

UNIVERSITÉ PARIS CITÉ

École Doctorale Sciences Mathématiques de Paris Centre (ED 386)
Institut de Recherche en Informatique Fondamentale

**Wait or Act: Parameterized Verification with
Wait-Only Protocols and Non-Blocking Communication**

Par **LUCIE GUILLOU**

THÈSE DE DOCTORAT EN INFORMATIQUE

Dirigée par **ARNAUD SANGNIER**

et co-encadrée par **NATHALIE SZNAJDER**

Présentée et soutenue publiquement le 30 septembre 2025

Devant un jury composé de :

ARNAUD SANGNIER	Directeur	MCF, DIBRIS, Università di Genova
JÉRÔME LEROUX	Rapporteur	DR, LaBRI, CNRS, Université Bordeaux
ROLAND MEYER	Rapporteur	PR, TU Braunschweig
NATHALIE BERTRAND	Examinatrice	DR, INRIA, IRISA, Université de Rennes
ANCA MUSCHOLL	Examinatrice	PR, LaBRI, Université Bordeaux
SYLVAIN SCHMITZ	Examineur	PR, IRIF, CNRS, Université Paris Cité

Membre invité:

NATHALIE SZNAJDER	Co-encadrante	MCF, LIP6, CNRS, Sorbonne Université
-------------------	---------------	--------------------------------------

RÉSUMÉ

Cette thèse s'inscrit dans le cadre de la vérification formelle de systèmes distribués/concurrents paramétrés, dans lesquels un nombre non borné de processus identiques exécutent le même programme et qui interagissent via deux mécanismes de communication :

1. les rendez-vous non bloquants : un processus envoie un message qui est reçu instantanément par un autre processus et si aucun processus n'est prêt à recevoir le message, celui-ci est simplement perdu (mais bien envoyé).
2. les broadcasts : un processus envoie un message reçu simultanément par tous les processus capables de le recevoir.

Le programme exécuté par les processus est donné comme une machine à états finis, appelé un protocole. Le nombre de processus n'est pas fixé, et il s'agit donc d'un paramètre du système. Vérifier ces systèmes revient alors à le faire pour tout nombre de processus possible. On obtient alors un nombre infini de systèmes rendant impossible l'utilisation de méthodes classiques de model checking. Dans ce cadre, nous étudions plusieurs problèmes de vérification paramétrée. Chaque problème vise à établir l'existence d'un nombre de processus pour lequel une exécution du système satisfait une propriété donnée : la couverture d'un état (un processus peut-il atteindre un état particulier ?), la couverture d'une configuration (le système peut-il évoluer vers une configuration qui couvre une cible ?), la synchronisation (tous les processus peuvent-ils atteindre un même état simultanément ?), et la couverture répétée (une transition peut-elle être exécutée indéfiniment ?). Nous montrons que les problèmes de couverture d'un état et de configuration sont EXPSPACE -complets dans les réseaux où les processus communiquent seulement avec le mécanisme de rendez-vous non-bloquant, améliorant un résultat antérieur qui ne démontrait que la décidabilité. En revanche, le problème de synchronisation est indécidable tout comme le problème de la couverture répétée. Notre contribution principale consiste en l'introduction des protocoles *Wait-Only*, une restriction syntaxique où chaque état est soit un état d'attente (ne pouvant effectuer aucune émission de message) soit un état d'action (ne pouvant recevoir aucun message). Cette séparation claire entre états d'action et d'attente permet d'obtenir une propriété fondamentale appelée la *propriété de copier-coller*, qui garantit que si un état d'action et un autre état sont séparément couvrables, alors il le sont également ensemble. Pour cette classe de protocoles, nous prouvons que la couverture d'un état est résoluble en temps polynomial, tandis que la couverture de configuration est elle, PSPACE -complète. Cela représente une réduction significative de complexité par rapport au cas général (Ackermann-complet et EXPSPACE -complet lorsque l'on se restreint aux rendez-vous non-bloquants). En restreignant la communication aux rendez-vous non-bloquants, la couverture de configuration peut être résolue en temps polynomial, avec pour conséquence remarquable que tout état couvable est soit couvable par au plus un processus, soit par un nombre arbitraire de processus. Pour les protocoles *Wait-Only* avec communication par broadcast, nous prouvons que la synchronisation est décidable (mais Ackermann-difficile), tandis que la synchronisation vers un état d'action est EXPSPACE -complète et que la couverture répétée est dans EXPSPACE . De manière surprenante, la synchronisation dans les protocoles *Wait-Only* où le seul mécanisme de communication est le rendez-vous non-bloquant, reste indécidable. Enfin, nous

introduisons les protocoles *Single Wait-Only*, où depuis chaque état d'attente, un unique message peut être reçu. Dans ce cadre, la couverture répétée devient décidable en temps polynomial.

Mots clés : vérification paramétrée, systèmes distribués, broadcast, rendez-vous, couverture, accessibilité.

ABSTRACT

This thesis falls within the framework of formal verification of parameterized distributed/concurrent systems, in which an unbounded number of identical processes execute the same program and interact via two communication mechanisms:

1. non-blocking rendez-vous: a process sends a message that is instantly received by another process, and if no process is ready to receive the message, it is simply lost (but still sent).
2. broadcast: a process sends a message received simultaneously by all processes capable of receiving it.

The program executed by the processes is given as a finite state machine, called a protocol. The number of processes is not fixed and is hence a parameter of the system. Verifying these systems then amounts to do so for all possible numbers of processes. Hence, we obtain an infinite number of systems, which makes usual model checking methods to verify such systems impossible. In this context, we study several problems of parameterized verification. Each problem aims to establish the existence of a number of processes for which an execution of the system satisfies a given property: state coverability (can a process reach a particular state?), configuration coverability (can the system evolve towards a configuration that covers a target configuration?), synchronization (can all processes reach the same state simultaneously?), and repeated coverability (can a transition be executed infinitely often?). We show that the state and configuration coverability problems are ExpSpace-complete in networks where processes communicate only with the non-blocking rendez-vous mechanism, improving a previous result that only demonstrated decidability. On the other hand, we show that the synchronization problem is undecidable, just like the repeated coverability problem. Our main contribution consists of introducing *Wait-Only* protocols, a syntactic restriction where each state is either a waiting state (unable to receive any message) or an action state (unable to perform any emission of message). This clear separation between action and waiting states allows us to obtain a fundamental property called the *copypaste property*, which guarantees that if an action state and another state are separately coverable, then they are also coverable together. For this class, we prove that state coverability becomes solvable in polynomial time, while configuration coverability is PSPACE-complete, representing a significant reduction in complexity compared to the general case (Ackermann-complete and ExpSpace-complete when restricted to non-blocking rendez-vous). By restricting communication to non-blocking rendez-vous, configuration coverability even becomes decidable in polynomial time, with the remarkable consequence that any coverable state is either coverable by at most one process or by an arbitrary number of processes. For Wait-Only protocols with broadcast communication, we prove that synchronization is decidable (but Ackermann-hard), while synchronization to an action state is ExpSpace-complete and repeated coverability is in ExpSpace. Finally, we introduce *Single Wait-Only* protocols, where exactly one message can be received from each waiting state. In this framework, repeated coverability becomes decidable in polynomial time.

Keywords: parameterized verification, distributed systems, broadcast, rendez-vous, coverability, reachability.

INTRODUCTION EN FRANÇAIS

LES SYSTÈMES DISTRIBUÉS

Les systèmes distribués et concurrents sont composés de multiples entités qui coopèrent pour atteindre un objectif commun. Dans les systèmes distribués, ces entités sont physiquement séparées et se trouvent généralement sur différentes machines reliées par un réseau. En revanche, dans les systèmes concurrents, les entités coexistent dans un même environnement, comme des threads s'exécutant sur un seul processeur ou au sein d'une même application (par exemple, sur un smartphone ou un système embarqué). À mesure que les systèmes computationnels modernes gagnent en échelle et en complexité, ils reposent de plus en plus sur des architectures concurrentes et distribuées. Ces systèmes jouent un rôle central dans de nombreux domaines. Les systèmes concurrents apparaissent notamment dans des applications où plusieurs tâches s'exécutent simultanément sur un même appareil, telles que les applications pour smartphone, les systèmes d'exploitation ou les contrôleurs embarqués dans les véhicules et dispositifs industriels. En permettant l'exécution parallèle et la communication entre tâches, ces architectures offrent de hautes performances et une grande scalabilité. Cependant, cette même complexité introduit des défis majeurs : l'entrelacement non déterministe des actions rend leur comportement difficile à comprendre, à prédire et à vérifier.

Les erreurs dans ces systèmes peuvent avoir de graves conséquences. Une défaillance peut entraîner des pertes financières, compromettre la sécurité, voire, lorsqu'elle se produit à grande échelle, engendrer des coûts écologiques importants. L'analyse et la vérification de leur comportement sont donc essentielles, à la fois pour garantir leur correction et pour optimiser l'usage des ressources. Pourtant, cette tâche est loin d'être triviale. Ces systèmes impliquent souvent une communication asynchrone, des topologies dynamiques, des données partagées sur des domaines infinis, ou encore un nombre non borné de participants, autant d'aspects qui posent d'importantes difficultés théoriques et pratiques. Bien que les systèmes distribués et les systèmes concurrents partagent certaines problématiques, comme l'évitement des interblocages (*deadlock*) ou l'atteinte d'états d'erreur, ils se distinguent aussi par leurs modèles de défaillance. Dans les systèmes distribués, un composant (comme un ordinateur) peut tomber en panne ou devenir inaccessible, et le système doit être résilient à ce type de défaillance. En revanche, les pannes sont généralement ignorées dans les systèmes concurrents, où tous les threads sont supposés s'exécuter dans un environnement unique et contrôlé.

Pour ancrer notre étude dans un cadre concret, nous introduisons maintenant un environnement de programmation largement utilisé. Un langage de programmation couramment employé pour concevoir des programmes comportant plusieurs composants (appelés *threads*) s'exécutant en parallèle est *Java*. Un *thread* est un sous-processus (généralement d'un système concurrent), c'est-à-dire une séquence d'exécution indépendante, qui permet à un programme d'accomplir plusieurs tâches simultanément. Java offre un support intégré pour le multithreading à travers sa classe *Threads* et des interfaces associées, une fonctionnalité connue sous le nom de *programmation multithread en Java*.

Pour coordonner l'exécution des threads et éviter les conflits lorsqu'ils accèdent à des ressources partagées, Java fournit des mécanismes de synchronisation. Une *section critique* est une portion du code qui ne doit être exécutée que par un seul thread à la fois, afin d'éviter des *conditions de compétition* (*race conditions*). Java implémente cela via le mot-clé `synchronized`, qui garantit qu'un seul thread peut entrer dans la section critique à un instant donné.

En plus des verrous, les threads peuvent coopérer en utilisant les méthodes `wait()`, `notify()` et `notifyAll()`, définies dans la classe `Object`. Lorsqu'un thread appelle `wait()` sur un objet, il suspend son exécution et attend d'être réveillé. La méthode `notify()` réveille au plus un thread en attente sur ce même objet, tandis que `notifyAll()` réveille tous les threads en attente. Le thread qui reprend effectivement l'exécution après un appel à `notify()` n'est pas déterminé à l'avance.

MODEL CHECKING

Comme discuté précédemment, les systèmes distribués impliquent de nombreux composants interagissant, ce qui conduit à une explosion combinatoire des chemins d'exécution possibles. Cela rend la vérification manuelle extrêmement difficile et motive le développement de différentes approches. Trois grandes familles de méthodes ont émergé :

1. **Les tests** sont la méthode la plus couramment utilisée en pratique. Ils consistent à exécuter l'implémentation sur différents jeux de données et à observer son comportement. Bien que les tests soient efficaces pour détecter des bogues, ils ne peuvent pas garantir leur absence. Des techniques ont été développées pour générer des ensembles de tests couvrant tous les cas nécessaires, mais cela n'est pas toujours possible.
2. **Les assistants de preuve** (par exemple, TLA+, Coq, Isabelle) permettent la construction de preuves formelles vérifiées par machine montrant qu'un système satisfait à une spécification donnée. Cette approche offre une forte garantie, puisqu'elle vérifie la correction du design lui-même. Toutefois, elle nécessite une expertise à la fois dans le système étudié et dans les outils de preuve. De plus, identifier des invariants ou des arguments inductifs appropriés peut s'avérer très difficile.
3. **Le model checking** désigne une classe de techniques automatisées qui prennent en entrée un *modèle* d'un système distribué ainsi qu'une *propriété* représentant une spécification souhaitée (par exemple, l'exclusion mutuelle, l'absence de deadlock, ou l'accessibilité d'une configuration particulière). Le modèle en entrée est typiquement un système de transitions d'états, où chaque état encode le point d'exécution actuel de chaque processus du système. Construire de tels modèles est une tâche complexe en soi, en particulier pour les programmes de grande taille reposant sur des bibliothèques externes dont le comportement interne n'est pas toujours connu à l'avance. Un *model checker* explore systématiquement l'espace des états du modèle afin de vérifier si toutes les exécutions possibles satisfont la propriété donnée. Parmi les model checkers notables, on peut citer UPPAAL [LPY97]. Cependant, le nombre d'états du modèle croît nécessairement de manière exponentielle avec le nombre de processus — un phénomène connu sous le nom de *problème d'explosion d'états*. Pour atténuer les limitations liées à la construction de modèles et à l'explosion de l'espace d'états, une autre ligne de recherche s'est concentrée sur le développement de *model checkers*

sans état (stateless model checkers) [MQ08; Kok+22; Ene+24]. Ces outils ne construisent pas explicitement le système de transitions global. À la place, ils explorent différents chemins d'exécution du programme en le réexécutant systématiquement sous divers ordonnancements des threads, évitant ainsi d'avoir à stocker en mémoire l'ensemble des états atteignables.

Cette thèse s'inscrit dans le contexte du *model checking*. Notre objectif est d'explorer des méthodes alternatives de vérification pour les systèmes distribués, en cherchant à contourner le problème d'explosion d'états, plutôt que de développer des modèles de systèmes plus efficaces.

VÉRIFICATION PARAMÉTRÉE

Le problème d'explosion d'états mentionné précédemment provient du fait que la taille du modèle croît avec le nombre de processus. Cela empêche la vérification d'algorithmes distribués conçus pour fonctionner avec un nombre arbitraire de participants n , où n est un paramètre de l'algorithme. Dans de tels cas, la vérification par model checking ne peut généralement garantir la correction que pour des valeurs fixes de n , sans couvrir le cas général.

C'est précisément ici qu'intervient la *vérification paramétrée*. Un système est dit paramétré lorsque certains aspects (comme le nombre de processus, l'ordonnanceur ou les variables du programme) ne sont pas fixés, mais considérés comme des paramètres. Un tel système définit alors une famille (potentiellement infinie) d'instances concrètes, chacune correspondant à une instantiation particulière des paramètres. Dans cette thèse, nous nous concentrons sur le cas où le paramètre est le nombre de processus, ce qui donne lieu à une famille infinie de systèmes : celui à un processus, deux processus, etc.

Les méthodes développées pour répondre à des questions du type :

Le système distribué paramétré donné satisfait-il une spécification donnée ?

sont appelées techniques de *vérification paramétrée*. Elles permettent de garantir la correction d'un système pour tout nombre de processus, tout en contournant le problème d'explosion d'états. Fait intéressant, la vérification peut parfois s'avérer plus facile dans le cas paramétré que pour un nombre fixe de participants. Par exemple, la vérification de vivacité (comme vérifier si un certain état est atteint infiniment souvent par un processus) dans des systèmes paramétrés à mémoire partagée est PSPACE-complète (décidable en espace polynomial) pour un nombre fixé de processus, et appartient à la classe NP (décidable de manière non déterministe en temps polynomial) dans le cas paramétré [Dur+17].

Cette thèse s'inscrit dans le contexte de la vérification paramétrée. Nous nous intéresserons à des questions de vivacité et d'atteignabilité, telles que : un état d'erreur donné peut-il être atteint par un processus ? Un état donné est-il atteignable infiniment souvent ? Ou encore : le système peut-il atteindre une configuration deadlock ? Nous étudierons des systèmes paramétrés précis, dans lesquels tous les processus exécutent la même machine à états finis et communiquent via des communications binaires (rendez-vous) ou par diffusion (broadcast). Nous présentons à présent un aperçu des principaux modèles et résultats en vérification paramétrée, afin de situer nos propres contributions.

La recherche sur la vérification paramétrée a débuté en 1986, lorsque Apt et Kozen ont démontré son indécidabilité dans [AK86]. Leur preuve construit un système dans lequel chaque processus simule une étape d’une machine de Turing, nécessitant des comportements distincts pour chaque processus. Cela a conduit à l’étude de réseaux composés d’entités identiques exécutant le même programme, éventuellement avec une unique entité distinguée, appelée *leader*, qui exécute un programme différent.

Dans ce cadre, une taxonomie des systèmes a été étudiée, selon différents critères : les primitives de communication (rendez-vous, diffusions, mémoire partagée ...), la puissance de calcul des processus (automate, machine à pile, machine équipée de données ...), l’ordonnanceur (synchronisé¹ ou non). Nous renvoyons à [Esp14] pour une présentation d’ensemble.

MÉMOIRE PARTAGÉE. Dans [Hag11], Hague introduit un modèle dans lequel les processus communiquent via une mémoire partagée : ils peuvent y lire et écrire, mais pas de manière atomique (i. e. une lecture et une écriture ne peuvent pas être effectuées en une seule étape). Il y démontre la décidabilité du problème de *couverture d’état* (consistant à déterminer si une configuration où au moins un processus est dans un certain état est atteignable), pour des systèmes avec un leader désigné, et où les processus sont modélisés par des machines à pile. Ce résultat a donné naissance à une ligne de recherche riche explorant des questions de vérification pour des systèmes interagissant via mémoire partagée sous diverses hypothèses : avec des processus modélisés soit par des automates finis, soit par des machines à pile, avec ou sans leader [EGM13; Dur+17; Bou+16; FMW17], dans des contextes où de nouveaux processus peuvent être créés dynamiquement au cours de l’exécution [MSW17], ou encore dans des cadres structurés en tours (chaque processus progressant de manière asynchrone à travers des tours et accédant à des variables partagées associées aux tours voisins) [Ber+22]. Dans les systèmes à mémoire partagée, la communication est par essence asynchrone, car les processus peuvent lire bien après l’écriture en mémoire.

Dans cette thèse, nous nous concentrons sur les systèmes à *communication synchrone* ; tous les systèmes décrits dans la suite de cette section utiliseront ce modèle de communication.

RENDEZ-VOUS. Dans leur article fondateur [GS92], German et Sistla étudient des réseaux où les processus communiquent via des *rendez-vous*. Les programmes des processus sont des automates finis munis de transitions d’émission et de réception. Une étape de communication par rendez-vous a lieu entre deux processus sur un même message.

Ils considèrent à la fois les systèmes avec et sans leader. Dans le premier cas, ils montrent que le problème de couverture, ainsi que des questions de vivacité telles que « un certain état peut-il être couvert infiniment souvent ? », sont décidables en espace exponentiel (EXPSPACE). Dans le cas sans leader, ils prouvent que ces problèmes sont résolubles en temps polynomial (dans PTIME). Dans [HS20], les auteurs étudient le *problème de seuil* (*cut-off*), qui consiste à déterminer s’il existe un nombre k tel que, pour tout nombre de processus supérieur ou égal à k , il existe une exécution menant tous les processus dans un état donné. Ils montrent

¹ Par *synchronisé*, on entend ici que les messages sont reçus instantanément. Cela diffère de la définition couramment utilisée dans la communauté des algorithmes distribués, où *synchronisé* signifie que la communication se fait par tours, tous les processus progressant simultanément d’un tour à l’autre. À chaque tour, tous les processus envoient et reçoivent typiquement des messages.

que, en présence d'un leader, ce problème est décidable mais Ackermann-difficile. Plus récemment, dans [BER21], les auteurs améliorent la borne de complexité dans le cas sans leader en montrant que le problème est PTIME-complet.

PROTOCOLES DE POPULATION. Étroitement liés aux systèmes à communication par rendez-vous, les protocoles de population ont été introduits en 2004 [Ang+04] comme un modèle conçu pour calculer des prédicats sur des données d'entrée. Dans ce cadre, chaque processus possède initialement une valeur d'entrée et exécute une machine à états finis. L'objectif est que le système atteigne un consensus : lorsque l'ensemble des entrées satisfait le prédicat, tous les processus doivent finir par se stabiliser dans un état d'acceptation désigné. La distinction principale avec les systèmes paramétrés utilisant les rendez-vous réside dans la notion d'entrée et d'évaluation de prédicat. Les protocoles de population sont étudiés principalement d'un point de vue calculatoire, et leur puissance expressive a été entièrement caractérisée : ils calculent exactement les prédicats définissables dans l'arithmétique de Presburger [AAE06; Blo+20]. La vérification d'un protocole de population consiste donc à vérifier qu'il calcule correctement un prédicat donné. Ce problème de vérification a été montré Ackermann-difficile [Esp+17], ce qui a motivé l'étude de sous-classes plus tractables ces dernières années [Esp+18; Esp+21].

VARIANTE NON-BLOQUANTE DU RENDEZ-VOUS. Dans cette thèse, nous ne considérons pas exactement le rendez-vous tel que défini par German et Sistla, mais une variante appelée *rendez-vous non-bloquant*. Dans ce modèle, une étape de communication peut être : (1) un échange de message entre deux processus (comme chez German et Sistla), ou (2) l'émission d'un message par un processus, sans qu'il soit *reçu*, car aucun processus n'est prêt à le recevoir (formellement, aucun n'est dans un état à partir duquel une transition de réception est définie pour ce message). Notons que ce second cas ne se produit que si *aucun* processus n'est en mesure de recevoir le message ; on ne peut donc pas parler de rendez-vous *perdant*, car aucun message n'est « perdu » lorsqu'il pourrait être reçu. À l'inverse du rendez-vous non-bloquant, nous appelons *rendez-vous bloquant* le modèle original de German et Sistla, dans lequel l'émission ne peut avoir lieu que si un récepteur est disponible. Reprenons ici l'illustration avec la programmation multithread en Java : le rendez-vous non-bloquant formalise naturellement la méthode `notify()` : lorsqu'un thread exécute cette méthode, il peut réveiller un thread en attente sur le même objet. En revanche, si aucun thread n'est en attente, l'exécution se poursuit sans effet sur le reste du système. Cette motivation a été introduite dans [DRVB02], où cette primitive a également été étudiée avec des communications par diffusion (broadcast) pour modéliser la méthode `notifyAll()`. Elle a toutefois reçu relativement peu d'attention depuis.

BROADCASTS. Une communication par *broadcast* a lieu lorsqu'un message est envoyé et reçu, de manière synchrone, par tous les autres processus du système. Un *réseau de broadcast* est un système distribué paramétré dans lequel les processus échangent des messages par broadcast et exécutent tous la même machine à états finis. Ce modèle a été introduit dans [EN98], puis étudié conjointement avec les rendez-vous bloquants dans [EFM99]. Dans ce dernier, les auteurs distinguent deux types de transitions : les unes mènent à des communications par rendez-vous, les autres à des broadcasts. Ils y montrent la décidabilité du problème de couverture grâce à un argument fondé sur un quasi-ordre bien fondé (*well quasi-order*), mais établissent l'*indécidabilité* des propriétés de vivacité. Dans [EK03], Emerson

et Kahlon introduisent des réseaux où les processus communiquent par broadcasts et *rendez-vous non-bloquants*, et prouvent la décidabilité du problème de couverture. Toutefois, ce résultat est tempéré par une complexité élevée : dans [SS13], il est montré que le problème est Ackermann-difficile pour les réseaux à broadcasts et rendez-vous bloquants. Cette difficulté persiste même lorsque seuls les broadcasts sont autorisés. Ce constat a conduit à la définition de systèmes à broadcasts dans des contextes alternatifs, notamment en faisant varier la *topologie* du réseau. Dans le modèle classique, la topologie est complète : chaque message est reçu par tous les autres processus. Dans [DSZ10], les auteurs explorent un modèle où le paramètre est non seulement le nombre de processus, mais aussi les liens entre eux (i. e. la topologie du réseau). Un message broadcasté est alors reçu uniquement par les *voisins* du processus émetteur. Ils y montrent que la couverture devient indécidable dans ce cadre. Cela les amène à considérer plusieurs variantes. Dans le même article [DSZ10], ils restreignent le modèle aux topologies dont le plus long chemin simple est borné par une constante, et prouvent alors la décidabilité de la couverture. L'année suivante, dans [DSZ11], ils introduisent une classe de topologies appelées *Maximal Clique Graphs with Bounded Paths* (Graphes à Clique Maximaux avec Chemins Bornés), et montrent également la décidabilité du problème, bien que la complexité soit Ackermann-difficile. En [DSZ12], ils étendent le modèle pour inclure des défaillances de processus (avec ou sans récupération) ainsi que la perte de messages. Ils identifient alors des scénarios dans lesquels la couverture reste décidable, et d'autres pour lesquels elle devient indécidable.

La variante ayant suscité le plus d'intérêt est celle des *réseaux de broadcast reconfigurables* (reconfigurable broadcast networks), introduite dans [DSZ10]. Dans ce modèle, la topologie du réseau peut évoluer dynamiquement au cours de l'exécution, ce qui permet de modéliser un déplacement spontané des processus. Les auteurs y montrent la décidabilité du problème de couverture. Ce résultat est raffiné dans [Del+12], où la couverture est prouvée en temps polynomial, tandis que des requêtes plus riches sont montrées PSPACE-complètes. Ce travail est étendu dans [BGW22], où les auteurs étudient le problème de *couverture de cubes* (cube reachability problem). Cependant, ces modèles restent trop permissifs pour capturer des déplacements réalistes : la topologie peut changer de manière arbitraire entre deux étapes, ce qui ne reflète pas les contraintes physiques ou logiques de nombreux systèmes distribués.

RÉSEAUX DE PETRI. Nous concluons cette section par un bref aperçu des réseaux de Petri, introduits par Petri dans [Pet62]. Conçus à l'origine comme un modèle formel pour les systèmes concurrents et distribués, les réseaux de Petri représentent naturellement des processus ainsi que la consommation et la production de ressources. Bien qu'ils ne soient pas des systèmes distribués à proprement parler, ils y sont étroitement liés : il existe des traductions naturelles des protocoles de population et des réseaux à rendez-vous bloquants vers les réseaux de Petri. Par conséquent, les réseaux de Petri constituent un outil fondamental dans l'étude de la vérification paramétrée. Un réseau de Petri est composé de places, de jetons et de transitions, chaque transition consommant des jetons dans certaines places et en produisant dans d'autres. Ils sont étroitement liés aux systèmes d'additions vectorielles avec états (Vector Addition Systems with States, VASS) et, plus généralement, aux machines à compteurs, qui seront présentées en détail dans Chapter 3. Nous mentionnons ici simplement que le problème de la couverture (décider s'il est possible, à partir d'un marquage initial, d'atteindre un marquage comportant au moins un jeton dans une place donnée) est EXPSPACE-complet (voir [Lip76] pour la difficulté et [Rac78] pour l'appartenance). Le problème de l'accessibilité (décider s'il est possible d'atteindre

exactement un marquage donné) est quant à lui Ackermann-complet (voir [LS19] pour l'appartenance et [Ler21; CO21] pour la difficulté).

CETTE THÈSE

Dans cette thèse, nous étudions la vérification paramétrée de systèmes dans lesquels tous les processus exécutent la même machine à états finis (que nous appellerons le *protocole*) et communiquent via des *rendez-vous non bloquants* et/ou des *broadcasts*. Nous pensons que ces systèmes conviennent bien pour modéliser des programmes écrits avec des cadres de programmation tels que le multithreading en Java, pour les raisons évoquées précédemment. Notre modèle suppose l'absence de défaillance des processus, ce qui est courant dans les systèmes concurrents où tous les processus s'exécutent sur la même machine, ce qui implique qu'un crash affecterait l'ensemble du système plutôt que des processus individuels.

Dans ce contexte, nous abordons plusieurs questions ouvertes. En particulier, le problème de couverture dans les systèmes où les processus communiquent uniquement via des rendez-vous non bloquants était seulement connu comme décidable, comme le montre [DRVB02]. Les auteurs y définissent une extension des réseaux de Petri pour étudier de tels réseaux. Dans le Chapter 5, nous établissons que ce problème est EXPSPACE -complet, ce qui peut surprendre puisque ce même problème est dans PTIME pour les rendez-vous bloquants.

Nous définissons également une restriction sur les protocoles exécutés par les processus : les *protocoles Wait-Only*. Un protocole Wait-Only ne possède aucun état à la fois capable d'envoyer et de recevoir des messages. Nous pensons que cette classe correspond bien à notre exemple illustratif, puisque dans un programme concurrent, un thread exécute typiquement soit des tâches (et n'attend donc pas de notifications sur des objets), soit il attend pour entrer dans une section critique, ne pouvant alors plus effectuer d'actions (notamment d'envoi de messages). Nous étudions plusieurs problèmes de vérification, pour des réseaux où les processus communiquent via rendez-vous non bloquants, broadcasts, ou les deux. Il s'avère que certains problèmes deviennent significativement plus simples pour les protocoles Wait-Only, par exemple le problème de couverture dans les réseaux combinant broadcasts et rendez-vous passe d'une complexité Ackermann-complete à PSPACE -complet. Ce n'est cependant pas une règle générale, puisque le problème de synchronisation, qui demande s'il existe un nombre de processus pour lequel une exécution met tous les processus dans un état donné, reste indécidable.

Enfin, nous allons plus loin en définissant une dernière restriction appelée *protocoles Single Wait-Only*, qui sont des protocoles Wait-Only où les états d'attente (états depuis lesquels des messages sont reçus mais pas envoyés) n'ont qu'une seule réception possible. Dans ce contexte, nous montrons que certains problèmes, comme celui de synchronisation ainsi qu'un problème de vivacité, deviennent plus accessibles.

Nous pensons que ces deux restrictions sont intéressantes et pourraient être étudiées dans d'autres contextes. Dans les protocoles (Single) Wait-Only, les interférences entre processus sont contraintes : par exemple, un broadcast n'a plus d'impact sur les processus qui sont dans des états d'action (c'est-à-dire des états où aucune réception n'est possible). Avec ces limitations, beaucoup de choses ne sont plus réalisables : par exemple, il n'est pas toujours possible d'élire un leader, puisque tous les processus doivent être dans un état d'attente prêts à recevoir le message d'élection. Selon la question de vérification et les primitives de communication utilisées, cette puissance d'expression peut être récupérée.

Cette thèse contient 7 chapitres.

- Au chapitre 2, nous définissons les réseaux de processus que nous considérons, où chaque processus exécute la même machine à états finis. Nous classifions ces réseaux en trois types selon les mécanismes de communication : les rendez-vous non bloquants (réseaux à rendez-vous), les broadcasts reçus par tous les processus (réseaux à broadcast), et une combinaison des deux (appelés simplement réseaux).

Nous présentons également les problèmes de vérification abordés dans cette thèse :

- Le problème de couverture d'état (resp. de couverture de configuration), qui consiste à savoir s'il existe un nombre de processus pour lequel il existe une exécution mettant au moins un processus dans un état donné (resp. couvrant une configuration donnée) ;
- Le problème de synchronisation, qui demande s'il existe un nombre de processus pour lequel une exécution met tous les processus dans un état donné ;
- Le problème de couverture répétée, qui demande s'il existe un nombre de processus et une exécution infinie dans laquelle un processus prend infiniment souvent une transition donnée.

Nous présentons l'historique de notre modèle et détaillons les résultats existants. À la fin de ce chapitre, nous proposons un tableau récapitulatif des résultats connus.

	Couverture	Synchronisation	Couverture répétée
Réseaux à Rendez-vous	Décidable [EK03; DRVB02]	??	Indécidable [EK03; EFM99]
Réseaux à Broadcast	Ackermann-complet		
Réseaux	[DRVB02; EK03; SS13]		

Table 1: Tableau obtenu à la fin du Chapitre 2.

- Le chapitre 3 est également un chapitre introductif qui présente plusieurs modèles, tous liés aux machines à compteurs et aux systèmes à additions de vecteurs avec états (VASS). Cette thèse ne vise pas à apporter de nouveaux résultats concernant les modèles introduits au chapitre 3, mais nous en ferons un usage intensif tout au long des preuves présentées dans ce travail. Nous y détaillons également leur historique ainsi que les résultats existants sur les deux problèmes principaux abordés ici : la couverture et l'accessibilité.

À partir du chapitre 4, les chapitres peuvent être lus indépendamment, même si les chapitres 7 et 8 peuvent nécessiter de revenir de temps en temps au chapitre 6.

- Le chapitre 4 présente deux petits résultats d'indécidabilité, tous deux concernant le même problème, appelé le problème de synchronisation, qui consiste à savoir s'il existe un nombre de processus pour lequel une exécution met tous les processus dans

un état donné. Nous démontrons l’indécidabilité de ce problème pour des réseaux de processus communiquant via des broadcasts, respectivement des rendez-vous non bloquants. Si le premier résultat n’est pas surprenant, nous n’avons toutefois pas trouvé de référence à ce sujet, c’est pourquoi nous le documentons ici. Le second résultat s’appuie sur notre étude des réseaux à rendez-vous non bloquants et est présenté dans un article de 2023 [GSS23].

- Dans le chapitre 5, nous étudions des réseaux où les processus communiquent uniquement via des rendez-vous non bloquants. Le résultat central est l’ExpSpace-complétude du problème de couverture, une question jusqu’ici ouverte. Nous démontrons d’abord cette complexité dans le cadre d’une nouvelle classe étendue de machines à compteurs, que nous montrons équivalente aux réseaux à rendez-vous pour le problème de couverture. Le contenu de ce chapitre a été publié dans un article de 2023 [GSS23].

À la fin des chapitres 4 et 5, il est possible de compléter la Table 1.1 pour obtenir la Table 1.2.

	Couverture	Synchronisation	Couverture répétée
Réseaux à Rendez-vous	ExpSpace-complet Chapter 5	Indécidable Chapter 4	Indécidable [EK03; EFM99]
Réseaux à Broadcast	Ackermann-complet		
Réseaux	[DRVB02; EK03; SS13]		

Table 2: Tableau obtenu à la fin du Chapitre 5.

- Au chapitre 6, nous introduisons la restriction syntaxique des protocoles *Wait-Only*. Dans un protocole *Wait-Only*, les états peuvent être partitionnés en deux ensembles : les états d’action, depuis lesquels les messages peuvent être envoyés et/ou broadcastés, et les états d’attente, depuis lesquels les messages peuvent être reçus. Ce chapitre se concentre sur l’étude des problèmes de couverture dans ce contexte. Nous établissons une propriété cruciale liée à cette restriction, appelée la propriété de *copypaste* (copier-coller), qui permet essentiellement d’assembler des exécutions tout en préservant la couverture de certains états. Cette propriété nous permet ensuite de montrer que le problème de couverture d’état est PTIME-complet, tandis que la couverture de configuration est PSPACE-complet. Nous étudions également le problème de couverture de configurations dans le cas où les processus communiquent uniquement via des rendez-vous et démontrons qu’il est alors PTIME-complet. Les résultats présentés dans ce chapitre ont été publiés dans [GSS24] pour les réseaux avec broadcasts et rendez-vous, et dans [GSS23] pour la partie sur les réseaux à rendez-vous.
- Le chapitre 7 porte toujours sur la restriction *Wait-Only*, mais cette fois-ci dans le cadre de réseaux de processus communiquant uniquement via des broadcasts. Ce chapitre s’appuie sur un VASS simulant ces réseaux. La traduction des réseaux à broadcasts vers des VASS n’est pas triviale : la manière naturelle de procéder est d’abstraire les processus en ne comptant que le nombre de processus dans chaque état, ce qui donne un compteur par état du protocole. Lorsqu’un broadcast a lieu

entre deux états q et p , un processus passe de l'état q à l'état p , ce qui se traduit dans le VASS par la décrémentation du compteur q et l'incrémement du compteur p . Comme un broadcast doit être reçu par tous les processus, tous les processus dans un état s pouvant recevoir le message doivent passer à l'état suivant r . Dans le VASS, cela correspond à vider le compteur s et à transférer sa valeur dans r . Or, ce type d'opération n'est pas réalisable dans un VASS, et d'ailleurs les extensions de VASS permettant ces opérations voient leur vérification devenir indécidable. Cependant, nous montrons que grâce à la restriction *Wait-Only*, il existe une autre manière de procéder qui permet de définir un VASS générique simulant le réseau. Il en découle que (1) le problème de synchronisation est décidable (ce qui constitue le premier résultat de décidabilité pour ce problème dans les cadres étudiés dans cette thèse), bien qu'il soit *Ackermann-difficile*, et (2) le problème de couverture répétée est dans EXPSPACE . De plus, nous montrons que lorsque l'état donné pour la synchronisation est un *état d'action*, on peut formuler une version relâchée du problème d'atteignabilité sur le VASS, et que ce problème est alors EXPSPACE -complet. Le contenu de ce chapitre est présenté a été publié dans [GSS25].

À la fin du chapitre 7, nous pouvons dresser la Table 1.3, où tous les problèmes sont restreints aux protocoles *Wait-Only*.

	Couverture...		Synchronisation		Couverture répétée
	...d'un état	...d'une configuration			
Réseaux à Rendez-vous	PTime-complet		Indécidable		
Réseaux à Broadcast	PTime complet	PSpace complet	<u>état d'action:</u> ExpSpace-complet	<u>état d'attente:</u> Ackermann-complet	dans ExpSpace
Réseaux			Indécidable		

Table 3: Tableau obtenu après les chapitres 6 et 7. Tous les résultats énoncés sont pour les protocoles *Wait-Only*. Les résultats en **rouge** peuvent être trouvés dans Chapter 6 et ceux en **orange** dans Chapter 7.

- Enfin, au chapitre 8, nous allons un cran plus loin en définissant les *protocoles Single Wait-Only*. Un protocole *Single Wait-Only* est un protocole *Wait-Only* dans lequel un seul message peut être reçu depuis chaque état d'attente. Nous y démontrons plusieurs résultats, dont le principal est que le problème de couverture répétée est dans PTIME . Le contenu de ce chapitre est inédit et n'a pas encore été publié. Tous les résultats sont résumés dans le tableau ci-dessous.

	Synchronisation sur un...		Couverture répétée
	état d'action	état d'attente	
Réseaux à Rendez-Vous	PTime-complet	Ackermann-difficile	dans PTime
Réseaux à Networks	PTime-complete	Décidable	
Réseaux			

Table 4: Tableau obtenu après le Chapter 8. Tous les résultats énoncés sont pour les protocoles Single Wait-Only.

CONTENTS

1	INTRODUCTION	1
1.1	Distributed systems	1
1.2	Model Checking	2
1.3	Parameterized verification	3
1.4	Taxonomy of parameterized systems	3
1.5	This thesis	6
1.6	Outline of the thesis	7
1.7	List of publications	9
1.7.1	Publications presented in this thesis	10
1.7.2	Publications not included in this thesis	10
2	NETWORKS OF PROCESSES	13
2.1	Introduction	13
2.2	Preliminaries	15
2.3	Protocols	16
2.4	Semantic	16
2.4.1	Configurations	16
2.4.2	Communication Step	17
2.4.3	Blocking versus Non-Blocking	20
2.5	Verification Problems	20
2.6	Multiset configurations	23
2.7	State of the Art	25
2.7.1	Networks with Broadcasts and Rendez-Vous	25
2.7.2	Blocking Rendez-Vous Networks	28
2.7.3	In this thesis	30
2.7.4	Recapitulative table of existing results.	31
3	COUNTER MACHINES	33
3.1	Introduction	33
3.2	Counter Machines	34
3.3	VASS and VAS	35
3.4	Non-Blocking Counter Systems	38
3.5	Summary	40
4	UNDECIDABILITY OF SYNCHRONIZATION	41
4.1	Broadcast Networks	42
4.2	Rendez-Vous Networks	45
4.3	Conclusions	53
5	COVERABILITY IN RENDEZ-VOUS NETWORKS	55

5.1	Introduction	55
5.2	Coverability in Non-Blocking Test-Free Reset Counter Machines	56
5.2.1	EXPSPACE-membership	56
5.2.2	EXPSPACE-hardness	61
5.3	Rendez-vous protocols	78
5.4	Reductions	79
5.4.1	From Rendez-vous Protocols to NB-CM	79
5.4.2	From NB-R-CM to Rendez-Vous Protocols	85
5.5	Conclusions	91
6	COVERABILITY IN WAIT-ONLY NETWORKS	93
6.1	Introducing Wait-Only Protocols	93
6.1.1	Synchronization Remains Undecidable	94
6.1.2	Notations and Examples	95
6.2	The copypaste property	96
6.3	State Coverability	103
6.3.1	Upper bound	103
6.3.2	Lower bound	106
6.4	Configuration Coverability	111
6.4.1	Main ideas	112
6.4.2	Reasoning with Abstract Configurations	112
6.4.3	Completeness of the algorithm	115
6.4.4	Soundness of the algorithm	118
6.4.5	Upper bound	120
6.4.6	Lower bound	120
6.5	Configuration Coverability for Wait-Only Rendez-Vous Networks	123
6.5.1	Token-Sets of Configurations	125
6.5.2	Computing Token-Sets of Configurations	128
6.5.3	Completeness	132
6.5.4	Soundness	134
6.5.5	Polynomial Time Algorithm	142
6.6	Conclusions	143
7	GOING BEYOND COVERABILITY IN BROADCAST WAIT-ONLY NETWORKS	145
7.1	Introduction	145
7.2	Preliminaries	146
7.3	A VASS simulating Wait-Only Broadcast Networks	148
7.3.1	Main idea	148
7.3.2	Preliminary properties	149
7.3.3	Summaries	152
7.3.4	Definition of the VASS	153
7.3.5	Soundness of the construction	156
7.3.6	Completeness of the construction	159
7.4	Synchronization in Wait-Only Broadcast Networks	169
7.4.1	Regaining decidability	169
7.4.2	Lower Bound	173
7.5	Synchronization is easier when the final state is an action state	177

CONTENTS

7.5.1	Upper bound	177
7.5.2	Lower bound	182
7.6	Repeated coverability problem for Wait-Only Protocols	183
7.6.1	Upper bound	183
7.6.2	Lower bound	195
7.7	Conclusions	198
8	ONE STEP FURTHER: SINGLE WAIT-ONLY NETWORKS	201
8.1	Introduction	201
8.2	Preliminaries	202
8.3	Repeated cover is easy	210
8.3.1	Notations	213
8.3.2	Local soundness property	214
8.3.3	Soundness lemma and main theorem	216
8.4	Synchronization to an action state is easy	220
8.4.1	Rendez-vous networks	221
8.4.2	Broadcast networks	224
8.4.3	Lower bound	225
8.5	Synchronization to a waiting state remains hard	227
8.6	Conclusions	228
9	CONCLUSIONS	231
	Bibliography	235
	Index	243

INTRODUCTION

1.1 DISTRIBUTED SYSTEMS

Distributed and concurrent systems are composed of multiple entities working together to achieve a common goal. In distributed systems, these entities are physically separated, often running on different machines connected via a network, while in concurrent systems, they typically coexist within the same environment, such as threads running on a single processor or within the same application (e.g., on a smartphone or an embedded system). As modern computing systems grow in scale and complexity, they increasingly rely on both distributed and concurrent architectures. These systems are central to a wide range of domains. Distributed systems are used in large-scale infrastructures like e-commerce platforms, cloud and grid computing, transportation systems, and mobile robot networks. Concurrent systems appear in applications where multiple tasks run simultaneously on a single device, such as smartphone apps, operating systems, or embedded controllers in vehicles and industrial devices. By enabling parallel execution and communication, these architectures offer high performance and scalability. However, this same complexity introduces major challenges: the non-deterministic interleaving of actions makes their behavior difficult to understand, predict, and verify.

Errors in these systems can have serious consequences. A failure may lead to financial loss, compromise safety, or even, when deployed at scale, incur substantial ecological costs. Analyzing and verifying their behavior is thus essential, both to ensure correctness and to optimize resource usage. Yet, this task is far from trivial. These systems often involve asynchronous communication, dynamic topologies, shared data over infinite domains, or even an unbounded number of participants, each of which raises significant theoretical and practical difficulties. While distributed and concurrent systems share common concerns, such as avoiding deadlocks or reaching error states, they also differ in their failure models. In distributed systems, a component (like a computer) may crash or become unreachable, and the system must be resilient to such failures. In contrast, crashes are usually not considered in concurrent systems, where all threads are assumed to run within a single controlled environment.

To ground our study in a concrete setting, we now introduce a widely used programming framework. One programming language commonly used to design programs that run multiple components, called threads, in parallel is Java. A thread is a subprocess (typically of a concurrent system), essentially a separate path of execution, that allows a program to perform multiple tasks at the same time. Java provides built-in support for multithreading through its Thread class and related APIs, a feature known as Java multithread programming. To coordinate the execution of threads and avoid conflicts when they access shared resources, Java provides mechanisms for synchronization. A critical section is a part of the code that

should be accessed by only one thread at a time to prevent race conditions. Java supports this through the `synchronized` keyword, which ensures that only one thread can enter the critical section at a time. In addition to locking, threads can cooperate using the `wait()`, `notify()`, and `notifyAll()` methods, which are defined in the `Object` class. When a thread calls `wait()` on an object, it pauses execution, waiting to be notified. The `notify()` method wakes up at most one thread that is waiting on the same object, whereas `notifyAll()` wakes up all waiting threads. The actual thread that resumes execution after a `notify()` call is not predictable.

1.2 MODEL CHECKING

As discussed earlier, distributed systems involve many interacting components, leading to a combinatorial explosion of possible execution paths. This makes manual verification extremely challenging and motivates the development of different methods. Three major families of approaches have emerged:

1. Testing is the most commonly adopted method in practice. It consists in executing the implementation on various inputs and observing its behavior. Although testing is effective at detecting bugs, it cannot guarantee their absence. Techniques have been developed to generate sets of tests that cover all needed executions, but it is not always possible.
2. Proof assistants (e.g., TLA+, Coq, Isabelle) support the construction of machine-checked formal proofs that a given system satisfies its specification. This approach offers high assurance, as it verifies the correctness of the design itself. However, it requires expertise in both the system under study and the proof tools. Moreover, identifying suitable invariants or inductive arguments can be very difficult.
3. Model checking refers to a class of automated techniques that take as input a *model* of a distributed system and a *property* representing a desired specification (e.g., mutual exclusion, absence of deadlock, or reachability of a particular configuration). The input model is typically a state-transition system, where each state captures the current execution point of each process in the system. Constructing such models is itself a challenging task, particularly for large-scale programs that rely on external libraries whose internal behavior may not be fully known a priori. A model checker systematically explores the state space of the model to verify whether all possible executions satisfy the given property. Notable model checkers include UPPAAL [LPY97]. However, the number of states of the model necessarily grows exponentially with the number of processes, this is known as the state explosion problem. To mitigate the limitations of model construction and state space explosion, another line of research has focused on the development of *stateless model checkers* [MQ08; Kok+22; Ene+24]. These tools do not explicitly build the global state-transition system. Instead, they explore different execution paths of the program by systematically re-executing it under different thread schedulings, thus avoiding the need to store all reachable states in memory.

This thesis is grounded in the context of model checking. Our focus is on exploring alternative verification methods for distributed systems, aiming to circumvent the state explosion problem rather than on developing more efficient system models.

1.3 PARAMETERIZED VERIFICATION

The aforementioned state explosion problem arises from the fact that the size of the model grows with the number of processes. This prevents the verification of distributed algorithms that are intended to work for any number of participants n , where n represents a parameter of the algorithm. In such cases, model checking can typically verify correctness only for fixed values of n , and not for the general case.

This is precisely where parameterized verification comes into play. A system is said to be parameterized when certain aspects (such as the number of processes, the scheduler, or program variables) are not fixed but treated as parameters. Such a system then defines a (potentially infinite) family of concrete instances, each corresponding to a specific parameter instantiation. In this thesis, we focus on the case where the parameter is the number of processes, yielding an infinite family of systems with one process, two processes, and so on.

Methods developed to address questions such as:

Does a given parameterized distributed system meet a given specification?

are referred to as *parameterized verification* techniques. This allows us to ensure that a system is correct for every number of processes, avoiding the state explosion problem. Interestingly enough, it sometimes makes verification easier than when considering a fixed number of participants. For instance, liveness verification (e.g., whether a given state is reached infinitely often by some process) of parameterized systems with shared memory is PSPACE-complete for a fixed number of processes and in NP when parameterized [Dur+17].

This thesis is situated in the context of parameterized verification. We will focus on liveness and reachability questions such as: can a given error state be reached by a process? Is a given state reachable infinitely often? Or even: can the system reach a configuration where all processes are in a deadlock state? We will look at precise parameterized systems, in which processes all evolve in the same finite-state machine and communicate via pairwise communication or broadcasts. We now present an overview of the main models and results in parameterized verification to provide context for our own contributions.

1.4 TAXONOMY OF PARAMETERIZED SYSTEMS

The line of research on parameterized verification began in 1986, when Apt and Kozen proved its undecidability in [AK86]. Their proof constructs a system where each process simulates a step of a Turing machine, requiring distinct behaviors for different processes. This led to the study of networks of identical entities running the same program, possibly with a single distinguished one, called the leader, executing a different program. In that setting, a taxonomy of systems have been considered, depending on the communication primitives (rendez-vous, broadcasts, shared memory...), the computational power of a process (automaton, push-down machine, machine equipped with data...), the scheduler (synchronous¹ or not). We refer to [Esp14] for a survey.

SHARED MEMORY. In [Hag11], Hague introduced a model in which processes communicate via a shared memory: they can read and write on it, but not atomically (i. e. a read

¹ By *synchronous*, we mean that messages are received instantaneously. This differs from the definition commonly used in the distributed algorithms community, where *synchronous* refers to communication occurring in rounds, with all processes moving synchronously from one round to the next. In each round, all processes typically send and receive messages.

and a write cannot be performed in a single step). Hague proved the decidability of the state coverability problem (asking for the reachability of a configuration with at least one process in a state) for systems in which there is a designated leader and processes are modeled as pushdown machines. This result gave rise to a rich line of research exploring verification questions for systems interacting through shared memory under various assumptions: with processes modeled either as finite-state or pushdown machines, with or without a leader [EGM13; Dur+17; Bou+16; FMW17], in settings where new processes may be dynamically spawned during execution [MSW17], or where executions are structured in rounds (each process progressing asynchronously through rounds and accessing shared variables associated with their neighboring rounds) [Ber+22]. In shared memory systems, the communication is, *par essence*, asynchronous as processes might read way after the writing in the memory. In this thesis, we focus on systems with *synchronous communication*; all systems described in the remainder of this section will use this communication model.

RENDEZ-VOUS. In their seminal paper [GS92], German and Sistla consider networks where processes communicate via *rendez-vous*. Processes' programs are finite-state machines equipped with *sending* transitions and *reception* transitions. A rendez-vous communication step happens between two processes on a same message. They explore both systems with and without the presence of a leader. In the first setting, they establish that the coverability problem, as well as liveness questions such as “is a given state can be covered infinitely often?” are in EXPSPACE , and in the leaderless case, they prove that the questions are solvable in polynomial time. In [HS20], the authors investigate the cut-off problem, which asks whether there exists a threshold such that for any number of processes above that threshold, one can find an execution that puts all processes in a given state. They prove that in the presence of a leader, the problem is decidable although Ackermann-hard. Later in [BER21], authors improved the complexity bound in the leaderless case by showing that the problem is PTIME -complete.

POPULATION PROTOCOLS. Closely related to systems with rendez-vous communication, population protocols were introduced in 2004 [Ang+04] as a model designed to compute predicates over input data. In this setting, each process initially holds an input value and executes a finite-state machine. The goal is for the system to eventually reach consensus: when the collection of inputs satisfies the predicate, all processes should eventually stabilize in a designated accepting state. The key distinction from parameterized systems using rendez-vous lies in the notion of input and predicate evaluation. Population protocols are studied primarily from a computational perspective, and their expressive power has been fully characterized: they compute exactly the predicates definable in Presburger arithmetic [AAE06; Blo+20]. Verifying a population protocol thus involves checking whether it correctly computes a given predicate. This verification problem has been shown to be Ackermann-hard [Esp+17], which motivates the investigation of more tractable subclasses in recent years [Esp+18; Esp+21].

NON-BLOCKING VARIANT OF RENDEZ-VOUS. In this thesis, we will not exactly look at rendez-vous as defined by German and Sistla but rather at a variant called *non-blocking rendez-vous*. In non-blocking rendez-vous, a step of communication, is either two processes exchanging a message (*à la* Germand and Sistla), or a process sending a message which is *not received* because no process were ready to receive it (formally, it corresponds to the

absence of processes in states from which there is a reception transition for this message). Note that the latter scenario (where a message is not received) occurs only when no process is in a state from which it can receive the message, and in that sense, it cannot be called *lossy* rendez-vous, as no message is lost when it can be received. In contrast to non-blocking rendez-vous, we refer to rendez-vous as defined by German and Sistla as *blocking* rendez-vous. Observe that, going back to our Java multithread programming illustration, the non-blocking rendez-vous primitive formalized well the `notify()` method: when a thread executes a `notify()` method, it can wake up one thread waiting on the same object. However, if no thread is waiting, the thread goes on with its execution and the execution of `notify()` has no impact on the rest of the system. This motivation was first discussed in [DRVBo2] where it was introduced with broadcast communications to modelize `notifyAll()` methods. It has not received a lot of attention since.

BROADCASTS. Broadcast communication happens when a message is sent and received synchronously by all other processes. A broadcast network is a parameterized distributed system where processes exchange messages via broadcasts and execute a finite-state machine. First discussed in [EN98], they have then been studied together with blocking rendez-vous communication in [EFM99]: depending on the type of the transition, the message is either sent in order to lead to a rendez-vous or to a broadcast. In [EFM99], they established decidability of coverability problem using a well-quasi order argument, but undecidability of liveness questions. In [EK03], they introduced networks where processes communicate via broadcasts *and non-blocking rendez-vous*, model for which they also establish decidability of the coverability problem. However, the result is contrasted as in [SS13], the problem is proven to be Ackermann-hard when considering networks with broadcasts and blocking rendez-vous. This result holds even when only broadcasts are allowed.

This led to define systems with broadcasts communication but different settings, mainly changing the topology of the network among processes (which is complete when messages are received by all processes). In [DSZ10], they explored systems where the parameter is not only the number of processes but also the link between processes (i. e. the topology). In that context, a broadcast is received by the neighbors of the sender. The negative result presented by Delzanno, Sangnier and Zavattaro is that coverability is in that context undecidable. It led them to consider several variants. In [DSZ10], they also considered a restriction of this model where allowed topologies are topologies with a longest simple path bounded by a constant. In that context, they show the decidability of coverability. The next year, in [DSZ11], Delzanno, Sangnier and Zavattaro introduced a class of topologies called Maximal Clique Graphs with Bounded Paths, and prove the decidability of coverability, although it is Ackermann-hard. The following year, in [DSZ12], they extended the model to include process crashes with possible recovery or permanent failure, and allowed message loss. For some scenarios, they established decidability of coverability, while other cases remain undecidable.

The variant that has received the most attention is the *reconfigurable broadcast networks* model, introduced in [DSZ10], where the network topology can change dynamically during execution to simulate spontaneous movement of processes. They showed that the coverability problem is decidable. In [Del+12], authors established that the coverability is in PTIME, and even showed that for richer queries it is PSPACE-complete. This work was extended in 2022 [BGW22], where the authors studied the *cube* reachability problem. However, such networks fail to capture realistic movements of processes: the topology can drastically change at each

step, and links can be deleted and created at will. An attempt to constrain the changes of topology has been made in [BBM18].

PETRI NETS. We conclude this section with a brief overview of Petri nets, introduced by Petri in [Pet62]. Originally designed as a formal model for concurrent and distributed systems, Petri nets naturally represent processes and the consumption and production of resources. Although they are not distributed systems per se, they are closely related: there exist natural translations from population protocols and blocking rendez-vous networks into Petri nets. Consequently, Petri nets have been a fundamental tool in studying parameterized verification. A Petri net consists of places, tokens, and transitions, where each transition consumes tokens from certain places and produces tokens in others. They are closely related to Vector Addition Systems with States (VASS) and, more generally, to counter machines, which will be introduced in detail in Chapter 3. Here, we simply mention that the coverability problem (deciding whether from an initial marking it is possible to reach a marking with at least one token in a given place) is ExpSPACE -complete (see [Lip76] for hardness and [Rac78] for membership). The reachability problem (deciding whether it is possible to reach a specific marking exactly) is Ackermann-complete (see [LS19] for membership and [Ler21; CO21] for hardness).

1.5 THIS THESIS

In this thesis, we investigate the parameterized verification of systems where all processes execute the same finite-state machine (which we will call the *protocol*) and communicate via *non-blocking rendez-vous* and/or *broadcasts*. We believe these systems are well suited to model programs written using programming frameworks such as Java multithreading, for the reasons mentioned earlier. Our model assumes no process failures, which is common in concurrent systems where all processes run on the same machine, meaning that a crash would affect the entire system rather than individual processes. In this context, we address several open questions. In particular, the coverability problem in systems where processes communicate only via non-blocking rendez-vous was only known to be decidable, as shown in [DRVB02]. The authors defined an extension of Petri nets to study such networks. In Chapter 5, we establish that the problem is ExpSPACE -complete, which may be surprising given that the problem is in PTime for blocking rendez-vous.

We also define a restriction on the protocols executed by processes: *Wait-Only protocols*. A Wait-Only protocol has no state both able to send messages and receive messages. We believe it fits well with our illustrative example, as in a concurrent program, a thread is typically either executing tasks (and thus not waiting for notifications on objects), or waiting to enter a critical section, because it cannot perform any further actions (particularly sending messages). We investigate several verification problems in this context, in networks where processes communicate via non-blocking rendez-vous, broadcasts, or both. It turns out that some verification problems become significantly easier in Wait-Only protocols, for instance the coverability problem in networks with both broadcasts and rendez-vous drops from Ackermann-complete to PSPACE -complete. This is not a universal rule as for instance the synchronization problem, which asks if there exists a number of processes for which there is an execution putting all processes in a given state, remains undecidable in that context. Finally, we take one step further by defining a final restriction called *Single Wait-Only protocols*, which are Wait-Only protocols where the waiting states (states from

which messages are received but not sent) have only one message reception. In this context, we show that in some cases, the synchronization problem becomes easier, as well as a liveness problem.

We believe these two restrictions are interesting, and could be studied in other settings. In (Single) Wait-Only protocols, the interferences between processes are constrained, for instance, a broadcast has now no impact on processes which are on action states (states from which no reception is possible). With these limitations, a lot of things cannot be done anymore: for example, it is not always possible to elect a leader, as all processes must be in a waiting state ready to receive the election message. Depending on the verification question and the communication primitives, this expressive power can be recovered.

1.6 OUTLINE OF THE THESIS

This thesis is structured in seven chapters.

- In chapter 2, we define the networks of processes we consider, where each process executes the same finite-state machine. We categorize these networks into three types based on the communication mechanisms: non-blocking rendez-vous (Rendez-vous networks), broadcasts received by all processes (Broadcast networks), and a combination of both (simply referred to as networks). We also present the verification problems addressed in this thesis:
 - The state coverability (resp. configuration coverability) problem asking if there exists a number of processes for which there is an execution putting one process at least on a given state (resp. covering the configuration)?
 - The synchronization problem asking if there exists a number of processes for which there is an execution putting all the processes on a given state?
 - The repeated coverability problem asking if there exists a number of processes for which there is an infinite execution in which a given transition is fires infinitely often?

We present the history of our model and detail the existing results. At the end of this chapter, we have the following table of existing results.

	Coverability	Synchronization	Repeated coverability
Rendez-vous Networks	Decidable [EK03; DRVB02]	??	Undecidable [EK03; EFM99]
Broadcast Networks	Ackermann-complete		
Networks	[DRVB02; EK03; SS13]		

Table 1.1: Table obtained at the end of Chapter 2.

- Chapter 3 is also an introductory chapter and presents several models, all related to counter machines and Vector Addition System with States. This thesis does not aim to present new results regarding the models introduced in Chapter 3, but we will

make extensive use of it throughout the proofs presented in this work. We also detail their history as well as the existing results on the two main problems considered here: coverability and reachability.

From Chapter 4, the chapters can be read independently, even if Chapter 7 and 8 might require to go back to Chapter 6 from time to time.

- Chapter 4 presents two small undecidability results, both on the same problem, called the synchronization problem, which asks if there exists a number of processes for which there is an execution putting all the processes on a given state. We prove the undecidability of this problem for networks of processes communicating via broadcasts, respectively non-blocking rendez-vous. While the first result is not surprising, we were unable to find a reference for it, so we document it here. The second result grounds in our study of networks with non-blocking rendez-vous and can be found on a 2023 paper [GSS23].
- In Chapter 5, we explore networks where processes communicate solely via non-blocking rendez-vous. The central result is the ExpSpace -completeness of the coverability problem, a previously open question. We first prove this complexity in the context of new extended class of counter machines, which we show to be equivalent to rendez-vous networks for the problem of coverability. The content of this chapter has been published in a 2023 paper [GSS23]. At the end of chapters 4 and 5, we can complete Table 1.1 and get Table 1.2.

	Coverability	Synchronization	Repeated coverability
Rendez-vous Networks	ExpSpace-complete Chapter 5	Undecidable Chapter 4	Undecidable [EK03; EFM99]
Broadcast Networks	Ackermann-complete		
Networks	[DRVB02; EK03; SS13]		

Table 1.2: Table obtained at the end of Chapter 5.

- In chapter 6, we introduce the syntactic restriction of *Wait-Only* protocols. In a Wait-Only protocol, one can partition states among two sets: the action states from which messages can be sent and/or broadcast, and the waiting states, from which messages can be received. This chapter focuses on the study of coverability problems in this context. We establish a crucial property when it comes to this restriction named the *copypaste* property, which essentially allow us to glue executions together while preserving coverability of certain states. This property then allows us to establish that the state coverability problem is PTIME -complete, whereas configuration coverability is PSPACE -complete. We also investigate the configuration coverability problem when processes communicate only via rendez-vous and prove that it is then PTIME -complete. The result presented in this chapter have been published in [GSS24] when it comes to networks with broadcasts and rendez-vous, and in [GSS23] for the part about rendez-vous networks.

- Chapter 7 still focuses on Wait-Only restriction but this time with networks of processes communicating only via broadcasts. The chapter is centered on a VASS simulating those networks. The translation from broadcast networks to VASS is not trivial: the natural way to proceed is to abstract processes' away and only count the number of processes in each state. This way, there is one counter per state of the protocol. When a broadcast happens between two states q and p , a process moves from state q to p which is translated in the VASS by decrementing counter q and incrementing counter p . Since a broadcast must be received by all processes, all processes in a state s that can receive the message must move to the next state r . In the VASS, it amounts to empty counter s and transfer its value to r . This is not something that can be done in a VASS, and, by the way, extensions of VASS allowing for this operations, have undecidable verification questions. However, we show that thanks to the Wait-Only restriction, there is another way to proceed, which will allow to define a generic VASS simulating the network. It then follows that (1) the synchronization problem is decidable (observe that this is the first decidability result for this problem in the settings considered in this thesis), although Ackermann-hard, and (2) the repeated coverability problem is in ExpSpace . Furthermore, we show that when the given state to synchronize on is an *action state*, one can ask for a relaxed version of the reachability problem on the VASS and we get that the problem is then ExpSpace -complete. The content of this chapter has been published in [GSS25]. At the end of chapter 7, we are able to draw Table 1.3, where all problems are restricted to *Wait-Only* protocols.

	Coverability...		Synchronization		Repeated coverability
	...of a state	...of a configuration			
Rendez-vous Networks	PTime-complete		Undecidable		
Broadcast Networks	PTime complete	PSpace complete	<u>action state:</u> ExpSpace-complete	<u>waiting state:</u> Ackermann-complete	in ExpSpace
Networks			Undecidable		

Table 1.3: Table obtained from chapters 6 and 7. All stated results are for Wait-Only protocols. Results in **red** can be found in Chapter 6 and results in **orange** in Chapter 7.

- Finally, in chapter 8, we go one step further and define *Single Wait-Only protocols*. A Single Wait-Only protocol is a Wait-Only protocol such that only one message can be received from each waiting state. We prove different results in that settings, the main one being that the repeated coverability problem is in PTime . The content of this chapter is new and has not been published yet. All the results can be found in the table below.

	Synchronization on a...		Repeated coverability
	action state	waiting state	
Rendez-vous Networks	PTime-complete	Ackermann-hard	in PTime
Broadcast Networks	PTime-complete	Decidable	
Networks			

Table 1.4: Table obtained from Chapter 8. All stated results are for Single Wait-Only protocols.

1.7.1 Publications presented in this thesis

As mentioned earlier, this thesis presents the results of three published papers:

1. **Safety Analysis of Parameterised Networks with Non-Blocking Rendez-Vous.** *With Arnaud Sangnier and Nathalie Sznajder. Published in CONCUR 2023.*
The results of this paper are presented in Chapters 5 and 6.
2. **Safety Verification of Wait-Only Non-Blocking Broadcast Protocols.** *With Arnaud Sangnier and Nathalie Sznajder. Published in Petri Nets 2024.*
The results of this paper are presented in Chapter 6.
3. **Wait-Only Broadcast Protocols are Easier to Verify.** *With Arnaud Sangnier and Nathalie Sznajder. Published in MFCS 2025.*
The results of this paper are presented in Chapter 7.

1.7.2 Publications not included in this thesis

In addition to the papers presented above, I have contributed to the following publications during my PhD:

1. **Parameterized Broadcast Networks with Registers: from NP to the Frontiers of Decidability.** *With Corto Mascle and Nicolas Waldburger. Published in FoSSaCS 2024.*
We consider the parameterized verification of networks of processes that communicate through unreliable broadcasts. In this model, processes have local registers whose values are unordered and initially distinct and may therefore be thought of as identifiers. When an agent broadcasts a message, it appends to the message the value stored in one of its registers. Upon reception, an agent can store the received value or test it for equality against one of its registers. We consider the coverability problem, where one asks whether a given state of the system may be reached by at least one agent. We establish that this problem is decidable, although non-primitive recursive. We contrast this with the undecidability of the closely related target problem where all agents must synchronize on a given state. On the other hand, we show that the coverability problem is NP-complete when each agent only has one register. The main result of this paper is the decidability proof, which is quite long and involved, and uses well-quasi order theory.
2. **Phase-Bounded Broadcast Networks over Topologies of Communication.** *With Arnaud Sangnier and Nathalie Sznajder. Published in CONCUR 2024.*

We study networks of processes that all execute the same finite state protocol and that communicate through broadcasts. The processes are organized in a graph (a topology) and only the neighbors of a process in this graph can receive its broadcasts. The coverability problem we study here asks, given a protocol and a state of the protocol, whether there is a topology for the processes such that one of them (at least) reaches the given state. This problem is undecidable [DSZ10]. We study here an under-approximation of the problem where processes alternate a bounded number of times k between phases of broadcasting and phases of receiving messages. To this end, we define k -phase-bounded protocols which ensure that each process executing the protocol alternates at most k times between phases of receptions and phases of broadcasts. We show that, if the problem remains undecidable when k is greater than 6, it becomes decidable for $k = 2$, and ExpSPACE -complete for $k = 1$. Furthermore, we show that if we restrict ourselves to line topologies, the problem is in PTIME for $k = 1$ and $k = 2$. The undecidability proof is quite involved. The ExpSPACE -completeness proof is a two-sense reduction with the coverability problem for VASS. The decidability proof for $k = 2$ is somewhat more interesting, as it follows several steps: first, we prove that when studying such networks, one can restrict itself to tree topologies (independently if the protocol is phase-bounded or not), next, we prove that when considering 2-phase-bounded protocols and tree topologies, one can restrict itself to tree topologies with branches bounded by a linear function in the size of the protocol. Lastly, using a result of [DSZ10], we conclude that the problem is decidable for 2-phase-bounded protocols. When restricting the problem to line topologies and 2-phase-bounded protocols, we get a PTIME algorithm which requires a meticulous study of the shape of the executions on such networks.

NETWORKS OF PROCESSES

This chapter introduces the formal model that serves as the foundation for this thesis.

2.1 INTRODUCTION

Distributed systems are ubiquitous in modern computing, and are inherently complex, often involving an unbounded number of processes that interact through various communication mechanisms. Ensuring their correctness—guaranteeing the absence of errors or deadlocks—is a critical yet challenging task. Formal verification provides a rigorous framework for analyzing such systems, but the parameterized nature of distributed systems, where the number of processes is unknown or unbounded, introduces significant complexity.

In distributed systems, the number of processes is often not fixed, leading to an infinite state space that complicates verification. While fixing the number of processes can sometimes simplify analysis by reducing the system to a finite-state model, this approach is not always practical or sufficient. Moreover, even in finite cases, the combinatorial explosion of possible states often renders verification intractable. Surprisingly, in some cases, allowing the number of processes to be unbounded can actually simplify verification. For instance, liveness verification of parameterised systems with shared memory is PSPACE-complete for a fixed number of processes and in NP when parameterised [Dur+17].

However, parameterized verification is generally undecidable, as established by the seminal work of [AK86]. Several settings come into play that can be adjusted to allow automatic verification. One key aspect to obtain decidability is to assume that the processes do not manipulate identities in the protocols and use simple communication mechanisms like pairwise synchronization (or blocking rendez-vous) [GS92], broadcast of a message to all the entities [EFM99] (which can as well be lossy in order to simulate mobility [DSZ10]), shared register containing values of a finite set [EGM13], and so on (see [Esp14] for a survey). In every aforementioned case, all the entities execute the same protocol given by a finite state automaton. The choice of communication primitives plays a crucial role, for example, broadcast communication, where a single message can simultaneously reach all processes, often leads to high verification complexity due to the global impact of each communication step. In contrast, pairwise communication mechanisms, such as blocking rendez-vous, offer a more localized form of interaction, which can simplify verification tasks.

In this thesis, we study a unified model that incorporates two fundamental communication primitives:

1. Broadcasts: A message sent by one process is received synchronously by all other processes.

2. Non-blocking rendez-vous: A message is sent to a single process, but the sender does not wait for the receiver to be ready. If no receiver is available, the message is lost.

This model, first introduced in [EK03], captures a wide range of distributed systems. It consists of an unbounded number of identical processes, each executing the same program modeled as a finite-state machine.

Within this model, we focus on several fundamental verification problems, each with practical implications for system reliability and performance:

1. State Coverability: Given a protocol state, does there exist a number of processes and an execution where at least one process reaches the given state? This problem is crucial for identifying reachable error states.
2. Configuration Coverability: Given a specific configuration (i.e., a distribution of processes over states), can the system reach a configuration that covers it (i.e., has at least as many processes in each state as the given configuration)? This generalizes the previously mentioned problem.
3. Synchronization: Does there exist a number of processes, such that all processes can simultaneously reach a given state? If the given state is a deadlock state, a negative answer to this problem ensures that the system will never end up in a deadlock, for any number of processes.
4. Repeated Coverability: Is there an infinite run where a specific transition is taken infinitely often?

The study of parameterized verification has a rich history. For instance, in [GS92], the authors showed that blocking rendez-vous networks admit polynomial-time solutions for many verification problems (including state coverability, configuration coverability and repeated coverability) while in [EK03], the authors demonstrated that state coverability is decidable for broadcast networks, but with high computational complexity (Ackermann-complete). In [EFM99] and later in [EK03], the repeated coverability problem is shown to be undecidable in networks with broadcast and non-blocking rendez-vous communication.

In this thesis, we build on these foundations to map the frontier between tractable and intractable verification problems. By studying a unified model that incorporates both broadcasts and non-blocking rendez-vous, we aim to identify the conditions under which verification remains feasible. This chapter lays the groundwork for this exploration by formally defining the model, its semantics, and the verification problems we will study.

We begin by introducing general mathematical notations used throughout the thesis (Section 2.2). We then define the program executed by each process, modeled as a finite-state machine called a protocol (Section 2.3). We then present configurations and communication steps in Section 2.4. We formally introduce the verification problems in Section 2.5 and we then present an alternative definitions of configurations, which does not make use of processes' identities in Section 2.6. We will extensively use this definition throughout the thesis, as it simplifies proofs and statements. Finally, we present the state-of-the-art for these models (Section 2.7).

2.2 PRELIMINARIES

We let $\mathbb{N}$ be the set of natural numbers and $\mathbb{Z}$ be the set of relative numbers. For $i, j \in \mathbb{Z}$ with $i \leq j$, we denote $[i, j]$ as the set $\{k \in \mathbb{Z} \mid i \leq k \text{ and } k \leq j\}$. We use $\mathbb{N}_{>0}$ to denote the set of positive natural numbers, i.e. $\mathbb{N} \setminus \{0\}$.

For a finite set of elements E , its number of elements is denoted $|E|$.

Let A be a countable set, we denote by A^* [resp. A^+] the set of [resp. non-empty] finite sequences of elements taken from A . For $w \in A^*$, the length of w is the number of elements in the sequence w , denoted $|w|$. For a sequence $w = a_0 \cdot a_2 \cdots a_k \in A^+$, we denote by $w[-1]$ the sequence $a_0 \cdot a_2 \cdots a_{k-1}$.

MULTISSETS. For a finite set E , the set $\mathbb{N}^E$ represents the multisets over E . For $M \in \mathbb{N}^E$ and an element $e \in E$, $M(e)$ denotes the multiplicity of element e in M . For instance, if M is the multiset with four elements a, b, b and c , then $M(a) = 1$, $M(b) = 2$ and $M(c) = 1$. We write M as $\langle a, b, b, c \rangle$ or $\langle a, 2 \cdot b, c \rangle$. For two elements $M, M' \in \mathbb{N}^E$, we use $M + M'$ to denote the multiset such that $(M + M')(e) = M(e) + M'(e)$ for all $e \in E$. We say that $M \leq M'$ if and only if $M(e) \leq M'(e)$ for all $e \in E$. If $M \leq M'$, then $M' - M$ is the multiset such that $(M' - M)(e) = M'(e) - M(e)$ for all $e \in E$. Given a subset $E' \subseteq E$ and $M \in \mathbb{N}^E$, we let $|M|_{E'} = \sum_{e \in E'} M(e)$ be the sum of the multiplicities in M of the elements in E' . The size of a multiset M is given by $|M| = |M|_E$.

VECTORS. For a finite set of elements E and a natural $d \in \mathbb{N}_{>0}$, we use E^d to denote the set of vectors of d elements also called d -dim vectors of E . Let $v \in \mathbb{Z}^d$ be a vector of relative numbers. The *norm* of v is denoted $\|v\|$ and is the maximal absolute value appearing in v , i.e. $\|v\| = \max_{1 \leq i \leq d} (|v(i)|)$ where $|v(i)|$ is the absolute value of $v(i)$. Given a set of vectors $V \subseteq \mathbb{Z}^d$, its norm $\|V\|$ is the maximal norm of its elements, i.e. $\|V\| = \max_{v \in V} (\|v\|)$. For two vectors $v, v' \in \mathbb{Z}^d$, we say that $v \leq v'$ whenever for all $i \in [1, d]$, $v(i) \leq v'(i)$. Finally, let $\mathbf{0}_d^1$ denote the null vector of dimension d , i.e. $\mathbf{0}_d \in \mathbb{Z}^d$ and $\mathbf{0}_d(i) = 0$ for all $i \in [1, d]$.

TRANSITION SYSTEMS. A transition system $\mathcal{S}$ consists of a tuple $(\Gamma, \rightarrow)$ where Γ is a set of states (perhaps infinite) and $\rightarrow$ is a transition relation between states. We let $\rightarrow^*$ [resp. $\rightarrow^+$] be the transitive and reflexive [resp. transitive] closure of $\rightarrow$.

Given a transition system $\mathcal{S} = (\Gamma, \rightarrow)$, a finite [resp. infinite] *run* of P w.r.t. $\mathcal{S}$ is a sequence of states $\gamma_0 \gamma_1 \cdots \gamma_{k-1}$ [resp. $\gamma_0 \gamma_1 \cdots$] such that $\gamma_i \rightarrow \gamma_{i+1}$ for all $0 \leq i < k$ [resp. for all $i \in \mathbb{N}$]. Its size is denoted $\text{size}(\rho)$ and is equal to k [resp. ω].

Given a set of initial states $\Gamma_{\text{init}} \subseteq \Gamma$ (perhaps infinite), a finite [resp. infinite] run $\gamma_0 \gamma_1 \cdots$ is a finite [resp. infinite] *execution* w.r.t. $\mathcal{S}$ if and only if $\gamma_0 \in \Gamma_{\text{init}}$.

The set of finite executions [resp. runs] w.r.t. $\mathcal{S}$ is denoted $\text{Exec}_f^{\mathcal{S}}$ [resp. $\text{Run}_f^{\mathcal{S}}$]. The set of infinite executions [resp. runs] is denoted $\text{Exec}_\omega^{\mathcal{S}}$ [resp. $\text{Run}_\omega^{\mathcal{S}}$]. Finally, the set of executions [resp. runs] is denoted $\text{Exec}^{\mathcal{S}} = \text{Exec}_f^{\mathcal{S}} \cup \text{Exec}_\omega^{\mathcal{S}}$ [resp. $\text{Run}^{\mathcal{S}} = \text{Run}_f^{\mathcal{S}} \cup \text{Run}_\omega^{\mathcal{S}}$].

¹ This thesis uses the `knowledge` package. When a *notion* is first defined, it appears in brown. Later occurrences of the *notion* (especially when they are far from the original definition) may be shown in blue: clicking on them brings the reader back to the corresponding definition.

2.3 PROTOCOLS

All processes will follow the same program that we will call a **protocol**. A **protocol** describes the behavior of processes: it is a transition system given as a finite-state machine. It has three different types of transitions: broadcast transitions labeled with $!!a$, sending transitions labeled with $!a$, and reception transitions labeled with $?a$. When a process takes a broadcast transition it sends a message to all other processes. When it takes a sending transitions, it sends a message to at most one other process, and reception transitions receive a message. For a finite alphabet Σ , we denote by $!!\Sigma$, $!\Sigma$, and $?\Sigma$ the sets $\{!!a \mid a \in \Sigma\}$, $\{!a \mid a \in \Sigma\}$ and $\{?a \mid a \in \Sigma\}$ respectively.

Definition 2.1 (Protocol). A **protocol** is a tuple $P = (Q, \Sigma, q_{init}, T)$, where

- Q is a finite set of states;
- Σ is a finite set of messages;
- $q_{init} \in Q$ is the initial state;
- $T \subseteq Q \times (!!\Sigma \cup !\Sigma \cup ?\Sigma) \times Q$ is a finite set of transitions.

The size of P is denoted by $|P|$ and is equal to $|T| \times (2\log_2(|Q|) + \log_2(|\Sigma|))$.

Remark 1. Remark that we don't consider internal transitions (transition which does not send or receive messages). However as we will explain later, it is easy to model internal actions with a broadcast (or a sending) of a special message τ for which there is no reception transitions in the protocol.

Let $P = (Q, \Sigma, q_{init}, T)$ be a **protocol**, $q \in Q$ a state of the **protocol**, and $a \in \Sigma$ a message. We denote $R(q)$ the set of messages which can be received from q , i. e. $\{b \in \Sigma \mid \exists q' \in Q, (q, ?b, q') \in T\}$, and conversely $Q?(a)$ the set of states from which there is an outgoing reception transition for message a , i. e. $\{p \in Q \mid \exists p' \in Q, (p, ?a, p') \in T\}$. We also denote by T_{Act} the set of action transitions (labeled with $!a$ or $!!a$), i. e. the set $T \cap (Q \times (!\Sigma \cup !!\Sigma) \times Q)$.

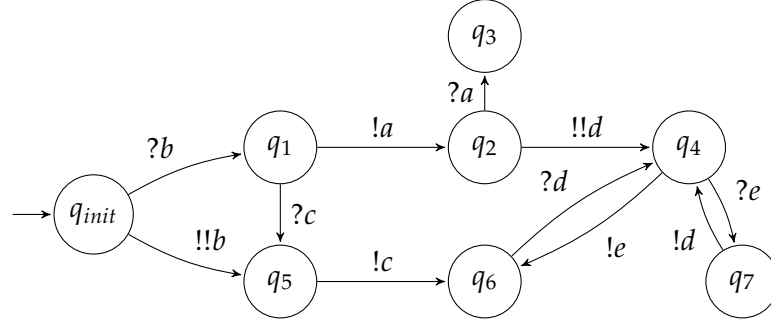
If a **protocol** $P = (Q, \Sigma, q_{init}, T)$ is such that $T \subseteq (Q \times (!!\Sigma \cup ?\Sigma) \times Q)$ (i. e. there is no sending transition) we say that P is a **broadcast protocol** (or **B-protocol**). Conversely, if $T \subseteq (Q \times (!\Sigma \cup ?\Sigma) \times Q)$ (i. e. there is no broadcast transitions) we say that P is a **rendez-vous protocol** (or **RDV-protocol**).

Example 2.1. In Figure 2.1, we display an example of a **protocol** with 8 states $q_{init}, q_1, q_2, \dots, q_7$. Its initial state is q_{init} and its set of messages is $\{a, b, c, d, e\}$. We have $R(q_4) = \{e\}$ and $Q?(d) = \{q_6\}$. It contains both sending transitions –for instance $(q_1, !a, q_2)$ – and broadcast transitions –for instance $(q_{init}, !!b, q_2)$ – hence it is neither a **B-protocol** nor a **RDV-protocol**.

2.4 SEMANTIC

2.4.1 Configurations

We now define the Network semantic. Since all processes execute the same protocol and the protocol does not use process identities, processes can be considered anonymous. However, in order to define certain verification problems and study the network, it is sometimes

Figure 2.1: Example of a protocol denoted P .

convenient (or even necessary) to distinguish between processes. For this reason, we will define configurations as vectors in Q^n for $n \in \mathbb{N}$.

A communication step occurs between two configurations when a process executes a transition $(q, \alpha, q') \in T$, where α represents either a broadcast or a sending event. If α is a broadcast, all processes receive the message and act accordingly, provided they are in a state with an outgoing reception transition for this message. If α is a sending event, the message is received by at most one other process. Specifically, if no process is in a state with an outgoing reception transition for this message, the message is not received by anyone. Otherwise, exactly one such process receives the message and executes the corresponding reception transition, while all other processes remain unchanged.

Consider a protocol $P = (Q, \Sigma, q_{init}, T)$, a *configuration* of P is a vector that assigns to each process its current state in the protocol.

Definition 2.2 (Configurations). Let $P = (Q, \Sigma, q_{init}, T)$ be a protocol and $n \in \mathbb{N}_{>0}$ a number of processes. A *configuration* over P of n processes is a vector $C \in Q^n$, and is sometimes called an *n -configuration*. The set of processes is $[1, n]$ and for a process $e \in [1, n]$, $C(e)$ depicts its current state.

The n -configuration C_{init}^n such that $C_{init}^n(e) = q_{init}$ for all processes $e \in [1, n]$, is called the *initial n -configuration*.

The set of all n -configurations over P is denoted $\mathcal{C}_n[P]$, and the set of all configurations is denoted $\mathcal{C}[P]$. The set of all initial configurations is denoted $\mathcal{C}_{init}[P]$.

The size of a configuration is the number of processes it contains and is denoted by $|C|$.

Note that the set of configurations is infinite, as well as the set of initial configurations. When the protocol is made clear from the context, we shall write $\mathcal{C}_n$, $\mathcal{C}_{init}$ and $\mathcal{C}$.

Definition 2.3. Let $P = (Q, \sigma, q_{init}, T)$ be a protocol. Given a configuration C and a set of states $Q' \subseteq Q$, we write $C^{-1}(Q')$ to denote the set of processes in a state $q \in Q'$, i. e. the set $\{e \in [1, |C|] \mid C(e) \in Q'\}$. When $Q' = \{q\}$ for a state $q \in Q$, we shall simply write $C^{-1}(q)$.

2.4.2 Communication Step

A step of communication is defined as follows. Consider two configurations $C, C' \in \mathcal{C}[P]$ with the same number of processes, i. e., $|C| = |C'|$. A communication step can be of three types:

- **(Broadcast step)** A process broadcasts a message m , that is received by other processes capable of handling it. Formally, given a broadcast transition $(q, !!m, q') \in T$, and a process $e \in [1, |C|]$, we have $C \xrightarrow{e, (q, !!m, q')}_P C'$ if the following conditions hold:

- $(C(e), !!m, C'(e)) = (q, !!m, q')$ and
- for all $e' \neq e$,

$$(m \notin R(C(e')) \text{ and } C(e') = C'(e')) \quad \text{or} \quad (C(e'), ?m, C'(e')) \in T.$$

i. e. either e' is not in a state from which it can receive the message and it remains still, or it receives it.

- **(Rendez-vous step)** A message is sent by one process and received by exactly one other process. Formally, given a sending transition $(q, !m, q') \in T$, and a process $e_1 \in [1, |C|]$, we have $C \xrightarrow{e_1, (q, !m, q')}_P C'$ if the following conditions hold:

- $(C(e_1), !m, C'(e_1)) = (q, !m, q')$ and
- there exists a process $e_2 \neq e_1$ such that $(C(e_2), ?m, C'(e_2)) \in T$, and
- for all $e' \in [1, |C|] \setminus \{e_1, e_2\}$, $C(e') = C'(e')$

(i. e. exactly one process receives the message and the other processes remain still).

- **(Sending step)** A message is sent by one process but is not received by any other process because no other process is capable of handling it. Formally, given a sending transition $(q, !m, q') \in T$, and a process $e \in [1, |C|]$, we have $C \xrightarrow{e, (q, !m, q')}_P C'$ if the following conditions hold:

- $(C(e), !m, C'(e)) = (q, !m, q')$ and
- For all $e' \neq e$, $m \notin R(C(e'))$ and $C(e') = C'(e')$; i. e. no process is in a state from which it can receive the message, and all processes remain still.

For a transition $t \in T_{Act}$, we let $C \xrightarrow{t}_P C'$ whenever there exists a process $e \in [1, |C|]$ such that $C \xrightarrow{e, t}_P C'$. We write $C \rightarrow_P C'$ whenever there exists a transition $t \in T_{Act}$ such that $C \xrightarrow{t}_P C'$. Remember that $\rightarrow_P^*$ [resp. $\rightarrow_P^+$] stands for the reflexive and transitive closure of $\rightarrow_P$ [resp. transitive closure]. When the protocol is made clear from the context, we drop the subscript and write $\rightarrow^*$, $\rightarrow^+$, $\rightarrow$, $\xrightarrow{t}$, and $\xrightarrow{e, t}$. Observe that $(\mathcal{C}[P], \rightarrow_P)$ is a transition system, we will abuse notation and write $\text{Exec}[P]$, $\text{Exec}_f[P]$ and $\text{Exec}_\omega[P]$ for the sets $\text{Exec}^{(\mathcal{C}[P], \rightarrow_P)}$, $\text{Exec}_f^{(\mathcal{C}[P], \rightarrow_P)}$ and $\text{Exec}_\omega^{(\mathcal{C}[P], \rightarrow_P)}$ respectively. Let $\rho \in \text{Exec}[P]$ we say that ρ is an *execution* of P .

Remark 2. A sending step is then the action of sending of a message with a transition labeled by a single exclamation point when no process is able to receive the message (i. e. no process is on a state with an outgoing transition reception for this message). Of course this can also happen when the transition is a broadcast, but this is already handled by the broadcast step definition. As we will see later, a lot of existing works do not consider sending steps.

Example 2.2. Consider protocol P depicted in Figure 2.1 and the initial 4-configuration. Then, the following execution is possible:

$$\begin{aligned}
 (q_{init}, q_{init}, q_{init}, q_{init}) &\xrightarrow{1, (q_{init}, !!b, q_5)} (q_5, q_1, q_1, q_1) \xrightarrow{1, (q_5, !c, q_6)} (q_6, q_5, q_1, q_1) \\
 &\xrightarrow{3, (q_1, !a, q_2)} (q_6, q_5, q_2, q_1) \xrightarrow{4, (q_1, !a, q_2)} (q_6, q_5, q_3, q_2) \\
 &\xrightarrow{4, (q_2, !!d, q_4)} (q_4, q_5, q_3, q_4).
 \end{aligned}$$

The following sequence of events happens:

- the first process broadcasts message b which is received by all other processes;
- the first process sends message c which leads to a rendez-vous since other processes are able to receive the message with transition $(q_1, ?c, q_5)$. The process receiving the message is the second process;
- the third process sends message a . Since no process is able to receive the message, the step is a *sending step*, and the message is not received by anyone;
- the fourth process sends message a . This time the third process can receive the message, hence it leads to a rendez-vous, putting the third process in state q_3 ;
- the fourth process broadcasts message d which is received by the only process able to do so, i. e. the first process.

For $C = (q_4, q_5, q_3, q_4)$, we have $|C| = 4$; $C^{-1}(\{q_1, q_2\}) = \emptyset$ and $C^{-1}(q_4) = \{1, 4\}$.

Remark 3. Following Remark 1, one can now clearly see that one can model an internal transition from state q to q' with a transition $(q, !!\tau, q')$ [resp. $(q, !\tau, q')$] where τ is an artificially added message such that there is no transitions labeled by $?\tau$ in the protocol. This way, broadcasting [resp. sending] τ has no effect on other processes, and allows the process to move from q to q' .

Let $P = (Q, \Sigma, q_{init}, T)$ and consider $C, C' \in \mathcal{C}$. We will say that $C \preceq_{\mathcal{C}[P]} C'$ if and only if $|C^{-1}(q)| \leq |C'^{-1}(q)|$ for all $q \in Q$.

We now introduce some useful vocabulary.

Definition 2.4. Consider a protocol $P = (Q, \Sigma, q_{init}, T)$. Let $C \in \mathcal{C}$, we use the following vocabulary:

- we say that C is *reachable* if there exists $C' \in \mathcal{C}_{init}$ with $C' \rightarrow^* C$;
- we say that C is *coverable* if there exists $C' \in \mathcal{C}_{init}$ and $C'' \in \mathcal{C}$ with $C' \rightarrow^* C''$ and $C \preceq_{\mathcal{C}[P]} C''$;
- if C is coverable and describes a configuration with one single process on a state $q \in Q$, then we say that q itself is coverable.

Example 2.3. From Example 2.2, we get that q_3 is coverable as well as the configuration with two processes in q_4 . We also have that the configuration with one process in q_3 , 2 in q_4 and one in q_5 is reachable.

2.4.3 Blocking versus Non-Blocking

In this thesis, we focus on communication steps as defined above: broadcast, rendez-vous, and **sending steps**. However, most existing works primarily address communication mechanisms that exclude **sending steps**. We refer to the semantic that omit **sending steps** as *blocking*, since a transition labeled with a single exclamation mark can be taken *only if* another process is ready to receive the message (corresponding to a **rendez-vous step**). If no process is able to receive it, transitions of this type are blocked.

In opposition, and to make it clear, we might refer to the semantic including the three types of communication steps as *non-blocking*.

Formally, given a protocol $P = (Q, \Sigma, q_{init}, T)$, we use $\rightarrow_{P, \flat}$ to denote the relation $\rightarrow_P$ restricted to **broadcast steps** and **rendez-vous steps** (i. e. excluding **sending steps**). As before, we write $\rightarrow_{\flat}$ when the context is clear. The tuple $(\mathcal{C}, \rightarrow_{\flat})$ is a transition system, and we will abuse notations and write $\text{Exec}^{\flat}[P]$ for the set of executions $\text{Exec}^{(\mathcal{C}[P], \rightarrow_{P, \flat})}$, and $\text{Exec}^{\flat}$ when P is clear from the context.

Note that all possible behaviors in the blocking semantic are possible as well in the semantic with **sending steps** but the converse is false. We give an example now.

Example 2.4. Consider the chain of events described in Example 2.2. Observe that there is one **sending step**: when the third process sends message a . If we compare this semantic with the blocking one, this step is unfeasible, as message a can not be sent if no process is able to receive it. As a consequence, in the blocking semantics, no process can ever reach state q_2 , as receiving a is only feasible from q_2 .

Since the number of processes is not fixed, observe that, to describe a network of processes, one only needs a **protocol** and the type of communication steps allowed (i. e. blocking or non-blocking semantics). Hence, we will use the following terminologies.

Name	Networks with...
<i>Blocking Rendez-Vous Networks</i>	RDV-protocols and Blocking semantics
<i>Rendez-Vous Networks</i>	RDV-protocols and Non-Blocking semantics
<i>Blocking Networks</i>	Protocols and Blocking semantics
<i>Broadcast Networks</i>	B-protocols
<i>Networks</i>	Protocols and Non-Blocking semantics

2.5 VERIFICATION PROBLEMS

In this section, we define the verification problems that will be of interest to us in this thesis. All the problems are formulated in the parameterized case in which the number of processes is unknown.

The first problem is the State-Coverability. Given a protocol P , it asks if there is an execution of P leading to a configuration in which at least one process is in q (i. e. is q coverable?). It is stated as follows for $\diamond \in \{nb, \flat\}$. When $\diamond = nb$, we consider the non-blocking semantic $\rightarrow$ by taking the convention that $\rightarrow_{nb} = \rightarrow$. When $\diamond = \flat$ we consider the blocking semantic $\rightarrow_{\flat}$.

State-Cover[◇]

Input: A Protocol $P = (Q, \Sigma, q_{init}, T)$ and a state $q \in Q$;

Question: Do there exist $C_{init} \in \mathcal{C}_{init}$, $C \in \mathcal{C}$, and $e \in [1, |C|]$, such that $C_{init} \rightarrow_{\diamond}^* C$ and $C(e) = q$?

In order to justify this problem, think of q_f as an error state, if q_f is not coverable, we conclude that starting from any number of processes, the error state q_f will never be reached by any of those agents.

The second problem is the Configuration-Coverability problem. Given a protocol P and a configuration C_f , it asks if it is coverable. It is stated as follows for $\diamond \in \{nb, b\}$.

Conf-Cover[◇]

Input: A Protocol $P = (Q, \Sigma, q_{init}, T)$ and a configuration $C_f \in \mathcal{C}$;

Question: Do there exist $C_{init} \in \mathcal{C}_{init}$, $C \in \mathcal{C}$, such that $C_{init} \rightarrow_{\diamond}^* C$ and $C_f \leq_{\mathcal{C}} C$?

This problem is a generalization of the STATE-COVER problem: STATE-COVER is an instance of the CONF-COVER problem, by taking the input configuration as a configuration with a single process on the state to cover.

Example 2.5. Consider the protocol P depicted in Figure 2.1. As explained in Example 2.4, (P, q_2) is a positive instance of STATE-COVER^{nb} whereas it is a negative instance of STATE-COVER^b. However, $(P, (q_2, q_2))$ is a negative instance of CONF-COVER^{nb}: consider any initial configuration $(q_{init}, q_{init}, \dots)$, and let us try to put (at least) two processes in q_2 :

$$(q_{init}, q_{init}, \dots) \xrightarrow{1, (q_{init}, !b, q_5)} (q_5, q_1, q_1, \dots) \xrightarrow{2, (q_1, !a, q_2)} (q_5, q_2, q_1, \dots)$$

The last configuration is such that: the first process is in q_5 , the second in q_2 and all other processes in q_1 . From the protocol's description process in q_5 has no chance to reach q_2 ever again. Let us try to put another process in q_2 :

$$(q_5, q_2, q_1, \dots) \xrightarrow{3, (q_1, !a, q_2)} (q_5, q_1, q_2, q_1, \dots) \xrightarrow{4, (q_1, !a, q_2)} (q_5, q_1, q_1, q_2, \dots) \xrightarrow{5, (q_1, !a, q_2)} \dots$$

Let us explain why we fail to put (at least) two processes in q_2 : from $(q_5, q_2, q_1, \dots)$, as there is one process that can receive message a with transition $(q_2, ?a, q_3)$ (the second process), then the communication step must be a rendez-vous step (and not a sending step). Hence, second process has to exit state q_2 and we did not manage to put more than one process on state q_2 .

We conclude that starting with any number of processes, in any execution, at most one process is on q_2 .

The next problem is the Synchronization problem. It asks given a protocol P and a special state q_f if there is an execution of P leading to a configuration in which all the processes are on q_f . It is formally stated as follows.

Synchro[◇]

Input: A Protocol $P = (Q, \Sigma, q_{init}, T)$ and a state $q_f \in Q$;

Question: Do there exist $C_{init} \in \mathcal{C}_{init}$, $C \in \mathcal{C}$, such that $C_{init} \rightarrow_{\diamond}^* C$ and $C(e) = q_f$ for all process $e \in [1, |C|]$?

To motivate this problem, think of q_f as the only deadlock state: if we cannot find an execution witnessing SYNCHRO for q_f , then one ensures that starting with any number of processes, there will never be a deadlock situation reached by the system.

Example 2.6. Consider the protocol P depicted in Figure 2.1, we present an execution attesting (P, q_4) is a positive instance of SYNCHRO^{nb}:

$$(q_{init}, q_{init}) \rightarrow (q_5, q_1) \rightarrow (q_5, q_2) \rightarrow (q_6, q_2) \rightarrow (q_4, q_4)$$

To contrast with, consider (P, q_1) : it is not a positive instance of SYNCHRO^{nb} as at least one process needs to broadcast message b for processes to reach q_1 . The broadcast of b leads to q_5 and once in q_5 , a process has no hope to reach q_1 ever again.

The last problem that we state is the Repeated-Coverability problem. Given a protocol P and a transition t_f of the protocol, it asks if there is an infinite execution of P in which a process takes transition t infinitely often. In contrast to the last three problems, a witness to a positive answer must this time be an infinite execution. Moreover, in the problem definition, we explicitly identify the process that takes t infinitely often. It is stated as follows for $\diamond \in \{nb, b\}$.

Rep-Cover[◇]

Input: A Protocol $P = (Q, \Sigma, q_{init}, T)$ and a transition $t_f = (q, \alpha, q') \in T$;

Question: Do there exist an infinite execution $C_0 C_1 \dots$ and a process $e \in [1, |C_0|]$ such that for all $N_1 \in \mathbb{N}$, there exists $N_2 \geq N_1$ with :

- if $t_f \in T_{Act}$, then $C_{N_2} \xrightarrow{e, t_f}_{\diamond} C_{N_2+1}$;
- otherwise $\alpha = ?a$ for an $a \in \Sigma$, and $(C_{N_2}(e), ?a, C_{N_2+1}(e)) = t_f$ and $C_{N_2} \xrightarrow{e', (p, \square a, p')}_{\diamond} C_{N_2+1}$ with $\square \in \{!, !!\}$, $p, p' \in Q$ and $e' \in [1, |C_0|]$.

This definition ensures that the state q (and q') is both visited and left infinitely often. Indeed, we do not want to consider as a positive instance an execution in which a process reaches the state and remains in it forever. This is why we define the problem in terms of a transition rather than a state.

Both problems are strongly related: if a transition (q, α, q') is taken infinitely often, then q (and q') are visited infinitely often. Furthermore, if a state q is visited and left infinitely often, then there exists a transition (q, β, p) that is taken infinitely often.

To motivate this problem, think of t_f as a bad-behavior transition. Answering 'no' to the REP-COVER problem ensures that, regardless of the initial number of processes, the bad behavior occurs only finitely often.

Example 2.7. Consider the protocol P depicted in Figure 2.1, starting from the *reachable* configuration (q_4, q_4) (as shown in Example 2.6), we explain why $(P, (q_4, ?e, q_7))$ is a positive instance of REP-COVER^{nb} :

$$(q_4, q_4) \rightarrow (q_6, q_7) \rightarrow (q_4, q_4) \rightarrow (q_6, q_7) \rightarrow \dots$$

Assume we continue the execution this way, then the second process takes infinitely often the transition $(q_4, ?e, q_7)$.

On the contrary, $(P, (q_5, !c, q_6))$ is a negative instance of REP-COVER^{nb} : there is no way for a process, once it took the transition $(q_5, !c, q_6)$, to reach back state q_5 , hence it can take the transition at most once during its execution. As any execution only contains a finite number of processes, the transition can only be taken a finite number of times during an execution.

2.6 MULTISSET CONFIGURATIONS

Since the protocol does not refer to process identities, we can describe behaviors without them. In such cases, a configuration is represented as a multiset over the set of states. Using multiset configurations simplifies the overall network representation and makes proofs more readable. While process identities are generally irrelevant in our networks—except in specific cases like the REP-COVER problem—working with a transition system that omits them is often more intuitive and pedagogical. For this reason, we formally define multiset configurations. To a particular configuration, we can associate to it a set of multiset configurations. We define all of this as follows.

Note that nothing defined in this section is new (the two transition relations we define with multiset configurations describe the same behaviors as the behaviors described by $\rightarrow$ and $\rightarrow_b$ respectively), however we will make extensive use of multiset configurations.

Definition 2.5 (Multiset configurations). Let $P = (Q, \Sigma, q_{init}, T)$ be a protocol. A *multiset configuration* over P is a *non-empty* multiset M over Q where $M(q)$ denotes the number of processes in state q and $|M|$ denotes the total number of processes in the configuration M . An *initial* multiset configuration M_{init} is a multiset configuration such that $M_{init}(q) = 0$ for all $q \in Q \setminus \{q_{init}\}$.

We let $\mathcal{M}[P]$ denote the set of all multiset configurations and $\mathcal{M}_{init}[P]$ denote the set of initial configurations. Finally, for $n \in \mathbb{N} \setminus \{0\}$, we use the notation $\mathcal{M}_n(P)$ to represent the set of configurations of size n , i.e. $\mathcal{M}_n(P) = \{M \in \mathcal{M}[P] \mid |M| = n\}$.

Given an integer $n \in \mathbb{N}$ and a *configuration* $C \in \mathcal{C}_n$, we define $\text{MS}(C) = \{C(1), \dots, C(n)\}$.

As before, when the protocol is made clear from the context, we shall write $\mathcal{M}$, $\mathcal{M}_{init}$ and $\mathcal{M}_n$.

We shall define the communication step relation between multiset configurations using the one defined for configurations in $\mathcal{C}$. We start by proving that, defining multiset configurations from configurations suffices to consider all possible multiset configurations.

Lemma 2.1. Given a protocol P , we have $\bigcup_{C \in \mathcal{C}[P]} \text{MS}(C) = \mathcal{M}[P]$, and $\bigcup_{C_{init} \in \mathcal{C}_{init}[P]} \text{MS}(C_{init}) = \mathcal{M}_{init}[P]$

Proof. By definition, $\bigcup_{C \in \mathcal{C}[P]} \text{MS}(C) \subseteq \mathcal{M}[P]$. Let $M \in \mathcal{M}[P]$, we denote it by $\wr q_1, \dots, q_k \wr$ with $k = |M|$. As M is a multiset, it might be that $q_i = q_j$ for some $i \neq j$. Let C_M be an k -configuration such that for all $e \in [1, k]$, $C_M(e) = q_e$. By definition, $M = \text{MS}(C_M)$. The second statement is trivial as $\bigcup_{C_{\text{init}} \in \mathcal{C}_{\text{init}}[P]} \text{MS}(C_{\text{init}}) = \bigcup_{n \in \mathbb{N}_{>0}} \wr n \cdot q_{\text{init}} \wr$. $\square$

Let $P = (Q, \Sigma, q_{\text{init}}, T)$ be a protocol. We now define $\rightarrow$ and $\rightarrow_b$ describing a communication step between multiset configurations: given two multiset configurations M, M' and a transition $t \in T_{\text{Act}}$, we have that $M \xrightarrow{t}_P M'$ [resp. $M \xrightarrow{t}_{P,b} M'$] if and only if there exist $C, C' \in \mathcal{C}$ two configurations and a process $e \in [1, |C|]$ such that $C \xrightarrow{e,t}_P C'$ [resp. $C \xrightarrow{e,t}_{P,b} C'$] and $M = \text{MS}(C)$ and $M' = \text{MS}(C')$.

In this context, broadcast steps, rendez-vous steps and sending steps can be rewritten as follows: let $M \xrightarrow{t}_P M' \in \mathcal{M}[P]$, then $|M| = |M'|$, and:

- **(Broadcast step)** If $t = (q, !m, q') \in T$, we can write M as $\wr q_1, q_2, \dots, q_n, q \wr$ and M' as $\wr q'_1, q'_2, \dots, q'_n, q' \wr$ where for all $1 \leq i \leq n$, either $m \notin R(q_i)$ and $q'_i = q_i$ or $(q_i, ?a, q'_i) \in T$;
- **(Rendez-vous step)** If $t = (q, !m, q') \in T$, then $M(q) > 0$, and there exists $p \in Q$ such that $(M - \wr q \wr)(p) > 0$ and $(p, ?m, p') \in T$ for some $p' \in Q$ and $M' = M - \wr p, q \wr + \wr p, q' \wr$.
- **(Sending step)** If $t = (q, !m, q') \in T$, then $M(q) > 0$ and $(M - \wr q \wr)(p) = 0$ for all $p \in Q$ such that $m \in R(p)$, and $M' = M - \wr q \wr + \wr q' \wr$.

Remark 4. In the rendez-vous step, we need to check that there is at least one process to send the message ($M(q) > 0$) and another process that can receive it via a transition $(p, ?m, p') \in T$. Since p may be equal to q , we must ensure that the receiver is not the sender. Therefore, we check that $(M - \wr q \wr)(p) > 0$.

We let $M \rightarrow_P M'$ [resp. $M \rightarrow_{P,b} M'$] whenever there exists a transition $t \in T_{\text{Act}}$ such that $M \xrightarrow{t}_P M'$ [resp. $M \xrightarrow{t}_{P,b} M'$], i. e. $\bigcup_{t \in T_{\text{Act}}} \xrightarrow{t}_P$ [resp. $\bigcup_{t \in T_{\text{Act}}} \xrightarrow{t}_{P,b}$]. When the protocol is made clear from the context, we shall write $\xrightarrow{t}$ and $\rightarrow$ [resp. $\xrightarrow{t}_b$ and $\rightarrow_b$]. As $(\mathcal{M}, \rightarrow)$ and $(\mathcal{M}, \rightarrow_b)$ are transition systems, we use $\rightarrow_P^*$, $\rightarrow_{P,b}^*$, as well as $\rightarrow_P^+$ and $\rightarrow_{P,b}^+$ to denote their respective transitive and reflexive closures, and their reflexive closures.

We now present an example, this time using multiset configurations.

Example 2.8. The sequence of events written in Example 2.2 can be re written as:

$$\begin{aligned} \wr q_{\text{init}}, q_{\text{init}}, q_{\text{init}}, q_{\text{init}} \wr &\xrightarrow{(q_{\text{init}}, !b, q_5)} \wr q_5, q_1, q_1, q_1 \wr \xrightarrow{(q_5, !c, q_6)} \wr q_6, q_5, q_1, q_1 \wr \\ &\xrightarrow{(q_1, !a, q_2)} \wr q_6, q_5, q_2, q_1 \wr \xrightarrow{(q_1, !a, q_2)} \wr q_6, q_5, q_3, q_2 \wr \\ &\xrightarrow{(q_2, !d, q_4)} \wr q_4, q_5, q_3, q_4 \wr. \end{aligned}$$

Note that now we do not care about identities, hence $\wr q_4, q_5, q_3, q_4 \wr$ describes the same configuration as $\wr q_3, q_4, q_4, q_5 \wr$.

We state the following lemmas making formally the link between configurations and multiset configurations.

Lemma 2.2. Let $P = (Q, \Sigma, q_{init}, T)$ be a protocol, $e \in \mathbb{N}$ and $t \in T$. Let $C, C' \in \mathcal{C}$ such that $C \xrightarrow{e,t} C'$ then $\text{MS}(C) \xrightarrow{t} \text{MS}(C')$.

Proof. Direct from $\xrightarrow{\cdot}$'s definition. $\square$

Lemma 2.3. Let $P = (Q, \Sigma, q_{init}, T)$ be a protocol, and $t \in T$. Let $M, M' \in \mathcal{M}$ such that $M \xrightarrow{t} M'$, then for all $C \in \mathcal{C}$ such that $M = \text{MS}(C)$, there exists $C' \in \mathcal{C}$ such that $C \xrightarrow{e,t} C'$ and $M' = \text{MS}(C')$ for a process $e \in [1, |C|]$.

Proof. Let $M, M' \in \mathcal{M}$ such that $M \xrightarrow{t} M'$. By definition of $\xrightarrow{\cdot}$, we get that there exists $e \in \mathbb{N}$ and $C_0, C_1 \in \mathcal{C}[P]$ such that $C_0 \xrightarrow{e,t} C_1$ and $\text{MS}(C_0) = M = \text{MS}(C)$ for a process $e \in [1, |C|]$. If $C_0 = C$, there is nothing else to be done. Otherwise, as $\text{MS}(C_0) = \text{MS}(C)$, there exists a permutation of processes $f : [1, |C|] \rightarrow [1, |C|]$ a bijective function such that $C(f(e)) = C_0(e)$ for all $e \in [1, |C|]$. Let $C'(f(e)) = C_1(e)$ for all $e \in [1, |C|]$. Then $C \xrightarrow{f(e),t} C'$ and $\text{MS}(C') = M'$ as f is a permutation of processes and $\text{MS}(C_1) = M'$. $\square$

Thanks to the two above lemmas, we now rewrite the verification problems presented before with multiset configurations.

Proposition 2.1. For a protocol $P = (Q, \Sigma, q_{init}, T)$, and a $\diamond \in \{nb, b\}$:

- given a state $q \in Q$, $\text{STATE-COVER}^\diamond(P, q)$ is equivalent to ask if there exist $M_{init} \in \mathcal{M}_{init}$, $M \in \mathcal{M}$ such that $M_{init} \xrightarrow{*}_\diamond M$ and $M(q) > 0$;
- given a configuration $C \in \mathcal{C}$, $\text{CONF-COVER}^\diamond(P, C)$ is equivalent to ask, if there is $M_{init} \in \mathcal{M}_{init}$, $M \in \mathcal{M}$ with $M_{init} \xrightarrow{*}_\diamond M$ and $\text{MS}(C) \leq M$;
- given a state $q \in Q$, $\text{SYNCHRO}^\diamond(P, q)$ is equivalent to find an integer $n \in \mathbb{N}$ such that $\{n \cdot q_{init}\} \xrightarrow{*}_\diamond \{n \cdot q\}$;

where $\xrightarrow{\cdot}_{nb} = \xrightarrow{\cdot}$.

As for configurations, we say that a multiset configuration M is *coverable* [resp. *reachable*] whenever there exists an execution reaching a multiset configuration M' such that $M' \geq M$ [resp. reaching M]. The CONF-COVER problem can be redefined as: given a protocol P and a multiset configuration M , is M coverable?

2.7 STATE OF THE ART

The results presented in this section constitute the state of the art in the verification of distributed systems with a parameterized number of processes, particularly in the context of communication protocols involving broadcast and rendez-vous mechanisms.

2.7.1 Networks with Broadcasts and Rendez-Vous

In the general case, the networks presented earlier are extremely challenging to verify as we will see in this section. Observe that, as the initial configuration is unfixed, exploring all

reachable configurations will only give us a semi-decision procedure for aforementioned problems. Indeed, there are infinitely many initial configurations, and we would need to explore all possible **reachable** configurations starting with any of the initial configurations, until finding the desired pattern (for instance for STATE-COVER: a configuration with at least one process on the state to cover), and stopping then. Hence, alternative techniques are necessary to establish decidability.

If we fix the initial configuration (i. e. the number of processes), the number of reachable configurations is finite, and bounded by an exponential number in the size of the configuration. As a consequence, all previously defined verification problems are decidable.

DECIDABILITY. State coverability and configuration coverability problems are decidable, as shown in [EKo3]. The authors use techniques of Well-Structured Transitions Systems, especially the one presented in [Abd+96; FSo1]. This theory proposes a methodology to solve some verification systems for which set of configurations can be paired with a well-quasi-order (wqo). A *Well-Structured Transitions Systems* (WSTS) correspond to a transition system equipped with a wqo on the configurations, and whose transition relation respects a property called *monotonicity*.

Given a set of configurations S , a relation $\preceq \subseteq S \times S$ is a well-quasi-order on S if it is a quasi-order (reflexive and transitive binary relation) such that any infinite sequence $s_0, s_1, \dots$ from S contains an increasing pair $s_i \preceq s_j$ with $i < j$. Given a transition system $(S, \rightarrow)$ and a wqo $\preceq \subseteq S \times S$, the relation $\rightarrow$ is said to be monotonic with respect to $\preceq$ if for all $s_1, s_2 \in S$ such that $s_1 \rightarrow s_2$, and for all s_3 such that $s_1 \preceq s_3$, there exists s_4 such that $s_2 \preceq s_4$ and $s_3 \rightarrow^* s_4$. A WSTS is defined as a tuple $(S, \rightarrow, \preceq)$ such that (i) $(S, \rightarrow)$ is a transition system, (ii) $\preceq$ is a wqo on S and (iii) $\rightarrow$ is monotonic with respect to $\preceq$.

Given a configuration s , asking if one can reach a state s' such that $s \preceq s'$ from an initial configuration s_0 is known to be decidable [Abd+96; FSo1]. This leads to the decidability of coverability issues of many infinite state systems, as authors show in [FSo1], that for instance Petri Nets and Lossy Channel Systems can be seen as WSTS. It is also the case for parameterized networks [EKo3]: taking as S the set of multiset configurations over a protocol P and $\preceq$ as the multiset order. It is easy to see that the relation obtained from communication steps is monotonic with respect to $\preceq$. We formally establish it below.

Lemma 2.4. Given a protocol $P = (Q, \Sigma, q_{init}, T)$, the transition system $(\mathcal{M}, \rightarrow)$ together with the order $\leq$ on multisets is a WSTS.

Proof. The order $\leq$ is a wqo on $\mathcal{M}$. Given two multiset configurations $M_1, M_2 \in \mathcal{M}$ such that $M_1 \xrightarrow{t} M_2$, and a multiset configuration M_3 such that $M_1 \leq M_3$, then:

- if $t = (q, !a, q')$ then let us rewrite $M_1 = \langle q, q_1, \dots, q_n \rangle$ and $M_2 = \langle q', q'_1, \dots, q'_n \rangle$. As $M_1 \leq M_3$, one can rewrite M_3 as $\langle q, q_1, \dots, q_n, p_1, \dots, p_k \rangle$. Define p'_i as $p'_i = p_i$ if $a \notin R(p_i)$ or a state such that $(p_i, ?a, p'_i) \in T$, for all $1 \leq i \leq k$. It then holds that $M_3 \xrightarrow{t} \langle q, q'_1, \dots, q'_n, p'_1, \dots, p'_n \rangle$ and $M_2 \leq \langle q, q'_1, \dots, q'_n, p'_1, \dots, p'_n \rangle$;
- if $t = (q, !a, q')$ with a **rendez-vous step**, then there exists $(p, ?a, p') \in T$ such that $(M_1 - \langle q \rangle)(p) > 0$ and $M_2 = M_1 - \langle p, q \rangle + \langle p', q' \rangle$. As $M_1 \leq M_3$, we write $M_3 = M_1 + M$. From the **rendez-vous step** definition, $M_3 \xrightarrow{t} M_2 + M \geq M_2$;
- if $t = (q, !a, q')$ with a **sending step**, then $(M_1 - \langle q \rangle)(p) = 0$ for all $p \in Q_?(a)$, and $M_2 = M_1 - \langle q \rangle + \langle q' \rangle$. As $M_1 \leq M_3$, we write $M_3 = M_1 + M$. If $M(p) = 0$ for all

$p \in Q_?(a)$, then, $M_3 \xrightarrow{t} M_3 - \langle q \rangle + \langle q' \rangle$ with a **sending step** and $M_3 - \langle q \rangle + \langle q' \rangle = M_2 + M \geq M_2$. Otherwise, there exists $(p, ?a, p') \in T$ such that $M(p) > 0$. Hence, $M_3 \xrightarrow{t} M_3 - \langle q, p \rangle + \langle q', p' \rangle$ with a **rendez-vous step** and $M_3 - \langle q, p \rangle + \langle q', p' \rangle = M_2 + M - \langle p \rangle + \langle p' \rangle$. As $M(p) > 0$, we get that $M_2 + M - \langle p \rangle + \langle p' \rangle \geq M_2$.

Hence $\rightarrow$ is monotonic with respect to $\leq$, and $(\mathcal{M}, \rightarrow, \leq)$ is a WSTS. $\square$

The above lemma's proof also justify that $((\mathcal{M}, \rightarrow_b), \leq)$ is a WSTS.

As shown in [EK03], this leads to the following theorem.

Theorem 2.1 (Section 4.1 in [EK03]). **STATE-COVER^{nb}**, **STATE-COVER^b**, **CONF-COVER^{nb}** and **CONF-COVER^b** are decidable.

ACKERMANN-HARDNESS. In [SS13][Fact 16, Remark 17], the authors prove Ackermann-hardness of both problems with the blocking semantics (when we don't allow sending steps, but only broadcast steps and rendez-vous steps). They rely on a simulation of a Minsky Machine bounded by a Hardy function.

Theorem 2.2 ([SS13]). **STATE-COVER^b** and **CONF-COVER^b** are Ackermann-hard.

However, it is possible to retrieve the lower bound for **B-protocols**, by using the fact that **rendez-vous steps** can be simulated by broadcast as shown in [ARZ15; EFM99].

The idea is illustrated in Figure 2.2. Instead of taking a transition labeled by $!a$, a process now *broadcasts* a request on message a : $\text{req}(a)$, and then waits for a process to answer its request with a message $\text{ans}(a)$ [left part of Figure 2.2]. Other processes, when receiving $\text{req}(a)$, pause their execution. If they were on a state from which they could receive message a , they can answer the request by broadcasting $\text{ans}(a)$, in that case the message is received by all other processes. If another process answers the request first, they go back to their old state: this way only one process answers the request [middle part of Figure 2.2]. If they were not on a state from which they could receive the message, they just wait for the rendez-vous to be completed (i. e. for one broadcast of $\text{ans}(a)$) [right part of Figure 2.2]. As authors consider the blocking semantics, a message is always received when sent, and so there is always at least one process to answer the request, and so this transformation does not create deadlocks.

Proposition 2.2 (Transformation from [ARZ15; EFM99]). Let P be a protocol, and P' be the B-protocol obtained from P thanks to the transformation described in Figure 2.2. If $C, C' \in \mathcal{C}[P]$ such that $C \rightarrow_P C'$, then $C, C' \in \mathcal{C}[P']$ and $C \rightarrow_{P'}^+ C'$.

Thanks to the above proposition, hardness results proved in [SS13] for protocols extends to B-protocols.

Theorem 2.3 ([ARZ15; EFM99; SS13]). **STATE-COVER^{nb}** and **CONF-COVER^{nb}** are Ackermann-hard, even when restricting the input protocols to **B-protocols**.

UNDECIDABILITY. In [EFM99], the authors show undecidability of **REP-COVER** problems when restricting the input protocols to B-protocols: they proceed by showing a reduction from the halting problem for 2-counter machines. A 2-counter machine is a finite state machine equipped with two counters, whose transitions can increment, decrement or test to

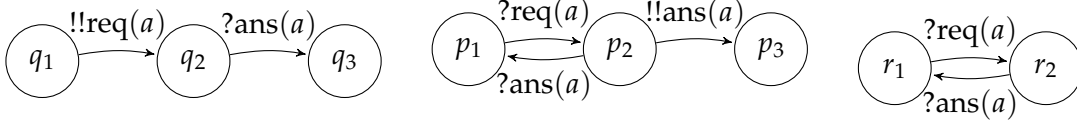


Figure 2.2: Illustration of the transformation for different types of transitions in order to eliminate sending transitions. On the left, the transformation for transition $(q_1, !a, q_3)$ where q_2 is a freshly added intermediate state. In the center, the transformation of a transition $(p_1, ?a, p_3)$ where p_2 is an freshly added intermediate state. On the right, the transformation of a state r_1 with no outgoing transition for message a , where r_2 is a freshly added state.

0 counters' values. The halting problem asks if one can find a run of the machine from an initial configuration (with both counters set to 0) to a final configuration (with both counters' values equal to 0). This problem is known to be undecidable. We will say more about that later, in the next chapter.

Theorem 2.4 (Theorem 5.1 in [EFM99]). REP-COVER^{nb} and REP-COVER^b are undecidable even when restricting the input protocols to *B-protocols*.

As in [EFM99], in [EK03], the authors show undecidability of REP-COVER^{nb} when restricting the input protocols to *RDV-protocols* using the same techniques.

Theorem 2.5 (Section 4.2 in [EK03]). REP-COVER^{nb} is undecidable even when restricting the input protocols to *RDV-protocols*.

2.7.2 Blocking Rendez-Vous Networks

When broadcasts are excluded (i.e., when we consider only *RDV-protocols*), and only rendez-vous communication steps are allowed, the verification problems presented in this thesis have been widely studied. The complexities are significantly lower, as all the problems can be solved with polynomial-time algorithms.

In [GS92], the authors introduce *blocking rendez-vous networks*. They present a polynomial-time algorithm that checks whether, given a logical formula, all process paths produced by the network's executions satisfy the formula.

The logic they consider is Linear Temporal Logic (LTL) without the “next” operator, meaning the formulas use a finite set of atomic propositions on the protocol's states, the constant True, the connectives “not” ($\neg$) and “or” ($\vee$), and the temporal operator “until” (U). With this logic, one can easily express the temporal operators “eventually” (F) and “always” (G).

STATE COVERABILITY. From this, we deduce that the state coverability problem can be solved using this algorithm, as:

- q is coverable if and only if *there exists* a process path produced by the network's executions that satisfies the formula “F q ”.

In other words, the algorithm presented in [GS92] answers negatively to the formula “ $\neg(\text{F } q)$ ” if and only if q is coverable.

This leads to the following theorem.

Theorem 2.6 (Section 4 in [GS92]). *STATE-COVER^b restricted to RDV-protocols is in PTIME.*

CONFIGURATION COVERABILITY. In blocking rendez-vous networks, the configuration coverability problem can be easily reduced to the state coverability problem. Consider a protocol $P = (Q, \Sigma, q_{init}, T)$ and a configuration $(q_1, \dots, q_k)$ that we want to cover. The reduction extends P by adding a sequence of transitions:

$$(q_{init}, ?\mathbf{q}_1, q_{cov,1}), (q_{cov,1}, ?\mathbf{q}_2, q_{cov,2}), \dots, (q_{cov,k-1}, ?\mathbf{q}_k, q_{cov,k}).$$

Messages $\mathbf{q}_i$ are newly introduced messages that can be sent once from the corresponding state, i.e., $(q_i, !\mathbf{q}_i, d)$, where d is a fresh state.

It follows that $q_{cov,k}$ is coverable in the extended version of P if and only if $(q_1, \dots, q_k)$ is coverable in P . If $(q_1, \dots, q_k)$ is coverable, then in the extended protocol, one extra process can remain in q_{init} while the configuration is being covered. Once the configuration is reached, each process covering one of the states $q_1, \dots, q_k$ can “witness” its state by sending the corresponding message. The remaining process receives the messages $\mathbf{q}_1, \dots, \mathbf{q}_k$ and reaches $q_{cov,k}$. Conversely, if $q_{cov,k}$ is coverable, we can identify k processes that sent $\mathbf{q}_1, \dots, \mathbf{q}_k$, respectively. We can reconstruct an execution of P : instead of sending the witness messages, these processes simply remain idle in their states. This execution successfully covers $(q_1, \dots, q_k)$. This leads to the following corollary.

Corollary 2.1. *CONF-COVER^b restricted to RDV-protocols is in PTIME.*

REPEATED COVERABILITY. The work of German and Sistla also proves that repeated coverability is in PTIME. Of course, the repeated coverability of a state q is easily expressible in the logic the authors considered:

- q is coverable infinitely often by a process if and only if *there exists* a process path produced by the network’s executions that satisfies the formula “ $\mathbf{G}(\mathbf{F}(q \wedge \mathbf{F} \neg q))$ ” (the process always eventually reaches q and leaves it).

In other words, the algorithm presented in [GS92] answers negatively to the formula $\neg \mathbf{G}(\mathbf{F}(q \wedge \mathbf{F} \neg q))$ if and only if q is coverable infinitely often by a process.

When we ask a process to take a transition infinitely often, there is a subtlety: if there are several outgoing transitions from q , we need to ensure that the appropriate transition is taken infinitely often. Consider a protocol $P = (Q, \Sigma, q_{init}, T)$ and a transition $t = (q, \alpha, q') \in T$ that we wish to cover infinitely often. We copy state q' into a state q'_{copy} : we only keep (q, α, q'_{copy}) as its incoming transition, and we copy all the outgoing transitions of state q' into outgoing transitions of state q'_{copy} . Covering state q'_{copy} infinitely often is then equivalent to repeatedly taking transition (q, α, q'_{copy}) . This leads to the following theorem:

Theorem 2.7 (Section 4 in [GS92]). *REP-COVER^b restricted to RDV-protocols is in PTIME.*

SYNCHRONIZATION. Unlike previous problems, the logic considered in [GS92] does not suffice to express the synchronization problem. Intuitively, synchronization expresses a *global* property concerning all processes in an execution. Thus, it is not enough to find a single path satisfying “ $\mathbf{F}(q \wedge \mathbf{G}q)$ ”. On the other hand, if *all* paths satisfy this formula, it

would imply that for *any* number of processes and *all* executions, every process eventually reaches and remains in q , which is far stronger than what synchronization requires.

In [HS20] and later in [BER21], the authors study the *cut-off problem* in *blocking rendez-vous networks*, which asks, given a protocol $P = (Q, \Sigma, q_{init}, T)$ and a state $q_f \in Q$, whether there exists a bound B such that, for any number of processes N above B , it holds that $\{N \cdot q_{init}\} \xrightarrow{*} \{N \cdot q_f\}$. To establish their result, they introduce a *scaling lemma* that relates the behavior of rendez-vous networks to that of *continuous Petri nets*, a relaxed model in which the number of tokens can be a rational value instead of an integer, and where transitions can fire partially. More precisely, in a continuous Petri net, a transition that normally produces one token can be taken, for instance, half a time, thereby generating half a token.

In [FH15], the authors show that the reachability problem for continuous Petri nets is in PTIME, a key ingredient in the analysis of [BER21]. The techniques developed in [BER21], in particular the scaling lemma, combined with the results from [FH15], suffice to prove that synchronization is in PTIME.

Theorem 2.8 ([BER21]). SYNCHRO^b restricted to rendez-vous protocols is in PTIME.

2.7.3 In this thesis

Our primary motivation stems from Java thread programming, where threads interact via `notify()` and `notifyAll()` methods. When a thread is suspended in a waiting state, it can be woken up either by `notify()`, which targets a single thread (leading to a rendez-vous), or by `notifyAll()`, which wakes up all waiting threads (similar to a broadcast). Notably, in Java, the sender thread is not blocked if no other thread is waiting for its message; the message is simply discarded. This non-blocking behavior inspired Delzanno et al. [DRVB02] to introduce non-blocking rendez-vous as a communication primitive, allowing the sender to proceed even if no process receives its message.

In [DRVB02], the authors generalize Petri nets by introducing Multi-Transfer Nets (MTNs) which feature multi-transfer transitions, enabling the simultaneous movement of multiple tokens between places. This captures synchronization mechanisms like broadcasts (used to model `notifyAll()`) and asynchronous rendez-vous (used for `notify()`). They establish a connection between MTNs and broadcast protocols, leveraging techniques from the verification of broadcast networks to analyze MTNs. The paper establishes decidability results for the control state reachability problem in MTNs. Specifically, it also proves that state coverability and configuration coverability are decidable using a symbolic backward reachability algorithm. Experimental results show the method's effectiveness in verifying safety properties of Java synchronization patterns.

However, while the verification of networks featuring `notifyAll()` (i.e., broadcast-based communication) has been extensively studied, the case where threads interact solely via `notify()` remains less explored.

In this thesis, we build on this work by analyzing verification problems in networks restricted to sending steps and rendez-vous steps, excluding broadcasts. We further extend this research by proposing restrictions well-suited to our motivation and analyzing the complexities of the verification problems relevant to us.

2.7.4 *Recapitulative table of existing results.*

We present a recapitulative table of the existing results.

	STATE-COVER [◇]	CONF-COVER [◇]	SYNCHRO [◇]	REP-COVER [◇]
Blocking Rdv Networks ($\diamond = \flat$ and RDV-protocol)	in PTIME [GS92; BER21]			
Rdv Networks ($\diamond = nb$ and RDV-protocol)	Decidable [DRVB02; EK03]		?	Undecidable [EK03; EFM99]
Broadcast Networks (B-protocol)	Decidable [DRVB02; EK03] but Ackermann-hard [ARZ15; EFM99; SS13]		?	
Blocking Networks ($\diamond = \flat$)			?	
Networks ($\diamond = nb$)			?	

COUNTER MACHINES

This chapter introduces different classes of counter systems, which will serve as key ingredients throughout this thesis for the study of networks defined in the previous chapter.

3.1 INTRODUCTION

Counter systems are finite-state automata equipped with counters that take values in non-negative numbers. Each transition can test or update the counters' values. Depending on the allowed operations, different families of systems can be defined. They can be used to represent real-life systems and also the behavior of programs working with integer variables. In general, these systems have Turing-complete computational power [Min67], mainly due to the ability to test whether a counter equals zero. In this context, alternative families of systems have been introduced, avoiding counter tests. In [Pet62], Petri nets were introduced as a well-known model, equivalent to Vector Addition Systems with States (VASS) [HP79], or even Vector Addition Systems (VAS) [KM69]. These models can encode certain distributed networks (e.g., rendez-vous networks [GS92]) by relying on a counting abstraction, where individual process identities are ignored, and only their number is kept. Verification problems such as reachability or coverability are decidable in this context but are often infeasible in practice: reachability is Ackermann-complete. Many classes of counter systems have been studied over the years, exploring various restrictions and extensions of VASS. These include models with affine updates [FGH13], reset or transfer operations [DFS98], arbitrarily decreasing counters (lossy VASS) [BM99], a single counter with zero-tests [Göl+10], or counters that can take negative values ($\mathbb{Z}$ -VASS) [HH14]. Another important variant is the monus VASS [Bau+23; Rei13], where transitions are always enabled but counters never go below zero: they remain at zero instead of taking negative values.

In this chapter, we introduce Non-Blocking Counter Machines (NB-CM), a new model that extends VASS by allowing a selective form of non-blocking decrements. This model generalizes both classical VASS and monus VASS [Bau+23; Rei13]. In classical VASS, a decrement transition is only enabled when the counter is strictly positive. In contrast, monus VASS allow decrement transitions to fire unconditionally, ensuring that counters never take negative values: when a decrement is applied at zero, the counter simply remains unchanged.

NB-CM introduce a mixed approach, where decrement transitions can be either regular or non-blocking. A regular decrement behaves as in classical VASS: it is only enabled when the counter is strictly positive and decreases its value by one. A non-blocking decrement, however, is always enabled. If the counter is positive, it decreases as expected; if it is zero, the transition still fires, but the counter remains at zero.

We begin in Section 3.2 by giving a general definition of counter machines. Then, in Section 3.3, we give the definitions of VASS and VAS and explain how it relates to counter machines. In Section 3.4, we formally define Non-Blocking Counter Machines and illustrate their behavior with an example.

3.2 COUNTER MACHINES

A canonical system is introduced in [Min67] where allowed operations are: decrementing a counter's value, incrementing it or testing it to 0. In this work, we call this system **Counter machines**. They are Turing-powerful already with two counters [Min67]. We define **Counter machines** formally below.

To define counter machines, given a set of integer variables (also called counters) X , we use the notation $\text{CAct}(X)$ to represent the set of associated actions, defined as $\{x+, x-, x=0 \mid x \in X\} \cup \{\perp\}$. Intuitively, $x+$ increments the value of the counter x , while $x-$ decrements it and $x=0$ checks if it is equal to 0. The action $\perp$ is an internal action with no effects on counters.

Definition 3.1 (Counter machine). A *counter machine* (abbreviated as CM) is a tuple $\text{CM} = (\text{Loc}, X, \Delta, \ell_{in})$ where Loc is a finite set of locations, $\ell_{in} \in \text{Loc}$ is the initial location, X is a finite set of counters, and $\Delta \subseteq \text{Loc} \times \text{CAct}(X) \times \text{Loc}$ is a finite set of transitions.

A configuration of a CM $\text{CM} = (\text{Loc}, X, \Delta, \ell_{in})$ is a pair (ℓ, v) where $\ell \in \text{Loc}$ specifies the current location of the CM and $v \in \mathbb{N}^X$ assigns a natural value to each counter. The size of a CM CM is defined as $|\text{CM}| = |\Delta| \times 2|\text{Loc}| \times 3|X|$ (we assume that we write the CM in unary).

Given two configurations (ℓ, v) and (ℓ', v') and a transition $\delta \in \Delta$, we define $(\ell, v) \xrightarrow{\delta}_{\text{CM}} (\ell', v')$ if and only if $\delta = (\ell, op, \ell')$ and one of the following holds:

- $op = \perp$ and $v = v'$;
- $op = x+$ and $v'(x) = v(x) + 1$ and $v'(x') = v(x')$ for all $x' \in X \setminus \{x\}$;
- $op = x-$ and $v'(x) = v(x) - 1$ and $v'(x') = v(x')$ for all $x' \in X \setminus \{x\}$;
- $op = x=0$ and $v(x) = 0$ and $v' = v$.

We write $(\ell, v) \rightsquigarrow_{\text{CM}} (\ell', v')$ if there exists $\delta \in \Delta$ such that $(\ell, v) \xrightarrow{\delta}_{\text{CM}} (\ell', v')$, and when the context is clear we drop the subscript CM. We let $\mathbf{0}_X$ be the valuation such that $\mathbf{0}_X(x) = 0$ for all $x \in X$. The initial configuration of CM is $(\ell_{in}, \mathbf{0}_X)$. The set of configurations of the counter machine together with relation $\rightsquigarrow$ is a transition system. This will be the case for all the systems defined in this chapter. Hence, we will use the notions of **runs** and **executions**, as defined for transitions systems in Section 2.2.

Example 3.1. In Figure 3.1, we display a **counter machine** with 6 locations denoted CM. Its initial location is ℓ_{in} and it has two counters x_1 and x_2 . For readability's sake, we write (ℓ, x_1, x_2) for a configuration (ℓ, v) with $v(x_1) = x_1$ and $v(x_2) = x_2$. Consider the following chain of events:

$$(\ell_{in}, 0, 0) \rightsquigarrow_{\text{CM}} (\ell_1, 1, 0) \rightsquigarrow_{\text{CM}} (\ell_2, 2, 0)$$

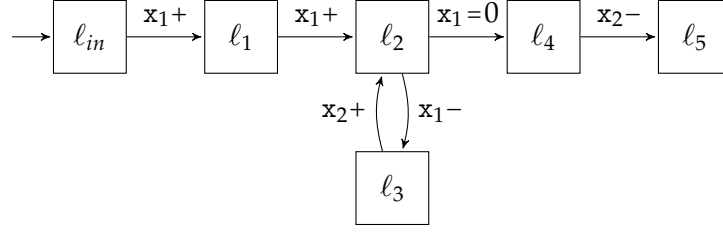


Figure 3.1: Example of a Counter Machine denoted CM.

At this point, x_1 has value 2 hence the transition to ℓ_4 can not be taken as it requires x_1 to be null. Hence, we can continue this way:

$$\begin{aligned}
 (\ell_2, 2, 0) &\rightsquigarrow_{\text{CM}} (\ell_3, 1, 0) \rightsquigarrow_{\text{CM}} (\ell_2, 1, 1) \rightsquigarrow_{\text{CM}} (\ell_3, 0, 1) \rightsquigarrow_{\text{CM}} \\
 (\ell_2, 0, 2) &\rightsquigarrow_{\text{CM}} (\ell_4, 0, 2) \rightsquigarrow_{\text{CM}} (\ell_5, 0, 1)
 \end{aligned}$$

This time, the transition $(\ell_2, x_1=0, \ell_4)$ can be taken as x_1 has value 0 in $(\ell_2, 0, 2)$.

In the context of counter systems, we will focus on two main problems: reachability and coverability. The coverability problem asks if, given a specific location, one can reach a configuration with this specific location. The reachability problem asks if, given a specific location, one can reach a configuration with this specific location and counters' values at 0. In the context of counter machine, it is stated as follows: Given a counter machine $\text{CM} = (\text{Loc}, X, \Delta, \ell_{in})$ and a location $\ell \in \text{Loc}$,

- **COVER[CM]** asks if there exists $v \in \mathbb{N}^X$ such that $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell, v)$?
- **REACH[CM]** asks if $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell, \mathbf{0}_X)$?

As we mentioned earlier, counter machines are Turing powerful, already with two counters, leading to the following theorem.

Theorem 3.1 ([Min67]). *COVER[CM] and REACH[CM] are undecidable, even with two counters.*

3.3 VASS AND VAS

We now dive into systems without tests to 0. Vector Addition Systems with States (VASS) introduced in [HP79], are counter systems without test to 0 where the updates are modeled as a vector of integers over the counters. They are formally defined below.

Definition 3.2 (VASS). A Vector Addition System with States (shortly VASS) is a tuple $V = (\text{Loc}, \ell_{in}, X, \Delta)$ where Loc is a finite set of locations; $\ell_{in} \in \text{Loc}$ is the *initial* location; X is a finite set of counters; and $\Delta \subseteq \text{Loc} \times \mathbb{Z}^X \times \text{Loc}$ is a finite set of transitions.

A configuration of a VASS $V = (\text{Loc}, X, \Delta, \ell_{in})$ is defined as before, by a pair (ℓ, v) in $\text{Loc} \times \mathbb{N}^X$. The initial configuration is $(\ell_{in}, \mathbf{0}_X)$. We consider the size of the VASS to be $|V| = |\Delta|(2|\text{Loc}| + |X|\|\Delta\|)$ (where $\|\Delta\| = \max(\|v\| \mid \exists \ell, \ell' \in \text{Loc} \text{ s.t. } (\ell, v, \ell') \in \Delta)$) as we will consider that counters are encoded in unary. If counters are encoded in binary, we would only need $|\Delta|(2|\text{Loc}| + |X|\log_2(\|\Delta\|))$ bits to write V . Complexity results are the same for VASS with a unary encoding and VASS with a binary encoding.

Let $(\ell, v), (\ell', v')$ be two configurations, and $(\ell, \delta, \ell') \in \Delta$. We say that $(\ell, v) \xrightarrow{\delta}_{\mathbf{V}} (\ell', v')$ whenever for all counters $x \in X$, $v'(x) = v(x) + \delta(x) \geq 0$. We write $(\ell, v) \rightsquigarrow (\ell', v')$ whenever there exists a transition $\delta \in \Delta$ such that $(\ell, v) \xrightarrow{\delta}_{\mathbf{V}} (\ell', v')$, and when the context is clear we drop the subscript $\mathbf{V}$.

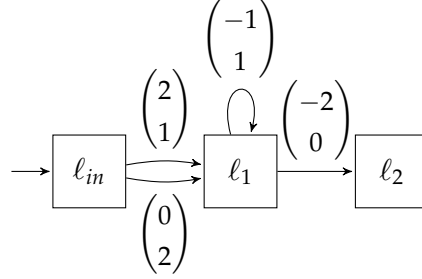


Figure 3.2: Example of a VASS denoted $\mathbf{V}$.

Example 3.2. A VASS with two counters x_1, x_2 is displayed in Figure 3.2. From $(\ell_{in}, 0, 0)$ two transitions can be taken: the one above adding 2 to x_1 and 1 to x_2 and the one below, adding 2 to x_2 . Assume the latter, then $(\ell_{in}, 0, 0) \rightsquigarrow_{\mathbf{V}} (\ell_1, 0, 2)$. Observe that from $(\ell_1, 0, 2)$, it is not possible to take the transition $(\ell_1, \begin{pmatrix} -2 \\ 0 \end{pmatrix}, \ell_2)$ as counter x_1 's value should stay above 0.

As for CM, coverability and reachability problems are defined as follows, given a $\mathbf{V} = (\text{Loc}, X, \Delta, \ell_{in})$ and a location $\ell \in \text{Loc}$:

- **COVER[VASS]** asks if there exists $v \in \mathbb{N}^X$ such that $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell, v)$?
- **REACH[VASS]** asks if $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell, \mathbf{0}_X)$?

We also define Vector Addition Systems (VAS), introduced in [KM69], which are essentially VASS without locations.

Definition 3.3 (VAS). Let $d \in \mathbb{N}$. A VAS of dimension d is a tuple $\mathbf{V} = (\Delta, v_{\text{init}})$ where $\Delta \subseteq \mathbb{Z}^d$ is a finite set of transitions and $v_{\text{init}} \in \mathbb{N}^d$ is an initial vector.

A configuration of a VAS of dimension $d \in \mathbb{N}$ is a vector $v \in \mathbb{N}^d$. The initial configuration is then v_{init} . We consider the size of the VAS to be $|\mathbf{V}| = d \times (\|v_{\text{init}}\| + |\Delta| \times \|\Delta\|)$, as before, we assume that counters are encoded in unary.

Given a VAS $\mathbf{V} = (\Delta, v_{\text{init}})$, two vectors $v, v' \in \mathbb{N}^d$ and a transition $\delta \in \Delta$, we say that $v \xrightarrow{\delta}_{\mathbf{V}} v'$ whenever for all counters $x \in X$, $v'(x) = v(x) + \delta(x) \geq 0$. We write $v \rightsquigarrow_{\mathbf{V}} v'$ whenever there exists a transition $\delta \in \Delta$ such that $v \xrightarrow{\delta}_{\mathbf{V}} v'$, and when the context is clear we drop the subscript $\mathbf{V}$.

In the context of VAS coverability and reachability problems are defined as follows, given a $\mathbf{V} = (\Delta, v_{\text{init}})$ of dimension d and a vector $v \in \mathbb{N}^d$:

- **COVER[VAS]** asks if there exists $v' \in \mathbb{N}^d$ such that $v' \geq v$ and $v_{\text{init}} \rightsquigarrow^* v'$?
- **REACH[VAS]** asks if $v_{\text{init}} \rightsquigarrow^* v$?

The model of **VASS** might seem more expressive than **VAS**, however they are equivalent in many ways. Of course, a **VAS** can be seen as a **VASS** with one single location. There is also a reduction from **VASS** to **VAS**, for instance in [HP79], where the authors present a way to turn a **VASS** into a **VAS** with only 3 additional counters, and an overall new size polynomial in the size of the **VASS**. As a result, both models are equivalent for coverability and reachability.

Coverability problem is decidable on those systems, as they transition system, together with the usual order on vectors, form a **well-structured transition system**. In [Lip76], the author shows that the coverability problem for **VASS** (and equivalently **VAS**) is **EXPSPACE-hard**. The proof shows that one can simulate a **2EXP**-bounded CM, i. e. a CM whose reachable configurations have counters' values bounded by a double exponential function in the size of the **VASS**, with a **VASS**, which is itself **2EXP**-bounded. As a **2EXP**-bounded CM can simulate a deterministic Turing machine with a tape using an exponential space in the size of the input, it leads to **EXPSPACE-hardness** of the coverability problem with **VASS**. Later in [Rac78], the author shows a matching lower bound by proving that the problem (for **VAS**, and equivalently **VASS**) is in **EXPSPACE**, by minimizing the length of executions witnessing coverability.

Theorem 3.2 (Hardness [Lip76], Membership [Rac78]). *COVER[VASS] and COVER[VAS] are EXPSPACE-complete.*

The reachability problem for **VASS** has been known to be decidable for a long time [May81]. In [LS19], Leroux and Schmitz proved that the problem is primitive-recursive. For many years, the known lower bound was the one of coverability, indeed in all the systems with locations we defined here above, the coverability reduces to the reachability by adding to the target location decrement transitions for each counters. In [CO21; Ler21], authors proved that it is Ackermann-hard.

Theorem 3.3 ([LS19; CO21; Ler21]). *REACH[VASS] and REACH[VAS] are Ackermann-complete.*

Note that **VASS** are closely related to Counter machines without tests to 0, we call such machines, **test-free CMs**.

Definition 3.4 (*test-free CM*). A CM $CM = (Loc, X, \Delta, \ell_{in})$ is **test-free** (shortly **test-free CM**) whenever $\Delta \cap (Loc \times \{x=0 \mid x \in X\} \times Loc) = \emptyset$.

The only difference being that the updates of counters: a transition of the **VASS** can perform multiple increments and decrements on several counters in one transition, whereas in a **test-free CM**, one transition only decrements or increments a single counter. Obviously, a **test-free CM** can be easily transformed into a **VASS**. To transform a **VASS** into a **test-free CM**, one can unfold each transition into several increments and decrements transitions. We show an example on how this can be done in Example 3.3, a transition of the **VASS** δ produces then $\sum_{x \in X} |\delta(x)|$ transitions in the **test-free CM**, leading to a **test-free CM** of size $\leq |\Delta|(1 + |X| \times \|\Delta\|)2(|Loc| + |X| \times \|\Delta\| \times |\Delta|)3|X|$, i. e. polynomial in the size of the **VASS**. The reduction transforming a **VASS** into a **test-free CM** can lead to an exponential blowup of the **test-free CM**'s space when components of the **VASS**'s transitions are encoded in binary. However, we previously mentioned that we would only consider **VASS** encoded in unary as complexity results are the same in both cases.

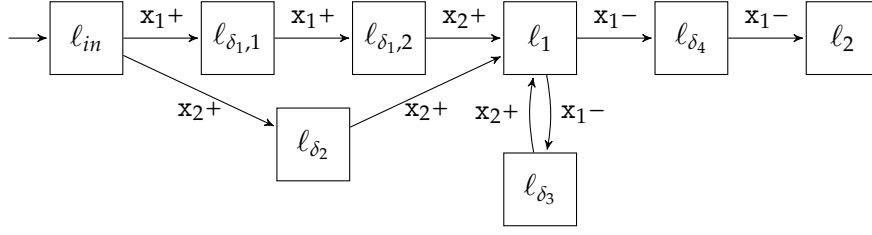


Figure 3.3: Example of a test-free CM with the same behavior as V of Figure 3.2. We denote it CM_V .

Example 3.3. We also present a **test-free CM** with the similar behavior as the **VASS** V depicted in Figure 3.3. It has the same number of counters, but its number of locations has increased; it is actually bounded by $\|\Delta\| \times |X| \times |\Delta| + |\text{Loc}|$ where Loc is the set of locations of the **VASS**, Δ its set of transitions and X its set of counters.

We denote by $\text{COVER}[\text{test-free CM}]$ and $\text{REACH}[\text{test-free CM}]$ the restrictions of $\text{COVER}[\text{CM}]$ and $\text{REACH}[\text{CM}]$, respectively, to test-free CM. We have the following corollary.

Corollary 3.1. *The two following results hold.*

- $\text{COVER}[\text{test-free CM}]$ is **EXPSPACE-complete**;
- $\text{REACH}[\text{test-free CM}]$ is **Ackermann-complete**.

3.4 NON-BLOCKING COUNTER SYSTEMS

In the ongoing effort to define VASS extensions and restrictions that ensure decidability of verification questions, unlike general counter machines, a notable example is $\mathbb{Z}$ -VASS [HH14], a variant where counters take values in $\mathbb{Z}$ rather than $\mathbb{N}$. As a result, transitions are always enabled, even when they put some counters below 0. In this setting, both coverability and reachability are NP-complete.

Another related approach considers VASS variants where transitions are non-blocking and is called *monus VASS*: they can always be fired, but counters never take negative values—if a counter is at zero, it remains unchanged. This model was first studied in [Rei13] and later in [Bau+23], where the authors established that coverability remains NP-complete. They also proved that reachability (when asking to reach a specific location with counters set to zero) is EXPSPACE-complete, whereas the variant requiring a specific valuation of counters is Ackermann-complete. Note that these two definitions of reachability are equivalent in the general case, but not for monus VASS.

Building on these ideas, we introduce a refined model where only some decrements are non-blocking, while others behave as in classical counter machines. Specifically:

- A *non-blocking decrement* can be applied regardless of the counter's value: if the counter is positive, it decreases by 1; if it is zero, the transition still fires without modifying the counter.
- A *regular decrement* can only be fired if the counter is strictly positive, in which case it decreases by 1. Otherwise, the transition is not enabled.

Definition 3.5 (NB-CM). A non-blocking test-free counter machine (shortly **NB-CM**) is a tuple $\text{CM} = (\text{Loc}, X, \Delta_b, \Delta_{nb}, \ell_{in})$ such that $(\text{Loc}, X, \Delta_b, \ell_{in})$ is a test-free CM and $\Delta_{nb} \subseteq \text{Loc} \times \{nb(x-) \mid x \in X\} \times \text{Loc}$ is a finite set of non-blocking decrements.

Observe that in a **NB-CM**, both blocking and non-blocking decrements are possible, depending on the type of transition taken. It is however closely related to monus VASS. Let $\text{CM} = (\text{Loc}, X, \Delta_b, \Delta_{nb}, \ell_{in})$, a configuration is given by a pair $(\ell, v) \in \text{Loc} \times \mathbb{N}^X$ and the initial configuration is $(\ell_{in}, \mathbf{0}_X)$. Its size is $|\text{CM}| = (|\Delta_b| + |\Delta_{nb}|) \times 2|\text{Loc}| \times 3|X|$.

Given two configurations (ℓ, v) and (ℓ, v') and $\delta \in \Delta_b \cup \Delta_{nb}$, we extend the transition relation $(\ell, v) \xrightarrow{\delta}_{\text{CM}} (\ell, v')$ over the set Δ_{nb} in the following way: for $\delta = (\ell, nb(x-), \ell') \in \Delta_{nb}$, we have $(\ell, v) \xrightarrow{\delta}_{\text{CM}} (\ell', v')$ if and only if $v'(x) = \max(0, v(x) - 1)$, and $v'(x') = v(x')$ for all $x' \in X \setminus \{x\}$.

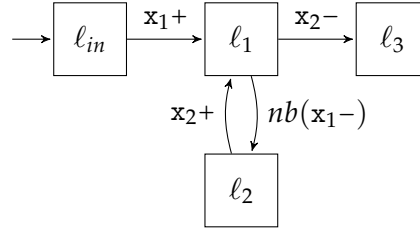


Figure 3.4: Example of a NB-CM denoted CM_{nb} .

Example 3.4. We depict an example of **NB-CM** with two counters in Figure 3.4, denoted CM_{nb} . Consider the following chain of events:

$$\begin{aligned}
 (\ell_{in}, 0, 0) &\rightsquigarrow_{\text{CM}_{nb}} (\ell_1, 1, 0) \rightsquigarrow_{\text{CM}_{nb}} (\ell_2, 0, 0) \rightsquigarrow_{\text{CM}_{nb}} (\ell_1, 0, 1) \rightsquigarrow_{\text{CM}_{nb}} \\
 (\ell_2, 0, 1) &\rightsquigarrow_{\text{CM}_{nb}} (\ell_1, 0, 2) \rightsquigarrow_{\text{CM}_{nb}} (\ell_3, 0, 1)
 \end{aligned}$$

Observe that the first time the transition $(\ell_1, nb(x_1-), \ell_2)$ is taken, x_1 is decremented as its value is above zero in configuration $(\ell_1, 1, 0)$. The second time the transition is taken, x_1 remains at 0 as its value is equal to 0 in $(\ell_1, 0, 1)$.

Let $\text{CM} = (\text{Loc}, X, \Delta_b, \Delta_{nb}, \ell_{in})$ be an NB-CM. We say that CM is an **NB-CM with restore** (shortly **NB-R-CM**) when $(\ell, \perp, \ell_{in}) \in \Delta_b$ for all $\ell \in \text{Loc}$, i.e. from each location, there is a transition leading to the initial location with no effect on the counters values. To the best of our knowledge, NB-CM have not been previously studied. We study both NB-CM and NB-R-CM in Chapter 5. We show that these two systems have the same complexity for the coverability problem.

This thesis focuses on networks of processes, and we introduce these machines to establish complexity results for **rendez-vous networks**. In Chapter 5, after analyzing the complexity of coverability for NB-CM and NB-R-CM, we establish a two-sense reduction between NB-R-CM and **rendez-vous networks** for coverability problems. In this part of our work, restore transitions are necessary, as we cannot fully simulate NB-CM using rendez-vous networks.

We extend the definition of $\text{COVER}[\text{CM}]$ to NB-CM and NB-R-CM:

- given a NB-CM [resp. a NB-R-CM] $\text{CM} = (\text{Loc}, X, \Delta_b, \Delta_{nb}, \ell_{in})$, and a location $\ell \in \text{Loc}$, $\text{COVER}[\text{NB-CM}]$ [resp. $\text{COVER}[\text{NB-R-CM}]$] asks if there exists $v \in \mathbb{N}^{|X|}$ such that $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell, v)$?

We also define Non-Blocking VAS (shortly NB-VAS). An NB-VAS is composed of a set of transitions over vectors of dimension d , sometimes called counters, and an initial vector of d non-negative integers, like a VAS so far. However, in an NB-VAS, a transition is a pair of vectors: one is a vector of d integers and is called the *blocking* part of the transition and the other one is a vector of d non-negative integers and is called the *non-blocking* part of the transition.

Definition 3.6. Let $d \in \mathbb{N}$. A d -dimension Non-blocking Vector Addition System (NB-VAS) V is a tuple $(\Delta, v_{\text{init}})$ such that $\Delta \subseteq \mathbb{Z}^d \times \mathbb{N}^d$ and $v_{\text{init}} \in \mathbb{N}^d$.

Its size is defined as $|V| = d \times (\|v_{\text{init}}\| + |\Delta| \times \max(\|\delta_b\| + \|\delta_{nb}\| \mid (\delta_b, \delta_{nb}) \in \Delta))$. Definition of configurations naturally extends from VAS to NB-VAS. Formally, for two vectors $v, v' \in \mathbb{N}^d$, and a transition $\delta = (\delta_b, \delta_{nb}) \in \Delta$, we write $v \xrightarrow{\delta}_V v'$ if there exists $v'' \in \mathbb{N}^d$ such that $v'' = v + \delta_b$ and, for all $i \in [1, d]$, $v'(i) = \max(0, v''(i) - \delta_{nb}(i))$. We write $\rightsquigarrow_V$ for $\bigcup_{\delta \in \Delta} \xrightarrow{\delta}_V$. When the context is clear, we drop the subscript V .

Intuitively, the blocking part δ_b of the transition has a strict semantics: to be taken, it needs to be applied to a vector large enough so that no value goes below 0. The non-blocking part δ_{nb} (which is subtracted from the vector) can be taken even if it decreases some component below 0: the corresponding component will simply be set to 0.

We extend the problem COVER[VAS] to NB-VAS:

- given an NB-VAS $V = (\Delta, v_{\text{init}})$ of dimension $d \in \mathbb{N}$ and a vector $v \in \mathbb{N}^d$, COVER[NB-VAS] asks if there exists $v' \in \mathbb{N}^d$ such that $v' \geq v$ and $v_{\text{init}} \rightsquigarrow^* v'$?

We will see in Section 5.2.1.2 that COVER[NB-CM] reduces to COVER[NB-VAS].

3.5 SUMMARY

We conclude this chapter by a summary table. We write $\simeq$ to say that systems are inter-reducible with respect to COVER and REACH. One reduction from NB-CM to NB-VAS is presented in Chapter 5.

	Counter Machines	test-free CM $\simeq$ VASS $\simeq$ VAS	NB-CM $\simeq$ NB-VAS	NB-R-CM
REACH	Undecidable [Min67]	Ackermann-complete [LS19; CO21; Ler21]	?	?
COVER	Undecidable [Min67]	EXPSPACE-complete [Lip76; Rac78]	EXPSPACE-complete (Chapter 5)	EXPSPACE-complete (Chapter 5)

UNDECIDABILITY OF SYNCHRONIZATION

We start by showing that SYNCHRO is quickly undecidable: it stays undecidable when considering broadcast networks or Rdv networks. In both cases, we can simulate a 2-counter machines, thanks to a mechanism that allows to isolate a process, called the *leader*, in a specific region of the protocol that corresponds to the control component of the counter machine. In broadcast networks, this can be achieved easily: from the initial configuration, the leader broadcasts a message received by all other (non-leader) processes, thereby establishing its special role. In rendez-vous networks, however, isolating the leader requires a more intricate construction, which we present in Section 4.2. Observe also that undecidable is more surprising in this latter case, since synchronization is solvable in polynomial time: in [blocking Rdv networks](#) (Theorem 2.8). Once isolated, the leader evolves through a sequence of states corresponding to the locations of the counter machine. The counters themselves are simulated by the non-leader processes: each such process represents one unit of one of the two counters. The value of a counter is then given by the number of processes in a specific state associated with that counter. Simulating an increment or decrement operation is straightforward: a rendez-vous (or a broadcast, depending on the model) between the leader and a non-leader process causes the latter to enter or leave the counter state, thereby increasing or decreasing the counter value. For a zero-test, the leader sends (or broadcasts) a message that can only be received by processes currently in the corresponding counter state. If the counter is not zero, then at least one process will receive the message and move to a *bad* state from which synchronization is no longer possible. As a result, any execution in which all processes eventually synchronize to a designated good state (e.g., the halting location ℓ_f of the counter machine) must correspond to a correct simulation, where all zero-tests were satisfied.

We recall that COVER[CM] and REACH[CM] are undecidable [Min67] (see Theorem 3.1 in Chapter 3). This holds even for a very restricted class of counter machines known as *Minsky machines* (or Minsky-CM), which are two-counter machines with no internal transitions (i.e., transitions that do not affect counters). In this chapter, we work with Minsky-CM.

Formally, given a Minsky-CM $CM = (\text{Loc}, \ell_{in}, \{x_1, x_2\}, \Delta)$, a configuration is a tuple (ℓ, v) where $\ell \in \text{Loc}$ is a location and $v \in \mathbb{N}^{\{x_1, x_2\}}$ is a valuation over the counters. Since there are only two counters, we denote a configuration (ℓ, v) by (ℓ, x_1, x_2) , where $x_1 = v(x_1)$ and $x_2 = v(x_2)$ for readability.

All definitions and proofs in this chapter are written using the *vector semantics*, rather than the multiset semantics.

4.1 BROADCAST NETWORKS

We now prove the undecidability of **SYNCHRO** over **broadcast networks**. In a broadcast network, all processes execute a **broadcast protocol** (without sending transitions). The proof relies on a reduction from the reachability problem of a Minsky-CM to **SYNCHRO**.

Given a Minsky-CM $CM = (Loc, \ell_{in}, \{x_1, x_2\}, \Delta)$ and a target location $\ell_f \in Loc$, we define the subset of test transitions as

$$\Delta_{\text{test}} = \{(\ell, \text{op}, \ell') \in \Delta \mid \text{op} \in \{x_1=0, x_2=0\}\} \subseteq \Delta.$$

We recall that in a Minsky-CM, Δ does not contain transitions of the form $(\ell, \perp, \ell')$ for any two locations $\ell, \ell' \in Loc$.

For the sake of simplicity, we assume that Δ is deterministic and complete, i.e. for all ℓ and for all $\text{op} \in \{x_+, x_-, x=0 \mid x \in \{x_1, x_2\}\}$, there exists *exactly one* $\ell' \in Loc$ such that $(\ell, \text{op}, \ell') \in \Delta$. We also assume that ℓ_f has no outgoing transition. Neither of these two restrictions affects the undecidability status of the reachability problem.

We define a protocol P depicted in Figure 4.1, where P_{CM} is defined as follows:

- $P_{CM} = (Loc \cup \{\odot\}, T_{CM})$ with
- $T_{CM} = \{(\ell, ?\text{op}, \ell') \mid (\ell, \text{op}, \ell') \in \Delta \setminus \Delta_{\text{test}}\} \cup \{(\ell, !!x=0, \ell') \mid (\ell, x=0, \ell') \in \Delta_{\text{test}}\}.$

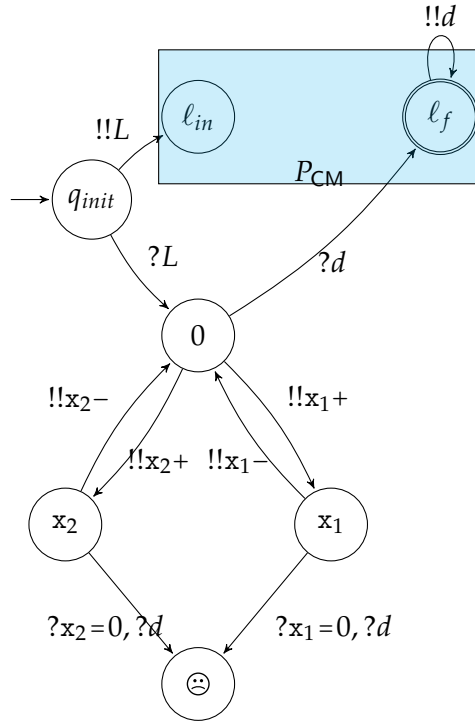


Figure 4.1: Protocol P associated to a machine $CM = (Loc, \ell_{in}, \{x_1, x_2\}, \Delta)$ and a location $\ell_f \in Loc$.

The protocol P simulates the machine as follows. A leader process starts by broadcasting L , which is received by all other processes. The values of the counters are encoded by the number of processes in states x_1 and x_2 . From ℓ_{in} , the leader simulates the machine:

- To simulate an increment of counter x , it receives a message $x+$ from a process moving from state 0 to x .
- To simulate a decrement of counter x , it receives a message $x-$ from a process moving from state x to 0.
- To simulate a zero test on counter x , the leader broadcasts a message $x=0$. If any process is in state x , it receives the message and moves to the deadlock state $\ominus$, from which it cannot reach ℓ_f .

Once the leader reaches ℓ_f , it broadcasts d . If any process is still in state x_1 or x_2 , it moves to $\ominus$ and cannot reach ℓ_f . Hence, any execution that brings all processes to ℓ_f must correctly simulate a run of the machine reaching $(\ell_f, 0, 0)$, with all zero tests being sound. We formally prove these ideas in the next two lemmas.

Lemma 4.1. If $(\ell_{in}, 0, 0) \rightsquigarrow^* (\ell_f, 0, 0)$, then there exist $C_{init} \in \mathcal{C}_{init}[P]$ and $C \in \mathcal{C}[P]$ such that $C_{init} \rightarrow^* C$ and $C(e) = \ell_f$ for all $e \in [1, |C|]$.

Proof. Let us denote $(\ell_{in}, 0, 0) \xrightarrow{\delta_1} (\ell_1, x_1^1, x_2^1) \xrightarrow{\delta_2} \dots \xrightarrow{\delta_k} (\ell_k, x_1^k, x_2^k) = (\ell_f, 0, 0)$ the execution of the counter machine. We define $\text{nbproc} = 1 + \max_{1 \leq i \leq k} (x_1^i + x_2^i)$ and let C_{init} be the initial configuration with nbproc processes.

From the initial configuration, a process (for instance process nbproc) broadcasts L and becomes *the leader*:

$$C_{init} \xrightarrow{\text{nbproc}, (q_{init}, !!L, \ell_{in})} C_0.$$

Observe that $C_0(\text{nbproc}) = \ell_{in}$ and $C_0(e) = 0$ for all $e \in [1, \text{nbproc} - 1]$.

Then, by induction on $i \in [0, k]$, we prove that $C_0 \rightarrow^* C_i$ where (i) $C_i(\text{nbproc}) = \ell_i$, and (ii) $|C_i^{-1}(x_1)| = x_1^i$, $|C_i^{-1}(x_2)| = x_2^i$, and $|C_i^{-1}(0)| = \text{nbproc} - (x_1^i + x_2^i + 1)$.

Base case: For $i = 0$, $(\ell_0, x_1^0, x_2^0) = (\ell_{in}, 0, 0)$ and C_0 satisfies (i) and (ii), hence there is nothing to do.

Induction case: Assume that $C_0 \rightarrow^* C_i$ with $C_i(\text{nbproc}) = \ell_i$, and $|C_i^{-1}(x_1)| = x_1^i$ and $|C_i^{-1}(x_2)| = x_2^i$ and $|C_i^{-1}(0)| = \text{nbproc} - (x_1^i + x_2^i + 1)$ for some $i \in [0, k - 1]$. We proceed by case induction on δ_{i+1} 's type:

- if $\delta_{i+1} = (\ell_i, x_j^+, \ell_{i+1})$ for $j \in \{1, 2\}$, then $x_1^{i+1} + x_2^{i+1} = x_1^i + x_2^i + 1 \leq \text{nbproc} - 1$, hence, by induction hypothesis $C_i^{-1}(0) \neq \emptyset$ and $C_i(\text{nbproc}) = \ell_i$. Hence, there exists e such that $C_i(e) = 0$. Let

$$C_i \xrightarrow{e, (0, !!x_j^+, x_j)} C_{i+1}$$

with $C_{i+1}(\text{nbproc}) = \ell_{i+1}$ (recall that $(\ell_i, ?x_j^+, \ell_{i+1}) \in T_{CM}$ by construction), and $|C_{i+1}^{-1}(x_j)| = |C_i^{-1}(x_j) \cup \{e\}| = x_j^i + 1 = x_j^{i+1}$, and $|C_{i+1}^{-1}(x_{3-j})| = |C_i^{-1}(x_{3-j})| = x_{3-j}^i = x_{3-j}^{i+1}$. Furthermore, $|C_{i+1}^{-1}(0)| = |C_i^{-1}(0) \setminus \{e\}| = \text{nbproc} - (x_1^i + x_2^i + 1) - 1 = \text{nbproc} - (x_1^i + x_2^i + 2) = \text{nbproc} - (x_1^{i+1} + x_2^{i+1} + 1)$. Hence C_{i+1} satisfies (i) and (ii) and $C_0 \rightarrow^* C_i \rightarrow C_{i+1}$.

- if $\delta_{i+1} = (\ell_i, \mathbf{x}_j, \ell_{i+1})$ for $j \in \{1, 2\}$, then $x_j^{i+1} = x_j^i - 1 \geq 0$, hence, by induction hypothesis $C_i^{-1}(\mathbf{x}_j) \neq \emptyset$ and $C_i(\text{nbproc}) = \ell_i$. Hence, there exists e such that $C_i(e) = \mathbf{x}_j$. Let

$$C_i \xrightarrow{e, (\mathbf{x}_j, !!\mathbf{x}_j, 0)} C_{i+1}$$

with $C_{i+1}(\text{nbproc}) = \ell_{i+1}$ (recall that $(\ell_i, ?\mathbf{x}_j, \ell_{i+1}) \in T_{\text{CM}}$ by construction), and $|C_{i+1}^{-1}(\mathbf{x}_j)| = |C_i^{-1}(\mathbf{x}_j) \setminus \{e\}| = x_j^i - 1 = x_j^{i+1}$, and $|C_{i+1}^{-1}(\mathbf{x}_{3-j})| = |C_i^{-1}(\mathbf{x}_{3-j})| = x_{3-j}^i = x_{3-j}^{i+1}$. Furthermore, $|C_{i+1}^{-1}(0)| = |C_i^{-1}(0) \cup \{e\}| = \text{nbproc} - (x_1^i + x_2^i + 1) + 1 = \text{nbproc} - (x_1^i + x_2^i - 1 + 1) = \text{nbproc} - (x_1^{i+1} + x_2^{i+1} + 1)$. Hence C_{i+1} satisfies (i) and (ii) and $C_0 \rightarrow^* C_i \rightarrow C_{i+1}$.

- if $\delta_{i+1} = (\ell_i, \mathbf{x}_j=0, \ell_{i+1})$ for $j \in \{1, 2\}$, then $x_j^{i+1} = x_j^i = 0$. By induction hypothesis, $C_i^{-1}(\mathbf{x}_j) = \emptyset$ and $C_i(\text{nbproc}) = \ell_i$. Recall that by construction, $(\ell_i, !!\mathbf{x}_j=0, \ell_{i+1}) \in T_{\text{CM}}$. Let

$$C_i \xrightarrow{\text{nbproc}, (\ell_i, !!\mathbf{x}_j=0, \ell_{i+1})} C_{i+1}$$

with $C_{i+1}(\text{nbproc}) = \ell_{i+1}$ and $C_{i+1}(e) = C_i(e)$ for all $e \in [1, \text{nbproc} - 1]$ because $\mathbf{x}_j$ is empty. Hence $|C_{i+1}^{-1}(\mathbf{x}_j)| = |C_i^{-1}(\mathbf{x}_j)| = x_j^i = x_j^{i+1} = 0$, and $|C_{i+1}^{-1}(\mathbf{x}_{3-j})| = |C_i^{-1}(\mathbf{x}_{3-j})| = x_{3-j}^i = x_{3-j}^{i+1}$. Furthermore, $|C_{i+1}^{-1}(0)| = |C_i^{-1}(0)| = \text{nbproc} - (x_1^i + x_2^i + 1) = \text{nbproc} - (x_1^{i+1} + x_2^{i+1} + 1)$. Hence C_{i+1} satisfies (i) and (ii) and $C_0 \rightarrow^* C_i \rightarrow C_{i+1}$.

We hence have an execution

$$C_{\text{init}} \rightarrow C_0 \rightarrow^* C_k \xrightarrow{\text{nbproc}, (\ell_f, !!d, \ell_f)} C_f$$

where $C_f(e) = \ell_f$ for all $e \in [1, \text{nbproc}]$, as $C_k(e) = 0$ for all $e \in [1, \text{nbproc} - 1]$. $\square$

Lemma 4.2. If there exist $C_{\text{init}} \in \mathcal{C}_{\text{init}}$ and $C \in \mathcal{C}$ such that $C_{\text{init}} \rightarrow^* C$ with $C(e) = \ell_f$ for all $e \in [1, |C|]$, then $(\ell_{\text{in}}, 0, 0) \rightsquigarrow^* (\ell_f, 0, 0)$.

Proof. We write the execution between C_{init} and C as: $C_0 \rightarrow C_1 \rightarrow \dots \rightarrow C_k$ with $C_0 = C_{\text{init}}$ and $C_k = C$. From the protocol's construction, there exists a process $\text{Leader} \in [1, |C|]$ such that

$$C_0 \xrightarrow{\text{Leader}, (q_{\text{init}}, !!L, \ell_{\text{in}})} C_1.$$

Hence, for all $e \neq \text{Leader}$, $C_1(e) = 0$. No process can then broadcast L anymore. As a consequence, Leader will necessarily broadcast d (as otherwise all other processes stay in states $0, \mathbf{x}_1, \mathbf{x}_2$ and $\ominus$) and will be the first to do it. Let i be the first index such that $C_i \xrightarrow{\text{Leader}, (\ell_f, !!d, \ell_f)} C_{i+1}$. It holds that $C_i^{-1}(\mathbf{x}_1) = C_i^{-1}(\mathbf{x}_2) = \emptyset$ as otherwise, there exists a process e such that $C_{i+1}(e) = \ominus$ (as P contains $(\mathbf{x}_1, ?d, \ominus)$ and $(\mathbf{x}_2, ?d, \ominus)$), and as $\ominus$ is deadlock state, it implies that $C_k(e) = \ominus$ which contradicts the fact that $C_k(e) = \ell_f$. Hence, the only occupied states in C_i are states 0 and ℓ_f , and so $C_{i+1}(e) = \ell_f$ for all $e \in [1, |C|]$. Hence, $C_{i+1} = C_k$.

We now focus on what happens between C_1 and C_{k-1} . For each $i \in [1, k-1]$, we associate to C_i a configuration (ℓ_i, x_1^i, x_2^i) of the machine such that:

$$\ell_i = C_i(\text{Leader}), x_1^i = |C_i^{-1}(x_1)| \text{ and } x_2^i = |C_i^{-1}(x_2)|.$$

This is well-defined as, by construction of the protocol, $C_i(\text{Leader}) \in \text{Loc}$ for all $1 \leq i \leq k-1$.

We prove by induction on $i \in [1, k-1]$ that $(\ell_{in}, 0, 0) \rightarrow^* (\ell_i, x_1^i, x_2^i)$.

Base case: ($i = 1$) We justified earlier that $\ell_1 = C_1(\text{Leader}) = \ell_{in}$ and $x_1^1 = |C_1^{-1}(x_1)| = 0$ and $x_2^1 = |C_1^{-1}(x_2)| = 0$. Hence there is nothing to do.

Induction case: Assume that $(\ell_{in}, 0, 0) \rightsquigarrow^* (\ell_i, x_1^i, x_2^i)$ for some $i \in [1, k-2]$. Let $e \in [1, |C|]$ and t_i the transition of P such that $C_i \xrightarrow{e, t_i} C_{i+1}$.

- if $t_i = (\ell, !!x_j=0, \ell')$ for $j \in \{1, 2\}$, then $e = \text{Leader}$. By construction, $(\ell, x_j=0, \ell') \in \Delta$, and as $C_i \rightarrow C_{i+1}$, we have $C_i(\text{Leader}) = \ell = \ell_i$ and $C_{i+1}(\text{Leader}) = \ell' = \ell_{i+1}$. Observe that $C_{i+1}^{-1}(x_{3-j}) = C_i^{-1}(x_{3-j})$ as message $x_j=0$ is not received from state x_{3-j} . If $|C_i^{-1}(x_j)| > 0$ then $|C_{i+1}^{-1}(\odot)| > 0$ as P contains $(x_j, ?x_j=0, \odot)$. Hence, as $\odot$ is a deadlock state, $|C_k^{-1}(\odot)| > 0$ which contradicts the definition of $C_k = C$. Thus, $|C_i^{-1}(x_j)| = |C_{i+1}^{-1}(x_j)| = 0$, and $(\ell_i, x_1^i, x_2^i) \rightsquigarrow (\ell_{i+1}, x_1^{i+1}, x_2^{i+1})$.
- if $t_i = (0, !!x_j+, x_j)$ [resp. $(x_j, !!x_j-, 0)$] for $j \in \{1, 2\}$, then as CM is complete, there exists a transition $(\ell_i, x_j+, \ell) \in \Delta$ [resp. $(\ell_i, x_j-, \ell) \in \Delta$] and so, by construction of T_{CM} , $(\ell_i, ?x_j+, \ell) \in T_{CM}$ [resp. $(\ell_i, ?x_j-, \ell) \in T_{CM}$] such that $C_{i+1}(\text{Leader}) = \ell = \ell_{i+1}$. Furthermore, $|C_{i+1}^{-1}(x_j)| = |C_i^{-1}(x_j)| + 1$ [resp. $|C_{i+1}^{-1}(x_j)| = |C_i^{-1}(x_j)| - 1 \geq 0$] and $|C_{i+1}^{-1}(x_{3-j})| = |C_i^{-1}(x_{3-j})|$, hence $(\ell_i, x_1^i, x_2^i) \rightsquigarrow (\ell_{i+1}, x_1^{i+1}, x_2^{i+1})$.

Hence, as $\ell_{k-1} = \ell_f$ (by definition of k) and $x_1^{k-1} = x_2^{k-1} = 0$ as we justified earlier, we get that $(\ell_{in}, 0, 0) \rightsquigarrow^* (\ell_f, 0, 0)$. □

The two previous Lemmas establish correctness and soundness of the reduction, leading to the following theorem.

Theorem 4.1. SYNCHRO over *broadcast networks* is undecidable.

4.2 RENDEZ-VOUS NETWORKS

We now prove that the synchronization problem is undecidable for *rendez-vous networks*. In a *rendez-vous networks*, all processes execute a *rendez-vous protocol* (without broadcast transitions) and can perform rendez-vous (one process sends a message and one process receives it) and *sending steps* (that happen when a process sends a message and no process is ready to receive the message).

As in the previous section, we proceed by simulating a Minsky-CM with respect to the reachability problem.

We fix a Minsky-CM $CM = (\text{Loc}, \ell_{in}, \{x_1, x_2\}, \Delta)$ with $\ell_f \in \text{Loc}$ as target state (in particular, Δ does not contain transitions with shape $(\ell, \perp, \ell')$ for two locations $\ell, \ell' \in \text{Loc}$). W.l.o.g.,

we assume that there is no outgoing transition from state ℓ_f in the machine. The protocol P simulating CM is described in Figures 4.2 and 4.3.

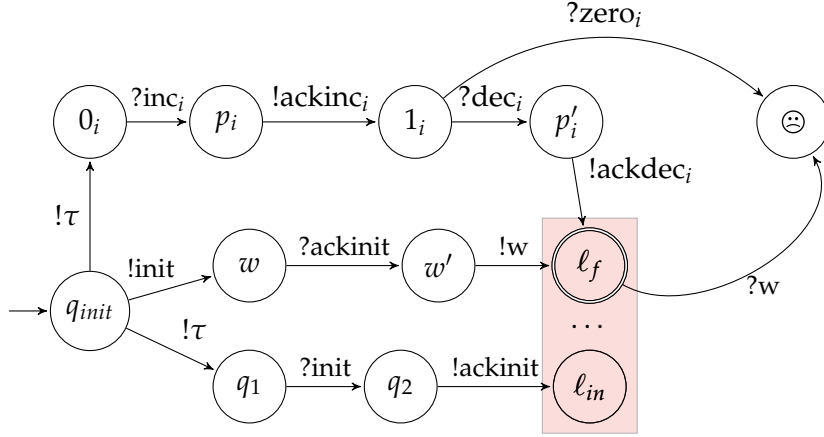


Figure 4.2: The protocol P – The coloured zone contains transitions pictured in Figure 4.3

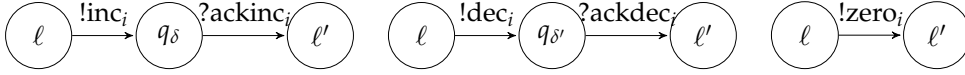


Figure 4.3: Translation of $\delta = (\ell, x_i+, \ell')$, $\delta' = (\ell, x_i-, \ell')$ and $(\ell, x_i=0, \ell')$ (respectively left figure, middle figure and right figure).

States $\{0_i, p_i, 1_i, p'_i \mid i = 1, 2\}$ will be visited by processes simulating values of counters, while the states in Loc will be visited by a process simulating the different locations in the Minsky-CM. If at the end of the computation, the counters are equal to 0, it means that each counter has been incremented and decremented the same number of times, so that all processes simulating the counters end up in the state ℓ_f . The first challenge is to appropriately check when a counter equals 0. This is achieved thanks to *sending steps*: the process sends a message $!zero_i$ to check if the counter i equals 0. If it does not, the message will be received by a process that will end up in the deadlock state $\ominus$. The second challenge is to ensure that only one process simulates the Minsky-CM in the states in Loc. This is ensured by the states $\{w, w'\}$. Each time a process arrives in the ℓ_{in} state, another must arrive in the w' state, as a witness that the simulation has begun. This witness must reach ℓ_f for the computation to be a testifier of a positive instance of SYNCHRO^{nb} , but it should be the first to do so, otherwise a process already in ℓ_f will receive the message “w” and reach the deadlock state $\ominus$. Thus, if two processes simulate the Minsky-CM, there will be two witnesses, and they won’t be able to reach ℓ_f together.

Remark 5. Observe that message τ can never be received: there is no transition labeled by $?\tau$. As a result, sending τ always lead to a sending step. We often use message τ to simulate an internal step: a process can send τ , and it will not have any impact on any other process.

We define $P = (Q, \Sigma, q_{init}, T)$ where:

- $Q = \text{Loc} \cup \{q_\delta \mid \delta \in \Delta\} \cup \{q_{init}, q_1, q_2, w, w', \ominus\} \cup \{0_i, 1_i, p_i, p'_i\}_{i=1,2}$, and
- $\Sigma = \{\tau, \text{init}, w, \text{ackinit}\} \cup \{\text{zero}_i, \text{inc}_i, \text{dec}_i, \text{ackdec}_i, \text{ackinc}_i\}_{i=1,2}$.

We define T as the union of two sets T' (described in Figure 4.2) and T^{CM} (described in Figure 4.3) is defined as follows:

$$\begin{aligned} T^{\text{CM}} = & \{(\ell, !\text{inc}_i, q_\delta), (q_\delta, ?\text{ackinc}_i, \ell') \mid (\ell, \mathbf{x}_i^+, \ell') \in \Delta\} \\ & \cup \{(\ell, !\text{dec}_i, q_\delta), (q_\delta, ?\text{ackdec}_i, \ell') \mid (\ell, \mathbf{x}_i^-, \ell') \in \Delta\} \\ & \cup \{(\ell, !\text{zero}_i, \ell') \mid (\ell, \mathbf{x}_i = 0, \ell') \in \Delta\}. \end{aligned}$$

We now proceed to prove completeness and soundness of the reduction. Completeness is the easy part as from an execution of the CM, one can choose an execution in the protocol.

Lemma 4.3. If there exists an execution of CM, $(\ell_{in}, 0, 0) \rightsquigarrow^* (\ell_f, 0, 0)$, then there exists an execution of P , $C_{init} \rightarrow^* C_f$ with $C_{init} \in \mathcal{C}_{init}$ and for all $e \in [1, |C_f|]$, $C_f(e) = \ell_f$.

Proof. Denote $\rho = (\ell_0, x_1^0, x_2^0) \rightsquigarrow (\ell_1, x_1^1, x_2^1) \rightsquigarrow \dots \rightsquigarrow (\ell_k, x_1^k, x_2^k)$ the execution of the CM such that $\ell_0 = \ell_{in}$, $\ell_k = \ell_f$ and $x_1^0 = x_2^0 = x_1^k = x_2^k = 0$. For all $1 \leq j \leq k$ and $i \in \{1, 2\}$, we denote NB-INC(j) [resp. NB-INC _{i} (j)] as the number of increment transitions triggered between (ℓ_0, x_1^0, x_2^0) and (ℓ_j, x_1^j, x_2^j) in ρ [resp. the number of counter x_i increment transitions between (ℓ_0, x_1^0, x_2^0) and (ℓ_j, x_1^j, x_2^j) in ρ]. Consider now C_{init} the initial configuration with NB-INC(k) + 2 processes. We denote $N = \text{NB-INC}(k) + 2$, and we consider the following execution:

$$C_{init} \xrightarrow{N, (q_{init}, !\tau, q_1)} C_1 \xrightarrow{N-1, (q_{init}, !\text{init}, w)} C_2 \xrightarrow{N, (q_2, !\text{ackinit}, \ell_{in})} C_3 \xrightarrow{N-1, (w', !w, \ell_f)} C_4$$

The first event is a non-blocking sending as there is no reception transition for message τ in the protocol. Second message is received by process N with transition $(q_1, ?\text{init}, q_2)$, which then can send message “ackinit” received itself by process $N - 1$ with transition $(w, ?\text{ackinit}, w')$, and finally, process $N - 1$ sends message “w” which is not received by anyone as $|C^{-1}(\ell_f)| = 0$. Hence, C_4 is such that:

$$\begin{aligned} C_4(N) &= \ell_{in} \text{ and } C_4(N - 1) = \ell_f \text{ and} \\ C_4(e) &= q_{init} \text{ for all } e \in [1, N - 2]. \end{aligned}$$

Hence, we can continue the execution this way:

$$\begin{aligned} C_4 &\xrightarrow{1, (q_{init}, !\tau, 0_1)} \xrightarrow{2, (q_{init}, !\tau, 0_1)} \dots \xrightarrow{\text{NB-INC}_1(k), (q_{init}, !\tau, 0_1)} \\ &\xrightarrow{\text{NB-INC}_1(k) + 1, (q_{init}, !\tau, 0_2)} \xrightarrow{\text{NB-INC}_1(k) + 2, (q_{init}, !\tau, 0_2)} \dots \xrightarrow{\text{NB-INC}(k), (q_{init}, !\tau, 0_2)} C'_0 \end{aligned}$$

Observe that:

$$\begin{aligned} C'_0(N) &= \ell_{in}, C'_0(N - 1) = \ell_f, \\ C'_0(e) &= 0_1 \text{ for all } e \in [1, \text{NB-INC}_1(k)] \\ C'_0(e) &= 0_2 \text{ for all } e \in [\text{NB-INC}_1(k) + 1, \text{NB-INC}(k)] \end{aligned}$$

From C'_0 , we prove by induction that for all $j \in [0, k]$, one can build a run $C'_0 \rightarrow^* C'_j$ such that:

$$C'_j(N) = \ell_j \text{ (process } N \text{ simulates the machine)}$$

$$C'_j(N-1) = \ell_f \text{ (process } N-1 \text{ stays in } \ell_f)$$

$$|C'^{-1}_j(1_i)| = x^j_i \text{ for all } i \in \{1, 2\} \text{ (coherent counters' values simulated)}$$

$$|C'^{-1}_j(0_i)| = \text{NB-INC}_i(k) - \text{NB-INC}_i(j) \text{ for all } i \in \{1, 2\} \text{ (unused processes are in } 0_i)$$

$$|C'^{-1}_j(\ell_f) \setminus \{N-1, N\}| = \text{NB-INC}(j) - (x^j_1 + x^j_2) \text{ (used processes are in } \ell_f)$$

For $j = 0$, it trivially holds: C'_0 satisfies the requirements, as $(\ell_0, x^0_1, x^0_2) = (\ell_{in}, 0, 0)$ and $\text{NB-INC}_i(0) = 0$ for $i \in \{1, 2\}$, and $C'_0 \rightarrow^* C'_0$.

Let $j \in [0, k-1]$ and assume we have built a run $C'_0 \rightarrow^* C'_j$ where C'_j satisfies the above requirements. Consider $(\ell_j, x^j_1, x^j_2) \xrightarrow{\delta} (\ell_{j+1}, x^{j+1}_1, x^{j+1}_2)$ with $\delta = (\ell_j, \text{op}, \ell_{j+1}) \in \Delta$. By hypothesis, $C'_j(N) = \ell_j$. We proceed by distinguishing cases:

- if $\text{op} = x_i+$ then there exists a transition $(\ell_j, !\text{inc}_i, q_\delta)$. Furthermore, $\text{NB-INC}_i(j+1) = \text{NB-INC}_i(j) + 1 \leq \text{NB-INC}_i(k)$. Hence, by hypothesis, $|C'^{-1}_{j-1}(0_i)| > 0$ and so there exists e such that $C'_j(e) = 0_i$. Hence, consider C such that $C(e) = p_i$,

$C(N) = q_\delta$ and $C(e') = C'_j(e')$ for all other e' . It holds that $C'_j \xrightarrow{N, (\ell_j, !\text{inc}_i, q_\delta)} C$. Define C'_{j+1} such that $C'_{j+1}(N) = \ell_{j+1}$, $C'_{j+1}(e) = 1_i$ and $C'_{j+1}(e') = C(e')$ for all $e' \notin \{N, e\}$. Observe that $C \xrightarrow{e, (p_i, !\text{ackinc}_i, 1_i)} C'_{j+1}$, and:

$$- C'_{j+1}(N) = \ell_{j+1} \text{ and } C'_{j+1}(N-1) = \ell_f,$$

$$- |C'^{-1}_{j+1}(1_i)| = 1 + x^j_i = x^{j+1}_i \text{ and } |C'^{-1}_{j+1}(1_{3-i})| = x^j_{3-i} = x^{j+1}_{3-i}$$

-

$$\begin{aligned} |C'^{-1}_{j+1}(0_i)| &= |C'^{-1}_j(0_i)| - 1 = \text{NB-INC}_i(k) - (\text{NB-INC}_i(j) + 1) \\ &= \text{NB-INC}_i(k) - \text{NB-INC}_i(j+1) \end{aligned}$$

-

$$\begin{aligned} |C'^{-1}_{j+1}(0_{3-i})| &= |C'^{-1}_j(0_{3-i})| = \text{NB-INC}_i(k) - \text{NB-INC}_i(j) \\ &= \text{NB-INC}_i(k) - \text{NB-INC}_i(j+1) \end{aligned}$$

-

$$\begin{aligned} |C'^{-1}_{j+1}(\ell_f) \setminus \{N-1, N\}| &= |C'^{-1}_j(\ell_f) \setminus \{N-1, N\}| \\ &= (\text{NB-INC}(j) + 1) - (x^j_1 + x^j_2 + 1) \\ &= \text{NB-INC}(j+1) - (x^{j+1}_1 + x^{j+1}_2). \end{aligned}$$

- if $\text{op} = x_i-$ then there exists a transition $(\ell_j, !\text{dec}_i, q_\delta)$. Furthermore, $x^j_i > 0$. Hence, by hypothesis, $|C'^{-1}_j(1_i)| > 0$ and so there exists e such that $C'_j(e) = 1_i$. Hence,

consider C such that $C(e) = p'_i$, $C(N) = q_\delta$ and $C(e') = C'_j(e')$ for all other e' . It holds that $C'_j \xrightarrow{N, (\ell_j, !\text{dec}_i, q_\delta)} C$. Define C'_{j+1} such that $C'_{j+1}(N) = \ell_{j+1}$, $C'_{j+1}(e) = \ell_f$ and $C'_{j+1}(e') = C(e')$ for all $e' \notin \{N, e\}$. Observe that $C \xrightarrow{e, (p'_i, !\text{lackdec}_i, 0_i)} C'_{j+1}$, and:

- $C'_{j+1}(N) = \ell_{j+1}$ and $C'_{j+1}(N-1) = \ell_f$,
- $|C'_{j+1}{}^{-1}(1_i)| = x_i^j - 1 = x_i^{j+1}$ and $|C'_{j+1}{}^{-1}(1_{3-i})| = x_{3-i}^j = x_{3-i}^{j+1}$,
- $|C'_{j+1}{}^{-1}(0_i)| = |C_j{}^{-1}(0_i)| = \text{NB-INC}_i(k) - \text{NB-INC}_i(j+1)$,
- $|C'_{j+1}{}^{-1}(0_{3-i})| = |C_j{}^{-1}(0_{3-i})| = \text{NB-INC}_i(k) - \text{NB-INC}_i(j+1)$,
-

$$\begin{aligned} |C'_{j+1}{}^{-1}(\ell_f) \setminus \{N-1, N\}| &= |C_j{}^{-1}(\ell_f) \setminus \{N-1, N\}| + 1 \\ &= \text{NB-INC}(j) - (x_1^j + x_2^j - 1) \\ &= \text{NB-INC}(j+1) - (x_1^{j+1} + x_2^{j+1}). \end{aligned}$$

- if $\text{op} = x_i = 0$ then there exists a transition $(\ell_j, !\text{zero}_i, \ell_{j+1})$. Furthermore, $x_i^j = 0$. Hence, by hypothesis, $C'_{j-1}{}^{-1}(1_i) = \emptyset$. Consider C'_{j+1} such that $C'_{j+1}(N) = \ell_{j+1}$ and $C(e') = C'_j(e')$ for all other e' . It holds that $C'_j \xrightarrow{N, (\ell_j, !\text{zero}_i, \ell_{j+1})} C'_{j+1}$ with a **sending step** as $C'_{j-1}{}^{-1}(1_i) = \emptyset$. Observe that C'_j is such that:

- $C'_{j+1}(N) = \ell_{j+1}$ and $C'_{j+1}(N-1) = \ell_f$,
- $|C'_{j+1}{}^{-1}(1_i)| = x_i^j = x_i^{j+1} = 0$ and $|C'_{j+1}{}^{-1}(1_{3-i})| = x_{3-i}^j = x_{3-i}^{j+1}$,
- $|C'_{j+1}{}^{-1}(0_i)| = |C_j{}^{-1}(0_i)| = \text{NB-INC}_i(k) - \text{NB-INC}_i(j+1)$,
- $|C'_{j+1}{}^{-1}(0_{3-i})| = |C_j{}^{-1}(0_{3-i})| = \text{NB-INC}_i(k) - \text{NB-INC}_i(j+1)$,
- $|C'_{j+1}{}^{-1}(\ell_f) \setminus \{N-1, N\}| = |C_j{}^{-1}(\ell_f) \setminus \{N-1, N\}| = \text{NB-INC}(j) - (x_1^j + x_2^j) = \text{NB-INC}(j+1) - (x_1^{j+1} + x_2^{j+1})$.

Hence, we proved the existence of the run $C'_0 \rightarrow^* C'_j$ for all $j \in [1, k]$ which concludes the proof as $C'_k(N) = \ell_f$, $C'_k(N-1) = \ell_f$, $|C'_k{}^{-1}(\ell_f) \setminus \{N, N-1\}| = \text{NB-INC}(k)$ (because $x_1^k = x_2^k = 0$). Hence for all $e \in [1, N]$, $C'_k(e) = \ell_f$. □

We now proceed to prove the soundness of the reduction: from an execution putting all processes in q_f , we extract an execution of the counter machine. To do so, we start by proving that there must exist a unique process which evolves in Loc. We prove that it rightfully simulates the counter machine: counters' values are then simulated by the number of processes on states 1_1 and 1_2 . When it simulates a test to 0 of counter x_i , it must be that the corresponding state (1_i) is empty as otherwise, a process receives zero_i and goes to state $\odot$. Any execution putting a process in state $\odot$ can not put all processes in state ℓ_f as $\odot$ is a deadlock state from which it is impossible to reach ℓ_f . This allows us to build a counter machine's run.

Lemma 4.4. If there exists an execution of P , $C_{init} \rightarrow^* C_f$ with $C_{init} \in \mathcal{C}_{init}$ and for all $e \in [1, |C_f|]$, $C_f(e) = \ell_f$, then there exists an execution of CM: $(\ell_{in}, 0, 0) \rightsquigarrow^* (\ell_f, 0, 0)$.

Proof. Denote $\rho = C_0 \rightarrow C_1 \rightarrow \dots \rightarrow C_k$ with $C_0 = C_{init}$ and $C_k = C_f$. We first start to prove a few facts about ρ .

Fact 1 There exists a unique $J_0 \in [0, k-1]$ such that $C_{J_0} \xrightarrow{e_0, (w', !w, \ell_f)} C_{J_0+1}$ for some process $e_0 \in [1, |C_f|]$.

Fact 2 There exists a unique $J_1 \in [J_0 + 1, k-1]$ such that $C_{J_1} \xrightarrow{e_1, (q_2, !ackinit, \ell_{in})} C_{J_1+1}$ for some process $e_1 \in [1, |C_f|]$ and $e_0 \neq e_1$.

We start by proving Fact 1. First observe that there must be an index J_0 and a process e_0 such that $C_{J_0} \xrightarrow{e_0, (w', !w, \ell_f)} C_{J_0+1}$: assume there is not, then no process takes transition $(q_{init}, !init, w')$ (as otherwise it will be blocked in states w or w' and can never reach ℓ_f without taking $(w', !w, \ell_f)$). If 'init' is never sent, then no process can ever reach a state from which inc_1 or inc_2 can be sent. As a result, the only reachable states are $q_{init}, q_1, 0_1$ and 0_2 . This contradicts the fact $C_f(e) = \ell_f$ for all processes $e \in [1, |C_f|]$. We now prove that J_0 is the unique index from which a process takes transition $(w', !w, \ell_f)$. First observe that, from the protocol's construction, $C_j(e_0) \in \{\ell_f, \ominus\}$ for all $j > J_0$. As state $\ominus$ is a deadlock state and, by hypothesis, $C_f(e_0) = \ell_f$, we deduce that $C_j(e_0) = \ell_f$ for all $j > J_0$. Assume there exists another index J'_0 and a process e'_0 such that $C_{J'_0} \xrightarrow{e'_0, (w', !w, \ell_f)} C_{J'_0+1}$. Then, as we justified earlier, $C_{J'_0}(e_0) = \ell_f$, in particular, $|C'^{-1}(\ell_f)| > 0$. Hence, the sending of 'w' by e'_0 must be received, and lead to a rendez-vous with a process on ℓ_f (ℓ_f is the only state from which 'w' can be received). Hence, there exists a process e such that $C_{J'_0+1}(e) = \ominus$ and so $C_f(e) = \ominus$ which is absurd, proving Fact 1.

We now proceed to prove Fact 2. Let J_0, e_0 be the unique index and process from Fact 1. From the construction of the protocol and Fact 1, we deduce that there exists a unique index j such that $C_j \xrightarrow{e_0, (q_{init}, !init, w)} C_{j+1}$. Note that, for e_0 to reach w' , there must exist J_1 such that $C_{J_1} \xrightarrow{e_1, (q_2, !ackinit, \ell_{in})} C_{J_1+1}$ for a process $e_1 \neq e_0$. From uniqueness of j , there is at most one process taking the transition $(q_1, ?init, q_2)$ and hence able to send 'ackinit', which concludes proving Fact 2.

We now denote $C_{j_1}, \dots, C_{j_k}$ the sequence of configurations with $j_1 < \dots < j_n \leq k$, and $j_1 = J_1 + 1$ such that for all $r \in [1, n]$; $C_{j_r}(e_1) \in \text{Loc}$. We prove by induction that

- (i) $|C_{j_r}^{-1}(p_i)| = |C_{j_r}^{-1}(p'_i)| = 0$ for all $i \in \{1, 2\}$ and $r \in [1, n]$ (no process is in a state in between a zero state and a unit state) and
- (ii) $(\ell_{in}, 0, 0) \rightsquigarrow^\epsilon (\ell_1, x_1^1, x_2^1) \rightsquigarrow^\epsilon (\ell_2, x_1^2, x_2^2) \rightsquigarrow^\epsilon \dots \rightsquigarrow^\epsilon (\ell_n, x_1^n, x_2^n)$ where $\rightsquigarrow^\epsilon$ is the reflexive closure of $\rightsquigarrow$, and:

$$\begin{aligned} \ell_r &= C_{j_r}(e_1) \text{ and} \\ x_i^r &= |C_{j_r}^{-1}(1_i)| \text{ for } i \in \{1, 2\} \end{aligned}$$

for all $r \in [1, n]$.

Base case ($r = 1$). Observe that by construction of J_1 , $C_{j_1}(e_1) = \ell_{in}$. From *Fact 2*, e_1 is the only process visiting a state $\ell \in \text{Loc} \setminus \{\ell_f\}$. We supposed wlog that ℓ_f is a deadlock location, hence e_1 is the only process able to send messages inc_1 and inc_2 . By construction of the protocol and J_1 , for all $j < j_1$ $C_j(e_1) \in \{q_{init}, q_1, q_2\}$. Hence, inc_1 and inc_2 have not been sent before j_1 . As a consequence, $|C_{j_1}^{-1}(p_i)| = |C_{j_1}^{-1}(p'_i)| = |C_{j_1}^{-1}(1_i)| = 0$ for $i \in \{1, 2\}$ hence (i) holds and (ii) as well as $(\ell_{in}, 0, 0) \rightsquigarrow^\epsilon (\ell_1, x_1^1, x_2^1)$ because $(\ell_1, x_1^1, x_2^1) = (\ell_{in}, 0, 0)$.

Induction case. Let $r \in [1, n - 1]$ and suppose we have proved that (i) $|C_{i_r}^{-1}(p_i)| = |C_{i_r}^{-1}(p'_i)| = 0$ for all $i \in \{1, 2\}$ and (ii) $(\ell_{in}, 0, 0) \rightsquigarrow^\epsilon \dots \rightsquigarrow^\epsilon (\ell_r, x_1^r, x_2^r)$ where $\rightsquigarrow^\epsilon$ is the reflexive closure of $\rightsquigarrow$, and:

$$\begin{aligned} \ell_r &= C_{j_r}(e_1) \text{ and} \\ x_i^r &= |C_{j_r}^{-1}(1_i)| \text{ for } i \in \{1, 2\} \end{aligned}$$

From the protocol's construction, *Facts 1-2* and the induction hypothesis, we know that we fall into one of the below mentioned scenarios:

- $C_{j_r} \xrightarrow{e, (q_{init}, !\tau, 0_i)} C$ for some $i \in \{1, 2\}$ and with $e \neq e_1$ and $C_{j_r+1} = C$. In that case, observe that $C = C_{j_r+1}$ as $C(e_1) = C_{j_r+1}(e_1) \in \text{Loc}$. Furthermore, $x_i^{r+1} = x_i^r$ for all $i \in \{1, 2\}$. Hence, (i) holds as $|C_{j_r+1}^{-1}(p_i)| = |C_{j_r}^{-1}(p_i)| = 0$ by induction hypothesis (and the same for p'_i) for $i \in \{1, 2\}$; and (ii) as well as $(\ell_r, x_1^r, x_2^r) = (\ell_{r+1}, x_1^{r+1}, x_2^{r+1})$.
- $C_{j_r} \xrightarrow{e_1, (\ell_r, !\text{inc}_i, q_\delta)} C$ for some $\delta = (\ell_r, x_i^r, \ell)$ and $C_{j_r+1} = C$. We start by justifying that the message is received by a process e_2 which moves from 0_i to p_i . Suppose the message is not received, i. e., $C_{j_r}^{-1}(0_i) = \emptyset$. Then, from the induction hypothesis, $C^{-1}(p_i) = \emptyset$ and $C(e_1) = q_\delta$. From *Fact 2*, e_1 is the only process able to send a message inc_i . Hence, from C , no message inc_i can be sent before e_1 receives a message ackinc_i . However, as $C^{-1}(p_i) = \emptyset$, for a message ackinc_i to be sent, a message inc_i needs to be sent first. Hence, we reached a deadlock and e_1 will never move from state q_δ and $C_f(e_1) = q_\delta$, which is absurd.

Hence, there exists e_2 such that $(C_{j_r}(e_2), ?\text{inc}_i, C(e_2)) \in T$ and, from the protocol's description, we get that $C(e_2) = p_i$. Using property (i) on C_{i_r} we get that $C^{-1}(p_i) = \{e_2\}$.

As in the previous case of this proof, it might be that before e_1 reaches a state in Loc again, some processes take transition $(q_{init}, !\tau, 0_1)$ or $(q_{init}, !\tau, 0_2)$. We consider $j'_r \geq j_r + 1$ as the first index such that $C_{j'_r} \xrightarrow{e, t} C_{j'_r+1}$ for some process e and $t \notin \{(q_{init}, !\tau, 0_i) \mid i = 1, 2\}$. Observe that $C_{j'_r}$ is such that:

- $C_{j'_r}(e_1) = C(e_1) = q_\delta$;
- $C_{j'_r}^{-1}(p_i) = C^{-1}(p_i) = \{e_2\}$;
- $C_{j'_r}^{-1}(1_b) = C^{-1}(1_b)$ for all $b = 1, 2$;
- $C_{j'_r}^{-1}(p'_b) = C^{-1}(p'_b) = \emptyset$ for all $b = 1, 2$;
- $C_{j'_r}^{-1}(p_{3-i}) = C^{-1}(p_{3-i}) = \emptyset$.

From *Fact 1-2*, we have that $t \notin \{(q_{init}, !init, w), (w', !w, \ell_f), (q_{init}, !\tau, q_1), (q_2, !ackinit, \ell_{in})\}$. Hence, we conclude that $t = (p_i, !ackinc_i, 1_i)$ and $e = e_2$. As a result,

- $C_{j'_r+1}(e_1) = \ell$;
- $C_{j'_r+1}^{-1}(p_i) = \emptyset$;
- $C_{j'_r+1}^{-1}(1_i) = C_{j_r}^{-1}(1_i) \cup \{e_2\}$;
- $C_{j'_r+1}^{-1}(1_{3-i}) = C_{j_r}^{-1}(1_{3-i})$;
- $C_{j'_r+1}^{-1}(p_{3-i}) = C_{j_r}^{-1}(p_{3-i}) = \emptyset$;
- $C_{j'_r+1}^{-1}(p'_b) = C^{-1}(p'_b) = \emptyset$ for all $b = 1, 2$;

Hence, $j_{r+1} = j'_r + 1$. We proved (i). Furthermore, we conclude that $x_i^{r+1} = |C_{j_r}^{-1}(1_i)| + 1 = x_i^r + 1$, and $x_{3-i}^{r+1} = x_{3-i}^r$; and $\ell_{r+1} = \ell$. As $(\ell_r, x_1^r, x_2^r) \rightsquigarrow (\ell_{r+1}, x_1^{r+1}, x_2^{r+1})$, it concludes this induction case.

- $C_{j_r} \xrightarrow{e_1, (\ell_r, !dec_i, q_\delta)} C$ for some $\delta = (\ell_r, x_i^r, \ell)$ and $C_{j_{r+1}} = C$. We start by justifying that the message is received by a process e_2 which moves from 1_i to p'_i . Suppose the message is not received, i. e., $C_{j_r}^{-1}(1_i) = \emptyset$. Then, from the induction hypothesis, $C^{-1}(p'_i) = \emptyset$ and $C(e_1) = q_\delta$. From *Fact 2*, e_1 is the only process able to send a message dec_i . Hence, from C , no message dec_i can be sent before e_1 receives a message $ackdec_i$. However, as $C^{-1}(p'_i) = \emptyset$, for a message $ackdec_i$ to be sent, a message dec_i needs to be sent first. Hence, we reached a deadlock and e_1 will never move from state q_δ and $C_f(e_1) = q_\delta$, which is absurd.

Hence, there exists e_2 such that $(C_{j_r}(e_2), ?dec_i, C(e_2)) \in T$ and, from the protocol's description, we get that $C(e_2) = p'_i$. Using property (i) on C_{j_r} we get that $C^{-1}(p'_i) = \{e_2\}$.

As in the previous case of this proof, it might be that before e_1 reaches a state in Loc again, some processes take transition $(q_{init}, !\tau, 0_1)$ or $(q_{init}, !\tau, 0_2)$. We consider $j'_r \geq j_r + 1$ as the first index such that $C_{j'_r} \xrightarrow{e, t} C_{j'_r+1}$ for some process e and $t \notin \{(q_{init}, !\tau, 0_i) \mid i = 1, 2\}$. Observe that $C_{j'_r}$ is such that:

- $C_{j'_r}(e_1) = C(e_1) = q_\delta$;
- $C_{j'_r}^{-1}(p'_i) = C^{-1}(p'_i) = \{e_2\}$;
- $C_{j'_r}^{-1}(1_b) = C^{-1}(1_b)$ for all $b = 1, 2$;
- $C_{j'_r}^{-1}(p_b) = C^{-1}(p_b) = \emptyset$ for all $b = 1, 2$;
- $C_{j'_r}^{-1}(p'_{3-i}) = C^{-1}(p'_{3-i}) = \emptyset$.

From *Fact 1-2*, we have that $t \notin \{(q_{init}, !init, w), (w', !w, \ell_f), (q_{init}, !\tau, q_1), (q_2, !ackinit, \ell_{in})\}$. Hence, we conclude that $t = (p_i, !ackdec_i, 1_i)$ and $e = e_2$. As a result,

- $C_{j'_r+1}(e_1) = \ell$;
- $C_{j'_r+1}^{-1}(p'_i) = \emptyset$;
- $C_{j'_r+1}^{-1}(1_i) = C_{j_r}^{-1}(1_i) \setminus \{e_2\}$;

- $C_{j_r+1}^{-1}(1_{3-i}) = C_{j_r}^{-1}(1_{3-i});$
- $C_{j_r+1}^{-1}(p'_{3-i}) = C_{j_r}^{-1}(p'_{3-i}) = \emptyset;$
- $C_{j_r+1}^{-1}(p_b) = C^{-1}(p_b) = \emptyset$ for all $b = 1, 2;$

Hence, $j_{r+1} = j'_r + 1$. We proved (i). Furthermore, we conclude that $x_i^{r+1} = |C_{j_r}^{-1}(1_i)| - 1 = x_i^r - 1$ and $x_i^r \geq 1$, and $x_{3-i}^{r+1} = x_{3-i}^r$; and $\ell_{r+1} = \ell$. As $(\ell_r, x_1^r, x_2^r) \rightsquigarrow (\ell_{r+1}, x_1^{r+1}, x_2^{r+1})$, it concludes this induction case.

- $C_{j_r} \xrightarrow{e_1, (\ell_r, \text{zero}_i, \ell)} C$ for some $\delta = (\ell_r, x_i^r, \ell)$, and $C_{j_r+1} = C$. Observe that $C_{j_r+1} = C$ as $C(e_1) \in \text{Loc}$. It is only left to justify that $x_i^r = x_i^{r+1} = 0$, i. e., $C_{j_r}^{-1}(1_i) = \emptyset$. Suppose there exists e such that $C_{j_r}(e) = 1_i$. Then, there exists a process e' who receives the message zero_i . Observe that $Q_?(\text{zero}_i) = \{1_i\}^1$; hence $C_{j_r}(e') = 1_i$ and $C(e') = \ominus$. As a consequence $C_f(e') = \ominus$, which is absurd. Hence, $C_{j_r}^{-1}(1_i) = \emptyset$; and $x_i^r = x_i^{r+1} = 0$. As $x_{3-i}^r = x_{3-i}^{r+1}$; hence $(\ell_r, x_1^r, x_2^r) \rightsquigarrow (\ell, x_1^{r+1}, x_2^{r+1})$.

Hence, we showed that $(\ell_{in}, 0, 0) \rightsquigarrow^* (\ell_n, x_1^n, x_2^n)$. Observe that $C_f(e_1) = \ell_f \in \text{Loc}$, hence, $j_n = k$ and $(\ell_n, x_1^n, x_2^n) = (\ell_f, 0, 0)$ which concludes the proof. $\square$

We proved that the presented reduction is sound and complete. Together with Theorem 3.1 the reduction leads to the following theorem.

Theorem 4.2. *SYNCHRO^{nb} over rendez-vous protocols is undecidable.*

4.3 CONCLUSIONS

In this chapter, we established that the synchronization problem is highly undecidable: both in broadcast and rendez-vous networks. In each case, requiring all processes to reach the final state prevents any cheating during the simulation of the counter machine.

In contrast, we will see that coverability problems (where some additional processes may reach other states) restore decidability. In broadcast networks, we already saw that coverability is decidable but Ackermann-complete (see Theorem 2.3). In the next chapter, we investigate coverability problems in rendez-vous networks.

¹ Recall that $Q_?(a) = \{q \in Q \mid \exists q' \in Q, (q, ?a, q') \in T\}$.

COVERABILITY IN RENDEZ-VOUS NETWORKS

5.1 INTRODUCTION

In this chapter, we study the coverability problems for parameterized networks of processes communicating through rendez-vous. In this setting, all processes execute the same protocol. A communication step can either be a *rendez-vous*, in which a message is sent and received by exactly one other process, or a *sending step*, where the message is sent but not received because no process is in a state where it can receive the message. Recall that when only rendez-vous steps are allowed, we refer to *blocking rendez-vous*.

While coverability for rendez-vous networks with blocking semantics is known to be solvable in polynomial time, (see their seminal paper [GS92]), the situation changes drastically in the presence of sending steps. We show in this chapter that coverability problems becomes ExpSpace -complete. This increase in complexity is due to the additional behaviors allowed by sending steps: for instance, it becomes possible to enforce that a given state contains *at most one* process, something that cannot be achieved with blocking rendez-vous alone.

Non-blocking rendez-vous were discussed to model Java multithread programming in [DRVB02]. In that work, and later in [RB03], the authors extend Petri nets to support non-blocking communication. They show that coverability remains decidable for this extended model, using techniques from Well-Structured Transition Systems [Abd+96; Abd+00; FS01]. However no bound on the complexity can be extracted from their work. More generally, Petri nets do not always yield efficient algorithms for parameterized systems: although blocking rendez-vous networks can be encoded in Petri nets, their coverability problem is solvable in polynomial time [GS92], whereas coverability in Petri nets is ExpSpace -complete [Rac78; Lip76].

In this chapter, we prove that coverability problems for non-blocking rendez-vous networks are ExpSpace -complete. This result complements known undecidability results in the same setting: repeated coverability was shown undecidable in [EK03], and we have proved synchronization to be undecidable in Chapter 4.

Our proof proceeds via an intermediate model: *non-blocking counter machines with restore transitions* (NB-R-CM), defined in chapter 3. These are non-blocking counter machines (NB-CM), where counters may be incremented, decremented, or non-blockingly decremented, and each location has a transition returning to the initial location. We show in section 5.2 that coverability for NB-R-CM (and NB-CM as well) is ExpSpace -complete. Although interesting on its own, this result serves mainly as a key step toward our main theorem. In section 5.3, we present a motivating example of a rendez-vous network. We then prove ExpSpace -completeness of coverability problems for rendez-vous networks in section 5.4, by reduction from NB-R-CM.

ON THE STRUCTURE OF OUR PROOFS. The upper and lower bounds for NB-R-CM are inspired by classical arguments for coverability in Vector Addition Systems with States (VASS) [Lip76; Rac78]. While the entire proof could have been developed directly in the rendez-vous setting, the detour through NB-R-CM allows us to distinguish between two sources of complexity: the *non-blocking behavior* (non-blocking decrements in NB-R-CM, sending steps in rendez-vous networks) and the *distributed nature* of parameterized networks. Handling the non-blocking aspect separately makes the argument clearer. It is worth noting that despite sharing the same coverability complexity, NB-CM and VASS are not intertranslatable: the non-blocking semantics breaks the assumptions needed for an encoding. This motivates the use of NB-R-CM, which can be simulated by rendez-vous protocols, provided they include restore transitions. Indeed, the reduction from NB-R-CM to RDV-protocol is detailed in section 5.4, and relies on this restore mechanism: while RDV-protocol can simulate NB-R-CM, they cannot simulate general NB-CM.

5.2 COVERABILITY IN NON-BLOCKING TEST-FREE RESET COUNTER MACHINES

5.2.1 EXPSPACE-membership

We start by proving EXPSPACE-membership of the coverability problem for **Non-Blocking test-free counter machines** (shortly **NB-CM**). As any **NB-R-CM** (an NB-CM in which each location has a transition with no effects on the counters going back to ℓ_{in}) is an NB-CM, this also establishes the upper bound of the complexity of coverability problem for NB-R-CM.

In [Rac78], Rackoff proves that the coverability problem for **Vector Addition Systems** is in EXPSPACE. This upper bound naturally extends to **Vector Addition Systems with States** as any **VASS** can be encoded into a **VAS** (see for example [HP79] for a transformation using only 3 additional counters and an overall size polynomial in the size of the VASS). For this upper bound proof (EXPSPACE membership), we will show that Rackoff's proof also works with a notion of non-blocking decrement. Hence, we prove that coverability is in EXPSPACE on **Non-Blocking VAS** (shortly **NB-VAS**) and then prove that this implies that coverability on NB-CM is also in EXPSPACE.

5.2.1.1 Non-blocking VAS

As a reminder: a **Non-blocking Vector Addition System (NB-VAS)** of dimension $d \in \mathbb{N}$ is a tuple (Δ, v_{init}) such that $\Delta \subseteq \mathbb{Z}^d \times \mathbb{N}^d$ and $v_{init} \in \mathbb{N}^d$. The blocking part δ_b of the transition has a strict semantics: to be taken, it needs to be applied to a vector large enough so no value goes below 0. The non-blocking part δ_{nb} can be taken even if it decreases some component below 0: the corresponding component will simply be set to 0. Formally, for two vectors $v, v' \in \mathbb{N}^d$, and a transition $\delta = (\delta_b, \delta_{nb}) \in \Delta$, we write $v \xrightarrow{\delta} v'$ if there exists $v'' \in \mathbb{N}^d$ such that $v'' = v + \delta_b$ and, for all $i \in [1, d]$, $v'(i) = \max(0, v''(i) - \delta_{nb}(i))$. We recall that COVER[NB-VAS] asks given an NB-VAS (Δ, v_{init}) and a vector v if its the case that $v_{init} \rightsquigarrow^* v$.

In [Rac78], Rackoff proves that the coverability problem for **VAS** is in EXPSPACE by showing that if coverability holds then there exists an execution of double-exponential length covering the given vector. We show that a similar argument holds on **NB-VAS**.

Fix an **NB-VAS** (Δ, v_{init}) of dimension $d \in \mathbb{N}$, we will extend the semantics of **NB-VAS** to a slightly *relaxed* semantics: let $v, v' \in \mathbb{N}^d$ and $\delta = (\delta_b, \delta_{nb}) \in T$, we will write $v \xrightarrow{\delta} v'$ when $v'(j) = \max(0, (v + \delta_b - \delta_{nb})(j))$ for all $1 \leq j \leq d$. We use $\rightarrow$ for $\bigcup_{\delta \in \Delta} \xrightarrow{\delta}$.

Note that $v \xrightarrow{\delta} v'$ implies that $v \xrightarrow{\delta} v'$ but the converse is false as shown in the following example.

Example 5.1. Consider an NB-VAS of dimension $d = 2$, with $\delta = (\delta_b, \delta_{nb}) \in \Delta$ such that $\delta_b = (-3, 0)$ and $\delta_{nb} = (0, 1)$, and let $v = (1, 2)$ and $v' = (0, 1)$. One can easily see that there does not exist $v'' \in \mathbb{N}^2$ such that $v'' = v + \delta_b$, as $1 - 3 < 0$. So, δ cannot be taken from v and it is not the case that $v \xrightarrow{\delta} v'$, however, $v \xrightarrow{\delta} v'$.

We now give a definition allowing to make the link between the relaxed semantics and the initial one.

Definition 5.1. Let $J \subseteq [1, d]$. A path $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_m$ is said to be J -correct if for all v_i such that $i < m$, there exists $\delta = (\delta_b, \delta_{nb}) \in \Delta$, with $v_i \xrightarrow{\delta} v_{i+1}$ and for all $j \in J$, $(v_i + \delta_b)(j) \geq 0$. We say that the path is *correct* if the path is $[1, d]$ -correct.

It follows from the definition that for all $v, v' \in \mathbb{N}^d$, $v \rightsquigarrow^* v'$ if and only if there exists a correct path between v and v' .

We now fix a target vector $v_f \in \mathbb{N}^d$, and define $N = \|v_f\|_1 + \max_{(\delta_b, \delta_{nb}) \in \Delta} (\|\delta_b\|_1 + \|\delta_{nb}\|_1)$, where $\|\cdot\|_1$ is the norm 1 of vectors in $\mathbb{Z}^d$, i.e. for a vector $v \in \mathbb{Z}^d$, $\|v\|_1 = \sum_{1 \leq i \leq d} |v(i)|$.

Definition 5.2. Let $\rho = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_m$ and $J \subseteq [1, d]$. We say the path ρ is J -covering if it is J -correct and $v_m(j) \geq v_f(j)$ for all $j \in J$.

Let $r \in \mathbb{N}$, we say that ρ is (J, r) -bounded if for all v_i , for all $j \in J$, $v_i(j) < r$. Let $v \in \mathbb{N}^d$, we define $\text{min-length}(J, v)$ as the length of the shortest J -covering path starting with v , and 0 if there is none.

Denote $\mathcal{J}_i = \{J \subseteq [1, d] \mid |J| = i\}$ and define the function f as follows: for $1 \leq i \leq d$, $f(i) = \max\{\text{min-length}(J_i, v) \mid J_i \in \mathcal{J}_i, \text{ and } v \in \mathbb{N}^d\}$. We will see that f is always well defined.

The rest of this proof is dedicated to prove that minimum lengths of J -covering paths are bounded by number that is exponential in the size of the set J . The result is obtained by an induction on the size of J .

Claim 5.1. $f(0) = 1$.

Proof. From any vector $v \in \mathbb{N}^d$, the path with one element v is $\emptyset$ -covering. □

Claim 5.2. $f(i+1) \leq (N \cdot f(i))^{i+1} + f(i)$ for all $0 \leq i < d$.

Proof. Let $J \in \mathcal{J}_{i+1}$ and $v \in \mathbb{N}^d$ such that there exists a J -covering path starting with v . Let $\rho = v_0 \xrightarrow{\delta^1} \dots \xrightarrow{\delta^m} v_m$ be the shortest such path (with $v_0 = v$).

First case: if ρ is $(J, N \cdot f(i))$ -bounded. We are going to bound the length of ρ by $(N \cdot f(i))^{i+1}$. In order to do this, we will prove that there are no two indices $k < \ell$ such that for all $j \in J$, $v_k(j) = v_\ell(j)$. Assume, for the sake of contradiction, that there exist two indices $k < \ell$, such that $v_k(j) = v_\ell(j)$ for all $j \in J$. Then we show that

$v_0 \rightarrow \dots v_k \rightarrow \bar{v}_{\ell+1} \dots \rightarrow \bar{v}_m$ is also a J -correct path, with the vectors $(\bar{v}_{\ell'})_{\ell < \ell' \leq m}$, defined as follows:

$$\bar{v}_{\ell+1}(j) = \begin{cases} v_{\ell+1}(j) & \text{for all } j \in J \\ \max(0, (v_k(j) + \delta_b^{\ell+1}(j) - \delta_{nb}^{\ell+1}(j))) & \text{otherwise,} \end{cases}$$

and for all $\ell + 1 < \ell' \leq m$,

$$\bar{v}_{\ell'}(j) = \begin{cases} v_{\ell'}(j) & \text{for all } j \in J \\ \max(0, (\bar{v}_{\ell'-1}(j) + \delta_b^{\ell'}(j) - \delta_{nb}^{\ell'}(j))) & \text{otherwise.} \end{cases}$$

We now prove that the built path satisfies what we want.

Claim 5.3. The path $v_0 \rightarrow \dots v_k \rightarrow \bar{v}_{\ell+1} \dots \rightarrow \bar{v}_m$ is J -correct.

Proof. We proof the claim by induction on $\ell' \in [\ell + 1, m]$.

Start with $\ell' = \ell + 1$. Since $v_k(j) = v_{\ell}(j)$ for all $j \in J$, we have that:

$$\begin{aligned} \bar{v}_{\ell+1}(j) &= v_{\ell+1}(j) \\ &= \max(0, (v_{\ell}(j) + \delta_b^{\ell+1}(j) - \delta_{nb}^{\ell+1}(j))) \\ &= \max(0, (v_k(j) + \delta_b^{\ell+1}(j) - \delta_{nb}^{\ell+1}(j))). \end{aligned}$$

Moreover, for $j \in J$, since $v_{\ell}(j) + \delta_b^{\ell+1}(j) \geq 0$ (because the path is J -correct), we get that $v_k(j) + \delta_b^{\ell+1}(j) \geq 0$. By definition, for $j \notin J$, $\bar{v}_{\ell+1}(j) = \max(0, (v_k(j) + \delta_b^{\ell+1}(j) - \delta_{nb}^{\ell+1}(j)))$. Hence, $v_k \xrightarrow{\delta^{\ell+1}} \bar{v}_{\ell+1}$, and $v_0 \xrightarrow{\delta^1} \dots v_k \xrightarrow{\delta^{\ell+1}} \bar{v}_{\ell+1}$ is J -correct.

Let now $\ell' \in [\ell + 1, m - 1]$ **and let us prove the claim for** $\ell' + 1$. By definition, for $j \in J$, $\bar{v}_{\ell'+1}(j) = v_{\ell'+1}(j)$. Hence,

$$\begin{aligned} \bar{v}_{\ell'+1}(j) &= \max(0, (v_{\ell'}(j) + \delta_b^{\ell'+1}(j) - \delta_{nb}^{\ell'+1}(j))) \\ &= \max(0, (\bar{v}_{\ell'}(j) + \delta_b^{\ell'+1}(j) - \delta_{nb}^{\ell'+1}(j))). \end{aligned}$$

Again, since ρ is J -correct, we deduce that for $j \in J$, $v_{\ell'}(j) + \delta_b^{\ell'+1}(j) \geq 0$, hence $\bar{v}_{\ell'}(j) + \delta_b^{\ell'+1}(j) \geq 0$. For $j \notin J$, $\bar{v}_{\ell'+1}(j) = \max(0, (\bar{v}_{\ell'}(j) + \delta_b^{\ell'+1}(j) - \delta_{nb}^{\ell'+1}(j)))$.

So $\bar{v}_{\ell'} \xrightarrow{\delta^{\ell'+1}} \bar{v}_{\ell'+1}$, and $v_0 \xrightarrow{\delta^1} \dots v_k \xrightarrow{\delta^{\ell'+1}} \bar{v}_{\ell'+1}$ is J -correct by induction. $\square$

Hence, the above claim allows us to build a J -correct path $\rho' = v_0 \rightarrow \dots v_k \rightarrow \bar{v}_{\ell+1} \dots \rightarrow \bar{v}_m$, and since $\bar{v}_m(j) = v_m(j)$ for all $j \in J$, it is also J -covering, contradicting the fact that ρ is minimal. It concludes our reasoning by contradiction, and hence, for all $k < \ell$, there exists $j \in J$ such that $v_k(j) \neq v_{\ell}(j)$. Hence, as our path is $(J, N \cdot f(i))$ -bounded with $|J| = i + 1$, the length of such a path is at most $(N \cdot f(i))^{i+1}$, so $\text{min-length}(J, v) \leq (N \cdot f(i))^{i+1} \leq (N \cdot f(i))^{i+1} + f(i)$.

Second case: if ρ is not $(J, N \cdot f(i))$ -bounded. We can then split ρ into two paths $\rho_1 \rho_2$ such that ρ_1 is $(J, N \cdot f(i))$ -bounded and $\rho_2 = v'_0 \dots v'_n$ is such that $v'_0(j) \geq N \cdot f(i)$ for

some $j \in J$. As we have just seen in the first case, $|\rho_1| \leq (N \cdot f(i))^{i+1}$. We now focus on bounding the length of ρ_2 .

Define $J' = J \setminus \{j\}$ with j such that $v'_0(j) \geq N \cdot f(i)$. Note that ρ_2 is J' -covering, therefore, by definition of f , there exists a J' -covering execution $\bar{\rho} = w_0 \dots w_k$ with $w_0 = v'_0$, and such that $|\bar{\rho}| \leq f(i)$.

Our challenge now is to show that $\bar{\rho}$ is also J -correct and J -covering.

Recall that:

$$N = \|v_f\|_1 + \max_{(\delta_b, \delta_{nb}) \in \Delta} (\|\delta_b\|_1 + \|\delta_{nb}\|_1).$$

Hence for all index $j' \in [1, d]$ and transition $(\delta_b, \delta_{nb}) \in \Delta$, it holds that:

$$N \geq \|\delta_b(j')\|_1 + \|\delta_{nb}(j')\|_1$$

Hence, $\delta_b(j') \geq -N$, and $\delta_b(j') - \delta_{nb}(j') \geq -N$. Hence, for all vector $v \in \mathbb{N}^d$, index $j' \in [1, d]$, and constant $c \in \mathbb{N}$ such that $v(j') \geq N + c$, it holds that for all transition $(\delta_b, \delta_{nb}) \in \Delta$:

$$(v + \delta_b)(j') \geq c \text{ and } (v + \delta_b - \delta_{nb})(j') \geq c.$$

Now, since $w_0 = v'_0$, we get $w_0(j) \geq N \cdot f(i)$. We deduce two things:

- first, for all $\ell \in [0, k-1]$, if $\delta = (\delta_b, \delta_{nb}) \in \Delta$ is such that $w_\ell \xrightarrow{\delta} w_{\ell+1}$, it holds that $(w_\ell + \delta_b)(j) \geq N \cdot (f(i) - \ell - 1)$. Since $k = f(i) - 1$, it yields that $\bar{\rho}$ is J -correct.
- second, for all $\ell \in [0, k]$, $w_\ell(j) \geq N \cdot (f(i) - \ell)$. Again, $k = f(i) - 1$, so $w_k(j) \geq N \geq v_f(j)$. Hence $\bar{\rho}$ is also J -covering.

Since ρ is the shortest J -covering path, we conclude that $|\rho| \leq |\rho_1 \cdot \bar{\rho}| \leq (N \cdot f(i))^{i+1} + f(i)$, and so $\min\text{-length}(J, v) \leq (N \cdot f(i))^{i+1} + f(i)$. $\square$

This allows to compute a bound on the length of a covering execution and establish the following result.

Theorem 5.1. *The COVER problem for NB-VAS is in EXPSPACE.*

Proof. Recall that for all $v, v' \in \mathbb{N}^d$, $v \rightsquigarrow^* v'$ if and only if there exists a correct path between v and v' . We define a function g such that $g(0) = 1$ and $g(i+1) = (N+1)^d(g(i))^d$ for all $0 \leq i < d$. Then $f(i) \leq g(i)$ for all $1 \leq i \leq d$, and so, $f(d) \leq g(d) \leq (N+1)^{d^{d+1}}$. As for the NB-VAS, we assume that v_f is written in unary, hence its size is $\|v_f\|_1$. Recall that $N = \|v_f\|_1 + \max_{(\delta_b, \delta_{nb}) \in \Delta} (\|\delta_b\|_1 + \|\delta_{nb}\|_1)$ and observe that it is linear in the size of the NB-VAS and the vector v_f to cover. Hence, $(N+1)^{d^{d+1}} \leq 2^{2^{cn}}$ for a constant c and $n = |V| + \|v_f\|_1$. Hence, we can cover vector v_f from v_{init} if and only if there exists a path (from v_{init}) of length $\leq 2^{2^{cn}}$ which covers v_f . As a consequence, there is a non-deterministic procedure that guesses a path of length $\leq 2^{2^{cn}}$, checks if it is a valid path and accepts it if and only if it covers v_f . As $\|v_{\text{init}}\|_1 \leq n$, $\|v_f\|_1 \leq n$ and for all $(\delta_b, \delta_{nb}) \in T$, $\|\delta_b\|_1 + \|\delta_{nb}\|_1 \leq n$, this procedure takes an exponential space in the size of the NB-VAS (recall that the size is $d \times (\|v_{\text{init}}\| + |\Delta| \times \max(\|\delta_b\| + \|\delta_{nb}\| \bmod (\delta_b, \delta_{nb}) \in \Delta))$). By Savitch's theorem, there exists a deterministic procedure in exponential space for the same problem. $\square$

5.2.1.2 From NB-VAS to NB-CM

We are now ready to prove that the COVER problem for NB-VAS is as hard as the COVER problem for NB-CM. To do this, we present a reduction from COVER[NB-CM] to COVER[NB-VAS].

Observe that there is also a reduction from COVER[NB-VAS] to COVER[NB-CM], using the same technique used to transform a VASS into a CM, presented in Chapter 3 (we unfold each transition). The reduction from COVER[NB-CM] to COVER[NB-VAS] is not difficult: we encode the NB-CM's locations into the NB-VAS's counters. The initial vector ensures that at most one location's counter is non null.

Let an NB-CM $CM = (Loc, X, \Delta_b, \Delta_{nb}, \ell_{in})$, for which we assume wlog that it does not contain any self-loop (replace a self loop on a location by a cycle using an additional internal transition and an additional location). We denote the set of counters and locations respectively by $X = \{x_1, \dots, x_m\}$, and $Loc = \{\ell_1 \dots \ell_k\}$, with $\ell_1 = \ell_{in}$ and $\ell_k = \ell_f$, and let $\Delta = \Delta_b \cup \Delta_{nb}$, $d = k + m$.

We define the NB-VAS $V_{CM} = (\Delta_{CM}, v_{init})$ of dimension d as follows: it has one counter by location of the NB-CM, and one counter for each one of the NB-CM.

The transitions will ensure that the sum of the values of the counters representing the locations of CM will always be equal to 1, hence a vector during an execution of V_{CM} will always represent a configuration of CM. First, for a transition $\delta = (\ell_i, op, \ell_{i'}) \in \Delta$, we define $t_\delta = (t_\delta^b, t_\delta^{nb}) \in \mathbb{Z}^d \times \mathbb{N}^d$ by $t_\delta^b(i) = -1$, $t_\delta^b(i') = 1$ and,

- if $op = \perp$, then $t_\delta^b(y) = 0$ for all other $y \in [1, d]$, and $t_\delta^{nb} = \mathbf{0}_d$, i. e. no other modification is made on the counters.
- if $op = x_j+$, then $t_\delta^b(k+j) = 1$, $t_\delta^b(y) = 0$ for all other $y \in [1, d]$, and $t_\delta^{nb} = \mathbf{0}_d$, i. e. the blocking part of the transition ensures the increment of the corresponding counter, while the non-blocking part does nothing.
- if $op = x_j-$, then $t_\delta^b(k+j) = -1$, $t_\delta^b(y) = 0$ for all other $y \in [1, d]$, and $t_\delta^{nb} = \mathbf{0}_d$, i. e. the blocking part of the transition ensures the decrement of the corresponding counter, while the non-blocking part does nothing.
- if $op = nb(x_j-)$, then $t_\delta^b(y) = 0$ for all other $y \in [1, d]$, $t_\delta^{nb}(k+j) = -1$ and $t_\delta^{nb}(y) = 0$ for all other $y \in [1, d]$, i. e. the blocking part of the transition only ensures the change in the location, and the non-blocking decrement of the counter is ensured by the non-blocking part of the transition.

We then let $\Delta_{CM} = \{t_\delta \mid \delta \in \Delta\}$, and v_{init} is defined by $v_{init}(1) = 1$ and $v_{init}(y) = 0$ for all $y \in [2, d]$. We also fix v_f by $v_f(k) = 1$, and $v_f(y) = 0$ for all other $y \in [1, d] \setminus \{k\}$.

We now proceed to prove soundness and completeness of this reduction.

Lemma 5.1. If there exists $w \in \mathbb{N}^X$ such that $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell_f, w)$, then there exists $v \in \mathbb{N}^d$ such that $v_{init} \rightsquigarrow^* v$ and $v_f \leq v$.

Proof. Any configuration (ℓ_i, w) of CM can be turned into a vector $v_{\ell_i, w}$ of $\mathbb{N}^{k+m}$ such that

$$v_{\ell_i, w}(j) = \begin{cases} 1 & \text{if } j = i \\ 0 & \text{if } j \in [1, k] \setminus \{i\} \\ w(x_{j-k}) & \text{if } j \in [k+1, k+m] \end{cases}$$

Observe that $v_{\ell_{\text{init}}, \mathbf{0}_X} = v_{\text{init}}$. It follows from the definition of Δ_{CM} that $(\ell_i, w) \rightsquigarrow (\ell_{i'}, w')$ if and only if $v_{\ell_i, w} \rightsquigarrow v_{\ell_{i'}, w'}$. Hence, $v_{\text{init}} \rightsquigarrow^* v_{\ell_f, w} \geq v_f$. $\square$

Lemma 5.2. If there exists $v \in \mathbb{N}^d$ such that $v_{\text{init}} \rightsquigarrow^* v$ and $v \geq v_f$, then there exists $w \in \mathbb{N}^X$ such that $(\ell_{\text{init}}, \mathbf{0}_X) \rightsquigarrow^* (\ell_f, w)$.

Proof. First observe that every vector v reachable from v_{init} is such that there exists only one index $i \in [1, k]$ such that $v(i) = 1$ and for all other index $i' \in [1, k] \setminus \{i\}$, $v(i') = 0$. Indeed, each transition of Δ_{CM} decrements one index in $[1, k]$ and increments another index of $[1, k]$. Hence, starting with a vector where exactly one index in $[1, k]$ is non-null and equal to 1, ensures that all reachable vectors have exactly one non-null component in $[1, k]$ and it is equal to 1.

Hence, given a reachable vector v , one can define a machine configuration as $\gamma_v = (\ell_i, w)$ where i is the unique index $i \in [1, k]$ such that $v(i) = 1$ and, for all $j \in [1, m]$, $w(x_j) = v(k+j)$. Note $v_{\text{init}} \rightsquigarrow v_1 \rightsquigarrow \dots \rightsquigarrow v_n = v$, and observe that $\gamma_{v_n} = (\ell_f, w)$ for some $w \in \mathbb{N}^X$. Using Δ_{CM} 's definition, we get that $\gamma_{v_{\text{init}}} \rightsquigarrow \gamma_{v_1} \rightsquigarrow \dots \rightsquigarrow \gamma_{v_n}$, which concludes the proof. $\square$

The two above lemmas lead to the following.

Lemma 5.3. COVER[NB-CM] reduces to COVER[NB-VAS].

Putting together theorem 5.1 and lemma 5.3, we obtain the proof of Theorem 5.2.

Theorem 5.2. COVER[NB-CM] and COVER[NB-R-CM] are in EXPSPACE.

5.2.2 EXPSPACE-hardness

In this subsection, we are interested in proving EXPSPACE-hardness for coverability with NB-R-CM (non-blocking test-free counter machines with restore). Recall that an NB-R-CM is an NB-CM (non-blocking test-free counter machines¹) equipped with reset transitions: from any location of the machine, there is a transition, with no effect on the counters, going to the initial location. Hence, EXPSPACE-hardness of coverability for NB-R-CM also proves EXPSPACE-hardness for the same problem on NB-CM. This proof is with no doubt the most challenging one of this chapter. The lower bound proof is inspired by Lipton's proof showing that coverability in Vector Addition Systems with States is EXPSPACE-hard (first presented in [Lip76], and later presented in [Esp96] in a more pedagogical manner).

In [Lip76], Lipton proves that, given a counter machine of size n (with tests to zero), whose counters are smaller than or equal to 2^{2^n} in any reachable configurations, one can

¹ A non-blocking test-free counter machine is a counter machine where transitions can either decrement, increment or non-blockingly decrement a counter. See Chapter 3 for the formal definition.

build a VASS of size $O(n^2)$ simulating the counter machine. Furthermore, the VASS counters are also smaller than or equal to 2^{2^n} in any reachable configurations.

We mimic Lipton's proof and rely on 2Exp-bounded test-free CM. We formalize the notion of 2Exp-bounded.

Definition 5.3. A CM $CM = (Loc, X, \Delta, \ell_{in})$ is *2Exp-bounded* if there exists $n \in O(|CM|)$ such that for any (ℓ, v) such that $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell, v)$, it holds that $v(x) \leq 2^{2^n}$ for all $x \in X$.

We use then the following result.

Theorem 5.1 (Part II, Section 7 of [Esp96], [Lip76]). *COVER[2Exp-bounded test-free CM] is EXPSPACE-hard.*

The above theorem comes from the fact that the VASS presented in [Esp96; Lip76] is 2Exp-bounded.

We now show how to simulate a 2Exp-bounded test-free CM with a NB-R-CM, by carefully handling restore transitions that may occur at any point during the execution. We ensure that each restore transition is followed by a reset of the counters, so that we can always extract from any execution of the NB-R-CM a valid initial execution of the original test-free CM. Note that in the absence of restore transitions, the reduction would be trivial, as any test-free CM is already a NB-CM. However, restore transitions complicate the reduction: they require adding to each location of the test-free CM a transition back to the initial location. Since this transition does not reset the counters, it enables additional executions. To overcome this, we devise a mechanism to reset the counters before resuming the simulation of the test-free CM. This is made possible by the use of non-blocking decrements. To implement the reset, we adapt the zero-test simulation mechanism introduced by [Lip76].

As in [Lip76; Esp96], we will describe the final NB-R-CM by means of several submachines. To this end, we define *procedural non-blocking counter machines* that are NB-CMs with several identified *output states*.

Definition 5.4. A procedural-NB-CM PM is a tuple $(Loc, X, \Delta_b, \Delta_{nb}, \ell_{in}, L_{out})$ such that $(Loc, X, \Delta_b, \Delta_{nb}, \ell_{in})$ is a NB-CM, $L_{out} \subseteq Loc$, and there is no outgoing transition from locations in L_{out} , i. e. each transition $(\ell, v, \ell') \in (\Delta_b \cup \Delta_{nb})$, satisfies $\ell \notin L_{out}$.

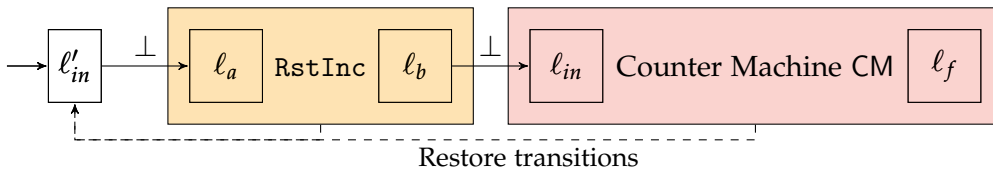


Figure 5.1: The NB-R-CM NM simulating the 2Exp-bounded counter machine CM.

Now, we fix a 2Exp-bounded test-free CM $CM = (Loc, X, \Delta, \ell_{in})$, and $\ell_f \in Loc$ the location to be covered. Thanks to Definition 5.3, there exists $n \in O(|CM|)$ such that any reachable configuration (ℓ, v) satisfies $v(x) < 2^{2^n}$ for all $x \in X$. We build a NB-R-CM NM as pictured in Figure 5.1. The goal of the procedural-NB-CM RstInc is to ensure that all counters in X are reset. Hence, after each reset transition, we are sure that we start over a fresh execution of the test-free CM CM.

For a counter x bounded by some value K , this is done by duplicating x into $\bar{x}$ and ensure along any execution that the sum of x and $\bar{x}$ is equal to K .

This leads us to define two families of sets of counters $(Y_i)_{0 \leq i \leq n}$ and $(\bar{Y}_i)_{0 \leq i \leq n}$ as follows. Let $Y_i = \{y_i, z_i, s_i\}$, $\bar{Y}_i = \{\bar{y}_i, \bar{z}_i, \bar{s}_i\}$ for all $0 \leq i < n$, $Y_n = X$, $\bar{Y}_n = \emptyset$ and $X' = \bigcup_{0 \leq i \leq n} Y_i \cup \bar{Y}_i$. All the machines we will describe from now on will work over the set of counters X' . Note that the n used here is the n defined earlier: $n = c \cdot |\text{CM}|$.

The procedural machines defined by Lipton were defined as machines without non-blocking decrements. However they can be of course considered as procedural-NB-CM with no non-blocking decrements ($\Delta_{nb} = \emptyset$).

We start by giving an overview of how the rest of this section will go.

Road map. Most of this section is dedicated to build the procedural-NB-CM `RstInc`, which is finally presented in Figure 5.7. We use procedural-NB-CMs presented in [Lip76; Esp96] as black boxes, meaning that we do not explicit them but just formalize the properties they ensure.

- We start by presenting a procedural-NB-CM named `TestSwap`: it tests the value of one counter $\bar{x}$, if it is equal to 0, it swaps it with counter x 's value, otherwise it remains unchanged. This procedural-NB-CM comes from [Lip76; Esp96]. (Section 5.2.2.1)
- We then proceed to build a family of procedural-NB-CM `Rst` _{$0 \leq i \leq n$} . It uses `TestSwap` and resets counters in $Y_i \cup \bar{Y}_i$ to 0. To do so it uses non-blocking decrements. This is the part that is particular to NB-CM and which represents our contribution. (Section 5.2.2.2)
- `Rst` _{i} requires precise hypothesis on other counters in $Y_j \cup \bar{Y}_j$ for $0 \leq j < i$. It requires the input counters valuation v to be such that $v(\bar{x}) = 2^{2^j}$ and $v(x) = 0$ for all $\bar{x} \in \bar{Y}_j$, and $x \in Y_j$. To realize this hypothesis, we use $(\text{Rst}_j)_{0 \leq j < i}$ as well as a family of procedural-NB-CM introduced in [Lip76; Esp96], named $(\text{Inc}_i)_{0 \leq i \leq n}$: the procedural-NB-CM `Inc` _{i} increases of 2^{2^i} counters in $\bar{Y}_i$ given that $v(\bar{y}) = 2^{2^j}$ and $v(y) = 0$ for all $\bar{y} \in \bar{Y}_j$, $y \in Y_j$ and $0 \leq j < i$. We recall this procedural-NB-CM in Section 5.2.2.3.
- We are then ready to present the procedural-NB-CM `RstInc`, which, given a counter valuation v with $v(x) \leq 2^{2^i}$ for all $x \in Y_i \cup \bar{Y}_i$ and $0 \leq i \leq n$, resets the counters in the following way: counters in Y_i are reset to 0 and counters in $\bar{Y}_i$ are set to 2^{2^i} . (Section 5.2.2.4)
- We finally present the reduction itself, leading to the main theorem of this section (Theorem 5.3). (Section 5.2.2.5)
- In Section 5.2.2.6, we present an example of our reduction. We give an example of an 2-EXPBOUNDED test-free CM as well as the NB-R-CM built by our reduction to simulate the test-free CM. One can typically either follow the formal proof from Section 5.2.2.1 to Section 5.2.2.5, or gain intuition from this example.

5.2.2.1 Procedural-NB-CM `TestSwap`.

We use a family of procedural-NB-CM defined in [Lip76; Esp96]: for all $0 \leq i < n$, for all $\bar{x} \in \bar{Y}_i$, `TestSwap` _{i} ($\bar{x}$) is a procedural-NB-CM with an initial location $\ell_{in}^{\text{TS},i,x}$, and two output locations $\ell_z^{\text{TS},i,x}$ and $\ell_{nz}^{\text{TS},i,x}$. It tests if the value of $\bar{x}$ is equal to 0, using the fact that the sum of the values of x and $\bar{x}$ is equal to 2^{2^i} . If $\bar{x} = 0$, it swaps the values of x and $\bar{x}$, and the execution ends in the output location $\ell_z^{\text{TS},i,x}$. Otherwise, counters values are left

unchanged and the execution ends in $\ell_{nz}^{TS,i,x}$. In any case, other counters are not modified by the execution. Note that $\text{TestSwap}_i(\bar{x})$ makes use of variables in $\bigcup_{1 \leq j < i} Y_j \cup \bar{Y}_i$.

Formally, these machines have the following property:

Proposition 5.1 ([Lip76; Esp96]). Let $0 \leq i < n$, and $\bar{x} \in \bar{Y}_i$. For all $v, v' \in \mathbb{N}^{X'}$, for $\ell \in \{\ell_z^{TS,i,x}, \ell_{nz}^{TS,i,x}\}$, we have $(\ell_{in}^{TS,i,x}, v) \rightsquigarrow^* (\ell, v')$ in $\text{TestSwap}_i(\bar{x})$ if and only if :

- (PreTest1): for all $0 \leq j < i$, for all $\bar{x}_j \in \bar{Y}_j$, $v(\bar{x}_j) = 2^{2^j}$ and for all $x_j \in Y_j$, $v(x_j) = 0$;
- (PreTest2): $v(\bar{s}_i) = 2^{2^i}$ and $v(s_i) = 0$;
- (PreTest3): $v(x) + v(\bar{x}) = 2^{2^i}$;
- (PostTest1): For all $y \notin \{x, \bar{x}\}$, $v'(y) = v(y)$;
- (PostTest2): either (i) $v(\bar{x}) = v'(x) = 0$, $v(x) = v'(\bar{x})$ and $\ell = \ell_z^i$, or (ii) $v'(\bar{x}) = v(\bar{x}) > 0$, $v'(x) = v(x)$ and $\ell = \ell_{nz}^{TS,i,x}$.

Moreover, if for all $j \in [0, n]$ and any counter $x \in Y_j \cup \bar{Y}_j$, $v(x) \leq 2^{2^j}$, then values of counters $Y_j \cup \bar{Y}_j$ will never go above 2^{2^j} for all $j \in [0, n]$ during the run.

To make the above proposition more readable, we illustrate it in Figure 5.2.

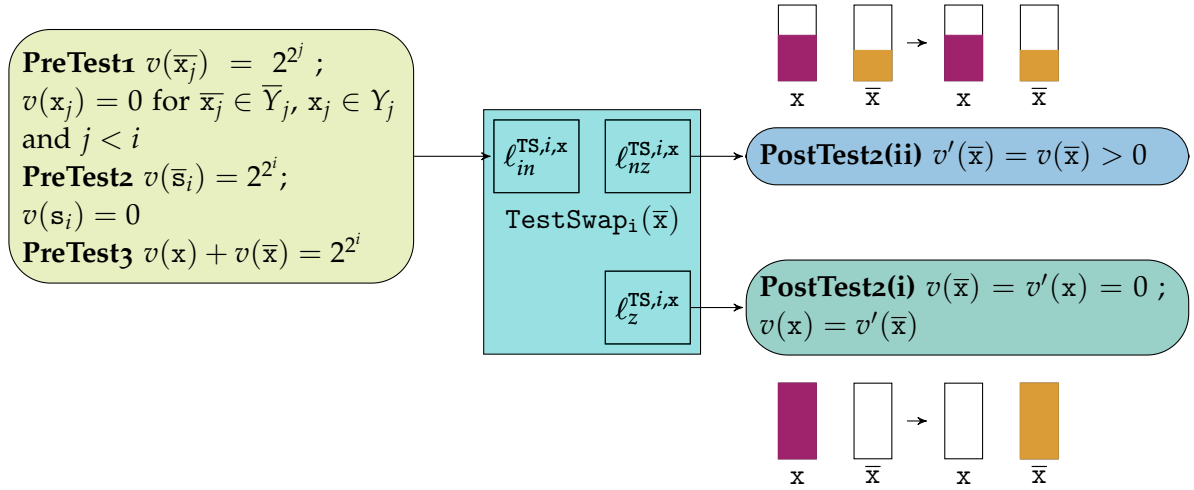


Figure 5.2: Illustration of Proposition 5.1 for procedural-NB-CM $\text{TestSwap}_i(\bar{x})$.

Note that for a valuation $v \in \mathbb{N}^{X'}$ that meets the requirements (PreTest1), (PreTest2) and (PreTest3), there is only one configuration (ℓ, v') with $\ell \in \{\ell_z^{TS,i,x}, \ell_{nz}^{TS,i,x}\}$ such that $(\ell_{in}, v) \rightsquigarrow^* (\ell, v')$.

5.2.2.2 Procedural-NB-CM Rst.

We use these machines to define a family of procedural-NB-CM called $(\text{Rst}_i)_{0 \leq i \leq n}$ that reset the counters in $Y_i \cup \bar{Y}_i$, assuming that their values are $\leq 2^{2^i}$. For $0 \leq i \leq n$, we let $\text{Rst}_i = (\text{Loc}^{R,i}, X', \Delta_b^{R,i}, \Delta_{nb}^{R,i}, \ell_{in}^{R,i}, \{\ell_{out}^{R,i}\})$. The machine Rst_0 is pictured Figure 5.3. More formally, it is defined as follows: $\text{Rst}_0 = (\text{Loc}^{R,0}, X', \Delta_b^{R,0}, \Delta_{nb}^{R,0}, \ell_{in}^{R,0}, \{\ell_{out}^{R,0}\})$ with

- $\text{Loc}^{R,0} = \{\ell_{in}^{R,0}, \ell_{out}^{R,0}\} \cup \{\ell_i^{R,0} \mid 1 \leq i \leq 11\}$;

- $\Delta_b^{R,0} = \emptyset$;
- Taking the convention that $\ell_0^{R,0} = \ell_{in}^{R,0}$ and $\ell_{12}^{R,0} = \ell_{out}^{R,0}$, we have:

$$\Delta_{nb}^{R,0} = \{(\ell_i^{R,0}, nb(x_0-), \ell_{i+1}^{R,0}) \mid \begin{array}{l} \text{if } i \in \{0, 1\} \text{ then } x_0 = y_0, \\ \text{if } i \in \{2, 3\} \text{ then } x_0 = \bar{y}_0, \\ \text{if } i \in \{4, 5\} \text{ then } x_0 = z_0, \\ \text{if } i \in \{6, 7\} \text{ then } x_0 = \bar{z}_0, \\ \text{if } i \in \{8, 9\} \text{ then } x_0 = s_0, \\ \text{if } i \in \{10, 11\} \text{ then } x_0 = \bar{s}_0 \end{array}\right\}$$

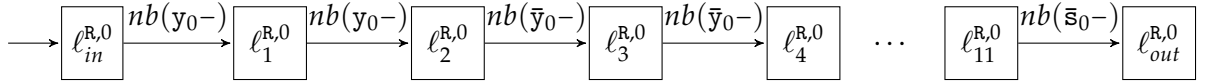


Figure 5.3: Description of Rst_0 . Between $\ell_4^{R,0}$ and $\ell_{out}^{R,0}$, we perform successive non-blocking decrements of z_0 (twice), $\bar{z}_0$ (twice), s_0 (twice), and $\bar{s}_0$ (twice).

The procedural-NB-CM decrements (in a non-blocking fashion) twice each counter in $Y_0 \cup \bar{Y}_0$. Hence, if counters $y \in Y_0 \cup \bar{Y}_0$ are such that $y \leq 2^{2^0} = 2$, then after executing Rst_0 , it holds that $y = 0$. Observe that if $y < 2$, the execution of Rst_0 still terminates because of the non-blocking decrement definition.

For all $0 \leq i < n$, the machine Rst_{i+1} uses counters from $Y_i \cup \bar{Y}_i$ and procedural-NB-CM $Testswap_i(\bar{z}_i)$ and $Testswap_i(\bar{y}_i)$ to control the number of times variables from Y_{i+1} and $\bar{Y}_{i+1}$ are decremented. It is pictured in Figure 5.4. Observe that since $Y_n = X$, and $\bar{Y}_n = \emptyset$, the machine Rst_n will be a bit different from the picture: there will only be non-blocking decrements over counters from Y_n , that is over counters X from the initial test-free CM CM. If $\bar{y}_i, \bar{z}_i$ (and $\bar{s}_i$) are set to 2^{2^i} and y_i, z_i (and s_i) are set to 0, then each time this procedural-NB-CM takes an outer loop, the variables of $Y_{i+1} \cup \bar{Y}_{i+1}$ are decremented (in a non-blocking fashion) 2^{2^i} times. This is ensured by Proposition 5.1. Moreover, the location $\ell_z^{TS,i,y}$ will only be reached when the counter $\bar{y}_i$ is set to 0, and this will happen after 2^{2^i} iterations of the outer loop, again thanks to Proposition 5.1. So, all in all, variables from Y_{i+1} and $\bar{Y}_{i+1}$ will take a non-blocking decrement $2^{2^i} \cdot 2^{2^i}$ times, that is $2^{2^{i+1}}$.

These properties are formalized in the following proposition.

Proposition 5.2. For all $0 \leq i \leq n$, for all $v \in \mathbb{N}^{X'}$ such that

- (PreRst1): for all $0 \leq j < i$, for all $\bar{x} \in \bar{Y}_j$, $v(\bar{x}) = 2^{2^j}$ and for all $x \in Y_j$, $v(x) = 0$,

for all $v' \in \mathbb{N}^{X'}$, if $(\ell_{in}^{R,i}, v) \rightsquigarrow^* (\ell_{out}^{R,i}, v')$ in Rst_i then

- (PostRst1): for all $x \in Y_i \cup \bar{Y}_i$, $v'(x) = \max(0, v(x) - 2^{2^i})$,
- (PostRst2): for all $x \notin Y_i \cup \bar{Y}_i$, $v'(x) = v(x)$.

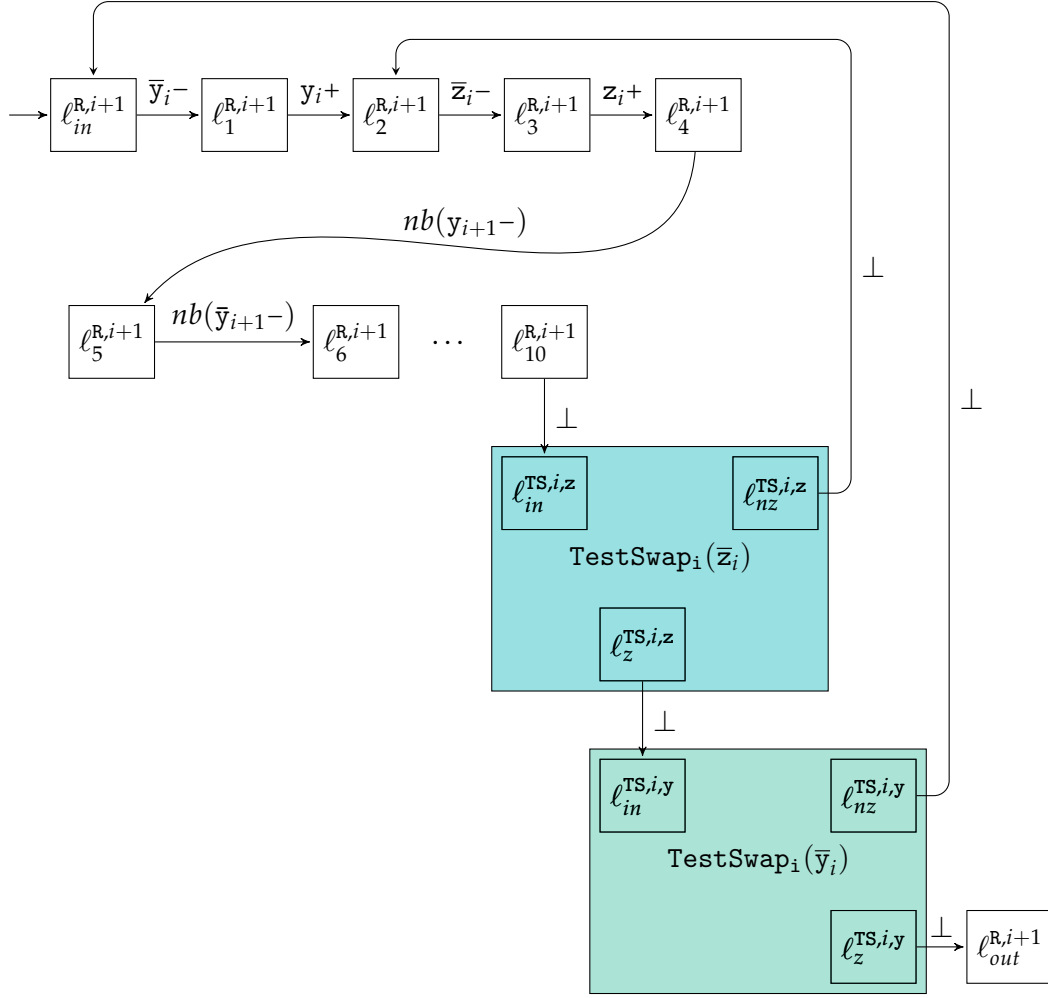


Figure 5.4: Description of Rst_{i+1} . Between $\ell_6^{R,i+1}$ and $\ell_{10}^{R,i+1}$, we perform successive non-blocking decrements of z_{i+1} , $\bar{z}_{i+1}$, s_{i+1} , and $\bar{s}_{i+1}$.

Proof. We start by observing that, for Rst_0 , (PreRst1) trivially holds, and it is easy to see that (PostRst1) and (PostRst2) hold. We now prove the Proposition for the machines $(\text{Rst}_i)_{1 \leq i \leq n}$. We fix $0 \leq i < n$, and work with the procedural-NB-CM Rst_{i+1} .

Let $v_0 \in \mathbb{N}^{X'}$ such that for all $0 \leq j < i + 1$, for all $\bar{x} \in \bar{Y}_j$, $v_0(\bar{x}) = 2^{2^j}$ and for all $x \in Y_j$, $v_0(x) = 0$, and let v_f such that $(\ell_{in}^{R,i+1}, v_0) \rightsquigarrow^+ (\ell_{out}^{R,i+1}, v_f)$ in Rst_{i+1} .

First, we show the following property.

Property ():* if there exist $v, v' \in \mathbb{N}^{X'}$ and $k \in \mathbb{N}$ such that $v(\bar{z}_i) = k$, $(\ell_{in}^{TS,i,z}, v) \rightsquigarrow^* (\ell_z^{TS,i,z}, v')$ with no other visit of $\ell_z^{TS,i,z}$ in between, then $v'(\bar{z}_i) = 2^{2^i}$, $v'(z_i) = 0$, for all $x \in Y_{i+1} \cup \bar{Y}_{i+1}$, $v'(x) = \max(0, v(x) - k)$, and $v'(x) = v(x)$ for all other $x \in X'$.

Proof of Property ():* The proof is by induction on k . If $k = 0$, then Proposition 5.1 ensures that $v'(\bar{z}_i) = 2^{2^i}$, $v'(z_i) = 0$, and for all other $x \in X'$, $v'(x) = v(x)$. Otherwise, assume that the property holds for some $k \geq 0$ and consider $(\ell_{in}^{TS,i,z}, v) \rightsquigarrow^* (\ell_z^{TS,i,z}, v')$ with no other visit of $\ell_z^{TS,i,z}$ in between, and $v(\bar{z}_i) = k + 1$. Then, thanks to Proposition 5.1

and the construction of the procedural-NB-CM Rst_{i+1} we get that $(\ell_{in}^{\text{TS},i,z}, v) \rightsquigarrow^* (\ell_{nz}^{\text{TS},i,z}, v) \rightsquigarrow (\ell_2^{\text{R},i+1}, v) \rightsquigarrow^* (\ell_{in}^{\text{TS},i,z}, v_1)$ with:

$$\begin{aligned} v_1(\bar{z}_i) &= k \text{ and } v_1(z_i) = v(z_i) + 1, \\ v_1(x) &= \max(0, v(x) - 1) \text{ for all } x \in Y_{i+1} \cup \bar{Y}_{i+1}, \\ v_1(x) &= v(x) \text{ for all other } x \in X'. \end{aligned}$$

Induction hypothesis tells us that $(\ell_{in}^{\text{TS},i,z}, v_1) \rightsquigarrow^* (\ell_z^{\text{TS},i,z}, v')$ with:

$$\begin{aligned} v'(\bar{z}_i) &= 2^{2^i} \text{ and } v'(z_i) = 0, \\ v'(x) &= \max(0, v(x) - k - 1) \text{ for all } x \in Y_{i+1} \cup \bar{Y}_{i+1}, \\ v'(x) &= v(x) \text{ for all other } x \in X'. \end{aligned}$$

This concludes the proof of Property (*).

Next, we show the following.

*Property (**):* if there exist $v, v' \in \mathbb{N}^{X'}$ such that $v(\bar{y}_i) = k$, $v(\bar{z}_i) = 2^{2^i}$, $v(z_i) = 0$, and $(\ell_{in}^{\text{TS},i,y}, v) \rightsquigarrow^* (\ell_z^{\text{TS},i,y}, v')$ with no other visit of $\ell_z^{\text{TS},i,y}$ in between, then $v'(\bar{y}_i) = 2^{2^i}$, $v'(y_i) = 0$, for all $x \in Y_{i+1} \cup \bar{Y}_{i+1}$, $v'(x) = \max(0, v(x) - k \cdot 2^{2^i})$, and $v'(x) = v(x)$ for all other $x \in X'$.

*Proof of Property (**):* The proof is by induction on k . If $k = 0$, then Proposition 5.1 ensures that $v'(\bar{y}_i) = 2^{2^i}$, $v'(y_i) = 0$, and $v'(x) = v(x)$ for all other $x \in X'$. Otherwise, assume that the property holds for some $k \geq 0$ and consider $(\ell_{in}^{\text{TS},i,y}, v) \rightsquigarrow^* (\ell_z^{\text{TS},i,y}, v')$ with no other visit of $\ell_z^{\text{TS},i,y}$ in between, and $v(\bar{y}_i) = k + 1$. Again, since $v(\bar{y}_i) = k + 1$, Proposition 5.1 and the construction of the procedural-NB-CM Rst_{i+1} ensure that $(\ell_{in}^{\text{TS},i,y}, v) \rightsquigarrow^* (\ell_{nz}^{\text{TS},i,y}, v) \rightsquigarrow (\ell_{in}^{\text{R},i+1}, v) \rightsquigarrow^* (\ell_{in}^{\text{TS},i,z}, v_1) \rightsquigarrow^* (\ell_z^{\text{TS},i,z}, v'_1) \rightsquigarrow (\ell_{in}^{\text{TS},i,y}, v'_1)$, with:

$$\begin{aligned} v_1(\bar{y}_i) &= v(\bar{y}_i) - 1 = k \\ v_1(y_i) &= v(y_i) + 1 \\ v_1(\bar{z}_i) &= v(\bar{z}_i) - 1 = 2^{2^i} - 1 \\ v_1(z_i) &= v(z_i) + 1 = 1 \\ v_1(x) &= \max(0, v(x) - 1) \text{ for all } x \in Y_{i+1} \cup \bar{Y}_{i+1} \\ v_1(x) &= v(x) \text{ for all other } x \in X'. \end{aligned}$$

By Property (*):

$$\begin{aligned} v'_1(\bar{z}_i) &= 2^{2^i} \text{ and } v'_1(z_i) = 0 \\ v'_1(x) &= \max(0, v(x) - 2^{2^i}) \text{ for all } x \in Y_{i+1} \cup \bar{Y}_{i+1} \\ v'_1(x) &= v_1(x) \text{ for all other } x \in X'. \end{aligned}$$

Induction hypothesis allows to conclude that since $(\ell_{in}^{\text{TS},i,y}, v'_1) \rightsquigarrow^* (\ell_z^{\text{TS},i,y}, v')$, we get:

$$\begin{aligned} v'(\bar{y}_i) &= 2^{2^i} \text{ and } v'(y_i) = 0 \\ v'(x) &= \max(0, v'_1(x) - k \cdot 2^{2^i}) \\ &= \max(0, v(x) - (k + 1) \cdot 2^{2^i}) \text{ for all } x \in Y_{i+1} \cup \bar{Y}_{i+1} \\ v'(x) &= v_1(x) = v(x) \text{ for all other } x \in X'. \end{aligned}$$

This concludes the proof of Property (**).

Since $(\ell_{in}^{R,i+1}, v_0) \rightsquigarrow^+ (\ell_{out}^{R,i+1}, v_f)$, we know that $(\ell_{in}^{R,i}, v_0) \rightsquigarrow^* (\ell_{in}^{TS,i,z}, v) \rightsquigarrow^* (\ell_z^{TS,i,z}, v') \rightsquigarrow (\ell_{in}^{TS,i,y}, v') \rightsquigarrow^* (\ell_z^{TS,i,y}, v'') \rightsquigarrow (\ell_{out}^{R,i}, v_f)$. By construction,

$$\begin{aligned} v(\bar{y}_i) &= v(\bar{z}_i) = 2^{2^i} - 1 \\ v(z_i) &= v(y_i) = 1 \\ v(x) &= \max(0, v_0(x) - 1) \text{ for all } x \in Y_{i+1} \cup \bar{Y}_{i+1} \\ v(x) &= v_0(x) \text{ for all other } x \in X'. \end{aligned}$$

By Property (*),

$$\begin{aligned} v'(\bar{z}_i) &= 2^{2^i} = v_0(\bar{z}_i) \\ v'(z_i) &= 0 = v_0(z_i) \\ v'(x) &= \max(0, v_0(x) - 2^{2^i}) \text{ for all } x \in Y_{i+1} \cup \bar{Y}_{i+1} \\ v'(x) &= v(x) \text{ for all other } x \in X'. \end{aligned}$$

By Property (**),

$$\begin{aligned} v''(\bar{y}_i) &= 2^{2^i} = v_0(\bar{y}_i) \\ v''(y_i) &= 0 = v_0(y_i) \\ v''(x) &= \max(0, v_0(x) - 2^{2^i} - (2^{2^i} - 1) \cdot 2^{2^i}) \\ &= \max(0, v_0(x) - 2^{2^i} \cdot 2^{2^i}) \\ &= \max(0, v_0(x) - 2^{2^{i+1}}) \text{ for all } x \in Y_{i+1} \cup \bar{Y}_{i+1} \\ v''(x) &= v'(x) = v_0(x) \text{ for all other } x \in X'. \end{aligned}$$

□

The procedural-**NB-CM** Rst_i is taking care of resetting counters in $Y_i \cup \bar{Y}_i$. The following lemma states that no counter in $Y_j \cup \bar{Y}_j$, will be increased over 2^{2^j} during this process (for any $1 \leq j \leq n$), and that it resets properly counters in $Y_i \cup \bar{Y}_i$.

Lemma 5.4. Let $0 \leq i \leq n$, and let $v \in \mathbb{N}^{X'}$ satisfying (PreRst1) for Rst_i . If for all $0 \leq j \leq n$, v is j -bounded, then for all $(\ell, v') \in \text{Loc}^{R,i} \times \mathbb{N}^{X'}$ such that $(\ell_{in}^{R,i}, v) \rightsquigarrow^* (\ell, v')$ in Rst_i :

- (a) v' is j -bounded for all $0 \leq j \leq n$;
- (b) the unique configuration such that $(\ell_{in}^{R,i}, v) \rightsquigarrow^* (\ell_{out}^{R,i}, v')$ in Rst_i is defined by $v'(x) = 0$ for all $x \in Y_i \cup \bar{Y}_i$ and $v'(x) = v(x)$ for all $x \notin Y_i \cup \bar{Y}_i$.

Proof. Observe that property (b) is a direct corollary of Proposition 5.2.

The rest of the proof is hence dedicated to prove the part (a) of the statement. For Rst_0 , the property (a) is trivial as only (non-blocking) decrements are performed. We now focus on Rst_{i+1} for $i \in [0, n-1]$.

We will prove (a) along with some other properties: (1) if ℓ is not a state of $\text{TestSwap}_i(\bar{z}_i)$ or $\text{TestSwap}_i(\bar{y}_i)$, then for all $0 \leq j < i$:

- $v'(\mathbf{x}) = 2^{2^j}$ for all $\mathbf{x} \in \bar{Y}_j$,
- $v'(\mathbf{x}) = 0$ for all $\mathbf{x} \in Y_j$,
- $v'(\bar{s}_i) = 2^{2^i}$ and $v'(s_i) = 0$

And (2) if ℓ is not a state of $\text{TestSwap}_i(\bar{z}_i)$ or $\text{TestSwap}_i(\bar{y}_i)$ and if $\ell \neq \ell_1^{R,i+1}$, then $v'(y_i) + v'(\bar{y}_i) = 2^{2^i}$, and if $\ell \neq \ell_3^{R,i+1}$, then $v'(z_i) + v'(\bar{z}_i) = 2^{2^i}$.

Let $0 \leq i < n$, and a valuation $v \in \mathbb{N}^{X'}$ such that:

- for all $j \in [0, i]$, for all $\bar{\mathbf{x}} \in \bar{Y}_j$, $v(\bar{\mathbf{x}}) = 2^{2^j}$ and for all $\mathbf{x} \in Y_j$, $v(\mathbf{x}) = 0$,
- for all $j \in [0, n]$, v is j -bounded.

Let now (ℓ, v') such that $(\ell_{in}^{R,i+1}, v) \rightsquigarrow^* (\ell, v')$ in Rst_{i+1} . We prove the properties by induction on the number of occurrences of $\ell_{in}^{\text{TS},i,z}$ and $\ell_{in}^{\text{TS},i,y}$.

Base case: If there is no occurrence of such state in $(\ell_{in}^{R,i+1}, v) \rightsquigarrow^* (\ell, v')$, then for all $\mathbf{x} \notin Y_i \cup \bar{Y}_i \cup Y_{i+1} \cup \bar{Y}_{i+1}$ it holds, by construction of Rst_{i+1} that $v'(\mathbf{x}) = v(\mathbf{x})$. Furthermore $v'(s_i) = v(s_i)$ and $v'(\bar{s}_{i+1}) = v(\bar{s}_i)$. Hence, as v respects (PreRst1), we get that (1) holds.

We now prove that (2) holds:

- if $\ell = \ell_{in}^{R,i+1}$, then $v'(y_i) = v(y_i)$ and $v'(\bar{y}_i) = v(\bar{y}_i)$, hence as v respects (PreRst1), (2) holds for y_i in that case;
- if $\ell \notin \{\ell_{in}^{R,i+1}, \ell_1^{R,i+1}\}$, then $v'(y_i) = v(y_i) + 1$ and $v'(\bar{y}_i) = v(\bar{y}_i) - 1$, hence $v'(y_i) + v'(\bar{y}_i) = v(y_i) + v(\bar{y}_i)$ and as v respects (PreRst1), (2) holds for y_i in that case;
- if $\ell \in \{\ell_{in}^{R,i+1}, \ell_1^{R,i+1}, \ell_2^{R,i+1}\}$, then $v'(z_i) = v(z_i)$ and $v'(\bar{z}_i) = v(\bar{z}_i)$, hence as v respects (PreRst1), (2) holds for z_i in that case;
- if $\ell \notin \{\ell_{in}^{R,i+1}, \ell_1^{R,i+1}, \ell_2^{R,i+1}, \ell_3^{R,i+1}\}$, then $v'(z_i) = v(z_i) + 1$ and $v'(\bar{z}_i) = v(\bar{z}_i) - 1$, hence $v'(z_i) + v'(\bar{z}_i) = v(z_i) + v(\bar{z}_i)$ and as v respects (PreRst1), (2) holds for z_i in this case;

Hence (2) holds.

As a consequence v' is j -bounded for all $j \in [0, i-1]$. For all $\mathbf{x} \in Y_j \cup \bar{Y}_j \cup \{s_i, \bar{s}_i\}$ and $j \notin \{i, i+1\}$, it holds that $v'(\mathbf{x}) = v(\mathbf{x})$ and so v' is j -bounded for all $j \notin \{i, i+1\}$. Also, for $\mathbf{x} \in \bar{Y}_i \cup Y_{i+1} \cup \bar{Y}_{i+1}$, $v'(\mathbf{x}) \leq v(\mathbf{x})$, and for all $\mathbf{x} \in Y_i$, $v'(\mathbf{x}) \leq v(\mathbf{x}) + 1 = 1$, hence v' is j -bounded for all $j \in [0, n]$.

Induction case: Assume now we proved the properties for k occurrences of $\ell_{in}^{\text{TS},i,z}$ and $\ell_{in}^{\text{TS},i,y}$, and let us prove the claim for $k+1$ such occurrences. Note $\ell_{k+1} \in \{\ell_{in}^{\text{TS},i,z}, \ell_{in}^{\text{TS},i,y}\}$ the last occurrence such that: $(\ell_{in}^{R,i+1}, v) \rightsquigarrow^+ (\ell_k, v_k) \rightsquigarrow (\ell_{k+1}, v_{k+1}) \rightsquigarrow^* (\ell, v')$. By induction hypothesis, v_k is j -bounded for all $j \in [0, n]$, and (ℓ_k, v_k) respects (1) and

(2), and by construction, $(\ell_k, \perp, \ell_{k+1})$ and $\ell_k \neq \ell_1^{R,i+1}$, $\ell_k \neq \ell_3^{R,i+1}$, hence v_{k+1} is j -bounded for all $j \in [0, n]$ and thanks to (1) and (2), it respects (PreTest1), (PreTest2), and (PreTest3) for $\text{TestSwap}_i(\bar{z}_i)$ and $\text{TestSwap}_i(\bar{y}_i)$.

As a consequence, if ℓ is a state of one of the machines such that $(\ell_{k+1}, v_{k+1}) \rightsquigarrow^* (\ell, v')$, then by Proposition 5.1, for all $0 \leq j \leq n$, as v_{k+1} is j -bounded, so is v' , for all $0 \leq j \leq n$ (and so (a) holds). As ℓ is a state of one of the machines, we do not need to check for (1) and (2).

Assume now ℓ not to be a state of one of the two machines, and keep in mind that $(\ell_k, v_k) = (\ell_k, v_{k+1})$ respects (1) and (2). Then, either $\ell = \ell_{out}^{R,i+1}$ or $\ell \in \{\ell_{in}^{R,i+1}, \ell_j^{R,i+1}\}_{j=1,2,3,4,5,6,\dots,10}$.

If $\ell = \ell_{out}^{R,i+1}$, by Proposition 5.2, $v'(x) = v(x)$ for all $x \notin Y_i \cup \bar{Y}_i$ and $v'(\bar{x}_i) = 2^{2^i}$ and $v'(x_i) = 0$ for all $x \in Y_i \cup \bar{Y}_i$. Hence, properties (a), (1) and (2) hold.

If $\ell \in \{\ell_{in}^{R,i+1}, \ell_j^{R,i+1}\}_{j=1,2,3,4,5,6,\dots,10}$, the execution is such that: $(\ell_{k+1}, v_{k+1}) \rightsquigarrow^+ (\ell_{nz,k+1}, v_{k+1}) \rightsquigarrow^* (\ell, v')$, where if $\ell_{k+1} = \ell_{in}^{TS,i,z}$, $\ell_{nz,k+1} = \ell_{nz}^{TS,i,z}$ and otherwise $\ell_{nz,k+1} = \ell_{nz}^{TS,i,y}$. In any case, for all $j \notin \{i, i+1\}$, and $x \in Y_j \cup \bar{Y}_j \cup \{s_i, \bar{s}_i\}$, it holds that $v'(x) = v_{k+1}(x)$, hence (1) holds in (ℓ, v') and v' is j -bounded for all $j < i$ and $j > i+1$. We prove that v' is $i+1$ -bounded. For all $x \in Y_{i+1} \cup \bar{Y}_{i+1}$, $v'(x) \leq v_{k+1}(x)$ by construction of Rst_{i+1} . By induction hypothesis, v_{k+1} is $i+1$ -bounded and so is v' .

We now proceed to prove that (2) holds before proving that v' is i -bounded.

- If $\ell \in \{\ell_{nz,k+1}, \ell_{in}^{R,i+1}\}$ [resp. $\{\ell_{nz,k+1}, \ell_{in}^{R,i+1}, \ell_1^{R,i+1}, \ell_2^{R,i+1}\}$] then $v'(y_i) = v_{k+1}(y_i)$ and $v'(\bar{y}_i) = v_{k+1}(\bar{y}_i)$ [resp. $v'(z_i) = v_{k+1}(z_i)$ and $v'(\bar{z}_i) = v_{k+1}(\bar{z}_i)$]. Hence, by induction hypothesis, $v'(y_i) + v'(\bar{y}_i) = 2^{2^i}$ [resp. $v'(z_i) + v'(\bar{z}_i) = 2^{2^i}$];
- If $\ell \in \{\ell_j^{R,i+1} \mid j \in [2, 10]\}$ [resp. $\{\ell_j^{R,i+1} \mid j \in [4, 10]\}$] then $v'(y_i) = v_{k+1}(y_i) + 1$ and $v'(\bar{y}_i) = v_{k+1}(\bar{y}_i) - 1$ [resp. $v'(z_i) = v_{k+1}(z_i) + 1$ and $v'(\bar{z}_i) = v_{k+1}(\bar{z}_i) - 1$]. Hence, by induction hypothesis, $v'(y_i) + v'(\bar{y}_i) = 2^{2^i} - 1 + 1 = 2^{2^i}$ [resp. $v'(z_i) + v'(\bar{z}_i) = 2^{2^i}$];

Hence (2) holds.

From (1), $v'(s_i) \leq 2^{2^i}$ and $v'(\bar{s}_i) \leq 2^{2^i}$. From (2), if $\ell \neq \ell_1^{R,i+1}$ [resp. $\ell \neq \ell_3^{R,i+1}$] then $v'(y_i) \leq 2^{2^i}$ and $v'(\bar{y}_i) \leq 2^{2^i}$ [resp. $v'(z_i) \leq 2^{2^i}$ and $v'(\bar{z}_i) \leq 2^{2^i}$]. If $\ell = \ell_1^{R,i+1}$ [resp. $\ell = \ell_3^{R,i+1}$], then the run is of the form $(\ell_{nz,k+1}, v_{k+1}) \rightsquigarrow (\ell_{in}^{R,i+1}, v'') \rightsquigarrow (\ell, v')$ [resp. $(\ell_{nz,k+1}, v_{k+1}) \rightsquigarrow^+ (\ell_2^{R,i+1}, v'') \rightsquigarrow (\ell, v')$]. Thanks to (2), $v''(y_i) \leq 2^{2^i}$ and $v''(\bar{y}_i) \leq 2^{2^i}$ [resp. $v''(z_i) \leq 2^{2^i}$ and $v''(\bar{z}_i) \leq 2^{2^i}$]. Hence, by construction, $v'(\bar{y}_i) < v''(\bar{y}_i) \leq 2^{2^i}$ [resp. $v'(\bar{z}_i) < v''(\bar{z}_i) \leq 2^{2^i}$] and $v'(\bar{z}_i) = v''(\bar{y}_i) \leq 2^{2^i}$ [resp. $v'(\bar{z}_i) = v''(\bar{z}_i) \leq 2^{2^i}$]. Hence v' is i -bounded which concludes the proof of (a) and of the lemma. $\square$

Definition 5.5. For all $x \in X'$, we say that x is *initialized* in a valuation v if there exists $i \in [0, n]$ such that:

$$(x \in Y_i \text{ and } v(x) = 0) \text{ or } (x \in \bar{Y}_i \text{ and } v(x) = 2^{2^i}).$$

For $0 \leq i \leq n$, we say that a valuation $v \in \mathbb{N}^{X'}$ is *i-bounded* if for all $x \in Y_i \cup \bar{Y}_i$, $v(x) \leq 2^{2^i}$.

The construction ensures that when one enters Rst_i with a valuation v that is i -bounded, and in which all variables in $\bigcup_{0 \leq j < i} Y_j \cup \bar{Y}_j$ are initialized, the location $\ell_{out}^{R,i}$ is reached with

a valuation v' such that: $v'(x) = 0$ for all $x \in Y_i \cup \bar{Y}_i$ and $v'(x) = v(x)$ for all $x \notin Y_i \cup \bar{Y}_i$. Moreover, if v is j -bounded for all $0 \leq j \leq n$, then any valuation reached during the execution remains j -bounded for all $0 \leq j \leq n$.

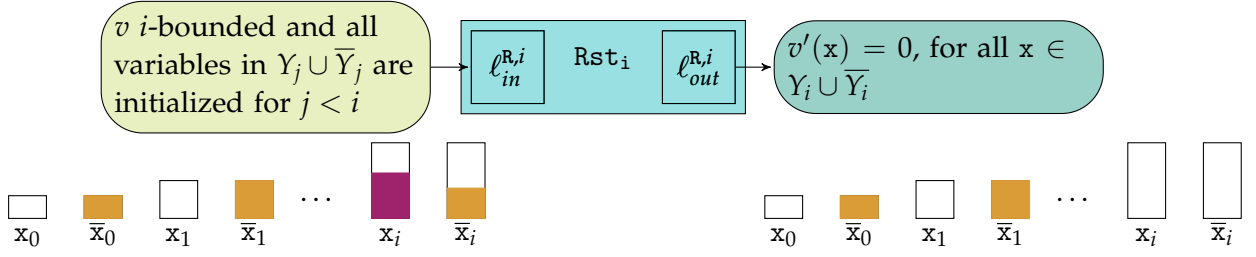


Figure 5.5: Putting together Proposition 5.2 and lemma 5.4: illustration of Rst_i . Each counter x_j [resp. $\bar{x}_j$] belongs to Y_j [resp. $\bar{Y}_j$] for $0 \leq j \leq i$.

Again, to make it more readable and pedagogical, we picture the global effect of the procedural-NB-CM Rst_i in Figure 5.5.

5.2.2.3 Procedural-NB-CM Inc_i .

The properties we seek for Rst_i are ensured whenever the variables in $\bigcup_{0 \leq j < i} Y_j \cup \bar{Y}_j$ are initialized. This is taken care of by a family of procedural-NB-CM introduced in [Lip76; Esp96]. For all $0 \leq i < n$, Inc_i is a procedural-NB-CM with initial location $\ell_{in}^{\text{Inc},i}$, and unique output location $\ell_{out}^{\text{Inc},i}$. They enjoy the following property:

Proposition 5.3 ([Lip76; Esp96]). For all $0 \leq i < n$, for all $v, v' \in \mathbb{N}^{X'}$, $(\ell_{in}^{\text{Inc},i}, v) \rightsquigarrow^* (\ell_{out}^{\text{Inc},i}, v')$ in Inc_i if and only if:

- (PreInc1) for all $0 \leq j < i$, for all $x \in Y_j \cup \bar{Y}_j$, x is initialized;
- (PreInc2) for all $x \in \bar{Y}_i$, $v(x) = 0$,
- (PostInc1) for all $x \in \bar{Y}_i$, $v'(x) = 2^{2^i}$;
- (PostInc2) for all $x \notin \bar{Y}_i$, $v'(x) = v(x)$.

Moreover, if for all $0 \leq j \leq n$, v is j -bounded, then for all (ℓ, v'') such that $(\ell_{in}^{\text{Inc},i}, v) \rightsquigarrow^* (\ell, v'')$ in Inc_i , then v'' is j -bounded for all $0 \leq j \leq n$.

We illustrate the above proposition in Figure 5.6.

5.2.2.4 Procedural-NB-CM RstInc .

Finally, let RstInc be a procedural-NB-CM with initial location ℓ_a and output location ℓ_b , over the set of counters X' and built as an alternation of Rst_i and Inc_i for $0 \leq i < n$, finished by Rst_n . It is depicted in Figure 5.7.

Thanks to the properties of the machines Rst_i and Inc_i , in the output location of each Inc_i machine, the counters in $\bar{Y}_i$ are set to 2^{2^i} , which allow counters in $Y_{i+1} \cup \bar{Y}_{i+1}$ to be set to 0 in the output location of Rst_{i+1} . Hence, in location $\ell_{out}^{\text{Rst},n}$, counters in $Y_n = X$ are set to 0.

The next proposition establishes the correctness of the construction RstInc .

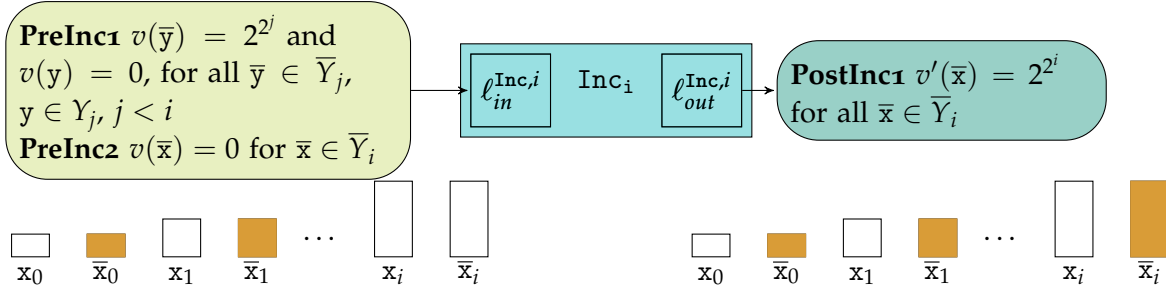


Figure 5.6: Illustration of Proposition 5.3 for procedural-NB-CM Inc_i . Each counter x_j [resp. $\bar{x}_j$] belongs to Y_j [resp. $\bar{Y}_j$] for $0 \leq j \leq i$.

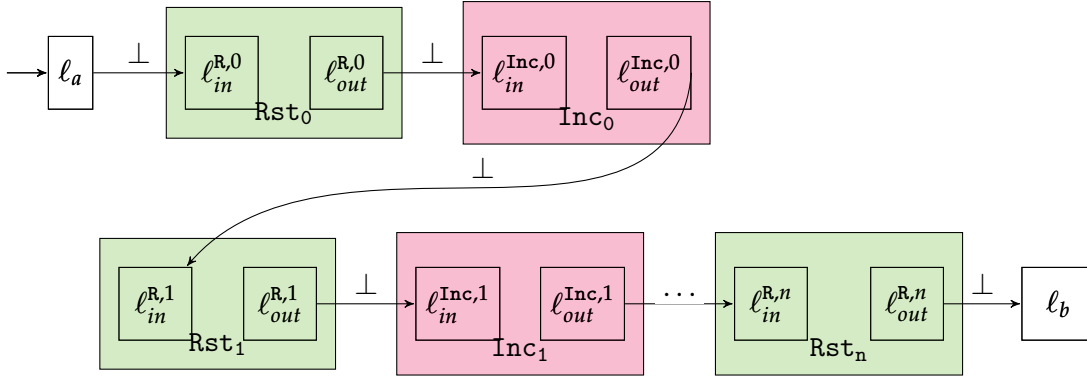


Figure 5.7: RstInc

Proposition 5.4. Let $v \in \mathbb{N}^{X'}$ be a valuation such that for all $i \in [0, n]$ and for all $x \in Y_i \cup \bar{Y}_i$, $v(x) \leq 2^{2^i}$. Then the unique valuation $v' \in \mathbb{N}^{X'}$ such that $(\ell_a, v) \rightsquigarrow^* (\ell_b, v')$ in RstInc is such that all counters in $Y_i \cup \bar{Y}_i$ are initialized for all $i \in [0, n]$. Moreover, for all (ℓ, v'') such that $(\ell_a, v) \rightsquigarrow^* (\ell, v'')$ in RstInc, v'' is i -bounded for all $i \in [0, n]$.

Proof. We can split the execution in

$$\begin{aligned}
 (\ell_a, v) &\rightsquigarrow (\ell_{in}^{R,0}, v) \rightsquigarrow^* (\ell_{out}^{R,0}, v_0) \rightsquigarrow (\ell_{in}^{Inc,0}, v_0) \\
 &\rightsquigarrow^* (\ell_{out}^{Inc,0}, v'_0) \rightsquigarrow (\ell_{in}^{R,1}, v'_0) \rightsquigarrow^* (\ell_{out}^{R,1}, v_1) \rightsquigarrow^* (\ell_{in}^{Inc,n-1}, v_{n-1}) \\
 &\rightsquigarrow^* (\ell_{out}^{Inc,n-1}, v'_{n-1}) \rightsquigarrow (\ell_{in}^{R,n}, v'_{n-1}) \rightsquigarrow^* (\ell_{out}^{R,n}, v_n) \rightsquigarrow (\ell_b, v')
 \end{aligned}$$

with $v' = v_n$ and $v = v'_{-1}$. We show that, for all $0 \leq i \leq n$:

- $P_1(i)$: For all $x \in Y_i \cup \bar{Y}_i$, $v_i(x) = 0$, and for all $x \notin Y_i \cup \bar{Y}_i$, $v_i(x) = v'_{i-1}(x)$.
- $P_2(i)$: For all $0 \leq j < i$, for all $x \in Y_j \cup \bar{Y}_j$, x is initialized in v'_{i-1} and for all other $x \in X'$, $v'_{i-1}(x) = v_{i-1}(x)$.
- $P_3(i)$: For all v'' such that $(\ell_a, v) \rightsquigarrow^* (\ell, v'') \rightsquigarrow^* (\ell_{out}^{R,i}, v_i)$, v'' is j -bounded, for all $0 \leq j \leq n$.

For $k = 0$, Lemma 5.4 implies that for all $x \in Y_0 \cup \bar{Y}_0$, $v_0(x) = 0$, and that for all other $x \in X'$, $v_0(x) = v(x)$. Moreover, for all v'' such that $(\ell_{in}^{R,0}, v) \rightsquigarrow^* (\ell, v'') \rightsquigarrow^* (\ell_{out}^{R,0}, v_0)$,

Lemma 5.4 ensures that v'' is i -bounded, for all $0 \leq i \leq n$. $P_2(0)$ is trivially true.

Let $0 \leq k < n$, and assume that $P_1(k)$, $P_2(k)$ and $P_3(k)$ hold. $P_1(k)$ and $P_2(k)$ ensure that (PreInc1) and (PreInc2) are true. Then Proposition 5.3 ensures that the valuation v'_k obtained at the output of Inc_k is such that for all $x \in \bar{Y}_k$, $v'_k(x) = 2^{2^k}$, and that for all other counter $x \in X'$, $v'_k(x) = v_k(x)$. Since $P_1(k)$ says that $v_k(x) = 0$ for all $x \in Y_k$, we obtain that $x \in Y_k \cup \bar{Y}_k$ is initialized in v'_k and $P_2(k+1)$ holds. Now Proposition 5.3 also ensures that v'_k is i -bounded for all $0 \leq i \leq n$. Then, thanks to $P_2(k+1)$, Lemma 5.4 implies that $v_{k+1}(x) = 0$ for all $x \in Y_{k+1} \cup \bar{Y}_{k+1}$, and that, for all other $x \in X'$, $v_{k+1}(x) = v'_k(x)$. So $P_1(k+1)$ holds. Moreover, by Lemma 5.4, for all v'' such that $(\ell_{out}^{\text{Inc},k}, v'_k) \rightsquigarrow (\ell_{in}^{\text{R},k+1}, v'_k) \rightsquigarrow^* (\ell, v'') \rightsquigarrow^* (\ell_{out}^{\text{R},k+1}, v_{k+1})$, v'' is i -bounded for all $0 \leq i \leq n$. Hence $P_3(k+1)$ holds.

By $P_1(n)$, $v'(x) = 0$ for all $x \in Y_n$, and since $\bar{Y}_n = \emptyset$, $v'(x) = 2^{2^n}$ for all $x \in \bar{Y}_n$. Let $x \notin (Y_n \cup \bar{Y}_n)$. Then $v'(x) = v'_{n-1}(x)$, and by $P_2(n)$, for all $0 \leq i < n$, for all $x \in \bar{Y}_i$, $v'(x) = 2^{2^i}$, and for all $x \in Y_i$, $v'(x) = 0$. By $P_3(n)$, for all (ℓ, v'') such that $(\ell_a, v) \rightsquigarrow^* (\ell, v'')$ in RstInc , for all $0 \leq i \leq n$, v'' is i -bounded. $\square$

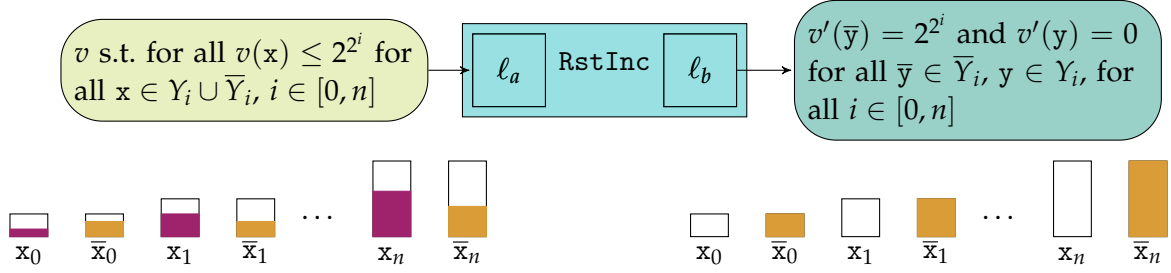


Figure 5.8: Illustration of Proposition 5.4 for procedural-NB-CM RstInc . Each counter x_j [resp. $\bar{x}_j$] belongs to Y_j [resp. $\bar{Y}_j$] for $0 \leq j \leq n$.

We illustrate the above proposition in Figure 5.8.

5.2.2.5 The reduction.

To build the final NB-R-CM NM, we compose the procedural-NB-CM RstInc with the test-free CM CM in the way described by Figure 5.1, which we picture again in Figure 5.9 for the sake of readability. And we add to every location ℓ of RstInc and CM a restore transition

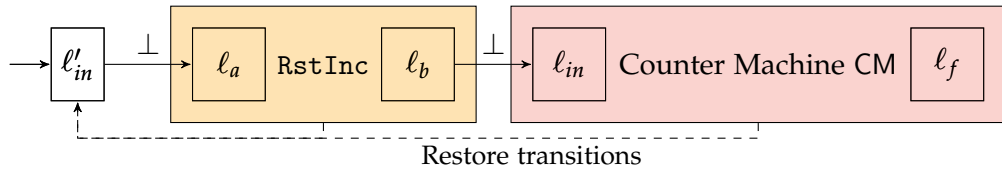


Figure 5.9: The NB-R-CM NM simulating the 2Exp -bounded counter machine CM.

$(\ell, \emptyset, \ell'_{in})$ which is represented in the figure in an abstract way with dashed arrows, for readability's sake.

We are now ready to prove that the reduction is sound and complete. For some subset of counters Y , we will note $v|_Y$ for the valuation v on counters Y , formally, $v|_Y : Y \rightarrow \mathbb{N}$ and is equal to v on its domain.

Lemma 5.5. If there exists $v \in \mathbb{N}^X$ such that $(\ell_{in}, \mathbf{0}_{X'}) \rightsquigarrow_{CM}^* (\ell_f, v)$, then there exists $v' \in \mathbb{N}^{X'}$ such that $(\ell'_{in}, \mathbf{0}_{X'}) \rightsquigarrow_{NM}^* (\ell_f, v')$.

Proof. From Proposition 5.4, we have that $(\ell'_{in}, \mathbf{0}_{X'}) \rightsquigarrow_{NM}^* (\ell_{in}, v_0)$ where v_0 is such that, for all $0 \leq j \leq n$, for all $x \in \bar{Y}_j$, $v_0(x) = 2^{2^j}$ and for all $x \in Y_j$, $v_0(x) = 0$ (recall that $Y_n = X$). By construction of NM, $(\ell_{in}, v_0) \rightsquigarrow_{NM}^* (\ell_f, v')$ with v' defined by: for all $0 \leq i < n$, for all $x \in \bar{Y}_j$, $v'(x) = 2^{2^j}$, for all $x \in Y_j$, $v'(x) = 0$, and, for all $x \in X$, $v'(x) = v(x)$. Note that in this path, there is no restore step. $\square$

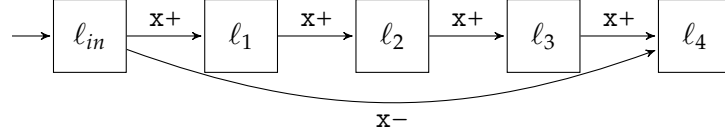
Lemma 5.6. If there exists $v' \in \mathbb{N}^{X'}$ such that $(\ell'_{in}, \mathbf{0}_{X'}) \rightsquigarrow_{NM}^* (\ell_f, v')$, then there exists $v \in \mathbb{N}^X$ such that $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow_{CM}^* (\ell_f, v)$.

Proof. We will write v_0 the valuation such that for all $0 \leq i \leq n$, and for all $x \in \bar{Y}_i$, $v_0(x) = 2^{2^i}$ and for all $x \in Y_i$, $v_0(x) = 0$. Observe that there might be multiple visits of location ℓ_{in} in the execution of NM, because of the restore transitions.

Formally, we show that for all (ℓ_{in}, v) such that $(\ell'_{in}, \mathbf{0}_{X'}) \rightsquigarrow_{NM}^* (\ell_{in}, v)$, we have that $v = v_0$. First let $(\ell'_{in}, w) \rightsquigarrow_{NM}^* (\ell'_{in}, w')$, with $w(x) \leq 2^{2^i}$, and ℓ'_{in} , ℓ_{in} not visited in between. Then for all $0 \leq i \leq n$, for all $x \in Y_i \cup \bar{Y}_i$, $w'(x) \leq 2^{2^i}$. Indeed, let $(\ell, \bar{w})$ be such that $(\ell'_{in}, w) \rightsquigarrow_{NM}^* (\ell, \bar{w}) \rightsquigarrow_{NM} (\ell'_{in}, w')$. By Proposition 5.4, we know that, for all $0 \leq i \leq n$, for all $x \in Y_i \cup \bar{Y}_i$, $\bar{w}(x) \leq 2^{2^i}$ (recall that we only visited locations of RstInc). Since the last transition is a restore transition, we deduce that, for all $0 \leq i \leq n$, for all $x \in Y_i \cup \bar{Y}_i$, $w'(x) = \bar{w}(x) \leq 2^{2^i}$.

- Let $v \in \mathbb{N}^{X'}$ be such that $(\ell'_{in}, \mathbf{0}_{X'}) \rightsquigarrow_{NM}^* (\ell_{in}, v)$, and (ℓ_{in}, v) is the first configuration where ℓ_{in} is visited. The execution is thus of the form $(\ell'_{in}, \mathbf{0}_{X'}) \rightsquigarrow_{NM}^* (\ell'_{in}, w) \rightsquigarrow_{NM}^* (\ell_{in}, v)$, with (ℓ'_{in}, w) the last time ℓ'_{in} is visited. We have stated above that $w(x) \leq 2^{2^i}$. Then, we have that $(\ell'_{in}, \mathbf{0}_{X'}) \rightsquigarrow_{NM}^* (\ell'_{in}, w) \rightsquigarrow_{NM} (\ell_a, w) \rightsquigarrow_{NM}^* (\ell_b, v) \rightsquigarrow_{NM} (\ell_{in}, v)$, and by Proposition 5.4, $v = v_0$.
- Let now $v_k, v_{k+1} \in \mathbb{N}^{X'}$ be such that $(\ell'_{in}, \mathbf{0}_{X'}) \rightsquigarrow_{NM}^* (\ell_{in}, v_k) \rightsquigarrow_{NM}^* (\ell_{in}, v_{k+1})$, and v_k and v_{k+1} are respectively the k^{th} and the $(k+1)^{\text{th}}$ time that ℓ_{in} is visited, for some $k \geq 0$. Assume that $v_k = v_0$. We have $(\ell_{in}, v_k) \rightsquigarrow_{NM}^* (\ell, v) \rightsquigarrow_{NM} (\ell'_{in}, v) \rightsquigarrow_{NM}^* (\ell'_{in}, \bar{v}) \rightsquigarrow_{NM} (\ell_a, \bar{v}) \rightsquigarrow_{NM}^* (\ell_b, v_{k+1}) \rightsquigarrow_{NM} (\ell_{in}, v_{k+1})$. Since the **test-free CM** is 2EXP-bounded, and $v_k = v_0$, we obtain that for all $x \in X = Y_n$, $v(x) \leq 2^{2^n}$. For all $0 \leq i < n$, for all $x \in Y_i \cup \bar{Y}_i$, $v(x) = v_0(x)$, then for all $0 \leq i \leq n$, for all $x \in Y_i \cup \bar{Y}_i$, $v(x) \leq 2^{2^i}$. Then, as proved above, $\bar{v}(x) \leq 2^{2^i}$ for all $0 \leq i \leq n$, for all $x \in Y_i \cup \bar{Y}_i$. By Proposition 5.4, $v_{k+1} = v_0$.

Consider now the execution $(\ell'_{in}, \mathbf{0}_{X'}) \rightsquigarrow_{NM}^* (\ell_{in}, v) \rightsquigarrow_{NM}^* (\ell_f, v')$, where (ℓ_{in}, v) is the last time the location ℓ_{in} is visited. Then, as proved above, $v = v_0$. From the execution $(\ell_{in}, v) \rightsquigarrow_{NM}^* (\ell_f, v')$, we can deduce an execution $(\ell_{in}, v|_X) \rightsquigarrow_{CM}^* (\ell_f, v'|_X)$. Since $v = v_0$ and for all $x \in X = Y_n$, $v(x) = 0$, we can conclude the proof. $\square$

Figure 5.10: A test-free CM CM_{ex} which is 2Exp-bounded.

From [Lip76; Esp96], each procedural machine $\text{TestSwap}_i(x)$ and Inc_i has size at most $C \times n^2$ for some constant C . Hence, observe that NM is of size at most B for some $B \in O(|CM|^3)$ as the procedural-NB-CM RstInc contains procedural-NB-CMs Rst_i , Inc_i for $i \in [0, n]$, and each Rst_i contains two procedural-NB-CMs $\text{TestSwap}_i(x)$. We showed that $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow_{CM}^* (\ell_f, v)$ for some $v \in \mathbb{N}^X$, if and only if $(\ell'_{in}, \mathbf{0}_{X'}) \rightsquigarrow_{NM}^* (\ell_f, v')$ for some $v' \in \mathbb{N}^{X'}$. Using Theorem 5.1, we obtain:

Theorem 5.3. $\text{COVER}[\text{NB-R-CM}]$ is EXPSPACE-hard .

5.2.2.6 An example of the reduction

Example 5.2. Consider the test-free CM $CM_{ex} = (\text{Loc}, X, \Delta, \ell_{in})$ depicted in Figure 5.10. Observe that $X = \{x\}$. Hence we simply write (ℓ, x) for a configuration (ℓ, v_x) with $v_x(x) = x$. Observe that all for any (ℓ, x) such that $(\ell_{in}, 0) \rightsquigarrow^* (\ell, x)$, it holds that $x \leq 4$. Hence, CM_{ex} is bounded by 4, and in particular 2Exp-bounded. Observe that transition $(\ell_{in}, x-, \ell_4)$ cannot be taken as in the initial configuration, counter x has value 0 and the counter's value can not go below 0.

The NB-R-CM built by our construction will work with the set of counters $X' = \{y_0, z_0, s_0\} \cup \{\bar{y}_0, \bar{z}_0, \bar{s}_0\} \cup \{x\}$. We picture the machine in Figure 5.11. For readability's sake, reset transitions are omitted, but keep in mind that from any location, there is a transition going to ℓ'_{in} with no effect on the counters.

First we explain the behavior of a run of the machine simulating CM_{ex} without any resets, i. e. when no reset transition $(\ell, \perp, \ell'_{in})$ is taken. Starting from $(\ell'_{in}, \mathbf{0}_{X'})$, consider the following sequence:

$$(\ell'_{in}, \mathbf{0}_{X'}) \rightsquigarrow (\ell_a, \mathbf{0}_{X'}) \rightsquigarrow (\ell_{in}^{R,0}, \mathbf{0}_{X'}) \rightsquigarrow \dots \rightsquigarrow (\ell_{out}^{R,0}, \mathbf{0}_{X'}) \rightsquigarrow (\ell_{in}^{Inc,0}, \mathbf{0}_{X'})$$

Observe that this sequence is possible as the decrements of Rst_0 are non-blocking, hence counters in $Y_0 \cup \bar{Y}_0$ remain at 0. From Proposition 5.3 (illustrated in Figure 5.6), we get that $(\ell_{in}^{Inc,0}, \mathbf{0}_{X'}) \rightsquigarrow^+ (\ell_{out}^{Inc,0}, v_0)$ with $v_0(\bar{y}_0) = v_0(\bar{z}_0) = v_0(\bar{s}_0) = 2^{2^0} = 2$ and $v_0(x) = v_0(y_0) = v_0(z_0) = v_0(s_0) = 0$.

Hence, we get that:

$$(\ell_{out}^{Inc,0}, v_0) \rightsquigarrow (\ell_{in}^{R,1}, v_0) \rightsquigarrow (\ell_1^{R,1}, v_1) \rightsquigarrow (\ell_2^{R,1}, v_2) \rightsquigarrow (\ell_3^{R,1}, v_3) \rightsquigarrow (\ell_4^{R,1}, v_4)$$

with $v_4(\bar{y}_0) = v_4(\bar{z}_0) = 1$ and $v_4(y_0) = v_4(z_0) = 1$. As $v_4(x) = 0$ we get that $(\ell_4^{R,1}, v_4) \rightsquigarrow (\ell_5^{R,1}, v_4) \rightsquigarrow (\ell_{in}^{TS,0,z}, v_4)$. Observe that $v_4(\bar{s}_0) = 2$ and $v_4(s_0) = 0$. Furthermore, $v_4(z_0) + v_4(\bar{z}_0) = 2$. Hence, from Proposition 5.1 (illustrated in Figure 5.2), and as $v_4(\bar{z}_0) = 1 > 0$, we get that $(\ell_{in}^{TS,0,z}, v_4) \rightsquigarrow^+ (\ell_{nz}^{TS,0,z}, v_4) \rightsquigarrow (\ell_2^{R,1}, v_4)$. Hence,

$$(\ell_2^{R,1}, v_4) \rightsquigarrow (\ell_3^{R,1}, v_5) \rightsquigarrow (\ell_4^{R,1}, v_6) \rightsquigarrow (\ell_5^{R,1}, v_6) \rightsquigarrow (\ell_{in}^{TS,0,z}, v_6)$$

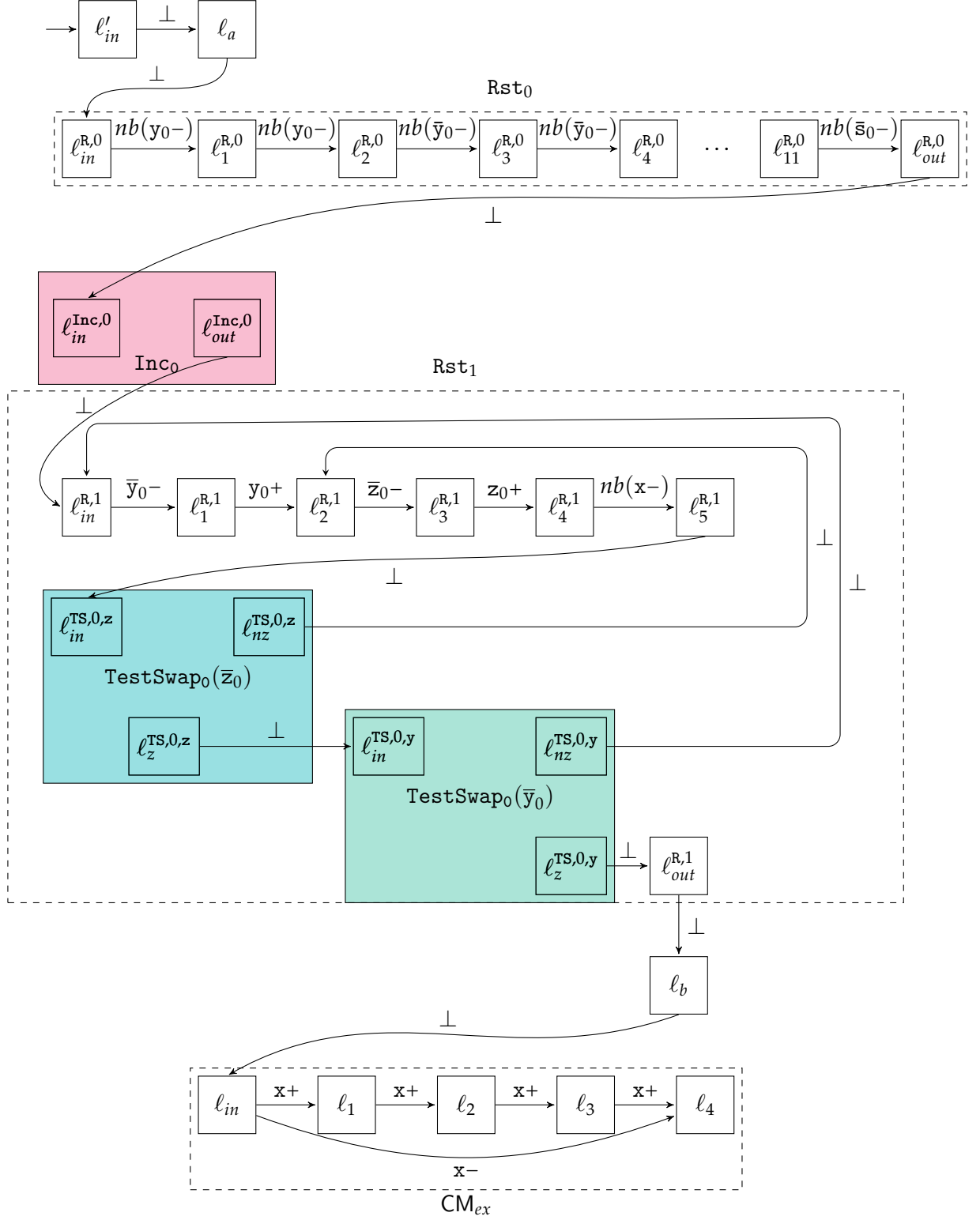


Figure 5.11: The NB-R-CM $NM_{CM_{ex}}$ built from CM_{ex} . For readability's sake, we omit the reset transitions: from each location ℓ , there is a transition going to ℓ'_{in} with no effects on the counters, i.e. $(\ell, \perp, \ell'_{in})$.

This time, $v_6(z_0) = 2$ and $v_6(\bar{z}_0) = 0$. Hence, from Proposition 5.1, we get that $(\ell_{in}^{TS,0,z}, v_6) \rightsquigarrow^+ (\ell_z^{TS,0,z}, v_7) \rightsquigarrow (\ell_{in}^{TS,0,y}, v_7)$ with $v_7(\bar{z}_0) = 2$ and $v_7(z_0) = 0$.

As $v_7(\bar{y}_0) = v_6(\bar{y}_0) = v_4(\bar{y}_0) = 1$, using Proposition 5.1 again we have that:

$$(\ell_{in}^{TS,0,y}, v_7) \rightsquigarrow^+ (\ell_{nz}^{TS,0,y}, v_7) \rightsquigarrow (\ell_{in}^{R,1}, v_7)$$

By iterating the reasoning made from configuration $(\ell_{in}^{R,1}, v_0)$, we reach a configuration $(\ell_{in}^{TS,0,y}, v_8)$ where $v_8(\bar{z}_0) = 2$ and $v_8(z_0) = 0$, and this time, $v_8(\bar{y}_0) = 0$ and $v_8(y_0) = 2$. From Proposition 5.1, we get that $(\ell_{in}^{TS,0,y}, v_8) \rightsquigarrow^+ (\ell_z^{TS,0,y}, v_9)$ where $v_9(\bar{y}_0) = 2$ and $v_9(y_0) = 0$. We get that:

$$(\ell_z^{TS,0,y}, v_9) \rightsquigarrow (\ell_{out}^{R,1}, v_9) \rightsquigarrow (\ell_b, v_9) \rightsquigarrow (\ell_{in}, v_9)$$

As $v_9(x) = 0$ and the rest of the machine (CM_{ex}) works only on counter x , the rest of the run faithfully simulates CM_{ex} . In particular, from (ℓ_{in}, v_9) , transition $(\ell_{in}, x-, \ell_4)$ can not be fired. All the first part of the counter machine can seem a bit complicated for nothing. However, remember that there are reset transitions from any location of the machine. In particular, the following run is possible:

$$(\ell_{in}, v_9) \rightsquigarrow (\ell_1, v_{10}) \rightsquigarrow (\ell'_{in}, v_{10})$$

where $v_{10}(x) = v_9(x) + 1 = 1$. Our construction must ensure that for all v_{11} such that $(\ell'_{in}, v_{10}) \rightsquigarrow^+ (\ell_{in}, v_{11})$, it holds that $v_{11}(x) = 0$. Otherwise transition $(\ell_{in}, x-, \ell_4)$ will become fireable, which would unfaithfully simulate the counter machine and create wrong behaviors. In fact, our construction ensures that the only configuration reachable from (ℓ'_{in}, v_{10}) with location ℓ_{in} is (ℓ_{in}, v_9) . Counter machine CM_{ex} ensures that, in any reachable configuration, counter x 's value is bounded by 4. Hence, for all configuration (ℓ_{in}, v) reached after a reset transition, we know that $v(x) \leq 4$. Our construction ensures that, for all $x_0 \in Y_0 \cup \bar{Y}_0$, $v(x_0) \leq 2$. Hence, the second line of the counter machine $NM_{CM_{ex}}(Rst_0)$ ensures that when we reach $\ell_{out}^{R,0}$, all counters in $Y_0 \cup \bar{Y}_0$ are set to 0 (because of the successive non-blocking decrements). Proposition 5.3 ensures that when one reaches $\ell_{out}^{Inc,0}$, counters in $\bar{Y}_0$ are set to 2 while counters in Y_0 remain unchanged. From there, the first time we reach $\ell_5^{R,1}$, counters $\bar{y}_0$ and $\bar{z}_0$ are equal to 1; and counter x is now *at most* 3 (if it was equal to 4 it has been decremented). At this point, entering in $TestSwap_0(\bar{z}_0)$ leads to $\ell_{nz}^{TS,0,z}$ and hence, the second time $\ell_5^{R,1}$ is reached; counters $\bar{y}_0$ is equal to 1, $\bar{z}_0$ is equal to 0, and counter x is now *at most* 2. This time, $TestSwap_0(\bar{z}_0)$ leads to $\ell_z^{TS,0,z}$ as $\bar{z}_0$ was equal to 0, and values of $\bar{z}_0$ and z_0 are swapped ($\bar{z}_0$ is now equal to 2). As counter $\bar{y}_0$ has value 1, $TestSwap_0(\bar{y}_0)$ leads to $\ell_{nz}^{TS,0,y}$. This leads to $\ell_{in}^{R,1}$. From there we redo the same loop, non-blockingly decrementing x twice again, and reaching $\ell_{in}^{TS,0,y}$ this time with counter y_0 set to 0. As a consequence, $TestSwap_0(\bar{y}_0)$ leads to $\ell_z^{TS,0,y}$ and x 's value is now 0.

This way, we rightfully reset counter x to 0 thanks to this double loop on counters $\bar{y}_0$ and $\bar{z}_0$. For this construction to work, we need to ensure that counters in $Y_0 \cup \bar{Y}_0$ never go above 2. This is ensured by our building of Rst_0, Rst_1 and Lipton's construction of procedural-NB-CMs Inc_0 and $TestSwap_0$.

5.3 RENDEZ-VOUS PROTOCOLS

To make the transition from the counter machines' proofs (previous section) to the rendez-vous networks' proofs (next section) smoother, we present in this section an example of rendez-vous protocol.

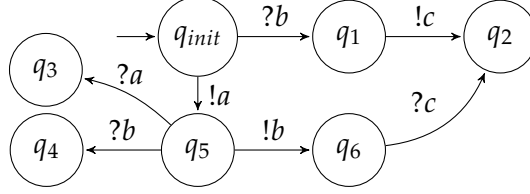


Figure 5.12: Example of a rendez-vous protocol P

We start with a small example. In this chapter, we will use **multiset configurations** as well as regular configurations (written as vectors). Recall that we are working with networks allowing for rendez-vous steps and sending steps. We make the use of some additional notations for readability's sake. For a protocol P and two multiset configurations M and M' we shall write $M \xrightarrow{\text{rdv.}(q,!a,q')}_P M'$ (resp. $M \xrightarrow{\text{s.}(q,!a,q')}_P M'$) whenever $M \xrightarrow{(q,!a,q')}_P M'$ thanks to a **rendez-vous step** (resp. a **sending step**). We remind the reader that a **sending step** happens when no process is on a state from which it can receive the message and involve only one process (the sender). As usual, when the context is clear, we simply write $M \xrightarrow{\text{rdv.}(q,!a,q')} M'$ and $M \xrightarrow{\text{s.}(q,!a,q')} M'$. For an integer $j \in \mathbb{N}$, we write $\xrightarrow{j}$ for the relation $\xrightarrow{\cdot}$ composed j times with itself.

Example 5.3. Figure 5.12 provides an example of a rendez-vous protocol where q_{init} is the initial state. A configuration associated to this protocol is for instance the multiset $\{2 \cdot q_1, 1 \cdot q_4, 1 \cdot q_5\}$ and the following sequence represents an execution:

$$\{2 \cdot q_{\text{init}}\} \xrightarrow{\text{s.}(q_{\text{init}},!a,q_5)} \{q_{\text{init}}, q_5\} \xrightarrow{\text{rdv.}(q_5,!b,q_6)} \{q_1, q_6\} \xrightarrow{\text{rdv.}(q_1,!c,q_2)} \{2 \cdot q_2\}.$$

We have that $\{2 \cdot q_{\text{init}}\} \xrightarrow{3} \{2 \cdot q_2\}$.

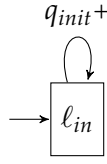
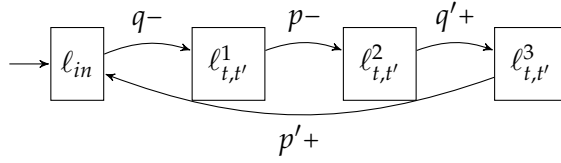
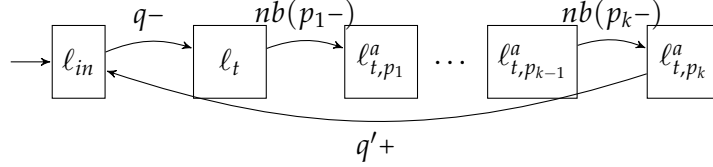
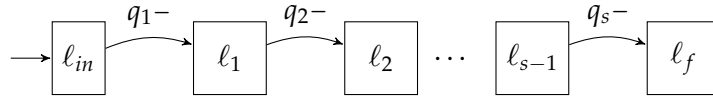
Hence, the rendez-vous protocol together with state q_1 of Figure 5.12 is a positive instance of STATE-COVER. However, this is not the case for SYNCHRO: if an execution brings a process in q_2 , this process cannot be brought afterwards to q_1 . If q_2 is the final state, P is now a positive instance of SYNCHRO (as proved just above). Note that if the target state is q_4 , P is not a positive instance of STATE-COVER anymore. In fact, the only way to reach a configuration with a process in q_4 is to put (at least) two processes in state q_5 as this is the only state from which one process can send the message b . However, this cannot happen, since from an initial configuration, the only available action consists in sending the message a without anyone ready to receive it. Once there is one process in state q_5 , any other attempt to put another process in this state will induce a reception of message a by the process already in q_5 , which will hence leave q_5 . Finally, note that for any $n \in \mathbb{N}$, the configuration $\{n \cdot q_3\}$ is coverable, even if P with q_3 as target state is not a positive instance of SYNCHRO. Indeed, to cover $\{n \cdot q_3\}$, it suffices to have $n + 1$ processes on q_{init} and to make them one by one take the transition $(q_{\text{init}}, !a, q_5)$.

5.4 REDUCTIONS

We are now ready to prove that STATE-COVER^{nb} and CONF-COVER^{nb} problems restricted to RDV-protocol are both ExpSPACE -complete. To this end, we present the following reductions: CONF-COVER^{nb} reduces to $\text{COVER}[\text{NB-CM}]$ and $\text{COVER}[\text{NB-R-CM}]$ reduces to STATE-COVER^{nb} . This will prove that CONF-COVER^{nb} is in ExpSPACE and STATE-COVER^{nb} is ExpSPACE -hard (from Theorem 5.2 and Theorem 5.3). As STATE-COVER^{nb} is an instance of CONF-COVER^{nb} , the two reductions suffice to prove ExpSPACE -completeness for both problems.

In the remainder of this chapter, we will make use of **multiset configurations**.

5.4.1 From Rendez-vous Protocols to NB-CM


 Figure 5.13: Incrementing q_{init} to initialize the number of processes

 Figure 5.14: Transitions for a rendez-vous $t = (q, !a, q'), t' = (p, ?a, p') \in T$.

 Figure 5.15: Transitions for a sending step $t = (q, !a, q') \in T$ and $Q?(a) = \{p_1 \dots p_k\}$

 Figure 5.16: Verification for the coverability of $C_F = \wr q_1 \wr + \wr q_2 \wr + \dots + \wr q_s \wr$

We present in this subsection, a reduction from the configuration coverability problem restricted to **rendez-vous protocols** to the coverability problem for **NB-CM**.

This would prove that the configuration coverability problem is in ExpSPACE . As the state coverability problem is an instance of the configuration coverability problem, it will follow that the state coverability problem is in ExpSPACE as well.

Let $P = (Q, \Sigma, q_{init}, T)$ be a **rendez-vous protocol** and M_f a multiset configuration of P to be covered. We shall also decompose M_f as a sum of multisets $\wr q_1 \wr + \wr q_2 \wr + \dots + \wr q_s \wr$. Observe that there might be $q_i = q_j$ for $i \neq j$. Given $P = (Q, \Sigma, q_{init}, T)$ and $a \in \Sigma$, we define $Q?(a) = \{p \mid \exists p' \text{ s.t. } (p, ?a, p') \in T\}$, i.e. the set of states from which message a can be received.

We build the **NB-CM** $\text{CM} = (\text{Loc}, X, \Delta_b, \Delta_{nb}, \ell_{in})$ where X and Loc are:

- $X = Q$;
- $\text{Loc} = \{\ell_{in}\} \cup \{\ell_{(t,t')}^1, \ell_{(t,t')}^2, \ell_{(t,t')}^3 \mid t = (q, !a, q'), t' = (p, ?a, p') \in T\} \cup \{\ell_t, \ell_{t,p_1}^a, \dots, \ell_{t,p_k}^a \mid t = (q, !a, q') \in T, Q?(a) = \{p_1, \dots, p_k\}\} \cup \{\ell_1 \dots \ell_s\}$.

Sets Δ_b and Δ_{nb} are given in the Figures 5.13 to 5.16. The location one tries to cover in CM is then ℓ_f . A configuration M of P is meant to be represented in CM by (ℓ_{in}, v) , with $v(q) = M(q)$ for all $q \in Q$. The only meaningful location of CM is then ℓ_{in} . The other ones are here to ensure correct updates of the counters when simulating a transition.

Thanks to the transition depicted in Figure 5.13, one can increment q_{init} at will to add any number of processes. A rendez-vous between two processes with transitions $(q, !a, q')$ and $(p, ?a, p')$ is simulated as one could expect (pictured in Figure 5.14): counters q and p are decremented and counters q' and p' are incremented.

The sending steps are handled by the transitions pictured Figure 5.15 (where the only non-blocking decrements of the **NB-CM** occur): to simulate the occurrence of $(q, !a, q')$, the **NB-CM** decrements the value of q by a decrement. It then takes a sequence of non-blocking decrements for each state in $Q?(a)$. The last step of the simulation of a non-blocking rendez-vous is to increment the counter q' . If the **NB-CM** faithfully simulates P , then this loop of non-blocking decrements is taken when the values of the counters in $Q?(a)$ are equal to 0 (i. e. no processes are on states from which they can receive a), and the configuration reached still corresponds to a configuration in P . However, it could be that this loop is taken in CM while some counters in $Q?(a)$ are strictly positive. In this case, a blocking rendez-vous has to be taken in P , e.g. $(q, !a, q')$ and $(p, ?a, p')$ if the counter p in CM is strictly positive. Therefore, the value of the reached configuration (ℓ_{in}, v) and the corresponding configuration C in P will be different: first, $M(p') > v(p')$, since the process in p has moved in the state p' in P when there has been no increment of p' in CM. Furthermore, all the other non-blocking decrements of counters from $Q?(a)$ in CM may have effectively decremented the counters, when in P no other process has left a state of $Q?(a)$. However, this ensures that $M \geq v$. The reduction then guarantees that if (ℓ_{in}, v) is reachable in CM, then a configuration $M \geq v$ is reachable in P . Then, if it is possible to reach a configuration (ℓ_{in}, v) in CM such that $(\ell_{in}, v) \rightsquigarrow^* (\ell_f, v')$, then the corresponding execution in P will reach a configuration $M \geq v$, which hence covers M_f .

Formal proofs are presented below. The completeness proof is the easiest one: one just has to follow the execution of the protocol choosing the right loop in the machine (one simulating a rendez-vous or one simulating a sending step), until reaching (ℓ_{in}, v_{M_f}) where $v_{M_f}(q) \geq M_f(q)$ for all $q \in Q$. Once it is done, it can just take the path to ℓ_f depicted in Figure 5.16. We start by proving the completeness of the reduction.

Lemma 5.7. Let $M_{init} \in \mathcal{M}_{init}$, and $M \geq M_f$. If $M_{init} \rightsquigarrow_p^* M$, then there exists $v \in \mathbb{N}^Q$ such that $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell_f, v)$.

Proof. For all $q \in Q$, we let $v_q(q) = 1$ and $v_q(q') = 0$ for all $q' \in X$ such that $q' \neq q$. Let $n = |M_{init}| = M_{init}(q_{init})$, and let $M_{init} \rightsquigarrow M_1 \dots \rightsquigarrow M_m \rightsquigarrow M$ be the execution leading to M . Then, applying n times the transition $(\ell_{in}, q_{init+}, \ell_{in})$, we get $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow (\ell_{in}, v^1) \rightsquigarrow \dots \rightsquigarrow (\ell_{in}, v^n)$ with $v^n(q_{init}) = n$ and $v^n(x) = 0$ for all $x \neq q_{init}$. We let $v_0 = v^n$.

For simplicity's sake, we write (ℓ, M) for the configuration (ℓ, v_M) with $v_M(q) = M(q)$ for all $q \in Q$. Let $i \geq 0$ and assume that $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell_{in}, M_i)$. We show that $(\ell_{in}, M_i) \rightsquigarrow^* (\ell_{in}, M_{i+1})$.

- If $M_i \xrightarrow{\text{rdv.}(r_1, !a, r'_1)}_P M_{i+1}$ let $t' = (r_2, ?a, r'_2) \in T$ the reception transition realizing the rendez-vous, i.e. the transition such that $(M_i - \{r_1\})(r_2) > 0$ and $M_{i+1} = M_i - \{r_1, r_2\} + \{r'_1, r'_2\}$. Then $(\ell_{in}, M_i) \rightsquigarrow (\ell_{(t,t')}, v_i^1) \rightsquigarrow (\ell_{(t,t')}, v_i^2) \rightsquigarrow (\ell_{(t,t')}, v_i^3) \rightsquigarrow (\ell_{in}, v_i^4)$, with $v_i^1 = M_i - v_{r_1}$, $v_i^2 = v_i^1 - v_{r_2}$, $v_i^3 = v_i^2 + v_{r'_1}$, $v_i^4 = v_i^3 + v_{r'_2}$. Observe that $v_i^4 = M_{i+1}$ and that $(\ell_{in}, M_i) \rightsquigarrow^* (\ell_{in}, M_{i+1})$.
- If $M_i \xrightarrow{\text{s.}(r_1, !a, r'_1)}_P M_{i+1}$, then $M_i(p) = 0$ for all $p \in Q_?(a)$. Let us denote $Q_?(a) = \{p_1, \dots, p_k\}$. Observe that $(\ell_{in}, M_i) \rightsquigarrow (\ell_t, v_i^1) \rightsquigarrow (\ell_{t,r_1}^a, v_i^2) \rightsquigarrow \dots \rightsquigarrow (\ell_{t,r_k}^a, v_i^{k+1}) \rightsquigarrow (\ell_{in}, v_i^{k+2})$ with:

$$\begin{aligned} v_i^1 &= M_i - v_{r_1} \\ v_i^{j+1}(p_j) &= \max(0, v_i^j(p_j) - 1) = 0 = v_i^j(p_j) \quad \text{for all } j+1 \in [2, k+1] \\ v_i^{k+2} &= v_i^{k+1} + v_{r'_1}. \end{aligned}$$

Indeed, as $M_i(p_j) = 0$ for all $j \in [1, k]$, it also holds that $v_i^1(p_j) = 0$. Hence the transitions $(\ell_{t,p_j}^a, nb(p_{j+1}-)), \ell_{t,p_{j+1}}^a)$ do not change the value of the counters.

Hence, $v_i^{k+2} = M_{i+1}$ and $(\ell_{in}, M_i) \rightsquigarrow^* (\ell_{in}, M_{i+1})$.

So we know that $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell_{in}, M)$. Moreover, since $M \geq M_f$, it holds that $M \geq v_{q_1} + v_{q_2} + \dots + v_{q_s}$. Then $(\ell_{in}, M) \rightsquigarrow^* (\ell_f, v)$ with $v = M - (v_{q_1} + v_{q_2} + \dots + v_{q_s})$. $\square$

We now prove soundness: starting from an execution of the machine, we construct a corresponding execution of the protocol. This lemma is more intricate than the previous one, for two main reasons. First, the increment of q_{init} can occur at any point during the machine execution, not necessarily at the beginning. Suppose this increment happens after several rendez-vous and sending steps, and that we have already constructed a protocol execution simulating this sequence. To account for the increment, we build a new protocol execution starting from a larger initial configuration, by adding extra processes in q_{init} . The challenge is that a sending step of a message a might instead be realized as a rendez-vous, if a can be received from a process in q_{init} . Therefore, adding a single process is not sufficient: we must add enough processes (specifically, one for each step in the sequence, plus one more) to ensure that at least one process remains in q_{init} at the end. This guarantees that the protocol configuration still covers the counter valuation after the increment.

Second, suppose the machine executes a loop corresponding to the sending of a message a , while the counter associated with some states in $Q_?(a)$ is nonzero. We show that this does not interfere with the protocol execution. In the protocol, this step is implemented as a rendez-vous (since some state in $Q_?(a)$ is indeed populated). If the rendez-vous uses a transition of the form $(p, ?a, p')$, then counter p is decreased by the non-blocking decrement that occurred during the loop. As a result, the resulting protocol configuration still covers the updated counter valuation.

Lemma 5.8. Let $v \in \mathbb{N}^Q$. If $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell_f, v)$, then there exists $M_{init} \in \mathcal{M}_{init}$, $M \geq M_f$ such that $M_{init} \rightsquigarrow_P^* M_f$.

Proof. Let $(\ell_{in}, v_0), (\ell_{in}, v_1) \dots (\ell_{in}, v_n)$ be the projection of the execution of CM on $\{\ell_{in}\} \times \mathbb{N}^X$. We prove that, for all $i \in [0, n]$, there exists $M_{init} \in \mathcal{M}_{init}$, and $M \geq v_i$ such that $M_{init} \rightsquigarrow_P^* M$. For $i = 0$, we let M_{init} be the multiset $\{q_{init}\}$, and the property is trivially true. Let $0 \leq i < n$, and assume that there exists $M_{init} \in \mathcal{M}_{init}$, $M \geq v_i$ such that $M_{init} \rightsquigarrow_P^* M$.

- If $(\ell_{in}, v_i) \xrightarrow{\delta} (\ell_{in}, v_{i+1})$ with $\delta = (\ell_{in}, q_{init} +, \ell_{in})$, then $v_{i+1} = v_i + v_{q_{init}}$. The execution $M_{init} \rightsquigarrow_P^* M$ built so far cannot be extended as it is, since it might not include enough processes. Hence, we build a new execution from a bigger initial configuration: we put enough additional processes on the initial state so that the same sequence of steps can be performed, and still leave an additional process on the initial state at the end. When the step is a rendez-vous, it has no impact on additional processes on q_{init} , but when it is a sending step of a message a , if the message can be received from the initial state, then one additional process needs to receive the message and exits q_{init} . Hence each step might exits one process on the initial state. Therefore, we need to add as many processes as the number of steps plus one on the initial step, so that after the sequence, there is at least one new process left in the initial state. Let N be the length of the execution between M_{init} and M , such that $M_{init} \rightsquigarrow_P M_1 \rightsquigarrow_P \dots \rightsquigarrow_P M_N = M$, and let $M'_{init} \in \mathcal{M}_{init}$ with $M'_{init}(q_{init}) = M_{init}(q_{init}) + N + 1$. We build, for all $j \in [0, N]$, a configuration M'_j such that $M'_0 \rightsquigarrow_P^j M'_j$, $M'_j \geq M_j$ and $M'_j(q_{init}) > M_j(q_{init}) + N - j$. For $j = 0$, taking $M'_0 = M'_{init}$ is enough. Assume now that, for $0 \leq j < N$, $M'_j \geq M_j$ and that $M'_j(q_{init}) > M_j(q_{init}) + N - j$.

- If $M_j \xrightarrow{\text{rdv.}(r_1, !a, r'_1)} M_{j+1}$, denote $(r_2, ?a, r'_2)$ the reception transition which realizes the rendez-vous, i. e. $(M_j - \{r_1\})(r_2) > 0$ and $M_{j+1} = M_j - \{r_1, r_2\} + \{r'_1, r'_2\}$. As $M'_j \geq M_j$, the rendez-vous is also possible from M'_j . We let $M'_{j+1} = M'_j - \{r_1, r_2\} + \{r'_1, r'_2\}$, and we get that $M'_j \xrightarrow{\text{rdv.}(r_1, !a, r'_1)} M'_{j+1}$. It is easy to see that $M'_{j+1} \geq M_{j+1}$. Moreover, $M'_{j+1}(q_{init}) > M_{j+1}(q_{init}) + N - j > M_{j+1}(q_{init}) + N - j - 1$.
- If $M_j \xrightarrow{\text{s.}(r_1, !a, r'_1)} M_{j+1}$, and no process can receive the message in M'_j , i. e., for all $q \in Q_?(a)$, $(M'_j - \{r_1\})(q) = 0$, we let $M'_{j+1} = M'_j - \{r_1\} + \{r'_1\}$. Observe that $M'_j \xrightarrow{\text{s.}(r_1, !a, r'_1)} M'_{j+1}$. Again, thanks to the induction hypothesis, we get that $M'_{j+1} \geq M_{j+1}$, and $M'_{j+1}(q_{init}) > M_{j+1}(q_{init}) + N - j > M_{j+1}(q_{init}) + N - j - 1$.
- If now $M_j \xrightarrow{\text{s.}(r_1, !a, r'_1)} M_{j+1}$, and a process can receive the message, i. e. there exists $r_2 \in Q_?(a)$ such that $(M'_j - \{r_1\})(r_2) > 0$, we let $(r_2, ?a, r'_2) \in T$, and then $M'_{j+1} = M'_j - \{r_1, r_2\} + \{r'_1, r'_2\}$. Hence, $M'_j \xrightarrow{\text{rdv.}(r_1, !a, r'_1)} M'_{j+1}$. We have that $M'_j(r_2) > M_j(r_2) = 0$, and so $M'_{j+1}(r_2) \in \{0, 1\}$ (1 if $r'_1 = r_2$), and $M'_{j+1}(r_2) \geq M_{j+1}(r_2)$. Furthermore, as $M_{j+1} = M_j - \{r_1\} + \{r'_1\}$, we have that $M'_{j+1}(q) \geq M_{j+1}(q)$ for all other $q \in Q$. Hence $M'_{j+1} \geq M_{j+1}$.

We now justify that $M'_{j+1}(q_{init}) > M_{j+1}(q_{init}) + N - j$. If $q_{init} \notin \{r_2, r'_2\}$ or if $q_{init} = r_2 = r'_2$, we have that $M'_{j+1}(q_{init}) = M'_j(q_{init}) + M_{j+1}(q_{init}) - M_j(q_{init}) > M_j(q_{init}) + N - j + M_{j+1}(q_{init}) - M_j(q_{init}) = M_{j+1}(q_{init}) + N - j$, resulting in $M'_{j+1}(q_{init}) > M_{j+1}(q_{init}) + N - j - 1$. If $q_{init} = r'_2$ and $r_2 \neq r'_2$, then $M'_{j+1}(q_{init}) - M'_j(q_{init}) = M_{j+1}(q_{init}) - M_j(q_{init}) + 1 > M_{j+1}(q_{init}) - M_j(q_{init})$. Hence, thanks to a similar reasoning as above, we get that $M'_{j+1}(q_{init}) > M_{j+1}(q_{init}) + N - j - 1$. Assume now $q_{init} = r_2$, i.e. an extra process in q_{init} received a whereas q_{init} was empty in M_j . Then, $M'_{j+1}(q_{init}) - M'_j(q_{init}) = M_{j+1}(q_{init}) - M_j(q_{init}) - 1$ and $M_j(q_{init}) = 0$. Hence, $M'_{j+1}(q_{init}) = M'_j(q_{init}) + M_{j+1}(q_{init}) - 1 > N - j + M_{j+1}(q_{init}) - 1$, concluding our induction.

So we have built an execution $M'_{init} \rightsquigarrow^* M'_N$ such that $M'_N \geq M_N$ and $M'_N(q_{init}) > M_N(q_{init})$. Hence, $M'_N \geq v_{i+1}$.

- If $(\ell_{in}, v_i) \rightsquigarrow (\ell_{(t,t')}, v_i^1) \rightsquigarrow (\ell_{(t,t')}, v_i^2) \rightsquigarrow (\ell_{(t,t')}, v_i^3) \rightsquigarrow (\ell_{in}, v_{i+1})$, with $t = (r_1, !a, r'_1)$ and $t' = (r_2, ?a, r'_2)$, then

$$v_i^1 = v_i - v_{r_1}, v_i^2 = v_i^1 - v_{r_2}, v_i^3 = v_i^2 + v_{r'_1} \text{ and } v_{i+1} = v_i^3 + v_{r'_2}.$$

Then by induction hypothesis, $M(r_1) \geq 1$, $(M - \wr r_1 \S)(r_2) \geq 1$. We let $M' = M - \wr r_1, r_2 \S + \wr r'_1, r'_2 \S$. We have $M \xrightarrow{\text{rdv.}(r_1, !a, r'_1)} M'$ and $M' \geq v_{i+1}$.

- If $(\ell_{in}, v_i) \rightsquigarrow (\ell_t, v_i^1) \rightsquigarrow (\ell_{t, p_1}^a, v_i^2) \rightsquigarrow \dots \rightsquigarrow (\ell_{t, p_k}^a, v_i^{k+1}) \rightsquigarrow (\ell_{in}, v_{i+1})$ with $t = (r_1, !a, r'_1)$ and $Q?(a) = \{p_1, \dots, p_k\}$, and there is no process on a state in $Q?(a)$ in M , i.e. $(M - \wr r_1 \S)(p) = 0$ for all $p \in Q?(a)$, we then let $M' = M - \wr r_1 \S + \wr r'_1 \S$. We get that $M \xrightarrow{\text{s.}(r_1, !a, r'_1)} M'$. Moreover, $v_i^1 = v_i - v_{r_1}$, and, for all $1 \leq j < k$, we have that $v_i^{j+1}(p_j) = \max(0, v_i^j(p_j) - 1)$. By induction hypothesis, $M \geq v_i$, hence $v_i^j(p) = 0$ for all $p \in Q?(a)$, for all $1 \leq j \leq k + 1$. Hence, $v_{i+1} = v_i^{k+1} + v_{r'_1} = v_i^1 + v_{r'_1}$, and $M' \geq v_{i+1}$.
- If $(\ell_{in}, v_i) \rightsquigarrow (\ell_t, v_i^1) \rightsquigarrow (\ell_{t, p_1}^a, v_i^2) \rightsquigarrow \dots \rightsquigarrow (\ell_{t, p_k}^a, v_i^{k+1}) \rightsquigarrow (\ell_{in}, v_{i+1})$ with $t = (r_1, !a, r'_1)$ and $Q?(a) = \{p_1, \dots, p_k\}$, and there is a process who can receive the message, i.e. $(M - \wr r_1 \S)(p_j) > 0$ for some $p_j \in Q?(a)$, we let $(p_j, ?a, p'_j) \in T$. This case happens when a sending steps is simulated whereas the message could be received by a process. We can then distinguish two cases, when the process receiving the message is counted by v_i , or not.

We define $M' = M - \wr r_1, p_j \S + \wr r'_1, p'_j \S$. Obviously, $M \xrightarrow{\text{rdv.}(r_1, !a, r'_1)} M'$. It remains to show that $M' \geq v_{i+1}$. This is due to the fact that in the NB-R-CM CM, the counter p'_j will not be incremented, unlike $M(p'_j)$. Moreover, if $M(p_j)$ loses a process, then either the non-blocking decrement of p_j has an effect, and hence, the counter also loses a unit, or the counter was at zero already. In both cases, $M'(p_j) \geq v_{i+1}(p_j)$.

Formally, by definition and by induction hypothesis, $M - \wr r_1 \S \geq v_i^1 = v_i - v_{r_1}$. Also, for all $p \in Q?(a)$, either $v_i^1(p) = v_i^{k+1}(p) = 0$, or $v_i^{k+1}(p) = v_i^1(p) - 1$. We will prove that $M' - \wr r'_1 \S \geq v_i^{k+1}$, we recall that $v_{i+1} = v_i^{k+1} + v_{r'_1}$. Let $p \in Q?(a)$

such that $p \neq p_j$. In both cases mentioned above, we get that $v_i^{k+1}(p) \leq v_i^1(p)$ (it is equal if $v_i^1(p) = 0$, and otherwise $v_i^{k+1}(p) = v_i^1(p) - 1$). Intuitively, all states on which a can be received are decremented by one (if they were above zero), but between M and M' only p_j is decremented. Hence, in all states $p \neq p_j$, $M(p) = M'(p)$ and thanks to the induction hypothesis, it is greater (or equal) to $v_i^{k+1}(p)$. We have that:

$$\begin{aligned} (M' - \wr r'_1)(p) &= (M - \wr r_1, p_j \wr + \wr p'_j \wr)(p) \\ &\geq (M - \wr r_1, p_j \wr)(p) \geq (M - \wr r_1 \wr)(p) \\ &\geq v_i^1(p) \geq v_i^{k+1}(p) \end{aligned}$$

We now show that this also holds for p_j . To do this, let us distinguish between two cases:

- if $v_i^1(p_j) = 0$, i.e. the process in p_j is not counted by the counter machine, then $v_i^j(p_j) = 0$ and by definition of non-blocking decrement, we get that $v_i^{k+1}(p_j) = 0$, trivially leading to $(M' - \wr r'_1 \wr)(p_j) \geq v_i^{k+1}(p_j) = 0$.
- if $v_i^1(p_j) > 0$, then, counter associated to p_j has been effectively decremented from the non-blocking decrement's definition, and so: $v_i^j(p_j) = v_i^1(p_j) - 1$. Hence, $v_i^{k+1}(p_j) = v_i^1(p_j) - 1$ and we get that

$$\begin{aligned} (M' - \wr r'_1 \wr)(p_j) &= (M - \wr r_1, p_j \wr + \wr p'_j \wr)(p_j) \\ &\geq (M - \wr r_1, p_j \wr)(p_j) \\ &\geq v_i^1(p_j) - 1 \geq v_i^{k+1}(p_j) \end{aligned}$$

Finally, for $p \in Q \setminus Q_?(a)$, $v_i^1(p) = v_i^{k+1}(p)$. Hence,

$$\begin{aligned} (M' - \wr r'_1 \wr)(p) &= (M - \wr r_1, p_j \wr + \wr p'_j \wr)(p) \\ &\geq (M - \wr r_1, p_j \wr)(p) \geq (M - \wr r_1 \wr)(p) \\ &\geq v_i^1(p) \geq v_i^{k+1}(p) \end{aligned}$$

Hence, $M' - \wr r'_1 \wr \geq v_i^{k+1}$, and so $M' \geq v_i^{k+1} + v_{r'_1} = v_{i+1}$, concluding this case.

From the counter machine's definition, we know that the execution has the following shape: $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell_{in}, v_n) \rightsquigarrow^* (\ell_f, v_f)$ with $v_f = v_n - (v_{q_1} + v_{q_2} + \dots + v_{q_s})$. Thus $v_n > v_{q_1} + v_{q_2} + \dots + v_{q_s}$. We have proved that we can build an execution of P : $M_0 \rightsquigarrow_P^* M_n$ and that $M_n \geq v_{q_1} + v_{q_2} + \dots + v_{q_s}$. Hence $M_n \geq M_f$. $\square$

Remember that the CONF-COVER^{nb} problem is equivalent to ask, given a protocol P and a multiset configuration M , if one can find an execution from an initial (multiset) configuration $M_{init} \in \mathcal{M}_{init}$ to a (multiset) configuration $M' \geq M$ (Proposition 2.1). Hence, thanks to Lemmas 5.7 and 5.8, we get the following lemma.

Theorem 5.4. CONF-COVER^{nb} and STATE-COVER^{nb} over rendez-vous protocols are in EXPSPACE .

5.4.2 From NB-R-CM to Rendez-Vous Protocols

We present a reduction from $\text{COVER}[\text{NB-R-CM}]$ to STATE-COVER^{nb} in rendez-vous protocols, showing that STATE-COVER^{nb} is ExpSPACE-hard . We choose to simulate an NB-R-CM, that is, a NB-CM with restore transitions, for a specific reason. A single process will simulate the counter machine using a mechanism similar to the one described in Example 5.3. It will evolve within a region of the protocol, denoted $P(\text{CM})$, after sending a message L , which can be received from any state within $P(\text{CM})$. This ensures that at most one process at a time evolves within $P(\text{CM})$. However, we cannot prevent another process from kicking out the one currently in $P(\text{CM})$. When this occurs, i. e. when one process replaces another inside $P(\text{CM})$, the new process reaches the initial location without resetting the counters. This behavior mirrors that of restore transitions in NB-R-CMs.

The reduction from $\text{COVER}[\text{NB-R-CM}]$ to STATE-COVER^{nb} in rendez-vous protocols mainly relies on the mechanism that can ensure that at most one process evolves in some given set of states, as explained in Example 5.3 with state q_5 . This will allow to somehow select a “leader” among the processes that will simulate the behavior of the NB-R-CM whereas other processes will simulate the values of the counters. Let $\text{CM} = (\text{Loc}, X, \Delta_b, \Delta_{nb}, \ell_{in})$ a NB-R-CM and $\ell_f \in \text{Loc}$ a final target location. We build the rendez-vous protocol P pictured in Figure 5.18, where $P(\text{CM})$ is the part that will simulate the NB-R-CM CM. The locations $\{1_x \mid x \in X\}$ will allow to encode the values of the different counters during the execution: for a configuration M , $M(1_x)$ will represent the value of the counter x . We give then $P(\text{CM}) = (Q_{\text{CM}}, \Sigma_{\text{CM}}, \ell_{in}, T_{\text{CM}})$ with

$$\begin{aligned} Q_{\text{CM}} &= \text{Loc} \cup \{\ell_\delta \mid \delta \in \Delta_b\} \\ \Sigma_{\text{CM}} &= \{\text{inc}_x, \overline{\text{inc}_x}, \text{dec}_x, \overline{\text{dec}_x}, \text{nbdec}_x \mid x \in X\} \cup \{\tau\} \\ T_{\text{CM}} &= \{(\ell_i, \text{inc}_x, \ell_\delta), (\ell_\delta, \overline{\text{inc}_x}, \ell_j) \mid \delta = (\ell_i, x+, \ell_j) \in \Delta_b\} \\ &\quad \cup \{(\ell_i, \text{dec}_x, \ell_\delta), (\ell_\delta, \overline{\text{dec}_x}, \ell_j) \mid \delta = (\ell_i, x-, \ell_j) \in \Delta_b\} \\ &\quad \cup \{(\ell_i, \text{nbdec}_x, \ell_j) \mid (\ell_i, \text{nb}(x-), \ell_j) \in \Delta_{nb}\} \\ &\quad \cup \{(\ell_i, \tau, \ell_j) \mid (\ell_i, \perp, \ell_j) \in \Delta_b, \ell_j \neq \ell_{in}\} \end{aligned}$$

Parts of $P(\text{CM})$ are depicted in Figure 5.17.

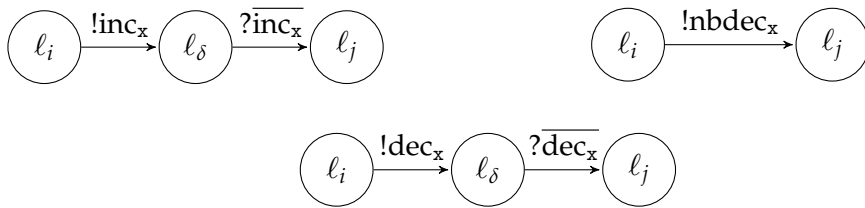


Figure 5.17: Translation of a transition $\delta = (\ell_i, x+, \ell_j)$ (top right figure), $(\ell_i, \text{nb}(x-), \ell_j)$ (top left) and $\delta = (\ell_i, x-, \ell_j)$ (below centered figure) in $P(\text{CM})$.

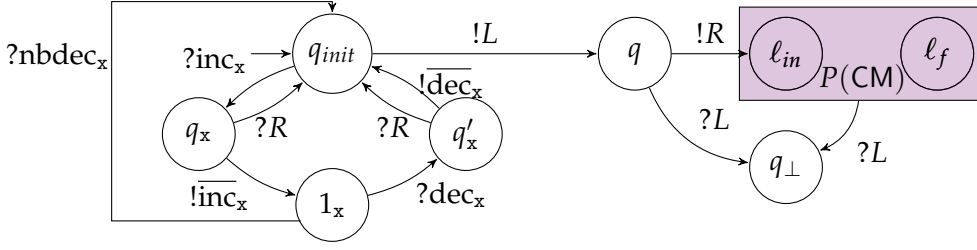


Figure 5.18: The rendez-vous protocol P built from the NB-R-CM CM. Note that there is one gadget with states $\{q_x, q'_x, 1_x\}$ for each counter $x \in X$.

We define $P = (Q, \Sigma, T, q_{init})$ as follows.

$$Q = Q_{CM} \cup \{1_x, q_x, q'_x \mid x \in X\} \cup \{q_{init}, q, q_\perp\}$$

$$\Sigma = \Sigma_{CM} \cup \{L, R\}$$

$$\begin{aligned} T = & T_{CM} \cup \{(q_{init}, !L, q), (q, !R, \ell_{in}), (q, ?L, q_\perp)\} \cup \{(\ell, ?L, q_\perp) \mid \ell \in Q_M\} \\ & \cup \{(q_{init}, ?inc_x, q_x), (q_x, !inc_x, 1_x), (1_x, ?dec_x, q'_x), (q'_x, !dec_x, q_{init}), (1_x, ?nbdec_x, q_{init}) \mid x \in X\} \\ & \cup \{(q_x, ?R, q_{init}), (q'_x, ?R, q_{init}) \mid x \in X\} \end{aligned}$$

With two non-blocking transitions on messages L (for Leader) and R (for Reset) at the beginning, a process can faithfully simulate the NB-R-CM CM, provided that the initial configuration contains enough processes to represent all counter values throughout the execution. Once a process enters the protocol in state ℓ_{in} , any transition of CM can be simulated within the zone $P(CM)$.

Conversely, multiple processes may enter the simulation zone $P(CM)$ during an execution of P , potentially disrupting the simulation. However, the protocol is designed to handle this: any process entering $P(CM)$ begins by sending message L . If another process is already in the simulation zone (i.e., in some state of Q_{CM}), it will move to a deadlock state $q_\perp$ upon receiving this message. Then, the new process sends message R , which is intended to be received by any process in the set $\{q_x, q'_x \mid x \in X\}$, if such a process exists. The construction ensures that at most one process can be in any of the states q_x or q'_x . The role of message R is thus critical: if a new process enters while a simulation was already ongoing, we want the simulation to restart from ℓ_{in} with the correct counter values (represented by the number of processes in 1_x), and the control state q_{init} , but without any intermediate state, such as q_x or q'_x , still being occupied due to an incomplete operation. Therefore, the new process must send message R before beginning the simulation, effectively clearing any processes from q_x or q'_x , given that at most one process occupies a state in $\{q_x, q'_x \mid x \in X\}$. As a result, if we reach a configuration simulating (ℓ, v) in CM, then sending a new process into $P(CM)$ leads to a configuration simulating (ℓ_{in}, v) , faithfully mimicking a restore transition in CM. Hence, every execution of P corresponds to a valid execution of CM.

We provide formal proofs of soundness and completeness below.

Lemma 5.9. For all $v \in \mathbb{N}^d$, if $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell_f, v)$, then there exists $M_{init} \in \mathcal{M}_{init}$, $M_f \geq \{\ell_f\}$ such that $M_{init} \rightsquigarrow^* M_f$.

Proof. For all $x \in X$, we let N_x be the maximal value taken by x in the execution $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell_f, v)$, and $N = \sum_{x \in X} N_x$. Now, we let $M_{init} \in \mathcal{M}_{init}$ be the initial

configuration with $N + 1$ processes. In the initial execution of P that we will build, one of the processes will evolve in the $P(\text{CM})$ part of the protocol, simulating the execution of the NB-R-CM, the others will simulate the values of the counters in the execution. Now, we show by induction on k that, for all $k \geq 0$, if $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^k (\ell, w)$, then $M_{init} \rightsquigarrow^* M$, with:

$$\begin{aligned} M(1_x) &= w(x) && \text{for all } x \in X \\ M(\ell) &= 1 \\ M(q_{init}) &= N - \sum_{x \in X} w(x) \\ M(s) &= 0 && \text{for all other } s \in Q; \end{aligned}$$

where $\rightsquigarrow^k$ is the relation $\rightsquigarrow$ concatenated with itself k times.

First for $k = 0$: the only reachable configuration in 0 step is $(\ell_{in}, \mathbf{0}_X)$, and $M_{init} \xrightarrow{s.(q_{init}, !L, q)} M_0^1$, and $M_0^2(q_{init}) = N$, $M_0^2(\ell_{in}) = 1$, and $M_0^2(s) = 0$ for all other $s \in Q$. So the property holds for $k = 0$. Suppose now that the property holds for $k \geq 0$ and consider $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^k (\ell, w) \xrightarrow{\delta} (\ell', w')$. By induction hypothesis, there exists M respecting the property hereabove, and such that $M_{init} \rightsquigarrow^* M$.

- if $\delta = (\ell, x+, \ell')$, then $M \xrightarrow{\text{rdv.}(\ell, !\text{inc}_x, \ell_\delta)} M_1$ with $M_1 = M - \wr \ell, q_{init} \wr + \wr \ell_\delta, q_x \wr$. Indeed, by induction hypothesis, $M(\ell) = 1 > 0$, and $M(q_{init}) > 0$, otherwise $\sum_{x \in X} w(x) = N$ and $w(x)$ is already the maximal value taken by x so no increment of x could have happened at that point of the execution of CM. We also have $M_1 \xrightarrow{\text{rdv.}(q_x, !\text{inc}_x, 1_x)} M'$, since $M_1(\ell_\delta) > 0$ and $M_1(q_x) > 0$ by construction, and $M' = M_1 - \wr \ell_\delta, q_x \wr + \wr \ell', 1_x \wr$. So $M'(\ell') = 1$, $M'(q_{init}) = N - \sum_{x \in X} w'(x)$ and $M'(1_x) = w'(x)$ for all $x \in X$.
- if $\delta = (\ell, x-, \ell')$, then $M(\ell) = 1 > 0$ and $M(1_x) > 0$ since $w(x) > 0$. Then $M \xrightarrow{\text{rdv.}(\ell, !\text{dec}_x, \ell_\delta)} M_1$ with $M_1 = M - \wr \ell, 1_x \wr + \wr \ell_\delta, q'_x \wr$. Then $M_1 \xrightarrow{\text{rdv.}(q'_x, !\text{dec}_x, q_{init})} M'$, with $M' = M_1 - \wr q'_x, \ell_\delta \wr + \wr q_{init}, \ell' \wr$. So $M'(\ell') = 1$, $M'(1_x) = M(1_x) - 1$, $M'(q_{init}) = M(q_{init}) + 1$.
- if $\delta = (\ell, nb(x-), \ell')$ and $w(x) > 0$ then $M \xrightarrow{\text{rdv.}(\ell, !nb\text{dec}_x, \ell')} M'$, and $M' = M - \wr \ell, 1_x \wr + \wr \ell', q_{init} \wr$ and the case is proved.
- if $\delta = (\ell, nb(x-), \ell')$ and $w(x) = 0$ then by induction hypothesis, $M(1_x) = 0$ and $M \xrightarrow{s.(\ell, !nb\text{dec}_x, \ell')} M'$, with $M' = M - \wr \ell \wr + \wr \ell' \wr$. Then, $M'(1_x) = 0 = w'(x)$, and $M'(\ell') = 1$.
- if $\delta = (\ell, \perp, \ell')$, then $M \xrightarrow{s.(\ell, !\tau, \ell')} M'$, with $M' = M - \wr \ell \wr + \wr \ell' \wr$. This includes the restore transitions.

Then $M_0 \rightsquigarrow^* M$ with $M(\ell_f) = 1$ and $M \geq \wr \ell_f \wr$.

□

We now state the soundness lemma.

Lemma 5.10. Let $M_{init} \in \mathcal{M}_{init}$, $M_f \geq \{ \ell_f \}$ such that $M_{init} \rightarrow^* M_f$, then $(\ell_0, \mathbf{0}_X) \rightsquigarrow^* (\ell_f, v)$ for some $v \in \mathbb{N}^X$.

Proof. We start by making the following observation.

Claim 5.4. Let $M_{init} \in \mathcal{M}_{init}$. For all $M \in \mathcal{M}$ such that $M_{init} \rightarrow^+ M$, we have $\sum_{p \in \{q\} \cup Q_{CM}} M(p) = 1$.

Indeed, from the protocol's construction, once a process is on $\{q\} \cup Q_{CM}$, any new process trying to join will exit the first one, preserving the constant number 1 of processes on $\{q\} \cup Q_{CM}$. Furthermore, the only possible action from M_{init} puts a process on q .

Let $M_0 \rightarrow M_1 \rightarrow \dots \rightarrow M_n = M_f$. Now, thanks to Claim 5.4, for all $1 \leq i \leq n$, we can denote $\text{leader}(M_i)$ the unique state s in $\{q\} \cup Q_{CM}$ such that $M_i(s) = 1$. In particular, observe that $\text{leader}(M_n) = \ell_f$. We say that a configuration M is CM-compatible if $\text{leader}(M) \in \text{Loc}$. For any CM-compatible configuration $M \in \mathcal{M}$, we define the configuration of the NB-R-CM $\text{confcm}(M) = (\text{leader}(M), v)$ with $v(x) = M(1_x)$ for all $x \in X$.

We let $M_{i_1} \dots M_{i_k}$ be the projection of $M_0 M_1 \dots M_n$ onto the CM-compatible configurations.

We show by induction on $j \in [1, k]$ that:

- $\text{Pr}(j)$: $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* \text{confcm}(M_{i_j})$, and $\sum_{x \in X} M_{i_j}(q_x) + M_{i_j}(q'_x) = 0$. Moreover, for all M such that $M_0 \rightarrow^* M \rightarrow M_{i_j}$, it holds that $\sum_{x \in X} M(q_x) + M(q'_x) \leq 1$.

By construction of the protocol, $M_0 \xrightarrow{s.(q_{init}, !L, q)} M_1 \xrightarrow{\text{rdv.}(q_{init}, !L, q)} M_2 \xrightarrow{s.(q, !R, \ell_{in})} M_{i_1}$ for some $k \in \mathbb{N}$. So $\text{confcm}(M_{i_1}) = (\ell_{in}, \mathbf{0}_X)$, and for all M such that $M_0 \rightarrow^* M \rightarrow M_{i_1}$, $\sum_{x \in X} M(q_x) + M(q'_x) = 0$, so $\text{Pr}(0)$ holds.

Let now $1 \leq j < k$, and suppose that $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* \text{confcm}(M_{i_j})$, and $\sum_{x \in X} M_{i_j}(q_x) + M_{i_j}(q'_x) = 0$. We know that $M_{i_j} \rightarrow^+ M_{i_{j+1}}$ by construction.

- If there is no $M \in \mathcal{M}$ such that $M(q) = 1$ and $M_{i_j} \rightarrow^+ M \rightarrow^* M_{i_{j+1}}$, the only possible transitions from M_{i_j} are in T_M . Let $\text{confcm}(M_{i_j}) = (\ell, v)$.
 - if $M_{i_j} \xrightarrow{\text{rdv.}(\ell, !\text{inc}_x, \ell_\delta)} M$ then $M = M_{i_j} - \{ \ell, q_{init} \} + \{ \ell_\delta, q_x \}$. Let $\delta = (\ell, x+, \ell') \in \Delta_b$. We have that $\sum_{x \in X} M(q_x) + M(q'_x) = 1$. Note that the message inc_x is necessarily received by some process, otherwise $M(q_x) = 0$ and M has no successor, which is in contradiction with the fact the execution reaches M_f . Moreover, the only possible successor configuration is $M \xrightarrow{\text{rdv.}(q_x, !\text{inc}_x, 1_x)} M_{i_{j+1}}$, with $M_{i_{j+1}} = M - \{ q_x, \ell_\delta \} + \{ 1_x, \ell' \}$. Hence, obviously, $\text{confcm}(M_{i_j}) \rightsquigarrow \text{confcm}(M_{i_{j+1}})$.
 - if $M_{i_j} \xrightarrow{\text{rdv.}(\ell, !\text{dec}_x, \ell_\delta)} M$ then $M = M_{i_j} - \{ \ell, 1_x \} + \{ \ell_\delta, q'_x \}$. Let $\delta = (\ell, x-, \ell') \in \Delta_b$. We have that $\sum_{x \in X} M(q_x) + M(q'_x) = 1$. Note that the message dec_x is necessarily received by some process, otherwise $M(q'_x) = 0$ and M has no successor, which is in contradiction with the fact the execution reaches M_f . Besides, $M_{i_j}(1_x) > 0$ hence $v(x) > 0$. Moreover, the only possible successor configuration is $M \xrightarrow{\text{rdv.}(q'_x, !\text{dec}_x, q_{init})} M_{i_{j+1}}$, with $M_{i_{j+1}} = M - \{ q'_x, \ell_\delta \} + \{ q_{init}, \ell' \}$. Hence, obviously, $\text{confcm}(M_{i_j}) \rightsquigarrow \text{confcm}(M_{i_{j+1}})$.

- if $M_{i_j} \xrightarrow{\text{rdv.}(\ell, \text{nbdec}_x, \ell')} M$ then $M = M_{i_{j+1}} = M_{i_j} - \{ \ell, 1_x \} + \{ \ell', q_{\text{init}} \}$. Observe that $\sum_{x \in X} M_{i_j}(q_x) + M_{i_j}(q'_x) = \sum_{x \in X} M_{i_{j+1}}(q_x) + M_{i_{j+1}}(q'_x) = 0$. Besides, $M_{i_j}(1_x) > 0$ hence $v(x) > 0$. Hence, obviously, $\text{confcm}(M_{i_j}) \rightsquigarrow \text{confcm}(M_{i_{j+1}})$.
- if $M_{i_j} \xrightarrow{\text{s.}(\ell, \text{nbdec}_x, \ell')} M$ then $M_{i_{j+1}} = M = M_{i_j} - \{ \ell \} + \{ \ell' \}$. Observe that $\sum_{x \in X} M(q_x) + M(q'_x) = 0$. Besides, $M_{i_j}(1_x) = 0$ hence $v(x) = 0$. Hence, obviously, $\text{confcm}(M_{i_j}) \rightsquigarrow \text{confcm}(M_{i_{j+1}})$.
- if $M_{i_j} \xrightarrow{\text{s.}(\ell, \tau, \ell')} M$ then $M = M_{i_{j+1}} = M_{i_j} - \{ \ell \} + \{ \ell' \}$. Observe that $\sum_{x \in X} M(q_x) + M(q'_x) = 0$. Besides, $M_{i_j}(1_x) = M_{i_{j+1}}(1_x)$ for all $x \in X$. Hence, obviously, $\text{confcm}(M_{i_j}) \rightsquigarrow \text{confcm}(M_{i_{j+1}})$.
- Otherwise, let M be the first configuration such that $M(q) = 1$ and $M_{i_j} \xrightarrow{+} M \xrightarrow{*} M_{i_{j+1}}$. The transition leading to M is necessarily a transition where the message L has been sent. Remember also that by induction hypothesis, $\sum_{x \in X} M_{i_j}(q_x) + M_{i_j}(q'_x) = 0$.
 - if $M_{i_j} \xrightarrow{\text{rdv.}(q_{\text{init}}, L, q)} M$, then $M(q) = 1$, and by induction hypothesis, $\sum_{x \in X} M(q_x) + M(q'_x) = 0$. Then the only possible successor configuration is $M \xrightarrow{\text{s.}(q, R, \ell_{\text{in}})} M_{i_{j+1}}$, with $\sum_{x \in X} M_{i_{j+1}}(q_x) + M_{i_{j+1}}(q'_x) = 0$, and $\text{confcm}(M_{i_{j+1}}) = (\ell_{\text{in}}, v)$, so $\text{confcm}(M_{i_j}) \rightsquigarrow \text{confcm}(M_{i_{j+1}})$, by a restore transition.
 - if $M_{i_j} \xrightarrow{\text{rdv.}(\ell, \text{inc}_x, \ell_\delta)} M_1 \xrightarrow{\text{rdv.}(q_{\text{init}}, L, q)} M$ then $M_1 = M_{i_j} - \{ \ell, q_{\text{init}} \} + \{ \ell_\delta, q_x \}$. We let $\delta = (\ell, x, \ell') \in \Delta_b$. Observe that $\sum_{x \in X} M_1(q_x) + M_1(q'_x) = 1$. Now, $M = M_1 - \{ \ell_\delta, q_{\text{init}} \} + \{ q_\perp, q \}$, so $M(q) = 1 = M(q_x)$, and $\sum_{x \in X} M(q_x) + M(q'_x) = 1$.
 - * If $M \xrightarrow{\text{rdv.}(q, R, \ell_{\text{in}})} M_{i_{j+1}}$, then $M_{i_{j+1}} = M - \{ q, q_x \} + \{ \ell_{\text{in}}, q_{\text{init}} \}$, then $\sum_{x \in X} M_{i_{j+1}}(q_x) + M_{i_{j+1}}(q'_x) = 0$ and $\text{confcm}(M_{i_{j+1}}) = (\ell_{\text{in}}, v)$, hence $\text{confcm}(M_{i_j}) \rightsquigarrow \text{confcm}(M_{i_{j+1}})$ by a restore transition.
 - * Otherwise, as $M(q_x) = 1$ and $M(p) = 0$ for all $p \in \{ q_y, q'_y \mid y \in X \} \setminus \{ q_x \}$, it must be that $M \xrightarrow{\text{s.}(q_x, \overline{\text{inc}_x}, 1_x)} M'$, with $M' = M - \{ q_x \} + \{ 1_x \}$. Here, $\sum_{x \in X} M'(q_x) + M'(q'_x) = 0$. However, $\text{leader}(M') = \{ q \}$ so M' is not CM-compatible. The only possible transition from M' is now $M' \xrightarrow{\text{s.}(q, R, \ell_{\text{in}})} M_{i_{j+1}}$ with $M_{i_{j+1}} = M' - \{ q \} + \{ \ell_{\text{in}} \}$. Hence, $M_{i_{j+1}}(1_x) = M'(1_x) = M_{i_j}(1_x) + 1 = v(x) + 1$, and $M_{i_{j+1}}(1_y) = M'(1_y) = M_{i_j}(1_y) = v(y)$ for all $y \neq x$. So $\text{confcm}(M_{i_j}) = (\ell, v) \rightsquigarrow (\ell', v + v_x) \rightsquigarrow (\ell_{\text{in}}, v + v_x) = \text{confcm}(M_{i_{j+1}})$, the last step being a restore transition. Finally, $\sum_{x \in X} M_{i_{j+1}}(q_x) + M_{i_{j+1}}(q'_x) = 0$.
 - if $M_{i_j} \xrightarrow{\text{rdv.}(\ell, \text{dec}_x, \ell_\delta)} M_1 \xrightarrow{\text{rdv.}(q_{\text{init}}, L, q)} M$, then $M_1 = M_{i_j} - \{ \ell, 1_x \} + \{ \ell_\delta, q'_x \}$. We let $\delta = (\ell, x, \ell') \in \Delta_b$. Observe that $\sum_{x \in X} M_1(q_x) + M_1(q'_x) = 1$. Now, $M = M_1 - \{ \ell_\delta, q_{\text{init}} \} + \{ q_\perp, q \}$, so $M(q) = 1 = M(q'_x)$, and $\sum_{x \in X} M(q_x) + M(q'_x) = 1$. Again, two transitions are available:

- * If $M \xrightarrow{\text{rdv.}(q, !R, \ell_{in})} M_{i_{j+1}}$, then $M_{i_{j+1}} = M - \wr q, q'_x \wr + \wr \ell_{in}, q_{init} \wr$, and $\sum_{x \in X} M_{i_{j+1}}(q_x) + M_{i_{j+1}}(q'_x) = 0$ and $\text{confcM}(M_{i_{j+1}}) = (\ell_{in}, v)$, hence $\text{confcM}(M_{i_j}) \rightsquigarrow \text{confcM}(M_{i_{j+1}})$ by a restore transition.
- * Otherwise, as $M(q'_x) = 1$ and $M(p) = 0$ for all $p \in \{q_y, q'_y \mid y \in X\} \setminus \{q'_x\}$, it must be that $M \xrightarrow{s.(q'_x, !\text{dec}_x, q_{init})} M'$, with $M' = M - \wr q'_x \wr + \wr q_{init} \wr$. Here, $\sum_{x \in X} M'(q_x) + M'(q'_x) = 0$. However, $\text{leader}(M') = \{q\}$ so M' is not CM-compatible. The only possible transition from M' is now $M' \xrightarrow{s.(q, !R, \ell_{in})} M_{i_{j+1}}$ with $M_{i_{j+1}} = M' - \wr q \wr + \wr \ell_{in} \wr$. Hence, $M_{i_{j+1}}(1_x) = M'(1_x) = M_{i_j}(1_x) - 1 = v(x) - 1$, and $M_{i_{j+1}}(1_y) = M'(1_y) = M_{i_j}(1_y) = v(y)$ for all $y \neq x$. So $\text{confcM}(M_{i_j}) = (\ell, v) \rightsquigarrow (\ell', v - v_x) \rightsquigarrow (\ell_{in}, v + v_x) = \text{confcM}(M_{i_{j+1}})$, the last step being a restore transition. Finally, $\sum_{x \in X} M_{i_{j+1}}(q_x) + M_{i_{j+1}}(q'_x) = 0$.
- If $M_{i_j} \xrightarrow{s.(\ell, !\text{inc}_x, 1_\delta)} M_1$ then, it follows that $M_{i_j}(q_{init}) = 0$. In that case, let $\delta = (\ell, x+, \ell') \in \Delta_b$, and $M_1 = M_{i_j} - \wr \ell \wr + \wr \ell_\delta \wr$. Since, by induction hypothesis, $M_1(q_x) = M_{i_j}(x) = 0$, the only possible transition from M_1 would be $M_1 \xrightarrow{\text{rdv.}(q_{init}, !L, q)} M_{i_{j+1}}$. However, $M_{i_j}(q_{init}) = M_1(q_{init}) = 0$, so this transition is not possible, and M_1 is a deadlock configuration, a contradiction with the hypothesis that $M_{i_j} \rightsquigarrow M_{i_{j+1}}$.
- If $M_{i_j} \xrightarrow{s.(\ell, !\text{dec}_x, \ell_\delta)} M_1$ then it means that $M_{i_j}(1_x) = 0$. In that case, let $\delta = (\ell, x-, \ell') \in \Delta_b$, and $M_1 = M_{i_j} - \wr \ell \wr + \wr \ell_\delta \wr$. Since, by induction hypothesis, $\sum_{x \in X} M_1(q_x) + M_1(q'_x) = \sum_{x \in X} M_{i_j}(q_x) + M_{i_j}(q'_x) = 0$, the only possible transition from M_1 is $M_1 \xrightarrow{\text{rdv.}(q_{init}, !L, q)} M$, with $M = M_1 - \wr q_{init}, \ell_\delta \wr + \wr q, q_\perp \wr$. Again, $\sum_{x \in X} M(q_x) + M(q'_x) = 0$, and $M(\ell) = 0$ for all $\ell \in Q_M$, so the only possible transition is $M \xrightarrow{s.(q, !R, \ell_{in})} M_{i_{j+1}}$. Observe that $M_{i_{j+1}}$ is CM-compatible, with $M_{i_{j+1}}(\ell_{in}) = 1$, and $M_{i_{j+1}}(1_x) = M_{i_j}(1_x)$ for all $x \in X$. Hence $\text{confcM}(M_{i_{j+1}}) = (\ell_{in}, v)$, and $\text{confcM}(M_{i_j}) \rightsquigarrow \text{confcM}(M_{i_{j+1}})$, thanks to a restore transition of CM.
- If $M_{i_j} \xrightarrow{(\ell, !\text{nbdec}_x, \ell')} M_{i_{j+1}}$ or $M_{i_j} \xrightarrow{(\ell, !\tau, \ell')} M_{i_{j+1}}$, then $M_{i_{j+1}}$ is CM-compatible and $i_{j+1} = i_j + 1$, and $M_{i_{j+1}}(q) = 0$, contradicting our case hypothesis.

Hence $\text{Pr}(j+1)$ holds and the property holds for all j by induction. We then have, by $\text{Pr}(k)$, that $(\ell_{in}, 0_X) \rightsquigarrow^* \text{confcM}(C_{i_k})$, with M_{i_k} CM-compatible and such that $M_{i_k} \rightsquigarrow^* M_f$, and M_{i_k} is the last CM-compatible configuration. Then, by definition of a CM-compatible configuration, $M_{i_k} = M_f$, and $\text{confcM}(M_{i_k}) = (\ell_f, v)$ for some $v \in \mathbb{N}^X$. $\square$

Thanks to Lemmas 5.9 and 5.10, together with Theorem 5.4 we get the main theorem of this section.

Theorem 5.5. STATE-COVER^{nb} and CONF-COVER^{nb} over rendez-vous protocols are EXPSpace complete.

5.5 CONCLUSIONS

In this chapter, we studied the impact of non-blocking rendez-vous communication on the coverability problems. We showed that, despite being more permissive than blocking rendez-vous, this communication primitive leads to a significantly higher verification complexity. We present the resulting table in Table 5.1. It thus appears to occupy an intermediate position between blocking rendez-vous and broadcast communication. In the next chapter, we will examine a restricted setting in which this complexity can be partially mitigated, making certain verification problems more tractable.

	STATE-COVER*	CONF-COVER*	SYNCHRO*	REP-COVER*
Blocking Rdv Networks ($\star = b$ and RDV-protocol)	in PTIME [GS92; BER21]			
Rdv Networks ($\star = nb$ and RDV-protocol)	Ackermann-complete [EK03; ARZ15; EFM99; SS13]		Undecidable (Chapter 4)	Undecidable [EK03; EFM99]
Broadcast Networks (B-protocol)				
Blocking Networks ($\star = nb$)				
Networks ($\star = nb$)				
	ExpSPACE-complete (this chapter)			

Table 5.1: Resulting table.

COVERABILITY IN WAIT-ONLY NETWORKS

6.1 INTRODUCING WAIT-ONLY PROTOCOLS

The previous chapter provided contrasting results (see table in Section 5.5). Restricting protocols to rendez-vous protocols significantly simplifies verification in terms of coverability problems, reducing the complexity from Ackermann-completeness (with broadcasts) to ExpSPACE-completeness. However, ExpSPACE remains a very high complexity class.

In this chapter, we introduce a further restriction on protocols, called *Wait-Only protocols*. In a Wait-Only protocol, no state allows both receiving and sending messages (either via broadcast or sending). This restriction is formally defined as follows.

Definition 6.1 (Wait-Only Protocols). A protocol $P = (Q, \Sigma, q_{init}, T)$ is **Wait-Only** if, for all states $q \in Q$, either:

- there is no transition $(q, \alpha, q') \in T$ with $\alpha \in ?\Sigma$ (i.e., from q , one can never receive messages), or
- there is no transition $(q, \alpha, q') \in T$ with $\alpha \in !!\Sigma \cup !\Sigma$ (i.e., from q , one can never send message).

A state satisfying the first condition (or both) is called an *action state*, while a state satisfying the second condition is a *waiting state*.

This restriction is natural when considering our initial inspiration: Java multithreaded programming. A thread typically either waits for a `notify()` or `notifyAll()` signal to enter a critical section, or is actively processing tasks. However, a thread does not simultaneously execute actions while waiting for messages.

We will see that this restriction simplifies coverability problems, even in the presence of broadcast transitions. The key reason behind this complexity reduction is the *copypaste property*: if an action state and a waiting state are separately coverable, then they are also coverable simultaneously. More precisely, if a configuration M is coverable and an action state q is coverable, then the configuration $M + \{q\}$ is coverable. Intuitively, once a process reaches an action state q , any external events (i.e., events not triggered by it) have no impact on it, as the process does not receive messages.

We formally establish this fundamental property in Section 6.2 and extensively use it in our analysis of Wait-Only protocols. The remainder of this chapter is structured as follows:

- In Section 6.3, we prove that the *state coverability problem* is solvable in PTIME, and that this complexity bound is tight (PTIME-complete).
- In Section 6.4, we establish a distinction between state coverability and configuration coverability, proving that the latter is PSPACE-complete.

- Finally, in Section 6.5, we show that when restricting to Wait-Only rendez-vous protocols (i.e., without broadcast transitions), the configuration coverability problem becomes PTIME-complete.

The complexity gap between state coverability and configuration coverability might seem surprising as we have seen in Chapter 2 that, in the general case, configuration coverability reduces to state coverability. We will explain in Section 6.4 why this is no longer the case with Wait-Only protocols.

6.1.1 Synchronization Remains Undecidable

A major limitation of Wait-Only protocols is that the SYNCHRO problem remains undecidable.

In Section 4.2, we proved that SYNCHRO is undecidable for rendez-vous networks by reducing the COVER[CM] problem (known to be undecidable Theorem 3.1 [Min67]) to SYNCHRO. The reduction constructs a protocol that encodes a counter machine $CM = (Loc, \ell_{in}, X, \Delta)$. As a reminder, we picture (again) in Figures 6.1 and 6.2 the protocol P we defined for CM.

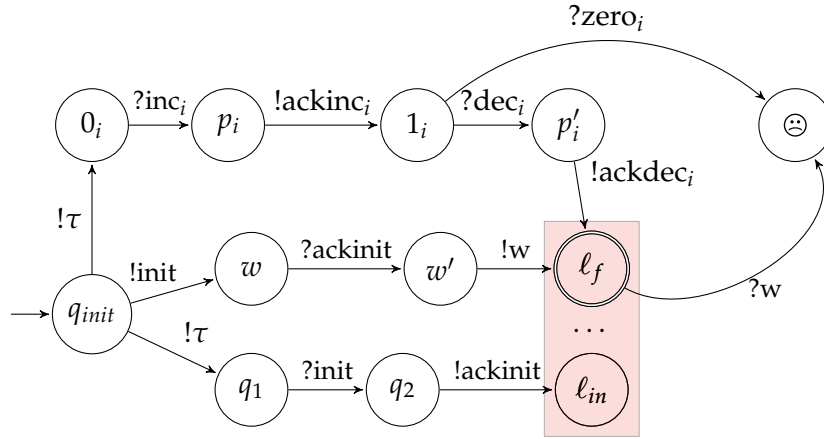


Figure 6.1: The protocol P – The coloured zone contains transitions pictured in Figure 6.2

Importantly, this protocol is Wait-Only:

- The states $\ell \in Loc \setminus \{\ell_f\}$, along with $q_{init}, p_1, p_2, p'_1, p'_2, w',$ and q_2 , are all action states.
- State ℓ_f is a waiting state (we assumed that there was no outgoing transition from ℓ_f in CM).
- The remaining states, including $0_1, 0_2, 1_1, 1_2, w, q_1$, and all states q_δ for $\delta \in \Delta$, are waiting states.

This observation directly yields the following theorem.

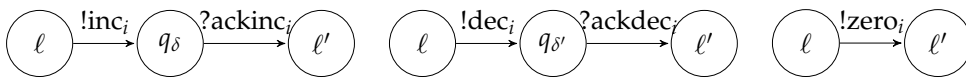
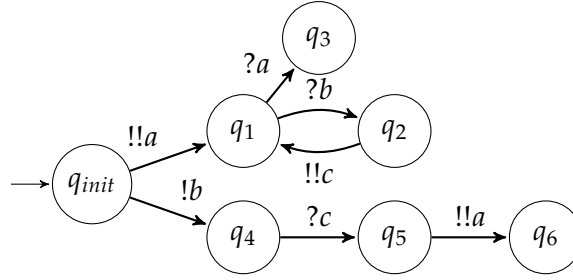


Figure 6.2: Translation of $\delta = (\ell, x_i+, \ell')$, $\delta' = (\ell, x_i-, \ell')$ and $(\ell, x_i=0, \ell')$ (respectively left figure, middle figure and right figure).


 Figure 6.3: Example of a Wait-Only protocol P .

Theorem 6.1. *The SYNCHRO^{nb} problem remains undecidable when restricted to Wait-Only rendez-vous protocols.*

Since any Wait-Only rendez-vous protocol is, in particular, a Wait-Only protocol, we immediately obtain:

Theorem 6.2. *The SYNCHRO^{nb} problem remains undecidable for Wait-Only protocols.*

We will see in the next chapter that SYNCHRO is however decidable for Wait-Only broadcast protocols.

6.1.2 Notations and Examples

We now introduce useful notations and provide an example of a Wait-Only protocol. We first formally define the sets of action and waiting states.

Definition 6.2 (Action and Waiting States). Let P be a Wait-Only protocol. We define:

- The set of *waiting states* as: $Q_W[P] = \{q \in Q \mid \exists q', (q, \alpha, q') \in T \text{ with } \alpha \in ?\Sigma\}$,
- The set of *action states* as its complement: $Q_A[P] = Q \setminus Q_W[P]$.

Thus, $Q_A[P]$ and $Q_W[P]$ form a partition of Q , and all sink states (states with no outgoing transition) are necessarily in $Q_A[P]$. When the context is clear, we will simply write Q_A and Q_W .

Remark 6. If the initial state q_{init} is a waiting state (i. e. $q_{init} \in Q_W$), then no execution is possible, as all processes would start in a state where no action can be performed. Thus, we assume throughout this thesis that $q_{init} \in Q_A$.

Example 6.1. An example of a Wait-Only protocol P is given in Figure 6.3. Here, action states are q_{init}, q_2, q_3, q_5 , and q_6 , while waiting states are q_1 and q_4 .

Consider the execution starting from the initial configuration $\{q_{init}, q_{init}, q_{init}\}$ with three processes:

$$\begin{aligned}
 \{q_{init}, q_{init}, q_{init}\} &\xrightarrow{(q_{init}, !!a, q_1)} \{q_1, q_{init}, q_{init}\} \xrightarrow{(q_{init}, !!b, q_4)} \{q_2, q_4, q_{init}\} \\
 &\xrightarrow{(q_{init}, !!b, q_4)} \{q_2, q_4, q_4\} \xrightarrow{(q_2, !!c, q_1)} \{q_1, q_5, q_5\} \\
 &\xrightarrow{(q_5, !!a, q_6)} \{q_3, q_6, q_5\}.
 \end{aligned}$$

It corresponds to the following sequence of events: one of the agents broadcasts message a (not received by anyone), then another agent sends message b which leads to a rendez-vous with the first agent on q_1 , the last agent sends message b which is not received by anyone, the agent in state q_2 broadcasts message c which is received by the two other agents, and finally, one of the agents in q_5 broadcasts letter a which is received by the process on q_1 .

Remark 7. In Chapter 2 (Remarks 1 and 3), we described how internal transitions can be simulated using send or broadcast transitions (with the sending or broadcast of a dummy message never received in the protocol). This remains possible in Wait-Only protocols, but only from action states. This is coherent with the Wait-Only restriction, since internal transitions correspond to actions and should only be possible from action states.

In the rest of this chapter, all the examples, statements, and proofs shall be presented using multiset configurations.

6.2 THE COPYPASTE PROPERTY

Wait-Only protocols enjoy a nice property on **coverable** states, which we present in this section. It is particularly useful for studying coverability and it makes a distinction between action states and waiting states. First, we show that an action state, if coverable, can be covered by arbitrarily many processes, unlike a waiting state. Indeed, it is possible that a waiting state can be covered by exactly one process at a time, and no more. However, we show that if two action states, or if an action state and a waiting state are coverable, then there is an execution that reaches a configuration where they are both covered.

This property relies on the fact that once the action state has been covered in an execution, it will not be emptied while performing the sequence of actions allowing to cover the second (waiting state), since no reception of message can happen in such a state. As we will see, this phenomenon can be generalised to a subset of action states.

Example 6.2. Going back to the protocol P of Figure 6.3, consider the action state q_2 . It is coverable as shown by the **execution**

$$\langle q_{init}, q_{init} \rangle \xrightarrow{(q_{init}, !!a, q_1)} \langle q_1, q_{init} \rangle \xrightarrow{(q_{init}, !b, q_4)} \langle q_2, q_4 \rangle.$$

From this execution, one can build an execution leading to a configuration covering $\langle n \cdot q_2 \rangle$ for any $n \in \mathbb{N}$. For instance, for $n = 2$, we build the following execution:

$$\begin{aligned} \langle q_{init}, q_{init}, q_{init}, q_{init} \rangle &\xrightarrow{(q_{init}, !!a, q_1)} \langle q_1, q_{init}, q_{init}, q_{init} \rangle \xrightarrow{(q_{init}, !b, q_4)} \langle q_2, q_4, q_{init}, q_{init} \rangle \\ &\xrightarrow{(q_{init}, !!a, q_1)} \langle q_2, q_4, q_1, q_{init} \rangle \xrightarrow{(q_{init}, !b, q_4)} \langle q_2, q_4, q_2, q_4 \rangle. \end{aligned}$$

Observe that this new execution replicates the original execution, effectively “copy-pasting” it. It starts with the two first processes performing $(q_{init}, !!a, q_1)$ $(q_{init}, !b, q_4)$, and then the two last processes do the same sequence of actions. Furthermore each waiting state that can be covered is always coverable alongside q_2 . For instance, $\langle q_2, q_4 \rangle$ is coverable as shown by the above execution.

Consider now a variant of P denoted P_{variant} depicted in Figure 6.4, where the transition labeled by $!!c$ ($(q_2, !!c, q_1)$ in P) and the transition labeled by $?c$ ($(q_4, ?c, q_5)$ in P) have been

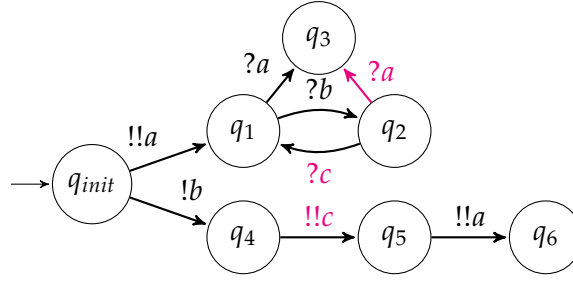


Figure 6.4: Example of a protocol denoted P_{variant} : it is protocol P with an additional edge $(q_2, ?a, q_3)$ and where broadcasts and reception of c have been swapped. P_{variant} is Wait-Only.

swapped and a reception transition of a has been added $((q_2, ?a, q_3) \text{ in } P_{\text{variant}})$. The protocol is Wait-Only, but unlike in P , q_2 is a waiting state and q_4 is an action state. One can easily check that q_2 and q_4 are still coverable in P_{variant} . According to the cypypaste property, $\langle q_2, q_4 \rangle$ should still be coverable, as witnessed by the following execution:

$$\langle q_{\text{init}}, q_{\text{init}} \rangle \xrightarrow{(q_{\text{init}}, !!a, q_1)} \langle q_1, q_{\text{init}} \rangle \xrightarrow{(q_{\text{init}}, !!b, q_4)} \langle q_2, q_4 \rangle$$

However, this time $\langle n \cdot q_2 \rangle$ for $n > 1$ is not coverable: at most one process at a time can cover q_2 . It remains coherent with the cypypaste property as now q_2 is a waiting state. This is because once a process is on q_1 or q_2 , any broadcast of a forces it to exit this state. In particular, when a new process wants to join in order to put two processes in q_2 , it has to broadcast a with transition $(q_{\text{init}}, !!a, q_1)$ and exits the process already there. For the same reason, $\langle q_1, q_2 \rangle$ is not coverable: waiting states are not always coverable together even when they can be covered separately. However, for any action state q (i. e. $q \in \{q_{\text{init}}, q_3, q_4, q_5, q_6\}$) the configuration $\langle n \cdot q, q_2 \rangle$ is coverable for any $n \in \mathbb{N}$.

Before stating the main lemma of this section (Proposition 6.1), we need an additional definition.

Definition 6.3. Let $P = (Q, \Sigma, q_{\text{init}}, T)$. For each *coverable* state $q \in Q$, let $\min_q$ be the minimal number of processes needed to cover q . More formally,

$$\min_q = \min\{n \mid n \in \mathbb{N}, \text{ there exists } M \in \mathcal{M} \text{ s. t. } \langle n \cdot q_{\text{init}} \rangle \xrightarrow{*} M \text{ and } M(q) > 0\}.$$

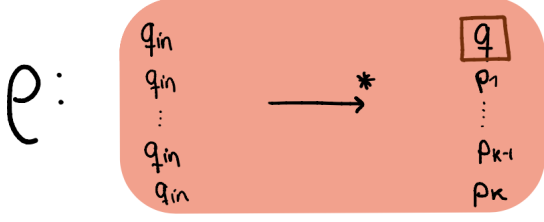
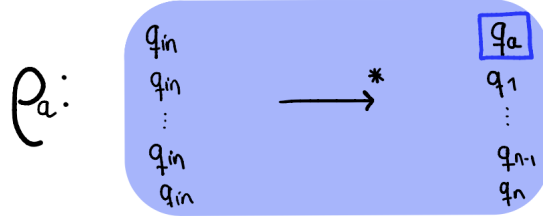
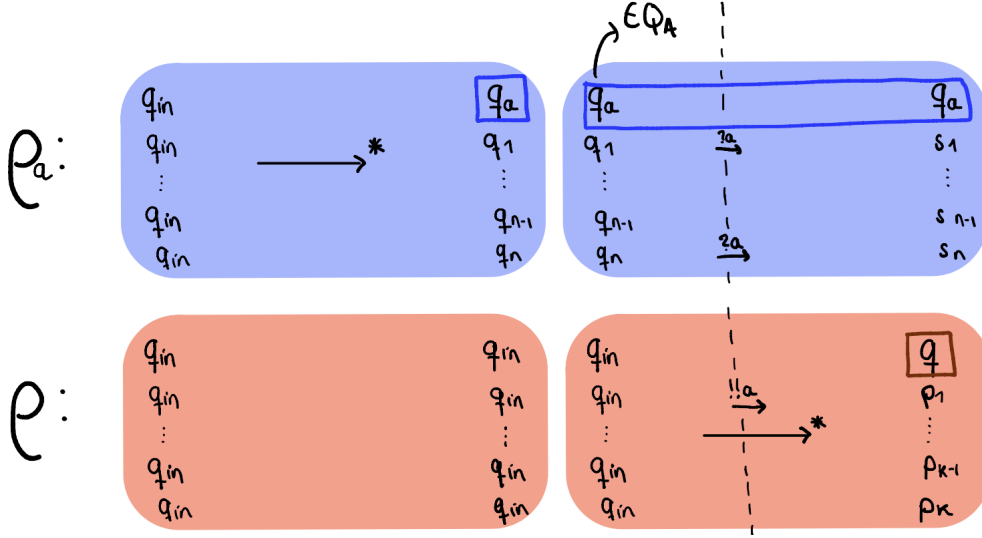
Note that $\min_q$ is defined only when q is coverable.

Along with ensuring the covering of multiple action states (and a waiting state) at a time, the cypypaste property also presents a bound on the number of processes to do so. In particular, to cover a configuration $\langle n \cdot q \rangle$, we need at most $n \cdot \min_q$ when state q is coverable. This is formalized in the lemma below.

Proposition 6.1 (The Cypypaste property). Let $P = (Q, \Sigma, q_{\text{init}}, T)$ be a Wait-Only protocol, $A = \{q_1, \dots, q_n\} \subseteq Q_A$ a subset of coverable *action* states and $p \in Q_W$ a coverable *waiting* state. Then, for all $N \in \mathbb{N}$, there exists an execution $M_{\text{init}} \xrightarrow{*} M$ such that $M_{\text{init}} \in \mathcal{M}_{\text{init}}$, and $\{N \cdot q_1, \dots, N \cdot q_n, p\} \leq M$. Moreover, $|M_{\text{init}}| = N \cdot \sum_{i=1}^n \min_{q_i} + \min_p$.

Let us explain the lemma before proving it formally. This lemma states that for any subset of coverable action states, and for any coverable waiting state, one can build an execution

covering all those states at the same time. Moreover, the execution can put an arbitrary number of processes on each action states. To clarify why we refer to this as the copypaste property, let us outline the proof intuition.

Figure 6.5: A run ρ covering q .Figure 6.6: A run ρ_a covering q_a .Figure 6.7: Gluing ρ and ρ_a together.

Let $P = (Q, \Sigma, q_{init}, T)$ be a Wait-Only protocol. Take a coverable action state $q_a \in Q_A$ and any coverable state $q \in Q$ (it might be that $q = q_a$). Consider the execution covering q_a [resp. q] and denote it ρ_a [resp. ρ]. We are able to build an execution ρ' with as many processes as in ρ_a and ρ combined, where ρ_a is glued before ρ , and such that a process remains in q_a when ρ is executed. This can be done thanks to the action states property. We illustrate the proof idea in Figures 6.5 to 6.7.

The proof of this lemma relies on two properties on executions of Wait-Only protocols. The first one formalizes what happens in the left part of Figure 6.7: given any initial configuration M_{init} (for example the starting configuration of ρ) and execution ρ_a , one can build a new execution ρ' which is composed of processes in M_{init} and processes of ρ_a . It executes ρ_a while processes of M_{init} remain idle in the initial state. This is possible as q_{init} is an action state (Remark 6), hence processes of M_{init} will not receive any broadcast messages or messages sent with a sending state. We formalize this in the following lemma.

Lemma 6.1. Given an execution $M_0 \xrightarrow{t_1} M_1 \xrightarrow{t_2} \dots \xrightarrow{t_k} M_k$ and another initial configuration M_{init} , we can build an execution $\widehat{M}_0 \xrightarrow{t_1} \widehat{M}_1 \xrightarrow{t_2} \dots \xrightarrow{t_k} \widehat{M}_k$ with $\widehat{M}_i = M_i + M_{init}$ for all $0 \leq i \leq k$.

Proof. Formally, we can build the execution described in Lemma 6.1 by induction on $0 \leq i \leq k$. For $i = 0$, let $\widehat{M}_0 = M_0 + M_{init}$. Hence as $M_0 \in \mathcal{M}_{init}$ and $M_{init} \in \mathcal{M}_{init}$, it

follows that $\widehat{M}_0 \in \mathcal{M}_{init}$. Now assume that $\widehat{M}_0 \xrightarrow{*} \widehat{M}_i$ with $\widehat{M}_i = M_i + M_{init}$ for some $i \in [0, k-1]$. We let $N = M_{init}(q_{init})$.

- If $t_{i+1} = (p_1, !m, p_2)$, let $M_i = \langle p_1, q_1, \dots, q_{N'-1} \rangle$ and $M_{i+1} = \langle p_2, q'_1, \dots, q'_{N'-1} \rangle$. For all $1 \leq i \leq N' - 1$, either $m \notin R(q_i)$, and $q'_i = q_i$, or $(q_i, ?m, q'_i) \in T$. By induction hypothesis, $\widehat{M}_0 \xrightarrow{*} \widehat{M}_i$ with $\widehat{M}_i = \langle p_1, q_1, \dots, q_{N'-1}, N \cdot q_{init} \rangle$. Since q_{init} is an action state (see Remark 6), we know that $m \notin R(q_{init})$, hence, with $\widehat{M}_{i+1} = \langle p_2, p'_1, \dots, p'_{N'-1}, N \cdot q_{init} \rangle$, it holds that $\widehat{M}_i \xrightarrow{t_{i+1}} \widehat{M}_{i+1}$ and $\widehat{M}_{i+1} = M_{i+1} + M_{init}$.
- If $t_{i+1} = (p_1, !m, p_2)$ and there exist $q, q' \in Q$ such that $(q, ?m, q') \in T$ and $(M_i - \langle p_1 \rangle)(q) > 0$ (observe that in Wait-Only protocols, as $Q_A \cap Q_W = \emptyset$, $p_1 \neq q$ and so $(M_i - \langle p_1 \rangle)(q) > 0$ if and only if $M_i(q) > 0$), then $M_{i+1} = M_i - \langle p_1, q \rangle + \langle p_2, q' \rangle$. By induction hypothesis, $\widehat{M}_0 \xrightarrow{*} \widehat{M}_i$ with $\widehat{M}_i = M_i + M_{init}$. Hence, $\widehat{M}_i(p_1) > 0$; $\widehat{M}_i(q) > 0$ and so $\widehat{M}_0 \xrightarrow{*} \widehat{M}_i \xrightarrow{t_{i+1}} \widehat{M}_i - \langle p_1, q \rangle + \langle p_2, q' \rangle$ and $\widehat{M}_i - \langle p_1, q \rangle + \langle p_2, q' \rangle = M_i + M_{init} - \langle p_1, q \rangle + \langle p_2, q' \rangle = M_{i+1} + M_{init}$.
- If $t_{i+1} = (p_1, !m, p_2)$ and for all $q, q' \in Q$ such that $(q, ?m, q') \in T$, $M_i(q) = 0$, then $M_{i+1} = M_i - \langle p_1 \rangle + \langle p_2 \rangle$. By induction hypothesis, $\widehat{M}_0 \xrightarrow{*} \widehat{M}_i$ with $\widehat{M}_i = M_i + M_{init}$. Observe that for all $q, q' \in Q$ such that $(q, ?m, q') \in T$, $\widehat{M}_i(q) = M_i(q) = 0$. Indeed, since q_{init} is an action state, $q_{init} \neq q$. Then $\widehat{M}_i \xrightarrow{t_{i+1}} \widehat{M}_i - \langle p_1 \rangle + \langle p_2 \rangle$ and $\widehat{M}_{i+1} = \widehat{M}_i - \langle p_1 \rangle + \langle p_2 \rangle = M_i + M_{init} - \langle p_1 \rangle + \langle p_2 \rangle = M_{i+1} + M_{init}$.

□

The second auxiliary lemma formalizes the fact that from an execution $M \xrightarrow{*} M'$, if one considers a configuration M'' , then from the configuration $M + M''$, one can execute the same sequence of events happening in the execution $M \xrightarrow{*} M'$ and reach a configuration M''' such that:

- $M'''(q) \geq M'(q)$ for all $q \in Q_W$;
- $M'''(q) \geq M'(q) + M''(q)$ for all $q \in Q_A$.

This is due to the fact that from states in Q_A , there is no outgoing reception transition, and hence, no broadcast nor sending happening between M and M' can make a process in an action state leave it. This is pictured on the right part of Figure 6.7: the process in q_a stays in q_a while ρ is executed. We generalize this statement to any configuration $M + N \cdot M''$ in the lemma below.

Lemma 6.2. Given an execution $M_0 \xrightarrow{t_1} M_1 \xrightarrow{t_2} \dots \xrightarrow{t_k} M_k$, and an integer $N \geq 1$, for all configurations (not necessarily initial) $\widetilde{M}_0$ such that $\widetilde{M}_0(q_{init}) \geq N \cdot M_0(q_{init})$, we can build a run $\widetilde{M}_0 \xrightarrow{t_1} \dots \xrightarrow{t_k} \widetilde{M}_k$ in which:

- $\widetilde{M}_i(q) \geq \widetilde{M}_0(q) + M_i(q)$ for all $q \in Q_A \setminus \{q_{init}\}$,
- $\widetilde{M}_i(q) \geq M_i(q)$ for all $q \in Q_W$, and
- $\widetilde{M}_i(q_{init}) \geq (N - 1) \cdot M_0(q_{init}) + M_i(q_{init})$

for all $0 \leq i \leq k$.

Proof. Again, we can build the run by induction on $0 \leq i \leq k$. For $i = 0$, it is obvious since $M_0(q) = 0$ for all $q \in Q \setminus \{q_{init}\}$. Let now $0 \leq i < k$ and assume that we have built a run $\tilde{M}_0 \rightsquigarrow^* \tilde{M}_i$ such that:

$$\begin{aligned}\tilde{M}_i(q) &\geq \tilde{M}_0(q) + M_i(q) \text{ for all } q \in Q_A \setminus \{q_{init}\} \\ \tilde{M}_i(q) &\geq M_i(q) \text{ for all } q \in Q_W \\ \tilde{M}_i(q_{init}) &\geq (N-1) \cdot M_0(q_{init}) + M_i(q_{init})\end{aligned}$$

Let $N_1 = M_0(q_{init})$.

- If $t_{i+1} = (p_1, !a, p_2)$, then by induction hypothesis (and because $p_1 \in Q_A$),
 - $\tilde{M}_i(p_1) \geq \tilde{M}_0(p_1) + M_i(p_1)$ if $p_1 \neq q_{init}$, and
 - $\tilde{M}_i(p_1) \geq (N-1) \cdot N_1 + M_i(p_1)$ if $p_1 = q_{init}$.

Moreover, $M_i(p_1) > 0$, hence the transition t_{i+1} can be taken from $\tilde{M}_i$.

Let $M_i = \langle p_1, q_1, \dots, q_{N_1-1} \rangle$, then, from the **broadcast step** definition, $M_{i+1} = \langle p_2, q'_1, \dots, q'_{N_1-1} \rangle$ such that for all $1 \leq i \leq N_1$, either $a \notin R(q_i)$ and $q'_i = q_i$, or $(q_i, ?a, q'_i) \in T$. Also, $\tilde{M}_i = \langle p_1, q_1, \dots, q_{N_1-1}, q''_1, \dots, q''_K \rangle$ and we now define $\tilde{M}_{i+1} = \langle p_2, q'_1, \dots, q'_{N_1-1}, p''_1, \dots, p''_K \rangle$, with, for all $1 \leq i \leq K$: if $a \notin R(q''_i)$ then $p''_i = q''_i$, and otherwise, let $p''_i \in Q$ such that $(q''_i, ?a, p''_i) \in T$. By definition of a **broadcast step**, we get that $\tilde{M}_i \xrightarrow{t_{i+1}} \tilde{M}_{i+1}$.

It naturally follows that for all $q \in Q$, $\tilde{M}_{i+1}(q) \geq M_{i+1}(q)$. Let now $q \in Q_A$. Either $\tilde{M}_{i+1}(q) = \tilde{M}_i(q) + \ell$ for some $\ell \geq 0$, or $\tilde{M}_{i+1}(q) = \tilde{M}_i(q) - 1$ (and $q = p_1$), i.e. $\tilde{M}_{i+1}(q) = \tilde{M}_i(q) + \ell$ for some $\ell \geq -1$. Hence, $M_{i+1}(q) = M_i(q) + \ell'$, with $\ell \geq \ell' \geq -1$. Indeed, let $\{q^1, \dots, q^r\}$ be the set of states such that $\tilde{M}_i(q^j) > 0$ and $(q^j, ?a, q) \in T$ with q being the chosen state of the corresponding component when defining $\tilde{M}_{i+1}$, for all $1 \leq j \leq r$. Then $M_i(q^j) \leq \tilde{M}_i(q^j)$ by induction hypothesis, so $M_{i+1}(q) = M_i(q) + \ell'$ and $\tilde{M}_{i+1}(q) = \tilde{M}_i(q) + \ell$ with $\ell \geq \ell' \geq -1$. Hence, if $q \neq q_{init}$, by induction hypothesis,

$$\begin{aligned}\tilde{M}_{i+1}(q) &\geq \tilde{M}_0(q) + M_i(q) + \ell \\ &= \tilde{M}_0(q) + M_{i+1}(q) - \ell' + \ell \\ &\geq \tilde{M}_0(q) + M_{i+1}(q).\end{aligned}$$

If otherwise $q = q_{init}$, by induction hypothesis,

$$\begin{aligned}\tilde{M}_{i+1}(q_{init}) &= \tilde{M}_i(q_{init}) + \ell \\ &\geq (N-1) \cdot N_1 + M_i(q_{init}) + \ell \\ &= (N-1) \cdot N_1 + M_{i+1}(q_{init}) - \ell' + \ell \\ &\geq (N-1) \cdot N_1 + M_{i+1}(q_{init}).\end{aligned}$$

- Let $t_{i+1} = (p_1, !a, p_2)$ and there exist $p, p' \in Q$ such that $(p, ?a, p') \in T$ and $M_i(p) > 0$. Then, $M_i(p_1) > 0$ and $M_{i+1} = M_i - \langle p_1, p \rangle + \langle p_2, p' \rangle$. By induction hypothesis,
 - $\tilde{M}_i(p_1) \geq \tilde{M}_0(p_1) + M_i(p_1)$ if $p_1 \neq q_{init}$

- $\tilde{M}_i(p_1) \geq (N-1) \cdot N_1 + M_i(p_1)$ if $p_1 = q_{init}$
- $\tilde{M}_i(p) \geq M_i(p)$.

Hence $\tilde{M}_i \xrightarrow{t_{i+1}} \tilde{M}_{i+1}$ where $\tilde{M}_{i+1} = \tilde{M}_i - \{p_1, p\} + \{p_2, p'\}$. Observe that for all $q \in Q$, $\tilde{M}_{i+1}(q) - \tilde{M}_i(q) = M_{i+1}(q) - M_i(q)$. Hence, if we let $q \in Q$, by induction hypothesis we get that:

$$\begin{aligned}
 \tilde{M}_{i+1}(q) &= \tilde{M}_i(q) - M_i(q) + M_{i+1}(q) \\
 &\geq \tilde{M}_0(q) + M_{i+1}(q) && \text{if } q \in Q_A \setminus \{q_{init}\} \\
 &\geq (N-1) \cdot N_1 + M_{i+1}(q) && \text{if } q = q_{init} \\
 &\geq M_{i+1}(q) && \text{if } q \in Q_W.
 \end{aligned}$$

- If $t_{i+1} = (p_1, !a, p_2)$ and for all $p, p' \in Q$ such that $(p, ?a, p') \in T$, we have that $M'_i(p) = 0$, then, either $\tilde{M}_i(p) = 0$ for all $p, p' \in Q$ such that $(p, ?a, p') \in T$, or there exist some $p, p' \in Q$ such that $(p, ?m, p') \in T$ and $\tilde{M}_i(p) > 0$. In the first case, since $\tilde{M}_i(p_1) \geq 0$ by induction hypothesis, $\tilde{M}_{i+1} = \tilde{M}_i - \{p_1\} + \{p_2\}$, and $M_{i+1} = M_i - \{p_1\} + \{p_2\}$. Then, as in the previous case $\tilde{M}_{i+1} - \tilde{M}_i = M_{i+1} - M_i$, which allows us to conclude.

In the second case, $\tilde{M}_{i+1} = \tilde{M}_i - \{p_1, p\} + \{p_2, p'\}$. Then the only states q for which $\tilde{M}_{i+1}(q) - \tilde{M}_i(q) \neq M_{i+1}(q) - M_i(q)$ are states p' and p . Hence, we only focus on those two states as for other states we can conclude using the reasoning of the previous case. Observe that the only interesting case is $p' \neq p$ as otherwise, we can again use the previous reasoning. Hence, consider $p \neq p'$. By construction, $p \in Q_W$, $\tilde{M}_i(p) > 0$ and $M_i(p) = 0$. If $p_2 = p$, we get that $M_{i+1}(p) = 1$ and $\tilde{M}_{i+1}(p) = \tilde{M}_i(p) + 1 - 1 = \tilde{M}_i(p) \geq 1 = M_{i+1}(p)$ which concludes this case. Otherwise, $M_{i+1}(p) = 0$ and $\tilde{M}_{i+1}(p) = \tilde{M}_i(p) - 1 \geq 0 = M_{i+1}(p)$ which concludes this case.

Consider now p' . Observe that $\tilde{M}_{i+1}(p') - \tilde{M}_i(p') > M_{i+1}(p') - M_i(p')$, hence $\tilde{M}_{i+1}(p') > \tilde{M}_i(p') + M_{i+1}(p') - M_i(p')$, we conclude with the reasoning of the previous item.

□

We are now ready to prove Proposition 6.1.

Proof of Proposition 6.1. Let $P = (Q, \Sigma, q_{init}, T)$ be a Wait-Only protocol, $A = \{q_1, \dots, q_n\} \subseteq Q_A$ a subset of coverable *action* states and $p \in Q_W$ a coverable *waiting* state. Let $N \in \mathbb{N}$. Using Lemmas 6.1 and 6.2, we can now prove the lemma.

We start by proving that there exists an execution $M_{init} \xrightarrow{*} M$ such that for all $q \in A$, $M(q) \geq N$ and $|M_{init}| = N \cdot \sum_{i=1}^n \min_{q_i}$.

We prove it by induction on the size of A . If $A = \emptyset$, the property is trivially true.

Let $n \in \mathbb{N}$, and assume the property to hold for all subsets $A \subseteq Q_A$ of size n . Take $A = \{q_1, q_2, \dots, q_{n+1}\} \subseteq Q_A$ of size $n+1$ such that all states $q \in A$ are coverable and let $A' = A \setminus \{q_1\}$. If $q_{init} \in A$, w.l.o.g. we assume that $q_1 = q_{init}$, and hence $q_{init} \notin A'$. Consider the execution

$$M_0 \xrightarrow{t_1} M_1 \xrightarrow{t_2} \dots \xrightarrow{t_k} M_k \text{ with } M_k(q_1) > 0 \text{ and } |M_0| = \min_{q_1}$$

and the execution

$$M'_0 \xrightarrow{t'_1} M'_1 \xrightarrow{t'_2} \dots \xrightarrow{t'_\ell} M'_\ell \text{ s.t. } M'_\ell(q') \geq N \text{ for all } q' \in A', \text{ and } |M'_0| = N \cdot \sum_{i=2}^{n+1} \min_{q_i}$$

(it exists by induction hypothesis).

We let $M_0^N = \wr(N \cdot \min_{q_1}) \cdot q_{init}$ and $M''_0 = M'_0 + M_0^N$. Thanks to Lemma 6.1, we can build an execution

$$M''_0 \xrightarrow{t'_1} M''_1 \xrightarrow{t'_2} \dots \xrightarrow{t'_\ell} M''_\ell \text{ with } M''_\ell = M'_\ell + M_0^N$$

In particular, for all $q' \in A'$,

$$M''_\ell(q') = M'_\ell(q') \geq N \text{ and } |M''_0| = |M'_0| + |M_0^N| = N \cdot \sum_{i=2}^{n+1} \min_{q_i} + N \cdot \min_{q_1}.$$

Now that we have shown how to build an execution that leads to a configuration with more than N processes on all states in A' and enough processes in the initial state, we show that mimicking N times the execution allowing to cover q_1 allows to obtain the desired result. Observe that if $q_1 = q_{init}$, there is nothing to do, as $\min_{q_1} = 1$. We assume now that $q_1 \neq q_{init}$. Let $M_{0,1} = M'_\ell$. We know that for all $q' \in A'$, $M_{0,1}(q') \geq N$, and $M_{0,1}(q_{init}) \geq N \cdot \min_{q_1}$. Since $|M_0| = \min_{q_1}$, using Lemma 6.2, we can build the execution

$$\begin{aligned} M_{0,1} \xrightarrow{t_1} \dots \xrightarrow{t_k} M_{k,1} \text{ with } M_{k,1}(q_{init}) &\geq (N-1) \cdot \min_{q_1}, \\ M_{k,1}(q') &\geq M_{0,k}(q') + M_k(q') \geq N \text{ for all } q' \in A' \text{ and} \\ M_{k,1}(q_1) &\geq M_{0,k}(q_1) + M_k(q_1) \geq 1 \end{aligned}$$

Iterating this construction and applying each time Lemma 6.2, we obtain that there is an execution

$$\begin{aligned} M_{0,1} \xrightarrow{t_1} \dots \xrightarrow{t_k} M_{k,1} \xrightarrow{t_1} \dots \xrightarrow{t_k} M_{k,2} \dots \xrightarrow{t_1} \dots \xrightarrow{t_k} M_{k,N-1} \xrightarrow{t_1} \dots \xrightarrow{t_k} M_{k,N} \\ \text{with for all } 1 \leq i \leq N: M_{k,i}(q_{init}) &\geq (N-i) \cdot \min_{q_1} \\ M_{k,i}(q') &\geq N \text{ for all } q' \in A', \text{ and} \\ M_{k,i}(q_1) &\geq M_{k,i-1}(q_1) + 1 \geq i \end{aligned}$$

Observe that to obtain that $M_{k,i}(q_1) \geq i$ from Lemma 6.2, we use the fact that $q_1 \in Q_A$. Hence, $M_{k,N}(q_1) \geq N$ and $M_{k,N}(q') \geq N$ for all $q' \in A'$ and we have built an execution where $M_{k,N}(q) \geq N$ for all $q \in A$ and $|M_{k,N}| = |M'_0| = N \cdot \sum_{i=1}^{|A|} \min_{q_i}$, as expected.

At last, consider $p \in Q_W$ the coverable state and $M'_0 \rightarrow^* M'_k$ such that $M'_k(p) \geq 1$ and $|M'_0| = \min_p$. Let $M_0 \rightarrow^* M_m$ be an execution such that $M_m(q) \geq N$ for all $q \in A$ and $|M_0| = N \cdot \sum_{i=1}^n \min_{q_i}$, as we have built before. By Lemma 6.1, we let $\hat{M}_0 = M_0 + M'_0$ and we have an execution $\hat{M}_0 \rightarrow^* \hat{M}_m$ with $\hat{M}_m = M_m + M'_0$. Hence, $\hat{M}_m(q) \geq N$ for all $q \in A$ and $\hat{M}_m(q_{init}) \geq M'_0(q_{init})$, and note that $|\hat{M}_m| = |\hat{M}_0| = |M_0| + |M'_0| = N \cdot \sum_{i=1}^n \min_{q_i} + \min_p$. Then, with $\tilde{M}_0 = \hat{M}_m$, by Lemma 6.2, we have a run $\tilde{M}_0 \rightarrow^* \tilde{M}_k$ with $\tilde{M}_k(q) \geq \tilde{M}_0(q) + M_k(q) \geq \tilde{M}_0(q) \geq N$ for all $q \in A$, and $\tilde{M}_k(p) \geq M_k(p) \geq 1$, and $|\tilde{M}_0| = |\hat{M}_0| = N \cdot \sum_{i=1}^n \min_{q_i} + \min_p$.

□

6.3 STATE COVERABILITY

In this section, we prove that the **STATE-COVER**^{nb} problem restricted to **Wait-Only protocols** is PTIME-complete. In Section 6.3.1, we present a PTIME algorithm answering **STATE-COVER**^{nb} relying on Proposition 6.1. We then present in Section 6.3.2 a reduction from the Circuit Value Problem (which is known to be PTIME-complete) to **STATE-COVER**^{nb}. This way, we prove that the **STATE-COVER**^{nb} problem is PTIME-hard.

6.3.1 Upper bound

We present here a polynomial time algorithm to solve the state coverability problem when the considered protocol is Wait-Only. Our algorithm computes in a greedy manner the set of coverable states using Proposition 6.1.

Given a Wait-Only protocol $P = (Q, \Sigma, q_{init}, T)$, we compute iteratively a set of states $S \subseteq Q$ containing all the states that are coverable by P , by relying on a family $(S_i)_{i \in \mathbb{N}}$ of subsets of Q formally defined as follows:

$$\begin{aligned} S_0 &= \{q_{init}\} \\ S_{i+1} &= S_i \cup \{q \mid \text{there exists } q' \in S_i, (q', \alpha, q) \in T, \alpha \in !!\Sigma \cup !\Sigma\} \\ &\quad \cup \{q'_2 \mid \text{there exist } q_1, q_2 \in S_i, q'_1 \in Q, a \in \Sigma \text{ s. t. } (q_1, !a, q'_1) \in T \text{ and } (q_2, ?a, q'_2) \in T\} \\ &\quad \cup \{q'_2 \mid \text{there exist } q_1, q_2 \in S_i, q'_1 \in Q, a \in \Sigma \text{ s. t. } (q_1, !!a, q'_1) \in T \text{ and } (q_2, ?a, q'_2) \in T\} \end{aligned}$$

Intuitively at each iteration, we add some control states to S_{i+1} either if they can be reached from a transition labeled with an action (in $!!\Sigma \cup !\Sigma$) starting at a state in S_i or if they can be reached by two transitions corresponding to a communication by broadcast or by rendez-vous starting from states in S_i . We then define $S = \bigcup_{n \in \mathbb{N}} S_n$. Observe that $(S_i)_{i \in \mathbb{N}}$ is an increasing sequence, and each time it grows it is because there exists a transition $t = (q, \alpha, q')$ such that $q' \in S_{i+1} \setminus S_i$. Hence, we reach a fix-point $N \leq |T|$ such that $S_N = S_{N+1} = S$. Hence S can be computed in polynomial time.

The two following lemmas show correctness of this algorithm. We first prove that any state $q \in S$ is indeed coverable by P . Moreover, we show that $\min_q$ the minimal number of processes necessary to cover $q \in Q$, is smaller than $2^{|T|}$.

Lemma 6.3. If $q \in S$, then there exists $M \in \mathcal{M}_{init}$ and $M' \in \mathcal{M}$ such that $M \rightarrow^* M'$, $M'(q) > 0$ and $|M| \leq 2^{|T|}$.

Proof. Let $N \in \mathbb{N}$ be the first natural such that $S_N = S_{N+1}$. We have then $S_N = S$ and $N \leq |T|$. We prove by induction that for all $0 \leq i \leq N$, for all $q \in S_i$, there exists $M \in \mathcal{M}_{init}$ and $M' \in \mathcal{M}$ such that $M \rightarrow^* M'$, $M'(q) > 0$ and $|M| \leq 2^i$.

As $S_0 = \{q_{init}\}$, the property trivially holds for $i = 0$, since $\{q_{init}\} \in \mathcal{M}_{init}$ and $\{q_{init}\}(q_{init}) > 0$.

Assume now the property to be true for $i < N$ and let $q \in S_{i+1}$. If $q \in S_i$, then by induction hypothesis, we have that there exists $M \in \mathcal{M}_{init}$ and $M' \in \mathcal{M}$ such that $M \rightarrow^* M'$, $M'(q) > 0$ and $|M| \leq 2^i < 2^{i+1}$. We suppose that $q \notin S_i$ and proceed by a case analysis on the way q has been added to S_{i+1} .

1. there exists $q' \in S_i$ and $t = (q', \alpha, q) \in T$ with $\alpha \in !!\Sigma \cup !\Sigma$. By induction hypothesis, there exists an execution $M \rightarrow^* M'$ such that $M'(q') > 0$ and $|M| \leq 2^i$. But we

have then $M' \xrightarrow{t} M''$ with $M''(q) > 0$, and consequently as well $M \xrightarrow{*} M''$. This is true because of the “non-blocking” nature of both broadcast and rendez-vous message in this model. Hence there is no need to check for a process to receive the message to ensure the execution $M \xrightarrow{*} M''$.

2. there exist $q_1, q_2 \in S_i$ and $q'_1 \in Q$ and there exists $a \in \Sigma$ such that $(q_1, !a, q'_1), (q_2, ?a, q) \in T$. By induction hypothesis, we have that there exist $M_1, M_2 \in \mathcal{M}_{init}$ and $M'_1, M'_2 \in \mathcal{M}$ such that $M_1 \xrightarrow{*} M'_1$ and $M_2 \xrightarrow{*} M'_2$ and $M'_1(q_1) > 0$ and $M'_2(q_2) > 0$ and $|M_1| \leq 2^i$ and $|M_2| \leq 2^i$. Note furthermore that by definition q_1 is in Q_A and as $(q_2, ?a, q) \in T$, q_2 does not belong to Q_A . Hence $q_1 \neq q_2$. By Proposition 6.1, we know that there exist $M \in \mathcal{M}_{init}$ and $M' \in \mathcal{M}$ such that $M \xrightarrow{*} M'$ and $M'(q_1) > 0$ and $M'(q_2) > 0$. Furthermore, recall that $\min_{q_i}$ for $i \in \{1, 2\}$ is the minimal number of processes needed to cover q_i and by Proposition 6.1, $|M| \leq \min_{q_1} + \min_{q_2}$. By induction hypothesis, $\min_{q_1} + \min_{q_2} \leq 2^i + 2^i$, hence $|M| \leq 2^{i+1}$.

We then have $M' \xrightarrow{(q_1, !a, q'_1)} M''$ with $M'' = M' - \{q_1, q_2\} + \{q'_1, q\}$. Hence $M \xrightarrow{*} M''$ with $M''(q) > 0$.

3. there exist $q_1, q_2 \in S_i$ and $q'_1 \in Q$ and there is some $a \in \Sigma$ such that $(q_1, !!a, q'_1), (q_2, ?a, q) \in T$. As above, we obtain the existence of an execution $M \xrightarrow{*} M'$ with $M'(q_1) > 0$ and $M'(q_2) > 0$, and $|M| \leq 2^{i+1}$. Then $M' = \{q_1, q_2, \dots, q_k\}$ and $M' \xrightarrow{(q_1, !!a, q'_1)} M''$ with $M'' = \{q'_1, q, \dots, q'_k\}$ with, for all $3 \leq j \leq k$, either $a \notin R(q_j)$ and $q_j = q'_j$ or $(q_j, ?a, q'_j) \in T$. In any case, we have $M \xrightarrow{*} M''$ with $M''(q) > 0$.

So, for any $q \in S$, $q \in S_M$ and we have an execution $M \xrightarrow{*} M'$ with $M \in \mathcal{M}_{init}$ such that $M'(q) > 0$ and $|M| \leq 2^N \leq 2^{|T|}$. $\square$

We now prove the completeness of our algorithm by showing that every coverable state belongs to S .

Lemma 6.4. If there exists $M \in \mathcal{M}_{init}$ and $M' \in \mathcal{M}$ and $q \in Q$ such that $M \xrightarrow{*} M'$ and $M'(q) > 0$, then $q \in S$.

Proof. We consider the execution $M_0 \xrightarrow{t_1} M_1 \xrightarrow{t_2} \dots \xrightarrow{t_n} M_n$ with $M = M_0$ and $M_n = M'$. We will prove by induction on $0 \leq i \leq n$ that for all q such that $M_i(q) > 0$, we have $q \in S_i$.

For $i = 0$, we have $M_0 = \{ |M| \cdot q_{init} \}$, and $S_0 = \{q_{init}\}$. Hence the property holds. Assume the property to be true for $i < n$, and let $q \in Q$ such that $M_{i+1}(q) > 0$. If $M_i(q) > 0$, then by induction hypothesis we have $q \in S_i$ and since $S_i \subseteq S_{i+1}$, we deduce that $q \in S_{i+1}$. Assume now that $M_i(q) = 0$. We proceed by a case analysis.

1. $t_{i+1} = (q', !a, q)$ or $t_{i+1} = (q', !!a, q)$ for some $a \in \Sigma$ and $q' \in Q$. Since $M_i \xrightarrow{t_{i+1}} M_{i+1}$, we have necessarily $M_i(q') > 0$. By induction hypothesis, $q' \in S_i$, and by construction of S_{i+1} , we deduce that $q \in S_{i+1}$.
2. $t_{i+1} = (q_1, !a, q'_1)$ or $t_{i+1} = (q_1, !!a, q'_1)$ with $q'_1 \neq q$. Since $M_i(q) = 0$ and $M_{i+1}(q) > 0$, there exists a transition of the form $(q_2, ?a, q)$ with $q_1 \neq q_2$ (because $q_1 \in Q_A$ and $(q_2, ?a, q) \in T$ hence $q_2 \notin Q_A$). Consequently, we know that we have

$M_i(q_1) > 0$ and $M_i(q_2) > 0$. By induction hypothesis q_1 and q_2 belong to S_i and by construction of S_{i+1} we deduce that $q \in S_{i+1}$. □

The two previous lemmas show the soundness and completeness of our algorithm to solve STATE-COVER^{nb} based on the computation of the set S . Since this set of states can be computed in polynomial time, we obtain the following result.

Theorem 6.3. STATE-COVER^{nb} is in PTIME for Wait-Only protocols.

Furthermore, completeness of the algorithm along with the bound on the number of processes established in Lemma 6.3 gives the following result.

Corollary 6.1. Given a Wait-Only protocol P , for all coverable state $q \in Q$, it holds that $\min_q \leq 2^{|T|}$.

We provide an example for which the above mentioned bound is (almost) reached.

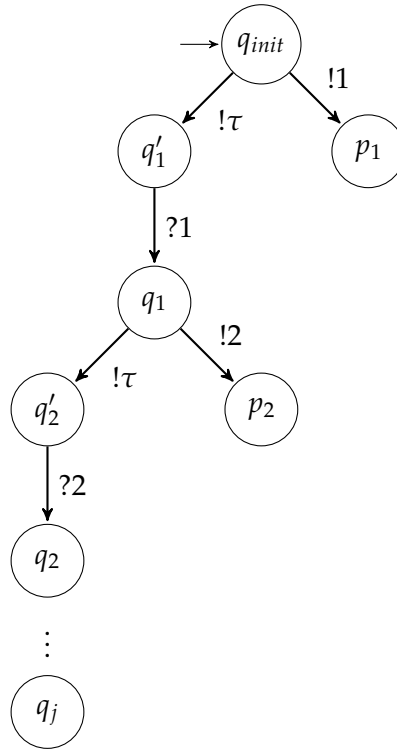


Figure 6.8: Example of a protocol denoted P_{exp} in which state q_j is coverable from any initial configurations with at least $2^j = 2^{\frac{|T|}{3}}$ processes for $j > 0$.

Example 6.3. Consider the protocol depicted in Figure 6.8 for $j = 2$. The state q_2 is coverable, as witnessed by the following execution:

$$\begin{aligned}
& \langle q_{\text{init}}, q_{\text{init}}, q_{\text{init}}, q_{\text{init}} \rangle \xrightarrow{(q_{\text{init}}, !\tau, q'_1)} \langle q'_1, q_{\text{init}}, q_{\text{init}}, q_{\text{init}} \rangle \xrightarrow{(q_{\text{init}}, !\tau, q'_1)} \langle q'_1, q'_1, q_{\text{init}}, q_{\text{init}} \rangle \\
& \xrightarrow{(q_{\text{init}}, !1, p_1)} \langle q_1, q'_1, p_1, q_{\text{init}} \rangle \xrightarrow{(q_{\text{init}}, !1, p_1)} \langle q_1, q_1, p_1, p_1 \rangle \\
& \xrightarrow{(q_1, !\tau, q'_2)} \langle q'_2, q_1, p_1, p_1 \rangle \xrightarrow{(q_1, !2, p_2)} \langle q_2, p_2, p_1, p_1 \rangle.
\end{aligned}$$

Observe that this execution requires $2^2 = 4$ processes.

More generally, to cover q_j , one must first reach a configuration where two processes are in state q_{j-1} . From there, one process sends a τ transition and moves to q'_j ; the other then sends message j , which is received by the first, allowing it to reach q_j . Using a simple induction, we show that reaching q_{j-1} with a single process requires 2^{j-1} processes and leads to configuration $\langle q_1, p_1, q_2, p_2, \dots, q_j, p_j \rangle$. Therefore, obtaining two such processes requires $2 \times 2^{j-1} = 2^j$ processes. Hence, covering q_j requires 2^j processes.

6.3.2 Lower bound

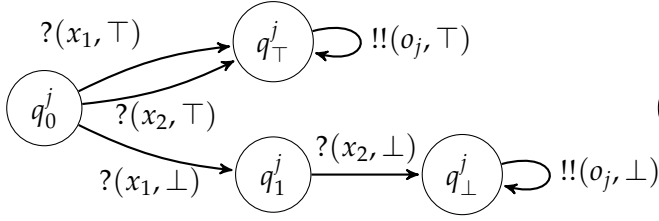
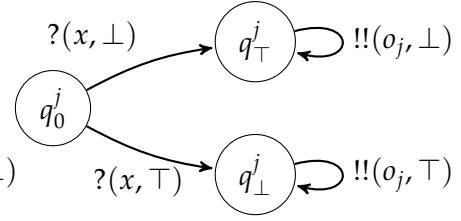
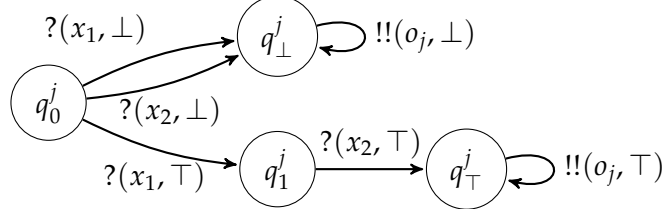
We now proceed to show that STATE-COVER^{nb} for Wait-Only protocols is PTIME-hard. For this, we provide a reduction from the Circuit Value Problem (CVP) which is known to be PTIME-complete [Lad75]. CVP is defined as follows: given an acyclic Boolean circuit with n input variables, one output variable, m boolean gates of type *and*, *or*, *not*, and a truth assignment for the input variables, is the value of the output equal to a given boolean value? Given an instance of the CVP, we build a protocol in which the processes broadcast variables (input ones or associated with gates) along with their boolean values. These broadcasts will be received by other processes that will use them to compute boolean value of their corresponding gate, and broadcast the obtained value. Hence, different values are propagated through the protocol representing the circuit, until the state representing the output variable value we look for is covered.

We start by introducing some notations. We denote an instance of CVP $C = (V, o, G, B, b)$ where:

- $V = \{v_1, \dots, v_n\}$ denotes the n input variables,
- o is the output variable,
- $G = \{g_1, \dots, g_m\}$ the m boolean gates (defined hereafter),
- $B = \{b_1, \dots, b_n\}$ the boolean assignment such that for all $1 \leq i \leq n$, boolean $b_i \in \{\top, \perp\}$ is the assignment of variable v_i , and
- b the boolean output value to test.

The size of C is defined as $2n + m$.

Let $V' = V \cup \{o_1, \dots, o_m\}$ where o_j is the output variable of gate j for $1 \leq j \leq m$. Wlog we can assume that $o = o_m$. For $1 \leq j \leq m$, we denote gate g_j by $g_j(\diamond, x_1, x_2, o_j)$ with $x_1, x_2 \in V'$ and $\diamond \in \{\vee, \wedge\}$ or by $g_j(\neg, x, o_j)$ with $x \in V'$. As C is acyclic, one can assume that $x_1, x_2, x \in V \cup \{o_1, o_2, \dots, o_{j-1}\}$.

Figure 6.9: P_j when $g_j(\vee, x_1, x_2, o_j)$.Figure 6.10: P_j when $g_j(\neg, x, o_j)$.Figure 6.11: P_j when $g_j(\wedge, x_1, x_2, o_j)$.

For some $x \in V'$, we denote by $\text{bv}(x)$ the boolean value of x with respect to the input B . Note that if $x \in V$, there exists $1 \leq i \leq n$ such that $x = v_i$ and so $\text{bv}(x) = b_i$.

From a CVP instance C , we construct a protocol in which variables, along with their boolean assignments, can be broadcast at will from the initial state. To simulate a gate, a process moves to a state where it waits for the required messages representing the truth values of the gate's inputs. Once the relevant messages are received, it broadcasts the corresponding output. For instance, if g_j is an 'or' gate of the form $g_j(\vee, x_1, x_2, o_j)$, and the process receives $(x_1, \top)$, it transitions to a state from which it can broadcast $(o_j, \top)$ at will. Conversely, if it receives both $(x_1, \perp)$ and $(x_2, \perp)$, it moves to a state that can broadcast $(o_j, \perp)$. In this construction, the only messages that can be broadcast correspond precisely to the boolean evaluation of the variables. In particular, a message (o, b) can be broadcast if and only if o evaluates to b .

Let $1 \leq j \leq m$, we describe $P_j = (Q_j, \Sigma_j, T_j)$ (and depict it in Figures 6.9 to 6.11) where $\Sigma_j = V' \times \{\top, \perp\}$ as follows:

- **(or gate)** if $g_j(\vee, x_1, x_2, o_j)$ (Figure 6.9):
 - $Q_j = \{q_0^j, q_\top^j, q_1^j, q_\perp^j\}$, and
 - $T_j = \{(q_0^j, ?(x_k, \top), q_\top^j) \mid k = 1, 2\} \cup \{(q_0^j, ?(x_1, \perp), q_1^j), (q_1^j, ?(x_2, \perp), q_\perp^j)\} \cup \{(q_\top^j, !!(\text{or}_j, \top), q_\top^j), (q_\perp^j, !!(\text{or}_j, \perp), q_\perp^j)\}$;
- **(not gate)** if $g_j(\neg, x, o_j)$ (Figure 6.10):
 - $Q_j = \{q_0^j, q_\top^j, q_\perp^j\}$, and
 - $T_j = \{(q_0^j, ?(x, \perp), q_\top^j), (q_0^j, ?(x, \top), q_\perp^j)\} \cup \{(q_\top^j, !!(\text{not}_j, \top), q_\top^j), (q_\perp^j, !!(\text{not}_j, \perp), q_\perp^j)\}$.
- **(and gate)** if $g_j(\wedge, x_1, x_2, o_j)$ (Figure 6.11):
 - $Q_j = \{q_0^j, q_\top^j, q_1^j, q_\perp^j\}$, and
 - $T_j = \{(q_0^j, ?(x_k, \perp), q_\perp^j) \mid k = 1, 2\} \cup \{(q_0^j, ?(x_1, \top), q_1^j), (q_1^j, ?(x_2, \top), q_\top^j)\} \cup \{(q_\top^j, !!(\text{and}_j, \top), q_\top^j), (q_\perp^j, !!(\text{and}_j, \perp), q_\perp^j)\}$;

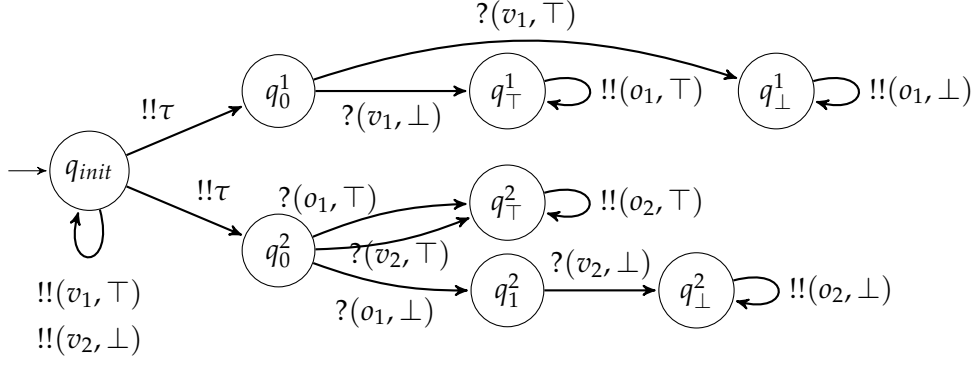


Figure 6.12: Protocol for a CVP instance with two variables v_1, v_2 , two gates $g_1(\neg, v_1, o_1)$ and $g_2(\vee, o_1, v_2, o_2)$, and input $\top$ for v_1 and $\perp$ for v_2 and output variable o_2 . Depending on the truth value of o_2 to test, the state we ask to cover can be $q_{\perp}^2$ or $q_{\top}^2$.

We are now ready to define the protocol associated to C , $P_C = (Q, \Sigma, q_{init}, T)$:

- $Q = \{q_{init}\} \cup \bigcup_{1 \leq j \leq m} Q_j$;
- $\Sigma = V' \times \{\top, \perp\}$;
- $T = \{(q_{init}, !!(\tau, b_j), q_{init}) \mid 1 \leq j \leq n\} \cup \{(q_{init}, !!\tau, q_0^j) \mid 1 \leq j \leq m\} \cup \bigcup_{1 \leq j \leq m} T_j$. The message τ is never received and only serves to simulate internal transitions.

Observe that P is Wait-Only: q_{init} is an action state, and for all $1 \leq j \leq m$: if g_j is a “not” gate, q_0^j is a waiting state, and $q_{\perp}^j, q_{\top}^j$ are action states, and if g_j is an “and” gate or an “or” gate, $q_{\perp}^j, q_{\top}^j$ are action states and q_0^j, q_1^j are waiting states. We recall that $|P| = |T|(\log_2(|Q|) + \log_2(|\Sigma|))$. Observe that $|P| \in O(m^2 + n \times m)$.

Example 6.4. Take for example a CVP instance C with two variables v_1, v_2 , and two gates: one *not* gate on variable v_1 denoted $g_1(v_1, \neg, o_1)$ (where o_1 stands for the output variable of g_1), and one *or* gate on variable v_2 and o_1 denoted $g_2(o_1, v_2, \vee, o_2)$ (where o_2 stands for the output variable of gate g_2). Assume the input boolean value for v_1 [resp. v_2] is $\top$ [resp. $\perp$]. The protocol associated to C is displayed on Figure 6.12. Assume the output value of C is o_2 , we will show that $q_{\top}^2$ [resp. $q_{\perp}^2$] is coverable if and only if o_2 evaluates to $\top$ [resp. $\perp$]. Note that with the truth assignment depicted earlier, o_2 evaluates to $\perp$, and indeed one can build an execution covering $q_{\perp}^2$ with three processes:

$$\begin{aligned} \{3 \cdot q_{init}\} &\rightarrow \{2 \cdot q_{init}, q_0^2\} \rightarrow \{q_{init}, q_0^1, q_0^2\} \xrightarrow{(q_{init}, !!(\tau, \top), q_{init})} \{q_{init}, q_{\perp}^1, q_0^2\} \\ &\xrightarrow{(q_{\perp}^1, !!(\tau, \perp), q_{\perp}^1)} \{q_{init}, q_{\perp}^1, q_1^2\} \xrightarrow{(q_{init}, !!(\tau, \perp), q_{init})} \{q_{init}, q_{\perp}^1, q_{\perp}^2\}. \end{aligned}$$

We now prove that $\text{bv}(o) = b$ if and only if there is an initial configuration $M_{init} \in \mathcal{M}_{init}$ and $M_f \in \mathcal{M}$ such that $M_{init} \rightarrow^* M_f$ and $M_f(q_b^m) > 0$.

Lemma 6.5. If $\text{bv}(o) = b$, then there exists $M_{init} \in \mathcal{M}_{init}$ and $M_f \in \mathcal{M}$ such that $M_{init} \rightarrow^* M_f$ and $M_f(q_b^m) > 0$.

Proof. Assume that $\text{bv}(o) = b$, and take $C_0 = \zeta(m+1) \cdot q_{\text{init}}\S$. There exists an execution $M_{\text{init}} \xrightarrow{*} M_f$ with $M_f = \zeta q_{\text{init}}, q_{y_1}^1, q_{y_2}^2, \dots, q_{y_m}^m\S$ where $y_j = \text{bv}(o_j)$ with the input boolean values B for $1 \leq j \leq m$. By definition, $y_m = \text{bv}(o) = b$. The execution is: $M_{\text{init}} \xrightarrow{+} M_1 \xrightarrow{+} \dots \xrightarrow{+} M_m$ where $M_j = \zeta q_{\text{init}}, q_{y_1}^1, \dots, q_{y_j}^j, q_{\text{init}}, \dots, q_{\text{init}}\S$ for all $0 \leq j < m$. Between M_j and M_{j+1} , the sequence of transitions is:

- if $g_{j+1}(\vee, x_1, x_2, o_{j+1})$ with $\text{bv}(x_k) = \top$ for some $k \in \{1, 2\}$, then $\text{bv}(o_{j+1}) = \top$. Either $x_k = v_i$ (and $b_i = \top$) for some $1 \leq i \leq n$, or $x_k = o_i$ (and $y_i = \top$) for some $1 \leq i \leq j$. In the first case, as $M_j(q_{\text{init}}) \geq 2$, then consider the sequence $M_j \xrightarrow{(q_{\text{init}}, !!\tau, q_0^{j+1})} M'_j \xrightarrow{(q_{\text{init}}, !!(v_i, \top), q_{\text{init}})} M_{j+1}$. It holds that $M'_j = M_j - \zeta q_{\text{init}}\S + \zeta q_0^{j+1}\S$ and $M_{j+1} = M'_j - \zeta q_0^{j+1}\S + \zeta q_{\top}^{j+1}\S$. Hence $M_{j+1} = \zeta q_{\text{init}}, q_{y_1}^1, \dots, q_{y_j}^j, q_{y_{j+1}}^{j+1}, \dots, q_{\text{init}}\S$.

In the second case, $M_j(q_{\top}^i) > 0$ as $i \leq j$ and $\top = \text{bv}(o_i)$, and $M_j(q_{\text{init}}) > 0$.

Consider the sequence $M_j \xrightarrow{(q_{\text{init}}, !!\tau, q_0^{j+1})} M'_j \xrightarrow{(q_{\top}^i, !!(o_i, \top), q_{\top}^i)} M_{j+1}$. It holds that $M'_j = M_j - \zeta q_{\text{init}}\S + \zeta q_0^{j+1}\S$ and $M_{j+1} = M'_j - \zeta q_0^{j+1}\S + \zeta q_{\top}^{j+1}\S$. Hence $M_{j+1} = \zeta q_{\text{init}}, q_{y_1}^1, \dots, q_{y_j}^j, q_{y_{j+1}}^{j+1}, \dots, q_{\text{init}}\S$.

- if $g_{j+1}(\wedge, x_1, x_2, o_{j+1})$ with $\text{bv}(x_k) = \perp$ for some $k = 1, 2$, then $\text{bv}(o_{j+1}) = \perp = y_{j+1}$. The sequence of transitions is built in an analogous way than the previous case, however this time the broadcast messages are τ and $(x_k, \perp)$ and the reached state is $q_{\perp}^{j+1}$.

- if $g_{j+1}(\vee, x_1, x_2, o_{j+1})$ with $\text{bv}(x_1) = \text{bv}(x_2) = \perp$, then $\text{bv}(o_{j+1}) = \perp$. Either $x_1 = v_i$ (and $b_i = \perp$) for some $1 \leq i \leq n$, or $x_1 = o_i$ (and $y_i = \perp$) for some $1 \leq i \leq j$. In the first case, as $M_j(q_{\text{init}}) \geq 2$, then consider the sequence $M_j \xrightarrow{(q_{\text{init}}, !!\tau, q_0^{j+1})} M_{j,1} \xrightarrow{(q_{\text{init}}, !!(v_i, \perp), q_{\text{init}})} M_{j,2}$. It holds that $M_{j,1} = M_j - \zeta q_{\text{init}}\S + \zeta q_0^{j+1}\S$ and $M_{j,2} = M_{j,1} - \zeta q_0^{j+1}\S + \zeta q_{\perp}^{j+1}\S$.

If $x_1 = o_i$ for some $i \leq j$, $M_j(q_{\perp}^i) > 0$ as $i \leq j$ and $\perp = \text{bv}(o_i)$, and $M_j(q_{\text{init}}) > 0$.

Consider the sequence $M_j \xrightarrow{(q_{\text{init}}, !!\tau, q_0^{j+1})} M_{j,1} \xrightarrow{(q_{\perp}^i, !!(o_i, \perp), q_{\perp}^i)} M_{j,2}$. It holds that $M_{j,1} = M_j - \zeta q_{\text{init}}\S + \zeta q_0^{j+1}\S$ and $M_{j,2} = M_{j,1} - \zeta q_0^{j+1}\S + \zeta q_{\perp}^{j+1}\S$.

In both cases, $M_{j,2} = \zeta q_{\text{init}}, q_{y_1}^1, \dots, q_{y_j}^j, q_{\perp}^{j+1}, \dots, q_{\text{init}}\S$.

We make the same cases distinctions in order to build the configuration M_{j+1} such that $M_{j,2} \xrightarrow{(q, !!(x_2, \perp), q)} M_{j+1}$ where $q = q_{\text{init}}$ if $x_2 \in V$, and otherwise $x_2 = o_i$ for some $i \leq j$ and $q = q_{\perp}^i$.

It holds that $M_{j+1} = M_{j,2} - \zeta q_{\perp}^{j+1}\S + \zeta q_{\perp}^{j+1}\S$, hence $M_{j+1} = \zeta q_{\text{init}}, q_{y_1}^1, \dots, q_{y_j}^j, q_{y_{j+1}}^{j+1}, \dots, q_{\text{init}}\S$.

- if $g_{j+1}(\wedge, x_1, x_2, o_{j+1})$ with $\text{bv}(x_1) = \text{bv}(x_2) = \top$, then $\text{bv}(o_{j+1}) = \top$. The sequence of transitions is built in an analogous way than the previous case, however this time the broadcast messages are τ , $(x_1, \top)$ and $(x_2, \top)$ and the reached state is $q_{\top}^{j+1}$.
- if $g_{j+1}(\neg, x, o_{j+1})$ with $\text{bv}(x) = \top$ (resp. $\perp$), then $\text{bv}(o_{j+1}) = \perp$ (resp. $\top$). Either $x = v_i$ for some $1 \leq i \leq n$, and as $M_j(q_{\text{init}}) \geq 2$, we build the following sequence:

$M_j \xrightarrow{(q_{init}, !!\tau, q_0^{j+1})} M'_j \xrightarrow{(q_{init}, (v_i, b_i), q_{init})} M_{j+1}$ where $\bar{b}_i = \perp$ (resp. $\bar{b}_i = \top$). It holds that $M'_j = M_j - \wr q_{init} \wr + \wr q_0^{j+1} \wr$ and $M_{j+1} = M'_j - \wr q_0^{j+1} \wr + \wr q_{\bar{b}_i}^{j+1} \wr$.

Otherwise, $x = o_i$ for some $i \leq j$, and so $M_j(q_{\top}^i) > 0$ (resp. $M_j(q_{\perp}^i) > 0$) and $M_j(q_{init}) > 0$. Hence, we can build the following sequence: $M_j \xrightarrow{(q_{init}, !!\tau, q_0^{j+1})} M'_j \xrightarrow{(q_{init}, (o_i, \text{bv}(o_i)), q_{init})} M_{j+1}$. It holds that $M'_j = M_j - \wr q_{init} \wr + \wr q_0^{j+1} \wr$ and $M_{j+1} = M'_j - \wr q_0^{j+1} \wr + \wr q_{\bar{b}_i}^{j+1} \wr$.

In both cases, $M_{j+1} = \wr q_{init}, q_{y_1}^1, \dots, q_{y_j}^j, q_{y_{j+1}}^{j+1}, \dots, q_{init} \wr$.

Hence, $M_0 \xrightarrow{+} M_m$ where $M_m(q_{y_m}^m) > 0$, and so if $b = y_m$, $M_m(q_b^m) > 0$. $\square$

Lemma 6.6. If there exists $M_{init} \in \mathcal{M}_{init}$ and $M_f \in \mathcal{M}$ such that $M_{init} \xrightarrow{*} M_f$ and $M_f(q_b^m) > 0$, then $\text{bv}(o) = b$.

Proof. Let $M_{init} \in \mathcal{M}_{init}$ and $M_f \in \mathcal{M}$ such that $M_{init} \xrightarrow{*} M_f$ and $M_f(q_b^m) > 0$.

First we show that all broadcast messages $(x, b_x) \in V' \times \{\top, \perp\}$ are such that $b_x = \text{bv}(x)$. We start by proving it for $x \in V$, and then proceed to prove it for $x \in \{o_1, \dots, o_m\}$ by induction on $k \in [1, m]$.

Let (x, b_x) be a broadcast message such that $x \in V$, we note $x = v_i$ with $1 \leq i \leq n$. By construction of the protocol P , the only broadcast transition labelled with (v_i, b) for some $b \in \{\top, \perp\}$ is the transition $(q_{init}, !!(v_i, b_i), q_{init})$ where b_i is the input boolean value for variable v_i , i.e. $\text{bv}(v_i) = b_i$. Hence, all broadcast messages (x, b_x) with $x \in V$, are such that $b_x = \text{bv}(x)$.

We prove now that for all $(x, b_x) \in \{o_1, \dots, o_m\} \times \{\top, \perp\}$ broadcast in the execution, $b_x = \text{bv}(x)$ and we do so by induction on $k \in [1, m]$. For $k = 1$, we have that $x = o_1$. Note that tuples containing o_1 can only be broadcast from $q_{\perp}^1$ or $q_{\top}^1$. Let $g_1(\diamond, x_1, x_2, o_1)$ or $g_1(\neg, x_3, o_1)$ with $\diamond \in \{\vee, \wedge\}$ be the corresponding gate, with $x_1, x_2, x_3 \in V$ by acyclicity.

- if $\diamond = \vee$, then a process reaching $q_{\top}^1$ has necessarily received $(x_1, \top)$ or $(x_2, \top)$, and a process reaching $q_{\perp}^1$ has necessarily received $(x_1, \perp)$ and $(x_2, \perp)$. As we proved, all broadcast messages containing x_1 (resp. x_2) are of the form $(x_1, \text{bv}(x_1))$ (resp. $(x_2, \text{bv}(x_2))$). Hence, only one state between $q_{\top}^1$ and $q_{\perp}^1$ is reachable. If it is $q_{\top}^1$ (resp. $q_{\perp}^1$), the only messages containing o_1 which can be broadcast are $(o_1, \top)$ (resp. $(o_1, \perp)$) and it holds that $\top = \text{bv}(x_1) \vee \text{bv}(x_2) = \text{bv}(o_1)$ (resp. $\perp$) as the process on $q_{\top}^1$ (resp. $q_{\perp}^1$) received either $(x_1, \top)$ or $(x_2, \top)$ (resp. $(x_1, \perp)$ and $(x_2, \perp)$);
- if $\diamond = \wedge$, the argument is analogous to the previous case;
- if $\diamond = \neg$, then a process reaching $q_{\top}^1$ has necessarily received $(x_3, \perp)$ and a process reaching $q_{\perp}^1$ has necessarily received $(x_3, \top)$. As we proved, all broadcast messages containing x_3 are of the form $(x_3, \text{bv}(x_3))$. Hence, only one state between $q_{\top}^1$ and $q_{\perp}^1$ is reachable. If it is $q_{\top}^1$ (resp. $q_{\perp}^1$), the only messages containing o_1 which can be broadcast are $(o_1, \top)$ (resp. $(o_1, \perp)$) and it holds that $\top = \neg \text{bv}(x_3) = \text{bv}(o_1)$ (resp. $\perp$) as the process on $q_{\top}^1$ (resp. $q_{\perp}^1$) received $(x_3, \perp)$ (resp. $(x_3, \top)$).

Assume the property true for $k < m$ gates, and let (o_{k+1}, b_{k+1}) a broadcast message and note $g_{k+1}(\diamond, x_1, x_2, o_{k+1})$ with $\diamond \in \{\wedge, \vee\}$, or $g_{k+1}(\neg, x_3, o_{k+1})$ with $x_1, x_2, x_3 \in V \cup \{o_1, \dots, k\}$. By induction hypothesis, the only broadcast messages containing x_1, x_2 or x_3 are $(x_1, \text{bv}(x_1))$, $(x_2, \text{bv}(x_2))$, and $(x_3, \text{bv}(x_3))$. The arguments are then the same than in the case $k = 1$.

To conclude, if q_b^m is reached, the message (o_m, b) can be broadcast, hence $b = \text{bv}(o_m) = \text{bv}(o)$. $\square$

Hence, we have proved with the two previous lemmas that $\text{bv}(o) = b$ if and only if q_b^m is coverable. Note that the protocol defined here is a Wait-Only broadcast protocol, hence we get the following theorem.

Theorem 6.4. *STATE-COVER^{nb} restricted to Wait-Only (broadcast) protocols is PTIME-hard.*

If we transform the Wait-Only protocol presented here into a Wait-Only rendez-vous protocol (by transforming all the broadcast transitions into sending transitions on the same message), the reduction remains sound and complete. Indeed, in the execution of P_C built in Lemma 6.5, all the broadcasts are received by only one process at a time. Hence, the same execution is possible by replacing broadcasts transitions by sending transitions. The proof of Lemma 6.6 works exactly the same way if the broadcasts transitions become sending transitions. Hence, we get the following corollary.

Corollary 6.2. *STATE-COVER^{nb} and CONF-COVER^{nb} restricted to Wait-Only rendez-vous protocols are PTIME-hard.*

Together with Theorem 6.3, we get the following theorem.

Theorem 6.5. *STATE-COVER^{nb} for Wait-Only protocols is PTIME-complete. This result holds when restricting protocols to Wait-Only rendez-vous protocols or Wait-Only broadcast protocols.*

6.4 CONFIGURATION COVERABILITY

In this section, we present the first complexity result that differentiates the state coverability problem from the configuration coverability problem.

Configuration coverability is a generalization of state coverability, so naturally, the complexity of the state coverability problem is always at most that of configuration coverability. As discussed in Chapter 2, in the general case (protocols with broadcast transitions and sending transitions), there is also a reduction in the other direction: configuration coverability reduces to state coverability. The idea is as follows: given a protocol P and a multiset to cover $M = \{q_1, \dots, q_k\}$, one could augment P with a sequence of states that allows an initial state to broadcast a “halt” message, received by all states, halting the execution of other processes. From there, the process that broadcasts “halt” can wait for messages from witness processes, one for each component $q_1, \dots, q_k$ of the multiset. Once the messages are received, the final state is covered. This shows that, in the general case, both problems are equivalent and have the same complexity. So, why is this different now?

In Wait-Only protocols, this reduction does not hold because the protocol constructed from P is inherently not Wait-Only. Adding a reception transition for the “halt” message to every state turns all action states into states with both outgoing action transitions (labeled

by elements of $!\Sigma \cup !!(\Sigma)$ and reception transitions. Simply adding reception transitions to waiting states, while allowing processes in action states to broadcast the witness message, is not enough. It desynchronizes the simulation. Each time a process in a waiting state witnesses the coverability of M , a new process can broadcast “halt”, putting this witness in a safe state where no further receptions can occur. At this point, a new execution can be simulated with fresh processes in the initial state (remember, q_{init} is an action state). Thus, in Wait-Only protocols, there is a clear distinction between the two problems, as demonstrated by the result established in this section.

We present an algorithm to solve the configuration coverability problem for Wait-Only protocols, using polynomial space in the size of the protocol and polylogarithmic space in the size of the multiset to cover.

6.4.1 Main ideas

For the remainder of the section, we fix a Wait-Only protocol $P = (Q, \Sigma, q_{init}, T)$ and a configuration $M_f \in \mathcal{M}$ to cover, where $K = |M_f|$. The intuition is the following: we (only) keep track of the K processes that will cover M_f . Of course, to reach the desired configuration, they might require other processes, especially if they need to receive messages. That is why we also maintain the set of reachable states along the execution. An abstract configuration will then be a multiset of K states (concrete part of the configuration) and a set of states (each of them being coverable). Proposition 6.1 ensures that it is enough to know which action states are reachable to ensure that both the concrete part and the action states of the abstract part are coverable at the same time. There is one case where this abstraction is insufficient: if one of the K processes sends a message that *should not* be received by the other $K - 1$ processes, and this message is received by a process in the abstracted part of the configuration. In this case, even if the (waiting) state is present in the set of reachable states, Proposition 6.1 does not guarantee that the configuration is reachable, and the transition to an abstract configuration where none of the $K - 1$ processes has received the message might be incorrect. This is why in that case we need to precisely keep track of the process that will receive the message, even if in the end it will not participate in the covering of M_f . This leads to the definition of the $\Rightarrow_{\text{switch}}$ transition below.

The proof is structured as follows: we present the formal definitions of the abstract configurations and semantic in Section 6.4.2. In Section 6.4.3, we provide the completeness proof, Section 6.4.4 is devoted to prove the soundness of the construction. In the latter, we also give some ingredients to prove an upper bound on the number of processes needed to cover the configuration. In Section 6.4.5, we present the main theorem of this section: it states that the CONF-COVER^{nb} problem is in PSPACE, and if the configuration is indeed coverable, it provides an upper bound on the number of processes needed to cover it. In Section 6.4.6, we prove that this bound is tight as the problem is PSPACE-hard.

6.4.2 Reasoning with Abstract Configurations

We present the abstract configurations we rely on. Consider the Wait-Only protocol P , a multiset configuration to cover M_f and $K = |M_f|$.

Definition 6.4. An **abstract configuration** γ is a pair (c, S) where c is a configuration in $\mathcal{M}$ such that $|c| = K$ and $S \subseteq Q$ is a subset of control states such that $\{q \in Q \mid c(q) > 0\} \subseteq S$.

We call c the c -part of γ and S its S -part. We denote by Γ the set of abstract configurations and by γ_{in} the initial abstract configuration $\gamma_{in} = (\lambda K \cdot q_{init}, \{q_{init}\})$.

An abstract configuration $\gamma = (c, S)$ represents a set of configurations $\llbracket \gamma \rrbracket$ defined by:

$$\llbracket \gamma \rrbracket = \{M \in \mathcal{M} \mid c \leq M \text{ and } M(q) > 0 \text{ implies } q \in S\}$$

Hence in $\llbracket \gamma \rrbracket$, we include all configurations M that are greater than or equal to c , as long as the states holding processes in M are stored in S (observe that this implies that all the states in c appear in S).

We now define an abstract transition relation for abstract configurations. For this matter, we define three transition relations $\Rightarrow_{\text{step}}, \Rightarrow_{\text{ext}}$ and $\Rightarrow_{\text{switch}}$ and let $\Rightarrow$ be defined by $\Rightarrow_{\text{step}} \cup \Rightarrow_{\text{ext}} \cup \Rightarrow_{\text{switch}}$. Consider $\gamma = (c, S)$ and $\gamma' = (c', S')$, two abstract configurations and $t = (q, \alpha, q')$ a transition in T with $\alpha = !a$ or $\alpha = !!a$. For $\kappa \in \{\text{step}, \text{ext}, \text{switch}\}$, we have $\gamma \xRightarrow{\kappa} \gamma'$ iff all the following conditions hold:

- $S \subseteq S'$, and,
- for all $p \in S' \setminus S$, either $p = q'$ or there exist $p' \in S$ and $(p', ?a, p)$ in T , and,
- one of the following cases is true:
 - $\kappa = \text{step}$ and $c \xrightarrow{t} c'$. This relation describes a message emitted from the c -part of the configuration;
 - $\kappa = \text{ext}$ and one of the following conditions holds:
 - * $q \in S$ and $c + \lambda q \xrightarrow{t} c' + \lambda q'$, or
 - * $\alpha = !a$ and there exists $(p, ?a, p')$ in T such that $q, p \in S$ and $c + \lambda q, p \xrightarrow{t} c' + \lambda q', p'$ (note that in that case $c = c'$).

The relation ext hence describes a message emitted from the S -part of the configuration;

- $\kappa = \text{switch}$, $q \in S$, $\alpha = !a$, and there exists $t' = (p, ?a, p') \in T$ such that:
 - * $c(p) > 0$,
 - * $c'(q') > 0$,
 - * $c - \lambda p = c' - \lambda q'$, and
 - * $c + \lambda q \xrightarrow{t} c' + \lambda p'$.

This relation models a rendez-vous where the sender comes from a state in the S -part of the abstract configuration, and the receiver is a process from c . After the interaction, we replace the receiver with the sender: hence the name “switch”. We need this switch transition because we do not want to allow ext transitions where a rendez-vous reception is handled by an external process while the sending is done by a process in c . This restriction is necessary when some processes (needed to cover the target configuration) must remain in waiting states where a is receivable, while another process must perform the actual sending. In such situations, we want to ensure that the message is received by a process that is not one of those needed for the final coverability. However, the copypaste property only ensures that for a coverable configuration c and an action state q , the configuration $c + \lambda q$ is also coverable. If we allowed external receptions (i.e., ext transitions like $c + \lambda p \xrightarrow{t} c' + \lambda p'$ with p being a waiting state), this

guarantee would fail: we would be unable to prove that $c + \wr p$ is itself coverable. To circumvent this, switch transitions let us preserve the waiting states intact while still making progress: the sender takes over the role of the receiver. This ensures that $c + \wr q$ remains coverable.

Note that in any case, q , the state from which the message is sent, belongs to S . We then write $\gamma \xRightarrow{t} \gamma'$ whenever $\gamma \xRightarrow{t}_\kappa \gamma'$ for $\kappa \in \{\text{step}, \text{ext}, \text{switch}\}$ and we do not always specify the transition t that is used. We denote by $\Rightarrow^*$ the reflexive and transitive closure of $\Rightarrow$.

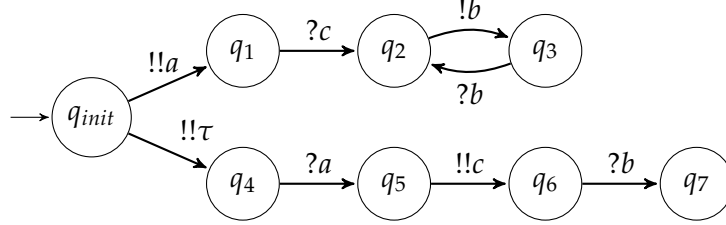


Figure 6.13: A Wait-Only protocol P' .

Example 6.5. We consider the Wait-Only protocol P' depicted on Figure 6.13 with set of states Q' . We want to cover $M_f = \wr q_3, q_3, q_6$ (hence $K = 3$).

In this example, the abstract configuration $\gamma = (\wr q_2, q_2, q_4, \{q_{init}, q_1, q_2, q_4, q_5, q_6\})$ represents all the configurations of P' with at least two processes on q_2 and one on q_4 , and no process on q_3 nor q_7 .

Considering the following abstract execution, we can cover M_f :

$$\begin{aligned}
 \gamma_{in} &\xrightarrow[\text{step}]{(q_{init}, !!\tau, q_4)} (\wr q_4, q_{init}, q_{init}, \{q_{init}, q_4\}) \xrightarrow[\text{step}]{(q_{init}, !!\tau, q_4)} (\wr q_4, q_4, q_{init}, \{q_{init}, q_4\}) \\
 &\xrightarrow[\text{step}]{(q_{init}, !!a, q_1)} (\wr q_5, q_5, q_1, \{q_{init}, q_1, q_4, q_5\}) \xrightarrow[\text{ext}]{(q_5, !!c, q_6)} (\wr q_5, q_5, q_2, Q' \setminus \{q_3, q_7\}) \\
 &\xrightarrow[\text{step}]{(q_2, !b, q_3)} (\wr q_5, q_5, q_3, Q') \xrightarrow[\text{step}]{(q_5, !!c, q_6)} (\wr q_6, q_5, q_3, Q') \\
 &\xrightarrow[\text{switch}]{(q_2, !b, q_3)} (\wr q_3, q_5, q_3, Q') \xrightarrow[\text{step}]{(q_5, !!c, q_6)} (\wr q_3, q_6, q_3, Q')
 \end{aligned}$$

It corresponds for instance to the following concrete execution:

$$\begin{aligned}
 \wr q_{init}, q_{init}, q_{init} &+ \wr q_{init}, q_{init} \rightsquigarrow^+ \wr q_4, q_4, q_{init} + \wr q_4, q_{init} \xrightarrow{(q_{init}, !!a, q_1)} \wr q_5, q_5, q_1 + \wr q_5, q_{init} \\
 &\xrightarrow{(q_{init}, !!a, q_1)} \wr q_5, q_5, q_1 + \wr q_5, q_1 \xrightarrow{(q_5, !!c, q_6)} \wr q_5, q_5, q_2 + \wr q_6, q_1 \\
 &\xrightarrow{(q_2, !b, q_3)} \wr q_5, q_5, q_3 + \wr q_7, q_1 \xrightarrow{(q_5, !!c, q_6)} \wr q_6, q_5, q_3 + \wr q_7, q_2 \\
 &\xrightarrow{(q_2, !b, q_3)} \wr q_3, q_5, q_3 + \wr q_7, q_7 \xrightarrow{(q_5, !!c, q_6)} \wr q_3, q_6, q_3 + \wr q_7, q_7
 \end{aligned}$$

The c-part of our abstract configurations is pictured in the first part of the multiset configurations of the concrete execution. We start with two more processes as in the abstract execution there are exactly two steps requiring an additional process ($\xrightarrow[\text{ext}]{(q_5, !!c, q_6)}$ and then

$$\xrightarrow[\text{switch}]{(q_2, !b, q_3)}).$$

The c-part of our abstract configuration $\langle q_6, q_5, q_3 \rangle$ reached just before the $\Rightarrow_{\text{switch}}$ transition does not correspond to the set of processes that finally cover M_f , at this point of time, the processes that will finally cover M_f are in states q_2 , q_5 , and q_3 . But here we ensure that the process on q_6 will actually receive the b sent by the process on q_2 , leaving the process on q_3 in its state. Once this has been ensured, process on q_6 is not useful anymore, and instead we follow the process that was on q_2 before the sending, hence the $\Rightarrow_{\text{switch}}$ transition.

The algorithm used to solve CONF-COVER^{nb} , is then to seek in the directed graph $(\Gamma, \Rightarrow)$ if a vertex of the form (M_f, S) is reachable from γ_{in} .

Before proving that this algorithm is correct, we establish the following property.

Lemma 6.7. Let (c, S) and (c', S') be two abstract configurations and $\tilde{S} \subseteq Q$ such that $S \subseteq \tilde{S}$. We have:

1. $\llbracket (c, S) \rrbracket \subseteq \llbracket (c, \tilde{S}) \rrbracket$.
2. If $(c, S) \Rightarrow (c', S')$ then there exists $S'' \subseteq Q$ such that $(c, \tilde{S}) \Rightarrow (c', S'')$ and $S' \subseteq S''$.

Proof. The first point is a direct consequence of the definition of $\llbracket \cdot \rrbracket$. For the second point, it is enough to take $S'' = S' \cup \tilde{S}$ and apply the definition of $\Rightarrow$. $\square$

6.4.3 Completeness of the algorithm

In this subsection we show that if M_f can be covered then there exists an abstract configuration $\gamma = (M_f, S)$ such that $\gamma_{in} \Rightarrow^* \gamma$. We use $\mathcal{M}_{\geq K}$ to represent the set $\{M \in \mathcal{M} \mid |M| \geq K\}$ of configurations with at least K processes and recall that $\mathcal{M}_K$ is the set $\{M \in \mathcal{M} \mid |M| = K\}$ of configurations with exactly K processes. This first lemma shows the completeness for a single step of our abstract transition relation (note that we focus on the c-part, as it is the one witnessing M_f in the end).

Lemma 6.8. Let $M, M' \in \mathcal{M}_{\geq K}$ and $t \in T$ such that $M \xrightarrow{t} M'$. Then for all $c' \in \mathcal{M}_K$ such that $c' \leq M'$, there exists $c \in \mathcal{M}_K$ and $S' \subseteq Q$ such that $(c, S) \Rightarrow (c', S')$ with $S = \{q \in Q \mid M(q) > 0\}$, $M \in \llbracket (c, S) \rrbracket$ and $M' \in \llbracket (c', S') \rrbracket$.

Proof. Let $c' \in \mathcal{M}_K$ such that $c' \leq M'$. We assume that $t = (q, \alpha, q')$ with $\alpha \in \{!a, !!a\}$. We let $S = \{p \in Q \mid M(p) > 0\}$ and $S' = S \cup \{p \in Q \mid M'(p) > 0\}$. By definition of $\llbracket \cdot \rrbracket$, we have that $M' \in \llbracket (c', S') \rrbracket$. By definition of S and S' , for all $p \in S' \setminus S$, either $p = q'$ or there exists $p' \in S$ and $(p', ?a, p)$ in T . In fact, let $p \in S' \setminus S$ such that $p \neq q'$. Since $M \xrightarrow{t} M'$, we have necessarily that there exists $p' \in Q$ such that $M(p') > 0$ (hence $p' \in S$), and $(p', ?a, p)$ in T . We now reason by a case analysis to determine $c \in \mathcal{M}_K$ such that $(c, S) \Rightarrow (c', S')$ and $M \in \llbracket (c, S) \rrbracket$. The different cases are: (i) $\alpha = !a$ and the message is received by a process, (ii) $\alpha = !!a$, and (iii) $\alpha = !a$ and the message is *not received*. The case exhibiting the most different abstract behaviors is case (i). Hence, we present it first, as we believe other cases can be skipped.

- (i) if $\alpha = !a$ and the message is received by a process (i.e. it is a rendez-vous), denote by $(p, ?a, p')$ the reception that has occurred between M and M' . Using

the definition of $\succrightarrow$, we get $M' = M - \downarrow q, p \rangle + \downarrow q', p' \rangle$. We consider the four following disjoint cases:

- $c'(q') = 0$ and $c'(p') = 0$. Since $\downarrow q', p' \rangle \leq M'$ and $c' \leq M'$, we get that $M' = c' + \downarrow q', p' \rangle + c_2$ for some multiset c_2 . We deduce that $M = c' + \downarrow q, p \rangle + c_2$. This allows us to deduce that $c' \leq M$ and consequently $M \in \llbracket (c', S) \rrbracket$. Moreover, $c' + \downarrow p, q \rangle \xrightarrow{t} c' + \downarrow q', p' \rangle$ and $q, p \in S$. Hence $(c', S) \xrightarrow{t}_{\text{ext}} (c', S')$.
- $c'(q') = 0$ and $c'(p') > 0$. In that case $M' = c' + \downarrow q' \rangle + c_2$ for some multiset c_2 . Let $c = c' - \downarrow p' \rangle + \downarrow p \rangle$. We have then

$$\begin{aligned} M &= M' + \downarrow q, p \rangle - \downarrow q', p' \rangle \\ &= c' + \downarrow q' \rangle + c_2 + \downarrow q, p \rangle - \downarrow q', p' \rangle \\ &= c' + c_2 - \downarrow p' \rangle + \downarrow p \rangle + \downarrow q \rangle \\ &= c + \downarrow q \rangle + c_2. \end{aligned}$$

This allows us to deduce that $c \leq M$ and consequently $M \in \llbracket (c, S) \rrbracket$. Furthermore $q \in S$ and $c + \downarrow q \rangle \xrightarrow{t} c' + \downarrow q' \rangle$. Hence $(c, S) \xrightarrow{t}_{\text{ext}} (c', S')$.

- $c'(q') > 0$ and $c'(p') = 0$. In that case $M' = c' + \downarrow p' \rangle + c_2$ for some multiset c_2 . Let $c = c' - \downarrow q' \rangle + \downarrow p \rangle$. Observe that, if we take $c = c - \downarrow q' \rangle + \downarrow q \rangle$, we can not have $(c, S) \Rightarrow_{\text{ext}} (c', S')$ (because the sender is accounted in the c -part) neither than $(c, S) \Rightarrow_{\text{step}} (c', S')$: if there exists another state $s \in Q_?(a)$, one process on s would need to receive the message to have $c \xrightarrow{t} c'$. We have then

$$\begin{aligned} M &= M' + \downarrow q, p \rangle - \downarrow q', p' \rangle \\ &= c' + \downarrow p' \rangle + c_2 + \downarrow q, p \rangle - \downarrow q', p' \rangle \\ &= c' + c_2 - \downarrow q' \rangle + \downarrow p \rangle + \downarrow q \rangle \\ &= c + \downarrow q \rangle + c_2. \end{aligned}$$

This allows us to deduce that $q \in S$ and $c \leq M$ and consequently $M \in \llbracket (c, S) \rrbracket$. We also have $\downarrow p \rangle \leq c$ and $\downarrow q' \rangle \leq c'$, $c - \downarrow p \rangle = c' - \downarrow q' \rangle$ and $c + \downarrow q \rangle \xrightarrow{t} c' + \downarrow p' \rangle$. Hence $(c, S) \xrightarrow{t}_{\text{switch}} (c', S')$. Observe that we need to use the $\Rightarrow_{\text{switch}}$ transition relation in this case.

- $c'(q') > 0$ and $c'(p') > 0$. In that case $M' = c' + c_2$ for some multiset c_2 . Let $c = c' - \downarrow p', q' \rangle + \downarrow p, q \rangle$. We have then $M = M' + \downarrow q, p \rangle - \downarrow q', p' \rangle = c + c_2$. This allows us to deduce that $c \leq M$ and consequently $M \in \llbracket (c, S) \rrbracket$ and that $c \xrightarrow{t} c'$. Hence $(c, S) \xrightarrow{t}_{\text{step}} (c', S')$.

The following cases are less involved.

2. if $\alpha = !!a$, then by definition of $\succrightarrow$, $M = \downarrow q_1, \dots, q_n, q \rangle$ and $M' = \downarrow q'_1, \dots, q'_n, q' \rangle$ such that for all $1 \leq i \leq n$, either $(q_i, ?a, q'_i) \in T$ or $a \notin R(q_i)$ and $q_i = q'_i$. We get the two following disjoint cases:

- $c'(q') = 0$. In this case, $c' = \downarrow q'_{i_1}, q'_{i_2}, \dots, q'_{i_K} \rangle$ where $1 \leq i_j \leq n$ for all $1 \leq j \leq K$ and $i_j \neq i_\ell$ if $j \neq \ell$. Let $c = \downarrow q_{i_1}, q_{i_2}, \dots, q_{i_K} \rangle$. Note that we have $c \leq M$, hence $M \in \llbracket (c, S) \rrbracket$. Using the definition of $\succrightarrow$, we also have $c + \downarrow q \rangle \succrightarrow c' + \downarrow q' \rangle$. Furthermore since $M(q) > 0$, we have $q \in S$ by definition of S . Applying the definition of $\Rightarrow_{\text{ext}}$ we have $(c, S) \xrightarrow{t}_{\text{ext}} (c', S')$.

- $c'(q') > 0$. In this case, $c' = \wr q'_{i_1}, q'_{i_2}, \dots, q'_{i_{K-1}}, q'\rangle$ where $1 \leq i_j \leq n$ for all $1 \leq j < K$ and $i_j \neq i_\ell$ if $j \neq \ell$. Let $c = \wr q_{i_1}, q_{i_2}, \dots, q_{i_{K-1}}, q\rangle$. Note that we have $c \leq M$, hence $M \in \llbracket (c, S) \rrbracket$. Using the definition of $\succrightarrow$, we also have $c \succrightarrow c'$. Applying the definition of $\Rightarrow_{\text{step}}$, we get $(c, S) \xRightarrow[\text{step}]{t} (c', S')$.
3. if $\alpha = !a$ and the message is *not* received (i.e. it is a sending), then using the definition of $\succrightarrow$, we have $M' = M - \wr q\rangle + \wr q'\rangle$ and $a \notin R(p)$ for all $p \in Q$ such that $(M - \wr q\rangle)(p) > 0$. We obtain the two following disjoint cases:
- $c'(q') = 0$. Since $M'(q') > 0$ and $c' \leq M'$, we deduce that $M' = c' + \wr q'\rangle + c_2$ for some multiset c_2 . By definition of M' , we also have $M = M' + \wr q\rangle - \wr q'\rangle$, hence $M = c' + \wr q\rangle + c_2$. We deduce that $c' \leq M$ and hence $M \in \llbracket (c', S) \rrbracket$. Furthermore we have $c' + \wr q\rangle \succrightarrow c' + \wr q'\rangle$ as $a \notin R(q)$ for all states $q \in Q$ such that $c'(q) > 0$. Consequently, $(c', S) \xRightarrow[\text{ext}]{t} (c', S')$.
 - $c'(q') > 0$. Since $c' \leq M'$, we have $M' = c' + c_2$ for some multiset c_2 . Let $c = c' + \wr q\rangle - \wr q'\rangle$. We hence have $M = M' + \wr q\rangle - \wr q'\rangle = c' + \wr q\rangle - \wr q'\rangle + c_2 = c + c_2$. Hence $c \leq M$ and $M \in \llbracket (c, S) \rrbracket$. Furthermore we have that $a \notin R(p)$ for all $p \in Q$ such that $(c - \wr q\rangle)(p) > 0$. We hence deduce that $c \xrightarrow{t} c'$. Applying the definition of $\Rightarrow_{\text{step}}$, we get $(c, S) \xRightarrow[\text{step}]{t} (c', S')$.

□

The two previous lemmas allow us to establish completeness of the construction, by a simple induction on the length of the considered execution.

Lemma 6.9. Let $M_{in} \in \mathcal{M}_{init}$ and $M \in \mathcal{M}_{\geq K}$ such that $M_{in} \succrightarrow^* M$. For all $c \in \mathcal{M}_{=K}$ such that $c \leq M$ there exists $S \subseteq Q$ such that $M \in \llbracket (c, S) \rrbracket$ and $\gamma_{in} \Rightarrow^* (c, S)$.

Proof. Suppose we have $M_0 \xrightarrow{t_1} M_1 \xrightarrow{t_2} \dots \xrightarrow{t_n} M_n$ with $M_0 = M_{in}$ and $M_n = M$. We show by induction on $0 \leq i \leq n$ that for all $c_i \in \mathcal{M}_{=K}$ such that $c_i \leq M_i$ there exists $S_i \subseteq Q$ such that $M_i \in \llbracket (c_i, S_i) \rrbracket$ and $\gamma_{in} \Rightarrow^* (c_i, S_i)$.

For $i = 0$, we have $M_{in} = \wr |M| \cdot q_{init}\rangle$ and $\wr K \cdot q_{init}\rangle$ is the unique configuration of size K smaller than M_{in} . Since $\gamma_{in} = (\wr K \cdot q_{init}\rangle, \{q_{init}\})$, we have $M_{in} \in \llbracket \gamma_{in} \rrbracket$.

Assume now that the property holds for $0 \leq i < n$. Let $c_{i+1} \in \mathcal{M}_K$ such that $c_{i+1} \leq M_{i+1}$. Since $M_i \xrightarrow{t_{i+1}} M_{i+1}$, by applying Lemma 6.8, there exists $c_i \in \mathcal{M}_K$, $S_{i+1} \subseteq Q$ such that $(c_i, S_i) \Rightarrow (c_{i+1}, S_{i+1})$ where $S_i = \{q \in Q \mid M_i(q) > 0\}$, $M_i \in \llbracket (c_i, S_i) \rrbracket$ and $M_{i+1} \in \llbracket (c_{i+1}, S_{i+1}) \rrbracket$. By induction hypothesis, there exists $S'_i \subseteq Q$ such that $M_i \in \llbracket (c_i, S'_i) \rrbracket$ and $\gamma_{in} \Rightarrow^* (c_i, S'_i)$. But since $M_i \in \llbracket (c_i, S'_i) \rrbracket$ and $S_i = \{q \in Q \mid M_i(q) > 0\}$, we have $S_i \subseteq S'_i$. Using Item 2 of Lemma 6.7, we deduce that there exists S'_{i+1} such that $S_{i+1} \subseteq S'_{i+1}$ and $(c_i, S'_i) \Rightarrow (c_{i+1}, S'_{i+1})$. Hence $\gamma_{in} \Rightarrow^* (c_{i+1}, S'_{i+1})$ and thanks to Item 1 of Lemma 6.7, $M_{i+1} \in \llbracket (c_{i+1}, S'_{i+1}) \rrbracket$. □

6.4.4 Soundness of the algorithm

We now prove that if we have $\gamma_{in} \Rightarrow^* (c, S)$ then the configuration c can be covered. We first establish that the S -part of a reachable abstract configuration stores only states that are reachable in a concrete execution.

Lemma 6.10. If $\gamma = (c, S)$ is an abstract configuration such that $\gamma_{in} \Rightarrow^* \gamma$, then all states $q \in S$ are coverable.

Proof. We suppose that we have $\gamma_{in} = \gamma_0 \Rightarrow \gamma_1 \Rightarrow \dots \Rightarrow \gamma_n = (c, S)$ and we prove this lemma by induction on n , the length of the abstract execution.

Case $n = 0$: In that case $(c, S) = \gamma_0 = (K \cdot \{q_{init}\}, \{q_{init}\})$, as q_{init} is trivially coverable, the property holds.

Case $n > 0$: We assume that the property holds for all $0 \leq m < n$ and consider the abstract execution $\gamma_0 \xRightarrow{t_1} \gamma_1 \xRightarrow{t_2} \dots \xRightarrow{t_n} \gamma_n$ where $\gamma_0 = \gamma_{in}$ and $\gamma_i = (c_i, S_i)$ for all $0 \leq i \leq n$. Let $p \in S_n$. If $p \in S_{n-1}$, then by induction hypothesis, p is coverable. Otherwise, $p \in S_n \setminus S_{n-1}$, and let $t_n = (q, \alpha, q')$ with $\alpha \in \{!a, !!a \mid a \in \Sigma\}$. By definition of $\Rightarrow$, $q \in S_{n-1}$ and:

- either $q' = p$, and by induction hypothesis, there exists an execution $M_0 \mapsto^* M$ with $M(q) > 0$ and in that case, $M_0 \mapsto^* M \xrightarrow{t_n} M'$ for a configuration M' such that $M'(p) > 0$ and so p is coverable.
- or $\alpha \in \{!a, !!a\}$ for some $a \in \Sigma$ and $(p', ?a, p) \in T$, with $p' \in S_{n-1}$. By induction hypothesis, both q and p' are coverable, with $q \in Q_A$ and $p \in Q_W$. By Proposition 6.1, there exists an execution $M_0 \mapsto^* M$ such that $M(q) \geq 1$ and $M(p') \geq 1$. We then have $M \xrightarrow{t_n} M'$ with $M'(p) > 0$ (if $\alpha = !a$ then the process on p' can receive the message a and move to p , and if $\alpha = !!a$ then the process on p' will necessary receive the broadcast and move to p), and p is coverable.

□

The next lemma establishes soundness of the algorithm. Moreover, it gives an upper bound on the minimal number of processes needed to cover a configuration.

Lemma 6.11. Let (c, S) be an abstract configuration such that $\gamma_{in} \Rightarrow \gamma_1 \Rightarrow \dots \Rightarrow \gamma_n = (c, S)$. Then, there exist $M_{in} \in \mathcal{M}_{init}$, $M \in \mathcal{M}$ such that $c \leq M$ and $M_{in} \mapsto^* M$. Moreover, $|M_{in}| = |M| \leq K + 2^{|T|} \times n$.

Observe that we do not have $M \in \llbracket (c, S) \rrbracket$ because to build the execution, the proof makes use of the copy-paste property, which does not give us any information about the covered states of the built execution. Hence, it might be that there is some state q such that $M(q) > 0$ and $q \notin S$.

Proof. We reason by induction on n , the length of the abstract execution.

Case $n = 0$: The property trivially holds for $M = \{K \cdot q_{init}\}$.

Case $n > 0$: We assume that the property holds for all $0 \leq m < n$ and consider the abstract execution $\gamma_0 \xRightarrow{t_1} \gamma_1 \xRightarrow{t_2} \dots \xRightarrow{t_n} \gamma_n$ where $\gamma_0 = \gamma_{in}$ and $\gamma_i = (c_i, S_i)$ for all $0 \leq i \leq n$. By induction hypothesis, we know that there exist $M_{in} \in \mathcal{M}_{init}$, $M_{n-1} \in \mathcal{M}$ such that $c_{n-1} \leq M_{n-1}$ and $M_{in} \mapsto^* M_{n-1}$ and $|M_{in}| = |M_{n-1}| \leq K + 2^{|T|} \times (n-1)$. If

$c_n = c_{n-1}$ then the property holds. Assume now that $c_n \neq c_{n-1}$. We let $t_n = (q, \alpha, q')$ with $\alpha = !a$ or $\alpha = !!a$. By definition of $\Rightarrow$, we know that $S_{n-1} \subseteq S_n$ and that $q \in S_{n-1}$. Thanks to Lemma 6.10, q is coverable. We now perform a case analysis:

- Assume $\gamma_{n-1} \xrightarrow{t_n}_{\text{step}} \gamma_n$. Then $c_{n-1} \xrightarrow{t_n} c_n$, and since $c_{n-1} \leq M_{n-1}$, we have $M_{n-1} = c_{n-1} + c$ for some multiset c .
 - If $\alpha = !a$, then $c_{n-1} + c = \langle q_1, \dots, q_{K-1}, q \rangle + \langle p_1, \dots, p_L \rangle$ and $c_n = \langle q'_1, \dots, q'_{K-1}, q' \rangle$ where for all $1 \leq i \leq K-1$, either $(q_i, ?a, q'_i) \in T$ or $a \notin R(q_i)$ and $q_i = q'_i$. For each $1 \leq i \leq L$, define p'_i as $p'_i = p_i$ if $a \notin R(p_i)$ or p'_i is such that $(p_i, ?a, p'_i) \in T$. If we let $c' = \langle p'_1, \dots, p'_L \rangle$ and $M_n = c_n + c'$, we have by definition that $M_{n-1} \xrightarrow{t_n} M_n$ with $c_n \leq M_n$.
 - If $\alpha = !a$ and $(c_{n-1} - \langle q \rangle)(p) > 0$ for some $p \in Q$ such that $(p, ?a, p') \in T$ (i.e., a rendez-vous occurred), it holds that $c_{n-1} + c \xrightarrow{t} c_n + c$ and we choose $M_n = c_n + c$.
 - If $\alpha = !a$ and $(c_{n-1} - \langle q \rangle)(p) = 0$ for all $p \in Q$ such that $a \in R(p)$ (i.e. it was a sending step), then either the message a can actually be received in c , i.e. there exists $(p, ?a, p') \in T$ such that $c(p) > 0$, or for all $p \in Q$ such that $c(p) > 0$, $a \notin R(p)$. In the first case, a rendez-vous will occur in the execution of t_n over M_{n-1} , and we have $c_{n-1} + c \xrightarrow{t_n} c_n + c - \langle p \rangle + \langle p' \rangle$. We then let $M_n = c_n + c - \langle p \rangle + \langle p' \rangle$. In the latter case, $c_{n-1} + c \xrightarrow{t_n} c_n + c$ and with $M_n = c_n + c$. In both cases, we have $M_{n-1} \xrightarrow{t_n} M_n$ and $c_n \leq M_n$.

In all the above cases, we have $M_{in} \xrightarrow{*} M_{n-1} \xrightarrow{*} M_n$ and $|M_{in}| = |M_n| = |M_{n-1}| \leq K + 2^{|T|} \times (n-1) \leq K + 2^{|T|} \times n$.

- Assume $\gamma_{n-1} \xrightarrow{t_n}_{\text{ext}} \gamma_n$ or $\gamma_{n-1} \xrightarrow{t_n}_{\text{switch}} \gamma_n$. As q is coverable, from Corollary 6.1, there exists an execution $M_{in}^q \xrightarrow{*} M^q$ such that $M_{in}^q \in \mathcal{M}_{init}$ and $M^q(q) > 0$ and $|M_{in}^q| \leq 2^{|T|}$. From Lemma 6.1, we have the following execution: $M_{in} + M_{in}^q \xrightarrow{*} M_{in} + M^q$. Recall that $M_{in} \xrightarrow{*} M_{n-1}$. From Lemma 6.2, with $\tilde{M}_0 = M_{in} + M^q$, we have an execution $M_{in} + M^q \xrightarrow{*} M_{n-1} + M'^q$ where $M'^q(q) > 0$. We deduce that $M_{n-1} + M'^q = c_{n-1} + \langle q \rangle + c$ for some multiset c . We now proceed with a case analysis:

- Case $\gamma_{n-1} \xrightarrow{t_n}_{\text{ext}} \gamma_n$ where $c_{n-1} + \langle q \rangle \xrightarrow{t_n} c_n + \langle q' \rangle$. By definition of $\xrightarrow{*}$, we also have $c_{n-1} + \langle q \rangle + c \xrightarrow{*} c_n + \langle q' \rangle + c'$ for some multiset c' . Letting $M_n = c_n + \langle q' \rangle + c'$ gives us that $M_{in} + M_{in}^q \xrightarrow{*} c_{n-1} + \langle q \rangle + c \xrightarrow{*} M_n$.
- Case $\gamma_{n-1} \xrightarrow{t_n}_{\text{switch}} \gamma_n$: by definition of $\Rightarrow_{\text{switch}}$, we know that there exists $(p, ?a, p') \in T$ with $p \in S_{n-1}$ such that $c_{n-1} - \langle p \rangle = c_n - \langle q' \rangle$ and $c_{n-1} + \langle q \rangle \xrightarrow{t_n} c_n + \langle p' \rangle$. Furthermore, by definition of $\xrightarrow{*}$, we have $c_{n-1} + \langle q \rangle + c \xrightarrow{*} c_n + \langle p' \rangle + c$. Hence letting $M_n = c_n + \langle p' \rangle + c$ gives us that $M_{in} + M_{in}^q \xrightarrow{*} c_{n-1} + \langle q \rangle + c \xrightarrow{*} M_n$, with $c_n \leq M_n$.

In both cases, we have shown that there exists M_n such that $c_n \leq M_n$ and $M_{in} + M_{in}^q \xrightarrow{*} M_n$. Furthermore we have that $|M_n| = |M_{in} + M_{in}^q| \leq K + 2^{|T|} \times (n-1) + 2^{|T|} \leq K + 2^{|T|} \times n$.

□

6.4.5 Upper bound

As we proved completeness and soundness of our abstraction, it leads to the following theorem.

Theorem 6.6. CONF-COVER^{nb} restricted to Wait-Only protocols is in PSPACE.

Proof. Let $P = (Q, \Sigma, q_{init}, T)$, and let $M_f \in \mathcal{M}$ be a configuration to cover. Using Lemmas 6.9 and 6.11, we know that there exists $M \in \mathcal{M}_{init}$ and $M' \in \mathcal{M}$ such that $M \mapsto^* M'$ and $M_f \leq M'$ if and only if there exists an abstract execution $\gamma_{in} \Rightarrow \gamma_1 \Rightarrow \dots \Rightarrow \gamma_n$ with $\gamma_n = (M_f, S)$ for some $S \subseteq Q$. Hence, the algorithm that decides whether a vertex of the form (M_f, S) is reachable from γ_{in} in the finite graph $(\Gamma, \Rightarrow)$ is correct. Each abstract configuration $\gamma = (c, S)$ can be encoded using $|Q| \cdot \log_2(|M_f|) + |Q|$ bits: the first part represents the multiset c by storing, for each state $q \in Q$, the number of processes in that state (bounded by $|M_f|$), and the second part encodes the subset $S \subseteq Q$. As a result, there exists a non-deterministic procedure that, starting from γ_{in} , maintains only one abstract configuration at each step, computes successors dynamically, and accepts if it reaches a configuration of the form (M_f, S) . If no such configuration is reachable, the procedure halts after at most $|M_f|^{|Q|} \cdot 2^{|Q|}$ steps, since some abstract configuration (c, S) must then repeat. This procedure uses space $|Q| \cdot \log_2(|M_f|) + |Q|$. By Savitch's theorem, we conclude that there exists a deterministic procedure solving CONF-COVER^{nb} for Wait-Only protocols using space $(|Q| \cdot \log_2(|M_f|) + |Q|)^2 \in O(|Q|^3 \cdot \log^2(|M_f|))$. Recall that $|P| = |T|(\log_2(|Q|) + \log_2(|Q|))$. We suppose w.l.o.g. that each state has either an outgoing or ingoing transition. Hence, $|Q| \leq |T|$ and the deterministic procedure uses space in $O(|T|^3 \cdot \log^2(|M_f|))$. Observe that this space bound is polynomial in the size of the protocol and polylogarithmic in the size of the target configuration. $\square$

Remark 8. Thanks to Lemma 6.11, we know that $\gamma_{in} \Rightarrow \gamma_1 \Rightarrow \dots \Rightarrow \gamma_n$ with $\gamma_n = (M_f, S)$ if there exists $M_{init} \in \mathcal{M}_{init}$, $M \in \mathcal{M}$ such that $M_f \leq M$ and $M_{init} \mapsto^* M$ and $|M_{init}| = |M| \leq |M_f| + 2^{|T|} \times n$. But due to the number of abstract configurations, we can assume that $n \leq 2^{|T|} \times |M_f|^{|Q|}$ as it is unnecessary in the abstract execution $\gamma_{in} \Rightarrow \gamma_1 \Rightarrow \dots \Rightarrow \gamma_n$ to visit twice the same abstract configuration. Hence the configuration M_f is coverable iff there is $M \in \mathcal{M}_{init}$ and $M' \in \mathcal{M}$ such $M \mapsto^* M'$ and $M_f \leq M'$ and $|M| = |M'| \leq |M_f| + 2^{|Q|} \times 2^{|T|} \times |M_f|^{|Q|}$.

6.4.6 Lower bound

To prove PSPACE-hardness of the CONF-COVER^{nb} problem for Wait-Only protocols, we reduce the intersection non-emptiness problem for deterministic finite automata, which is known to be PSPACE-complete [Koz77]. PSPACE-hardness holds even when restricting to Wait-Only broadcast protocols, i. e. without transitions of the form $(q, !a, q')$.

AUTOMATA. Let $\mathcal{A}_1, \dots, \mathcal{A}_n$ be a list of deterministic finite and *complete* automata with $\mathcal{A}_i = (\Sigma, Q_i, q_i^0, \{q_i^f\}, \Delta_i)$ for all $1 \leq i \leq n$. Observe that we restrict our reduction to automata with a unique accepting state, which does not change the complexity of the problem. We

denote by Σ^* the set of words over the finite alphabet Σ and Δ_i^* the function extending Δ_i to Σ^* , i.e., for all $q \in Q_i$, $\Delta_i^*(q, \varepsilon) = q$, and for all $w \in \Sigma^*$ and $a \in \Sigma$, $\Delta_i^*(q, wa) = \Delta_i(\Delta_i^*(q, w), a)$.

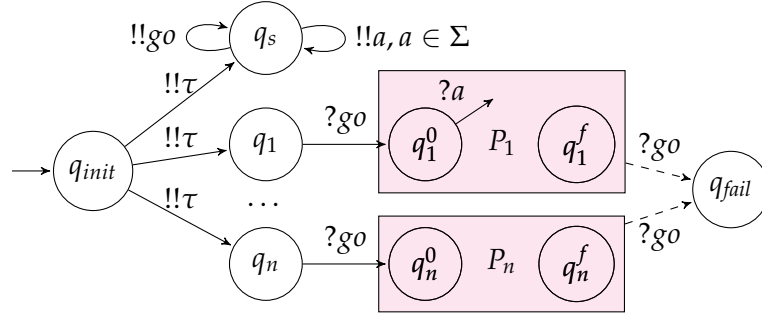


Figure 6.14: Wait-Only protocol P for PSPACE-hardness of CONF-COVER^{nb} .

REDUCTION. We build the protocol P with set of states Q , displayed in Figure 6.14 where P_i for $1 \leq i \leq n$ is a protocol mimicking the behaviour of the automaton $\mathcal{A}_i$:

$$P_i = (Q_i, \Sigma, q_i^0, T_i), \text{ with } T_i = \{(q, ?a, q') \mid (q, a, q') \in \Delta_i\}.$$

Moreover, from any state $q \in \bigcup_{1 \leq i \leq n} Q_i$, there is an outgoing transition $(q, ?go, q_{fail})$. These transitions are depicted by the outgoing dashed transitions labeled by $?go$ from the orange rectangles.

Note that P is Wait-Only as all states in P_i are waiting states (for all $1 \leq i \leq n$) and the only action states are q_{init} and q_s . We show that $\bigcap_{1 \leq i \leq n} L(\mathcal{A}_i) \neq \emptyset$ if and only if there is an initial configuration $M \in \mathcal{M}_{init}[P]$ and a configuration $M' \in \mathcal{M}[P]$ such that $M \mapsto^* M'$ and $M_f \leq M'$ with $M_f = \wr q_1^f, \dots, q_n^f \wr$.

The idea is to synchronize (at least) n processes into simulating the n automata. To this end, we need an additional (leader) process that will broadcast a message go , which will be received by the n processes, leading each of them to reach a different automaton initial state. Then, the leader process will broadcast a word letter by letter. Since the automata are all complete, these broadcast will be received by all the processes that simulate the automata, mimicking an execution. If the word belongs to all the automata languages, then each process simulating the automata ends the simulation on the unique final state of the automaton. Note that if the leader process broadcasts the message go a second time, then all the processes simulating the automata stop their simulation and reach the state q_{fail} .

Formal proofs are given below.

Lemma 6.12. If there exists a word $w \in \bigcap_{1 \leq i \leq n} L(\mathcal{A}_i)$, then $\wr n+1 \cdot q_{init} \wr \mapsto^* \wr q_s, q_1^f, \dots, q_n^f \wr$.

Proof. Assume that there exists a word $w = a_1 \dots a_k \in \bigcap_{1 \leq i \leq n} L(\mathcal{A}_i)$, i.e., $q_i^f = \Delta_i^*(q_i^0, a_1 \dots a_k)$ for all $1 \leq i \leq n$. Then take $M = \wr (n+1) \cdot q_{init} \wr$. There exists an execution $M \mapsto^* M'$ with $M' = \wr q_s, q_1^f, \dots, q_n^f \wr \geq M_f$. This execution is

$$\begin{aligned} M &\xrightarrow{(q_{init}, !!\tau, q_s)} \tilde{M}_1 \xrightarrow{(q_{init}, !!\tau, q_1)} \tilde{M}_2 \dots \xrightarrow{(q_{init}, !!\tau, q_n)} \tilde{M}_0 \\ &\xrightarrow{(q_s, !!go, q_s)} M_0 \xrightarrow{(q_s, !!a_1, q_s)} M_1 \xrightarrow{(q_s, !!a_2, q_s)} M_2 \dots \xrightarrow{(q_s, !!a_k, q_s)} M'. \end{aligned}$$

One can check that $M_0(q_i^0) = 1$ for all $1 \leq i \leq n$, and hence, by definition of $(T_i)_{1 \leq i \leq n}$, $M_k(q_i^f) = 1$ for all $1 \leq i \leq n$. Hence $M_f \leq M' = \langle q_s, q_1^f, \dots, q_n^f \rangle$. $\square$

Lemma 6.13. If there exists $M \in \mathcal{M}_{init}$ and $M' \in \mathcal{M}$ such that $M \mapsto^* M'$ and $M' \geq M_f$ then $\bigcap_{1 \leq i \leq n} L(\mathcal{A}_i) \neq \emptyset$.

Proof. Assume that there exists an initial configuration M and an execution $M \mapsto^* M'$ with $M' \leq M_f$. We first make easy observations about the executions of this protocol. For this we define the following notation: Let $M_1, M_2 \in \mathcal{M}[P]$. We write $M_1 \xrightarrow{\tau}^* M_2$ if there exists a sequence of k transitions $M_1 \xrightarrow{t_1} \dots \xrightarrow{t_k} M_2$ such that $t_i \in \{(q_{init}, !!\tau, q) \mid q \in \{q_s, q_1, \dots, q_n\}\}$ for all $1 \leq i \leq k$.

Observation 1. Let $M_1, M_2 \in \mathcal{M}[P]$. If $M_1 \xrightarrow{\tau}^* M_2$ then $M_2(q) \geq M_1(q)$ for all $q \in \{q_s, q_1, \dots, q_n\}$, $M_2(q_{init}) \leq M_1(q_{init})$, and $M_2(q) = M_1(q)$ for all other $q \in Q$.

Observation 2. Let $M_1, M_2 \in \mathcal{M}[P]$ such that $M_1 \xrightarrow{t_1} \dots \xrightarrow{t_k} M_2$ and $t_j \neq (q_s, !!go, q_s)$ for all $1 \leq j \leq k$. If $M_1(q) = 0$ for all $q \in Q_i$, then $M_2(q) = 0$ for all $q \in Q_i$, for any $1 \leq i \leq n$.

Observation 3. Let $M_1, M_2 \in \mathcal{M}[P]$ such that $M_1 \xrightarrow{(q_s, !!go, q_s)} M_2$. Then, $M_2(q_s) = M_1(q_s) > 0$, and $M_2(q) = 0$ for all $q \in \bigcup_{1 \leq i \leq n} (Q_i \setminus \{q_i^0\})$.

We can deduce from these observations that the execution $M \mapsto^* M'$ can be decomposed in $M \xrightarrow{\tau}^* \hat{M} \xrightarrow{(q_s, !!go, q_s)} M_0 \xrightarrow{t_0} \dots \xrightarrow{t_k} M'$ where $t_i \neq (q_s, !!go, q_s)$ for all $0 \leq i \leq k$. Indeed, since $M(q) = 0$ for all $q \in \bigcup_{1 \leq i \leq n} Q_i$, if no transition $(q_s, !!go, q_s)$ appears in the execution, Observation 2 allows to conclude that $M'(q_i^f) = 0$ for all $1 \leq i \leq n$, which contradicts the fact that $M' \geq M_f$. By Observation 3, $M_0(q_s) > 0$ and $M_0(q) = 0$ for all $q \in \bigcup_{1 \leq i \leq n} (Q_i \setminus \{q_i^0\})$.

We also deduce that $M_0(q_i^0) > 0$ for all $1 \leq i \leq n$. Otherwise, if there exists $1 \leq j \leq n$ such that $M_0(q_j^0) = 0$, then $M_0(q) = 0$ for all $q \in Q_j$, and so by Observation 2, $M'(q_j^f) = 0$ which is a contradiction with $M' \geq M_f$.

Now the run $M_0 \xrightarrow{t_0} \dots \xrightarrow{t_k} M'$ is of the form

$$M_0 \xrightarrow{\tau}^* M'_0 \xrightarrow{(q_s, !!a_1, q_s)} M_1 \xrightarrow{\tau}^* M'_1 \xrightarrow{(q_s, !!a_2, q_s)} M_2 \dots \xrightarrow{(q_s, !!a_k, q_s)} M_k \xrightarrow{\tau}^* M'.$$

Using Observation 1, we can obtain a new run $M_0 \xrightarrow{(q_s, !!a_1, q_s)} \tilde{M}_1 \xrightarrow{(q_s, !!a_2, q_s)} \tilde{M}_2 \dots \xrightarrow{(q_s, !!a_k, q_s)} \tilde{M}_k$ such that for all $1 \leq j \leq k$:

$$\begin{aligned} \tilde{M}_j(q_s) &= M_0(q_s) \\ \tilde{M}_j(q_i) &= M_0(q_i) && \text{for all } 1 \leq i \leq n \\ \tilde{M}_j(q_{init}) &= M_0(q_{init}) \\ \tilde{M}_j(q) &= M_j(q) && \text{for all other } q \in Q \setminus \{q_s, q_{init}, q_i \mid 1 \leq i \leq n\} \end{aligned}$$

In particular, $\tilde{M}_k(q_i^f) = M_k(q_i^f) = M'(q_i^f) \geq M_f(q_i^f) = 1$.

We can show now that the word $a_1 \dots a_k$ belongs to $\bigcap_{1 \leq i \leq n} L(\mathcal{A}_i)$. Let $1 \leq i \leq n$. It is easy to see that for all $1 \leq j \leq k$, there exists a unique $q_i^j \in Q_i$ such that $\tilde{M}_j(q_i^j) > 0$ and $\tilde{M}_j(q) = 0$ for all $q \in Q_i \setminus \{q_i^j\}$ (because $\mathcal{A}_i$ is deterministic and by construction of T_i).

Moreover, as q_i^0 is the unique state in $q \in Q_i$ such that $\tilde{M}_j(q) > 0$, it holds that for all $1 \leq j \leq k$, $q_i^j = \Delta_i^*(q_i^0, a_1 \dots a_j)$ (this holds from the construction of T_i and because $\mathcal{A}_i$ is complete, hence all the letters are received and lead to the corresponding transition of the automaton). As $\tilde{M}_k(q_i^f) = M'(q_i^f) > 0$, we get that $q_i^f = \Delta_i^*(q_i^0, a_1 \dots a_k)$, and hence $a_1 \dots a_k \in L(\mathcal{A}_i)$ for all $1 \leq i \leq n$. □

Soundness and completeness of the reduction (Lemmas 6.12 and 6.13) together with Theorem 6.6 lead to the main theorem of this section.

Theorem 6.7. *CONF-COVER^{nb} restricted to Wait-Only protocols is PSPACE-complete, even when restricting to Wait-Only broadcast protocols.*

6.5 CONFIGURATION COVERABILITY FOR WAIT-ONLY RENDEZ-VOUS NETWORKS

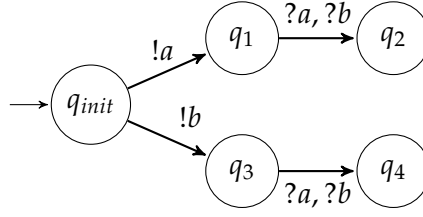
In this section, we focus on Wait-Only *rendez-vous* protocols, which do not involve broadcast transitions.

In Chapter 2, we saw why, in the general case, the configuration coverability problem can be reduced to state coverability using broadcasts. Later in Section 6.4, we explained why it no longer holds with Wait-Only protocols. The same phenomenon arises with rendez-vous networks. First let us explain how the configuration coverability problem reduces to state coverability in rendez-vous networks (without the Wait-Only restriction). Of course, we can no longer stop everyone in their execution with a single sending of a message “halt” as we explained in Chapter 2. However, we modify the protocol so that, once a configuration covering the target one is reached, a fresh process starts waiting for messages from witness processes. Each process in the target configuration can send a message to the fresh process, witnessing its state before transitioning to a newly added sink state. To ensure that execution stops once this checking phase begins, we enrich the protocol so that if the process receiving messages from witnesses receives any other messages, it also goes to a sink state and stops its execution. We are able to guarantee that this checking phase is always done by at most one process at a time thanks to the mechanism we have seen multiple times now (e. g. in Section 5.4.2): to start its check, the process sends (without anyone receiving it) a message “go”, which can be received from all the states of the checking phase. Each time a process wants to join, it must send the same message, forcing the current checking process to exit upon receiving it.

As in the case of broadcasts, this reasoning no longer holds in a Wait-Only setting: this reduction inherently produces non-Wait-Only protocols, as we add sending transitions to all states, including waiting states.

Example 6.6. Consider the rendez-vous protocol P of Figure 6.15 and the multiset $\wr q_1, q_2 \wr$. The construction described here-above yields the protocol P' of Figure 6.16, with $\odot$ as target state. From an execution covering $\wr q_1, q_2 \wr$ in P , we get an execution covering $\odot$ in P' . In P , an execution covering $\wr q_1, q_2 \wr$ is:

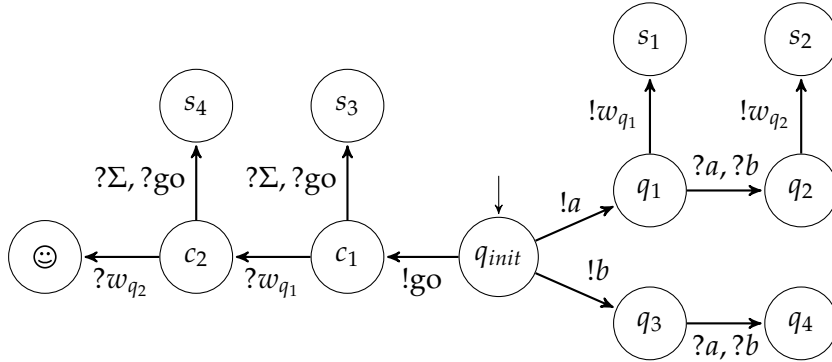
$$\wr q_{init}, q_{init} \wr \rightsquigarrow \wr q_1, q_{init} \wr \rightsquigarrow \wr q_2, q_1 \wr$$

Figure 6.15: A Wait-Only rendez-vous protocol P .

And the one covering $\odot$ in P' is:

$$\begin{aligned}
 \wr q_{init}, q_{init}, q_{init} \wr &\rightarrow \wr q_1, q_{init}, q_{init} \wr \rightarrow \wr q_2, q_1, q_{init} \wr \\
 &\rightarrow \wr q_2, q_1, c_1 \wr \text{ (checking process starts)} \\
 &\rightarrow \wr q_2, s_1, c_2 \wr \text{ (first witness sends its message)} \\
 &\rightarrow \wr s_2, s_1, \odot \wr \text{ (second witness sends its message and } \odot \text{ is reached)}
 \end{aligned}$$

Here, w_{q_1} and w_{q_2} correspond to the witness messages, while s_1 and s_2 are sink states ensuring the checking process terminates. The final state $\odot$ confirms that the target configuration has been covered. Observe that while P is Wait-Only, P' is not, as it contains the transitions $(q_1, ?a, q_2)$ and $(q_1, !w_{q_1}, s_1)$.

Figure 6.16: The rendez-vous protocol P' obtained from P (Figure 6.15) and the multiset $\wr q_1, q_2 \wr$.

In Theorem 6.7, we showed that the configuration coverability problem for Wait-Only broadcast protocols (restricted to broadcast transitions) is PSPACE-complete. The algorithm we presented runs in polynomial space in the size of the protocol and logarithmic space in the size of the configuration. This implies that we cannot expect to characterize the set of coverable configurations in an explicit or compact form: if such a characterization were available, the algorithm would run in time independent of the size of the target configuration. In particular, while it is possible to determine whether a configuration of the form $\wr j \cdot q \wr$ is coverable for a given $j \in \mathbb{N}$, there is no known bound on the maximum value of j for which q is still coverable. As a result, computing the maximal number of processes that can simultaneously cover a given waiting state q would require repeatedly querying the coverability procedure for increasing values of j , until the answer becomes negative. In the worst case, this upper bound may be very large. This reflects a limitation of verification in

broadcast networks: some receiving states may be coverable only by a specific number of processes, and identifying that number can be computationally expensive. Then, in the case of Wait-Only rendez-vous protocols (RDV-protocols), the situation improves significantly. These networks enjoy better verification properties: we will see that it is possible to compute, in polynomial time, a succinct representation of the set of coverable configurations. In particular, we are able to determine the maximum number of processes that can be simultaneously present in each reception state. This leads to an efficient method for reasoning about coverability in RDV-protocols, independent of the size of the configuration we aim to cover. Observe that the cypaste property (Proposition 6.1) remains valid when considering Wait-Only rendez-vous protocols.

In the sequel, we will often refer to paths in the underlying graph of a protocol. Formally, given a protocol $P = (Q, \Sigma, q_{init}, T)$, a *path* is either a state $q \in Q$ or a finite sequence of transitions $(q_0, \alpha_0, q_1)(q_1, \alpha_1, q_2) \dots (q_k, \alpha_k, q_{k+1})$. In the first case, the path starts and ends at q ; in the second case, it starts at q_0 and ends at q_{k+1} .

6.5.1 Token-Sets of Configurations

To solve the coverability problem for Wait-Only RDV-protocols in polynomial time, we rely on a sound and complete abstraction of the set of reachable configurations. We represent this set abstractly as a token-set of configurations, consisting of:

- a set S of states that can host an unbounded number of processes simultaneously (including action states, i.e., those in Q_A), and
- a set $Toks$ of tokens that represent states which can host only a bounded number of processes at a time (specifically, at most one) along with the message that allowed reaching them.

This situation arises when a message m can be received from a waiting state q , and m must have been sent earlier along the path leading to q . Therefore, each token in $Toks$ is a pair (q, m) , where $q \in Q_W$ is a waiting state and $m \in \Sigma$ is a message that was sent before reaching q . Note that since multiple paths can lead to q with different messages being sent along the way, $Toks$ may contain distinct tokens (q, m_1) and (q, m_2) with $m_1 \neq m_2$.

Example 6.7. Consider the Wait-Only rendez-vous protocol P of Figure 6.15. Action states $(q_{init}, q_2$ and $q_4)$ can be populated by any number of processes, in particular for q_2 :

$$\begin{aligned} \{q_{init}, q_{init}, q_{init}, \dots\} \xrightarrow{(q_{init}, !a, q_1)} \{q_1, q_{init}, q_{init}, \dots\} \xrightarrow{(q_{init}, !a, q_1)} \{q_2, q_1, q_{init}, \dots\} \\ \xrightarrow{(q_{init}, !a, q_1)} \{q_2, q_2, q_1, \dots\} \xrightarrow{(q_{init}, !a, q_1)} \dots \end{aligned}$$

However q_1 (a waiting state) is coverable by *at most* one process: the message a is necessarily sent to reach q_1 , and is received from q_1 . Hence, a second process trying to reach q_1 will exit the first one already there.

This is not necessarily the case for all waiting states: if only b can be received from q_1 (and a is no longer received), the above argument no longer holds and q_1 can be covered by any number of processes with successive sendings of a from an initial configuration.

In the sequel, we consider a Wait-Only RDV-protocol $P = (Q, \Sigma, q_{init}, T)$.

Definition 6.5. A **token-set of configurations** γ is a pair $(S, Toks)$ such that:

- $S \subseteq Q$ is a subset of states, and,
- $Toks \subseteq Q_W \times \Sigma$ is a subset of pairs (called tokens) composed of a waiting state and a message, and,
- $q \notin S$ for all $(q, m) \in Toks$.

We denote by Γ the set of token-sets of configurations.

Let $\gamma = (S, Toks)$ be a token-set of configurations. Before we go into the configurations represented by γ , we need some preliminary definitions.

- We note $st(Toks)$ the set $\{q \in Q_W \mid \text{there exists } m \in \Sigma \text{ such that } (q, m) \in Toks\}$ of control states appearing in $Toks$.
- Given a state $q \in Q$, we recall that $R(q)$ is the set $\{m \in \Sigma \mid \text{there exists } q' \in Q \text{ such that } (q, ?m, q') \in T\}$ of messages that can be received in state q (if q is not a waiting state, this set is empty);
- Given two different waiting states q_1 and q_2 in $st(Toks)$, we say q_1 and q_2 are **conflict-free** in γ if there exist $m_1, m_2 \in \Sigma$ such that $m_1 \neq m_2$, $(q_1, m_1), (q_2, m_2) \in Toks$ and $m_1 \notin R(q_2)$ and $m_2 \notin R(q_1)$.

Intuitively, two states q and p are conflict-free if a process can reach q by sending a message a , and another can reach p by sending a message b , such that the message a is not received from p , and b is not received from q . In other words, there exist two tokens (q, a) and (p, b) , and the emission of a (resp. b) does not trigger a reception from p (resp. q). This ensures that both states are jointly reachable, regardless of the order in which q and p are reached.

We can now make the link between configurations in $\mathcal{M}$ and token-sets of configurations.

Definition 6.6. Let $\gamma = (S, Toks)$ be a token-set of configurations. We say that a configuration $M \in \mathcal{M}[P]$ **respects** γ if and only if for all $q \in Q$ such that $M(q) > 0$ one of the following two conditions holds:

1. $q \in S$, or,
2. $q \in st(Toks)$, $M(q) = 1$ and for all $q' \in st(Toks) \setminus \{q\}$ such that $M(q') = 1$, we have that q and q' are **conflict-free**.

Let $\llbracket \gamma \rrbracket$ be the set of configurations respecting γ .

Observe that $\llbracket (\{q_{init}\}, \emptyset) \rrbracket = \mathcal{M}_{init}$, hence $(\{q_{init}\}, \emptyset)$ will be our *initial* token-set.

Note that these conditions only speak about states q such that $M(q) > 0$ as we are only interested in characterising the reachable states (and unreachable states should not appear in S or $st(Toks)$). Note that in $\llbracket \gamma \rrbracket$, for q is in S there is no restriction on the number of processes that can be put in q , whereas if q in $st(Toks)$, it can host at most one process. Two states from $st(Toks)$ can both host a process in the same configuration if they are conflict-free.

Example 6.8. Going back to Example 6.7 and protocol P of Figure 6.15, the token-set characterizing all the coverable configurations is $\gamma = (S, Toks)$, with:

- $S = \{q_{init}, q_2, q_4\}$, and

- $Toks = \{(q_1, a), (q_3, b)\}$ (last message sent to reach q_1 [resp. q_3] is a [resp. b]).

Hence, coverable configurations (configurations respecting γ) can host at most one process in q_1 and one process in q_2 . Observe that q_1 and q_3 are *not* conflict-free: $a \in R(q_3)$ and $b \in R(q_1)$. Hence, any configuration M **respecting** γ can not host a process in q_1 and a process in q_3 . Indeed, in P it is impossible to cover q_1 and q_3 at the same time: once a process is in q_1 , if a new process tries to reach q_3 it has to send b , which has to be received by the process in q_1 .

Consider now the protocol P' depicted in Figure 6.17. The only difference with P is that state q_1 can no longer receive message a . Surprisingly, this has a significant impact on the set of coverable configurations: now, both q_1 and q_3 can be covered simultaneously, as witnessed by the following execution:

$$\wr q_{init}, q_{init} \wr \rightarrow \wr q_1, q_{init} \wr \rightarrow \wr q_1, q_3 \wr.$$

This was not possible in P , since the transition $(q_{init}, !b, q_3)$ would have caused the first process (on q_1) to leave due to a reception of b . However, note that q_1 and q_3 are *not* conflict-free in the configuration γ , since $a \in R(q_3)$. But unlike in P , we now have $b \notin R(q_1)$ in P' , which is crucial. In fact, the configuration γ should not be considered a “good” token-set configuration for P' . Observe that, thanks to $b \notin R(q_1)$, any number of processes can be added on q_1 without interfering with the behavior of those on q_3 . For example:

$$\begin{aligned} \wr q_{init}, q_{init}, q_{init}, q_{init}, q_{init} \wr &\rightarrow \wr q_1, q_{init}, q_{init}, q_{init}, q_{init} \wr \rightarrow \wr q_1, q_3, q_{init}, q_{init}, q_{init} \wr \\ &\rightarrow \wr q_1, q_4, q_1, q_{init}, q_{init} \wr \rightarrow \wr q_1, q_4, q_1, q_2, q_{init} \wr \\ &\rightarrow \wr q_1, q_4, q_1, q_4, q_1 \wr. \end{aligned}$$

Hence, in P' , q_3 should be in S . A “good” token-set configuration should satisfy the following condition: for *every* pair of tokens (q, m) and (q', m') with $q \neq q'$, either:

- $m \notin R(q')$ and $m' \notin R(q)$ (i.e., q and q' are conflict-free, this time we ask that each pair of tokens with q and q' realizes the conflict-freeness), or
- $m \in R(q')$ and $m' \in R(q)$ (i.e., q and q' cannot be reached together).

We formalize this notion below and call it *consistency*.

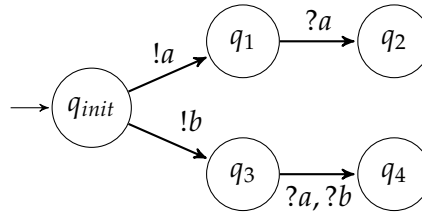
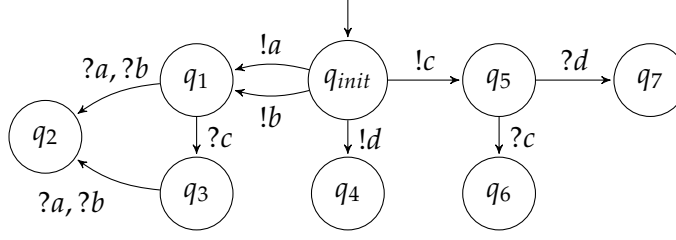


Figure 6.17: Another Wait-Only rendez-vous protocol P' .

Definition 6.7. A token-set of configurations $\gamma = (S, Toks)$ is **consistent** if

- (i) for all $(q, m) \in Toks$, there exists a path $(q_0, !m, q_1)(q_1, ?m_1, q_2) \dots (q_k, ?m_k, q)$ in P such that $q_0 \in S$, and there exists $(q'_i, !m_i, q''_i) \in T$ with $q'_i \in S$ for all $1 \leq i \leq k$;

Figure 6.18: Wait-only protocol P_1 .

- (ii) for two tokens $(q, m), (q', m') \in \text{Toks}$ either $m \in R(q')$ and $m' \in R(q)$, or, $m \notin R(q')$ and $m' \notin R(q)$.

Condition (i) ensures that processes in S can indeed lead to a process in the states from $\text{st}(\text{Toks})$. Condition (ii) ensures that if in a configuration M , some states in $\text{st}(\text{Toks})$ are pairwise conflict-free, then they can all host a process together no matter the path chosen to reach them (hence conflict freeness is realized for each pair of tokens with q and q'). If they are not pairwise conflict-free, then, there is no way to reach them together.

We assume w.l.o.g. that each state and each message appears in at least one transition, i. e. $|Q| \leq |T|$ and $|\Sigma| \leq |T|$. We now prove that checking that a configuration respects a token-set can be done in polynomial time and that $\llbracket \gamma \rrbracket$ is *downward-closed*, i. e. for any $M \leq M'$ such that $M' \in \llbracket \gamma \rrbracket$, it holds that $M \in \llbracket \gamma \rrbracket$.

Lemma 6.14. Given $\gamma \in \Gamma$ and a configuration M , there exists $M' \in \llbracket \gamma \rrbracket$ such that $M' \geq M$ if and only if $M \in \llbracket \gamma \rrbracket$. Checking that $M \in \llbracket \gamma \rrbracket$ can be done in polynomial time.

Proof. Let $M' \in \llbracket \gamma \rrbracket$ such that $M' \geq M$. Let $q \in Q$ such that $M(q) > 0$. Then we have $M'(q) > 0$. If $q \notin S$, then $q \in \text{st}(\text{Toks})$ and $M'(q) = 1$ and $M(q) = 1$ too. Furthermore for all $q' \in \text{st}(\text{Toks}) \setminus \{q\}$ such $M(q') = 1$, we have that $M'(q') = 1$ and q and q' are conflict-free. This allows us to conclude that $M \in \llbracket \gamma \rrbracket$. Checking whether M belongs to $\llbracket \gamma \rrbracket$ can be done in polynomial time applying the definition of $\llbracket \cdot \rrbracket$. $\square$

6.5.2 Computing Token-Sets of Configurations

Our polynomial time algorithm is based on the computation of a sequence of consistent token-sets of configurations. This sequence, of a polynomial length, leads to a final token-set representing a correct abstraction for the set of coverable configurations. This will be achieved by a function $F : \Gamma \rightarrow \Gamma$, that inductively computes this final token-set starting from $\gamma_0 = (\{q_{\text{init}}\}, \emptyset)$. Formal definition of the function F relies on intermediate sets $S'' \subseteq Q$ and $\text{Toks}'' \subseteq Q_W \times \Sigma$, which are the smallest sets satisfying the conditions described in Table 6.1.

From S and Toks , rules described in Table 6.1 add states and tokens to S'' and Toks'' from the outgoing transitions from states in S and $\text{st}(\text{Toks})$. It must be that every state added to S'' can host an unbounded number of processes, and every state added to Toks'' can host at most one process. Furthermore, two conflict-free states in Toks'' should be able to host at most one process at the same time. We now give an example of computation of S'' and Toks'' .

Construction of intermediate states S'' and $Toks''$

1. $S \subseteq S''$ and $Toks \subseteq Toks''$
2. for all $(p, !a, p') \in T$ with $p \in S$, we have:
 - a) $p' \in S''$ if $a \notin R(p')$ or if there exists $(q, ?a, q') \in T$ with $q \in S$;
 - b) $(p', a) \in Toks''$ otherwise (i.e. when $a \in R(p')$ and for all $(q, ?a, q') \in T$, $q \notin S$);
3. for all $(q, ?a, q') \in T$ with $q \in S$ or $(q, a) \in Toks$, we have $q' \in S''$ if there exists $(p, !a, p') \in T$ with $p \in S$;
4. for all $(q, ?a, q') \in T$ with $(q, m) \in Toks$ with $m \neq a$, if there exists $(p, !a, p') \in T$ with $p \in S$, we have:
 - a) $q' \in S''$ if $m \notin R(q')$;
 - b) $(q', m) \in Toks''$ if $m \in R(q')$.

Table 6.1: Definition of $S'', Toks''$ for $\gamma = (S, Toks)$.

Example 6.9. Consider the Wait-Only protocol P_1 depicted on Figure 6.18. From $(\{q_{in}\}, \emptyset)$, rules described in Table 6.1 construct the following pair:

$$(S_1'', Toks_1'') = (\{q_{in}, q_4\}, \{(q_1, a), (q_1, b), (q_5, c)\}).$$

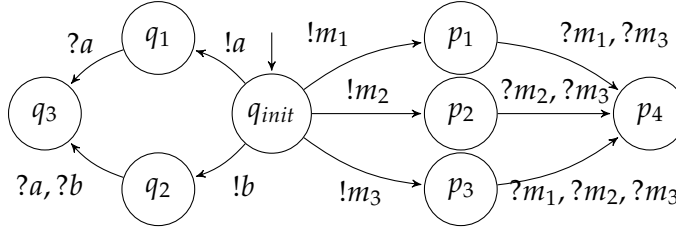
In P_1 , it is indeed possible to reach a configuration with as many processes as one wishes in the state q_4 by repeating the transition $(q_{in}, !d, q_4)$ (rule 2a). On the other hand, it is possible to put *at most* one process in the waiting state q_1 (rule 2b), because any other attempt from a process in q_{init} will yield a reception of the message a (resp. b) by the process already in q_1 . Similarly, we can put at most one process in q_5 . Note that in $Toks_1''$, the states q_1 and q_5 are conflict-free and it is hence possible to have simultaneously one process in both of them. If we apply rules of Table 6.1 one more time to $(S_1'', Toks_1'')$, we get

$$S_2'' = \{q_{init}, q_2, q_4, q_6, q_7\} \text{ and } Toks_2'' = \{(q_1, a), (q_1, b), (q_3, a), (q_3, b), (q_5, c)\}.$$

We can put at most one process in q_3 : to add one, a process will take the transition $(q_1, ?c, q_3)$. Since $(q_1, a), (q_1, b) \in Toks_1''$, there can be at most one process in state q_1 , and this process arrived by a path in which the last message sent was $!a$ or $!b$. Since $\{a, b\} \subseteq R(q_3)$, by rule 4b, $(q_3, a), (q_3, b)$ are added. On the other hand one can put as many processes as one wants in the state q_7 (rule 4a): from a configuration with one process on state q_5 , successive sendings of letter c , and rendez-vous on letter d will allow to increase the number of processes in state q_7 .

However, one can observe that q_5 can in fact host an unbounded number of processes: once two processes have been put on states q_1 and q_5 respectively (remember that q_1 and q_5 are conflict-free in $(S_1'', Toks_1'')$), iterating rendez-vous on letter c (with transition $(q_1, ?c, q_3)$) and rendez-vous on letter a can increase unboundedly the number of processes on q_5 .

As a consequence we need to apply another transformation to $(S_2'', Toks_2'')$ to obtain $F(S_1'', Toks_1'')$. We shall see that this second step has no impact when computing $F((\{q_{init}\}, \emptyset))$ hence we have that $F((\{q_{init}\}, \emptyset)) = (S_1'', Toks_1'')$.

Figure 6.19: Wait-only protocol P_2 .

We shall finally set $F(\gamma)$ equals to $(S', Toks')$, where the construction of S' from $(S'', Toks'')$ is given by Table 6.2 and $Toks' = Toks'' \setminus (S' \times \Sigma)$, i.e. all states added to S' are removed from $Toks'$ so a state belongs either to S' or to $st(Toks')$.

Construction of state S' , the smallest set including S'' and such that:

6. for all $(q_1, m_1), (q_2, m_2) \in Toks''$ such that
 - $m_1 \neq m_2$ and $m_2 \notin R(q_1)$ and $m_1 \in R(q_2)$,
 we have $q_1 \in S'$;
 7. for all $(q_1, m_1), (q_2, m_2), (q_3, m_2) \in Toks''$ such that
 - $m_1 \neq m_2$, and $(q_2, ?m_1, q_3) \in T$,
 we have $q_1 \in S'$;
 8. for all $(q_1, m_1), (q_2, m_2), (q_3, m_3) \in Toks''$ such that
 - $m_1 \neq m_2, m_1 \neq m_3$ and $m_2 \neq m_3$ and
 - $m_1 \notin R(q_2)$ and $m_1 \in R(q_3)$ and
 - $m_2 \notin R(q_1)$ and $m_2 \in R(q_3)$, and
 - $m_3 \in R(q_2)$ and $m_3 \in R(q_1)$,
 we have $q_1 \in S'$.
-

Table 6.2: Definition of S' where $F(\gamma) = (S', Toks')$ for $(S'', Toks'')$.

Example 6.10. Now the case of state q_5 evoked in the previous example leads to the application of rule 7, since $(q_5, c), (q_1, a), (q_3, a) \in Toks''_2$, and $(q_1, ?c, q_3) \in T$. Finally, we get that

$$F(S''_1, Toks''_1) = F(F(\{q_{in}\}, \emptyset)) = (\{q_{in}, q_2, q_4, q_5, q_6, q_7\}, \{(q_1, a), (q_1, b), (q_3, a), (q_3, b)\}).$$

Since q_1 and q_3 are not conflict-free, they won't be reachable together in a configuration. We consider now the wait-only protocol P_2 depicted on Figure 6.19. In that case, to compute $F(\{q_{in}\}, \emptyset)$ we will first have

$$S'' = \{q_{in}\} \text{ and } Toks'' = \{(q_1, a), (q_2, b), (p_1, m_1), (p_2, m_2), (p_3, m_3)\}$$

(using rule 2b), to finally get

$$F(\{q_{in}\}, \emptyset) = (\{q_{in}, q_1, p_1\}, \{(q_2, b), (p_2, m_2), (p_3, m_3)\}).$$

Applying rule 6 to tokens (q_1, a) and (q_2, b) from $Toks''$, we obtain that $q_1 \in S'$: whenever one manages to obtain one process in state q_2 , this process can receive message a instead of processes in state q_1 , allowing one to obtain as many processes as desired in state q_1 . Now, since (p_1, m_1) , (p_2, m_2) and (p_3, m_3) are in $Toks''$ and respect the conditions of rule 8, p_1 is added to the set S' of unbounded states. This case is a generalization of the previous one, with 3 processes. Once one process has been put on state p_2 from q_{init} , iterating the following actions: rendez-vous over m_3 , rendez-vous over m_1 , sending of m_2 , will ensure as many processes as one wants on state p_1 . Finally applying successively F , we get in this case the token-set $(\{q_{in}, q_1, q_3, p_1, p_2, p_3, p_4\}, \{(q_2, b)\})$.

We show that F satisfies several properties: the following lemma ensures that when repeatedly applying F to $(\{q_{in}\}, \emptyset)$, the computation eventually reaches a consistent token-set γ_f such that $\gamma_f = F(\gamma_f)$. Moreover, each step runs in polynomial time.

Lemma 6.15. For all consistent $\gamma \in \Gamma$, $F(\gamma)$ is also consistent and can be computed in polynomial time in the size of P .

Proof. The fact that $F(\gamma)$ can be computed in polynomial time is a direct consequence of the definition of F (see Table 6.1 and 6.2).

Assume $\gamma = (S, Toks) \in \Gamma$ to be consistent. Note $(S'', Toks'')$ the intermediate sets computed during the computation of $F(\gamma)$, and note $F(\gamma) = (S', Toks')$.

To prove that $F(\gamma)$ is consistent, we need to argue that

- (1) for all $(q, m) \in Toks'' \setminus Toks$, there exists a path $(q_0, \alpha_0, q_1)(q_1, \alpha_1, q_2) \dots (q_k, \alpha_k, q)$ in P such that $q_0 \in S$, $\alpha_0 = !m$, and $\alpha_i = ?m_i$ and there exists $(q'_i, !m_i, q'_i) \in T$ with $q'_i \in S$ for all $1 \leq i \leq k$;
- (2) for all $(q, m), (q', m') \in Toks'$ either $m \in R(q')$ and $m' \in R(q)$ or $m \notin R(q')$ and $m' \notin R(q)$.

We start by proving property (1). If (q, m) has been added to $Toks''$ with rule 2b, then by construction, there exists $p \in S$ such that $(p, !a, p') \in T$, and $(q, m) = (p', a)$. The sequence of transitions, is the single transition $(p, !a, q)$.

If (q, m) has been added to $Toks''$ with rule 4b, then there exists $(q', m) \in Toks$, and $(q', ?a, q)$ with $m \neq a$. Furthermore, $m \in R(q)$ and there exists $(p, !a, p') \in T$ with $p \in S$. By hypothesis, γ is consistent, hence there exists a finite sequence of transitions $(q_0, \alpha_0, q_1) \dots (q_k, \alpha_k, q')$ such that $q_0 \in S$, and $\alpha_0 = !m$ and for all $1 \leq i \leq k$, we have that $\alpha_i = ?m_i$ and that there exists $(q'_i, !m_i, q'_{i+1}) \in T$ with $q'_i \in S$. By completing this sequence with transition $(q', ?a, q)$ we get an appropriate finite sequence of transitions.

It remains to prove property (2). Assume there exist $(q, m), (q', m') \in Toks'$ such that $m \in R(q')$ and $m' \notin R(q)$, then as $Toks' \subseteq Toks''$, $(q, m), (q', m') \in Toks''$. By condition 6, $q \in S'$, therefore, as $Toks' = \{(p, a) \in Toks'' \mid p \notin S'\}$, we have that $(q, m) \notin Toks'$, and we reached a contradiction. $\square$

The next lemma provides the key ingredient to guarantee that the algorithm terminates in a polynomial number of steps. It states that by successively applying F , we always make “progress”: either new states are added to S , or the set of tokens grows (or remains equal). When both S and $Toks$ remain unchanged after applying F , we have reached a fixpoint.

Lemma 6.16. If $(S', \text{Toks}') = F(S, \text{Toks})$ then $S \subsetneq S'$ or $\text{Toks} \subseteq \text{Toks}'$.

Proof. From the construction of F (see Table 6.1 and 6.2), we have $S \subseteq S'' \subseteq S'$. Assume now that $S = S'$. First note that $\text{Toks} \subseteq \text{Toks}''$ (see Table 6.1) and that $\text{st}(\text{Toks}) \cap S = \emptyset$. But $\text{Toks}' = \{(q, m) \in \text{Toks}'' \mid q \notin S'\} = \{(q, m) \in \text{Toks}'' \mid q \notin S\}$. Hence the elements that are removed from Toks'' to obtain Toks' are not elements of Toks . Consequently $\text{Toks} \subseteq \text{Toks}'$. $\square$

6.5.3 Completeness

The next lemma proves the completeness of the computed abstraction. We consider a configuration M that respects a consistent token-set γ and show that a single step from M yields a configuration M' that respects $F(\gamma)$.

Specifically, we show that: (1) each state q populated in M' appears in $F(\gamma)$; (2) states that appear in the token-part of $F(\gamma)$ are populated by at most one process. Moreover, if two such states are populated, they must be conflict-free.

Lemma 6.17. For all consistent $\gamma \in \Gamma$, if $M \in \llbracket \gamma \rrbracket$ and $M \rightarrow M'$ then $M' \in \llbracket F(\gamma) \rrbracket$.

Proof. Let $\gamma = (S, \text{Toks}) \in \Gamma$ be a consistent abstract set of configurations, and $M \in \mathcal{M}$ such that $M \in \llbracket \gamma \rrbracket$ and $M \rightarrow M'$. We denote by (S', Toks') the token-set $F(\gamma)$ and by $\gamma' = (S'', \text{Toks}'')$ the intermediate sets used to compute $F(\gamma)$. We will first prove that:

- (1) for all state q such that $M'(q) > 0$, $q \in S'$ or $q \in \text{st}(\text{Toks}')$,

and then we will prove that

- (2) for all states q such that $q \in \text{st}(\text{Toks}')$ and $M'(q) > 0$, (2a) $M'(q) = 1$ and (2b) for all other states $p \in \text{st}(\text{Toks}')$ such that $M'(p) > 0$, p and q are **conflict-free**.

Observe that $S \subseteq S'' \subseteq S'$, $\text{Toks} \subseteq \text{Toks}''$, and $\text{st}(\text{Toks}'') \subseteq \text{st}(\text{Toks}') \cup S'$.

Proof of (1). First, let us prove that for every state q such that $M'(q) > 0$, it holds that $q \in S' \cup \text{st}(\text{Toks}')$. Note that for all q such that $M(q) > 0$, because M respects γ , $q \in \text{st}(\text{Toks}) \cup S$. As $\text{st}(\text{Toks}) \cup S \subseteq \text{st}(\text{Toks}'') \cup S'$, the property holds for q . Hence, we only need to consider states q such that $M(q) = 0$ and $M'(q) > 0$.

If $M \xrightarrow{(q', !a, q)} M'$ then q' is an action state. Then $q' \in S$, (recall that $\text{Toks} \subseteq Q_W \times \Sigma$). Hence, q should be added to $\text{st}(\text{Toks}'') \cup S'$ by condition 2a or 2b.

If $M \xrightarrow{(p, !a, p')} M'$ and $p' \neq q$, then $(q', ?a, q) \in T$ with $q' \in S \cup \text{st}(\text{Toks})$, and it should be added to $\text{st}(\text{Toks}'') \cup S'$ by condition 3, 4a, or 4b. Therefore, we proved that for all state q such that $M'(q) > 0$, it holds that $q \in \text{st}(\text{Toks}') \cup S'$.

The rest of the proof is devoted to prove (2).

Let q such that $M'(q) > 0$. Note that if $q \in \text{st}(\text{Toks})$ and $M(q) = M'(q) = 1$, then for every state p such that $p \in \text{st}(\text{Toks})$ and $M(p) = M'(p) = 1$, it holds that q and p are conflict-free.

Let $q \in \text{st}(\text{Toks}') \setminus \text{st}(\text{Toks})$ with $M'(q) > 0$, we shall prove that $M'(q) = 1$ and for all $p \in \text{st}(\text{Toks}')$ and $M'(p) > 0$, q and p are conflict-free. If $q \in \text{st}(\text{Toks}') \setminus \text{st}(\text{Toks})$, it implies that $M(q) = 0$ because M respects γ . Hence, we fall in one of the following cases:

- **Case A.** $M \xrightarrow{s.(q',!a,q)} M'$ with $(q',!a,q) \in T$;
- **Case B.** $M \xrightarrow{rdv.(q_1,!a,q'_1)} M'$ for a transition $(q_1,!a,q'_1) \in T$ and there exists $(q_2,?a,q'_2) \in T$ with $M' = M - \{q_1, q_2\} + \{q'_1, q'_2\}$. In that case, $q = q'_1$ or $q = q'_2$. Then we should be careful as we need to prove that $q'_2 \neq q'_1$, otherwise, $M'(q) = 2$.

Case A: Note that as only one process moves between M and M' and $M(q) = 0$, it is trivial that $M'(q) = 1$ (2a). In this case, as it is a sending of a between M and M' (and not a rendez-vous), it holds that: for all $p \in \text{st}(Toks)$ such that $M(p) = 1$, $a \notin R(p)$ (as otherwise, the process in p needs to receive a). Take $p \in \text{st}(Toks')$, such that $p \neq q$ and $M'(p) = 1$. Observe that $(q, a) \in Toks'$ by construction. Then $M'(p) = M(p) = 1$ and so $p \in \text{st}(Toks)$, and $a \notin R(p)$. Suppose $(p, m) \in Toks'$ such that $m \in R(q)$, then we found two tokens in $Toks'$ such that $m \in R(q)$ and $a \notin R(p)$ which contradicts $F(\gamma)$'s consistency (Lemma 6.15). Hence, p and q are conflict-free (2b).

Case B: We start by proving some small facts.

- (i) If $q'_2 \in \text{st}(Toks')$, then $q_2 \in \text{st}(Toks)$.

Proof. Otherwise, q'_2 should be in S' by condition 3. □

- (ii) If $q'_1 \in \text{st}(Toks')$, then $a \in R(q'_1)$.

Proof. Otherwise, q'_1 should be in S' by condition 2a. □

- (iii) If $q'_1 \in \text{st}(Toks')$, then $(q'_1, a) \in Toks'$.

Proof. From (ii) and condition 2b. □

- (iv) If $q'_1 \in \text{st}(Toks')$, then $q_2 \in \text{st}(Toks)$.

Proof. Otherwise q'_1 should be added to S' by condition 2a. □

We now prove that either $q'_1 \in S'$, or $q'_2 \in S'$.

For the sake of contradiction, assume this is not the case. Then, thanks to (i), let $(q_2, m) \in Toks$, with $(q'_2, m) \in Toks'$ (by condition 4b it must exist such two tokens with the same message m). Then, together with (iii), there are three tokens $(q'_1, a), (q_2, m), (q'_2, m) \in Toks' \subseteq Toks''$, such that $(q_2, ?a, q'_2) \in T$. If $m \neq a$, from condition 7, q'_1 should be added to S' and so $(q'_1, a) \notin Toks'$. It is not possible that $m = a$, as otherwise, $q'_2 \in S'$ from condition 3. Hence, $q'_1 \in S'$, or $q'_2 \in S'$.

Note that, as a consequence $q'_1 \neq q'_2$ or $q'_1 = q'_2 \in S'$. Take $q \in \text{st}(Toks') \setminus \text{st}(Toks)$ such that $M'(q) > 0$, if such a q exists, then $q = q'_1$ or $q = q'_2$ and $q'_1 \neq q'_2$. As a consequence, $M'(q) = 1$ (note that if $q'_1 = q_2$, $M(q_2) = 1$) which proves (2a).

It is left to prove (2b). Take $p \in \text{st}(Toks') \setminus \{q\}$ such that $M'(p) > 0$, we now prove that q and p are conflict-free, i. e. (2b). We first justify that $p \notin \{q_1, q'_1, q_2, q'_2\}$. As $p \neq q$ and $q \in \{q'_1, q'_2\}$ and $(q'_1 \in S' \text{ or } q'_2 \in S')$, we deduce that $p \notin \{q'_1, q'_2\}$. If $p = q_1$, then $M'(q_1) = 1$. As $M' = M - \{q_1, q_2\} + \{q'_1, q'_2\}$, it implies that

$$M(q_1) = 2 \text{ or } p = q_1 = q'_1 \text{ or } p = q_1 = q'_2.$$

In the first case, it implies that $p = q_1 \in S$ because M respects γ , which is absurd given that $p \in \text{st}(Toks')$. Hence $M(p) = M'(p)$ and $p \in \text{st}(Toks)$.

The two latter cases are absurd as we justified that $p \notin \{q'_1, q'_2\}$. We now distinguish between cases.

- **(Case $q = q'_1$)** Assume $q = q'_1$ and assume q and p are not **conflict-free**. As $q'_1 \in \text{st}(\text{Toks}')$, it holds that $q_2 \in \text{st}(\text{Toks})$ by (iv). As M respects γ , there exists (p, m_p) and $(q_2, m) \in \text{Toks}$ such that $m_p \notin R(q_2)$ and $m \notin R(p)$ (q_2 and p are conflict-free). As $p \in \text{st}(\text{Toks}')$, $(p, m_p) \in \text{Toks}'$ and so $m_p \in R(q)$ or $a \in R(p)$ (q and p are not conflict-free). As $F(\gamma)$ is consistent, $m_p \in R(q)$ and $a \in R(p)$. Note that $a \neq m_p$ because $a \in R(q_2)$, $a \neq m$ because $m \notin R(p)$, and obviously $m \neq m_p$. Note also that if $m \notin R(q)$, then we found two tokens (q, a) and (q_2, m) in Toks' such that $a \in R(q_2)$ and $m \notin R(q)$, which contradicts the fact that $F(\gamma)$ is consistent (Lemma 6.15). Hence, $m \in R(q)$. Note that even if q_2 is added to S'' , it still is in Toks'' . As $\text{Toks}' \subseteq \text{Toks}''$ we found three tokens $(p, m_p), (q_2, m), (q, a)$ in Toks'' , satisfying condition 8, and so p should be added to S' , which is absurd as $p \in \text{st}(\text{Toks}')$. We reach a contradiction and so q and p should be conflict-free.
- **(Case $q = q'_2$)** Finally assume $q = q'_2$. As $q'_2 \in \text{st}(\text{Toks}')$, it holds that $q_2 \in \text{st}(\text{Toks})$ by (i). If $q = q_2$, then, because M respects γ , q and p are conflict-free. Otherwise, as q_2 is conflict-free with p , there exists (q_2, m) and (p, m_p) in Toks such that $m \notin R(p)$ and $m_p \notin R(q_2)$. Note that $(q, m) \in \text{Toks}''$ from condition 4b (otherwise, $q \in S''$ which is absurd). Hence, $(q, m) \in \text{Toks}'$ and, as $p \in \text{st}(\text{Toks}')$, (p, m_p) is conserved from Toks to Toks' . It remains to show that $m_p \notin R(q)$. Assume this is not the case, then there exists (p, m_p) and $(q, m) \in \text{Toks}'$ such that $m \notin R(p)$ and $m_p \in R(q)$ which is absurd given $F(\gamma)$'s consistency. As a consequence, q and p are conflict-free.

We managed to prove that for all q such that $M'(q) > 0$, $q \in S' \cup \text{st}(\text{Toks}')$, and if $q \in \text{st}(\text{Toks}')$, then $M'(q) = 1$ and for all others $p \in \text{st}(\text{Toks}')$ such that $M'(p) = 1$, p and q are conflict-free. □

6.5.4 Soundness

In order to prove soundness, we first present two technical lemmas. The first lemma states that if there is an execution from M to M' , then the same sequence of steps can be realized from any larger configuration D . This property follows from the fact that the network's transition system is well-structured. Furthermore, in a rendez-vous network, each state loses at most 2ℓ processes, where ℓ is the length of the run between M and M' . Indeed, at each step, at most two processes change states: one if it is a sending step, and two if it is a rendez-vous. Thus, a state loses at most two processes per step, which leads to the following lemma.

Lemma 6.18. Let $M, M' \in \mathcal{M}$ such that $M = M_0 \rightarrow M_1 \cdots \rightarrow M_\ell = M'$. Then, the following two properties hold.

1. For all $q \in Q$ such that $M(q) = 2 \cdot \ell + x$ for some $x \in \mathbb{N}$, we have $M'(q) \geq x$.
2. For all $D_0 \in \mathcal{M}$ such that $D_0 \geq M_0$, there exist $D_1, \dots, D_\ell$ such that $D_0 \rightarrow D_1 \cdots \rightarrow D_\ell$ and $D_i \geq M_i$ for all $1 \leq i \leq \ell$.

Proof. By the semantics of rendez-vous protocols, each step in the execution from M to M' removes at most two processes per state q , which proves the first item. The second item is a direct consequence from the fact that $(\mathcal{M}, \rightarrow, \leq)$ is a WSTS, which we proved in Lemma 2.4 (Chapter 2, page 26). $\square$

We have seen that in the networks under study in this thesis in general, states can be in conflict when one aims at covering them. We show here that in the Rdv networks as soon as two states can be covered separately by an unbounded number of processes, they can be covered together by an unbounded number of processes. Note that this result is *stronger* than the cypypaste property (Proposition 6.1), as the subset of states may include both action and waiting states. Thus, we can ensure that at least n processes reach each state in the subset simultaneously, including waiting states.

Lemma 6.19. Let γ be a consistent token-set of configurations. Given a subset of states $U \subseteq Q$, if for all $n \in \mathbb{N}$ and for all $q \in U$ there exists $M_q \in \llbracket \gamma \rrbracket$ and $M'_q \in \mathcal{M}$ such that $M_q \rightarrow^* M'_q$ and $M'_q(q) \geq n$, then for all $n \in \mathbb{N}$, there exists $M \in \llbracket \gamma \rrbracket$ and $M' \in \mathcal{M}$ such that $M \rightarrow^* M'$ and $M'(q) \geq n$ for all $q \in U$.

Proof. We suppose $\gamma = (S, Toks)$ a consistent token-set of configurations, and a set of states $U \subseteq Q$, and reason by induction on the number of elements in $U \setminus S$.

Base case. The base case is obvious. Indeed assume $U \setminus S = \emptyset$ and let $n \in \mathbb{N}$. We define the configuration M such that $M(q) = n$ for all $q \in S$ and $M(q) = 0$ for all $q \in Q \setminus S$. It is clear that $M \in \llbracket \gamma \rrbracket$ and that $M(q) \geq n$ for all $q \in U$ (since $U \setminus S = \emptyset$, we have in fact $U \subseteq S$).

Induction case. We now assume that the property holds for a set U and we shall see it holds for $U \cup \{p\}$, $p \notin S$. We assume hence that for all $n \in \mathbb{N}$ and for all $q \in U \cup \{p\}$ there exists $M_q \in \llbracket \gamma \rrbracket$ and $M'_q \in \mathcal{M}$ such that $M_q \rightarrow^* M'_q$ and $M'_q(q) \geq n$. Let $n \in \mathbb{N}$. By induction hypothesis,

there exists $M_U \in \llbracket \gamma \rrbracket$, $M'_U \in \mathcal{M}$ s.t. $M_U \rightarrow^* M'_U$ and $M'_U(q) \geq n$ for all $q \in U$.

We denote by ℓ_U the minimal number of steps in a run from M_U to M'_U . We will see that we can build a configuration $M \in \llbracket \gamma \rrbracket$ such that:

$$M \rightarrow^* M''_U \text{ with } M''_U \geq M_U \text{ and } M''_U(p) \geq n + 2 \cdot \ell_U.$$

Using Lemma 6.18, we will then have that:

$$M''_U \rightarrow^* M' \text{ with } M' \geq M'_U \text{ and } M'(p) \geq n.$$

This will allow us to conclude.

Having $M_U \in \llbracket \gamma \rrbracket$, we name $(q_1, m_1) \dots (q_k, m_k)$ the tokens in $Toks$ such that $M_U(q_j) = 1$ for all $1 \leq j \leq k$, (and for all $q \in \text{st}(Toks) \setminus \{q_j\}_{1 \leq j \leq k}$, $M_U(q) = 0$). Since γ is consistent, for each (q_j, m_j) there exists a path $(q_{0,j}, !m_j, q_{1,j})(q_{1,j}, ?m_{1,j}, q_{2,j}) \dots (q_{\ell_j,j}, ?m_{\ell_j,j}, q_j)$ in P such that $q_{0,j} \in S$ and such that there exists $(q'_{i,j}, !m_{i,j}, q''_{i,j}) \in T$ with $q'_{i,j} \in S$ for all $1 \leq i \leq \ell_j$. We denote by $\ell = \max_{1 \leq j \leq k} (\ell_j) + 1$. We know also that there exist $M_p \in \llbracket \gamma \rrbracket$ and $M'_p \in \mathcal{M}$ such that $M_p \rightarrow^* M'_p$ and $M'_p(p) \geq n + 2 \cdot \ell_U + (k \cdot \ell)$. We denote by ℓ_p

the minimum number of steps in a run from M_p to M'_p . We build the configuration M as follows:

$$\begin{aligned} M(q) &= M_U(q) + 2 \cdot \ell_p + (k \cdot \ell) + M_p(q) && \text{for all } q \in S \\ M(q) &= M_p(q) && \text{for all } q \in \text{st}(\text{Toks}) \end{aligned}$$

note that since $M_p \in \llbracket \gamma \rrbracket$, we have that $M \in \llbracket \gamma \rrbracket$. Furthermore, we have $M \geq M_p$, hence using again Lemma 6.18, we know that there exists a configuration M''_p such that $M \rightarrow^* M''_p$ and $M''_p \geq M'_p$, i.e.

$$\begin{aligned} M''_p(p) &\geq n + 2 \cdot \ell_U + (k \cdot \ell) \\ M''_p(q) &\geq M_U(q) + (k \cdot \ell) + M_p(q) \text{ for all } q \in S. \end{aligned}$$

We now prove the following claim.

Claim 6.1. For all $1 \leq i < j \leq k$, for all $(q_i, m_i), (q_j, m_j) \in \text{Toks}$, $m_i \notin R(q_j)$ and $m_j \notin R(q_i)$.

Proof. Assume there exists $1 \leq i < j \leq k$ such that $(q_i, m_i), (q_j, m_j) \in \text{Toks}$ and $M_U(q_i) = M_U(q_j) = 1$, and $m_i \in R(q_j)$ and $m_j \in R(q_i)$, because γ is consistent. Since M_U respects $\llbracket \gamma \rrbracket$, q_i and q_j are conflict-free: there exist $(q_i, m), (q_j, m') \in \text{Toks}$ such that $m \notin R(q_j)$ and $m' \notin R(q_i)$. Hence, $(q_i, m_i), (q_i, m), (q_j, m_j), (q_j, m') \in \text{Toks}$, and $m \notin R(q_j)$ and $m_j \in R(q_i)$. Therefore, we have $(q_i, m), (q_j, m_j) \in \text{Toks}$ and $m \notin R(q_j)$ and $m_j \in R(q_i)$, which is in contradiction with the fact that γ is consistent. Hence, for all $1 \leq i < j \leq k$, for all $(q_i, m_i), (q_j, m_j) \in \text{Toks}$, $m_i \notin R(q_j)$ and $m_j \notin R(q_i)$. $\square$

We shall now explain how from M''_p we reach M''_U in $k \cdot \ell$ steps, i.e. how we put (at least) one process in each state q_j such that $q_j \in \text{st}(\text{Toks})$ and $M_U(q_j) = 1$ in order to obtain a configuration $M''_U \geq M_U$. We begin by q_1 . Let a process on $q_{0,1}$ send the message m_1 (remember that $q_{0,1}$ belongs to S) and let ℓ_1 other processes on states of S send the messages needed for the process to reach q_1 following the path $(q_{0,1}, !m_1, q_{1,1})(q_{1,1}, ?m_{1,1}, q_{2,1}) \dots (q_{\ell_1,1}, ?m_{\ell_1,1}, q_1)$. At this stage, we have that the number of processes in each state q in S is bigger than $M_U(q) + ((k-1) \cdot \ell) + M_p(q)$ and we have (at least) one process in q_1 . We proceed similarly to put a process in q_2 , note that the message m_2 sent at the beginning of the path cannot be received by the process in q_1 since, as explained above, $m_2 \notin R(q_1)$.

We proceed again to put a process in the states q_1 to q_k and at the end we obtain the configuration M''_U with the desired properties. $\square$

We are now ready to prove the soundness of our construction.

Lemma 6.20. For all consistent $\gamma \in \Gamma$, if $M' \in \llbracket F(\gamma) \rrbracket$, then there exists $M'' \in \mathcal{M}$ and $M \in \llbracket \gamma \rrbracket$ such that $M \rightarrow^* M''$ and $M'' \geq M'$.

Let γ be a consistent token-set of configurations and $M' \in \llbracket F(\gamma) \rrbracket$. We write $\gamma = (S, \text{Toks})$ and $F(\gamma) = \gamma' = (S', \text{Toks}')$. We first show that, starting from a configuration in $\llbracket \gamma \rrbracket$, we can cover all states in S' simultaneously with any number of processes. To do so, we prove that

each state in S' is individually coverable by any number of processes, and then we apply Lemma 6.19.

Claim 6.2. For any $n \in \mathbb{N}$, there exist $M \in \llbracket \gamma \rrbracket$ and $M' \in \mathcal{M}$ such that $M \mapsto^* M'$ and $M'(q) \geq n$ for all $q \in S'$.

Proof. We will first show that for all $n \in \mathbb{N}$, for all $q \in S'$ there exists a configuration $M_q \in \llbracket \gamma \rrbracket$ and a configuration $M'_q \in \mathcal{M}$ such that $M_q \mapsto^* M'_q$ and $M'_q(q) \geq n$. This will allow us to rely then on Lemma 6.19 to conclude.

Take $n \in \mathbb{N}$ and $q \in S'$, if $q \in S$, then take $M_q \in \llbracket \gamma \rrbracket$ to be $\wr n \cdot q \S$. Clearly $M_q \in \llbracket F(\gamma) \rrbracket$, $M_q(q) \geq n$ and $M_q \mapsto^* M_q$. Now let $q \in S' \setminus S$. Note $(Toks'', S'')$ the intermediate sets of $F(\gamma)$'s computation.

Case 1: $q \in S''$. As a consequence, q was added to S'' by one of the conditions 2a, 3 or 4a.

- Case 2a and $a \notin R(q)$. Denote q' the state such that $(q', !a, q)$, and consider the configuration $M_q = \wr n \cdot q' \S$. By doing n sendings, we reach $M'_q = \wr n \cdot q \S$. Note that messages are not received as $q' \in Q_A$ and $a \notin R(q)$. It holds that $M'_q \in \llbracket F(\gamma) \rrbracket$.
- Case 2a and $a \in R(q)$ or case 3. Note $(q_1, !a, q'_1)$ and $(q_2, ?a, q'_2)$ the two transitions realizing the conditions. As a consequence $q_1, q_2 \in S$. Take the configuration $M_q = \wr n \cdot q_1, n \cdot q_2 \S$. $M_q \in \llbracket \gamma \rrbracket$ and by doing n successive rendez-vous on the letter a , we reach configuration $M'_q = \wr n \cdot q'_1, n \cdot q'_2 \S$. Hence, $M'_q \in \llbracket F(\gamma) \rrbracket$, and as $q \in \{q'_1, q'_2\}$, $M'_q(q) \geq n$.
- In case 4a, there exists $(q', m) \in Toks$ such that $(q', ?a, q) \in T$, $m \notin R(q)$, and there exists $p \in S$ such that $(p, !a, p') \in T$. Remember that γ is consistent, and so there exists a finite sequence of transitions $(q_0, !m, q_1)(q_1, ?m_1, q_2) \dots (q_k, ?m_k, q')$ such that $q_0 \in S$ and there exists $(q'_i, !m_i, q''_i) \in T$ with $q'_i \in S$ for all $1 \leq i \leq k$. Consider

$$M_q = \wr (n-1) \cdot q_0, (n-1) \cdot q'_1, \dots, (n-1) \cdot q'_k, n \cdot p, q' \S.$$

Clearly $M_q \in \llbracket \gamma \rrbracket$ as all states except q' are in S and $q' \in \text{st}(Toks)$ with $M_q(q') = 1$. We shall show how to put 2 processes on q from M_q and then explain how to repeat the steps in order to put n . Consider the following run:

$$M_q \xrightarrow{\text{rdv.}(p, !a, p')} M_1 \xrightarrow{(q_0, !m, q_1)} M_2 \xrightarrow{\text{rdv.}(q'_1, !m_1, q''_1)} \dots \xrightarrow{\text{rdv.}(q'_k, !m_k, q''_k)} M_{k+2} \xrightarrow{\text{rdv.}(p, !a, p')} M_{k+3}.$$

The first rendez-vous on a is made with transitions $(p, !a, p')$ and $(q', ?a, q)$. Then either $m \notin R(p')$ and $M_1 \xrightarrow{s.(q_0, !m, q_1)} M_2$, otherwise $M_1 \xrightarrow{\text{rdv.}(q_0, !m, q_1)} M_2$. In any case, the rendez-vous or sending is made with transition $(q_0, !m, q_1)$ and the message is not received by the process on q (because $m \notin R(q)$) and so $M_2 \geq \wr q, q_1 \S$. Then, each rendez-vous on m_i is made with transitions $(q'_i, !m_i, q''_i)$ and $(q_i, ?m_i, q_{i+1})$ ($q_{k+1} = q'$), and the last rendez-vous with transition $(q', ?a, q)$. Hence

$$M_{k+3} \geq \wr (n-2) \cdot q_0, (n-2) \cdot q'_1, \dots, (n-2) \cdot q'_k, (n-2) \cdot p, 2 \cdot q \S.$$

We can reiterate this run (without the first rendez-vous on a) $n - 2$ times to reach a configuration M'_q such that $M'_q \geq \wr n \cdot q \S$.

Case 2: $q \notin S''$. Hence, q should be added to S' by one of the conditions 6, 7, and 8.

- If it was added with condition 6, let $(q_1, m_1), (q_2, m_2) \in \text{Toks}''$ such that $q = q_1$, $m_1 \neq m_2$, $m_2 \notin R(q_1)$ and $m_1 \in R(q_2)$. From the proof of Lemma 6.15, one can actually observe that all tokens in Toks'' correspond to “feasible” paths regarding states in S , i.e there exists a finite sequence of transitions $(p_0, !m_1, p_1)(p_1, ?b_1, p_2) \dots (p_k, ?b_k, q_1)$ such that $p_0 \in S$ and there exists $(p'_i, !b_i, p''_i) \in T$ with $p'_i \in S$ for all $1 \leq i \leq k$. The same such sequence exists for the token (q_2, m_2) , we note the sequence $(s_0, !m_2, s_1)(s_1, ?c_1, s_2) \dots (s_\ell, ?c_\ell, q_2)$ such that $s_0 \in S$ so there exists $(s'_i, !c_i, s''_i) \in T$ with $s'_i \in S$ for all $1 \leq i \leq \ell$. Consider

$$M_q = \wr n \cdot p_0, n \cdot s_0, n \cdot p'_1, \dots, n \cdot p'_k, n \cdot s'_1, \dots, n \cdot s'_\ell \S.$$

Clearly, $M_q \in \llbracket \gamma \rrbracket$, as all states are in S . Consider the following run:

$$M_q \xrightarrow{s.(p_0, !m_1, p_1)} M_1 \xrightarrow{\text{rdv.}(p'_1, !b_1, p''_1)} \dots \xrightarrow{\text{rdv.}(p'_k, !b_k, p''_k)} M_{k+1}$$

Each rendez-vous on letter b_i is made with transitions $(p'_i, !b_i, p''_i)$ and $(p_i, ?b_i, p_{i+1})$ ($p_{k+1} = q_1$). Hence, M_{k+1} is such that

$$M_{k+1} \geq \wr (n-1) \cdot p_0, n \cdot s_0, (n-1) \cdot p'_1, \dots, (n-1) \cdot p'_k, n \cdot s'_1, \dots, n \cdot s'_\ell, q_1 \S$$

From M_{k+1} , consider the following run:

$$M_{k+1} \xrightarrow{(s_0, !m_2, s_1)} M_{k+2} \xrightarrow{\text{rdv.}(s'_1, !c_1, s''_1)} \dots \xrightarrow{(s'_\ell, !c_\ell, s''_\ell)} M_{k+\ell+2} \xrightarrow{(p_0, !m_1, p_1)} M_{k+\ell+3}$$

If no process is on a state in $Q_?(m_2)$ (recall that $Q_?(m_2) = \{q \mid \exists q' \in Q \text{ s.t. } (q, ?m_2, q') \in T\}$) then $M_{k+1} \xrightarrow{s.(s_0, !m_2, s_1)} M_{k+2}$, otherwise $M_{k+1} \xrightarrow{\text{rdv.}(s_0, !m_2, s_1)} M_{k+2}$. In any case, as $m_2 \notin R(q_1)$, hence $M_{k+2} \geq \wr q_1 \S$. Moreover each rendez-vous on letter c_i is made with transitions $(s'_i, !c_i, s''_i)$ and $(s_i, ?c_i, s_{i+1})$ ($s_{k+1} = q_2$), the last rendez-vous on m_1 is made with transitions $(p_0, !m_1, p_1)$ and $(q_2, ?m_1, q'_2)$ (such a q'_2 exists as $m_1 \in R(q_2)$). Hence, $M_{k+\ell+3} \geq \wr p_1, q_1 \S$.

$$M_{k+2} \geq \wr (n-1) \cdot p_0, (n-1) \cdot s_0, (n-1) \cdot p'_1, \dots, (n-1) \cdot p'_k, \\ (n-1) \cdot s'_1, \dots, (n-1) \cdot s'_\ell, q_1, p_1 \S$$

By repeating the two sequences of steps (without the first sending of m_1) $n - 1$ times (except for the last time where we don't need to repeat the second run), we reach a configuration M'_q such that $M'_q \geq \wr n \cdot q_1 \S$.

- If it was added with condition 7, then let $(q_1, m_1), (q_2, m_2), (q_3, m_2) \in \text{Toks}''$ such that $m_1 \neq m_2$ and $(q_2, ?m_1, q_3) \in T$ with $q = q_1$. From the proof of Lemma 6.15, Toks'' is made of “feasible” paths regarding S and so there exists a finite sequence of transitions $(p_0, !m_2, p_1)(p_1, ?b_1, p_2) \dots (p_k, ?b_k, q_2)$ such that $p_0 \in S$ and there exists $(p'_i, !b_i, p''_i) \in T$ with $p'_i \in S$ for all $1 \leq i \leq k$. The same sequence exists for the token (q_1, m_1) , we write the sequence $(s_0, !m_1, s_1)(s_1, ?c_1, s_2) \dots (s_\ell, ?c_\ell, q_1)$

such that $s_0 \in S$ and there exists $(s'_i, !c_i, s''_i) \in T$ with $s'_i \in S$ for all $1 \leq i \leq \ell$. Consider

$$M_q = \wr n \cdot p_0, n \cdot s_0, n \cdot p'_1, \dots, n \cdot p'_k, n \cdot s'_1, \dots, n \cdot s'_\ell \S.$$

Clearly, $M_q \in \llbracket \gamma \rrbracket$, as all states are in S . We do the same run from M_q to M_{k+1} as in the previous case:

$$M_q \xrightarrow{s.(p_0, !m_2, p_1)} M_1 \xrightarrow{\text{rdv.}(p'_1, !b_1, p''_1)} \dots \xrightarrow{\text{rdv.}(p'_k, !b_k, p''_k)} M_{k+1}.$$

Here M_{k+1} is then such that:

$$M_{k+1} \geq \wr (n-2) \cdot p_0, n \cdot s_0, (n-1) \cdot p'_1, \dots, (n-1) \cdot p'_k, n \cdot s'_1, \dots, n \cdot s'_\ell, q_2 \S$$

Then, from M_{k+1} we do the following:

$$M_{k+1} \xrightarrow{(s_0, !m_1, s_1)} M_{k+2} \xrightarrow{(s'_1, !c_1, s''_1)} \dots \xrightarrow{(s'_\ell, !c_\ell, s''_\ell)} M_{k+\ell+2} \xrightarrow{(p_0, !m_2, p_1)} M_{k+\ell+3}$$

The rendez-vous on letter m_1 is made with transits $(s_0, !m_1, s_1)$ and $(q_2, ?m_1, q_3)$. Then, each rendez-vous on letter c_i is made with transitions $(s'_i, !c_i, s''_i)$ and $(s_i, ?c_i, s_{i+1})$ ($s_{k+1} = q_1$), and the last rendez-vous on letter m_2 is made with transitions $(p_0, !m_2, p_1)$ and $(q_3, ?m_2, q'_3)$ (such a state q'_3 exists as $(q_3, m_2) \in \text{Toks}''$ and so $m_2 \in R(q_3)$). Hence, $M_{k+\ell+3}$ is such that:

$$M_{k+\ell+3} \geq \wr (n-2) \cdot p_0, (n-1) \cdot s_0, (n-1) \cdot p'_1, \dots, (n-1) \cdot p'_k, \\ (n-1) \cdot s'_1, \dots, (n-1) \cdot s'_\ell, q_1, p_1 \S$$

We can repeat the steps from M_q (except the first sending of m_2 from p_0), $n-1$ times (except for the last time where we don't need to repeat the second part of the run), to reach a configuration M'_q such that $M'_q \geq \wr n \cdot q_1 \S$.

- If it was added with condition 8, then let $(q_1, m_1), (q_2, m_2), (q_3, m_3) \in \text{Toks}''$, such that $m_1 \neq m_2, m_2 \neq m_3, m_1 \neq m_3$, and $m_1 \notin R(q_2), m_1 \in R(q_3)$, and $m_2 \notin R(q_1), m_2 \in R(q_3)$ and $m_3 \in R(q_2)$ and $m_3 \in R(q_1)$, and $q_1 = q$. Then there exists three finite sequences of transitions:

$$\begin{array}{ll} (p_0, !m_1, p_1)(p_1, ?b_1, p_2) \dots (p_k, ?b_k, p_{k+1}) & \text{with } p_{k+1} = q_1 \\ (s_0, !m_2, s_1)(s_1, ?c_1, s_2) \dots (s_\ell, ?c_\ell, s_{\ell+1}) & \text{with } s_{\ell+1} = q_2 \\ (r_0, !m_3, r_1)(r_1, ?d_1, r_2) \dots (r_j, ?d_j, r_{j+1}) & \text{with } r_{j+1} = q_3 \end{array}$$

We denote the *multiset* of messages $\wr b_{i_1}, c_{i_2}, d_{i_3} \mid 1 \leq i_1 \leq k, 1 \leq i_2 \leq \ell, 1 \leq i_3 \leq j \S$ by Mess . For all messages $a \in \text{Mess}$, there exists $q_a \in S$ such that $(q_a, !a, q'_a)$. Consider

$$M_q = \wr n \cdot p_0, n \cdot s_0, n \cdot r_0 \S + \sum_{a \in \text{Mess}} \wr n \cdot q_a \S.$$

From M_q , consider the following run:

$$M_q \xrightarrow{s.(p_0, !m_1, p_1)} M_1 \xrightarrow{\text{rdv.}(q_{b_1}, !b_1, q'_{b_1})} \dots \xrightarrow{\text{rdv.}(q_{b_k}, !b_k, q'_{b_k})} M_{k+1}.$$

Each rendez-vous with letter b_i is made with transitions $(q_{b_i}, !b_i, q'_{b_i})$ and $(p_i, ?b_i, p_{i+1})$. Hence,

$$M_{k+1} \geq \langle q_1, (n-1) \cdot p_0, n \cdot s_0, n \cdot r_0 \rangle + \sum_{a \in \text{Mess} - \{b_1 \dots b_k\}} \langle n \cdot q_a \rangle \\ + \sum_{a \in \{b_1 \dots b_k\}} \langle (n-1) \cdot q_a \rangle$$

Then, we continue the run in the following way:

$$M_{k+1} \xrightarrow{(s_0, !m_2, s_1)} M_{k+2} \xrightarrow{\text{rdv.}(q_{c_1}, !c_1, q'_{c_1})} \dots \xrightarrow{(q_{c_\ell}, !c_\ell, q'_{c_\ell})} M_{k+\ell+2}$$

If there is no process on a state in $Q?(m_2)$ then $M_{k+1} \xrightarrow{s.(s_0, !m_2, s_1)} M_{k+2}$, and otherwise $M_{k+1} \xrightarrow{\text{rdv.}(s_0, !m_2, s_1)} M_{k+2}$. In any case, the rendez-vous is not answered by a process on state q_1 because $m_2 \notin R(q_1)$. Furthermore, each rendez-vous with letter c_i is made with transitions $(q_{c_i}, !c_i, q'_{c_i})$ and $(s_i, ?c_i, s_{i+1})$. Hence,

$$M_{k+\ell+2} \geq \langle q_2, q_1, (n-1) \cdot p_0, (n-1) \cdot s_0, n \cdot r_0 \rangle + \sum_{a \in \{d_1 \dots d_k\}} \langle n \cdot q_a \rangle \\ + \sum_{a \in \text{Mess} - \{d_1 \dots d_k\}} \langle (n-1) \cdot q_a \rangle$$

From $M_{k+\ell+2}$ let the following run be:

$$M_{k+\ell+2} \xrightarrow{\text{rdv.}(r_0, !m_3, r_1)} M_{k+\ell+3} \xrightarrow{\text{rdv.}(q_{d_1}, !d_1, q'_{d_1})} \dots \xrightarrow{\text{rdv.}(q_{d_j}, !d_j, q'_{d_j})} M_{k+\ell+j+3}$$

where the rendez-vous on letter m_3 is made with transitions $(r_0, !m_3, r_1)$ and $(q_2, ?m_3, q'_2)$ (this transition exists as $m_3 \in R(q_2)$). Each rendez-vous on d_i is made with transitions $(q_{d_i}, !d_i, q'_{d_i})$ and $(r_i, ?d_i, r_{i+1})$. Hence, the configuration $M_{k+\ell+j+3}$ is such that:

$$M_{k+\ell+2} \geq \langle q_3, q_1, (n-1) \cdot p_0, (n-1) \cdot s_0, (n-1) \cdot r_0 \rangle + \sum_{a \in \text{Mess}} \langle (n-1) \cdot q_a \rangle$$

Then from $M_{k+\ell+j+3}$: $M_{k+\ell+j+3} \xrightarrow{\text{rdv.}(p_0, !m_1, p_1)} M_{k+\ell+j+4}$ where the rendez-vous is made with transitions $(p_0, !m_1, p_1)$ and $(q_3, ?m_1, q'_3)$ (this transition exists as $m_1 \in R(q_3)$). By repeating $n-1$ times the run from configuration M_q (without the first sending of m_1) from $M_{k+\ell+j+4}$, we reach a configuration M'_q such that $M'_q(q_1) \geq n$.

Hence, for all $n \in \mathbb{N}$, for all $q \in S'$, there exists $M_q \in \llbracket \gamma \rrbracket$, such that $M_q \xrightarrow{*} M'_q$ and $M'_q(q) \geq n$. From Lemma 6.19, there exists M'_n and $M_n \in \llbracket \gamma \rrbracket$ such that $M_n \xrightarrow{*} M'_n$ and for all $q \in S'$, $M_n(q) \geq n$. $\square$

The above claim shows that, from a configuration in $\llbracket \gamma \rrbracket$, we can cover S' with any number of processes simultaneously. It remains to show that we can also cover Toks' (simultaneously with S'). However, we do not need to (and, in fact, cannot) cover all states in $\text{st}(\text{Toks}')$ simultaneously: we only need to cover the states occupied by M' . The proof of Lemma 6.20 below explains how this follows from the definition of conflict-freeness.

Proof of Lemma 6.20. We have γ a consistent token-set of configurations and we let $M' \in \llbracket F(\gamma) \rrbracket$.

Let us remind what we want to prove: there exists $M \in \llbracket \gamma \rrbracket$ and $M'' \in \mathcal{M}$ such that $M \rightarrow^* M''$ and $M'' \geq M'$.

Thanks to Claim 6.2, we know how to build for any $n \in \mathbb{N}$, a configuration M'_n such that $M'_n(q) \geq n$ for all states $q \in S'$ and a configuration $M_n \in \llbracket \gamma \rrbracket$, such that $M_n \rightarrow^* M'_n$. In particular for n bigger than the maximal value $M'(q)$ for $q \in S'$, M'_n is greater than M' on all the states in S' .

We need to prove that from a configuration M'_n for a particular $n \in \mathbb{N}$, we can reach a configuration M'' such that $M''(q) \geq M'(q)$ for $q \in S' \cup \text{st}(\text{Toks}')$. As M' respects $F(\gamma)$, remember that for all $q \in \text{st}(\text{Toks}')$, $M'(q) \leq 1$. The run is actually built in the same fashion as the one we built at the end of the proof of Lemma 6.19.

We enumerate states $q_1, \dots, q_m$ in $\text{st}(\text{Toks}')$ such that $M'(q_i) = 1$. From Lemma 6.15, $F(\gamma)$ is consistent, and so we write $(p_0^j, ?m_1^j, p_1^j) \dots (p_{k_j}^j, ?m_{k_j}^j, p_{k_j+1}^j)$ the sequence of transitions associated to state q_j such that: $p_{k_j+1}^j = q_j$, and $(q_j, m^j) \in \text{Toks}$ and for all m_i^j , there exists $(q_{m_i^j}, !m_i^j, q'_{m_i^j})$ with $q_{m_i^j} \in S'$. Note that for all $i \neq j$, q_i and q_j are conflict-free (because M' respects $F(\gamma)$) and so there exists $(q_i, m), (q_j, m') \in \text{Toks}'$ such that $m \notin R(q_j)$ and $m' \notin R(q_i)$. As $F(\gamma)$ is consistent, it should be the case for all pairs of tokens $(q_i, a), (q_j, a')$. Hence $m^j \notin R(q_i)$ and $m^i \notin R(q_j)$.

Let $\ell_j = k_j + 1$ and $n_{\max}$ be the maximum value for any $M'(q)$, i.e. $n_{\max} = \max_{q \in Q} M'(q)$. For $n = n_{\max} + \sum_{1 \leq j \leq m} \ell_j$, there exists a configuration M'_n such that there exists $M_n \in \llbracket \gamma \rrbracket$, $M_n \rightarrow^* M'_n$, and $M'_n(q) \geq n$ for all $q \in S'$ (Claim 6.2). In particular, for all $q \in S'$, $M'_n(q) \geq M'(q) + \sum_{1 \leq j \leq m} \ell_j$.

Then, we still have to build a run leading to a configuration M'' such that for all $q \in \text{st}(\text{Toks}')$, $M''(q) \geq M'(q)$. We then use the sequences of transitions associated to each state q_j . With ℓ_1 processes we can reach a configuration $M'_{n+\ell_1}$ such that $M'_{\ell_1}(q_1) \geq 1$:

$$M'_n \xrightarrow{(p_0^1, !m_1^1, p_1^1)} M'_{n+1} \xrightarrow{\text{rdv.}(q_{m_1^1}, !m_1^1, q'_{m_1^1})} \dots \xrightarrow{\text{rdv.}(q_{m_{k_1}^1}, !m_{k_1}^1, q'_{m_{k_1}^1})} M'_{n+\ell_1}$$

If there is no process on $Q_?(m^1)$ then $M'_n \xrightarrow{s.(p_0^1, !m_1^1, p_1^1)} M'_{n+1}$, and $M'_n \xrightarrow{\text{rdv.}(p_0^1, !m_1^1, p_1^1)} M'_{n+1}$ otherwise. Each rendez-vous on m_i^1 is made with transitions $(p_i^1, ?m_i^1, p_{i+1}^1)$ and $(q_{m_i^1}, !m_i^1, q'_{m_i^1})$. As a result, for all $q \in S'$, $M'_{n+\ell_1}(q) \geq M'(q) + \sum_{2 \leq j \leq m} \ell_j$ and $M'_{n+\ell_1}(q_1) \geq 1$. We then do the following execution from $M'_{n+\ell_1}$:

$$M'_{n+\ell_1} \xrightarrow{(p_0^2, !m_2^2, p_1^2)} M'_{n+\ell_1+1} \xrightarrow{\text{rdv.}(q_{m_2^2}, !m_2^2, q'_{m_2^2})} \dots \xrightarrow{\text{rdv.}(q_{m_{k_2}^2}, !m_{k_2}^2, q'_{m_{k_2}^2})} M'_{n+\ell_1+\ell_2}$$

If there is no process on $Q_?(m^2)$, then $M'_{n+\ell_1} \xrightarrow{s.(p_0^2, !m_2^2, p_1^2)} M'_{n+\ell_1+1}$, otherwise $M'_{n+\ell_1} \xrightarrow{s.(p_0^2, !m_2^2, p_1^2)} M'_{n+\ell_1+1}$. Remember that we argued that $m^2 \notin R(q_1)$, and therefore $M'_{n+\ell_1+\ell_2}(q_1) \geq M'_{n+\ell_1}(q_1) \geq 1$.

Each rendez-vous on m_i^2 is made with transitions $(p_i^2, ?m_i^2, p_{i+1}^2)$ and $(q_{m_i^2}, !m_i^2, q'_{m_i^2})$. As a result, $M'_{n+\ell_1+\ell_2}(q) \geq M'(q) + \sum_{3 \leq j \leq m} \ell_j$ for all $q \in S'$ and $M'_{n+\ell_1+\ell_2} \geq \{q_1, q_2\}$.

We can then repeat the reasoning for each state q_i and so reach a configuration M'' such that $M''(q) \geq M'(q)$ for all $q \in S'$ and, $M'' \geq \{q_1, q_2, \dots, q_m\}$. We built the following execution: $M_n \rightarrow^* M'_n \rightarrow^* M''$, such that $M'' \geq M'$, and $M'_n \in \llbracket \gamma \rrbracket$ which concludes the proof. $\square$

6.5.5 Polynomial Time Algorithm

We now present a polynomial-time algorithm to solve CONF-COVER^{nb} for Wait-Only protocols. We define the sequence $(\gamma_n)_{n \in \mathbb{N}}$ as follows:

$$\gamma_0 = (\{q_{init}\}, \emptyset) \quad \text{and} \quad \gamma_{i+1} = F(\gamma_i) \quad \text{for all } i \in \mathbb{N}.$$

First, note that γ_0 is consistent and that $\llbracket \gamma_0 \rrbracket = \mathcal{M}_{init}$ is the set of initial configurations. By Lemma 6.15, we deduce that γ_i remains consistent for all $i \in \mathbb{N}$.

Each application of F to a token-set of configurations $(S, Toks)$ either increases S or $Toks$, or leaves $(S, Toks)$ unchanged (Lemma 6.16). Since S increases at most $|Q|$ times and $Toks$ $|Q| \cdot |\Sigma|$ times, the sequence (γ_n) stabilizes within at most $|Q|^2 \cdot |\Sigma|$ iterations. Let $\gamma_f = \gamma_{|Q|^2 \cdot |\Sigma|}$. We are now ready to prove the following Lemma:

Lemma 6.21. For any $M \in \mathcal{M}$, there exist $M_0 \in \mathcal{M}_{init}$ and $M' \geq M$ such that $M_0 \rightarrow^* M'$ if and only if there exists $M'' \in \llbracket \gamma_f \rrbracket$ such that $M'' \geq M$.

Proof. Let $M \in \mathcal{M}$. We start by proving completeness: assume there exist $M_0 \in \mathcal{M}_{init}$ and $M' \geq M$ such that $M_0 \rightarrow^* M'$. Let k be the length of the run between M_0 and M' . As $M_{init} \in \llbracket \gamma_0 \rrbracket$ and thanks to Lemma 6.17, $M' \in \llbracket F^k(\gamma_0) \rrbracket$. From γ_f definition, either $F^k(\gamma_0) = \gamma_f$ or there exists $j > k$ such that $F^j(\gamma_0) = \gamma_f$. In the first case we immediately have that $M' \in \llbracket \gamma_f \rrbracket$. Otherwise, thanks to Lemma 6.16, we get that $\llbracket F^k(\gamma_0) \rrbracket \subseteq \llbracket F^j(\gamma_0) \rrbracket$, hence $M' \in \llbracket \gamma_f \rrbracket$.

We now prove soundness: assume there exists $M'' \in \llbracket \gamma_f \rrbracket$ such that $M'' \geq M$. Let k be the first integer such that $F^k(\gamma_0) = F^{k+1}(\gamma_0) = \gamma_f$. We prove by induction on $i \in [0, k]$ that there exist $M_k \geq M''$ and $M_{k-i} \in \llbracket F^{k-i}(\gamma_0) \rrbracket$ such that $M_{k-i} \rightarrow^* M_k$.

For $i = 0$, $M_k = M''$ satisfies the claim. Let now $i \in [0, k-1]$ and assume that there exist $M_k \geq M''$ and $M_{k-i} \in \llbracket F^{k-i}(\gamma_0) \rrbracket$ such that $M_{k-i} \rightarrow^* M_k$. As $i < k-1$, $F^{k-i-1}(\gamma_0)$ is defined (with the convention that $F^0(\gamma_0) = \gamma_0$). From lemma 6.20, there exists $M'_{k-i} \geq M_{k-i}$ and $M_{k-i-1} \in \llbracket F^{k-i-1}(\gamma_0) \rrbracket$ such that $M_{k-i-1} \rightarrow^* M'_{k-i}$. Thanks to Item 2 of Lemma 6.18 there exists M'_k such that

$$M_{k-i-1} \rightarrow^* M'_{k-i} \rightarrow^* M'_k \text{ with } M'_k \geq M_k \geq M''.$$

Hence, the claim holds for $i+1$ and so for all $i \in [0, k]$. As a consequence, there exists $M_0 \in \llbracket \gamma_0 \rrbracket = \mathcal{M}_{init}$ and $M_k \geq M'' \geq M$ such that $M_0 \rightarrow^* M_k$, concluding the proof. $\square$

To compute γ_f , we iterate F at most $|Q|^2 \cdot |\Sigma|$ times. Remember that we have assumed that $|Q| \leq |T|$, and $|\Sigma| \leq |T|$. Each application of F runs in polynomial time (Lemma 6.15). Furthermore, checking whether there exists $M'' \in \llbracket \gamma_f \rrbracket$ such that $M'' \geq M$ can be done in polynomial time by Lemma 6.14.

Since STATE-COVER^{nb} is easier than CONF-COVER^{nb} and by Theorem 6.5, we obtain the desired result.

Theorem 6.8. CONF-COVER^{nb} restricted to Wait-Only rendez-vous protocols is PTIME-complete.

6.6 CONCLUSIONS

In this chapter, we introduced the restriction of Wait-Only protocols. We showed that, under this restriction, coverability problems become significantly easier: from Ackermann-complete to PTIME-complete and PSPACE-complete for the state and configuration coverability problems, respectively. This stands in contrast to the synchronization problem, which remains undecidable, even for rendez-vous networks. In the next chapter, we will see that, in the case of broadcast networks, synchronization becomes decidable. This is made possible by a precise abstraction of broadcast networks with Wait-Only protocols into a Vector Addition System with States. The main results of this chapter are summarized in Table 6.3.

	STATE-COVER^{nb}	CONF-COVER^{nb}	SYNCHRO^{nb}
Rdv Networks (RDV-protocol)	EXPSPACE-complete (Chapter 5) Ackermann-complete [EK03; ARZ15; EFM99; SS13]		Undecidable (Chapter 4)
Broadcast Networks (B-protocol)			
Networks			
Wait-Only Rdv Networks (Wait-Only RDV-protocol)	PTIME-complete		Undecidable
Wait-Only Networks (Wait-Only protocols)			
	PTIME-complete	PSPACE-complete	

Table 6.3: Results presented in this chapter are written in red.

GOING BEYOND COVERABILITY IN BROADCAST WAIT-ONLY NETWORKS

7.1 INTRODUCTION

In this chapter, we investigate the behavior of the *Wait-Only* restriction in networks where only broadcasts are allowed (i.e., without transitions labeled by $!a$). We recall that a *Wait-Only broadcast protocol* is a protocol in which no state has both outgoing broadcast transitions and outgoing reception transitions.

For general broadcast protocols (without the *Wait-Only* restriction), throughout this thesis, we established the following:

- **State and configuration coverability** –the problems asking given a protocol and a state (respectively a configuration) if there is an execution putting at least one process on the state (respectively covering the configuration)–are Ackermann-complete (Chapter 2, Theorem 2.3, see [ARZ15; EFM99; SS13]).
- **Repeated coverability** –the problem asking given a protocol and a transition if there is an infinite execution in which the transition is fired infinitely often– is undecidable (Chapter 2, Theorem 2.4, see [EFM99]).
- **Synchronization** –the problem asking given a protocol and a state if there is an execution putting everyone in the state– is undecidable (Chapter 4, Theorem 4.1).

In Chapter 6, we studied coverability problems in networks allowing both broadcasts and rendez-vous communications with *Wait-Only protocols*: protocols in which no state has both outgoing action transitions (i.e. labeled with $!!a$ or $!a$) and outgoing reception transitions (i.e. labeled with $?b$). However, the lower bounds for state and configuration coverability were proven using broadcast protocols alone. For *Wait-Only* broadcast protocols, we recall:

- **State coverability** is PTIME-complete (Theorem 6.5).
- **Configuration coverability** is PSPACE-complete (Theorem 6.7).

CONTRIBUTIONS OF THIS CHAPTER We resolve the remaining problems: repeated coverability and synchronization. Our first result is that synchronization becomes decidable for *Wait-Only* broadcast networks, although Ackermann-hard. Interestingly, while rendez-vous protocols often simplify problem analysis (e.g., coverability is EXPSpace-complete for rendez-vous networks, as shown in Chapter 5), the *Wait-Only* restriction does not help for synchronization as it remains undecidable (Chapter 6 and theorem 6.1), whereas in

Wait-Only broadcast networks, as proved in this chapter, synchronization becomes decidable under the Wait-Only assumption.

Our proof relies on a carefully constructed VASS (Vector Addition System with States) that simulates Wait-Only broadcast networks. The VASS tracks processes in action states via a counting abstraction, forgetting identities and only maintaining processes' number per state. When a process broadcasts and leaves an action state, the VASS decrements the corresponding counter. However, this abstraction does not trivially extend to waiting states: upon receiving a message, *all* processes in a waiting state must transition, requiring a transfer of values, that a VASS cannot handle. We solve this by abstracting *zones* of waiting states: the VASS tracks the total number of processes in a zone and encodes which states are occupied in its location. Broadcasts update the occupied states while preserving the zone's total count.

Using this VASS construction, we establish that:

- **Synchronization** is decidable – and we prove an Ackermann lower bound.
- **Synchronization to an action state** is in ExpSPACE – and we prove an ExpSPACE lower bound.
- **Repeated coverability** is in ExpSPACE – and we prove a PSPACE lower bound.

In Section 7.2, we introduce some new notations and give an example of Wait-Only broadcast protocol. In Section 7.3, we give the definition of the VASS which would be the key ingredient of all the upper bounds presented in this chapter. In Section 7.4, we show that the synchronization problem is Ackermann-complete, and in Section 7.5, we show that if the state to synchronize on is a waiting state then the problem is ExpSPACE -complete. Finally, in Section 7.6, we show that the repeated coverability problem is in ExpSPACE and PSPACE -hard.

7.2 PRELIMINARIES

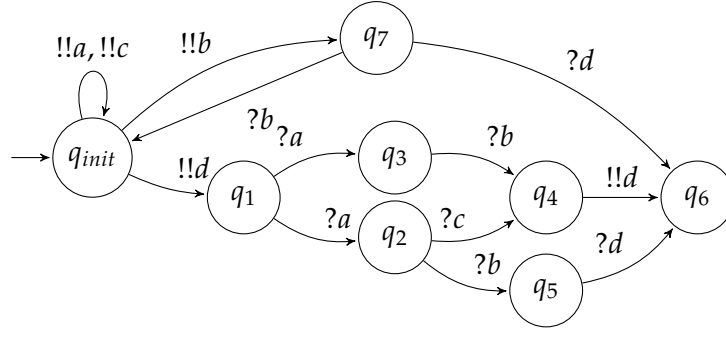
We will be working with vector configurations as it will be helpful to identify processes throughout the proofs of this chapter. We now introduce some useful notations, but first let us recall what is a Wait-Only protocol. Let $P = (Q, \Sigma, q_{\text{init}}, T)$ be a Wait-Only broadcast protocol, in other words, for all $q \in Q$,

$$\{(q, ?a, q') \in T \mid a \in \Sigma, q' \in Q\} = \emptyset \text{ or } \{(q, !!a, q') \in T \mid a \in \Sigma, q' \in Q\} = \emptyset.$$

If q satisfies the first condition it is called an action state, and otherwise, a waiting state. Recall that Q is then partitioned into the set of waiting states denoted by Q_W , and the set of action states denoted by Q_A . We always assume that $q_{\text{init}} \in Q_A$ as otherwise, nothing can happen in the network.

Example 7.1. Figure 7.1 illustrates an example of a Wait-Only protocol: no state allows both broadcasting and receiving messages simultaneously. The set of waiting states is $Q_W = \{q_1, q_2, q_3, q_5, q_7\}$, and the set of action states is $Q_A = \{q_{\text{init}}, q_4, q_6\}$.

Given a protocol $P = (Q, \Sigma, q_{\text{init}}, T)$, we recall that a configuration C is a vector in Q^n for an integer $n \in \mathbb{N}_{>0}$. For a configuration $C \in Q^n$ with $n \in \mathbb{N}_{>0}$, we use $C(A)$ to identify the set of states of processes in $A \subseteq [1, n]$, formally $C(A) = \{C(e) \mid e \in A\}$.

Figure 7.1: Example of a Wait-Only protocol P .

We recall that a finite [resp. infinite] **execution** of P is a finite [resp. infinite] sequence $\rho = C_0 C_1 \dots C_{k-1}$ [resp. $\rho = C_0 C_1 \dots$] such that for all $0 \leq i < k-1$ [resp. for all $i \in \mathbb{N}$], $C_i \rightarrow C_{i+1}$. Furthermore, its size is denoted by $\text{size}(\rho)$ and is equal to k [resp. ω]. In this chapter, given an execution $\rho = C_0 C_1 \dots$, and $0 \leq i < \text{size}(\rho)$, we use ρ_i to denote C_i , the i -th configuration. Additionally, $\#\text{proc}(\rho)$ represents $|C_0|$, the number of processes in the execution. For $0 \leq i < j < \text{size}(\rho)$, we define $\rho_{i,j}$ as the sequence $\rho_i \dots \rho_j$. We also let $\rho_{\geq i}$ be the sequence $\rho_i \rho_{i+1} \dots$ and $\rho_{\leq i}$ be $\rho_0 \dots \rho_i$. For $e \in [1, \#\text{proc}(\rho)]$, we denote by $\rho(e)$ the sequence of states $\rho_0(e) \rho_1(e) \dots$.

The set of finite [resp. infinite] executions of P is denoted $\text{Exec}_f[P]$ [resp. $\text{Exec}_\omega[P]$]. The set of all executions of P is denoted by $\text{Exec}[P]$.

Example 7.2. Here is an execution with 3 processes of the protocol of Figure 7.1:

$$\begin{aligned} \rho = (q_{\text{init}}, q_{\text{init}}, q_{\text{init}}) &\xrightarrow{1, (q_{\text{init}}, !!d, q_1)} (q_1, q_{\text{init}}, q_{\text{init}}) \xrightarrow{2, (q_{\text{init}}, !!d, q_1)} (q_1, q_1, q_{\text{init}}) \\ &\xrightarrow{3, (q_{\text{init}}, !!a, q_{\text{init}})} (q_2, q_3, q_{\text{init}}) \xrightarrow{3, (q_{\text{init}}, !!c, q_{\text{init}})} (q_4, q_3, q_{\text{init}}) \\ &\xrightarrow{3, (q_{\text{init}}, !!b, q_7)} (q_4, q_4, q_7). \end{aligned}$$

In particular: $\text{size}(\rho) = 6$ and $\#\text{proc}(\rho) = 3$. Furthermore, $\rho(1) = q_{\text{init}} \cdot q_1 \cdot q_1 \cdot q_2 \cdot q_4 \cdot q_4$.

We recall that:

- the **synchronization** problem (SYNCHRO) asks, given a protocol $P = (Q, \Sigma, q_{\text{init}}, T)$ and a state $q_f \in Q$, if there is an initial configuration C_{init} such that $C_{\text{init}} \rightarrow^* C_f$ and $C_f(e) = q_f$ for all $e \in [1, |C_{\text{init}}|]$.
- the **repeated coverability** problem (REP-COVER) asks, given a protocol $P = (Q, \Sigma, q_{\text{init}}, T)$ and a transition $t_f = (q, \alpha, q') \in T$, if there exist an infinite execution $C_0 C_1 \dots$ and a process $e \in [1, |C_0|]$ such that for all $N_1 \in \mathbb{N}$, there exists $N_2 \geq N_1$ with :
 - if $\alpha = !!a$ for an $a \in \Sigma$, then $C_{N_2} \xrightarrow{e, t_f} C_{N_2+1}$;
 - otherwise $\alpha = ?a$ for an $a \in \Sigma$, and $(C_{N_2}(e), ?a, C_{N_2+1}(e)) = t_f$ and $C_{N_2} \xrightarrow{e', (p, !!a, p')} C_{N_2+1}$ with $p, p' \in Q$ and $e' \in [1, |C_0|]$.

Observe that in broadcast protocols, we do not need to make the distinction between $\text{SYNCHRO}^{\text{nb}}$ (synchronisation when allowing broadcast steps, rendez-vous steps and sending

steps) and SYNCHRO^b (synchronisation when allowing only broadcast steps and rendez-vous steps) as the two semantics are the same when it comes to broadcast protocols. Hence, we simply write SYNCHRO and REP-COVER.

Remark 9. Throughout the remainder of this chapter, we will consider protocols without self-loop broadcast transitions of the form $(q, !!a, q)$. Such transitions can be transformed into two transitions $(q, !!a, p_q), (p_q, !!$, $q)$ where p_q is a new state added to the set of states and $$$ is a new message added to the alphabet. This transformation of the protocol is equivalent to the original one with respect to SYNCHRO and REP-COVER.$

7.3 A VASS SIMULATING WAIT-ONLY BROADCAST NETWORKS

In this section, we present the VASS which will be the keystone for all the upper bounds results presented in this chapter. We first give the main idea in Section 7.3.1, before presenting a preliminary result which will later allow us to bound the size of the VASS (Section 7.3.2). Then, in Section 7.3.3, we define *summaries* which will be the formal way of representing processes on zones of waiting states. We finally define the VASS in Section 7.3.4, and present a soundness local property (Section 7.3.5) and a completeness local property (Section 7.3.6).

7.3.1 Main idea

In this section, we show that we can build a VASS that simulates the behavior of a broadcast network running a Wait-Only broadcast protocol. An intuitive way to proceed, inspired by the counting abstraction proposed for protocols communicating by pairwise rendez-vous communication [GS92], consists in having one counter per state of the protocol, that stores the number of processes populating that state. Initially, the counter associated with the initial state can be incremented as much as one wants: this will determine the number of processes in the execution simulated by the VASS. Then, the simulation of broadcast messages $(q, !!a, q')$ amounts to decrement the counter associated to q and increment the counter associated to q' . However, simulating receptions of the message (e.g. a transition $(p, ?a, p')$), requires to empty the counter associated to p and transfer its value to the counter associated to p' . This transfer operation is not something that can be done in VASS unless the model is extended with special transfer transitions, leading to an undecidable reachability problem [Val78]. To circumvent this problem, we rely on two properties of Wait-Only protocols:

- (1) processes that occur to be in the same waiting state follow the same path of reception in the protocol, and end in the same action state (we show how to take care of potential non-determinism in the next subsection) and
- (2) processes in an action state cannot be influenced by the behavior of other processes as they cannot receive any message.

Thanks to property (1), instead of precisely tracking the number of processes in each waiting state, we only count the processes in a “waiting region” – a connected component of waiting states populated by processes that will all reach the same action state simultaneously. The waiting region allows us also to monitor when the processes go out from a waiting region to an action state. We will explain later how we use property (2) to handle the transfer

of values of counters within the VASS semantics, and thus simulating processes leaving a waiting region.

For the rest of this chapter, we fix $P = (Q, \Sigma, q_{init}, T)$ a Wait-Only protocol (with Q_W the set of waiting states and Q_A the set of action states) for which we assume w.l.o.g. the existence of an *uncoverable state* $q_u \in Q$ verifying $q \neq q_u$ and $q' \neq q_u$ for all $(q, \alpha, q') \in T$ and $q_u \neq q_{init}$.

7.3.2 Preliminary properties

We present here properties satisfied by Wait-Only protocols, which we will rely on in our construction. We first show with Lemma 7.1 that if, during an execution, two processes fulfill two conditions:

- (i) they are on the same waiting state q_w , and
- (ii) the next action state they reach (not necessarily at the same time) is the same (namely q_a),

then one can build an execution where they both follow the same path between q_w and q_a . This is trivial for *deterministic* protocols (where for all $q \in Q$ and $\alpha \in !!\Sigma \cup ?\Sigma^1$, there is at most one transition of the form $(q, \alpha, q') \in T$); in fact, it is also true for non-deterministic protocols.

For an execution $\rho \in \text{Exec}_f[P] \cup \text{Exec}_\omega[P]$ of P , an index $0 \leq j < \text{size}(\rho)$, a waiting state $q \in Q_W$ and a process number $e \in [1, \#\text{proc}(\rho)]$ such that $\rho_j(e) = q$, we define the index of its next action state to be:

$$\mathbf{na-index}(\rho, j, e) = \min_{j \leq k < \text{size}(\rho)} \{\rho_k(e) \in Q_A\},$$

i.e. the first moment after ρ_j where the process e reaches an action state (if such moment does not exist, it is set to $\text{size}(\rho)$). We define its next action state to be:

$$\mathbf{na-state}(\rho, j, e) = \rho_{\mathbf{na-index}(\rho, j, e)}(e)$$

i.e. the next action state for process e from ρ_j if $\mathbf{na-index}(\rho, j, e) \neq \text{size}(\rho)$ and otherwise we take the convention that $\mathbf{na-state}(\rho, j, e)$ is equal to q_u , the uncoverable state of the protocol P . Finally, we let its next action be:

$$\mathbf{na}(\rho, j, e) = (\mathbf{na-state}(\rho, j, e), \mathbf{na-index}(\rho, j, e)).$$

We are now ready to prove the technical lemma allowing us to resolve the non-determinism of our protocol.

¹ We remind the reader that $!!\Sigma = \{!!a \mid a \in \Sigma\}$ and $?\Sigma = \{?a \mid a \in \Sigma\}$.

Lemma 7.1. Let $\rho \in \text{Exec}[P]$ be an execution of P . If there exist $e_1, e_2 \in [1, \#\text{proc}(\rho)]$ and $0 \leq j < \text{size}(\rho)$ such that:

- (i) $\rho_j(e_1) = \rho_j(e_2) \in Q_W$, and
- (ii) $\mathbf{na}(\rho, j, e_1) = (q, j_1)$, and $\mathbf{na}(\rho, j, e_2) = (q, j_2)$ with $j_1 \leq j_2 < \text{size}(\rho)$

then there exists an execution ρ' of the form

$$\rho_{\leq j} \cdot \rho'_{j+1} \cdots \rho'_{j_2} \cdot \rho_{\geq j_2+1}$$

such that:

- $\rho'_k(e_1) = \rho'_k(e_2) = \rho_k(e_1)$ for all $j+1 \leq k \leq j_1$ and,
- $\rho'_k(e_2) = q$ and $\rho'_k(e) = \rho_k(e)$ for all $e \neq e_2$ and $j_1 < k \leq j_2$.

In particular $\mathbf{na}(\rho', j, e_1) = \mathbf{na}(\rho', j, e_2) = (q, j_1)$.

Proof. Let $\rho \in \text{Exec}[P]$ be an execution of P such that there exist $e_1, e_2 \in [1, \#\text{proc}(\rho)]$ and $0 \leq j_0 < \text{size}(\rho)$ with $\rho_{j_0}(e_1) = \rho_{j_0}(e_2) \in Q_W$ and $\mathbf{na}(\rho, j_0, e_1) = (q, j_1)$ and $\mathbf{na}(\rho, j_0, e_2) = (q, j_2)$ with $j_1 \leq j_2 < \text{size}(\rho)$.

We define the sequence of configurations $\rho' = \rho'_{j_0+1} \cdots \rho'_{j_2}$ as follows:

- for all $j_0 < k \leq j_1$, $\rho'_k(e_2) = \rho_k(e_1)$ (and $\rho'_k(e) = \rho_k(e)$ for all other process e);
- for all $j_1 < k \leq j_2$, $\rho'_k(e_2) = q$ (and $\rho'_k(e) = \rho_k(e)$ for all other process e).

Observe that $\rho_{j_2}(e_2) = q$ as $\mathbf{na}(\rho, j_0, e_2) = (q, j_2)$. Hence, $\rho'_{j_2} = \rho_{j_2}$. Hence, it remains to prove that $\rho_{j_0} \rightarrow \rho'_{j_0+1} \rightarrow \cdots \rightarrow \rho'_{j_2}$ in order to prove that $\rho_{\leq j_0} \cdot \rho' \cdot \rho_{\geq j_2+1}$ is an execution.

We do so by induction: we let $\rho'_{j_0} = \rho_{j_0}$ and we prove by induction on $j_0 \leq k \leq j_2$ that $\rho'_{j_0} \rightarrow \rho'_{j_0+1} \rightarrow \cdots \rightarrow \rho'_k$.

For $k = j_0$, there is nothing to do. Let $j_0 \leq k < j_2$ and assume that

$$\rho'_{j_0} \rightarrow \rho'_{j_0+1} \rightarrow \cdots \rightarrow \rho'_k.$$

Let us prove that $\rho'_k \rightarrow \rho'_{k+1}$. To do so, let $\rho_k \xrightarrow{e_0, t} \rho_{k+1}$ with $t = (q, !!a, q')$. We distinguish between three cases:

- Case $k < j_1$. First observe that $k < j_1 = \mathbf{na-index}(\rho, j_0, e_1) \leq \mathbf{na-index}(\rho, j_0, e_2) = j_2$, hence $\rho_k(e_1) \in Q_W$, $\rho_k(e_2) \in Q_W$ and so $e_0 \neq e_1$ and $e_0 \neq e_2$.

Observe that we are in the case where $\rho'_k(e_2) = \rho_k(e_1)$ and $\rho'_{k+1}(e_2) = \rho_{k+1}(e_1)$. As $\rho_k \rightarrow \rho_{k+1}$, either $(\rho'_k(e_2), ?a, \rho'_{k+1}(e_2)) = (\rho_k(e_1), ?a, \rho_{k+1}(e_1)) \in T$ or $a \notin R(\rho_k(e_1)) = R(\rho'_k(e_2))^2$ and $\rho'_{k+1}(e_2) = \rho_{k+1}(e_1) = \rho_k(e_1) = \rho'_k(e_2)$. Hence, as for all other process e , $\rho'_k(e) = \rho_k(e)$ and $\rho'_{k+1}(e) = \rho_{k+1}(e)$, it holds that $\rho'_k \xrightarrow{e_0, t} \rho'_{k+1}$.

- Case $k > j_1$. First observe that $k < j_2 = \mathbf{na-index}(\rho, j_0, e_2)$, hence $\rho_k(e_2) \in Q_W$, and so $e_0 \neq e_2$. Observe that we are in the case where $\rho'_k(e_2) = \rho'_{k+1}(e_2) = q$.

² We recall that $R(q) = \{a \in \Sigma \mid (q, ?a, q'), q' \in Q\}$.

Recall that $q \in Q_A$. Hence, $a \notin R(q) = \emptyset$. As $\rho'_k(e) = \rho_k(e)$ and $\rho'_{k+1}(e) = \rho_{k+1}(e)$ for all other process e , it holds that $\rho'_k \xrightarrow{e,t} \rho'_{k+1}$. As in the previous case, we conclude that $\rho'_k \xrightarrow{e_0,t} \rho'_{k+1}$.

- Case $k = j_1$. In that case $\rho'_{j_1}(e_2) = \rho_{j_1}(e_1) = q$ and $\rho'_{j_1+1}(e_2) = q$. As $q \in Q_A$, $a \notin R(q) = \emptyset$ and so, as $\rho'_k(e) = \rho_k(e)$ and $\rho'_{k+1}(e) = \rho_{k+1}(e)$ for all other processes $e \neq e_2$, it holds that $\rho'_k \xrightarrow{e_0,t} \rho'_{k+1}$.

□

Example 7.3. Consider the protocol P in Figure 7.1 and the execution ρ of Example 7.2. We have $\rho_2(1) = \rho_2(2) = q_1$ and $\mathbf{na}(\rho, 2, 1) = (q_4, 4)$ and $\mathbf{na}(\rho, 2, 2) = (q_4, 5)$. Applying Lemma 7.1, we can build:

$$\begin{aligned} \rho' &= (q_{\text{init}}, q_{\text{init}}, q_{\text{init}}) \rightarrow (q_1, q_{\text{init}}, q_{\text{init}}) \rightarrow (q_1, q_1, q_{\text{init}}) \\ &\rightarrow (q_2, q_2, q_{\text{init}}) \rightarrow (q_4, q_4, q_{\text{init}}) \rightarrow (q_4, q_4, q_7) \end{aligned}$$

where process 1 and process 2 follow the same path between q_1 and q_4 . However, if you consider the following execution:

$$\begin{aligned} \rho'' &= (q_{\text{init}}, q_{\text{init}}, q_{\text{init}}) \rightarrow (q_1, q_{\text{init}}, q_{\text{init}}) \rightarrow (q_1, q_1, q_{\text{init}}) \\ &\rightarrow (q_2, q_3, q_{\text{init}}) \rightarrow (q_5, q_4, q_7), \end{aligned}$$

we cannot apply the Lemma as from q_1 , processes 1 and 2 reach two different action states, namely q_4 and q_6 .

The above lemma enables us to focus on a specific subset of executions which we refer to as well-formed. In these executions, at any moment, two processes in the same waiting state and with the same next action state, follow the same path of receptions.

Definition 7.1 (Well-formed execution). An execution ρ of a protocol P is *well-formed* if for all $0 \leq i < \text{size}(\rho)$, for all $e_1, e_2 \in [1, \#\text{proc}(\rho)]$ such that

- $\rho_i(e_1) = \rho_i(e_2) \in Q_W$ and
- $\mathbf{na}\text{-state}(\rho, i, e_1) = \mathbf{na}\text{-state}(\rho, i, e_2) = q$,

it holds that $\rho_k(e_1) = \rho_k(e_2)$ for all $i \leq k \leq \mathbf{na}\text{-index}(\rho, i, e_1)$.

We finally provide a result which will allow us to bound the size of the VASS that we will build from the given Wait-Only protocol. Given an execution $\rho \in \text{Exec}[P]$ of P , an index $0 \leq j < \text{size}(\rho)$ and an action state $q_a \in Q_A$, we define $\mathbf{na}\text{-index}\text{-set}(\rho, j, q_a)$ as the set of indices where processes in a waiting state at ρ_j will reach q_a , if it is their next action state:

$$\begin{aligned} \mathbf{na}\text{-index}\text{-set}(\rho, j, q_a) &= \{i \mid \text{there exists } e \in [1, \#\text{proc}(\rho)] \\ &\text{s.t. } \rho_j(e) \in Q_W \text{ and } \mathbf{na}(\rho, j, e) = (q_a, i)\} \end{aligned}$$

Lemma 7.2. For all well-formed execution ρ , for all $0 \leq j < \text{size}(\rho)$, and for all $q_a \in Q_A$, we have $|\mathbf{na}\text{-index}\text{-set}(\rho, j, q_a)| \leq |Q_W|$.

Proof. If we have $|\mathbf{na-index-set}(\rho, j, q_a)| > |Q_W|$, by pigeon hole principle there exist $e_1, e_2 \in [1, \#\mathbf{proc}(\rho)]$ such that $\rho_j(e_1) = \rho_j(e_2) \in Q_W$ and such that $\mathbf{na}(\rho, j, e_1) = (q_a, i_1)$ and $\mathbf{na}(\rho, j, e_2) = (q_a, i_2)$ with $i_1 < i_2$. Consequently $\rho_{i_1}(e_1) = q_a \in Q_A$ and $\rho_{i_1}(e_2) \in Q_W$ hence $\rho_{i_1}(e_1) \neq \rho_{i_1}(e_2)$, which contradicts the definition of well-formedness. $\square$

The VASS we build faithfully simulates *well-formed* executions. We will see why this is sufficient to solve SYNCHRO. When it comes to REP-COVER, we would need a slightly bigger set of executions, and we will see why, and how to adapt the VASS in order to handle those.

7.3.3 Summaries

We present here the formal tool we use to represent processes in waiting states. We begin by introducing some notations.

A **print** $\mathbf{pr}$ is a set of waiting states. Given a non empty subset of states $A = \{q_1, \dots, q_n\} \subseteq Q$, we define a configuration $\dot{A} \in Q^n$ such that $\dot{A}(e) = q_e$ for all $e \in [1, n]$. When $A = \{q\}$, we write $\dot{q} \in Q^1$ instead of $\{\dot{q}\}$. Conversely, given a configuration C , we define $\mathbf{set}(C) = \{q \in Q \mid C^{-1}(q) \neq \emptyset\}$. For two configurations $C \in Q^n$ and $C' \in Q^m$, we let $C \oplus C'$ be the configuration $C'' \in Q^{n+m}$ defined by: $C''(e) = C(e)$ if $1 \leq e \leq n$ and $C''(e) = C'(e - n)$ if $n + 1 \leq e \leq n + m$.

A **summary** S is a tuple $(\mathbf{pr}, q_a, k)$ such that $\mathbf{pr} \subseteq Q_W$ is a print, $q_a \in Q_A$ is an *exit state*, and $k \in [1, |Q_W| + 1]$ is an identifier. The label of S is $\mathbf{lab}(S) = (q_a, k)$ and we denote its print by $\mathbf{print}(S)$. Finally, we define a *special* summary **Done**.

In our construction, each process in a waiting state is associated with a summary while processes in action states are handled by the counters of the VASS. Intuitively, a summary $(\mathbf{pr}, q_a, k)$ represents a set of processes lying in the set $\mathbf{pr} \subseteq Q_W$ that will reach the same next action state q_a simultaneously (this restriction is justified by Lemma 7.1). Since the set of waiting states $\mathbf{pr}$ evolve during the simulation, each summary must be uniquely identified. To achieve this, we use an integer $k \in [1, |Q_W| + 1]$. Hence each summary is identified with the pair (q_a, k) . We do not need more than $|Q_W| + 1$ different identifiers, because when a process enters a new summary (i.e it arrives in a waiting state q_w from an action state), aiming for next action state q_a , it either joins an existing summary with exit state q_a , or create a new one. In the latter case, well-formed executions ensure that no existing summary $S = (\mathbf{pr}, q_a, k)$ with exit state q_a is such that $q_w \in \mathbf{pr}$. Otherwise two processes would be in the same state q_w , aiming for the same action state, but at different moments, contradicting Lemma 7.1. Note that we need $|Q_W| + 1$ different identifiers, and not $|Q_W|$ for technical reasons. Finally, the special summary **Done** is used to indicate when the processes described by a summary reach the exit action state.

We now describe how summaries evolve with the sequence of transitions. Let $S = (\mathbf{pr}, q_a, k)$ and $S' = (\mathbf{pr}', q_a, k)$ be two summaries (with the same exit state and identifier) and $t = (q, !!b, q') \in T$ such that there exists a configuration C' verifying $\dot{q} \oplus \mathbf{pr} \xrightarrow{1t} \dot{q}' \oplus C'$.

- We then write $S \xRightarrow{t} S'$ whenever $\mathbf{set}(C') = \mathbf{pr}' \subseteq Q_W$. This represents the evolution of a summary upon reception of message b , when the processes all stay in waiting states, and no new process join the “waiting region” of the given summary.

- We write $S \xrightarrow{t+q'} S'$ whenever $q' \in Q_W$ and $\mathbf{set}(C') \cup \{q'\} = \mathbf{pr}' \subseteq Q_W$. In that latter case, the process responsible for the transition t joins the “waiting region” represented by S .
- Finally, we use $S \xrightarrow{t} \text{Done}$ whenever $\mathbf{set}(C') = \{q_a\}$. This represents the evolution (and disappearance) of the summary when all the processes reach q_a (they all reach it simultaneously).

Observe that the two first cases are not mutually exclusives: it is possible to have both $S \xrightarrow{t} S'$ and $S \xrightarrow{t+q'} S'$ (when $q' \in \mathbf{set}(C')$). We will see later how to ensure in the VASS that the process responsible for t joins exactly *one identified* waiting region.

Example 7.4. Returning to the protocol P of Figure 7.1, consider the summary $S_0 = (\{q_1\}, q_4, 1)$. Observe that $S_0 \xrightarrow{(q_{\text{init}}, !!a, q_{\text{init}})} (\{q_3\}, q_4, 1)$ and $S_0 \xrightarrow{(q_{\text{init}}, !!a, q_{\text{init}})} (\{q_2\}, q_4, 1)$. However, by definition, we do not have $S_0 \xrightarrow{(q_{\text{init}}, !!a, q_{\text{init}})} (\{q_2, q_3\}, q_4, 1)$. Indeed, processes summarized in S_0 are forced to all go either on q_3 or on q_2 upon receiving a : we consider only well-formed executions, where all processes summarized in S_0 choose the same next state. Additionally, we have $(\{q_3\}, q_4, 1) \xrightarrow{(q_{\text{init}}, !!b, q_7)} \text{Done}$ and

$$(\{q_1\}, q_6, 1) \xrightarrow{(q_{\text{init}}, !!a, q_{\text{init}})} (\{q_2\}, q_6, 1) \xrightarrow{(q_{\text{init}}, !!b, q_7), +q_7} (\{q_5, q_7\}, q_6, 1) \xrightarrow{(q_{\text{init}}, !!d, q_1)} \text{Done}.$$

However, note that we do not have $(\{q_3\}, q_6, 1) \xrightarrow{(q_{\text{init}}, !!b, q_7)} \text{Done}$, since the action state q_4 reached from q_3 upon receiving b is not the exit state q_6 .

7.3.4 Definition of the VASS

We now explain how we use the summaries in the VASS simulating the executions of the network.

First we say that a set $\mathcal{S}$ of summaries is **coherent** if for all distinct pairs $(\mathbf{pr}_1, q_1, k_1), (\mathbf{pr}_2, q_2, k_2) \in \mathcal{S}$ such that $q_1 = q_2$, we have $k_1 \neq k_2$ and $\mathbf{pr}_1 \cap \mathbf{pr}_2 = \emptyset$. We denote by **CoSets** the set of coherent sets of summaries. For a set of summaries $\mathcal{S}$ we let $\mathbf{lab}(\mathcal{S}) = \{\mathbf{lab}(S) \mid S \in \mathcal{S}\}$. Observe that, when $\mathcal{S}$ is **coherent**, for a label $(q, k) \in \mathbf{lab}(\mathcal{S})$, there is a unique $S \in \mathcal{S}$ such that $\mathbf{lab}(S) = (q, k)$.

We define the VASS $\mathbf{V}_P$ simulating the protocol P as follows: $\mathbf{V}_P = (\text{Loc}, s_0, X, \Delta)$, where $\text{Loc} = \text{CoSets} \cup \{s_0\}$, the set of counters is

$$X = \{\mathbf{x}_q \mid q \in Q_A\} \cup \{\mathbf{x}_{(q,k)} \mid q \in Q_A, k \in [1, |Q_W| + 1]\},$$

and the set of transitions $\Delta = \Delta_0 \cup (\bigcup_{t \in T} \Delta_t) \cup \Delta_e$, is defined as follows:

- Δ_0 contains exactly the following transitions:
 - (s_0, δ_0, s_0) where $\delta_0(\mathbf{x}_{q_{\text{init}}}) = 1$ and $\delta_0(\mathbf{x}) = 0$ for all other counters;
 - $(s_0, \delta_0, \emptyset)$. Observe that the empty set is a coherent set of summaries;
- For $t = (q, !!a, q')$ in T , we have $(\mathcal{S}, \delta, \mathcal{S}') \in \Delta_t$ iff one of the following conditions holds:

- a. $q' \in Q_A$ and $\mathcal{S} = \{S_1, \dots, S_k\}$. Then, for all $1 \leq i \leq k$, there exists S'_i such that $S_i \xrightarrow{t} S'_i$ and $\mathcal{S}' = \{S'_j \mid 1 \leq j \leq k\} \setminus \{\text{Done}\}$. Moreover, $\delta(\mathbf{x}_q) = -1$, $\delta(\mathbf{x}_{q'}) = 1$ and $\delta(\mathbf{x}) = 0$ for all $\mathbf{x} \in X \setminus \{\mathbf{x}_q, \mathbf{x}_{q'}\}$.

Here we simply update the sets of states populated by processes in waiting states after the transition t . Some summaries may have disappeared from $\mathcal{S}$ if the processes represented by this summary have reached their exit action state when receiving a . For now, we only modify the counters associated to the states q and q' . Note that this is well-defined, since we assume in Remark 9 that there is no self-loop of the form $(q, !!a, q)$.

- b. $q' \in Q_W$, $\mathcal{S} = \{S_1, \dots, S_k\}$, and there exists $1 \leq i \leq k$ and S'_i such that $S_i \xrightarrow{t, +q'} S'_i$. For all $j \neq i$, there exists S'_j such that $S_j \xrightarrow{t} S'_j$. Then $\mathcal{S}' = \{S'_i \mid 1 \leq i \leq k\} \setminus \{\text{Done}\}$. Moreover, $\delta(\mathbf{x}_q) = -1$, $\delta(\mathbf{x}_{\text{lab}(S_i)}) = 1$ and $\delta(\mathbf{x}) = 0$ for all $\mathbf{x} \in X \setminus \{\mathbf{x}_q, \mathbf{x}_{\text{lab}(S_i)}\}$.

In that case, the process performing the transition joins an existing summary S_i . We modify accordingly the counters associated to q and to the appropriate summary.

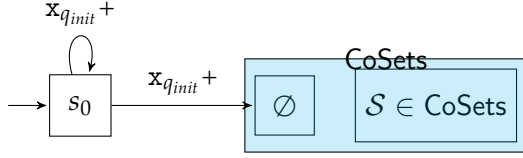
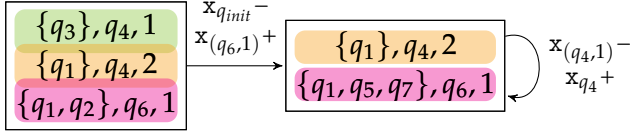
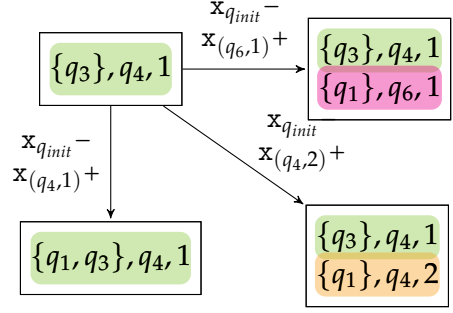
- c. $q' \in Q_W$ and $\mathcal{S} = \{S_1, \dots, S_k\}$. For all $1 \leq j \leq k$, there exists S'_j such that $S_j \xrightarrow{t} S'_j$. Then $\mathcal{S}' = (\{S'_1, \dots, S'_k\} \setminus \{\text{Done}\}) \cup \{(\{q'\}, q_a, r)\}$ for some $q_a \in Q_A$ and $r \in [1, |Q_W| + 1]$ such that $(q_a, r) \notin \text{lab}(\mathcal{S})$. Moreover, $\delta(\mathbf{x}_q) = -1$, $\delta(\mathbf{x}_{(q_a, r)}) = 1$, and $\delta(\mathbf{x}) = 0$ for all $\mathbf{x} \in X \setminus \{\mathbf{x}_q, \mathbf{x}_{(q_a, r)}\}$.

This case happens when the process sending the message a reaches a waiting state, but its expected instant of joining its next action state and its expected next action state do not correspond to any existing summary. In that case, it joins a new summary, the counter associated to the state q is decremented and the counter of this new summary is incremented.

- Finally, Δ_e allows to empty the counters associated to summaries that have disappeared from the set of locations. It is defined as the set of transitions $(\mathcal{S}, \delta, \mathcal{S}')$ such that: $\mathcal{S} = \mathcal{S}'$ and there exists $(q, k) \notin \text{lab}(\mathcal{S})$ with $\delta(\mathbf{x}_q) = +1$ and $\delta(\mathbf{x}_{(q, k)}) = -1$ and for all other counters, $\delta(\mathbf{x}) = 0$.

Transitions from Δ_e allow the transfer of counter values from summaries that have disappeared to the counter of their exit action state. This transfer is not immediate, as it is done through iterative decrements and increments of counters. In particular, it is possible for the execution of the VASS to continue without the counter of a disappeared summary being fully transferred. The processes represented by the counter of a disappeared summary behave in the same way as processes in the exit action state that do not take any action. These counters can be emptied later, but always from a location from where no summary with the same label exists. This ensures that there is no ambiguity between processes in a waiting region and those on an action state that have not yet been transferred to the appropriate counter.

For two configurations of the VASS $\lambda = (\ell, v)$ and $\lambda' = (\ell', v')$, we abuse notation and write $\lambda \xrightarrow{\delta} \lambda'$ for $\lambda \xrightarrow{tr} \lambda'$ and $tr = (\ell, \delta, \ell') \in \Delta$.

Figure 7.2: The structure of the VASS V_P .Figure 7.3: One successor of location $((\{q_3\}, q_4, 1), (\{q_1\}, q_4, 2), (\{q_1, q_2\}, q_6, 1))$ (left location) after the broadcast transition $(q_{init}, !!b, q_7)$ Figure 7.4: Three successors of location $((\{q_3\}, q_4, 1))$ (top left location) after the broadcast transition $(q_{init}, !!d, q_1)$.

Example 7.5. Figures 7.2 to 7.4 illustrate the VASS V_P defined for protocol P in Figure 7.1. Figure 7.2 shows the overall structure of the VASS: any execution starts by incrementing the counter associated with the initial state.

In Figure 7.4, we detail how message receptions and summary creations are handled. The top-left location has a single summary, $(\{q_3\}, q_4, 1)$, representing configurations where processes in waiting states are all in q_3 , progressing together to q_4 . The three successors depict behaviors after the broadcast $(q_{init}, !!d, q_1)$. In the first scenario (right location), the broadcasting process joins a *new* summary (pink) $(\{q_1\}, q_6, 1)$. In the second scenario (diagonal location), it joins another *new* summary (orange) $(\{q_1\}, q_4, 2)$, indicating a different timing for reaching q_4 . In the third scenario (below location), the process joins an existing summary, and q_1 is added to the print. Note that well-formed executions limit the number of summaries with the same exit state since there are at most $|Q_W|$ different moments where processes can reach a same action state (Lemma 7.2).

Observe that, if a wrong choice is made in the VASS execution, the system reaches a deadlock. For instance, consider the location on the left below, with a single summary $(q_1, q_3, q_4, 1)$. If one tries to simulate a broadcast of message b , this is not possible: it would require the process in q_3 to move to q_4 , while the one in q_1 would remain unchanged. Since they do not reach q_4 together, they should not belong to the same summary. Accordingly, in the VASS, there is no transition from $(q_1, q_3, q_4, 1)$ simulating such a broadcast, as no summary S (not even $S = \text{Done}$) satisfies $(q_1, q_3, q_4, 1) \xrightarrow{t} S$. Sometimes, bad behaviour are prevented because they do not represent coherent sets of summaries. For instance, from the right-hand location, with two summaries: $(q_3, q_4, 1)$ and $(q_1, q_4, 2)$, when simulating the transition $(q_{init}, !!a, q_{init})$, the process in q_1 must move to q_2 . Otherwise, the resulting location would contain $(q_3, q_4, 1)$ and $(q_3, q_4, 2)$: a non-coherent set of summaries, which is not a VASS location.

Figure 7.3 illustrates summary deletion. The transition $(q_{init}, !!b, q_7)$ causes the broadcasting process to join the pink summary (the bottom one), incrementing its counter. Processes in the green summary $(\{q_3\}, q_4, 1)$ receive the message via $(q_3, ?b, q_4)$, reaching their exit state q_4 . The green summary is deleted in the next location. By taking the loop transition at the right location, the value of $x_{(q_4,1)}$ is transferred to the counter x_{q_4} . If the loop is not taken enough times, $x_{(q_4,1)}$ may remain non-zero, but this does not affect correctness, as the

counter will be decremented eventually, from a state in which there is no summary labeled with $(q_4, 1)$, and not decrementing soon enough only delay the moment the corresponding processes will move from the action state q_4 .

Remark 10. In the sequel of this paper, the size of V_P will be of interest to us (Sections 4.1 and 5.1). Hence, observe that $|\text{Loc}| = |\text{CoSets}| + 1 \leq 2^{|Q_A| \times (|Q_W| + 1) \times 2^{|Q_W|}} + 1$ (as one summary is composed of a state in Q_A , one label in $[1, |Q_W| + 1]$ and one set of waiting states), and $|X| = |Q_A| + |Q_A| \times (|Q_W| + 1)$. As $q_{init} \in Q_A$, we have that $|Q_W| + 1 \leq |Q|$, and so $|Q_A| \times |Q_W| + 1 \leq |Q|^2$, which we will extensively use in this chapter. As each transition (ℓ, δ, ℓ') is such that δ has at most 2 non-null components, and non-null components are in $\{-1, 1\}$, we get that $|\Delta| \leq 2 + |\text{CoSets}|^2 \times 2|X|$ (the first two transitions are $(s_0, \mathbf{0}_X, s_0)$ and $(s_0, \delta_0, \emptyset)$). Hence,

$$\begin{aligned} |V_P| &= |\Delta|(2|\text{Loc}| + |X| \times \|\Delta\|) \\ &= |\text{Loc}|^2 2|X|(2|\text{Loc}| + |X|) \\ &\leq |\text{CoSets}|^2 2(|Q| + 1)^2 (2|\text{CoSets}| + (|Q| + 1)^2) \\ &\leq 2^{2|Q|^2 \times 2^{|Q|} + 1} (|Q| + 1)^4 (2^{|Q|^2 \times 2^{|Q|}} + 1) \\ &\leq 2^{3|Q|^2 \times 2^{|Q|} + 2} (|Q| + 1)^4. \end{aligned}$$

In this chapter, we consider that all states of the protocol except q_u appear in at least one transition, hence we get $|Q| \leq |T| + 1$ allowing us to state that the VASS as a size doubly exponential in the size of the protocol, defined as $|T|(2 \log_2(|Q|) + \log_2(|\Sigma|))$.

7.3.5 Soundness of the construction

We show now how we go from an execution of the VASS V_P to an execution of P . To do that, we start by giving the set of configurations of P associated with a configuration of the VASS.

We say that a configuration (ℓ, v) of V_P is an **S-configuration** if $\ell = \mathcal{S}$ for some $\mathcal{S} \in \text{CoSets}$. The **implementation** of an S-configuration $\lambda = (\mathcal{S}, v)$ is the set of network configurations $\llbracket \lambda \rrbracket \subseteq \mathcal{C}$ defined as follows: $C \in \llbracket \lambda \rrbracket$ if and only if there exists a function $f : [1, |C|] \rightarrow X$ such that $|f^{-1}(x)| = v(x)$ for all $x \in X$, and

CondImpl1 for all $x_q \in X$ with $q \in Q_A$, we have $C(e) = q$ for all $e \in f^{-1}(x_q)$;

CondImpl2 for all $x_{(q,k)} \in X$, if there exists $(pr, q, k) \in \mathcal{S}$ for some $pr \subseteq Q_W$, then $C(e) \in pr \cup \{q\}$ for all $e \in f^{-1}(x_{q,k})$;

CondImpl3 for all $x_{(q,k)} \in X$, if $(q, k) \notin \text{lab}(\mathcal{S})$, then $C(e) = q$ for all $e \in f^{-1}(x_{q,k})$.

In an implementation of λ , the processes populate states according to the values of the counters. All processes associated with a counter x_q of an action state will populate this exact state (**CondImpl1**). However, processes associated with a counter $x_{(q,k)}$ of a summary do not necessarily populate waiting states. This occurs when the label (q, k) does not appear in $\mathcal{S}$ while the associated counter remains strictly positive. Such a situation arises when a summary has been previously deleted, but its counter has not been emptied. Since all associated processes have already reached the exit action state, the processes associated to the corresponding counter have to populate this active state (**CondImpl3**). If the summary with label (q, k) is active (i.e. its label appears in $\mathcal{S}$), the processes associated with $x_{(q,k)}$ will

be in the corresponding waiting region, or in the exit action state (**CondImpl2**). This may seem contradictory, as all processes are supposed to reach their exit state simultaneously. However, the processes already on the exit action state are “old” processes, that remained after a previous deletion of this summary (cf. **CondImpl3**). These processes will be allowed to send messages only when they are identified with the counter of the action state, meaning when the corresponding transition in Δ_e will be taken.

The next lemma will allow us to build an execution of the protocol P from an execution of the VASS V_P .

Lemma 7.3. Let λ, λ' be two S-configurations of V_P and $C \in \llbracket \lambda \rrbracket$. If $\lambda \xrightarrow{\delta} \lambda'$ in V_P then there exists $C' \in \llbracket \lambda' \rrbracket$ such that $C \rightarrow^* C'$.

Proof. We begin by giving an overview of the proof. Assume $\lambda = (\mathcal{S}, v)$ and $\lambda' = (\mathcal{S}', v')$. Then either $(\mathcal{S}, \delta, \mathcal{S}') \in \Delta_e$ or $(\mathcal{S}, \delta, \mathcal{S}') \in \Delta_t$ for some $t \in T$. In the first case, one can show that if $C \in \llbracket \lambda \rrbracket$, then $C \in \llbracket \lambda' \rrbracket$. Otherwise, let $t = (q, !!a, q')$ and there is a process e such that $C(e) = q$. The transition δ in the VASS makes all the summaries and counters evolve according to the reception of the message a . According to the different cases a., b. or c., one can build a configuration $C' \in \llbracket \lambda' \rrbracket$ such that $C \rightarrow C'$.

We now present the technical proof of the Lemma.

We first explain how we build C' s.t. $C \rightarrow C'$ and then proceed to show that $C' \in \llbracket \lambda' \rrbracket$.

Let $\lambda \xrightarrow{\delta} \lambda'$ with $\lambda = (\mathcal{S}, v)$ and $\lambda' = (\mathcal{S}', v')$. Let $C \in \llbracket \lambda \rrbracket$ with $|C| = n$ and $f : [1, n] \rightarrow X$ realizing **CondImpl1**, **CondImpl2** and **CondImpl3** w.r.t. C and γ .

- If $(\mathcal{S}, \delta, \mathcal{S}') \in \Delta_e$, then $\mathcal{S} = \mathcal{S}'$ and let $(p, k) \notin \mathbf{lab}(\mathcal{S})$ such that $\delta(x_p) = 1$ and $\delta(x_{p,k}) = -1$. In that case, we let $C' = C$ and $C \rightarrow^* C'$.

It remains to show that $C = C' \in \llbracket \lambda' \rrbracket$. For that, we will build another function $f' : [1, n] \rightarrow X$, that reflects the modification of counters $x_{p,k}$ and x_p . Since $\delta(x_{p,k}) = -1$, $v(x_{p,k}) > 0$ and hence $|f^{-1}(x_{p,k})| > 0$. Thus, there exists a process $e \in f^{-1}(x_{p,k})$. Since $(p, k) \notin \mathbf{lab}(\mathcal{S})$, by **CondImpl3**, the process reached state p already: $C(e) = C'(e) = p$. Then, we let $f'(e) = x_p$, and for all other processes $e' \in [1, n]$, we let $f'(e) = f(e')$. Then,

$$|f'^{-1}(x_p)| = |f^{-1}(x_p)| + 1, \text{ and } |f'^{-1}(x_{p,k})| = |f^{-1}(x_{p,k})| - 1,$$

and for all other counter x , $|f'^{-1}(x)| = |f^{-1}(x)|$. Then,

$$v'(x_p) = v(x_p) + 1 = |f^{-1}(x_p)| + 1 = |f'^{-1}(x_p)|,$$

$$v'(x_{p,k}) = v(x_{p,k}) - 1 = |f^{-1}(x_{p,k})| - 1 = |f'^{-1}(x_{p,k})|,$$

and for all other counter $v'(x) = v(x) = |f^{-1}(x)|$. Since $C'(e) = p$, it is easy to see that f' meets the requirements **CondImpl1**, **CondImpl2** and **CondImpl3**, hence $C' \in \llbracket \lambda' \rrbracket$.

- Otherwise, $(\mathcal{S}, \delta, \mathcal{S}') \notin \Delta_e$ and let $t = (q, !!a, q')$ such that $(\mathcal{S}, \delta, \mathcal{S}') \in \Delta_t$. Since $\delta(x_q) = -1$, $v(x_q) > 0$ and hence there exists $e_q \in [1, n]$ such that $f(e_q) = x_q$, and, by **CondImpl1**, $C(e_q) = q$. We build C' as follows:

- $C'(e_q) = q'$,
- for all $e \in [1, n]$, such that $a \notin R(C(e))^3$, we let $C'(e) = C(e)$,
- for all $e \in [1, n]$ such that $a \in R(C(e))$, we need to distinguish cases. Observe that it must be that $C(e) \in Q_W$. From the fact that $C \in \llbracket \lambda \rrbracket$, we deduce that $f(e) = x_{(p,k)}$, with $(p, k) \in \mathbf{lab}(\mathcal{S})$. Let $S = (pr, p, k) \in \mathcal{S}$, from **CondImpl2**, it holds that $C(e) \in pr \cup \{p\}$. As $C(e) \in Q_W$ and $p \in Q_A$, we deduce that $C(e) \in pr$.
 - * If $(p, k) \notin \mathbf{lab}(\mathcal{S}')$, it means that $S \xrightarrow{t} \text{Done}$, and we let $C'(e) = p$. By definition of $S \xrightarrow{t} \text{Done}$, we deduce that $(C(e), ?a, p) \in T$.
 - * Otherwise, let $S' = (pr', p, k) \in \mathcal{S}'$. By definition of the VASS, it means that either $S \xrightarrow{t} S'$, or $S \xrightarrow{t, +q'} S'$. In both cases, there exists a configuration C'' such that $\dot{q} \oplus \dot{pr} \xrightarrow{1, t} \dot{q}' \oplus C''$ and either $\mathbf{set}(C'') = pr'$ or $\mathbf{set}(C'') \cup \{q'\} = pr'$. Let e' such that $(\dot{q} \oplus \dot{pr})(e') = C(e)$ (it exists as $C(e) \in pr$), then, we define $C'(e) = (\dot{q}' \oplus C'')(e')$. Then we know that $(C(e), ?a, C'(e)) \in T$ or $C(e) = C'(e)$ and $a \notin R(C(e))$.

It is clear from the construction of C' that $C \rightarrow C'$. We show now that $C' \in \llbracket \lambda' \rrbracket$ by defining $f' : [1, n] \rightarrow X$ as follows.

$$f'(e_q) = \begin{cases} x_{q'} & \text{if } q' \in Q_A \\ x_{p,k} & \text{if } q' \in Q_W, \text{ and } \mathbf{lab}(\mathcal{S}') \subseteq \mathbf{lab}(\mathcal{S}), \text{ with } x_{p,k} \text{ the unique counter} \\ & \text{such that } \delta(x_{p,k}) = 1 \\ x_{p,k} & \text{if } q' \in Q_W, \text{ and } \mathbf{lab}(\mathcal{S}') \not\subseteq \mathbf{lab}(\mathcal{S}), \text{ with } (p, k) \in \mathbf{lab}(\mathcal{S}') \setminus \mathbf{lab}(\mathcal{S}) \end{cases}$$

$$f'(e) = f(e) \quad \text{for all other } e \in [1, n]$$

We have to prove **CondImpl1**, **CondImpl2** and **CondImpl3**, but also that for all $x \in X$, $P(x): |f'^{-1}(x)| = v'(x)$.

*Proof of **CondImpl1** and $P(x_q)$ for all $q \in Q_A$.*

Let $p \in Q_A$ and $e \neq e_q$ such that $f'(e) = p$. By construction $f(e) = f'(e) = x_p$. Then $C(e) = p$, and by definition of C' , $C'(e) = p$.

If e_q is such that $f'(e_q) = x_p$, it means that $p = q' \in Q_A$. In that case, $C'(e_q) = p$. In any case, **CondImpl1** is satisfied. Moreover, if $p \neq q'$, $v'(x_p) = v(x_p)$ and $f'^{-1}(x_p) = f^{-1}(x_p)$, hence $|f'^{-1}(x_p)| = v'(x_p)$. If $p = q'$, then $v'(x_p) = v(x_p) + 1$ and $f'^{-1}(x_p) = f^{-1}(x_p) \cup \{e_q\}$ thus $|f'^{-1}(x_p)| = v'(x_p)$.

*Proof of **CondImpl2** and $P(x_{p,k})$ for all $(p, k) \in \mathbf{lab}(\mathcal{S}')$.*

Let (p, k) such that $S' = (pr', p, k) \in \mathcal{S}'$, and $e \in f'^{-1}(x_{p,k})$.

- First case: if $(pr, p, k) \in \mathcal{S}$ for some $pr \subseteq Q_W$.

³ Recall that $R(p) = \{a \mid \exists p' \in Q, (p, ?a, p') \in T\}$.

- * If $e \neq e_q$, then, from f' definition, $e \in f'^{-1}(x_{p,k})$. Hence, from **CondImpl2** on C and λ , we have that $C(e) \in \text{pr} \cup \{p\}$. By construction of C' , either $C(e) = p = C'(e)$, or $C(e) \in \text{pr}$ and $C'(e) \in \text{pr}'$.
- * If $e = e_q$, by definition of f' , $q' \in Q_W$, and as $(\text{pr}, p, k) \in \mathcal{S}$, we have that $\text{lab}(\mathcal{S}) = \text{lab}(\mathcal{S}')$, and $\delta(x_{p,k}) = 1$ (by definition of the VASS). By definition of V_P again, this means that $S \xrightarrow{t, q'} S'$, hence $q' \in \text{pr}'$. Hence, $C'(e) = q' \in \text{pr}'$.
- Second case: if $(p, k) \notin \text{lab}(\mathcal{S})$.
 - * If $e \neq e_q$, then, from f' definition, $e \in f'^{-1}(x_{p,k})$. Since $(p, k) \notin \text{lab}(\mathcal{S})$, by **CondImpl3**, $C(e) = p \in Q_A$, and then $C'(e) = p \in \text{pr}' \cup \{p\}$.
 - * If $e = e_q$, $C'(e) = q'$, and by definition of the VASS V_P , $S' = (\{q'\}, p, k)$.

Then **CondImpl2** is met. Moreover, if $f'(e_q) = x_{p,k}$ then $|f'^{-1}(x_{p,k})| = |f^{-1}(x_{p,k})| + 1$, and in that case, $v'(x_{p,k}) = v(x_{p,k}) + 1$. Otherwise, $|f'^{-1}(x_{p,k})| = |f^{-1}(x_{p,k})| = v(x_{p,k}) = v'(x_{p,k})$.

*Proof of **CondImpl3** and $P(x_{p,k})$ for all $(p, k) \notin \text{lab}(\mathcal{S}')$.*

Finally, let $(p, k) \notin \text{lab}(\mathcal{S}')$, and let $e \in f'^{-1}(x_{p,k})$. By definition of f' , $e \in f^{-1}(x_{p,k})$. If $(p, k) \notin \text{lab}(\mathcal{S})$, then by **CondImpl3** on C , $C(e) = p \in Q_A$, and then $C'(e) = p$. Otherwise, let $S = (\text{pr}, p, k) \in \mathcal{S}$, we must have $S \xrightarrow{t} \text{Done}$. By construction, $C'(e) = p$. Then **CondImpl3** is satisfied by C' . Moreover, $v(x_{p,k}) = v'(x_{p,k})$ and $|f'^{-1}(x_{p,k})| = |f^{-1}(x_{p,k})|$.

We have hence proved that $C' \in \llbracket \lambda' \rrbracket$.

□

7.3.6 Completeness of the construction

We show now how we go from a *well-formed* execution of P to an execution of V_P . This part is more complex than the previous step, as to choose the right successor in the VASS, we need to know “the future” of a process, meaning its next action (if any). Hence, the associated S-configuration of a configuration of P depends on the whole execution.

Let us first introduce some new definitions. Given a well-formed execution $\rho \in \text{Exec}[P]$, two indices $0 \leq i < j \leq \text{size}(\rho)$ and an action state $q_a \in Q_A$, we define $E_{q_a, j}^{\rho, i}$, the set of processes in waiting states in ρ_i , and whose next action state is q_a , reached at ρ_j . Formally,

$$E_{q_a, j}^{\rho, i} = \{e \in [1, \# \text{proc}(\rho)] \mid \rho_i(e) \in Q_W \text{ and } \text{na}(\rho, i, e) = (q_a, j)\}.$$

Furthermore, an S-configuration $\lambda = (\mathcal{S}, v)$ is a **representative** of the configuration ρ_i in ρ iff the following conditions are respected:

CondRepr1 for all $q \in Q_A$, $v(x_q) = |\rho_i^{-1}(q)|$;

and for all $q_a \in Q_A$, there is an injective function $r_{q_a} : \mathbf{na-index-set}(\rho, i, q_a) \rightarrow [1, |Q_W| + 1]$ s.t.⁴:

CondRepr2 $\mathcal{S} = \{(\rho_i(E_{q_a,j}^{\rho,i}), q_a, r_{q_a}(j)) \mid q_a \in Q_A, j \in \mathbf{na-index-set}(\rho, i, q_a)\}$

CondRepr3 for all $q_a \in Q_A$, we have $v(x_{(q_a, r_{q_a}(j))}) = |E_{q_a,j}^{\rho,i}|$ for all $j \in \mathbf{na-index-set}(\rho, i, q_a)$ and $v(x_{(q_a, k)}) = 0$ for all $k \notin r_{q_a}(\mathbf{na-index-set}(\rho, i, q_a))$.

In a **representative** of the configuration ρ_i , the counters faithfully count the number of processes on active states (**CondRepr1**), and on waiting regions. The set $E_{q_a,j}^{\rho,i}$ gathers exactly the set of processes that populate a same waiting region in a representative of ρ_i in ρ : they all reach the same next action state q_a at the same instant j . The set $\mathbf{na-index-set}(\rho, i, q_a)$ being bounded by $|Q_W|$ (cf. Lemma 7.2), we can associate with each of these indices a unique identifier, given by the injective function r_{q_a} . We use a spare identifier for technical reasons, to handle more easily situations when $|Q_W|$ summaries exist in the location of the VASS, and a new summary is created while one of the previous summaries is deleted at the next step. Then, **CondRepr2** and **CondRepr3** require that the counters corresponding to the summaries and the summaries themselves reflect faithfully the situation in the configuration ρ_i with respect to ρ . Observe that such a representative is tightly linked to the simulated execution, since we need to know the future behavior of the different processes to determine the different summaries.

Finally, if we let $\mathbf{repr}(\rho, i)$ be the set of all S -configurations that are representatives of ρ_i in ρ , we have the following result.

Lemma 7.4. Let ρ be a *well-formed* execution, and $0 \leq i < \text{size}(\rho) - 1$. For all $\lambda_i \in \mathbf{repr}(\rho, i)$, there exists $\lambda_{i+1} \in \mathbf{repr}(\rho, i+1)$ such that $\lambda_i \rightsquigarrow^* \lambda_{i+1}$ in V_P .

Proof. The proof will proceed as follows: from an S -configuration $\lambda_i = (S, v)$ in $\mathbf{repr}(\rho, i)$, we can compute the effect on $\mathcal{S}$ of the transition $t = (q, !!a, q')$ taken in ρ to go from ρ_i to ρ_{i+1} , and find a matching transition $(S, \delta, S') \in \Delta_t$ (according to whether q' is a waiting state or not, and in the latter case, whether it joins an existing summary or a new one, depending of the execution ρ). We then obtain a new configuration $\lambda' = (S', v')$ such that $\lambda_i \xrightarrow{\delta} \lambda'$. Note that λ' is not necessarily in $\mathbf{repr}(\rho, i+1)$: if some summaries have been deleted between λ_i and λ' , the transition (S, δ, S') in V_P has not updated the corresponding counters. Hence we need to apply transitions from Δ_e to empty counters corresponding to deleted summaries and accordingly increase corresponding counters on matching action states to obtain $\lambda_{i+1} = (S', v_{i+1})$ in $\mathbf{repr}(\rho, i+1)$.

We start by building λ' . Let $C_i = \rho_i$ and let $\lambda_i = (S, v) \in \mathbf{repr}(\rho, i)$, then:

- **CondRepr1** for all $q \in Q_A$, $v(x_q) = |C_i^{-1}(q)|$;

and for all $q_a \in Q_A$, there is an injective function $r_{q_a} : \mathbf{na-index-set}(\rho, i, q_a) \rightarrow [1, |Q_W| + 1]$ s.t.:

- **CondRepr2** $\mathcal{S} = \{(C_i(E_{q_a,j}^{\rho,i}), q_a, r_{q_a}(j)) \mid q_a \in Q_A, j \in \mathbf{na-index-set}(\rho, i, q_a)\}$, and,

⁴ Recall that $\mathbf{na-index-set}(\rho, i, q_a)$ is the set of indices j such that there exists a process e whose next action from ρ_i is (q_a, j) .

- **CondRepr3** for all $q_a \in Q_A$, we have $v(x_{q_a, r_{q_a}(j)}) = |E_{q_a, j}^{\rho, i}|$ for all $j \in \mathbf{na-index-set}(\rho, i, q_a)$ and $v(x_{q_a, k}) = 0$ for all $k \notin r_{q_a}(\mathbf{na-index-set}(\rho, i, q_a))$.

Let $t = (q, !!m, q')$ and $e_q \in [1, \#proc(\rho)]$ such that $C_i \xrightarrow{e_q, t} C_{i+1} = \rho_{i+1}$.

We start by building an S -configuration $\lambda' = (S', v')$ such that $\lambda_i \xrightarrow{\delta} \lambda'$ for a transition $\delta \in \Delta_t$. We will then show that an appropriate sequence of transitions from Δ_e allows to go from λ' to a configuration $\lambda_{i+1} \in \mathbf{repr}(\rho, i+1)$.

We state first some remarks about $E_{q_a, j}^{\rho, i}$, for $q_a \in Q_A$ and $j \in \mathbf{na-index-set}(\rho, i, q_a)$. Observe that, since

$$E_{q_a, j}^{\rho, i} = \{e \in [1, \#proc(\rho)] \mid C_i(e) \subseteq Q_W \text{ and } \mathbf{na}(\rho, i, e) = (q_a, j)\},$$

if $C_{i+1}(e) \in Q_A$ for some $e \in E_{q_a, j}^{\rho, i}$, then it means that $j = i+1$ and $C_{i+1}(e) = q_a$ for all $e \in E_{q_a, j}^{\rho, i}$. Hence,

$$C_{i+1}(E_{q_a, j}^{\rho, i}) \subseteq Q_W \text{ or } C_{i+1}(E_{q_a, j}^{\rho, i}) = \{q_a\}. \quad (7.1)$$

Observe also that if $j \in \mathbf{na-index-set}(\rho, i, q_a)$:

$$E_{q_a, j}^{\rho, i+1} = \begin{cases} \emptyset & \text{if } j = i+1 \\ E_{q_a, j}^{\rho, i} \cup \{e_q\} & \text{if } q' \in Q_W \text{ and } \mathbf{na}(\rho, i+1, e_q) = (q_a, j) \\ E_{q_a, j}^{\rho, i} & \text{otherwise.} \end{cases} \quad (7.2)$$

Moreover, if $q' \in Q_W$, then

if $\mathbf{na}(\rho, i, e_q) = (q_a, j)$ s.t. $j \notin \mathbf{na-index-set}(\rho, i, q_a)$, then $E_{q_a, j}^{\rho, i} = \emptyset$ and $E_{q_a, j}^{\rho, i+1} = \{e_q\}$. (7.3)

Definition of λ' . For all $S = (pr, q_a, k) \in \mathcal{S}$, we let $j = r_{q_a}^{-1}(k)$ and we define

$$S^{+1} = \begin{cases} (C_{i+1}(E_{q_a, j}^{\rho, i}), q_a, k) & \text{if } C_{i+1}(E_{q_a, j}^{\rho, i}) \subseteq Q_W \\ \text{Done} & \text{otherwise.} \end{cases}$$

Recall that $t = (q, !!m, q')$. S^{+1} represents the set of states reached by the states in the print of S after the sending of the message m in ρ . We then let

$$\mathcal{S}_0 = \{S^{+1} \mid S \in \mathcal{S}\} \setminus \{\text{Done}\}.$$

We first show that $\mathcal{S}_0$ is **coherent**, i. e. for all distinct pairs $(pr_1, q_1, k_1), (pr_2, q_2, k_2) \in \mathcal{S}_0$ such that $q_1 = q_2$, we have $k_1 \neq k_2$ and $pr_1 \cap pr_2 = \emptyset$. If it is coherent then $\mathcal{S}_0$ is indeed a location of the VASS.

Claim 7.1. $\mathcal{S}_0$ is **coherent**.

Proof. Let $S_1 = (pr_1, q_a, k_1), S_2 = (pr_2, q_a, k_2) \in \mathcal{S}$ such that $S_1^{+1} = (pr'_1, q_a, k_1)$ and $S_2^{+1} = (pr'_2, q_a, k_2)$ are two summaries in $\mathcal{S}_0$. Since $\mathcal{S}$ is coherent, we know that $k_1 \neq k_2$. Assume that there is some $p \in pr'_1 \cap pr'_2$. It means that there exist $e_1, e_2 \in [1, \#proc(\rho)]$ such that $C_{i+1}(e_1) = C_{i+1}(e_2) = p$, and $e_1 \in E_{q_a, r_{q_a}^{-1}(k_1)}^{\rho, i}$ and $e_2 \in E_{q_a, r_{q_a}^{-1}(k_2)}^{\rho, i}$. Hence by definition, $\mathbf{na}(\rho, i, e_1) = (q_a, r_{q_a}^{-1}(k_1)) = \mathbf{na}(\rho, i+1, e_1)$ and $\mathbf{na}(\rho, i, e_2) = (q_a, r_{q_a}^{-1}(k_2)) = \mathbf{na}(\rho, i+1, e_2)$. Since ρ is **well-formed**, we have that $r_{q_a}^{-1}(k_1) = r_{q_a}^{-1}(k_2)$, and hence, $k_1 = k_2$, which is a contradiction. Then for any two distinct summaries $S_1^{+1} = (pr'_1, q_a, k_1)$ and $S_2^{+1} = (pr'_2, q_a, k_2)$, it holds that $k_1 \neq k_2$ and $pr'_1 \cap pr'_2 = \emptyset$. $\square$

Definition of $\mathcal{S}'$. Now that we have defined $\mathcal{S}_0$, which captures the behavior of the processes that were in waiting states when the transition t has been taken, we define $\mathcal{S}'$ that incorporates the process e_q that arrives in state q' , along with the new valuation v' . In particular, if $q' \in Q_W$, then e_q reaches a zone of waiting states and must enter one of the summaries. Observe first that $C_i(e_q) = q$, then $v(\mathbf{x}_q) = |C_i^{-1}(q)| > 0$, then we let $v'(\mathbf{x}_q) = v(\mathbf{x}_q) - 1$.

- If $q' \in Q_A$, $\mathcal{S}' = \mathcal{S}_0$ and $v'(\mathbf{x}_{q'}) = v(\mathbf{x}_{q'}) + 1$. For all other counter $\mathbf{x}$, $v'(\mathbf{x}) = v(\mathbf{x})$.
- If $q' \in Q_W$, let $\mathbf{na}(\rho, i+1, e_q) = (p, j)$.
 1. If $j \in \mathbf{na-index-set}(\rho, i, p)$, then e_q will join an existing summary

$$S = (pr, p, r_p(j)) \in \mathcal{S}.$$

Since $\mathbf{na}(\rho, i+1, e_q) = (p, j)$ and $q' \in Q_W$, $j \neq i+1$ and $C_{i+1}(E_{p,j}^{\rho, i}) \subseteq Q_W$. Hence, $S^{+1} \neq \text{Done}$. We then let

$$\mathcal{S}' = (\mathcal{S}_0 \setminus \{S^{+1}\}) \cup \{(\mathbf{print}(S^{+1}) \cup \{q'\}, p, r_p(j))\}.$$

Then $v'(\mathbf{x}_{p, r_p(j)}) = v(\mathbf{x}_{p, r_p(j)}) + 1$ and $v'(\mathbf{x}) = v(\mathbf{x})$ for all other counter $\mathbf{x}$.

2. If $j \notin \mathbf{na-index-set}(\rho, i, p)$, then e_q will enter a new summary. Observe first that

$$\mathbf{na-index-set}(\rho, i+1, p) = (\mathbf{na-index-set}(\rho, i, p) \cup \{j\}) \setminus \{i+1\}.$$

So

$$|\mathbf{na-index-set}(\rho, i, p)| \leq |\mathbf{na-index-set}(\rho, i+1, p)| \leq |Q_W| < 1 + |Q_W|,$$

and there exists $k \in [1, |Q_W| + 1]$ such that $k \notin r_p(\mathbf{na-index-set}(\rho, i, p))$. This means that (p, k) is a label that is not used in the summaries of $\mathcal{S}_0$ and we let $\mathcal{S}' = \mathcal{S}_0 \cup \{(\{q'\}, p, k)\}$. Now, $v'(\mathbf{x}_{p, k}) = v(\mathbf{x}_{p, k}) + 1$, and for all other counter $\mathbf{x}$, $v'(\mathbf{x}) = v(\mathbf{x})$.

We now prove that $(\mathcal{S}', v')$ actually represents a configuration of the VASS V_p (i. e. that $\mathcal{S}'$ is **coherent**).

Claim 7.2. $\lambda' = (\mathcal{S}', v')$ is an S -configuration.

Proof. To prove that λ' is an S -configuration, we need to show that $\mathcal{S}'$ is coherent. The definition of $\mathcal{S}'$ depends on the belonging of q' to Q_A or Q_W .

- If $q' \in Q_A$, then $\mathcal{S}' = \mathcal{S}_0$, and we have proved in the previous claim that $\mathcal{S}_0$ is coherent.
- if $q' \in Q_W$, let $S_1 = (pr_1, q_a, k_1)$ and $S_2 = (pr_2, q_a, k_2) \in \mathcal{S}'$, for some $q_a \in Q_A$. If $S_1, S_2 \in \mathcal{S}_0$, by coherence of $\mathcal{S}_0$, we have that $k_1 \neq k_2$ and $pr_1 \cap pr_2 = \emptyset$. Otherwise, assume that $S_2 \in \mathcal{S}' \setminus \mathcal{S}_0$. Depending on whether e_q has joined an existing summary or a new one, $S_2 = (pr \cup \{q'\}, p, k_2)$ with pr empty or not. Then $S_1 = (pr_1, p, k_1)$ and if $pr \neq \emptyset$, $(pr, p, k_2) \in \mathcal{S}_0$. By coherence of $\mathcal{S}_0$, $k_1 \neq k_2$ and $pr_1 \cap pr = \emptyset$. Moreover, if $S_2 = (\{q'\}, p, k_2)$, by construction, $k_2 \neq k_1$. Assume now that there exists $p' \in pr_1 \cap pr_2$. Then $p' = q'$, and there exists $e_1 \in [1, \#proc(\rho)]$ such that $C_{i+1}(e_1) = q'$ and $na(\rho, i, e_1) = (p, r_p^{-1}(k_1))$. But $na(\rho, i, e_q) = (p, j)$. Since ρ is **well-formed**, $j = r_p^{-1}(k_1)$, which is impossible because $k_2 \neq k_1$. Hence $pr_1 \cap pr_2 = \emptyset$, and $\mathcal{S}'$ is coherent. □

We now proceed to prove that λ' is a successor of λ_i . To do this, we prove the intermediate claim.

Claim 7.3. For all $S \in \mathcal{S}$, $S \xRightarrow{t} S^{+1}$.

Proof. Let $S = (pr, q_a, k) \in \mathcal{S}$ with $j' = r_{q_a}^{-1}(k)$. By construction, $pr = C_i(E_{q_a, j'}^{\rho, i})$. Assume now that there exist $e_1, e_2 \in E_{q_a, j'}^{\rho, i}$ such that $C_i(e_1) = C_i(e_2)$. Since ρ is **well-formed**, $C_{i+1}(e_1) = C_{i+1}(e_2)$. Then, for all $s \in pr$, there exists a unique $s^{+1} \in Q$ such that $C_{i+1}(e) = s^{+1}$ for all $e \in C_i^{-1}(s)$. Moreover, $C_{i+1}(E_{q_a, j'}^{\rho, i}) = \{s^{+1} \mid s \in pr\}$. We define a configuration C' as follows: Let $e \in [1, |\dot{pr}|]$ such that $\dot{pr}(e) = p_e \in pr$, then $C'(e) = p_e^{+1}$. Hence $\text{set}(C') = \{s^{+1} \mid s \in pr\} = C_{i+1}(E_{q_a, j'}^{\rho, i})$.

We show now that $\dot{q} \oplus \dot{pr} \xrightarrow{1, t} \dot{q}' \oplus C'$. Let $e \in [1, |\dot{pr}|]$ such that $\dot{pr}(e) = p_e$, then there exists $e' \in E_{q_a, j'}^{\rho, i}$ such that $C_i(e') = p_e$ and $p_e^{+1} = C_{i+1}(e')$.

Since $C_i \xrightarrow{e_q, t} C_{i+1}$, then either $(p_e, ?m, p_e^{+1}) \in T$ or $m \notin R(p_e)$ and $p_e = p_e^{+1}$, and $\dot{q} \oplus \dot{pr} \xrightarrow{1, t} \dot{q}' \oplus C'$. We have already established that $\text{set}(C') = C_{i+1}(E_{q_a, j'}^{\rho, i}) \subseteq Q_W$ or $\text{set}(C') = C_{i+1}(E_{q_a, j'}^{\rho, i}) = \{q_a\}$ (cf. 7.1). In the latter case, $S^{+1} = \text{Done}$. In the former case, $S^{+1} = (C_{i+1}(E_{q_a, j'}^{\rho, i}), q_a, k)$ and $S \xRightarrow{t} S^{+1}$. In any case, we have that $S \xRightarrow{t} S^{+1}$. □

We are now ready to prove that λ' is reachable from λ_i

Claim 7.4. There exists $(\mathcal{S}, \delta, \mathcal{S}') \in \Delta_t$ such that $\lambda_i \xrightarrow{\delta} \lambda'$.

Proof. • If $q' \in Q_A$, from V_P 's construction, we must have $\delta(x_q) = -1$, $\delta(x_{q'}) = 1$ and for all other counter x , $\delta(x) = 0$. Observe that we have $v'(x_q) = v(x_q) - 1$, $v'(x_{q'}) = v(x_{q'}) + 1$, and $v'(x) = v(x)$ for all other counter x . Moreover, $\mathcal{S}' = \mathcal{S}_0 = \{S^{+1} \mid S \in \mathcal{S}\} \setminus \{\text{Done}\}$. Since $S \xrightarrow{t} S^{+1}$ for all $S \in \mathcal{S}$, we have $(\mathcal{S}, \delta, \mathcal{S}') \in \Delta_t$ and $\lambda_i \xrightarrow{\delta} \lambda'$.

• If $q' \in Q_W$ and there exists some $S = (\text{pr}, p, r_p(j))$ such that $\mathcal{S}' = (\mathcal{S}_0 \setminus \{S^{+1}\}) \uplus \{(\text{print}(S^{+1}) \cup \{q'\}, p, r_p(j))\}$. Then $v'(x_q) = v(x_q) - 1$, $v'(x_{p, r_p(j)}) = v(x_{p, r_p(j)}) + 1$ and $v'(x) = v(x)$ for all other counter x . By definition, $S \xrightarrow{t, q'} (\text{print}(S^{+1}) \cup \{q'\}, p, r_p(j))$, because $S \xrightarrow{t} S^{+1}$. Then, with $\delta(x_q) = -1$, $\delta(x_{p, r_p(j)}) = 1$ and $\delta(x) = 0$ for all other counter, we have $(\mathcal{S}, \delta, \mathcal{S}') \in \Delta_t$, and $\lambda_i \xrightarrow{\delta} \lambda'$.

• If $q' \in Q_W$ and $\mathcal{S}' = \mathcal{S}_0 \cup \{(\{q'\}, p, k)\}$ for some $k \in [1, |Q_W| + 1]$. Then $(p, k) \notin \text{lab}(\mathcal{S})$ by construction. Moreover, $v'(x_q) = v(x_q) - 1$, and $v'(x_{p, k}) = 1$. For all other counter x , $v'(x) = v(x)$. For δ such that $\delta(x_q) = -1$, $\delta(x_{p, k}) = 1$ and $\delta(x) = 0$ for all other x , $(\mathcal{S}, \delta, \mathcal{S}') \in \Delta_t$ and $\lambda_i \xrightarrow{\delta} \lambda'$.

We successfully proved the claim. $\square$

We are now ready to build λ_{i+1} which will be the configuration in $\text{repr}(\rho, i+1)$: from λ' , we should empty counters labeled by summaries which have arrived between ρ_i and ρ_{i+1} , so that we get an S-configuration in $\text{repr}(\rho, i+1)$.

Definition of λ_{i+1} . First observe that for all $q_a \in Q_A$, there is at most one $S = (\text{pr}, q_a, k) \in \mathcal{S}$ such that $S^{+1} = \text{Done}$. Indeed, suppose there exist $S_1 = (\text{pr}_1, q_a, k_1)$, $S_2 = (\text{pr}_2, q_a, k_2) \in \mathcal{S}$ such that $S_1^{+1} = S_2^{+1} = \text{Done}$. By definition, $\text{pr}_1 = C_i(E_{q_a, r_{q_a}^{-1}(k_1)}^{\rho, i})$ and $\text{pr}_2 = C_i(E_{q_a, r_{q_a}^{-1}(k_2)}^{\rho, i})$. Let $j_1 = r_{q_a}^{-1}(k_1)$ and $j_2 = r_{q_a}^{-1}(k_2)$. Then, since $C_{i+1}(E_{q_a, j_i}^{\rho, i}) \notin Q_W$ for $i \in \{1, 2\}$, necessarily $C_{i+1}(E_{q_a, j_1}^{\rho, i}) = C_{i+1}(E_{q_a, j_2}^{\rho, i}) = \{q_a\}$. This implies that $j_1 = j_2 = i+1$ and then that $k_1 = k_2$, which contradicts the coherence of $\mathcal{S}$. This means that, there is at most one summary for each $q_a \in Q_A$ that disappears while going from $\mathcal{S}$ to $\mathcal{S}'$. This corresponds to a set of processes leaving a waiting zone and reaching the state q_a together.

We capture this new configuration of the protocol by transferring the value of the counter $x_{(q_a, r_{q_a}(i+1))}$ into the counter x_{q_a} .

So we define v_{i+1} as follows: for all $q_a \in Q_A$,

- if $i+1 \in \text{na-index-set}(\rho, i, q_a)$, let $k = r_{q_a}(i+1)$ and $v_{i+1}(x_{q_a}) = v'(x_{q_a}) + v'(x_{q_a, k})$ and $v_{i+1}(x_{q_a, k}) = 0$.
- Otherwise, $v_{i+1}(x_{q_a}) = v'(x_{q_a})$.

For all other counter x , $v_{i+1}(x) = v'(x)$. We let $\lambda_{i+1} = (\mathcal{S}', v_{i+1})$.

We now proceed to prove that $\lambda_{i+1} \in \mathbf{repr}(\rho, i+1)$, before finally proving that $\lambda' \rightsquigarrow^* \lambda_{i+1}$. To prove that $\lambda_{i+1} \in \mathbf{repr}(\rho, i+1)$, we successively prove that it respects **CondRepr1**, **CondRepr3**, and **CondRepr2**.

Claim 7.5. λ_{i+1} respects **CondRepr1**⁵.

Proof. Let $q_a \in Q_A$ and $e_q \in [1, \# \mathbf{proc}(\rho)]$ such that $C_i \xrightarrow{e_q, t} C_{i+1}$. Then, $C_i(e_q) = q$, $C_{i+1}(e_q) = q'$. For all $e \in [1, \# \mathbf{proc}(\rho)]$, $C_{i+1}(e) = q_a$ if and only if

$$\begin{aligned} & (C_i(e) = q_a \text{ and } q \neq q_a) \text{ or } (C_i(e) = q_a \text{ and } e \neq e_q) \\ & \text{or } ((C_i(e), ?m, q_a) \in T \text{ and } \mathbf{na}(\rho, i, e) = (q_a, i+1)) \\ & \text{or } (q_a = q' \wedge e = e_q). \end{aligned}$$

From this we can deduce that

$$C_{i+1}^{-1}(q_a) = \begin{cases} C_i^{-1}(q_a) \cup E_{q_a, i+1}^{\rho, i} & \text{if } q_a \neq q \text{ and } q_a \neq q' \\ C_i^{-1}(q_a) \cup \{e_q\} \cup E_{q_a, i+1}^{\rho, i} & \text{if } q_a = q' \\ (C_i^{-1}(q_a) \setminus \{e_q\}) \cup E_{q_a, i+1}^{\rho, i} & \text{if } q_a = q. \end{cases}$$

Since $C_i^{-1}(q_a)$ and $E_{q_a, i+1}^{\rho, i}$ are disjoint, and if $e_q \in C_i^{-1}(q) \setminus C_i^{-1}(q')$ (as $q \neq q'$, it is always true⁶), we have that

$$|C_{i+1}^{-1}(q_a)| = \begin{cases} |C_i^{-1}(q_a)| + |E_{q_a, i+1}^{\rho, i}| & \text{if } q_a \neq q \text{ and } q_a \neq q' \\ |C_i^{-1}(q_a)| + 1 + |E_{q_a, i+1}^{\rho, i}| & \text{if } q_a = q' \\ |C_i^{-1}(q_a)| - 1 + |E_{q_a, i+1}^{\rho, i}| & \text{if } q_a = q. \end{cases}$$

Since $\lambda = (\mathcal{S}, v) \in \mathbf{repr}(\rho, i)$, $v(\mathbf{x}_{q_a}) = |C_i^{-1}(q_a)|$ and, by definition of v' , we have that

$$|C_{i+1}^{-1}(q_a)| = v'(\mathbf{x}_{q_a}) + |E_{q_a, i+1}^{\rho, i}|$$

Now, if $i+1 \notin \mathbf{na-index-set}(\rho, i, q_a)$, it means that $E_{q_a, i+1}^{\rho, i} = \emptyset$. In that case, by definition of v_{i+1} , $v_{i+1}(\mathbf{x}_{q_a}) = v'(\mathbf{x}_{q_a})$. By, definition of v' ,

- if $q_a = q$, $v'(\mathbf{x}_{q_a}) = v(\mathbf{x}_{q_a}) + 1$,
- if $q_a = q'$, $v'(\mathbf{x}_{q_a}) = v(\mathbf{x}_{q_a}) - 1$,
- otherwise, $v'(\mathbf{x}_{q_a}) = v(\mathbf{x}_{q_a})$.

Hence, in all cases, $v_{i+1}(\mathbf{x}_{q_a}) = v'(\mathbf{x}_{q_a}) = |C_{i+1}^{-1}(q_a)|$.

⁵ **CondRepr1**: for all $q \in Q_A$, $v_{i+1}(\mathbf{x}_q) = |C_{i+1}^{-1}(q)|$.

⁶ Remember that we assumed the protocol did not contain any self loop in this chapter.

Otherwise, i.e. if $i + 1 \in \mathbf{na-index-set}(\rho, i, q_a)$, $v_{i+1}(\mathbf{x}_{q_a}) = v'(\mathbf{x}_{q_a}) + v'(\mathbf{x}_{(q_a, r_{q_a}(i+1))})$. The only case where $v'(\mathbf{x}_{(q_a, r_{q_a}(i+1))}) \neq v(\mathbf{x}_{(q_a, r_{q_a}(i+1))})$ is if $q' \in Q_W$ and $\mathbf{na}(\rho, i + 1, e_q) = (q_a, i + 1)$ which is impossible. So $v'(\mathbf{x}_{(q_a, r_{q_a}(i+1))}) = v(\mathbf{x}_{(q_a, r_{q_a}(i+1))})$ and, since $\lambda \in \mathbf{repr}(\rho, i)$, $v(\mathbf{x}_{(q_a, r_{q_a}(i+1))}) = |E_{q_a, i+1}^{\rho, i}|$. Hence, $v_{i+1}(\mathbf{x}_{q_a}) = v'(\mathbf{x}_{q_a}) + |E_{q_a, i+1}^{\rho, i}|$. Using the same argument as for the first case, we get $v_{i+1}(\mathbf{x}_{q_a}) = v'(\mathbf{x}_{q_a}) + |E_{q_a, i+1}^{\rho, i}| = |C_{i+1}^{-1}(q_a)|$ proving **CondRepr1**. $\square$

In order to prove **CondRepr3**, and **CondRepr2**, we now define the new renaming function for λ_{i+1} . Let $q_a \in Q_A$. We define an injective function $r'_{q_a} : \mathbf{na-index-set}(\rho, i + 1, q_a) \rightarrow [1, |Q_W| + 1]$ as follows.

Recall that

$$\mathbf{na-index-set}(\rho, i + 1, q_a) = \begin{cases} (\mathbf{na-index-set}(\rho, i, q_a) \setminus \{i + 1\}) \cup \{\mathbf{na-index}(\rho, i + 1, e_q)\} & \text{if } q' \in Q_W \text{ and } \mathbf{na-state}(\rho, i + 1, e_q) = q_a \\ \mathbf{na-index-set}(\rho, i, q_a) \setminus \{i + 1\} & \text{otherwise.} \end{cases}$$

The new injective function keeps the labeling of the summaries as in the previous configuration, and if needed, associates the labeling of the newly created summary to the index of the next moment e_q will reach an action state.

$$r'_{q_a}(j) = \begin{cases} r_{q_a}(j) & \text{if } j \in \mathbf{na-index-set}(\rho, i, q_a) \setminus \{i + 1\} \\ k & \text{otherwise, for the unique } k \text{ s.t. } (\{q'\}, q_a, k) \in \mathcal{S}' \text{ and } (q_a, k) \notin \mathbf{lab}(\mathcal{S}) \end{cases}$$

Claim 7.6. λ_{i+1} respects **CondRepr3**⁷.

Proof. Let $q_a \in Q_A$, and $j \in \mathbf{na-index-set}(\rho, i + 1, q_a)$.

- If $j \in \mathbf{na-index-set}(\rho, i, q_a) \setminus \{i + 1\}$, then $v_{i+1}(\mathbf{x}_{q_a, r'_{q_a}(j)}) = v_{i+1}(\mathbf{x}_{q_a, r_{q_a}(j)}) = v'(\mathbf{x}_{q_a, r_{q_a}(j)})$. By definition of v' ,

$$v'(\mathbf{x}_{q_a, r_{q_a}(j)}) = \begin{cases} v(\mathbf{x}_{q_a, r_{q_a}(j)}) + 1 & \text{if } q' \in Q_W \text{ and } \mathbf{na}(\rho, i + 1, e_q) = (q_a, j) \\ v(\mathbf{x}_{q_a, r_{q_a}(j)}) & \text{otherwise.} \end{cases}$$

We know that $v(\mathbf{x}_{q_a, r_{q_a}(j)}) = |E_{q_a, j}^{\rho, i}|$ because $\lambda_i \in \mathbf{repr}(\rho, i)$. Moreover,

$$E_{q_a, j}^{\rho, i+1} = \begin{cases} E_{q_a, j}^{\rho, i} \cup \{e_q\} & \text{if } q' \in Q_W \text{ and } \mathbf{na}(\rho, i + 1, e_q) = (q_a, j) \\ E_{q_a, j}^{\rho, i} & \text{otherwise.} \end{cases}$$

Hence, $v_{i+1}(\mathbf{x}_{q_a, r'_{q_a}(j)}) = v'(\mathbf{x}_{q_a, r_{q_a}(j)}) = |E_{q_a, j}^{\rho, i+1}|$.

⁷ **CondRepr3**: for all $q_a \in Q_A$, $v_{i+1}(\mathbf{x}_{q_a, r'_{q_a}(j)}) = |E_{q_a, j}^{\rho, i+1}|$ for all $j \in \mathbf{na-index-set}(\rho, i + 1, q_a)$ and $v_{i+1}(\mathbf{x}_{(q_a, k)}) = 0$ for all $k \notin r_{q_a}(\mathbf{na-index-set}(\rho, i + 1, q_a))$.

- If $j \notin \mathbf{na-index-set}(\rho, i, q_a)$, then $\mathbf{na}(\rho, i+1, e_q) = (q_a, j)$. By definition $v_{i+1}(\mathbf{x}_{q_a, r'_{q_a}(j)}) = v'(\mathbf{x}_{q_a, r'_{q_a}(j)}) = v'(\mathbf{x}_{q_a, k})$. This corresponds to the case 2 of the definition of v' , where $v'(\mathbf{x}_{q_a, k}) = v(\mathbf{x}_{q_a, k}) + 1$. By definition of k , $k \notin r_p(\mathbf{na-index-set}(\rho, i, q_a))$, then, since $\lambda_i = (\mathcal{S}, v) \in \mathbf{repr}(\rho, i)$, $v(\mathbf{x}_{q_a, k}) = 0$ and $v_{i+1}(\mathbf{x}_{q_a, r'_{q_a}(j)}) = 1$. Moreover,

$$E_{q_a, j}^{\rho, i} = \{e \in [1, \#proc(\rho)] \mid C_i(e) \in Q_W \text{ and } \mathbf{na}(\rho, i, e) = (q_a, j)\};$$

since $j \notin \mathbf{na-index-set}(\rho, i, q_a)$, $E_{q_a, j}^{\rho, i} = \emptyset$. Hence, $E_{q_a, j}^{\rho, i+1} = E_{q_a, j}^{\rho, i} \cup \{e_q\} = \{e_q\}$ and $v_{i+1}(\mathbf{x}_{q_a, r'_{q_a}(j)}) = 1 = |E_{q_a, j}^{\rho, i+1}|$.

Let $x \notin r'_{q_a}(\mathbf{na-index-set}(\rho, i+1, q_a))$. Then, either $x \notin \mathbf{na-index-set}(\rho, i, q_a)$, or $x = r_{q_a}(i+1)$. In the first case, $v(\mathbf{x}_{q_a, x}) = 0$, and $v_{i+1}(\mathbf{x}_{q_a, x}) = v'(\mathbf{x}_{q_a, x}) = v(\mathbf{x}_{q_a, x}) = 0$. In the second case, $v_{i+1}(\mathbf{x}_{q_a, x}) = 0$ by construction. $\square$

Claim 7.7. λ_{i+1} respects **CondRepr2**⁸.

Proof. Finally we show that **CondRepr2** holds, i.e. that $\mathcal{S}' = \{(C_{i+1}(E_{q_a, j}^{\rho, i+1}), q_a, r'_{q_a}(j)) \mid q_a \in Q_A, j \in \mathbf{na-index-set}(\rho, i+1, q_a)\}$.

- First, let $S \in \mathcal{S}'$. We prove that $S = (C_{i+1}(E_{q_a, j}^{\rho, i+1}), q_a, r'_{q_a}(j))$ for some $q_a \in Q_A$ and $j \in \mathbf{na-index-set}(\rho, i+1, q_a)$.

By construction, $S \in \mathcal{S}_0$ or $S = (\{q'\}, q_a, k)$ or there exists $S_0 = (\mathbf{pr}, q_a, r_{q_a}(j)) \in \mathcal{S}_0$ s.t. $S = (\mathbf{pr} \cup \{q'\}, q_a, r_{q_a}(j))$.

- If $S \in \mathcal{S}_0$, then $S = S_i^{+1}$ for some $S_i \in \mathcal{S}$. Since $(\mathcal{S}, v) \in \mathbf{repr}(\rho, i)$,

$$S_i = (C_i(E_{q_a, j}^{\rho, i}), q_a, r_{q_a}(j)) \text{ for some } q_a \in Q_A \text{ and } j \in \mathbf{na-index-set}(\rho, i, q_a).$$

Then, by definition of $\mathcal{S}_0$, $S = (C_{i+1}(E_{q_a, j}^{\rho, i}), q_a, r_{q_a}(j))$. Furthermore, as $S \in \mathcal{S}_0$, then $q' \in Q_A$ or $\mathbf{na}(\rho, i, e_q) \neq (q_a, j)$ and so $E_{q_a, j}^{\rho, i+1} = E_{q_a, j}^{\rho, i}$. Moreover, $S_i^{+1} \neq \text{Done}$ hence $j \neq i+1$, then $r'_{q_a}(j) = r_{q_a}(j)$. Then $S = (C_{i+1}(E_{q_a, j}^{\rho, i+1}), q_a, r'_{q_a}(j))$.

- If there exists $S_0 = (\mathbf{pr}, q_a, r_{q_a}(j)) \in \mathcal{S}$ such that $S = (\mathbf{pr} \cup \{q'\}, q_a, r_{q_a}(j))$, then by construction of $\mathcal{S}'$, $q' \in Q_W$ and $\mathbf{na}(\rho, i+1, e_q) = (q_a, j)$. In that case, $E_{q_a, j}^{\rho, i+1} = E_{q_a, j}^{\rho, i} \cup \{e_q\}$ and $j \neq i+1$ so $r'_{q_a}(j) = r_{q_a}(j)$. Then $S = (C_{i+1}(E_{q_a, j}^{\rho, i+1}), q_a, r'_{q_a}(j))$.

- Finally, if $S = (\{q'\}, q_a, k)$, then $q' \in Q_W$, with $\mathbf{na}(\rho, i+1, e_q) = (q_a, j)$ and $j \notin \mathbf{na-index-set}(\rho, i, q_a)$. Hence $E_{q_a, j}^{\rho, i+1} = \{e_q\}$ and $r'_{q_a}(j) = k$. Then, $S = (C_{i+1}(E_{q_a, j}^{\rho, i+1}), q_a, r'_{q_a}(j))$.

⁸ **CondRepr2**: $\mathcal{S}' = \{(C_{i+1}(E_{q_a, j}^{\rho, i+1}), q_a, r'_{q_a}(j)) \mid q_a \in Q_A, j \in \mathbf{na-index-set}(\rho, i+1, q_a)\}$

- Now, let $S = (C_{i+1}(E_{q_a,j}^{\rho,i+1}), q_a, r'_{q_a}(j))$ for some $q_a \in Q_A$, $j \in \mathbf{na-index-set}(\rho, i+1, q_a)$. We prove that $S \in \mathcal{S}'$.
 - If $j \in \mathbf{na-index-set}(\rho, i, q_a)$, then $j \neq i+1$ and $r'_{q_a}(j) = r_{q_a}(j)$. Let $S_0 = (C_i(E_{q_a,j}^{\rho,i}), q_a, r_{q_a}(j)) \in \mathcal{S}$. Such a summary exists because $\lambda_i = (\mathcal{S}, v)$ is in $\mathbf{repr}(C_i, i)$. If $q' \in Q_W$ and $\mathbf{na}(\rho, i+1, e_q) = (q_a, j)$, then, $E_{q_a,j}^{\rho,i+1} = E_{q_a,j}^{\rho,i} \cup \{e_q\}$. Moreover, by construction of $\mathcal{S}'$, $(C_{i+1}(E_{q_a,j}^{\rho,i}) \cup \{q'\}, q_a, r_{q_a}(j)) = (C_{i+1}(E_{q_a,j}^{\rho,i+1}) \cup C_{i+1}(e_q), q_a, r_{q_a}(j)) = (C_{i+1}(E_{q_a,j}^{\rho,i+1}), q_a, r'_{q_a}(j)) \in \mathcal{S}'$. Otherwise, $E_{q_a,j}^{\rho,i+1} = E_{q_a,j}^{\rho,i}$ and, by construction of $\mathcal{S}'$, $S_0^{+1} = (C_{i+1}(E_{q_a,j}^{\rho,i}), q_a, r_{q_a}(j)) = (C_{i+1}(E_{q_a,j}^{\rho,i+1}), q_a, r'_{q_a}(j)) \in \mathcal{S}'$.
 - If $j \notin \mathbf{na-index-set}(\rho, i, q_a)$, then $r'_{q_a}(j) = k$ and $(\{q'\}, q_a, k) \in \mathcal{S}'$. Moreover, since $j \in \mathbf{na-index-set}(\rho, i+1, q_a) \setminus \mathbf{na-index-set}(\rho, i, q_a)$, it means that $q' \in Q_W$ and $\mathbf{na}(\rho, i+1, e_q) = (q_a, j)$. Then $E_{q_a,j}^{\rho,i+1} = \{e_q\}$ and $S = (C_{i+1}(E_{q_a,j}^{\rho,i+1}), q_a, r'_{q_a}(j)) = (\{q'\}, q_a, k) \in \mathcal{S}'$.

□

The three above claims lead us to the following result.

Claim 7.8. $\lambda_{i+1} \in \mathbf{repr}(\rho, i+1)$.

It is left to prove that $\lambda' \rightsquigarrow^* \lambda_{i+1}$ to complete the proof, as we already proved that $\lambda_i \rightsquigarrow \lambda'$.

Claim 7.9. $\lambda' \rightsquigarrow^* \lambda_{i+1}$.

Proof. The only difference between λ' and λ_{i+1} is in the valuation.

We show that $\lambda' \rightsquigarrow^* \lambda_{i+1}$ by successive transitions from Δ_e in order to transfer the values of counters associated to summaries that have reached Done to the counter associated to the corresponding action state.

Let $\{q_1, \dots, q_n\} \subseteq Q_A$ the set of states q_ℓ such that $i+1 \in \mathbf{na-index-set}(\rho, i, q_\ell)$ for all $1 \leq \ell \leq n$. It is the set of action states that are reached by processes in waiting zones in C_{i+1} . We build a sequence of valuations $v_0, v_1, \dots, v_n$ with $v_0 = v'$ and $v_n = v_{i+1}$ such that $(\mathcal{S}', v_\ell) \rightsquigarrow^* (\mathcal{S}', v_{\ell+1})$, for all $0 \leq \ell < n$.

If this set is empty, it means that for all $q_a \in Q_A$, $i+1 \notin \mathbf{na-index-set}(\rho, i, q_a)$, hence, by definition of v_{i+1} , $v_{i+1} = v'$ and there is nothing to do.

Assume the set is not empty and denote $k_\ell = r_{q_\ell}(i+1)$, for all $1 \leq \ell < n$. Observe that $k_\ell \notin r'_{q_\ell}(\mathbf{na-index-set}(\rho, i+1, q_\ell))$ for all $1 \leq \ell \leq n$. Indeed, by construction, if $\mathbf{na}(\rho, i+1, e_q) = (q_\ell, j)$, then $r'_{q_\ell}(j) \neq r_{q_\ell}(i+1)$ (in particular, when building $\mathcal{S}'$ in Item 2, we chose a fresh index k). Hence, as $(\mathcal{S}', v_{i+1})$ is in $\mathbf{repr}(\rho, i+1)$, it holds that $(\mathcal{S}', \delta_\ell, \mathcal{S}') \in \Delta_e$ with $\delta_\ell(x_{q_\ell}) = 1$ and $\delta_\ell(x_{q_\ell, k_\ell}) = -1$ and $\delta_\ell(x) = 0$ for all other $x \in X$.

Hence,⁹

$$(\mathcal{S}', v_0) (\overset{\delta_1}{\rightsquigarrow})^{x_1} (\mathcal{S}', v_1) (\overset{\delta_2}{\rightsquigarrow})^{x_2} \dots (\overset{\delta_n}{\rightsquigarrow})^{x_n} (\mathcal{S}', v_n)$$

where:

- $x_\ell = v'(\mathbf{x}_{q_\ell, k_\ell})$ for all $1 \leq \ell \leq n$, and
- $v_{\ell+1}(\mathbf{x}_{q_\ell, k_\ell}) = 0$, $v_{\ell+1}(\mathbf{x}_{q_\ell}) = v_\ell(\mathbf{x}_{q_\ell}) + v_\ell(\mathbf{x}_{q_\ell, k_\ell})$, and $v_{\ell+1}(\mathbf{x}) = v_\ell(\mathbf{x})$ for all other counter $\mathbf{x} \in X$, for all $1 \leq \ell < n$.

Hence:

- $v_n(\mathbf{x}_{q_\ell, k_\ell}) = 0$ for all $1 \leq \ell \leq n$;
- $v_n(\mathbf{x}_{q_\ell}) = v_0(\mathbf{x}_{q_\ell}) + v_0(\mathbf{x}_{q_\ell, k_\ell}) = v'(\mathbf{x}_{q_\ell}) + v'(\mathbf{x}_{q_\ell, k_\ell})$ for all $1 \leq \ell \leq n$;
- $v_n(\mathbf{x}) = v_0(\mathbf{x}) = v'(\mathbf{x})$ for all other counter $\mathbf{x} \in X$.

And so $v_n = v_{i+1}$ which concludes the proof. □

□

7.4 SYNCHRONIZATION IN WAIT-ONLY BROADCAST NETWORKS

7.4.1 Regaining decidability

We now proceed to show how to adapt V_P in order to solve SYNCHRO in Wait-Only broadcast networks. First, let us make the following observation: from Lemma 7.1, we immediately get the following result.

Corollary 7.1. *Let P be a Wait-Only broadcast protocol and q_f a state of P . There exists an execution $\rho = C_0 \dots C_n$ such that $C_n(e) = q_f$ for all $e \in [1, \#proc(\rho)]$ iff there exists a well-formed execution from C_0 to C_n .*

Hence, we focus on well-formed executions. Let us consider $P = (Q, \Sigma, q_{init}, T)$ a Wait-Only broadcast protocol and consider a final waiting state $q_f \in Q_W$. We still assume that P contains an unreachable state $q_u \in Q$, which is a sink state (and hence $q_u \in Q_A$). To ease the reading, we assume in this section that the final state q_f is a waiting state, but our construction could be adapted to the case where q_f is an action state. Moreover, we show in the next section that SYNCHRO in this latter case is in fact easier to solve. We denote by $\mathcal{C}_f$ the set of configurations where all processes are in q_f , i. e.:

$$\mathcal{C}_f = \{C \in \mathcal{C} \mid \text{for all } e \in [1, |C|], C(e) = q_f\}.$$

We consider the summaries $S_f^k = (\{q_f\}, q_u, k)$ for each $k \in [1, |Q_W| + 1]$. Each of them represents a waiting region where all processes are on state q_f or q_u . As q_u is an artificially added unreachable state, this ensures that processes are all in q_f . As we will mainly use S_f^1 , we define $S_f = S_f^1$.

⁹ $(\overset{\delta_1}{\rightsquigarrow})^{x_1}$ is the x_1 -concatenation of relation $\overset{\delta_1}{\rightsquigarrow}$.

In fact, as $q_f \in Q_W$, we get the following result.

Lemma 7.5. Let $C_{init} \in \mathcal{C}_{init}$, and $C \in \mathcal{C}$ such that $C_{init} \rightarrow^* C$ and $C \in \bigcup_{n \in \mathbb{N}_{>0}} \llbracket (\{S_f\}, v_n) \rrbracket$ where $v_n(x_{q_u,1}) = n$, and $v_n(x) = 0$ for all $x \neq x_{q_u,1}$ and $n \in \mathbb{N}_{>0}$. Then it holds that $C \in \mathcal{C}_f$.

Proof. Let $C_{init} \in \mathcal{C}_{init}$, and $C \in \mathcal{C}$ such that $C_{init} \rightarrow^* C$ and $C \in \bigcup_{n \in \mathbb{N}_{>0}} \llbracket (\{S_f\}, v_n) \rrbracket$. From the definition of $\llbracket \cdot \rrbracket$ we get that: $\bigcup_{n \in \mathbb{N}_{>0}} \llbracket (\{S_f\}, v_n) \rrbracket = \bigcup_{n \in \mathbb{N}_{>0}} \{C \in \mathcal{C}_n \mid C(e) \in \{q_f\} \cup \{q_u\}\}$. Hence, for all $e \in [1, |C|]$, we have that $C(e) \in \{q_f, q_u\}$. As q_u is unreachable by definition, and $C_{init} \rightarrow^* C$, we get that $C(e) = q_f$ for all $e \in [1, |C|]$. $\square$

Conversely, we justify that the representative of the final configuration of an execution witnessing SYNCHRO, is w.l.o.g. in $\bigcup_{n \in \mathbb{N}_{>0}} \llbracket (\{S_f\}, v_n) \rrbracket$. Intuitively, once a summary S_{q_u} labeled with q_u is created, it is never deleted as the processes it represents never reach q_u . Furthermore, all processes represented by a summary labeled with q_u have the same next action index, namely, the size of the execution¹⁰. Hence, S_{q_u} is the only summary labeled by q_u in the rest of the VASS's run. We can hence *choose* its index when created, which we w.l.o.g. choose to be 1. This is formalized in the next lemma.

Lemma 7.6. Let $\rho = C_0 \dots C_{k-1} \in \text{Exec}[P]$ s.t. $C_{k-1} \in \mathcal{C}_f$ and $\lambda_0 \in \mathbf{repr}(\rho, 0)$, $\lambda \in \mathbf{repr}(\rho, k-1)$ such that $\lambda_0 \rightsquigarrow^* \lambda$, then there exists λ' s.t. $\lambda_0 \rightsquigarrow^* \lambda'$ and $\lambda' = (\{S_f\}, v')$ for a vector v' with $v'(x) = 0$ for all $x \neq x_{q_u,1}$.

Proof. Let $\rho = C_0 \dots C_{k-1} \in \text{Exec}[P]$ s.t. $C_{k-1} \in \mathcal{C}_f$ and $\lambda_0 \in \mathbf{repr}(\rho, 0)$, $\lambda \in \mathbf{repr}(\rho, k-1)$ such that $\lambda_0 \rightsquigarrow^* \lambda$. Observe that for all $e \in [1, |C_{k-1}|]$, $\mathbf{na}(\rho, k-1, e) = (q_u, k)$. From the definition of $\mathbf{repr}(\rho, k-1)$, we get that $\lambda = (\mathcal{S}, v)$ with:

- $v(x_{q_u, j_k}) = |C_{k-1}|$ for an index $j_k \in [1, |Q_W| + 1]$, and
- $v(x) = 0$ for all $x \in X \setminus \{x_{q_u, j_k}\}$, and
- $\mathcal{S} = \{S_f^{j_k}\}$.

If $j_k = 1$, then there is nothing to do, otherwise, we write $\lambda_0 \rightsquigarrow \lambda_1 \rightsquigarrow \dots \rightsquigarrow \lambda_m = \lambda$ the run of V_P leading to λ , where $\lambda_i = (\mathcal{S}_i, v_i)$ for all $1 \leq i \leq m$. Observe that for all process e , $C_0(e) = q_{init} \in Q_A$, hence $E_{q_a, j}^{\rho, 0} = \emptyset$ ¹¹ for all $q_a \in Q_A$ and $j \in [0, \text{size}(\rho)]$. As $\lambda_0 \in \mathbf{repr}(\rho, 0)$, we get that $\lambda_0 = (\emptyset, v_0)$ for a vector v_0 . Denote the first moment i_0 where there exists $j \in [1, |Q_W| + 1]$ such that there exists $S_{i_0} \in \mathcal{S}_{i_0}$ with $\mathbf{lab}(S) = (q_u, j)$. As q_u as no ingoing edges (in particular no edges shaped as $(q, ?m, q_u)$ for $q \in Q_W$ and $m \in \Sigma$), the following holds:

for all S s.t. $\mathbf{lab}(S) = (q_u, j)$, and for all $t \in T$, it is not the case that $S \xRightarrow{t} \text{Done}$.

Hence, from V_P 's construction, we get that for all $i_0 \leq i \leq m$, there exists a summary $S_i \in \mathcal{S}_i$ such that $\mathbf{lab}(S) = (q_u, j)$. As $\mathcal{S}_m = \{S_f^{j_k}\}$, we get that $j = j_k$ and no other summary labeled by (q_u, j') with $j' \neq j$ appears in the VASS's run (as otherwise, it would not be deleted and would appear in λ_m which contradicts $\mathcal{S}_m = \{S_f^{j_k}\}$).

¹¹ We remind the reader that $E_{q_a, j}^{\rho, i} = \{e \in [1, \#\text{proc}(\rho)] \mid \rho_i(e) \in Q_W \text{ and } \mathbf{na}(\rho, i, e) = (q_a, j)\}$.

¹⁰ Remember that for a process e which never reaches again an action state from an index j , we took the convention that $\mathbf{na}(\rho, j, e) = (q_u, \text{size}(\rho))$.

Consider δ such that $\lambda_{i_0-1} \xrightarrow{\delta} \lambda_{i_0}$. By definition of i_0 , a new summary with a fresh label is added between λ_{i_0-1} and λ_{i_0} hence, we are necessary in the case Item c. of the VASS's definition, where we choose a fresh label (q_u, r) such that $(q_u, r) \notin \mathbf{lab}(\mathcal{S}_{i_0-1})$. By definition of i_0 , $(q_u, 1) \notin \mathbf{lab}(\mathcal{S}_{i_0-1})$ ($\mathcal{S}_{i_0}$ is the first summary labeled by q_u to appear in the run). Hence, from the VASS's construction there is also a transition $(\mathcal{S}_{i_0-1}, \delta', \mathcal{S}'_{i_0})$ where:

- $\mathcal{S}'_{i_0} = \mathcal{S}_{i_0} \setminus \{\mathcal{S}_{i_0}\} \cup \{(\mathbf{print}(\mathcal{S}_{i_0}), q_u, 1)\}$,
- $\delta'(\mathbf{x}_{q_u,1}) = \delta(\mathbf{x}_{q_u,j_k}) = 1$, and $\delta'(\mathbf{x}_{q_u,j_k}) = \delta(\mathbf{x}_{q_u,1}) = 0$, and
- $\delta'(\mathbf{x}) = \delta(\mathbf{x})$ for all other counters $\mathbf{x}$.

Hence, by defining $\lambda'_i = (\mathcal{S}'_i, v'_i)$ where:

- $\mathcal{S}'_i = \mathcal{S}_i \setminus \{\mathcal{S}_i\} \cup \{(\mathbf{print}(\mathcal{S}_i), q_u, 1)\}$ (where $\mathcal{S}_i$ is the summary labeled by (q_u, j_k) appearing in $\mathcal{S}_i$),
- $v'_i(\mathbf{x}_{q_u,1}) = v_i(\mathbf{x}_{q_u,j_k})$, and $v'_i(\mathbf{x}_{q_u,j_k}) = v_i(\mathbf{x}_{q_u,1}) = 0$, and
- $v'_i(\mathbf{x}) = v_i(\mathbf{x})$ for all other counters $\mathbf{x}$,

we can easily rebuild a run of the VASS from λ_{i-1} to λ'_m (the only difference is index j_k which is now index 1). As $\mathcal{S}'_m = \{\mathcal{S}_f\}$ it respects the claim. $\square$

Hence, checking for the reachability of one configuration in $\bigcup_{n \in \mathbb{N}_{>0}} \llbracket (\{\mathcal{S}_f\}, v_n) \rrbracket$ where $v_n(\mathbf{x}_{q_u,1}) = n$, and $v_n(\mathbf{x}) = 0$ for all $\mathbf{x} \neq \mathbf{x}_{q_u,1}$ and $n \in \mathbb{N}_{>0}$, suffices to decide SYNCHRO.

We are now ready to introduce V_P^{SYN} which is V_P enriched with a decrement self loop on counter $\mathbf{x}_{q_u,1}$ in state $\{\mathcal{S}_f\}$. Formally, if $V_P = (\text{Loc}, s_0, X, \Delta)$, then $V_P^{\text{SYN}} = (\text{Loc}, s_0, X, \Delta \cup \{(\{\mathcal{S}_f\}, \delta_{q_u,1}^-, \{\mathcal{S}_f\})\})$ where $\delta_{q_u,1}^-(\mathbf{x}_{q_u,1}) = -1$ and $\delta_{q_u,1}^-(\mathbf{x}) = 0$ for all other counters. It is pictured in Figure 7.5.

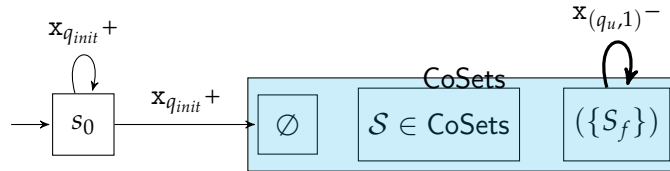


Figure 7.5: The structure of the VASS V_P^{SYN} .

Thanks to the two previous lemmas, we get the following result.

Lemma 7.7. There exist $C_{init} \in \mathcal{C}_{init}$, $C \in \mathcal{C}_f$ such that $C_{init} \rightarrow^* C$ if and only if $(s_0, \mathbf{0}_X) \rightsquigarrow_{V_P^{\text{SYN}}}^* (\{\mathcal{S}_f\}, \mathbf{0}_X)$.

Proof. • If there exist $C_{init} \in \mathcal{C}_{init}$, $C \in \mathcal{C}_f$ such that $C_{init} \rightarrow^* C$, then from Corollary 7.1, there exists a well-formed execution ρ from C_{init} to $\mathcal{C}_f$. Let $K = |C_{init}|$ and $n = \text{size}(\rho) - 1$. First, consider the sequence $(s_0, \mathbf{0}_X) \rightsquigarrow_{V_P^{\text{SYN}}}^* (s_0, v'_0) \rightsquigarrow_{V_P^{\text{SYN}}} (\emptyset, v_0)$ where $v_0(\mathbf{x}_{q_{init}}) = K - 1$, $v'_0(\mathbf{x}_{q_{init}}) = K$ and for all other counters $\mathbf{x}$, $v_0(\mathbf{x}) = v'_0(\mathbf{x}) = 0$. Observe that, since $q_{init} \in Q_A$, $(\emptyset, v_0) \in \mathbf{repr}(\rho, 0)$. From the VASS

V_P^{SYN} 's construction, every run of V_P is also a run of V_P^{SYN} . Hence, by applying inductively lemma 7.4, we get that $(\emptyset, v_0) \rightsquigarrow_{V_P^{\text{SYN}}}^* (\mathcal{S}_1, v_1) \rightsquigarrow_{V_P^{\text{SYN}}}^* \dots \rightsquigarrow_{V_P^{\text{SYN}}}^* (\mathcal{S}_n, v_n)$ with $(\mathcal{S}_n, v_n) \in \mathbf{repr}(\rho, n)$. Thanks to Lemma 7.6, we get that there exists $\lambda' = (\{S_f\}, v')$ such that $(\emptyset, v_0) \rightsquigarrow_{V_P^{\text{SYN}}}^* \lambda'$ and $v'(x) = 0$ for all $x \neq x_{qu,1}$. As we added a transition from $\{S_f\}$ to itself, by letting $x = v'(x_{qu,1})$, and decrementing counter $x_{qu,1}$, we get that $\lambda' (\rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_{x_{qu,1},1}^-})^x (\{S_f\}, \mathbf{0}_X)^{12}$. Hence, we have built the following execution:

$$(s_0, \mathbf{0}_X) \rightsquigarrow_{V_P^{\text{SYN}}}^* (s_0, v'_0) \rightsquigarrow_{V_P^{\text{SYN}}} (\emptyset, v_0) \rightsquigarrow_{V_P^{\text{SYN}}}^* (\{S_f\}, v') (\rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_{x_{qu,1},1}^-})^x (\{S_f\}, \mathbf{0}_X).$$

- Assume now that $(s_0, \mathbf{0}_X) \rightsquigarrow_{V_P^{\text{SYN}}}^* (\{S_f\}, \mathbf{0}_X)$. From V_P^{SYN} 's construction, the execution is of the form:

$$(s_0, \mathbf{0}_X) \rightsquigarrow_{V_P^{\text{SYN}}}^* (s_0, v'_0) \rightsquigarrow_{V_P^{\text{SYN}}} (\emptyset, v_0) \rightsquigarrow_{V_P^{\text{SYN}}}^* (\{S_f\}, \mathbf{0}_X)$$

where $v_0(x_{qu,1}) = n$ for some $n \in \mathbb{N}_{>0}$ and $v_0(x) = 0$ for all other counter x . We now rewrite the run between $(\emptyset, v_0)$ and $(\{S_f\}, \mathbf{0}_X)$ as

$$(\mathcal{S}_0, v_0) \rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_1} (\mathcal{S}_1, v_1) \rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_2} \dots \rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_k} (\mathcal{S}_k, v_k) = (\{S_f\}, \mathbf{0}_X).$$

We transform the run so that all decrements of counter $x_{qu,1}$ with transition $(\{S_f\}, \delta_{qu,1}^-, \{S_f\})$ appear at the end of the run. This is possible as $\mathcal{S}_k = \{S_f\}$

and if $(\{S_f\}, v_1) \rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_{qu,1}^-} (\{S_f\}, v_2)$ then $v_2 \leq v_1$ and, for all δ such that $(\{S_f\}, v_2) \rightsquigarrow_{V_P^{\text{SYN}}}^{\delta} (\mathcal{S}, v'_2)$, we have: $(\{S_f\}, v_1) \rightsquigarrow_{V_P^{\text{SYN}}}^{\delta} (\mathcal{S}, v'_1)$ with $v'_1(x_{qu,1}) = v'_2(x_{qu,1}) + 1$ and $v'_1(x) = v'_2(x)$ for all other counter x . We finally get a run of the form:

$$(\mathcal{S}_0, v_0) \rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_1} (\mathcal{S}_1, v_1) \rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_2} \dots \rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_j} (\{S_f\}, v_j) \rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_{qu,1}^-} \rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_{qu,1}^-} \dots \rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_{qu,1}^-} (\{S_f\}, \mathbf{0}_X)$$

where, for all $1 \leq i \leq j$, $\delta_i \in \Delta$. Hence, the run between $(\mathcal{S}_0, v_0)$ and $(\{S_f\}, v_j)$ is also a run in V_P . Furthermore, as $(\{S_f\}, v_j) \rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_{qu,1}^-} \rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_{qu,1}^-} \dots \rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_{qu,1}^-} (\{S_f\}, \mathbf{0}_X)$, v_j is of the form: $v_j(x) = 0$ for all $x \neq x_{qu,1}$.

Define C as the initial configuration in $\mathcal{C}_{init}$ with n processes (i.e. $|C| = n$). Trivially, $C \in \llbracket (\emptyset, v_0) \rrbracket$ (recall that $\mathcal{S}_0 = \emptyset$) and with Lemma 7.3 and a simple induction, we get that there exists $C' \in \llbracket (\{S_f\}, v_j) \rrbracket$ such that $C \rightarrow^* C'$. In particular, $|C'| = |C|$ and so $v_j(x_{qu,1}) = |C'| = |C| = n > 0$. Using Lemma 7.5, we get that $C \in \mathcal{C}_f$, concluding the proof. $\square$

12 $(\rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_{x_{qu,1},1}^-})^x$ stands for relation $\rightsquigarrow_{V_P^{\text{SYN}}}^{\delta_{x_{qu,1},1}^-}$ applied x times.

The reduction proves that SYNCHRO is at most as hard as the reachability problem for VASS which is proven to be decidable (Theorem 3.3, [LS19]), leading to the main theorem of this section.

Theorem 7.1. *SYNCHRO restricted to Wait-Only broadcast protocols is decidable.*

7.4.2 Lower Bound

In this subsection we reduce the reachability problem for **test-free Counter Machines** (shortly test-free CM) to SYNCHRO. Remember that in Chapter 3, we explained why the reachability problem for test-free CM has the same complexity that the one for VASS, i. e., Ackermann-complete Theorem 3.3, [LS19; CO21; Ler21]. We fix a test-free CM $CM = (Loc, \ell_{in}, X, \Delta)$ and a final location $\ell_f \in Loc$.

We explain how to build a protocol P_{CM} that simulates CM: it is depicted in Figure 7.6.

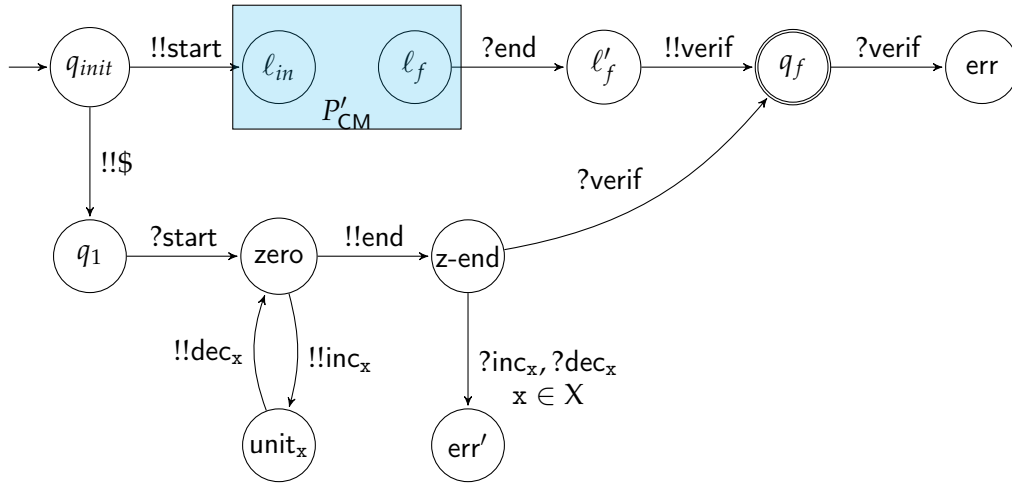


Figure 7.6: Protocol P_{CM} associated to a test-free CM CM .

The states $zero$ and $\{unit_x \mid x \in X\}$ represent the evolution of the different counters of the test-free CM while the blue box P'_{CM} describes the evolution of CM with respect to its locations. Formally, P'_{CM} consists in a set of states $Q_{CM} = Loc \cup \{\ominus\}$ and a set of transitions

$$T_{CM} = \{(\ell, ?inc_x, \ell') \mid (\ell, x+, \ell') \in \Delta\} \cup \\ \{(\ell, ?dec_x, \ell') \mid (\ell, x-, \ell') \in \Delta\} \cup \\ \{(\ell, ?m, \ominus) \mid m \in \{inc_x, dec_x \mid x \in X\}\}$$

Some transitions of T_{CM} are depicted in Figure 7.7.

Before going into the technical details of the proof, we give the ideas that prove that the reduction described above is correct. Assume first that $(\ell_{in}, \mathbf{0}) \rightsquigarrow^* (\ell_f, \mathbf{0})$ in CM. Let $K \in \mathbb{N}$ be the maximum value $\sum_{x \in X} v(x)$, where v is the counter valuation, encountered during the execution of CM. We choose C_{init} such that $|C_{init}| = K + 1$. The execution of the protocol goes as follows. All processes except for one (called the leader) broadcast a (dummy message) $\$$. The remaining leader process broadcasts start and reaches ℓ_{in} , causing all processes in q_1 to reach zero. These processes will simulate the counter values throughout the execution of the test-free CM, while the leader at ℓ_{in} simulates the machine itself by traversing its locations



Figure 7.7: Part of P'_{CM} that simulates respectively the transition $(\ell, x+, \ell') \in \Delta$ at the left, and $(\ell'', x-, \ell''') \in \Delta$ at the right.

in P'_{CM} . The value of the counter x in the different steps of the execution of the VASS is represented by the number of processes in unit_x . To simulate an increment of x in CM from (ℓ_i, v_i) to (ℓ_{i+1}, v_{i+1}) , one of the processes in state zero sends the message inc_x . This message is received by the leader, making it transitioning to ℓ_{i+1} . If the protocol's configuration correctly represented (ℓ_i, v_i) , then the new configuration faithfully represents (ℓ_{i+1}, v_{i+1}) . Once the simulation of the execution of the CM is finished, the leader process is in ℓ_f , and other processes remain in zero. At this point, all processes in zero broadcast the message end, and the leader, now on ℓ'_f , broadcasts the message verif , which gathers everyone in q_f .

To prove the other direction, we need to prove that the processes in the protocol cannot cheat in their simulation of the test-free CM.

First observe that to gather all the processes in q_f , an execution of the protocol needs to see the broadcast of verif exactly once (a second broadcast of this message will necessarily send the first process to have broadcast it in the state err). This enforces that in the execution of the protocol, exactly one process will evolve in the P_{CM} part, simulating correctly the states of the machine.

Second, observe that the modification of the counters must follow the transitions of the test-free CM. The processes cannot modify a counter in a way that is not allowed by the machine: the process in the states of P'_{CM} would receive the message and would be sent to the losing state $\odot$.

Thirdly, as the end of the simulation (when the leader reaches ℓ_f), all processes must be in state zero: if it is not the case, then one process (called the helper process) broadcasts end, the message is received by the leader, which reaches ℓ'_f . Then, all processes (in states zero and unit_x) need to reach state z-end before the unique broadcast of verif , as otherwise, they can never receive the message and reach q_f . Processes in state zero can safely join state z-end as message end is not received by anyone at that point. However, when a process in state unit_x joins state z-end, it has to broadcast message dec_x , causing processes in state z-end (including the helper process) to receive the message and reach state err' . As a consequence, the helper process can no longer receive verif and reach q_f , preventing the execution to put everyone in q_f . Hence, the simulation correctly ends in a configuration where all processes but the leader are on state zero, faithfully simulating counters' values.

We are then able to prove that this reduction is sound and complete.

Lemma 7.8. There exist $C_0 \in \mathcal{C}_{init}$ and $C_f \in \mathcal{C}$ such that $C_0 \rightarrow^* C_f$ in P_{CM} and $C_f(e) = q_f$ for all $e \in [1, |C_f|]$ if and only if $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell_f, \mathbf{0}_X)$ in CM.

Proof. We first show completeness, i. e. if $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell_f, \mathbf{0}_X)$, then there exist $C_0 \in \mathcal{C}_{init}$ and $C_f \in \mathcal{C}$ such that $C_0 \rightarrow^* C_f$ and $C_f(e) = q_f$ for all $e \in [1, |C_f|]$. Let the run of the VASS be $(\ell_0, v_0) \rightsquigarrow (\ell_1, v_1) \rightsquigarrow \dots \rightsquigarrow (\ell_n, v_n)$ with $v_0 = v_n = \mathbf{0}_X$ and $\ell_0 = \ell_{in}$, $\ell_n = \ell_f$. We denote $K = \max_{0 \leq i \leq n} \|v_i\|_1$ where $\|v_i\|_1 = \sum_{x \in X} v_i(x)$.

We build an execution of the protocol starting from the initial configuration C_{init} with $K + 1$ processes. The first part of the execution goes as follows.

$$C_{init} = C_0 \xrightarrow{1, (q_{init}, !!\$, q_1)} C_1 \xrightarrow{2, (q_{init}, !!\$, q_1)} \dots \xrightarrow{K, (q_{init}, !!\$, q_1)} C_K \xrightarrow{K+1, (q_{init}, !!\text{start}, \ell_{in})} C_{K+1}.$$

Observe that $C_{K+1}(j) = \text{zero}$ for all $1 \leq j \leq K$ and $C_{K+1}(K+1) = \ell_{in}$. We rename C_{K+1} to C'_0 .

For all $0 \leq i \leq n$, we define

$$\begin{aligned} \mathbb{C}(\ell_i, v_i) &= \{C \in \mathcal{C} \mid C(K+1) = \ell_i, \text{ and} \\ &\quad C([1, K]) \subseteq \{\text{zero}, \text{unit}_x \mid x \in X\}, \text{ and} \\ &\quad |C^{-1}(\text{unit}_x)| = v_i(x) \text{ for all } x \in X\} \end{aligned}$$

as the set of protocol configurations corresponding to (ℓ_i, v_i) (recall that $\ell_0 = \ell_{in}$). We show by induction on $i \in [0, n]$ that, for all $C \in \mathbb{C}(\ell_i, v_i)$, $C'_0 \rightarrow^* C$.

For $i = 0$, observe that $\mathbb{C}(\ell_0, v_0) = \{C'_0\}$ hence the property trivially holds. Let $0 \leq i < n$ and $\delta_i = (\ell_i, \text{op}_i, \ell_{i+1}) \in \Delta$ such that $(\ell_i, v_i) \xrightarrow{\delta_i} (\ell_{i+1}, v_{i+1})$. Let now $C \in \mathbb{C}(\ell_{i+1}, v_{i+1})$.

- *Case $\text{op}_i = x+$ for a counter $x \in X$.* Then $v_{i+1}(x) = v_i(x) + 1 > 0$, and since $C \in \mathbb{C}(\ell_{i+1}, v_{i+1})$, there exists $1 \leq e \leq K$ such that $C(e) = \text{unit}_x$. Let then $C_i \in \mathcal{C}$ such that $C_i(K+1) = \ell_i$, $C_i(e) = \text{zero}$ and for all other process $1 \leq j \leq K$, $C_i(j) = C(j)$. By definition of the protocol, $(\ell_i, ?\text{inc}_x, \ell_{i+1}) \in T$, hence $C_i \xrightarrow{e, (\text{zero}, !!\text{inc}_x, \text{unit}_x)} C$. Furthermore, it is easy to see that, since $|C_i^{-1}(\text{unit}_x)| = |C^{-1}(\text{unit}_x)| - 1 = v_{i+1}(x) - 1 = v_i(x)$, and for all other counter y , $|C_i^{-1}(\text{unit}_y)| = |C^{-1}(\text{unit}_y)| = v_{i+1}(y) = v_i(y)$, $C_i \in \mathbb{C}(\ell_i, v_i)$. Hence, by induction hypothesis, $C'_0 \rightarrow^* C_i \rightarrow C$.
- *Case $\text{op}_i = x-$ for a counter $x \in X$.* Then $v_{i+1}(x) = v_i(x) - 1 < K$ and there exists a process e such that $C(e) = \text{zero}$. Let $C_i \in \mathcal{C}$ such that $C_i(K+1) = \ell_i$, $C_i(e) = \text{unit}_x$ and for all other process $1 \leq j \leq K$, $C_i(j) = C(j)$. By definition of the protocol, $(\ell_i, ?\text{dec}_x, \ell_{i+1}) \in T$, hence $C \xrightarrow{e, (\text{unit}_x, !!\text{dec}_x, \text{zero})} C_i$. Furthermore, it is easy to see that, since $|C_i^{-1}(\text{unit}_x)| = |C^{-1}(\text{unit}_x)| + 1 = v_{i+1}(x) + 1 = v_i(x)$, and for all other counter y , $|C_i^{-1}(\text{unit}_y)| = |C^{-1}(\text{unit}_y)| = v_{i+1}(y) = v_i(y)$, $C_i \in \mathbb{C}(\ell_i, v_i)$. Hence, by induction hypothesis, $C'_0 \rightarrow^* C_i \rightarrow C$.

Then, since $\mathbb{C}(\ell_n, v_n) = \{C'_n\}$ with $C'_n(K+1) = \ell_n = \ell_f$, and $C'_n(j) = \text{zero}$ for all $1 \leq j \leq K$, we have that $C'_0 \rightarrow^* C'_n$. The execution ends in the following way:

$$\begin{aligned} C'_n &\xrightarrow{1, (\text{zero}, !!\text{end}, z\text{-end})} C'_{n+1} \xrightarrow{2, (\text{zero}, !!\text{end}, z\text{-end})} \dots \xrightarrow{K, (\text{zero}, !!\text{end}, z\text{-end})} \\ &\quad C'_{n+K} \xrightarrow{K+1, (s'_f, !!\text{verif}, q_f)} C'_{n+K+1} \end{aligned}$$

One can observe that for all process e , $C'_{n+K+1}(e) = q_f$ which concludes the completeness proof.

We now prove the other direction, i.e. soundness of the reduction. It relies on the following intermediate lemma.

Lemma 7.9. Let $C_0 \rightarrow^* C_f$ such that $C_0 \in \mathcal{C}_{init}$ and for all $e \in [1, |C_f|]$, $C_f(e) = q_f$. Then, the message `verif` is broadcast exactly once.

Proof. We let $C_0 \rightarrow C_1 \rightarrow \dots \rightarrow C_n = C_f$ be the execution that gathers everyone in q_f . The broadcast of `verif` can be done only with the transition $(\ell'_f, !!\text{verif}, q_f)$.

First, observe that there must exist an index $0 \leq j \leq n$ such that $C_j \xrightarrow{e_1, (\ell'_f, !!\text{verif}, q_f)} C_{j+1}$; otherwise, q_f is unreachable. Let j_1 denote the first such index. If there exists an index $j_2 > j_1$ such that $C_{j_2} \xrightarrow{e_2, (\ell'_f, !!\text{verif}, q_f)} C_{j_2+1}$, then, by construction of the protocol, $e_2 \neq e_1$ (there is no way for e_1 to go back to ℓ'_f), and $C_{j_2+1}(e_1) = \text{err}$. Then, $C_n(e_1) = \text{err}$ which contradicts the definition of the execution. $\square$

We let $C_0 \rightarrow C_1 \rightarrow \dots \rightarrow C_n = C_f$ be the execution that gathers all processes in q_f .

From Lemma 7.9, there exists a unique index j such that $C_j \xrightarrow{e_1, (\ell'_f, !!\text{verif}, q_f)} C_{j+1}$. Hence, there exists a unique process e_1 evolving in P_{CM} ; indeed, any process in P_{CM} needs to broadcast `verif` in order to reach q_f , and the only way to broadcast `verif` is to go through P_{CM} . So there exists a unique index $j_1 < j$ such that $C_{j_1} \xrightarrow{e_1, (q_{init}, !!\text{start}, \ell_{in})} C_{j_1+1}$. Hence, all other processes receive the message `start` in C_{j_1} ; otherwise they will never reach q_f . As a consequence, for each process $e \neq e_1$, $C_{j_1+1}(e) = \text{zero}$.

We now prove that there exists an index $j_1 < j_2 < j$ such that $C_{j_2}(e_1) = \ell_f$ and for any other process $e \neq e_1$, $C_{j_2}(e) = \text{zero}$. First note that, unless $\ell_{in} = \ell_f$, there must be at least one process e such that $C_{j_1+1}(e) = \text{zero}$. Otherwise, no process ever broadcasts a message, and e would not be able to reach ℓ_f and then ℓ'_f .

We claim that j_2 is the first index such that $C_{j_2} \xrightarrow{e_2, (\text{zero}, !!\text{end}, \text{z-end})} C_{j_2+1}$ for some process e_2 . This transition must occur during the execution, otherwise, processes other than e_1 would be unable to reach q_f during the unique broadcast of `verif` in the execution. First, we prove that $C_{j_2}(e_1) = \ell_f$. If it is not the case, it needs to receive some messages in the set $\{\text{inc}_x, \text{dec}_x \mid x \in X\}$ before broadcasting message `!!verif`. Observe from Lemma 7.9, there is exactly one broadcast of `verif`, and so the sending of one message in $\{\text{inc}_x, \text{dec}_x \mid x \in X\}$ is received by e_2 which goes to state err' . As a consequence, it never reaches q_f which contradicts definition of C_f . Hence, $C_{j_2}(e_1) = \ell_f$.

We prove now that for all other process $e \neq e_1$, $C_{j_2}(e) = \text{zero}$. If it is not the case, since `end` is not broadcast before, there is one process e' such that $C_{j_2}(e') \in \{\text{unit}_x \mid x \in X\}$. In that case, for the same reason as before, it will need to broadcast a message from $\{\text{dec}_x \mid x \in X\}$ while e_2 is in `z-end`. This would lead e_2 to err' and contradicts the definition of C_f .

Hence, $C_{j_2}(e_1) = \ell_f$ and for all other process $e \neq e_1$, $C_{j_2}(e) = \text{zero}$.

We rename the configurations visited during this part of the execution to $C_{j_1+1} = C'_0 \rightarrow C'_1 \rightarrow \dots \rightarrow C'_k = C_{j_2}$, with $k = j_2 - j_1 - 1$. First observe that since $C'_0(e_1) = \ell_{in}$, and $C'_k(e_1) = \ell_f$, the structure of the protocol implies that $C'_i(e_1) \in \text{Loc}$ for all $0 \leq i \leq k$. Similarly, since, for all $e \neq e_1$, $C'_0(e) = C'_k(e) = \text{zero}$, the structure of the protocol implies that $C'_i(e) \in \{\text{zero}, \text{unit}_x \mid x \in X\}$, for all $e \neq e_1$, and for all $0 \leq i \leq k$. We build

now, by induction on $0 \leq i \leq k$, an execution $(\ell_0, v_0) \rightsquigarrow (\ell_1, v_1) \rightsquigarrow \dots \rightsquigarrow (\ell_k, v_k)$ of the test-free CM such that $\ell_i = C'_i(e_1)$, and $v_i(x) = |C'^{-1}_i(\text{unit}_x)|$ for all $x \in X$.

With $v_0 = \mathbf{0}_X$, the definition of C'_0 allows to conclude immediately for the case $i = 0$.

Let now $0 \leq i < k$ and assume the property true. Let $C'_i \xrightarrow{e^t} C'_{i+1}$ the transition taken in the execution of the protocol. Since, by construction of the protocol, $C'_i(e_1) \in Q_W$, we know that $e \neq e_1$, and then $t \in \{(\text{zero}, !!\text{inc}_x, \text{unit}_x), (\text{unit}_x, !!\text{dec}_x, \text{zero}) \mid x \in X\}$. We distinguish two cases:

- $t = (\text{zero}, !!\text{inc}_x, \text{unit}_x)$ for some $x \in X$. In that case, by definition of the protocol, $(\ell_i, x+, \ell_{i+1}) \in \Delta$. Furthermore, $|C'^{-1}_{i+1}(\text{unit}_x)| = |C'^{-1}_i(\text{unit}_x)| + 1 = v_i(x) + 1$ by induction hypothesis, and for all other counters y , $|C'^{-1}_{i+1}(\text{unit}_y)| = |C'^{-1}_i(\text{unit}_y)| = v_i(y)$, by induction hypothesis. Hence, $(\ell_i, v_i) \rightsquigarrow (\ell_{i+1}, v_{i+1})$ with $v_{i+1}(x) = |C'^{-1}_{i+1}(\text{unit}_x)|$ for all $x \in X$.
- $t = (\text{unit}_x, !!\text{dec}_x, \text{zero})$ for some $x \in X$. In that case, by definition of the protocol, $(\ell_i, x-, \ell_{i+1}) \in \Delta$. Furthermore, $|C'^{-1}_{i+1}(\text{unit}_x)| = |C'^{-1}_i(\text{unit}_x)| - 1 = v_i(x) - 1 \geq 0$, by induction hypothesis. For all other counters x' , $|C'^{-1}_{i+1}(\text{unit}_{x'})| = |C'^{-1}_i(\text{unit}_{x'})| = v_i(x')$, by induction hypothesis. Hence, $(\ell_i, v_i) \rightsquigarrow (\ell_{i+1}, v_{i+1})$ where $v_{i+1}(x) = |C'^{-1}_{i+1}(\text{unit}_x)|$ for all $x \in X$.

Hence, $(\ell_{in}, v_0) \rightsquigarrow^* (\ell_k, v_k)$ with $\ell_k = C'_k(e_1) = C_{j_2}(e_1) = \ell_f$ and $v_k(x) = |C'^{-1}_k(\text{unit}_x)| = 0$ for all counters x , which concludes the proof. $\square$

Thanks to Theorem 7.1 and the above lemma, we then obtain the following theorem.

Theorem 7.2. *SYNCHRO with Wait-Only protocols is Ackermann-complete.*

7.5 SYNCHRONIZATION IS EASIER WHEN THE FINAL STATE IS AN ACTION STATE

In this section, we prove that when $q_f \in Q_A$, the SYNCHRO problem becomes EXPSPACE-complete. For the upper bound, we show that SYNCHRO can be reduced to a simpler problem on VASS. The lower bound comes from the coverability problem on VASS.

7.5.1 Upper bound

We show that SYNCHRO when $q_f \in Q_A$ can be reduced to the mutual reachability problem on VASS. We will also mention the mutual reachability problem for VAS (Vector Addition Systems). They are both defined as follows.

- Given a VASS $V = (\text{Loc}, \ell_{in}, X, \Delta)$ and a location $\ell_f \in \text{Loc}$, **MUTUREACH[VASS]** asks if $(\ell_{in}, \mathbf{0}_X) \rightsquigarrow^* (\ell_f, \mathbf{0}_X)$ and $(\ell_f, \mathbf{0}_X) \rightsquigarrow^* (\ell_{in}, \mathbf{0}_X)$?
- Given a VAS $V = (\Delta, v_{\text{init}})$ of dimension $d \in \mathbb{N}$, and a vector v , **MUTUREACH[VAS]** asks if $v_{\text{init}} \rightsquigarrow^* v$ and $v \rightsquigarrow^* v_{\text{init}}$?

7.5.1.1 About mutual reachability complexity

In [Ler11], Leroux proved that $\text{MUTUREACH}[\text{VAS}]$ is ExpSPACE -complete. Using a VASS-to-VAS transformation (as for instance the one presented in [HP79]) we get that $\text{MUTUREACH}[\text{VASS}]$ is also ExpSPACE -complete. To reduce our problem to $\text{MUTUREACH}[\text{VASS}]$, we will define a VASS V_P^{MR} from V_P (and so from our protocol P), so that V_P^{MR} together with a location ℓ_f is a positive instance of MUTUREACH if and only if P together with q_f is one of SYNCHRO . However, as we shall see, the VASS V_P^{MR} has a doubly exponential number of locations in the size of the protocol P (and so requires an exponential space in the size of P), and so Leroux's result is not enough, as it would give us a 2ExpSPACE membership.

Hence, one needs to be precise, and we will use Leroux's Corollary 10.6 [Ler11]. We recall that $\|v\| = \max_{1 \leq i \leq d} |v(i)|$.

Lemma 7.10 (Corollary 10.6 of [Ler11]). Given a VAS $V = (\Delta, v_{\text{init}})$ of dimension $d \in \mathbb{N}$, and a vector v , there exist two runs from v_{init} to v and from v to v_{init} if and only if there exist two runs whose lengths are bounded by $17d^2x^{15d^{d+2}}$ where

$$x = (1 + 2\|\Delta\|)(1 + \max(\|v_{\text{init}}\|, \|v\|)).$$

Given a VASS $V = (\text{Loc}, X, \ell, \Delta)$, we can transform it into a VAS $V' = (\Delta', v_{\text{init}})$ of dimension d , preserving the reachability properties and using transformation presented in [HP79], such that $d = |X| + 3$ and $\|\Delta'\| = \max((|\text{Loc}| + 1)^2, \|\Delta\|)$. The mutual reachability problem in the VASS between $(\ell_0, \mathbf{0}_X)$ and $(\ell_f, \mathbf{0}_X)$ is now equivalent to ask for mutual reachability in the VAS between vectors $v_0 = (1, (|\text{Loc}| + 1) \times |\text{Loc}|, 0, \dots)$ and $v_f = (|\text{Loc}|, |\text{Loc}| + 1, 0, \dots)$.

We hence get the following result.

Corollary 7.2. Given a VASS $V = (\text{Loc}, X, \ell, \Delta)$, and a location ℓ_f , there exist two runs from $(\ell_{\text{in}}, \mathbf{0}_X)$ to $(\ell_f, \mathbf{0}_X)$ and from $(\ell_f, \mathbf{0}_X)$ to $(\ell_{\text{in}}, \mathbf{0}_X)$ if and only if there exist two runs (one from $(\ell_{\text{in}}, \mathbf{0}_X)$ to $(\ell_f, \mathbf{0}_X)$, and one from $(\ell_f, \mathbf{0}_X)$ to $(\ell_{\text{in}}, \mathbf{0}_X)$) whose lengths are bounded by $17(|X| + 3)^2x^{15(|X|+3)^{|X|+5}}$ where

$$x = (1 + \max(\|\Delta\|, (|\text{Loc}| + 1)^2))(1 + (|\text{Loc}| + 1)^2).$$

7.5.1.2 Our reduction

Given a Wait-Only broadcast protocol $P = (Q, \Sigma, q_{\text{init}}, T)$ and a target state $q_f \in Q_A$, we explain how to transform the VASS V_P presented in Section 7.3.4 into a VASS where mutual reachability is equivalent to SYNCHRO . We let again $V_P = (\text{Loc}, X, s_0, \Delta)$ be the VASS defined in Section 7.3.4, and we reuse the following set of configurations:

$$\mathcal{C}_f = \{C \in \mathcal{C} \mid C(e) = q_f \text{ for all } e \in [1, |C|]\}$$

Observe that the S-configurations corresponding to the set $\mathcal{C}_f$ are of the form $(\emptyset, v)$, where the only non-zero counters are x_{q_f} and $x_{(q_f, k)}$ for some $k \in [1, |Q_W| + 1]$. Since from location $\emptyset$, the values of counters $x_{(q_f, k)}$ can be easily transferred to x_{q_f} , we can reduce to the case of configurations $(\emptyset, v)$ where $v(x) = 0$ for all $x \neq x_{q_f}$. This leads to the following lemma.

Lemma 7.11. There exists $\rho = C_0 \dots C_k \in \text{Exec}[P]$ with $C_k \in \mathcal{C}_f$ if and only if there exists v such that $(s_0, \mathbf{0}_X) \rightsquigarrow_{V_P}^* (\emptyset, v)$ with $v(x) = 0$ for all $x \neq x_{q_f}$.

Proof. • Assume that there exists an execution between C_{init} and C_f with $C_f \in \mathcal{C}_f$, thanks to Corollary 7.1, there exists a *well-formed* execution $\rho = C_0 C_1 \dots C_k$ with $C_0 = C_{init}$ and $C_k = C_f$. Denote $K = |C_0|$, as in Lemma 7.7's proof, consider the beginning of execution: $(s_0, \mathbf{0}_X) \rightsquigarrow \dots \rightsquigarrow (s_0, v'_0) \rightsquigarrow (\emptyset, v_0)$ with $v_0(x_{q_{init}}) = K$ and $v_0(x) = 0$ for all other counters x . It holds that $(\emptyset, v_0) \in \mathbf{repr}(\rho, 0)$. Applying successively Lemma 7.4, we get that $(\emptyset, v_0) \rightsquigarrow^* \lambda$ with $\lambda \in \mathbf{repr}(\rho, k)$. Observe that, as $C_k^{-1}(Q_W) = \emptyset$ and $C_k^{-1}(Q_A \setminus \{q_f\}) = \emptyset$, we have that $\mathbf{repr}(\rho, k) = \{(\emptyset, v)\}$ where $v(x_{q_f}) = |C_k^{-1}(q_f)| = K$ and $v(x) = 0$ for all other counter x .

• Now assume that there exists v such that $(s_0, \mathbf{0}_X) \rightsquigarrow_{V_P}^* (\emptyset, v)$ with $v(x) = 0$ for all $x \neq x_{q_f}$. From V_P 's construction, the execution is of the form: $(s_0, \mathbf{0}_X) \rightsquigarrow_{V_P}^* (s_0, v'_0) \rightsquigarrow_{V_P} (\emptyset, v_0) \rightsquigarrow_{V_P}^* (\emptyset, v)$ where $v_0(x_{q_{init}}) = n$ for some $n \in \mathbb{N}_{>0}$ and $v_0(x) = 0$ for all other counter x . Consider C_{init} the initial configuration with $v_0(x_{q_{init}})$ processes (note that $v_0(x_{q_{init}}) > 0$ as $v_0(x_{q_{init}}) = v'_0(x_{q_{init}}) + 1$). It holds that $C_{init} \in \llbracket (\emptyset, v_0) \rrbracket$. Applying successively Lemma 7.3, we get that there exists $C \in \llbracket (\emptyset, v) \rrbracket$, and $C_{init} \rightarrow^* C$. By definition of $\llbracket \cdot \rrbracket$, we get that $C \in \mathcal{C}_f$. □

We now enrich the VASS so that:

- (1) counter x_{q_f} 's value can be decremented from state $\emptyset$ in order to reach $(\emptyset, \mathbf{0}_X)$: we add a transition $(\emptyset, \delta_f, \emptyset)$ where $\delta_f(x_{q_f}) = -1$ and $\delta_f(x) = 0$ for all other counter x ;
- (2) there is a *trivial* run from $(\emptyset, \mathbf{0}_X)$ to $(s_0, \mathbf{0}_X)$: we add a transition $(\emptyset, \mathbf{0}_X, s_0)$.

Item (1) allows us to build execution reaching $(\emptyset, \mathbf{0}_X)$ from any S-configuration $(\emptyset, v)$ with a vector v s.t. $v(x) = 0$ for all $x \neq x_{q_f}$. Item (2) allows the VASS to become a positive instance of MUTUREACH (by taking $\ell_f = \emptyset$).

Formally:

$$V_P^{\text{MR}} = (\text{Loc}, X, s_0, \Delta \cup \{(\emptyset, \delta_f, \emptyset), (\emptyset, \mathbf{0}_X, s_0)\})$$

where $\delta_f(x_{q_f}) = -1$ and $\delta_f(x) = 0$ for all other counter x . The VASS V_P^{MR} is pictured in Figure 7.8.

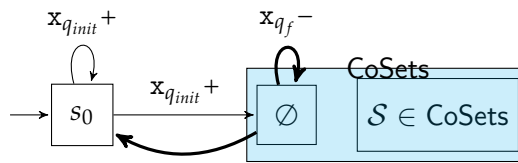


Figure 7.8: The structure of the VASS V_P^{MR} .

What could go wrong? Thanks to a reasoning similar to the one used in the proof of Lemma 7.7, Item (1) does not introduce any new undesired behaviors: decrementing a counter does not enable new executions. However, Item (2) would now allow an execution from $(s_0, \mathbf{0}_X)$ to $(\emptyset, \mathbf{0}_X)$ to revisit s_0 multiple times, incrementing $x_{q_{init}}$ at will. We explain why this is not problematic, and why it would be, if q_f were a waiting state.

Why is it okay to restart the simulation at will? The key observation is that the ability to return from $\emptyset$ to s_0 during a run from $(s_0, \mathbf{0}_X)$ to $(\emptyset, \mathbf{0}_X)$ only adds processes in q_{init} , and then resumes the simulation at precisely the point where it was left, namely in $\emptyset$. Since $q_{init} \in Q_A$, adding such processes in q_{init} is harmless: we can reconstruct an execution

where these additional processes were already present in q_{init} from the beginning, staying idle throughout. As q_{init} belongs to Q_A , these processes do not receive any messages and can safely remain, regardless of what happens elsewhere.

When restarting from $\emptyset$, no information is lost: everything relevant is encoded in the counters. The counters x_{q_a} track the number of processes in state $q_a \in Q_A$. Non-null counters $x_{q_a,k}$ represent processes either in waiting zones or in state q_a (if the counter was not properly emptied). As we left the simulation in location $\emptyset$ (where no waiting states are occupied), the processes counted by $x_{q_a,k}$ are in fact located in $q_a \in Q_A$.

In contrast, if q_f were a waiting state, asking for mutual reachability with location $\{S_f\}$ and allowing a transition from $\{S_f\}$ to s_0 would lead to unwanted behaviors. The simulation could then restart from s_0 , freely add processes in q_{init} , and jump back to $\emptyset$, effectively discarding the processes represented by S_f . These processes would still be counted in $x_{q_u,1}$, but we would have no way of tracking their states anymore.

Lemma 7.12. There exist $C \in \mathcal{C}_{init}$ and $C' \in \mathcal{C}_f$ such that $C \rightarrow^* C'$ if and only if there is a run $(s_0, \mathbf{0}_X) \rightsquigarrow^* (\emptyset, \mathbf{0}_X)$ and a run $(\emptyset, \mathbf{0}_X) \rightsquigarrow^* (s_0, \mathbf{0}_X)$ in V_P .

Proof. Assume there exist $C \in \mathcal{C}_{init}$ and $C' \in \mathcal{C}_f$. From V_P^{MR} 's construction there is always a trivial run $(\emptyset, \mathbf{0}_X) \xrightarrow{\mathbf{0}_X}_{V_P^{MR}} (s_0, \mathbf{0}_X)$.

From Lemma 7.11, we get that there exists an execution of V_P between $(s_0, \mathbf{0}_X)$ and $(\emptyset, v)$ for some vector v such that $v(x) = 0$ for all $x \neq x_{q_f}$. As V_P^{MR} extends V_P , it is also the case that: $(s_0, \mathbf{0}_X) \rightsquigarrow_{V_P^{MR}}^* (\emptyset, v)$. We end the execution by taking $v(x_{q_f})$ times the transition $(\emptyset, \delta_f, \emptyset)^{13}$ added to V_P^{MR} , successfully reaching $(\emptyset, \mathbf{0}_X)$.

We now proceed to prove that if there are runs $(s_0, \mathbf{0}_X) \rightsquigarrow^* (\emptyset, \mathbf{0}_X)$ and $(\emptyset, \mathbf{0}_X) \rightsquigarrow^* (s_0, \mathbf{0}_X)$ in the VASS V_P^{MR} , there is an execution in P that leads all the processes in q_f . From a run of the VASS going from $(s_0, \mathbf{0}_X)$ to $(\emptyset, \mathbf{0}_X)$, we build a run from $(s_0, \mathbf{0}_X)$ to $(\emptyset, \mathbf{0}_X)$ that never takes the transition $(\emptyset, \mathbf{0}_X, s_0)$. We also put all the transitions $(\emptyset, \delta_f, \emptyset)$ at the end of the run (remember that we used a similar reasoning in Lemma 7.7).

We write the run from $(s_0, \mathbf{0}_X)$ to $(\emptyset, \mathbf{0}_X)$ as

$$(\mathcal{S}_0, v_0) \xrightarrow{\delta_1} (\mathcal{S}_1, v_1) \xrightarrow{\delta_2} \dots \xrightarrow{\delta_k} (\mathcal{S}_k, v_k) \quad (7.4)$$

We now explain how to get a run of the form:

$$(s_0, \mathbf{0}_X) \xrightarrow{\delta_0} \dots \xrightarrow{\delta_0} (s_0, v_n) \xrightarrow{\delta_0} (\emptyset, v'_0) \xrightarrow{\delta_1} (\mathcal{S}_1, v'_1) \xrightarrow{\delta_2} \dots \xrightarrow{\delta_n} (\emptyset, v'_n) \xrightarrow{\delta_f} \dots \xrightarrow{\delta_f} (\emptyset, \mathbf{0}_X) \quad (7.5)$$

such that for all $1 \leq i \leq n$:

1. $\delta_i \neq \delta_f$, i.e. all the decrements of x_{q_f} happens at the end, and
2. $\mathcal{S}_i \neq s_0$ (with the convention that $\mathcal{S}_n = \emptyset$) i.e. the run between $(\emptyset, v'_0)$ and $(\emptyset, v'_n)$ never takes transition $(\emptyset, \mathbf{0}_X, s_0)$.

We first explain how to transform the run 7.4, into a run where all the decrements of x_{q_f} happens at the end. Take an index $i \in [0, k-1]$ of the run 7.4 such that

¹³ Recall that $\delta_f(x_{q_f}) = -1$ and $\delta_f(x) = 0$ for all other counters x .

$(\mathcal{S}_i, v_i) \xrightarrow{\delta_f} (\mathcal{S}_{i+1}, v_{i+1})$. By definition of V_P^{MR} , $\mathcal{S}_i = \mathcal{S}_{i+1} = \emptyset$ and $v_{i+1}(x_{q_f}) = v_i(x_{q_f}) - 1$, and $v_{i+1}(x) = v_i(x)$ for all other counters x . Recall that $(\mathcal{S}_k, v_k) = (\emptyset, \mathbf{0}_X)$. Hence,

$$(\mathcal{S}_i, v_i) \xrightarrow{\delta_{i+2}} (\mathcal{S}_{i+2}, v'_{i+2}) \xrightarrow{\delta_{i+3}} \dots \xrightarrow{\delta_k} (\mathcal{S}_k, v'_k)$$

where $v'_j(x_{q_f}) = v_j(x_{q_f}) + 1$ and $v'_j(x) = v_j(x)$ for all other counters x , for all $i + 2 \leq j \leq k$. This holds because no decrements nor increments is prevented from happening when one component is increased. Hence, $v'_k(x_{q_f}) = 1$ and $v'_k(x) = 0$ for all other counters. We can conclude the run adding the previously withdrawn decrement: $(\mathcal{S}_k, v'_k) \xrightarrow{\delta_f} (\emptyset, \mathbf{0}_X)$. Hence, by iterating, one can obtain a run shaped as:

$$(\mathcal{S}_0, v_0) \xrightarrow{\delta_1} (\mathcal{S}_1, v_1) \xrightarrow{\delta_2} \dots \xrightarrow{\delta_k} (\mathcal{S}_k, v_k) \xrightarrow{\delta_f} \xrightarrow{\delta_f} \dots \xrightarrow{\delta_f} (\emptyset, \mathbf{0}_X) \quad (7.6)$$

such that for all $1 \leq i \leq k$, $\delta_i \neq \delta_f$. We now work with this run.

We now explain how to get rid of transitions $(\mathcal{S}_i, v_i) \xrightarrow{\mathbf{0}_X} (\mathcal{S}_{i+1}, v_{i+1})$ with $\mathcal{S}_i = \emptyset$ and $\mathcal{S}_{i+1} = s_0$. Let $i_1 \in [0, k - 1]$ such that $(\mathcal{S}_{i_1}, v_{i_1}) = (\emptyset, v_{i_1}) \xrightarrow{\mathbf{0}_X} (s_0, v_{i_1}) = (\mathcal{S}_{i_1+1}, v_{i_1+1})$. Then, by construction of V_P^{MR} , there exists $i_2 > i_1$ such that

$$(s_0, v_{i_1}) \xrightarrow{\delta_{i_1+2}} (s_0, v_{i_1+2}) \xrightarrow{\delta_{i_1+3}} \dots \xrightarrow{\delta_{i_2}} (s_0, v_{i_2}) \xrightarrow{\delta_{i_2+1}} (\emptyset, v_{i_2+1})$$

and $\delta_i = \delta_0$ for all $i_1 + 2 \leq i \leq i_2 + 1$.¹⁴ Hence, $v_{i_2+1}(x_{q_{\text{init}}}) = v_{i_1}(x_{q_{\text{init}}}) + i_2 + 1 - i_1$, and $v_{i_2+1}(x) = v_{i_1}(x)$ for all $x \neq x_{q_{\text{init}}}$. Let now $x = i_2 + 1 - i_1$. We have the following run:

$$(\mathcal{S}_0, v_0) = (s_0, \mathbf{0}_X) \xrightarrow{(\delta_0)^x} (s_0, v'_0) = (\mathcal{S}_0, v'_0) \xrightarrow{\delta_1} (\mathcal{S}_1, v'_1) \xrightarrow{\delta_2} \dots \xrightarrow{\delta_{i_1-1}} (\mathcal{S}_{i_1}, v'_{i_1})$$

where $v'_i(x_{q_{\text{init}}}) = v_i(x_{q_{\text{init}}}) + x$ and $v'_i(x) = v_i(x)$ for all $x \neq x_{q_{\text{init}}}$. Hence, $v'_{i_1} = v_{i_2+1}$, and $\mathcal{S}_{i_1} = \emptyset$ by construction. Hence:

$$(\mathcal{S}_{i_1}, v'_{i_1}) \xrightarrow{\delta_{i_2+1}} (\mathcal{S}_{i_2+2}, v_{i_2+2}) \xrightarrow{\delta_{i_2+3}} \dots \xrightarrow{\delta_k} (\mathcal{S}_k, v_k).$$

By iterating the reasoning, we get a run shaped as 7.5.

Hence, the run $(s_0, \mathbf{0}_X) \rightsquigarrow^* (\emptyset, v'_n)$ extracted from 7.5 is also a run of V_P as none of the transitions added to V_P^{MR} is taken during the run. Hence we get that: $(s_0, \mathbf{0}_X) \rightsquigarrow_{V_P}^* (\emptyset, v'_n)$, and so, thanks to Lemma 7.11, we get that there exists $\rho = C_0 \dots C_k \in \text{Exec}[P]$ with $C_k \in \mathcal{C}_f$, concluding the proof. $\square$

The above lemma shows us that the reduction to MUTUREACH is sound and complete, together with Corollary 7.2, we get the following theorem.

Theorem 7.3. *SYNCHRO for Wait-Only broadcast protocols is in EXPSPACE when $q_f \in Q_A$.*

Proof. We recall that $V_P = (\text{Loc}, X, \ell_{\text{in}}, \Delta)$ and that $V_P^{\text{MR}} = (\text{Loc}, X, \ell_{\text{in}}, \Delta \cup \{(\emptyset, \mathbf{0}_X, s_0), (\emptyset, \delta_f, \emptyset)\})$. Thanks to the previous lemma (Lemma 7.12), checking for

¹⁴ Recall that δ_0 is the vector incrementing $x_{q_{\text{init}}}$.

two runs from $(s_0, \mathbf{0}_X)$ to $(s_f, \mathbf{0}_X)$ and from $(s_f, \mathbf{0}_X)$ to $(s_0, \mathbf{0}_X)$ in V_p^{MR} is enough to answer the synchronization problem.

Thanks to Corollary 7.2, those two runs exist if and only if there exist two runs, one from $(s_0, \mathbf{0}_X)$ to $(s_f, \mathbf{0}_X)$ and one from $(s_f, \mathbf{0}_X)$ to $(s_0, \mathbf{0}_X)$ whose lengths are bounded by:

$$17(|X| + 3)^2((1 + \max(\|\Delta\|, (|\text{Loc}| + 1)^2))(1 + (|\text{Loc}| + 1)^2))^{15(|X|+3)^{|X|+5}}$$

Thanks to Remark 10, we get that the lengths are bounded by:

$$\begin{aligned} & 17(|T| + 5)^4(1 + (|\text{Loc}| + 1)^2)^{30(|T|+5)^{2(|T|+8)^2}} \\ & \leq 17(|T| + 5)^4(|\text{CoSets}| + 3)^{60(|T|+5)^{2(|T|+8)^2}} \\ & \leq 17(|T| + 5)^4(2^{(|T|+1)^2 2^{|T|+1}} + 3)^{60(|T|+5)^{2(|T|+8)^2}} \\ & \leq 17(|T| + 5)^4 2^{(|T|+1)^2 2^{|T|+1} 180(|T|+5)^{2(|T|+8)^2}} \end{aligned}$$

In particular, the counters' values during the runs are bounded by a double exponential number in the size of the protocol. Hence, after building V_p^{MR} (which takes a polynomial space according to Remark 10), we can guess non deterministically two runs of V_p^{MR} on the fly, one starting from $(s_0, \mathbf{0}_X)$ and one from $(s_f, \mathbf{0}_X)$. As double exponential numbers can be store in exponential space, we get a non-deterministic procedure working in exponential space on the size of the protocol. From Savitch's theorem, $\text{EXPSPACE} = \text{NEXPSPACE}$, which allows us to conclude. $\square$

7.5.2 Lower bound

We get the EXPSPACE -hardness result by proving that if one can solve SYNCHRO problem with $q_f \in Q_A$, then one can solve the coverability problem in **test-free Counter Machines** (shortly **test-free CM**). We recall that this problem is EXPSPACE -hard Corollary 3.1, [Lip76], and is stated as follows: given a test-free CM $\text{CM} = (\text{Loc}, \ell_0, X, \Delta)$ and a location $\ell_f \in \text{Loc}$, decide whether there is an execution from $(\ell_0, \mathbf{0}_X)$ to (ℓ_f, v) for some $v \in \mathbb{N}^{|X|}$.

Fix a test-free CM $\text{CM} = (\text{Loc}, \ell_0, X, \Delta)$ and a location $\ell_f \in \text{Loc}$. We let $P_{\text{CM}} = (Q, \Sigma, q_{\text{init}}, T)$ the protocol simulating CM depicted in Figure 7.9. Part of the protocol P_{CM}' is the same as the one depicted in Section 7.4.2. We give again the figures describing it in Figure 7.10.

Then ℓ_f is coverable if and only if there exists an execution $C_0 \rightarrow^* C_f$ in which all processes reach ℓ'_f starting from an initial configuration $C_0 \in \mathcal{C}_{\text{init}}$. If ℓ_f is coverable, the execution of the VASS can be simulated in the protocol like in section 7.4.2. Only the end of the execution differs: once the leader process is in ℓ_f , this time it might be that other processes still populate states unit_x for $x \in X$. All processes (excepted the leader) broadcast end, which is received by the leader, putting everyone in ℓ'_f .

Conversely, if there exists an execution $C_0 \rightarrow^* C_f$ in which all processes reach ℓ'_f , it means that ℓ_f is coverable. Let e_0 be the first process to broadcast start, then an execution of the VASS covering ℓ_f can be built based on the sequence of states visited by e_0 between ℓ_0 and ℓ_f . This relies on three keys observations:

- (1) The process e_0 must visit ℓ_f , otherwise it cannot reach ℓ'_f .

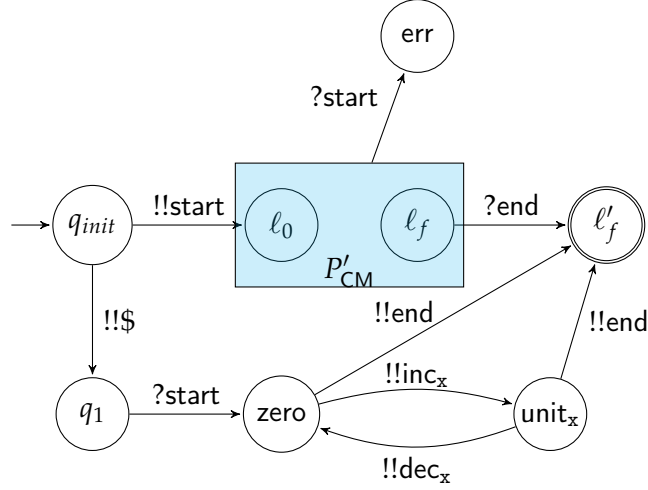

 Figure 7.9: Protocol P'_{CM} associated to a test-free CM CM.

 Figure 7.10: Part of P'_{CM} that simulates respectively the transition $(\ell, x+, \ell') \in \Delta$ at the left, and $(\ell'', x-, \ell''') \in \Delta$ at the right.

- (2) When e_0 is between ℓ_0 and ℓ_f , e_0 is the only process in this region. If another process attempts to join by broadcasting start, e_0 will receive the message and go to err , preventing it from reaching ℓ'_f .
- (3) When e_0 reaches ℓ_0 for the first time, no other process is in the states $\{unit_x \mid x \in X\}$, since start is sent for the first time at this point.

Together with theorem 7.3, this establishes the following result.

Theorem 7.4. SYNCHRO for Wait-Only protocols is EXPSPACE-complete when $q_f \in Q_A$.

7.6 REPEATED COVERABILITY PROBLEM FOR WAIT-ONLY PROTOCOLS

7.6.1 Upper bound

We show now that REP-COVER on Wait-Only protocols is in EXPSPACE. We remind how the REP-COVER problem is defined:

- the **repeated coverability** problem (REP-COVER) asks, given a protocol $P = (Q, \Sigma, q_{init}, T)$ and a transition $t_f = (q, \alpha, q') \in T$, if there exist an infinite execution $C_0 C_1 \dots$ and a process $e \in [1, |C_0|]$ such that for all $N_1 \in \mathbb{N}$, there exists $N_2 \geq N_1$ with :

- if $\alpha = !!a$ for an $a \in \Sigma$, then $C_{N_2} \xrightarrow{e, t_f} C_{N_2+1}$;

- otherwise $\alpha = ?a$ for an $a \in \Sigma$, and $(C_{N_2}(e), ?a, C_{N_2+1}(e)) = t_f$ and $C_{N_2} \xrightarrow{e', (p, !!a, p')} C_{N_2+1}$ with $p, p' \in Q$ and $e' \in [1, |C_0|]$.

We prove the upper bound by reducing the problem to the repeated coverability problem in VASS, which is stated as follows:

- given a VASS $V = (\text{Loc}, \ell_{\text{init}}, X, \Delta)$ and a location $\ell_f \in \text{Loc}$, **REP-COVER[VASS]** asks if there is an infinite execution $(\ell_{\text{init}}, \mathbf{0}_X) \rightsquigarrow (\ell_1, v_1) \rightsquigarrow \dots$ such that, for all $i \in \mathbb{N}$, there exists $j > i$ such that $\ell_j = \ell_f$?

In [Hab97], Habermehl proves that the problem is in ExpSpace , but more precisely he proves the following result.

Theorem 7.5 (Theorem 3.1 of [Hab97]). *For a VASS $V = (\text{Loc}, \ell_{\text{init}}, X, \Delta)$, the repeated coverability problem is solvable in $O(\log(l) + \log(|\text{Loc}|))2^{c|X|\log(|X|)}$ nondeterministic space for some constant c independent of V , and where the absolute values of components of vectors in Δ are smaller than 1.*

We will need this precise bound to get our result.

7.6.1.1 New subset of executions

Given a protocol P and a transition $t = (q, \alpha, q')$, between two configurations C and C' , and a process e , we say that e **fires** t if and only if:

- if $t \in T_{\text{Act}}$, then $C \xrightarrow{e, t} C'$;
- otherwise $\alpha = ?a$ for a message $a \in \Sigma$, and $(C(e), ?a, C'(e)) = t$ and $C \xrightarrow{e', (p, !!a, p')} C'$ with $p, p' \in Q$ and $e' \in [1, |C|]$

Let $P = (Q, \Sigma, q_{\text{init}}, T)$ be a Wait-Only protocol and t_f the transition that has to occur infinitely often. A major difference from the synchronization problem is that we can no longer focus on well-formed execution: assume t_f to be a reception transition. Then it might be that a process firing t_f infinitely often needs to take a different reception path than the “faster” one, taken by other processes realizing the well-formedness of the execution.

Example 7.6. Consider the protocol P' depicted in Figure 7.11, and let $t_f = (q_2, ?b, q_{\text{init}})$. One can build an execution where t_f is fired infinitely often as attested by the following prefix:

$$\begin{aligned} \rho = (q_{\text{init}}, q_{\text{init}}, q_{\text{init}}, q_{\text{init}}) &\xrightarrow{1, (q_{\text{init}}, !!c, q_1)} (q_1, q_{\text{init}}, q_{\text{init}}, q_{\text{init}}) \xrightarrow{2, (q_{\text{init}}, !!c, q_1)} (q_1, q_1, q_{\text{init}}, q_{\text{init}}) \\ &\xrightarrow{3, (q_{\text{init}}, !!c, q_1)} (q_1, q_1, q_1, q_{\text{init}}) \xrightarrow{4, (q_{\text{init}}, !!a, q_3)} (q_{\text{init}}, q_{\text{init}}, q_2, q_3) \\ &\xrightarrow{1, (q_{\text{init}}, !!c, q_1)} (q_1, q_{\text{init}}, q_2, q_4) \xrightarrow{2, (q_{\text{init}}, !!c, q_1)} (q_1, q_1, q_2, q_5) \\ &\xrightarrow{3, (q_5, !!b, q_{\text{init}})} (q_1, q_1, q_{\text{init}}, q_{\text{init}}) \end{aligned}$$

Hence, process 3 takes transition $(q_2, ?b, q_{\text{init}})$ in the segment between $(q_1, q_1, q_{\text{init}}, q_{\text{init}})$ and $(q_1, q_1, q_{\text{init}}, q_{\text{init}})$. Hence, one can build an execution $(q_{\text{init}}, q_{\text{init}}, q_{\text{init}}, q_{\text{init}}) \rightarrow^+ (q_1, q_1, q_{\text{init}}, q_{\text{init}}) \rightarrow^+ (q_1, q_1, q_{\text{init}}, q_{\text{init}}) \rightarrow^+ \dots$ where process 3 takes t_f infinitely often. Observe that ρ is *not* well-formed, when, from $\rho_3 = (q_1, q_1, q_1, q_{\text{init}})$, a is broadcast, processes 1 and 2 take the reception transition to q_{init} , whereas process 3 takes the reception transition to q_2 . However their next action state is the same, i. e. q_{init} . Transforming ρ into a well-formed

execution would force process 3 to take the transition $(q_1, ?a, q_{init})$ rather than $(q_1, ?a, q_2)$, preventing it to take transition $t_f = (q_2, ?b, q_{init})$. Hence, an execution where a process takes infinitely t_f *can not be* well-formed. However, observe that the only process that needs to “diverge” from the reception path of other processes is the process taking t_f . Hence, we define a notion where all processes except one (in our example, process 3), follow the same path of reception.

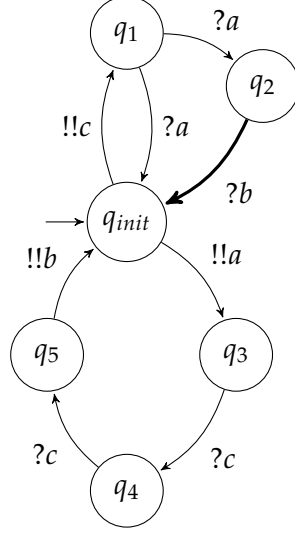


Figure 7.11: A Protocol P' , for which there exists an execution in which $(q_2, ?b, q_{init})$ is fired infinitely often.

As illustrated by the above example, only one process needs to differ from the deterministic path of receptions. We then introduce the notion of *almost well formed*.

Definition 7.2 (Almost well-formed). An execution ρ of a protocol P is *almost well-formed* if there exists a process e_0 such that for all $0 \leq i < \text{size}(\rho)$, for all $e_1, e_2 \in [1, \#\text{proc}(\rho)] \setminus \{e_0\}$, if

- $\rho_i(e_1) = \rho_i(e_2) \in Q_W$, and
- $\text{na-state}(\rho, i, e_1) = \text{na-state}(\rho, i, e_2) = q$,

it holds that $\rho_k(e_1) = \rho_k(e_2)$ for all $i \leq k \leq \text{na-index}(\rho, i, e_1)$. The process e_0 is called the *nonconformist* process.

Let ρ be an *almost well-formed* execution of P , with nonconformist process e_0 . We define, for all $0 \leq j \leq \text{size}(\rho)$ and $q_a \in Q_A$:

$$\begin{aligned} \text{na-index-set}^{\text{conf}}(\rho, j, q_a) &= \{i \mid \text{there exists } e \in [1, \#\text{proc}(\rho)] \setminus \{e_0\} \\ &\quad \text{s.t. } \rho_j(e) \in Q_W \text{ and } \text{na}(\rho, j, e) = (q_a, i)\} \end{aligned}$$

We naturally get an analogous result of Lemma 7.2 that for well-formed execution. The proof is the same as the proof of Lemma 7.2.

Lemma 7.13. For all almost well-formed execution ρ , for all $0 \leq j < \text{size}(\rho)$, and for all $q_a \in Q_A$, we have $|\text{na-index-set}^{\text{conf}}(\rho, j, q_a)| \leq |Q_W|$.

We now state that **almost well-formed** executions is the right subset of executions to look at for studying the REP-COVER problem. This is a direct consequence from Lemma 7.1: consider an execution ρ with a process e_0 where e_0 takes infinitely often t_f , by applying Lemma 7.1 successively it to every processes but e_0 , we get an *almost well-formed* execution ρ' where $\rho(e_0) = \rho'(e_0)$ ¹⁵, and its nonconformist process is e_0 .

Corollary 7.3. *Given a protocol $P = (Q, \Sigma, q_{init}, T)$ and a transition $t_f = (q, \alpha, q') \in T$, there exist an infinite execution $C_0 C_1 \dots$ and a process $e \in [1, |C_0|]$ such that for all $N_1 \in \mathbb{N}$, there exists $N_2 \geq N_1$ where e fires t_f between C_{N_2} and C_{N_2+1} , if and only if there exists an infinite almost well-formed execution respecting the same conditions with e as nonconformist process.*

7.6.1.2 Definition of the VASS

Now that we have the right notion for executions, we proceed to define the VASS simulating almost well-formed executions of a network running a Wait-Only broadcast protocol. As for the SYNCHRO problem, we build the VASS from the VASS V_P defined in Section 7.3.4. Intuitively, we add to the location of V_P the state of the non-conformist process which will fires the transition infinitely often. In addition, we need to identify the “good” locations which are the locations we want to see infinitely often in the VASS. In order to do that, we add to the locations an emoji, either $\ominus$ or $\odot$: the ones labeled with a $\odot$ are the ones to see infinitely often in the VASS execution.

We fix a Wait-Only broadcast protocol $P = (Q, \Sigma, q_{init}, T)$ and a transition $t_f \in T$. The VASS V_P^{RC} extends the VASS $V_P = (Loc, X, s_0, \Delta)$ defined in Section 7.3.4. We recall that $\Delta = \Delta_0 \cup \bigcup_{t \in T} \Delta_t \cup \Delta_e$. We define the new VASS in the following way: $V_P^{RC} = (Loc', X, s_0, \Delta')$ where:

- $Loc' = \{s_0\} \cup \{\odot, \ominus\} \times Q \times CoSets$, the new locations are labeled with an emoji and the state of the nonconformist process.
- $\Delta' = \Delta'_0 \cup \bigcup_{t \in T} \Delta_t^{sum} \cup \bigcup_{t \in T} \Delta_t^{proc} \cup \Delta'_e \cup \Delta_{\odot}$, is defined as follows:
 - $\Delta'_0 = \{(s_0, \delta_0, s_0), (s_0, \delta_0, (\odot, q_{init}, \emptyset))\}$, where $\delta_0(x_{q_{init}}) = 1$ and $\delta_0(x) = 0$ for all other counters. Executions of V_P^{RC} start as in V_P . In the starting location of the simulation, the nonconformist process is in q_{init} , and did not fired t_f yet (hence $\odot$).
 - $\Delta'_e = \{((\odot, q, S), \delta, (\odot, q', S')) \mid q = q' \text{ and } (S, \delta, S') \in \Delta_e\}$. As in V_P , Δ'_e allows to empty the counters associated to summaries that have disappeared.
 - $\Delta_{\odot} = \{((\odot, q, S), 0_X, (\odot, q, S))\}$, transitions in $\Delta_{\odot}$ make the transition from $\odot$ locations to $\odot$ locations. This way, from a $\odot$ location, the only possible action is to get back to the corresponding $\odot$ location.
 - For $t = (p, !!a, p')$, we have $((\odot, q, S), \delta, (\square, q', S')) \in \Delta_t^{sum}$ iff $(S, \delta, S') \in \Delta_t$ and one of the following conditions holds:
 1. $(q, ?a, q') \in T$, $t_f \neq (q, ?a, q')$ and $\square = \odot$;
 2. $a \notin R(q)$ with $q = q'$, and $\square = \odot$;
 3. $t_f = (q, ?a, q') \in T$ and $\square = \odot$;

¹⁵ Recall that $\rho(e_0)$ is the sequence $\rho_0(e_0)\rho_1(e_0)\dots$.

Transitions in Δ_t^{sum} describe a broadcast from a process counted in the valuation, i. e. not from the nonconformist process. Hence, going from $\mathcal{S}$ to $\mathcal{S}'$ is done as in V_P . The state of the nonconformist process evolve depending on if it can receive the message or not. When it receives the message with transition t_f , it goes to a $\ominus$ location, otherwise, to a $\oplus$ location.

- For $t = (q, !!a, q')$, we have $((\ominus, q, \mathcal{S}), \mathbf{0}_X, (\square, q', \mathcal{S}')) \in \Delta_t^{\text{proc}}$ if and only if $\mathcal{S} = \{S_1, \dots, S_k\}$ and for all $1 \leq i \leq k$, there exists S'_i such that $S_i \xrightarrow{t} S'_i$ and $\mathcal{S}' = \{S'_1, \dots, S'_k\} \setminus \{\text{Done}\}$, and one of the following conditions holds:
 4. $\square = \ominus$ and $t_f = (q, !!a, q')$;
 5. $\square = \ominus$ and $(q, !!a, q') \neq t_f$.

Transitions in Δ_t^{proc} describe a broadcast from the nonconformist process, when it is done via $t = t_f$, the next location is a $\ominus$ location, otherwise, a $\oplus$ location. Summaries in $\mathcal{S}$ then evolve according to t , which we faithfully define with relation $\xrightarrow{t}$. The set $\mathcal{S}'$ only keeps the summaries that are not arrived in their next action state.

Let $\gamma = (\ell, v)$ be a configuration of V_P^{RC} , we say that γ is a *P-configuration* whenever $\ell \neq s_0$. We say that γ is a $\ominus$ -configuration (resp. $\oplus$ -configuration) whenever $\ell = (\ominus, q, \mathcal{S})$ (resp. $\ell = (\oplus, q, \mathcal{S})$).

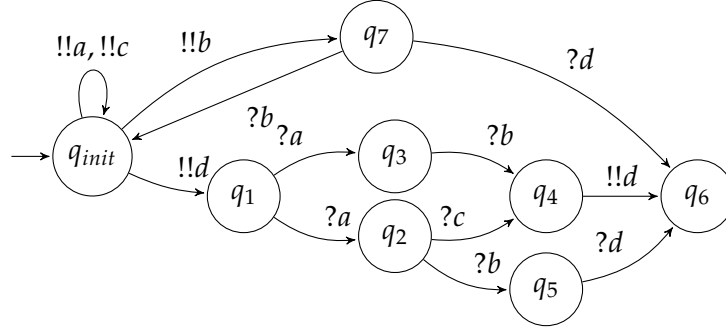


Figure 7.12: Example of a Wait-Only protocol P .

Example 7.7. We illustrate the transformation from V_P to V_P^{RC} with an example. We re picture the protocol displayed at the beginning of this chapter in Figure 7.12. Previously, we presented a part of the VASS V_P , which we give again in Figure 7.13. The VASS V_P^{RC} transforms the part described in Figure 7.13 into the part of the V_P^{RC} depicted in Figure 7.14 (when $t_f = (q_7, ?b, q_{\text{init}})$). In this example, the non conformist process takes transition t_f hence, the VASS goes to a $\ominus$ location before transitioning back to the corresponding $\oplus$

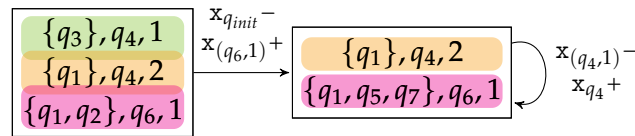


Figure 7.13: One successor of location $\{(\{q_3\}, q_4, 1), (\{q_1\}, q_4, 2), (\{q_1, q_2\}, q_6, 1)\}$ (left location) after the broadcast transition $(q_{\text{init}}, !!b, q_7)$ in V_P .

location from which there is the transition allowing to empty the counter of the disappeared summary.

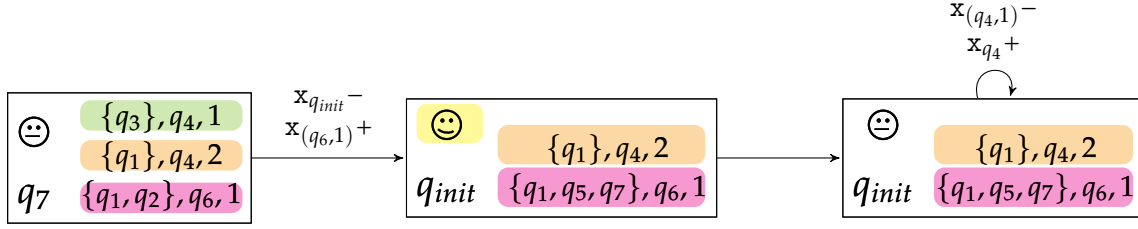


Figure 7.14: The transformation of VASS V_P^{RC} when $t_f = (q_7, ?b, q_{\text{init}})$: the nonconformist process takes transition $(q_7, ?b, q_{\text{init}})$ and hence the first transition leads to a ☹️ location.

7.6.1.3 Soundness of the construction

We now proceed to prove the soundness of our reduction. Given a configuration $\gamma = ((\square, q, \mathcal{S}), v)$, the new implementation of γ is the set of network configurations C such that

$$C = \dot{q} \oplus C' \text{ and } C' \in \llbracket (\mathcal{S}, v) \rrbracket.$$

The set of such configurations is denoted $\llbracket (\mathcal{S}, v) \rrbracket^{\text{RC}}$. Recall that $\llbracket (\mathcal{S}, v) \rrbracket$ is the set of configurations respecting **CondImpl3**, **CondImpl1** and **CondImpl2** with regard to $(\mathcal{S}, v)$ in the VASS V_P described in Section 7.3.4. Perhaps surprisingly, the set of configurations in the implementation of a VASS configuration, does not depend on its emoji. As the emoji ☹️ means that transition t_f have just been fired, it is only relevant when we look at a transition between two VASS configurations.

We now proceed to adapt the soundness proof presented in Section 7.3.5 (Lemma 7.3). As before, we prove that from a configuration implementing a VASS configuration γ , we can reach a configuration in the implementation of the successor of γ . Furthermore, we prove that from a VASS configuration, if we can reach a ☹️-configuration, then from any configuration in its implementation, t_f can be fired. Furthermore, as before, emptying a counter of a deleted summary does not change the implementation set of a VASS configuration, and changing its smiley from ☹️ to ☹️ neither.

Lemma 7.14. Let $\gamma = ((\square, q, \mathcal{S}), v)$ and $\gamma' = ((\diamond, q', \mathcal{S}'), v')$ be two P -configurations of V_P^{RC} and $C \in \llbracket \gamma \rrbracket^{\text{RC}}$. If $\gamma \xrightarrow{\delta} \gamma'$ in V_P^{RC} then:

- if $(\gamma, \delta, \gamma') \in \Delta_{\text{☹️}}$ or $(\gamma, \delta, \gamma') \in \Delta_e$, then $C \in \llbracket \gamma' \rrbracket^{\text{RC}}$;
- otherwise there exists $C' \in \llbracket \gamma' \rrbracket^{\text{RC}}$ such that $C \rightarrow C'$. Furthermore, γ' is a ☹️-configuration only if process 1 fires t_f between C and C' .

Proof. Let $\gamma = ((\square, q, \mathcal{S}), v)$ and $\gamma' = ((\diamond, q', \mathcal{S}'), v')$. We prove that when $(\gamma, \delta, \gamma') \in \Delta_{\text{☹️}} \cup \Delta_e$ then if $C \in \llbracket \gamma \rrbracket$ then $C \in \llbracket \gamma' \rrbracket$. Furthermore, if $(\gamma, \delta, \gamma') \in \Delta_i^{\text{sum}}$, then one can use Lemma 7.3 to define a configuration $C' = \dot{q}' \oplus C''$ where C'' is the configuration obtained from the lemma. We justify that q' is defined so the nonconformist process

behaves well regarding transition t . If $(\gamma, \delta, \gamma') \in \Delta_t^{\text{proc}}$, we justify that $\mathcal{S}'$ is well defined so processes described by $\mathcal{S}'$ are behaving accordingly to the broadcast transition t fired by the non conformist process. Hence, we are able to build a configuration $C' \in \llbracket \gamma' \rrbracket$ such that $C \rightarrow C'$. For each case, we justify that $\diamond = \odot$ if and only if t_f is fired by the nonconformist process.

- if $(\gamma, \delta, \gamma') \in \Delta_{\odot}$, then $\delta = \mathbf{0}_X$, $(\mathcal{S}, v) = (\mathcal{S}', v')$ and $\square = \odot$ and $\diamond = \odot$. Then $C' = C \in \llbracket \gamma' \rrbracket^{\text{RC}}$.
- if $(\gamma, \delta, \gamma') \in \Delta_e$, then one can prove using a similar technique as in Lemma 7.3's proof, and prove that $C \in \llbracket \gamma' \rrbracket^{\text{RC}}$. Observe that in that case $\square = \diamond = \odot$.

From the definition of $\llbracket \cdot \rrbracket^{\text{RC}}$, we now write C as $C = \dot{q} \oplus C_1$ with $C_1 \in \llbracket \mathcal{S}, v \rrbracket$.

- if $(\gamma, \delta, \gamma') \in \Delta_t^{\text{sum}}$ for a transition $t = (p, !!a, p')$, then from V_p^{RC} , $(\mathcal{S}, \delta, \mathcal{S}') \in \Delta_t$. Hence, $(\mathcal{S}, v) \xrightarrow{\delta} (\mathcal{S}', v')$, and so from Lemma 7.3, we get that there exists C'_1 such that $C_1 \xrightarrow{e, t} C'_1$ for a process e , and $C'_1 \in \llbracket \mathcal{S}', v' \rrbracket$. Furthermore, either $q = q'$ and $a \notin R(q)$, or $(q, ?a, q') \in T$ (from the definition of V_p^{RC}). Hence $\dot{q} \oplus C_1 \xrightarrow{t} \dot{q}' \oplus C'_1$. We denote $C' = \dot{q}' \oplus C'_1$. We have that $C' \in \llbracket \gamma' \rrbracket^{\text{RC}}$. Furthermore, observe that $\diamond = \odot$ only if $(q, ?a, q') = t_f$.

- if $(\gamma, \delta, \gamma') \in \Delta_t^{\text{proc}}$, then $t = (q, !!a, q')$, $\delta = \mathbf{0}_X$, and if we denote $\mathcal{S} = \{S_1, \dots, S_n\}$, from V_p^{MR} 's definition, we know for all $1 \leq i \leq n$, there exists S'_i such that $S_i \xrightarrow{t} S'_i$ and $\mathcal{S}' = \{S'_i \mid 1 \leq i \leq n\} \setminus \{\text{Done}\}$. We define C'_1 as in the proof of Lemma 7.3 when $(\mathcal{S}, \delta, \mathcal{S}') \in \Delta_t$, excluding cases about the process firing $(q, !!a, q')$ (as it is now outside of $(\mathcal{S}, v)$). It follows, from the definition of C'_1 , that $C = \dot{q} \oplus C_1 \xrightarrow{1, t} \dot{q}' \oplus C'_1$. We now define $C_2 = \dot{q}' \oplus C'_1$.

To prove that $C'_1 \in \llbracket \mathcal{S}', v \rrbracket$, we consider the function f realizing $C_1 \in \llbracket \mathcal{S}, v \rrbracket$. The function attesting that $C'_1 \in \llbracket \mathcal{S}', v \rrbracket$, is the same hence $f' = f$ (the function does not change as the valuation over counters neither). Proving **CondImpl1**, **CondImpl2** and **CondImpl3** is then easier and can easily be extracted from the proof of Lemma 7.3: only consider cases concerning processes different from the one firing $(q, !!a, q')$. This leads to $C'_1 \in \llbracket \mathcal{S}', v \rrbracket$ and so $C_2 \in \llbracket \gamma' \rrbracket^{\text{RC}}$. From V_p^{RC} 's definition, if γ' is a $\odot$ -configuration, then $t_f = t = (q, !!a, q')$ hence e_0 fires t between C_1 and C_2 .

□

We can then lift this local lemma to prove the soundness of our reduction, i. e. from an infinite execution of the VASS hitting infinitely often a $\odot$ location, we extract an infinite execution of P in which a process fires transition t_f infinitely often.

Lemma 7.15. If there is an infinite execution of V_p^{RC} visiting infinitely often a location $(\odot, q, \mathcal{S})$ for a state $q \in Q$ and a set $\mathcal{S} \in \text{CoSets}$, then there is an infinite execution of P where a process fires infinitely transition t_f .

Proof. Let $\gamma_0\gamma_1\cdots$, and $\ell_{\odot} = (\odot, q, S) \in \text{Loc}'$ such that for all $i \in \mathbb{N}$, there exists $j > i$ such that $\gamma_j = (\ell_{\odot}, v_j)$ for some valuation v_j .

From the VASS V_p^{RC} 's definition, we get that the execution starts with a prefix of the form:

$$\gamma_0 = (s_0, \mathbf{0}_X) \rightsquigarrow^* (s_0, v) \rightsquigarrow ((\odot, q_{\text{init}}, \emptyset), v_{\text{init}}) = \gamma_{j_0}$$

for an index $j_0 \in \mathbb{N}_{>0}$, where $v_{\text{init}}(\mathbf{x}_{q_{\text{init}}}) = n \in \mathbb{N}_{>0}$ and $v_{\text{init}}(\mathbf{x}) = 0$ for all other counters $\mathbf{x}$. We let C_{init} be the initial configuration with $n + 1$ processes. We get that $C_{\text{init}} = q_{\text{init}} \oplus C'_{\text{init}}$ where C'_{init} is the initial configuration with n processes, and $C_{\text{init}} \in \llbracket (\emptyset, v_{\text{init}}) \rrbracket$. Hence, $C_{\text{init}} \in \llbracket \gamma_0' \rrbracket^{\text{RC}}$.

We denote $w = \gamma_{i_0}\gamma_{i_1}\gamma_{i_2}\cdots$ the subword of $\gamma_0\gamma_1\cdots$ s.t.:

- $i_0 = j_0$, and
- i_k is the smallest index $i_k > i_{k-1}$ such that $\gamma_{i_{k-1}} \xrightarrow{\delta_{i_k}} \gamma_{i_k}$ with $\gamma_{i_{k-1}} = (\ell, v)$, $\gamma_{i_k} = (\ell', v)'$ and $(\ell, \delta_{i_k}, \ell') \in \Delta_t^{\text{sum}} \cup \Delta_t^{\text{proc}}$ for a transition $t \in T$ (i. e. $(\ell, \delta_{i_k}, \ell') \notin \Delta_e' \cup \Delta_{\odot}'$).

We first prove the following claim.

Claim 7.10. The word w is infinite and visits infinitely often $(\odot, q, S)$.

Proof. If w is finite, there exists $k \in \mathbb{N}_{>0}$ such that, for all $j \geq k$, $\gamma_{j-1} \xrightarrow{\delta_j} \gamma_j$ with $\gamma_{j-1} = (\ell, v)$, $\gamma_j = (\ell', v)'$ and $(\ell, \delta_j, \ell') \in \Delta_e' \cup \Delta_{\odot}'$.

If $(\ell, \delta_j, \ell') \in \Delta_{\odot}' \cup \Delta_e'$, then γ_{j+1} is necessarily a $\odot$ -configuration, and as no transition in $\bigcup_{t \in T} (\Delta_t^{\text{sum}} \cup \Delta_t^{\text{proc}})$ is taken after k , there is no $\odot$ -configuration happening after γ_j (observe that the ongoing transitions to $\odot$ locations are all in $\bigcup_{t \in T} (\Delta_t^{\text{sum}} \cup \Delta_t^{\text{proc}})$), which contradicts the hypothesis on $\gamma_0\gamma_1\cdots$. Hence w is infinite.

Observe that, as there is no transitions $((\odot, q, S), \delta, (\odot, q, S))$, $(\odot, q, S)$ must be left infinitely often, and so entered infinitely often, in other words, for all $i \in \mathbb{N}$, there exists $j \geq i$ such that $\gamma_j = (\ell_j, v_j)$ and $\gamma_{j+1} = ((\odot, q, S), v_{j+1})$ and $\ell_j \neq (\odot, q, S)$. Again, the only ongoing transitions in V_p^{RC} to $(\odot, q, S)$ are transitions in $\bigcup_{t \in T} \Delta_t^{\text{sum}} \cup \Delta_t^{\text{proc}}$. Hence, for all $m \in \mathbb{N}$ such that $\gamma_m = (\ell_m, v_m)$ and $\gamma_{m+1} = ((\odot, q, S), v_{m+1})$ with $\ell_m \neq (\odot, q, S)$, there exists i_r such that $w = \gamma_{i_0}\cdots\gamma_{i_r}\cdots$ and $i_r = m + 1$, which concludes the proof. $\square$

We now proceed to build for all $j \in \mathbb{N}$ a configuration $C_j \in \llbracket \gamma_{i_j} \rrbracket$ such that $C_{\text{init}} \rightarrow^* C_j$, and if γ_{i_j} is a $\odot$ -configuration with $j > 0$, then process 1 fires t_f between C_{j-1} and C_j . For $j = 0$, we take $C_0 = C_{\text{init}}$, as $C_{\text{init}} \in \llbracket \gamma_0' \rrbracket^{\text{RC}}$ and $C_{\text{init}} \rightarrow^* C_{\text{init}}$, C_0 satisfies the requirements. Let $j > 0$ and assume we have built $C_j \in \llbracket \gamma_{i_j} \rrbracket$ such that $C_{\text{init}} \rightarrow^* C_j$. We denote $\gamma_i = (\ell_i, v_i)$ for all $i \in \mathbb{N}$. From the construction of w ,

$$\gamma_{i_j} \xrightarrow{\delta_{i_{j+1}}} \cdots \xrightarrow{\delta_{i_{j+1}-1}} \gamma_{i_{j+1}-1} \xrightarrow{\delta_{i_{j+1}}} \gamma_{i_{j+1}}$$

where $(\ell_x, \delta_x, \ell_{x+1}) \in \Delta_e' \cup \Delta_{\odot}'$ for all $x \in [i_j + 1, i_{j+1} - 1]$, and $(\ell_{i_{j+1}-1}, \delta_{i_{j+1}}, \ell_{i_{j+1}}) \in \Delta_t^{\text{sum}} \cup \Delta_t^{\text{proc}}$ for a transition $t \in T$. From Lemma 7.14, we get that $C_j \in \llbracket \gamma_{i_{j+1}-1} \rrbracket$, and

applying Lemma 7.14 once again, we get that there exists $C_{j+1} \in \llbracket \gamma_{i+1} \rrbracket$ such that $C_j \rightarrow C_{j+1}$ and if γ_{i+1} is a $\ominus$ -configuration, then process 1 fires t_f between C_j and C_{j+1} . Hence, we have built an infinite execution $C_{init}C_1 \cdots$ such that for all $i \in \mathbb{N}_{>0}$, there exists $j \geq i$ such that process 1 fires t_f between C_j and C_{j+1} . $\square$

7.6.1.4 Completeness of the construction

We now prove the completeness of our reduction. Given an almost well-formed execution ρ , consider the **nonconformist** process e_0 . We assume w.l.o.g. that $e_0 = \# \text{proc}(\rho)$. We write ρ^{conf} the sequence of configurations $\rho'_0 \rho'_1 \cdots$ where ρ'_i is the configuration ρ_i excluding process e_0 , for all $0 \leq i < \text{size}(\rho)$. As $e_0 = \# \text{proc}(\rho)$, each ρ'_i is a well-defined configuration of $\# \text{proc}(\rho) - 1$ processes. The sequence $\rho'_0 \rho'_1 \cdots$ is not technically an execution (as it might not be true that $\rho'_i \rightarrow \rho'_{i+1}$ when $\rho_i \xrightarrow{e_0, t} \rho_{i+1}$ for a transition t) but definitions of well-formedness can easily be lifted to sequence of configurations. The new representative set of ρ_i is the set of VASS configurations $((\square, q, \mathcal{S}), v)$ such that:

$$(\mathcal{S}, v) \in \mathbf{repr}(\rho^{\text{conf}}, i) \text{ and } q = \rho_i(e_0)$$

We denote the set of representative by $\mathbf{repr}^{\text{RC}}(\rho, i)$.

Lemma 7.16. Let ρ be an almost well-formed execution with nonconformist process e_0 . Let $0 \leq i < \text{size}(\rho) - 1$, for all $\gamma_i \in \mathbf{repr}^{\text{RC}}(\rho, i)$, there exists $\gamma_{i+1} \in \mathbf{repr}^{\text{RC}}(\rho, i+1)$ such that $\gamma_i \rightsquigarrow^+ \gamma_{i+1}$. Furthermore, if e_0 fires t_f between ρ_i and ρ_{i+1} , then $\gamma_i \rightsquigarrow^+ \gamma^{\ominus} \rightsquigarrow^* \gamma_{i+1}$ with $\gamma^{\ominus}$ an $\ominus$ -configuration.

*Proof.*¹⁶ The proof will proceed as for Lemma 7.4: from a configuration $\gamma_i = ((\square, q, \mathcal{S}), v)$ in $\mathbf{repr}^{\text{RC}}(\rho, i)$, we can compute the effect on $\mathcal{S}$ of the broadcast transition $t = (q, !!a, q')$ depending if its broadcast by the nonconformist process or not. When it is not broadcast by the nonconformist process, we have that $\rho'_i \rightarrow \rho'_{i+1}$ (where ρ'_j is ρ_j without e_0) and we can apply Lemma 7.4 to obtain a new configuration $\lambda_{i+1} \in \mathbf{repr}(\rho^{\text{conf}}, i+1)$ such that $\lambda_i \rightsquigarrow^\delta \lambda_{i+1}$. Building $\gamma_{i+1} \in \mathbf{repr}^{\text{RC}}(\rho, i+1)$ then naturally follows. The emoji of γ_{i+1} depends on the behavior of e_0 .

When it is broadcast by the nonconformist process, we re use the intermediate set of summaries $\mathcal{S}_0$ defined in Lemma 7.4's proof: it was the evolution of summaries $\mathcal{S}$ upon the reception of the message. This time, this set will be our new summary set as no process joins the summaries as the broadcast is made by e_0 . As in Lemma 7.4, it leads to the definition of γ' in which some deleted summaries might still need to be emptied and such that $\gamma_i \rightsquigarrow^+ \gamma'$. The emoji of γ' then depends on if $t = t_f$ or not. We then define γ_{i+1} to be γ' but with the right counters' valuations on labels of deleted summaries and justify that $\gamma' \rightsquigarrow^* \gamma_{i+1}$. We have that $\gamma_{i+1} \in \mathbf{repr}^{\text{RC}}(\rho, i+1)$.

Let ρ be an almost well-formed execution with nonconformist process e_0 and $0 \leq i < \text{size}(\rho) - 1$. We assume w.l.o.g. that $e_0 = \# \text{proc}(\rho)$. We write $\rho_i \xrightarrow{e, t} \rho_{i+1}$. First, if

¹⁶ This proof extends Lemma 7.4's proof on $\mathbb{V}_P$, and we will refer to it throughout the proof, instead of copying parts of it.

$\gamma_i = ((\ominus, q, \mathcal{S}), v)$ then we consider $\gamma_i \xrightarrow{0_X} \gamma$ where $\gamma = ((\ominus, q, \mathcal{S}), v)$. Otherwise we let $\gamma = \gamma_i$. As $\gamma_i \in \mathbf{repr}^{\text{RC}}(\rho, i)$, it holds that $\gamma \in \mathbf{repr}^{\text{RC}}(\rho, i)$ from $\mathbf{repr}^{\text{RC}}(\cdot, \cdot)$'s definition. We now build γ_{i+1} from γ .

- *First case: $e \neq e_0$.* Then it holds that $\rho'_i \xrightarrow{e, t} \rho'_{i+1}$, where $\rho^{\text{conf}} = \rho'_0 \rho'_1 \dots$. From the proof of Lemma 7.4, there exists $\lambda' = (\mathcal{S}', v')$ and $\lambda_{i+1} = (\mathcal{S}', v_{i+1})$ such that:

$$(\mathcal{S}, v) \xrightarrow{\delta} (\mathcal{S}', v') \rightsquigarrow^* \lambda_{i+1}$$

with $(\mathcal{S}, \delta, \mathcal{S}') \in \Delta_t$, and $(\mathcal{S}', v_{i+1}) \in \mathbf{repr}(\rho^{\text{conf}}, i)$. We define $\gamma' = ((\square, \rho_{i+1}(e_0), \mathcal{S}'), v')$ with

$$\square = \ominus \text{ if and only if } (\rho_i(e_0), ?a, \rho_{i+1}(e_0)) = t_f.$$

From V_P^{RC} 's definition, we get that $\gamma \xrightarrow{\delta} \gamma'$ as now $((\ominus, \rho_i(e), \mathcal{S}), \delta, (\square, \rho_{i+1}(e), \mathcal{S}')) \in \Delta_t^{\text{sum}}$. Observe that if $\lambda_{i+1} = \lambda'$ then $\gamma' \in \mathbf{repr}^{\text{RC}}(\rho, i+1)$ and we are done. Otherwise, we define $\gamma_{i+1} = ((\ominus, \rho_{i+1}(e_0), \mathcal{S}'), v_{i+1})$. As $\lambda_{i+1} = (\mathcal{S}', v_{i+1}) \in \mathbf{repr}(\rho, i+1)$, we get that $\gamma_{i+1} \in \mathbf{repr}^{\text{RC}}(\rho, i+1)$. Recall that (from the proof of Lemma 7.4):

$$(\mathcal{S}', v') \xrightarrow{\delta_1}_{V_P} (\mathcal{S}', v^1) \xrightarrow{\delta_2}_{V_P} \dots \xrightarrow{\delta_k}_{V_P} (\mathcal{S}', v_{i+1}) = \lambda_{i+1}$$

for some $k \in \mathbb{N}$, and with $(\mathcal{S}', \delta_j, \mathcal{S}') \in \Delta_e$ for all $j \in [1, k]$. Hence, by definition of V_P^{RC} , we get that $((\ominus, \rho_{i+1}(e_0), \mathcal{S}'), \delta_j, (\ominus, \rho_{i+1}(e_0), \mathcal{S}')) \in \Delta'_e$, and so, if $\gamma' = ((\ominus, \rho_{i+1}(e_0), \mathcal{S}'), v')$ we get:

$$\gamma' \xrightarrow{0_X} ((\ominus, \rho_{i+1}(e_0), \mathcal{S}'), v') \xrightarrow{\delta_1}_{V_P} \xrightarrow{\delta_2}_{V_P} \dots \xrightarrow{\delta_k}_{V_P} ((\ominus, \rho_{i+1}(e_0), \mathcal{S}'), v_{i+1}) = \gamma_{i+1}$$

and otherwise

$$\gamma' = ((\ominus, \rho_{i+1}(e_0), \mathcal{S}'), v') \xrightarrow{\delta_1}_{V_P} \xrightarrow{\delta_2}_{V_P} \dots \xrightarrow{\delta_k}_{V_P} ((\ominus, \rho_{i+1}(e_0), \mathcal{S}'), v_{i+1}) = \gamma_{i+1}.$$

Observe that if e_0 fires t_f between ρ_i and ρ_{i+1} , then $(\rho_i(e_0), ?a, (\rho_{i+1}(e_0))) = t_f$ with $t = (p, !!a, p')$ for some $p, p' \in Q$, and γ' is an $\ominus$ -configuration.

- *Second case: $e = e_0$.* For this case, we will need to dive deeper into Lemma 7.4's proof. We now let $t = (p, !!a, p')$ be the transition such that $\rho_i \xrightarrow{e_0, t} \rho_{i+1}$. As $\gamma \in \mathbf{repr}^{\text{RC}}(\rho, i)$, $\gamma = (\ominus, p, \mathcal{S})$ (because $\rho_i(e_0) = p$).

Let $\mathcal{S} = \{S_1, \dots, S_k\}$. We recall that

$$E_{q_a, j}^{\rho^{\text{conf}}, i} = \{e \in [1, \#\text{proc}(\rho) - 1] \mid \rho_i(e) \subseteq Q_W \text{ and } \mathbf{na}(\rho^{\text{conf}}, i, e) = (q_a, j)\},$$

for $q_a \in Q_A$ and $j \in \mathbf{na-index-set}(\rho^{\text{conf}}, i)$. As $(\mathcal{S}, v) \in \mathbf{repr}(\rho^{\text{conf}}, i)$, for all $q_a \in Q_A$, there is an injective function $r_{q_a} : \mathbf{na}(\rho^{\text{conf}}, i) \rightarrow [1, |Q_W| + 1]$ such that:

$$\text{-- CondRepr2: } \mathcal{S} = \{(\rho_i(E_{q_a, j}^{\rho^{\text{conf}}, i}), q_a, r_{q_a}(j)) \mid q_a \in Q_A, j \in \mathbf{na-index-set}(\rho^{\text{conf}}, i, q_a)\}$$

- **CondRepr3**: for all $q_a \in Q_A$, we have $v(x_{(q_a, r_{q_a}(j))}) = |E_{q_a, j}^{\rho^{\text{conf}}, i}|$ for all $j \in \mathbf{na-index-set}(\rho^{\text{conf}}, i, q_a)$ and $v(x_{(q_a, k)}) = 0$ for all $k \notin r_{q_a}(\mathbf{na-index-set}(\rho^{\text{conf}}, i, q_a))$.

As in Lemma 7.4's proof, for all $S = (pr, q_a, k) \in \mathcal{S}$, we let $j = r_{q_a}^{-1}(k)$ and we define

$$S^{+1} = \begin{cases} (\rho_{i+1}(E_{q_a, j}^{\rho^{\text{conf}}, i}), q_a, k) & \text{if } C_{i+1}(E_{q_a, j}^{\rho^{\text{conf}}, i}) \subseteq Q_W \\ \text{Done} & \text{otherwise.} \end{cases}$$

S^{+1} represents the set of states reached by the states in the print of S after the broadcast of a in ρ^{conf} . We then let

$$\mathcal{S}_0 = \{S^{+1} \mid S \in \mathcal{S}\} \setminus \{\text{Done}\}.$$

In Lemma 7.4's we proved that $\mathcal{S}_0$ is a coherent set of summaries (Claim 7.1), and for all $S \in \mathcal{S}$, $S \xrightarrow{t} S^{+1}$ (Claim 7.3). Hence,

$$((\ominus, p, \mathcal{S}), \mathbf{0}_X, (\square, p', \mathcal{S}_0)) \in \Delta_t^{\text{proc}}$$

where,

$$\square = \begin{cases} \ominus & \text{if } (p, !!a, !p') = t_f \\ \ominus & \text{otherwise.} \end{cases}$$

We let $\gamma' = ((\square, p', \mathcal{S}_0), v)$. It holds that: $\gamma \xrightarrow{\mathbf{0}_X} \gamma'$ and if γ' is an $\ominus$ -configuration, then e_0 fired t_f between ρ_i and ρ_{i+1} .

As in Lemma 7.4's proof, γ' is not necessarily in $\mathbf{repr}^{\text{RC}}(\rho, i+1)$ as we should empty counters labeled by summaries which have arrived between ρ_i and ρ_{i+1} , so that we get a configuration in $\mathbf{repr}^{\text{RC}}(\rho, i+1)$. To do this, we define γ_{i+1} in the same fashion as done in Lemma 7.4. We let $\gamma_{i+1} = ((\ominus, p', \mathcal{S}_0), v_{i+1})$ with v_{i+1} defined as follows: for all $q_a \in Q_A$,

- if $i+1 \in \mathbf{na-index-set}(\rho^{\text{conf}}, i, q_a)$, let $k = r_{q_a}(i+1)$ and $v_{i+1}(x_{q_a}) = v'(x_{q_a}) + v'(x_{q_a, k}) = v(x_{q_a}) + v(x_{q_a, k})$ and $v_{i+1}(x_{q_a, k}) = 0$.
- Otherwise, $v_{i+1}(x_{q_a}) = v'(x_{q_a}) = v(x_{q_a})$.

For all other counter x , $v_{i+1}(x) = v'(x) = v(x)$.

We now need to prove that $(\mathcal{S}_0, v_{i+1}) \in \mathbf{repr}(\rho^{\text{conf}}, i+1)$. Proofs of Claims 7.5, 7.6 and 7.7 (proving that $(\mathcal{S}', v_{i+1})$ respects **CondRepr1**, **CondRepr2** and **CondRepr3**, with respect to ρ^{conf} and hence that $(\mathcal{S}', v_{i+1}) \in \mathbf{repr}(\rho^{\text{conf}}, i+1)$) can be easily adapted to our case by only keeping the cases where the process e_q (the one broadcasting a message in the old proof) is not considered, nor the transition broadcasting the message.

As $\rho_{i+1}(e_0) = p'$, we get that $\gamma_{i+1} \in \mathbf{repr}^{\text{RC}}(\rho, i+1)$.

Finally, in Lemma 7.4's proof, we proved that

$$(\mathcal{S}', v') \xrightarrow{\delta_1}_{V_P} (\mathcal{S}', v^1) \xrightarrow{\delta_2}_{V_P} \dots \xrightarrow{\delta_k}_{V_P} (\mathcal{S}', v_{i+1}) = \lambda_{i+1}$$

where $\delta_j \in \Delta_e$ for all $1 \leq j \leq k$. Hence, by definition of V_P^{RC} , we get that $((\ominus, \rho_{i+1}(e_0), S'), \delta_j, (\ominus, \rho_{i+1}(e_0), S')) \in \Delta'_e$, and so, if $\gamma' = ((\ominus, \rho_{i+1}(e_0), S'), v')$ we get:

$$\gamma' \xrightarrow{0_X} ((\ominus, \rho_{i+1}(e_0), S'), v') \xrightarrow{\delta_1} \xrightarrow{\delta_2} \dots \xrightarrow{\delta_k} ((\ominus, \rho_{i+1}(e_0), S'), v_{i+1}) = \gamma_{i+1}$$

and otherwise

$$\gamma' = ((\ominus, \rho_{i+1}(e_0), S'), v') \xrightarrow{\delta_1} \xrightarrow{\delta_2} \dots \xrightarrow{\delta_k} ((\ominus, \rho_{i+1}(e_0), S'), v_{i+1}) = \gamma_{i+1}.$$

Hence, we have built γ' such that $\gamma_i \rightsquigarrow^* \gamma \rightsquigarrow \gamma' \rightsquigarrow^* \gamma_{i+1}$ with $\gamma_{i+1} \in \mathbf{repr}^{\text{RC}}(\rho, i+1)$ and if e_0 fires t_f between and ρ_i and ρ_{i+1} , γ' is a $\ominus$ -configuration. $\square$

We can now extend the previous local lemma to prove the completeness of our reduction.

Lemma 7.17. *If there is an infinite execution of P in which a process fires t_f infinitely often, then there exists an infinite execution of V_P^{RC} such that there exists $q \in Q$, $S \in \text{CoSets}$ and $(\ominus, q, S)$ is visited infinitely often.*

Proof. If there is an infinite execution of P in which a process fires t_f infinitely often, then, from Corollary 7.3, there exists an infinite *almost well-formed* execution $\rho = C_0 C_1 \dots$ of P , where nonconformist process e_0 fires infinitely often t_f . W.l.o.g we suppose that $e_0 = \# \mathbf{proc}(\rho)$. Let $n = \# \mathbf{proc}(\rho) - 1$, we can assume w.l.o.g. that $n \geq 1$. Then consider the following prefix of execution of V_P^{RC} :

$$(s_0, \mathbf{0}_X) \rightsquigarrow \dots \rightsquigarrow (s_0, v) \rightsquigarrow ((\ominus, q_{\text{init}}, \emptyset), v_{\text{init}})$$

where $v(x_{q_{\text{init}}}) = n - 1$, $v_{\text{init}}(x_{q_{\text{init}}}) = n$ and $v(x) = v_{\text{init}}(x) = 0$ for all other counter x . As we assumed $\# \mathbf{proc}(\rho) \geq 2$, we have that $v_{\text{init}}(x_{q_{\text{init}}}) \geq 1$. Hence $(\emptyset, v_{\text{init}}) \in \mathbf{repr}(\rho^{\text{conf}}, 0)$, and so it holds that $((\ominus, q_{\text{init}}, \emptyset), v_{\text{init}}) \in \mathbf{repr}^{\text{RC}}(\rho, 0)$. For all $i \in \mathbb{N}$, we build a configuration $\gamma_i \in \mathbf{repr}^{\text{RC}}(\rho, i)$ such that $(s_0, \mathbf{0}_X) \rightsquigarrow^* \gamma_0 \rightsquigarrow^+ \gamma_1 \rightsquigarrow^+ \dots \rightsquigarrow^+ \gamma_i$ and if $i > 0$ and e_0 fires t_f between C_{i-1} and C_i , then $\gamma_{i-1} \rightsquigarrow^+ \gamma_i^{\ominus} \rightsquigarrow^* \gamma_i$. For $i = 0$, it suffices to consider $\gamma_0 = ((\ominus, q_{\text{init}}, \emptyset), v_{\text{init}})$. Let $i \in \mathbb{N}$ and assume we have built such a $\gamma_i \in \mathbf{repr}^{\text{RC}}(\rho, i)$. then from Lemma 7.16, we get that there exists $\gamma_{i+1} \in \mathbf{repr}^{\text{RC}}(\rho, i+1)$ such that $(s_0, \mathbf{0}_X) \rightsquigarrow^* \gamma_0 \rightsquigarrow^+ \gamma_i \rightsquigarrow^+ \gamma_{i+1}$. Furthermore, if e_0 fires t_f between C_i and C_{i+1} , then $\gamma_{i+1} \rightsquigarrow^+ \gamma_{i+1}^{\ominus} \rightsquigarrow^* \gamma_{i+1}$.

Hence, we have built an infinite execution of V_P^{RC} visiting infinitely often locations with a $\ominus$. In particular, as there is a finite number of $\ominus$ locations, there must exist $q \in Q$, and $S \in \text{CoSets}$ such that $(\ominus, q, S)$ is visited infinitely often. $\square$

7.6.1.5 Conclusions

As we have proved completeness and soundness of our construction we get the following result.

Theorem 7.6. *REP-COVER for Wait-Only broadcast protocols is in EXPSPACE.*

Proof. Given a protocol P and a transition t_f , thanks to Lemmas 7.15 and 7.17, we get that there exists an execution in which a process fires infinitely often t_f if and only if there exists a location $(\odot, q, S)$ in V_P^{RC} for which there is an infinite execution visiting infinitely often $(\odot, q, S)$. The procedure to decide REP-COVER is then the following: in V_P^{RC} , guess a state $q \in Q$ and a coherent set $S \in \text{CoSets}$ solve the repeated coverability problem for V and $\ell_f = (\odot, q, S)$.

Observe that $|\text{Loc}'| = 1 + 2 \times |Q| \times |\text{CoSets}|$, and that $|\Delta'| \leq 2 + |\text{Loc}'|^2 \times (2|X| + 1)$ as each transition between two locations either decrements a counter and increments another, or does not change the counters' values. Hence, using Theorem 7.5, the nondeterministic procedure takes $O(\log_2(2) + \log_2(1 + 2 \times |Q| \times |\text{CoSets}|))2^{c(|Q|+1)^2 \log_2((|Q|+1)^2)}$ space, i.e. $O((|T| + 1)^3 2^{c(|T|+2)^3})$ space (recall that $|\text{CoSets}| \leq 2^{|Q|^2 2^{|Q|}}$ and we assumed $|Q| \leq |T| + 1$, see Remark 10). The overall procedure is then in $\text{NEXPSPACE} = \text{EXPSPACE}$. Hence, we proved the theorem. $\square$

7.6.2 Lower bound

We now prove that the REP-COVER problem for Wait-Only broadcast protocols is PSPACE-hard. In order to do that, we reduce the intersection non-emptiness problem for deterministic finite automata, which is known to be PSPACE-complete [Koz77]. We already reduced this problem to the CONF-COVER^{nb} problem for Wait-Only protocols in Chapter 6. We give again the formal tools to present this reduction.

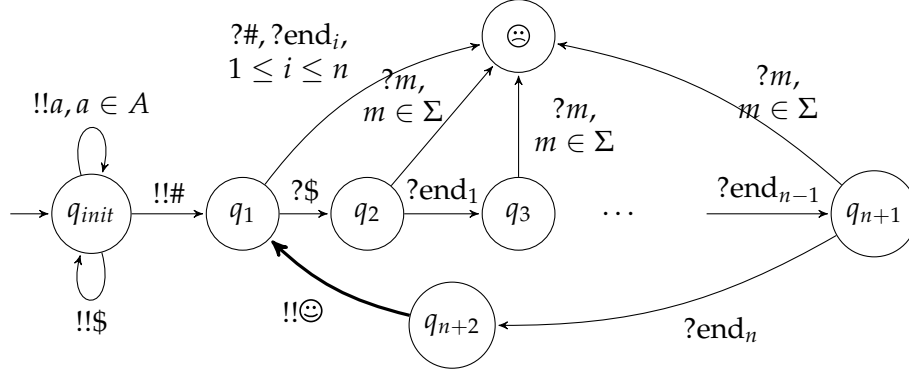
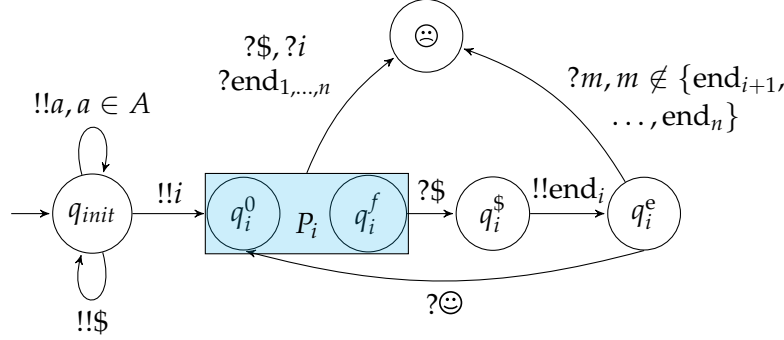
AUTOMATA. Let $\mathcal{A}_1, \dots, \mathcal{A}_n$ be a list of deterministic finite and *complete* automata with $\mathcal{A}_i = (A, Q_i, q_i^0, \{q_i^f\}, \Delta_i)$ for all $1 \leq i \leq n$. Observe that we restrict our reduction to automata with a unique accepting state, which does not change the complexity of the problem. We denote by A^* the set of words over the finite alphabet A and Δ_i^* the function extending Δ_i to Σ^* , i.e. for all $q \in Q_i$, $\Delta_i^*(q, \varepsilon) = q$, and for all $w \in A^*$ and $a \in A$, $\Delta_i^*(q, wa) = \Delta_i(\Delta_i^*(q, w), a)$. For an automaton $\mathcal{A} = (A, Q_i, q_i^0, \{q_i^f\}, \Delta_i)$, we let $\mathcal{L}(\mathcal{A})$ be the language of automaton $\mathcal{A}$, i.e. $\mathcal{L}(\mathcal{A}) = \{w \mid \Delta^*(q_i^0, w) = q_i^f\}$ where q_i^0 is its initial state and q_i^f its final state.

THE REDUCTION. We define a protocol $P = (Q, \Sigma, q_{\text{init}}, T)$ that consists of several components. For each automaton $\mathcal{A}_i$, P contains one component pictured on Figure 7.16. Here, the blue box $P_i = (Q^i, T^i)$ is defined as follows:

- Q^i is the set of states of automaton i (i.e. $Q^i = Q_i$) and
- $T^i = \{(q_1^i, ?a, q_2^i) \mid (q_1^i, a, q_2^i) \in \Delta_i\}$.

The other main component, shown in Figure 7.15, includes the transition $t_f = (q_{n+2}, !!\odot, q_1)$, that must be taken infinitely often. Any process taking t_f infinitely often will also receive infinitely often the sequence of messages $\$ \cdot \text{end}_1 \cdot \text{end}_2 \dots \text{end}_n$. Each message end_i is broadcast from the component corresponding to $\mathcal{A}_i$ (see Figure 7.16).

If there is a word $w = a_1 \dots a_n$ in the intersection of the languages of the n automata, the execution of P should start by putting one process in q_1 , one process in q_{init} , and one process in each of the q_i^0 . Then, the process in q_{init} broadcasts the sequence of messages $a_1 \dots a_n$, received by all the processes in the parts simulating the automata. Since the word is accepted by all the automata, they all reach q_i^f together. Then, the process in q_1 broadcasts $\$$,


 Figure 7.15: The part of P with transition $t_f = (q_{n+2}, !!\odot, q_1)$.

 Figure 7.16: The part of P simulating automaton i .

receives the sequence of $\text{end}_1 \dots \text{end}_n$, and broadcasts $\odot$, that gathers all the processes that are q_i^e in q_i^0 . This can be repeated an infinite number of times, and t_f will occur infinitely often.

Formally, we get the following lemma.

Lemma 7.18. If there exists a word $w \in \bigcap_{1 \leq i \leq n} \mathcal{L}(\mathcal{A}_i)$, then there exists an infinite execution $\rho \in \text{Exec}_\omega[P]$ and $e \in [1, \#\text{proc}(\rho)]$ such that for all $i \in \mathbb{N}$, there exists $j > i$ such that $\rho_j \xrightarrow{e, (q_{n+2}, !!\odot, q_1)} \rho_{j+1}$.

Proof. We denote $w = a_1 \dots a_k$. Let C_0 be the initial configuration with $n + 2$ processes. We form the following prefix of execution:

$$C_0 \xrightarrow{1, (q_{\text{init}}, !!1, q_1^0)} C_1 \xrightarrow{2, (q_{\text{init}}, !!2, q_2^0)} \dots \xrightarrow{n, (q_{\text{init}}, !!n, q_n^0)} C_n \xrightarrow{n+1, (q_{\text{init}}, !!\#, q_1)} C_{n+1}$$

C_{n+1} is such that: $C_{n+1}(i) = q_i^0$ for all $i \in [1, n]$, and $C_{n+1}(n+1) = q_1$ and $C_{n+1}(n+2) = q_{\text{init}}$. Next, consider the following sequence of configurations:

$$\begin{aligned} C_{n+1} &\xrightarrow{n+2, (q_{\text{init}}, !!a_1, q_{\text{init}})} C_{n+2} \xrightarrow{n+2, (q_{\text{init}}, !!a_2, q_{\text{init}})} \dots \xrightarrow{n+2, (q_{\text{init}}, !!a_k, q_{\text{init}})} C_{n+k+1} \\ &\xrightarrow{n+2, (q_{\text{init}}, !!\$, q_{\text{init}})} C_{n+k+2} \xrightarrow{1, (q_1^{\$}, !!\text{end}_1, q_1^e)} \dots \xrightarrow{n, (q_n^{\$}, !!\text{end}_n, q_n^e)} C_{2n+k+2} \\ &\xrightarrow{n+1, (q_{n+2}, !!\odot, q_1)} C_{2n+k+3} \end{aligned}$$

As $a_1 \dots a_k = w \in \bigcap_{1 \leq i \leq n} \mathcal{L}(\mathcal{A}_i)$, it holds that: $C_{n+k+1}(i) = q_i^f$ for all $i \in [1, n]$. Hence, $C_{n+k+2}(i) = q_i^{\$}$ for all $i \in [1, n]$. Furthermore, $C_{n+k+2}(n+1) = q_2$. We

conclude that $C_{2n+k+3} = C_{n+1}$. Hence, we can form an infinite execution $\rho = C_0 C_1 \dots C_{n+1} \dots C_{2n+k+3} \dots C_{3n+k+5} \dots$ such that for all $i \in \mathbb{N}$, $C_{n+1+i(n+k+2)} = C_{n+1}$ and $C_{n+i(n+k+2)} \xrightarrow{n+1, (q_{n+2}, !!\odot, q_1)} C_{n+1+i(n+k+2)}$, i. e. the process $n+1$ fires t_f , which concludes the proof. $\square$

Conversely, assume that the transition t_f occurs infinitely often. At the beginning of the execution, it is possible that the automata are not faithfully simulated, because several processes populate the blue zone of some P_i . A process can then reach the state q_i^f with a different word than the others. However, this can happen only a finite number of times in an infinite execution, because every time a new process reaches the blue zone, the processes that already populate it are sent to the state $\odot$ (by receiving message i), which is a sink state. Also, only one process at a time populates the zone pictured in Figure 7.15. So in an execution where t_f occurs infinitely often, there exists a process e_0 that takes it infinitely often, and eventually no process reaches $\odot$. Once we have reached this point, when e_0 takes transition t_f , there is exactly one process in q_i^0 for all $1 \leq i \leq n$. When $\$$ is broadcast, all the processes have received the same word, and they are all in a final state of their respective automaton (unless they go to $\odot$, contradicting our assumption). So the word is in the intersection of languages.

Lemma 7.19. If there exists an infinite execution $\rho \in \text{Exec}_\omega[P]$ and $e \in [1, \#\text{proc}(\rho)]$ such that for all $i \in \mathbb{N}$, there exists $j > i$ such that $\rho_j \xrightarrow{e, (q_{n+2}, !!\odot, q_1)} \rho_{j+1}$, then there exists $w \in \bigcap_{1 \leq i \leq n} \mathcal{L}(\mathcal{A}_i)$.

Proof. Denote ρ such an execution. We start by proving the following lemma.

Lemma 7.20. There exists $i_0 \in \mathbb{N}$, from which no new processes reach state $\odot$, in other words, for all process e , if $\rho_{i_0}(e) \neq \odot$ then $\rho_i(e) \neq \odot$ for all $i > i_0$.

Proof. We reason by contradiction. Otherwise, for all $i_0 \in \mathbb{N}$, either all processes are on $\odot$, or there exists an process not in $\odot$ who reach it later. As $\odot$ is a sink state and there is a finite number of processes, eventually all processes are on $\odot$, leading to a deadlock configuration, which contradicts the fact that ρ is infinite. $\square$

We now denote e_0 a process taking infinitely often transition $(q_{n+2}, !!\odot, q_1)$. We easily see that for all $i \in \mathbb{N}$, $\rho_i(e_0) \neq \odot$, as otherwise, $\rho_j(e_0) = \odot$ for all $j > i$. Consider now $i_1, i_2, i_3 \in \mathbb{N}$ three indexes such that $i_0 < i_1 < i_2 < i_3$, and

$$\rho_{i_j} \xrightarrow{e_0, (q_{n+2}, !!\odot, q_1)} \rho_{i_{j+1}} \text{ for } j \in \{1, 2, 3\}.$$

By construction of the protocol, there exist $e_1, \dots, e_n$ which respectively broadcast messages $\text{end}_1, \dots, \text{end}_n$ which are received by e_0 between i_1 and i_2 . Observe that from the protocol's construction, once process e_0 is on state q_2 , the only sequence of broadcast

messages that does not put e_0 (before it reaches q_{n+2}) in $\ominus$ is $\text{end}_1 \cdot \text{end}_2 \cdots \text{end}_n$. Meaning that there exists $i_1 < i_4 < i_2$ such that $\rho_{i_4}(e_0) = q_2$ and:

$$\rho_{i_4} \xrightarrow{e_1, (q_1^{\$}, !!\text{end}_1, q_1^e)} \xrightarrow{e_2, (q_2^{\$}, !!\text{end}_2, q_2^e)} \cdots \xrightarrow{e_n, (q_n^{\$}, !!\text{end}_1, q_n^e)} \rho_{i_4+n}$$

as any other behavior would put e_0 in $\ominus$. Observe that for all $i \in [1, n]$, $\rho_{i_4+n}(e_i) = q_i^e$. Note that $i_4 + n = i_2$ as if $\rho_{i_4+n} \xrightarrow{(q, !!m, q')} \rho_{i_4+n+1}$ with $m \neq \ominus$, then $\rho_{i_4+n}(e_n) = \ominus$. Hence:

$$\begin{aligned} \rho_{i_2}(e_i) &= q_i^0 \text{ for all } i \in [1, n] \\ \rho_{i_2}(e_0) &= q_1 \end{aligned}$$

Denote i_5 the next index from which $\$$ is broadcast (it exists as, in particular, e_0 needs to take the cycle with $(q_2, ?\text{end}_1, q_3)$).

As $i_3 > i_0$, no process e_i reaches $\ominus$, hence $\rho_{i_5}(e_i) = q_i^f$ as otherwise, $\rho_{i_5+1}(e_i) = \ominus$ for all $i \in [1, n]$.

Observe that between $i_3 + 1$ and i_5 , the only broadcasts are of the form $(q_{\text{init}}, !!a, q_{\text{init}})$ with $a \in \Sigma$ as any other broadcast would put e_0 in $\ominus$.

Hence, the sequence of letters $a \in \Sigma$ broadcast between $i_3 + 1$ and i_5 is received by all processes $e_1, \dots, e_n$ (as automata are complete), and thanks to the protocol's construction, it forms a word accepted by all automata. $\square$

The two previous lemmas prove that the reduction is sound and complete, leading to the following result.

Theorem 7.7. *REP-COVER with Wait-Only broadcast protocols is PSPACE-hard.*

7.7 CONCLUSIONS

In this chapter, we investigated the Wait-Only restriction in broadcast protocols. While the state and configuration coverability problems were already known to be PTIME-complete and PSPACE-complete, respectively (as shown in Chapter 6), we complemented these results by providing a generic VASS construction simulating Wait-Only broadcast networks. This led to several new results, summarized in Table 7.1, including (sometimes matching) lower bounds. In summary, the Wait-Only restriction significantly reduces the complexity of coverability problems. However, for other verification problems, the complexity remains high. In the next chapter, we take this idea one step further by introducing a coarser restriction: each waiting state is allowed to receive only one message. This new setting aims to yield more reasonable complexity for problems beyond coverability.

	SYNCHRO ^{nb}		REP-COVER ^{nb}
Rdv Networks (RDV-protocol)	Undecidable (Chapter 4)		Undecidable [EK03; EFM99]
Broadcast Networks (B-protocol)			
Networks			
Wait-Only Rdv Networks (Wait-Only RDV-protocol)	Undecidable (Chapter 6)		?
Wait-Only Networks (Wait-Only protocols)			
Wait-Only Broadcast Networks (Wait-Only B-protocol)	<u>$q_f \in Q_W$:</u> Ackermann-c.	<u>$q_f \in Q_A$:</u> EXPSPACE-c.	in EXPSPACE and PSPACE-hard

Table 7.1: Results presented in this chapter are written in red.

ONE STEP FURTHER: SINGLE WAIT-ONLY NETWORKS

8.1 INTRODUCTION

In this chapter, we build on the *Wait-Only* restriction introduced in Chapter 6 and studied in the previous two chapters, and introduce a stronger restriction: Single Wait-Only (SWO) protocols. A protocol is said to be Single Wait-Only if, in addition to being Wait-Only, each of its waiting states has exactly one outgoing reception transition.

We now give the formal definition.

Definition 8.1 (Single Wait-Only protocols). A protocol $P = (Q, \Sigma, q_{init}, T)$ is a *Single Wait-Only* (SWO) protocol if it is Wait-Only, with sets of action states Q_A and waiting states Q_W , and:

$$\text{for all } q \in Q_W, |\{(q, ?a, q') \in T \mid a \in \Sigma, q' \in Q\}| = 1.$$

We believe that this restriction remains faithful to our initial motivation: typically, a thread waits for exactly one type of message in order to enter a critical section.

In the previous two chapters, we established the following results for Wait-Only protocols:

- State coverability is PTIME-complete, while configuration coverability is PSPACE-complete.
- The synchronization problem remains undecidable in general, but becomes decidable (although Ackermann-complete) when restricted to Wait-Only broadcast networks. Furthermore, when considering synchronization with an action final state, the problem becomes EXPSpace-complete.
- The repeated coverability problem, restricted to Wait-Only broadcast protocols, lies in EXPSpace and is PSPACE-hard.

In this chapter, we focus on synchronization and repeated coverability, aiming to identify restrictions under which these problems have more reasonable complexities. In this context, we establish the following results for SWO protocols:

- Repeated coverability is in PTIME.
- Synchronization with an action final state is PTIME-complete for both broadcast and rendez-vous networks.
- Synchronization with a waiting final state is Ackermann-hard, even when restricted to rendez-vous networks. The decidability status remains open.

In SWO protocols, once a process reaches a waiting state, its future is *deterministic* until it reaches an action state: only one reception path is possible. All the upper bounds established in this chapter rely on this observation. In the repeated coverability problem, we do not need to track precisely which processes are in which waiting states; it suffices to ensure that a given transition is fired infinitely often, though we keep track precisely of processes on action states. If the transition is an action, it is enough to ensure that the necessary reception paths are eventually traversed. If it is a reception transition, checking that the corresponding reception path is eventually taken is sufficient. Due to the unique reception sequence, it guarantees that the reception transition has indeed been fired. When synchronization is restricted to an *action* final state, this determinism also helps: we must ensure that all processes eventually reach the final state, but we can afford to *delay* receptions during the execution. A process in a waiting state has a single way out, so postponing a reception (from time t to $t + \delta$) does not affect the process behavior: it will still move to the same next state, and no other state can be reached in the meantime.

In contrast, synchronization with a *waiting* final state is significantly more difficult. In the final segment of the execution, receptions can no longer be delayed: doing so might allow a process to *avoid* crossing the final state, although it should have. This breaks the guarantee that all processes synchronize in the final state.

We start by defining new notions related to SWO protocols and prove key properties in Section 8.2. Then, in Section 8.3, we show that the repeated coverability problem is in PTIME. Our proof relies on a reduction to a VASS problem: the existence of a cycle $(\ell, v) \rightsquigarrow^+ (\ell, v')$ for two vectors $v' \geq v$.

In Section 8.4, we study the synchronization problem with an action final state and establish its PTIME-completeness for both rendez-vous and broadcast networks. For rendez-vous networks, we reduce to *blocking* rendez-vous networks (where sending steps are not allowed). As established in Chapter 2, the problem is then in PTIME. For broadcast networks, we reduce to *reconfigurable broadcast networks* (RBN), where a broadcast message is received by a *subset* of processes. RBNs were introduced in [DSZ10] and have been extensively studied since, with many verification problems shown to have reasonable complexity [DSZ10; Del+12; BGW22; BW21; BFS15; CMS19]. In particular, synchronization on RBNs is known to be in PTIME, as established in [Fou15].

Finally, in Section 8.5, we show that when considering the synchronization problem for SWO protocols with a waiting final state, the problem is Ackermann-hard, already for rendez-vous networks.

8.2 PRELIMINARIES

We start by giving an example in Figure 8.1.

In SWO protocols, the behavior of a process on a waiting state is deterministic until the next action state it may reach (if any). That is, each process on a waiting state q follows a unique sequence of reception transitions leading either to an action state, or back to q , and in the latter case, forming a reception cycle from which a process cannot escape.

We formalize this by defining the set of future states of a waiting state as the set of states it must visit before reaching an action state or looping back to itself.

Definition 8.2. Let $P = (Q, \Sigma, q_{init}, T)$ be a SWO protocol, with states partitioned into action states Q_A and waiting states Q_W . For a waiting state $q \in Q_W$, we define $\text{future}(q)$ as the set

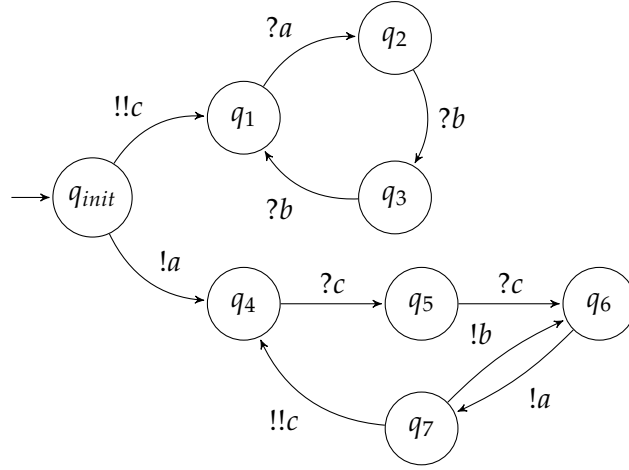


Figure 8.1: Example of a Single Wait-Only protocol P . Its action states are q_{init} , q_6 , and q_7 , and its waiting states are q_1, q_2, q_3, q_4 , and q_5 . Observe that each waiting state has exactly one outgoing reception transition.

of states visited along the unique reception path starting from q , until an action state (if any) is reached.

Formally, $\text{future}(q)$ is the smallest set such that:

- $q \in \text{future}(q)$, and
- if $p \in \text{future}(q)$ and $(p, ?a, p') \in T$, then $p' \in \text{future}(q)$.

Note that in Single Wait-Only protocols, $|\text{future}(p) \cap Q_A| \leq 1$ for all $p \in Q_W$, since action states have no outgoing reception transitions, and waiting states have at most one.

Example 8.1. Returning to the protocol P from Figure 8.1, we have:

- $\text{future}(q_4) = \{q_4, q_5, q_6\}$,
- $\text{future}(q_5) = \{q_5, q_6\}$,
- $\text{future}(q_1) = \text{future}(q_2) = \text{future}(q_3) = \{q_1, q_2, q_3\}$.

Processes reaching q_1 , q_2 , or q_3 are trapped in a reception cycle, and thus their future does not include any action state. In contrast, the futures of q_4 and q_5 include the action state q_6 , which is the next action reachable from those states.

We now extend this notion of future to configurations.

Definition 8.3. Let $P = (Q, \Sigma, q_{init}, T)$ be a SWO protocol, with $Q = Q_A \uplus Q_W$. For a configuration C of P , we define **Future**(C) as the set of configurations in which all processes are in the same or more advanced positions along their respective reception paths. Formally, $C' \in \text{Future}(C)$ if $|C'| = |C|$ and, for all $e \in [1, |C|]$:

$$C'(e) = \begin{cases} C(e) & \text{if } C(e) \in Q_A, \\ p \in \text{future}(C(e)) & \text{if } C(e) \in Q_W. \end{cases}$$

These notions will play a crucial role throughout this chapter. All the upper bounds we present will rely on various abstractions that compute (not necessarily reachable)

configurations whose future set contains reachable ones. In particular, if we can show that there is a reachable configuration in the future set of a configuration C such that $C^{-1}(Q_W) = \emptyset$, then C itself is reachable, since in that case $\text{Future}(C) = \{C\}$. We will exploit this property to study the synchronization problem restricted to an action final state.

We now establish that this future relation is transitive, both at the level of states and configurations.

Lemma 8.1. The future relation is transitive:

- If $p \in \text{future}(q)$ and $q \in \text{future}(s)$, then $p \in \text{future}(s)$.
- If $C \in \text{Future}(C')$ and $C' \in \text{Future}(C'')$, then $C \in \text{Future}(C'')$.

Proof. Directly follows from the definitions of $\text{future}(\cdot)$ and $\text{Future}(\cdot)$. $\square$

Example 8.2. Consider the configuration $C = (q_{\text{init}}, q_2, q_5, q_7, q_6)$. Then any configuration $C' = (q_{\text{init}}, \square, \triangle, q_7, q_6)$ such that $\square \in \text{future}(q_2) = \{q_1, q_2, q_3\}$ and $\triangle \in \{q_5, q_6\}$ belongs to $\text{Future}(C)$. For instance,

$$C' = (q_{\text{init}}, q_2, q_6, q_7, q_6) \in \text{Future}(C).$$

Furthermore, let $C'' = (q_{\text{init}}, q_3, q_5, q_7, q_4)$. Then:

- $q_2 \in \text{future}(q_2)$,
- $q_5 \in \text{future}(q_5)$,
- $q_6 \in \text{future}(q_4)$,

hence $C \in \text{Future}(C'')$. By transitivity, we also get $C' \in \text{Future}(C'')$.

The next lemma shows that being in the future of a configuration preserves the same possible actions. Moreover, communication steps maintain the future relation: if $C_1 \rightarrow C'_1$ and $C_2 \in \text{Future}(C_1)$, then there exists C'_2 such that $C_2 \rightarrow C'_2$ and $C'_2 \in \text{Future}(C'_1)$.

Intuitively, since processes on action states are preserved between C_1 and C_2 , any action available from C_1 is as well from C_2 . And since the processes in C_2 are either at the same or further positions along their reception paths, they either receive the same messages as in C_1 or are in a further position already.

This result is particularly meaningful in the presence of reception cycles. In such cases, $\text{future}(q)$ may include the entire cycle, so $C'_2 \in \text{Future}(C'_1)$ does not necessarily imply that the same receptions have occurred. The lemma ensures that, whenever possible, reception steps are preserved.

Lemma 8.2. Let $P = (Q, \Sigma, q_{\text{init}}, T)$ be a SWO protocol and $C_1, C'_1 \in \mathcal{C}$, such that $C_1 \xrightarrow{e,t} C'_1$ for a process $e \in [1, |C|]$ and a transition $t \in T$. For any $C_2 \in \text{Future}(C_1)$, there exists $C'_2 \in \text{Future}(C'_1)$ such that $C_2 \xrightarrow{e,t} C'_2$. Furthermore, for all $t' = (q, ?a, q') \in T$, such that t' is fired between C_1 and C'_1 , then:

- t' is also fired between C_2 and C'_2 , or
- $C_2^{-1}(q) = \emptyset$.

Proof. Let $C_1, C'_1 \in \mathcal{C}$, such that $C_1 \xrightarrow{e_1, t} C'_1$ for a process $e_1 \in [1, |C|]$ and a transition $t = (q, \alpha, q') \in T$ with $\alpha \in !\Sigma \cup !!\Sigma$. By definition, $q \in Q_A$. Let $C_2 \in \text{Future}(C_1)$, then $C_2(e_1) = C_1(e_1) = q$. Hence, t can be fired. We now explain how we define C'_2 .

1. if $\alpha = !!a$, then we define C'_2 as follows:

$$C'_2(e) = \begin{cases} q' & \text{if } e = e_1 \\ C_2(e) & \text{if } C_2(e) \in Q_A \text{ and } e \neq e_1 \\ C_2(e) & \text{if } C_2(e) \in Q_W \text{ and } a \notin R(C_2(e)) \\ p & \text{if } (C_2(e), ?a, p) \in T \end{cases}$$

It is clear from C'_2 definition that $C_2 \xrightarrow{e_1, t} C'_2$. Furthermore, for all $t' = (p, ?a, p') \in T$ fired between C_1 and C'_1 such that $C_2^{-1}(p) \neq \emptyset$, we have that $(p, ?a, p')$ is the only outgoing reception transition from p because P is SWO, hence for all $e \in C_2^{-1}(p)$, $C'_2(e) = p'$ and so t' is fired.

2. if $\alpha = !a$, then we fall into one the four cases:

- a) Message a is received between C_1 and C'_1 by a process e_2 and the same rendez-vous can be made from C_2 with e_2 , i. e. there exists e_2 such that $(C_1(e_2), ?a, C'_1(e_2)) \in T$ and $C_2(e_2) = C_1(e_2)$. Then we define C'_2 as follows:

$$C'_2(e) = \begin{cases} q' & \text{if } e = e_1 \\ C'_1(e_2) & \text{if } e = e_2 \\ C_2(e) & \text{otherwise} \end{cases}$$

It is clear from C'_2 definition that $C_2 \xrightarrow{e_1, t} C'_2$. Furthermore, $t' = (C_1(e_2), ?a, C'_1(e_2))$ is the only reception transition fired between C_1 and C'_1 by definition of a **rendez-vous** hence all fired transitions between C_1 and C'_1 are also fired between C_2 and C'_2 .

- b) Message a is received between C_1 and C'_1 by a process e and transition $(p', ?a, p) \in T$ but e is not on p in C_2 and there exists e_2 on p in C_2 , i. e. there exists $e_2 \neq e_1$, s.t. $t' = (C_2(e_2), ?a, p) \in T$ and t' is fired between C_1 and C'_1 , hence we define C'_2 as follows:

$$C'_2(e) = \begin{cases} q' & \text{if } e = e_1 \\ p & \text{if } e = e_2 \\ C_2(e) & \text{otherwise} \end{cases}$$

It is clear from C'_2 definition that $C_2 \xrightarrow{e_1, t} C'_2$. There is at most one reception transition fired between C_1 and C'_1 by definition of a **rendez-vous**, and it is also fired between C_2 and C'_2 by construction.

If we are not in the two previous case then either $C_2^{-1}(R(a)) = \emptyset$, or there exists $t' = (p, ?a, p') \in T$ such that $C_2^{-1}(p) = \emptyset$. In the latter case, observe that t' is not fired between C_1 and C'_1 because otherwise, we would have been in case a) or b).

- c) In the first case, we have that for all $e \neq e_1$, it holds that $a \notin R(C_2(e))$. We define C'_2 as follows:

$$C'_2(e) = \begin{cases} q' & \text{if } e = e_1 \\ C_2(e) & \text{otherwise} \end{cases}$$

It is clear from C'_2 definition that $C_2 \xrightarrow{e_1, t} C'_2$. Furthermore, if there exists $t' = (p, ?a, p') \in T$ a reception transition fired between C_1 and C'_1 then $t' = (p, ?a, p') \in T$ with $a \in R(p)$, hence $C_2^{-1}(p) = \emptyset$.

- d) In the latter case, there exists $e_2 \neq e_1$, s.t. $t' = (C_2(e_2), ?a, p) \in T$. We define C'_2 as follows:

$$C'_2(e) = \begin{cases} q' & \text{if } e = e_1 \\ p & \text{if } e = e_2 \\ C_2(e) & \text{otherwise} \end{cases}$$

It is clear from C'_2 's definition that $C_2 \xrightarrow{e_1, t} C'_2$. If there exists $t' = (p, ?a, p') \in T$ a reception transition fired between C_1 and C'_1 then by hypothesis of that case, $C_2^{-1}(p) = \emptyset$.

It remains to prove that $C'_2 \in \text{Future}(C'_1)$.

Claim 8.1. For all e such that $C_2(e) = C_1(e)$, it holds that $C'_2(e) = C'_1(e)$ or $(C'_1(e), ?a, C'_2(e)) \in T$ and $C'_1(e') = C_1(e')$.

Proof. In all cases, $C'_2(e_1) = q' = C'_1(e_1)$, hence the claim holds true for e_1 . Let now $e \neq e_1$ such that $C_1(e) = C_2(e)$.

1. In case 1, if $C_2(e) \in Q_A$ or $a \notin R(C_2(e))$, then $C'_2(e) = C_2(e)$ and as $C_1(e) = C_2(e)$, it holds that $C'_1(e) = C_1(e)$, concluding this case. If $(C_2(e), ?a, p) \in T$, then $(C_1(e), ?a, p) \in T$, and so $C'_1(e) = p = C'_2(e)$, concluding this case.
2. a) In case 2a, either $e \neq e_2$ and $C'_2(e) = C_2(e)$ and $C_1(e) = C'_1(e)$, concluding this case, or $e = e_2$ and $C'_2(e) = C'_1(e)$, by construction.
- b) In case 2b, if $e \neq e_2$, then $C_1(e) = C'_1(e)$: otherwise $(C_1(e), ?a, C'_1(e)) \in T$ and it contradicts the hypothesis of case 2b which is that the transition can not be taken by the same process as the one between C_1 and C'_1 . Hence, $C'_2(e) = C_2(e) = C_1(e) = C'_1(e)$. If now $e = e_2$, then $(C_2(e), ?a, C'_2(e)) \in T$, and $C_2(e) = C_1(e)$. Furthermore, $C_1(e) = C'_1(e)$ (as otherwise we are in case 2a), hence $(C'_1(e), ?a, C'_2(e)) \in T$, concluding the proof.
- c) In case 2c, we show that $C_1(e) = C'_1(e)$. Otherwise $(C_1(e), ?a, C'_1(e)) \in T$ then $(C_2(e), ?a, C'_1(e)) \in T$, and it contradicts the fact that $a \notin R(C_2(e))$. Hence, $C'_2(e) = C_2(e) = C_1(e) = C'_1(e)$.

- d) In case 2d, either $e \neq e_2$ and hence, $C'_2(e) = C_2(e) = C_1(e)$. In that case, it can not be that $(C_1(e), ?a, C'_1(e)) \in T$ as either it is a sending step between C_1 and C'_1 or it is a rendez-vous with a process e_3 and a transition $(p, ?a, p')$ such that $C_2^{-1}(p) = \emptyset$. Hence, $C_2(e_3) \neq C_1(e_3)$ and $e_3 \neq e$, it follows that $C'_1(e) = C_1(e) = C'_2(e)$. If $e = e_2$ then by construction, $(C_2(e_2), ?a, C'_2(e_2)) \in T$ and $C_2(e_2) = C_1(e_2) = C'_1(e_2)$, as if $C_1(e_2) \neq C'_1(e_2)$ then it means that $(C_1(e_2), ?a, C'_1(e_2)) \in T$ and so $C_2(e_2) \neq C_1(e_2)$ as otherwise it is not consistent with hypothesis of case 2d.

□

Finally, we prove that, all processes which were ahead in their reception paths, are either still ahead or, have arrived in their next action state.

Claim 8.2. For all e such that $C_2(e) \neq C_1(e)$, then, either $C'_2(e) = C'_1(e) \in Q_A$ or $C'_1(e) \in Q_W$ and $C'_2(e) \in \text{future}(C'_1(e))$.

Proof. First observe that if $C_2(e) \neq C_1(e)$, then as $C_2 \in \text{Future}(C_1)$, it follows that $C_1(e) \in Q_W$ and $C_2(e) \in \text{future}(C_1(e))$. We have to consider four cases:

- **Case 1:** $C'_1(e) = C_1(e)$ and $C'_2(e) = C_2(e)$. Then the claim is trivially true.
- **Case 2:** $(C_1(e), ?a, C'_1(e)) \in T$ and $C'_2(e) = C_2(e)$. If $C'_1(e) \in Q_W$, then $C_2(e) \in \text{future}(C_1(e)) \setminus \{C_1(e)\}$. By definition, $\text{future}(C_1(e)) \setminus \{C_1(e)\} \subseteq \text{future}(C'_1(e))$, hence, $C_2(e) = C'_2(e) \in \text{future}(C'_1(e))$.
If $C'_1(e) \in Q_A$, then by definition $\text{future}(C_1(e)) = \{C_1(e), C'_1(e)\}$: indeed since the protocol is SWO, there exists a unique outgoing reception from $C_1(e)$, and as $C'_1(e) \in Q_A$, there is none from $C'_1(e)$. Hence, as $C_2(e) \in \text{future}(C_1(e))$ and $C_2(e) \neq C_1(e)$, we conclude that $C'_2(e) = C_2(e) = C'_1(e) \in Q_A$.
- **Case 3:** $C'_1(e) = C_1(e)$ and $(C_2(e), ?a, C'_2(e)) \in T$. Then, $C_2(e) \in Q_W$ and $C'_2(e) \in \text{future}(C_2(e))$. Since $C_2(e) \in \text{future}(C_1(e))$, we get that $C'_2(e) \in \text{future}(C_1(e))$, which concludes this case as $C'_1(e) = C_1(e)$.
- **Case 4:** $(C_1(e), ?a, C'_1(e)) \in T$ and $(C_2(e), ?a, C'_2(e)) \in T$. Hence, $C_2(e) \in \text{future}(C_1(e)) \setminus \{C_1(e)\}$ and $C_2(e) \in Q_W$. It follows that $C'_1(e) \in Q_W$ since otherwise we would have $\text{future}(C_1(e)) \setminus \{C_1(e)\} = \{C'_1(e)\} \subseteq Q_A$. Hence, $C'_2(e) \in \text{future}(C_2(e)) \subseteq \text{future}(C'_1(e))$.

□

The two claims above prove that $C'_2 \in \text{Future}(C'_1)$ and so prove the lemma.

□

Example 8.3. Consider once again protocol P of Figure 8.1. We illustrate the lemma with two examples:

1. Consider $C_1 = (q_4, q_4, q_7)$ and $C_2 = (q_6, q_5, q_7)$. We have that $C_2 \in \text{Future}(C_1)$. Furthermore, $C_1 \xrightarrow{3, (q_7, !c, q_4)} (q_5, q_5, q_4)$, and $C_2 \xrightarrow{3, (q_7, !c, q_4)} (q_6, q_6, q_4)$ and indeed $(q_6, q_6, q_4) \in \text{Future}((q_5, q_5, q_4))$. Observe that $(q_4, ?c, q_5)$ is fired from C_1 but it is not from C_2 as $C_2^{-1}(q_4) = \emptyset$.
2. Let now $C_3 = (q_1, q_2, q_7)$ and $C_4 = (q_3, q_2, q_7)$, we have that $C_4 \in \text{Future}(C_3)$. Also, $C_3 \xrightarrow{3, (q_7, !b, q_6)} (q_1, q_3, q_6)$, and $C_4 \xrightarrow{3, (q_7, !b, q_6)} (q_3, q_3, q_6)$. Indeed we have that $(q_3, q_3, q_6) \in \text{Future}((q_1, q_3, q_6))$, and transition $(q_2, ?b, q_3)$ fired from C_3 is also fired from C_4 . Although we also have $C_4 \xrightarrow{3, (q_7, !b, q_6)} (q_1, q_2, q_6)$ and $(q_1, q_2, q_6) \in \text{Future}((q_1, q_3, q_6))$, it cannot be taken as a successor configuration, since $(q_2, ?b, q_3)$ is not fired from C_4 in that case.

In Chapter 6, we proved that computing the set of coverable states of a Wait-Only protocol, can be done in PTIME (Theorem 6.5). Hence, the set of coverable states S can be computed in polynomial time, since any SWO protocol is, in particular, a Wait-Only one.

In the following lemma, we show that for any configuration in which all processes are in coverable states, there exists a configuration in its future-set that is itself coverable. Intuitively, we extend the cypaste lemma established for Wait-Only protocols to the Single Wait-Only setting: when we glue executions, additional sendings or broadcasts may cause some processes to leave their waiting states. However, since each waiting state has a unique outgoing reception transition, any process receiving one of these extra messages moves to a state in the future-set of its original state.

Lemma 8.3. Let P be a SWO protocol and S its set of coverable states. Then, for any configuration $C \in \mathcal{C}$ such that $\text{set}(C) \subseteq S^1$, there exists $C' \in \text{Future}(C)$ such that C' is coverable.

Proof. We prove by induction that for all configuration C with k processes on waiting states, i. e. $k = |C^{-1}(Q_W)|$, there exists $C' \in \text{Future}(C)$ such that C' is coverable.

Case $k = 0$. Let C such that $C^{-1}(Q_W) = \emptyset$. Then, by definition, $\text{Future}(C) = \{C\}$. Furthermore, P is a Wait-Only protocol, hence the cypaste property (Proposition 6.1) holds and with $N = \max_{q \in Q_A} |C^{-1}(q)|$, the lemma ensures that there is an execution $C_{\text{init}} \rightarrow^* C'$ such that for all $q \in Q_A$, $|C'^{-1}(q)| \geq N \geq |C^{-1}(q)|$. Hence C is coverable, proving the base case.

Case $k + 1$. Let C be a configuration such that $|C^{-1}(Q_W)| = k + 1$, then w.l.o.g. we can consider that $C = C'' \oplus \dot{p}$ with $p \in Q_W$ and C'' a configuration with k processes on waiting states. From the SWO shape and because $p \in S$, there exists $q \in Q_A \cap S$ and a sequence of transitions in T

$$(q, \alpha, p_1)(p_1, ?a_1, p_2)(p_2, ?a_2, p_3) \cdots (p_n, ?a_n, p)$$

¹ Recall that $\text{set}(C) = \{q \mid C^{-1}(q) \neq \emptyset\}$

² Recall that $\dot{p}$ is the configuration C_p with a single process e such that $C_p(e) = p$. We also recall the definition of $\oplus$: the configuration $C_3 = C_1 \oplus C_2$ is defined as the configuration with $|C_1| + |C_2|$ processes and such that $C_3(e) = C_1(e)$ for all $e \in [1, |C_1|]$ and $C_3(e) = C_2(e - |C_1|)$ for all $e \in [|C_1| + 1, |C_1| + |C_2|]$.

such that for all $1 \leq i \leq n$, there exists $q_i \in Q_A \cap S$ such that $(q_i, \diamond_i a_i, q'_i) \in T$ with $\diamond_i \in \{!, !!\}$. We define C_p as the configuration with $|C''| + n + 1$ processes:

$$C_p = C'' \oplus (q, q_1, \dots, q_n)$$

Observe that C_p has as many processes on waiting states as processes on waiting states in C'' , i.e. k . Hence, we can apply the induction hypothesis. We get that there exists $C''' \in \text{Future}(C_p)$ and a configuration C'_p such that $|C'^{-1}_p(s)| \geq |C'''^{-1}(s)|$ for all $s \in Q$, and an execution

$$C_{\text{init}} \rightarrow^* C'_p$$

for an initial configuration $C_{\text{init}} \in \mathcal{C}_{\text{init}}$.

Observe that by definition of $\text{Future}(C_p)$, for all e such that $C_p(e) \in Q_A$, we have that $C_p(e) = C'''(e)$. Hence, w.l.o.g. we can rewrite $C''' = C_f \oplus (q, q_1, \dots, q_n)$ for a configuration $C_f \in \text{Future}(C')$. So since C'_p covers C''' ,

$$C'_p = C_f \oplus (q, q_1, \dots, q_n) \oplus C_1$$

for some configuration C_1 . We let $j = |C_f|$ be the number of processes in C_f (i.e. the number of processes in C excluding the process on p). From C'_p , one can perform the following sequence of communication steps:

$$C'_p \xrightarrow{j+1, (q, \alpha, p_1)} C'_1 \xrightarrow{j+2, (q_1, \diamond_1 a_1, q'_1)} C'_2 \xrightarrow{j+3, (q_2, \diamond_2 a_2, q'_2)} \dots \xrightarrow{j+n+1, (q_n, \diamond_n a_n, q'_n)} C'_{n+1}$$

such that whenever $\diamond_i = !$, we choose the receiver process to be process $j + 1$. Hence, C'_{n+1} is such that:

- the first j processes still form a configuration in $\text{Future}(C'')$, and so respects the condition required for the $j + 1$ first processes to form a configuration in $\text{Future}(C)$:
 - for all $e \in [1, j]$ such that $C'_p(e) \in Q_A$, it holds that $C'_{n+1}(e) = C'_p(e) = C_f(e) \in \text{future}(C''(e)) = \text{future}(C(e))$.
 - for all $e \in [1, j]$ such that $C'_p(e) \in Q_W$, either $C'_{n+1}(e) = C'_p(e) = C''(e) = C(e)$ or, there exists a subword $b_1 \dots b_m$ of $aa_1a_2 \dots a_n$ (where a is the first message sent by process $j + 1$, i.e. $\alpha = \diamond a$ for $\diamond \in \{!, !!\}$) such that there exists $s_1, \dots, s_m \in S$ and a sequence in T :

$$(C'_p(e), ?b_1, s_1)(s_1, ?b_2, s_2) \dots (s_m, ?b_{m+1}, C'_{n+1}(e))$$

Hence, $C'_{n+1}(e) \in \text{future}(C'_p(e))$ and $C'_p(e) = C_f(e) \in \text{future}(C''(e)) = \text{future}(C(e))$, hence by transitivity, $C'_{n+1}(e) \in \text{future}(C(e))$.

- the $j + 1$ process is on $C(j + 1) = p$: $C'_{n+1}(j + 1) = p = C(j + 1)$ so p is covered in C'_{n+1} .

Hence, C'_{n+1} can be written as

$$C_{n+1} \oplus C_2$$

for two configurations C_{n+1}, C_2 such that $C_{n+1} \in \text{Future}(C)$, which concludes the induction proof. □

8.3 REPEATED COVER IS EASY

In this section, we prove that the repeated coverability problem for Single Wait-Only networks is in PTIME in contrast with WO networks where it is PSPACE-hard..

It relies on the positive-cycle problem on VASS defined as follows: given a VASS $V = (\text{Loc}, X, \ell_{in}, \Delta)$, the positive-cycle problem asks if there exists $v, v' \in \mathbb{N}^X$ such that $(\ell_{in}, v) \rightsquigarrow^* (\ell_{in}, v')$ and $v \leq v'$?

We show that repeated coverability on SWO networks actually reduce to the positive cycle problem on VASS. In [KS88], the authors establish the following result.

Theorem 8.1 ([KS88]). *The positive-cycle problem is in PTIME.*

The idea is that in the VASS, we simulate rendez-vous, sendings, and broadcasts in a “relaxed” manner, thanks to a counting abstraction. The relaxation lies in two aspects, both related to 0-testing a counter:

1. when simulating a broadcast, counters associated to a state should be emptied because of the broadcasts. The VASS will obviously not check this.
2. when simulating a sending step, again, the VASS will not check this, hence counters associated to states from which the message can be received should be empty, hence a sending step can be simulated whereas actually there are processes ready to receive the message.

We argue that these relaxations are harmless: finding a positive cycle in the VASS still guarantees the existence of an infinite execution of the protocol in which the given transition is fired infinitely often.

Both relaxations allow some receptions to be skipped when they should have occurred. However, in SWO protocols, this only amounts to *delaying* these receptions: eventually the process has to receive the message in order to complete the VASS positive-cycle. Because reception paths are deterministic in SWO protocols, and because no alternative behavior is possible while a process waits, the delay does not interfere with the rest of the execution. In particular, the waiting process cannot receive other messages nor reach other states.

We fix a SWO protocol $P = (Q, \Sigma, q_{init}, T)$, a transition $t_f \in T$, and let S be its set of coverable states. We define the VASS represented in Figure 8.2, and show that there exists a positive-cycle from s if and only if there is an execution in which t_f is fired infinitely often.

The VASS $V = (\text{Loc}, X, s, \Delta)$ has two main locations: s and s_1 . Location s is the starting point from which we ask for a positive-cycle, while s_1 is the location where communication steps are simulated. The counter x_{t_f} ensures that the positive-cycle simulates at least one communication step where t_f is fired: it is decremented when moving from s to s_1 , and must be incremented before returning to s to close the cycle.

Once in s_1 , the simulation proceeds as follows:

- **Broadcast** $(q, !!a, q')$: The simulation moves to a location $s_{(q, !!a, q')}$. From there, for every transition $(p, ?a, p') \in T$, the counter x_p can be decremented while $x_{p, p'}$ is incremented, this models processes receiving message a . Then, from an intermediate state $s'_{(q, !!a, q')}$ the values in counters $x_{p, p'}$ are transferred to $x_{p'}$, and the simulation returns to s_1 . This intermediate step is needed to ensure that each process receives message a at most once during a single broadcast. For instance, consider the protocol depicted in Figure 8.3, merging $s_{(q_{init}, !!a, q_4)}$ and $s'_{(q_{init}, !!a, q_4)}$ together will result in incorrect behaviors: it would

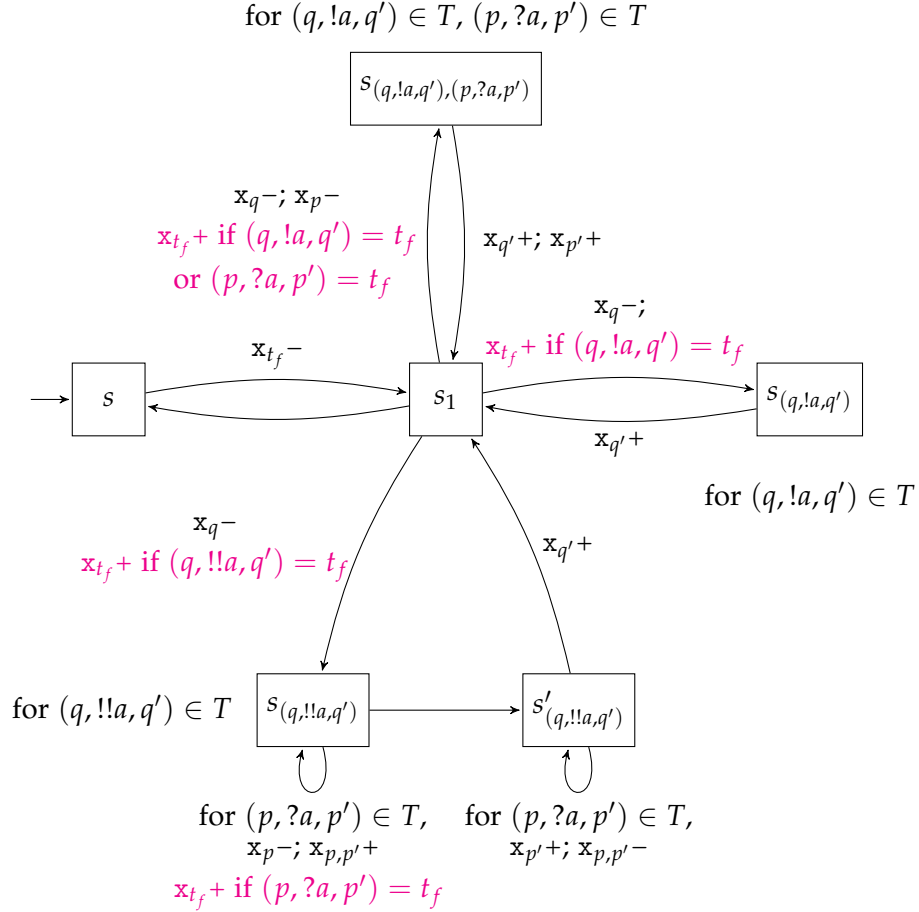


Figure 8.2: The VASS V built from the SWO protocol $P = (Q, \Sigma, q_{init}, T)$ and a transition $t_f \in T$.

be possible for a process to receive message a twice during a single broadcast by successively applying the VASS transitions corresponding to $(q_1, ?a, q_2)$ and $(q_2, ?a, q_3)$. Note that x_p is not necessarily emptied, some processes may delay their reception. However, since the protocol is SWO and receptions are deterministic, these processes will eventually receive the message. Each time the simulation includes a transition that corresponds to t_f , the counter x_{t_f} is incremented.

- **Rendez-vous** ($t_1 = (q, !a, q')$, $t_2 = (p, ?a, p')$): The simulation moves to a location s_{t_1, t_2} , and updates counters accordingly.
- **Sending** $(q, !a, q')$ **not received by any process**: The simulation moves to a location s_t , and only updates counters x_q and $x_{q'}$. There is no mechanism in the VASS to ensure this only happens when there is no possible receivers of a (i.e., counters of states in $R(a)$ are zero). Therefore, it may happen that a process that should receive a misses it in the simulation. As in the broadcast case, this only delays the reception, which is not problematic in a SWO protocol, since the reception will eventually occur.

We formally define it below.

Definition 8.4. We consider the VASS $V = (\text{Loc}, X, s, \Delta)$ pictured in Figure 8.2 and such that:

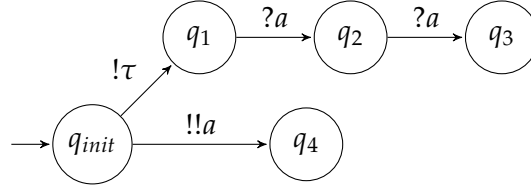


Figure 8.3: Example of why we need an intermediate location in the VASS.

- $\text{Loc} = \{s, s_1\} \cup \{s_{(q,!a,q')} \mid (q,!a,q') \in T \text{ and } q, q' \in S\} \cup \{s_{(q,!!a,q')}, s'_{(q,!!a,q')} \mid (q,!!a,q') \in T \text{ and } q, q' \in S\} \cup \{s_{(q,!a,q'),(p,?a,p')} \mid (q,!a,q'), (p,?a,p') \in T \text{ and } q, q', p, p' \in S\}$, and
- $X = \{x_q \mid q \in S\} \cup \{x_{q,q'} \mid q, q' \in S\} \cup \{x_{t_f}\}$, and
- Δ contains exactly the following transitions:
 - (s, δ_{t_f}, s_1) where $\delta_{t_f}(x_{t_f}) = -1$ and $\delta_{t_f}(x) = 0$ for all other counters x . We make sure x_{t_f} is decreased at least once in any positive-cycle going from s to s , so that t_f is taken at least once.
 - $(s_1, \mathbf{0}_X, s)$, the transition makes sure that a positive-cycle from s to itself can exist.
 - for all $(q,!a,q') \in T$, $(p,?a,p') \in T$ such that $q, q', p, p' \in S$, Δ contains the following transitions simulating a rendez-vous (and decreasing x_{t_f} when one of the two involved transition is t_f):
 - * $(s_1, \delta^-, s_{(q,!a,q'),(p,?a,p')})$ with $\delta^-(x_q) = \delta^-(x_p) = -1$ and, if $t_f \in \{(q,!a,q'), (p,?a,p')\}$, then $\delta^-(x_{t_f}) = 1$ and otherwise, $\delta^-(x_{t_f}) = 0$. Moreover, $\delta^-(x) = 0$ for all other counters x .
 - * $(s_{(q,!a,q'),(p,?a,p')}, \delta^+, s_1)$ with $\delta^+(x_{q'}) = \delta^+(x_{p'}) = 1$ (and if $q' = p'$, $\delta^+(x_{q'}) = 2$), and $\delta^+(x) = 0$ for all other counters x .
 - for all $(q,!a,q') \in T$ with $a \in \Sigma$, such that $q, q' \in S$, Δ contains the following transitions, simulating a sending step (and decreasing x_{t_f} when $t_f = (q,!a,q')$):
 - * $(s_1, \delta^-, s_{(q,!a,q')})$ with $\delta^-(x_q) = -1$ and, if $t_f = (q,!a,q')$, then $\delta^-(x_{t_f}) = 1$ and otherwise, $\delta^-(x_{t_f}) = 0$, and $\delta^-(x) = 0$ for all other counters x .
 - * $(s_{(q,!a,q')}, \delta^+, s_1)$ with $\delta^+(x_{q'}) = 1$, and $\delta^+(x) = 0$ for all other counters x .
 - for all $(q,!!a,q') \in T$ and $(p,?a,p') \in T$ such that $q, q', p, p' \in S$, Δ contains the following transitions simulating a broadcast steps:
 - * $(s_1, \delta^-, s_{(q,!!a,q')})$ with $\delta^-(x_q) = -1$ and, if $t_f = (q,!!a,q')$, then $\delta^-(x_{t_f}) = 1$ and otherwise, $\delta^-(x_{t_f}) = 0$, and $\delta^-(x) = 0$ for all other counters x .
 - * $(s_{(q,!!a,q')}, \mathbf{0}_X, s'_{(q,!!a,q')})$,
 - * $(s'_{(q,!!a,q')}, \delta^+, s_1)$ with $\delta^+(x_{q'}) = 1$, and $\delta^+(x) = 0$ for all other counters x .
 - * $(s_{(q,!!a,q')}, \delta_p^-, s_{(q,!!a,q')})$ where $\delta(x_p) = -1$ and $\delta(x_{p,p'}) = 1$, and $\delta(x) = 0$ for all other counters x . In a first step, x_p is decremented for each process receiving a , and kept in transit in $x_{p,p'}$.
 - * $(s'_{(q,!!a,q')}, \delta_{p'}^+, s'_{(q,!!a,q')})$ where $\delta(x_{p,p'}) = -1$ and $\delta(x_{p'}) = 1$, and $\delta(x) = 0$ for all other counters x . In a second step, $x_{p,p'}$ is transferred to $x_{p'}$.

Observe that the VASS transitions only consider states in S , i.e. coverable states. This will allow us to establish that all positive-cycles of the VASS lead to infinite run of the protocol reachable from an initial configuration.

We establish the following completeness result.

Lemma 8.4. If there exists an infinite execution of P in which t_f is fired infinitely often, then there exists a positive-cycle in the VASS.

Proof. Let $\rho = C_0 C_1 \cdots$ an execution of P in which t_f is fired infinitely often. There must exist $i < j$ such that $C_i C_{i+1} \cdots C_j$ with $C_j = C_i$ and t_f is fired between C_i and C_j . Denote by x_k the number of times t_f is fired in the segment $C_i \cdots C_k$ for all $k \in [i, j]$. We let v_k be the valuation such that: $v_k(x_q) = |C_k^{-1}(q)|$ for all $q \in Q$ and $v_k(x_{t_f}) = x_k$, and $v_k(x) = 0$ for all other counters x , for all $k \in [i, j]$. We easily prove that $(s_1, v_k) \rightsquigarrow^* (s_1, v_{k+1})$ for all $k \in [i, j-1]$, by taking the appropriate transition simulating the transition between C_k and C_{k+1} . For instance, if $C_k \xrightarrow{(q, !!a, q')} C_{k+1}$, then from (s_1, v_k) , take transition to $s_{(q, !!a, q')}$, transfer the values of counters associated to state p such that there exists p' with $(p, ?a, p') \in T$ to counters $x_{pp'}$, then take the transition to state $s'_{(q, !!a, q')}$ and transfer values of $x_{pp'}$ to counters $x_{p'}$. Hence, we get that $(s_1, v_i) \rightsquigarrow^* (s_1, v_j)$. The value of counter x_{t_f} is increased by the number of processes which fired t_f , hence at least once in the segment between (s_1, v_i) and (s_1, v_j) . Furthermore, for all $x \neq x_{t_f}$, $v_i(x) = v_j(x)$ because $C_i = C_j$, and $v_i(x_{t_f}) = 0$ and $v_j(x_{t_f}) = x_j \geq 1 > v_i(x_{t_f})$. We define v the valuation such that $v(x_{t_f}) = 1$ and $v(x) = v_i(x)$ for all other counter x . Hence $(s, v) \rightsquigarrow (s_1, v_i) \rightsquigarrow^+ (s_1, v_j) \rightsquigarrow (s, v_j)$ concluding the proof. $\square$

The rest of this section is devoted to prove the soundness of the construction.

8.3.1 Notations

We now define useful notations. For two configurations (s_1, v) and (s_1, v') and a transition $t = (q, \alpha, q')$ with $\alpha \in !\Sigma \cup !!\Sigma$, we write $(s_1, v) \xrightarrow{t} (s_1, v')$ whenever there exists a run of the VASS $(s_1, v) \xrightarrow{\delta_1} (s^1, v^1) \xrightarrow{\delta_2} \cdots \xrightarrow{\delta_n} (s_1, v')$ s.t. $s^i \in \{s_{(q, \alpha, q')}, s'_{(q, \alpha, q')}, s_{(q, \alpha, q'), t'} \mid t' \in T\}$ for all $1 \leq i \leq n-1$, i.e. when the communication step simulated between (s_1, v) and (s_1, v') corresponds to firing t .

Furthermore, for a transition $t' = (p, ?a, p')$, we write $(s_1, v) \rightsquigarrow_{t'} (s_1, v')$ whenever there exists $t = (q, \alpha, q')$, and a run $(s_1, v) \xrightarrow{\delta_1} (s^1, v^1) \xrightarrow{\delta_2} \cdots \xrightarrow{\delta_n} (s_1, v)$ such that:

- $\alpha = !a$ and $s^1 = s_{(q, !a, q'), (p, ?a, p')}$ for all $1 \leq i \leq n-1$ (in fact in that case $n = 2$);
- $\alpha = !!a$, $s^i \in \{s_{(q, !!a, q')}, s'_{(q, !!a, q')}\}$ for all $1 \leq i \leq n-1$ and there exists $j \in [2, n-2]$ such that $\delta_j(x_p) = -1$; $\delta_j(x_{p'}) = 1$ and $\delta_j(x) = 0$ for all other counters x .

We now let v be a valuation over the set of counters X , and we associate a set of configurations of P to it, called representatives:

$$\mathbf{C}[v] = \{C \in \mathcal{C} \mid \forall q \in Q, |C^{-1}(q)| = v(x_q) + \sum_{q' \in Q_W} v(x_{q'q})\}.$$

Processes counted by $\sum_{q' \in Q} v(x_{q'q})$ correspond to processes who previously took a transition $(q, ?a, q')$ but the counter has not been correctly emptied. Hence, we decide that the processes it counts are in q and not in q' .

Finally, we abuse notation and write $\text{Future}(v)$ for the set of configurations

$$\text{Future}(v) = \{C \in \mathcal{C} \mid \text{there exists } C' \in \mathcal{C}[v] \text{ s.t. } C \in \text{Future}(C')\}.$$

8.3.2 Local soundness property

We now prove a first local soundness property. It states that whenever $(s_1, v) \xrightarrow{t} (s_1, v')$, then for any configuration C representing v , we can find a configuration C' such that $C \rightarrow C'$ and C' belongs to the future set of v .

When simulating a rendez-vous, this is immediate: the definition of $\mathcal{C}[v]$ guarantees that two processes are in the correct states to perform the rendez-vous. When simulating a sending step $(q, !a, q')$, one process is in state q in C and can perform the transition. If no process is able to receive the message, we directly reach a configuration in $\mathcal{C}[v']$. Otherwise, a process receives the message and moves to a state belonging to the future set of its starting state, ensuring that the reached configuration is still in $\mathcal{C}[v']$. The reasoning is analogous for broadcasts: processes that move without being explicitly moved by the VASS simply reach states in the future set of their starting states.

Finally, the lemma also ensures that when a reception transition is simulated between (s_1, v) and (s_1, v') , it is also fired between C and the constructed configuration C' . This will later allow us to build an execution where t_f (when it is a reception transition) is fired infinitely often.

Lemma 8.5. Let v, v' such that $(s_1, v) \xrightarrow{t} (s_1, v')$ for a transition $t \in T$. For all $C \in \mathcal{C}[v]$, there exists $C' \in \text{Future}(v')$ such that $C \xrightarrow{e, t} C'$ for a process $e \in [1, |C|]$. Furthermore, for all $t' \in T$ such that $(s_1, v) \rightsquigarrow_{t'} (s_1, v')$, there exists e' taking t' between C and C' .

Proof. Let $(s_1, v) \xrightarrow{t} (s_1, v')$ for a transition $t = (q, \alpha, q') \in T$ and $C \in \mathcal{C}[v]$. Observe that, from the shape of V , in any case, counter x_q is decreased from (s_1, v) , hence $v(x_q) > 0$ and so there exists a process e such that $C(e) = q$. We distinguish between three cases.

- if $\alpha = !!a$ then the run between (s_1, v) and (s_1, v') of the form:

$$(s_1, v) \rightsquigarrow (s_t, v_1) \rightsquigarrow \dots \rightsquigarrow (s_t, v_k) \rightsquigarrow (s'_t, v_{k+1}) \rightsquigarrow \dots \rightsquigarrow (s'_t, v_n) \rightsquigarrow (s_1, v')$$

We let k_p be the number of times x_p is decreased between (s_t, v_1) and (s_t, v_k) for all $p \in Q_W$. We now let C' be a configuration such that $C \xrightarrow{e, t} C'$ (it exists as $C(e) = q$). We now have to show that $C' \in \text{Future}(v')$. In order to do that, we first define $C'' \in \mathcal{C}[v']$: we choose it so that exactly k_p processes on p take the transition $(p, ?a, p')$ when there is one, instead of all processes. That is to say:

- $C''(e) = q'$, and
- Let now $p \in Q_W$. By construction of the VASS, $v(x_p) = v_1(x_p) \geq k_p$ for all $p \in Q_W$, hence, as $C \in \mathcal{C}[v]$, there exists $e_1, \dots, e_{k_p}$ such that $C(e_i) = p$. For all $1 \leq i \leq k_p$, we let $C''(e_i) = p'$ where $(p, ?a, p') \in T$. We denote this subset of processes E_p for all $p \in Q_W$.

- for all other processes e' , $C''(e') = C(e')$.

It is easy to see that for all $p \in Q$,

$$v_n(x_p) + \sum_{p' \in Q_W} v_n(x_{p'p}) = v_1(x_p) + \sum_{p' \in Q_W} v_1(x_{p'p}) - k_p + \sum_{(p', ?a, p) \in T} k_{p'}$$

(we take the convention that $k_p = 0$ for $p \in Q_A$). Recall that $v_1(x_q) = v(x_q) - 1$ (resp. $v_n(x_{q'}) = v'(x_{q'}) - 1$) and $v_1(x) = v(x)$ (resp. $v_n(x) = v'(x)$) for all other counters. Furthermore,

$$|C''^{-1}(p)| = \begin{cases} |C^{-1}(p)| - k_p + \sum_{(p', ?a, p) \in T} k_{p'} & \text{if } p \in Q \setminus \{q, q'\} \\ |C^{-1}(p)| - k_p + \sum_{(p', ?a, p) \in T} k_{p'} & \text{if } p = q = q' \\ |C^{-1}(p)| - k_p + \sum_{(p', ?a, p) \in T} k_{p'} - 1 & \text{if } p = q \text{ and } q \neq q' \\ |C^{-1}(p)| - k_p + \sum_{(p', ?a, p) \in T} k_{p'} + 1 & \text{if } p = q' \text{ and } q \neq q' \end{cases}$$

Hence, $C'' \in \mathbb{C}[v']$.

We show now that $C' \in \text{Future}(C'')$. Observe that the only processes e' such that $C'(e') \neq C''(e')$ are processes such that $e' \notin \bigcup_{p \in Q_W} E_p$ and $(C(e'), ?a, C'(e')) \in T$. Furthermore, $C''(e') = C(e')$. Hence, by definition of $\text{future}(\cdot)$, we have that $C'(e') \in \text{future}(C''(e'))$, which concludes this case.

- if $\alpha = !a$ and there exists $t' = (p, ?a, p') \in T$ such that the run has the following shape:

$$(s_1, v) \rightsquigarrow (s_{t'}, v'') \rightsquigarrow (s_1, v'),$$

then $p \in Q_W$ and $q \in Q_A$ hence $q \neq p$ and so $v(x_q) > 0$ and $v(x_p) > 0$. Hence, as $C \in \mathbb{C}[v]$, there exists e' such that $C(e) = p$. Hence, $C \xrightarrow{e, t} C'$ where $C'(e') = p'$, i. e. e sends a leading to a rendez-vous with process e' . We naturally get that $C' \in \mathbb{C}[v']$ and so $C' \in \text{Future}(v')$.

- if $\alpha = !a$ and the run has the following shape:

$$(s_1, v) \rightsquigarrow (s_t, v'') \rightsquigarrow (s_1, v'),$$

there are two subcases to consider: either there exists $e_?$ such that $(C(e_?), ?a, p') \in T$ or for all process e' , $a \notin R(C(e'))$. In the first case, we let C' be the configuration such that:

$$C'(e') = \begin{cases} q' & \text{if } e' = e \\ p' & \text{if } e' = e_? \\ C(e) & \text{otherwise} \end{cases}$$

We have that $C \xrightarrow{e, t} C'$. We now prove that $C' \in \text{Future}(v')$. To do this let C'' be a configuration such that

$$C''(e') = \begin{cases} q' & \text{if } e' = e \\ C(e) & \text{otherwise} \end{cases}$$

Clearly, $C'' \in \mathbb{C}[v']$. Furthermore the only process e' such that $C'(e') \neq C''(e')$ is process $e_?$ and $(C(e_?), ?a, C'(e_?)) \in T$. Furthermore, $C''(e_?) = C(e_?)$. Hence, by definition of $\text{future}(\cdot)$, we have that $C'(e_?) \in \text{future}(C''(e_?))$, which concludes this case.

Now assume that for all process e' , $a \notin R(C(e'))$, then let C' be the configuration such that:

$$C'(e') = \begin{cases} q' & \text{if } e' = e \\ C(e) & \text{otherwise} \end{cases}$$

We have that $C \xrightarrow{e,t} C'$ and $C' \in \mathbb{C}[v']$, and hence $C' \in \text{Future}(v')$, which concludes the proof. □

Thanks to Lemma 8.3 and this lemma, we get the following result, which will be our key ingredient to build an infinite execution of P from a positive cycle of the VASS.

Lemma 8.6. Let v, v' be two valuations such that $(s_1, v) \xrightarrow{t} (s_1, v')$ for a transition $t \in T$. For all $C \in \text{Future}(v)$, there exists $C' \in \text{Future}(v')$ such that $C \xrightarrow{e,t} C'$ for a process e .

Proof. Let $C \in \text{Future}(v)$, then there exists $C_1 \in \mathbb{C}[v]$ such that $C \in \text{Future}(C_1)$. From Lemma 8.5, there exists $C_2 \in \text{Future}(v')$ such that $C_1 \xrightarrow{e,t} C_2$ for a process e . More precisely, there exists $C_3 \in \mathbb{C}[v']$ such that $C_2 \in \text{Future}(C_3)$. From Lemma 8.2, there exists C' such that $C \xrightarrow{e,t} C'$ and $C' \in \text{Future}(C_2)$. Finally, thanks to Lemma 8.1, we get that $C' \in \text{Future}(C_3)$, i. e. $C' \in \text{Future}(v')$. □

8.3.3 Soundness lemma and main theorem

Proving the soundness of the construction requires a bit more work than simply applying Lemma 8.6 inductively, as the lemma alone does not guarantee that t_f is fired infinitely often.

If t_f is an action transition, the property follows immediately from Lemma 8.6. However, if t_f is a reception transition, we must ensure that, in the constructed execution, there is always a process that eventually fires it.

If t_f belongs to a cycle that also contains action transitions, then we can argue that since a process fires infinitely often the action transition leading to the reception segment containing t_f , and because the protocol is SWO (ensuring that there is no alternative path back to an action state), it must also traverse this segment infinitely often. Hence, t_f is fired infinitely often. If, on the other hand, t_f is part of a cycle composed only of reception transitions, we first argue that at least one process evolves within this cycle. Then, using the second part of Lemma 8.2 and the second part of Lemma 8.1, we show by contradiction that there must be at least one process that keeps making progress within the cycle, thereby firing t_f infinitely often.

Lemma 8.7. If there exists a positive-cycle in the VASS, then there exists an infinite execution of protocol in which t_f is fired infinitely often.

Proof. Consider a run $(s, v) \rightsquigarrow^+ (s, v')$ the run such that $v' \geq v$ and v is a minimal vector from which a positive cycle is possible. From V 's shape, we can decompose the run into

$$(s, v) \rightsquigarrow (s_1, v_1) \xrightarrow{t_1} (s_1, v_2) \xrightarrow{t_2} \dots \xrightarrow{t_{k-1}} (s_1, v_k) \rightsquigarrow (s, v')$$

where $v' \geq v$.

We start by proving the following claim.

Claim 8.3. $v'(x) = v(x)$ for all $x \in X \setminus \{x_{t_f}\}$.

Proof. Observe that from the construction of the VASS, it holds that for all $i \in [1, k-1]$,

$$\sum_{q \in Q} v_i(x_q) + \sum_{q \in Q_W, q' \in Q} v_i(x_{qq'}) = \sum_{q \in Q} v_{i+1}(x_q) + \sum_{q \in Q_W, q' \in Q} v_{i+1}(x_{qq'})$$

i.e. the sum of counters describing states is the same in all the (s_1, v_i) . This ensures that no process is nor created or deleted during the simulation. Hence, we have that

$$\sum_{q \in Q} v(x_q) + \sum_{q \in Q_W, q' \in Q} v(x_{qq'}) = \sum_{q \in Q} v'(x_q) + \sum_{q \in Q_W, q' \in Q} v'(x_{qq'})$$

and $v'(x) \geq v(x)$ for all $x \in X$. Therefore, we get that

$$v'(x) = v(x) \text{ for all } x \in X \setminus \{x_{t_f}\}.$$

□

We now prove the following intermediate lemma:

Lemma 8.8. There exists an infinite execution $C_{init} \rightarrow^+ C_0 \xrightarrow{e_1, t_1} \xrightarrow{e_2, t_2} \dots \xrightarrow{e_k, t_k} C_k \xrightarrow{e_{k+1}, t_1} \dots$ such that for all $i \in \mathbb{N}_{>0}$, $e_i \in [1, |C_{init}|]$.

Proof. As v is a minimal vector from which there is a run $(s, v) \rightsquigarrow^+ (s, v')$ with $v' \geq v$, it holds that $v(x_q) = v(x_{q'q}) = 0$ for all $q \notin S$ as for all $q \notin S$, x_q (resp. $x_{q'q}$) is never nor increased nor decreased, and so the same cycle can be performed with null counter's value.

Hence let $C \in \mathbb{C}[v_1]$.³ From Lemma 8.3, there exists $C' \in \text{Future}(C)$ such that C' is coverable: i.e. there exists $C_0 = C' \oplus C'_0$ for some C'_0 , and an initial configuration $C_{init} \rightarrow^* C_0$. Thanks to a simple induction and Lemma 8.6, we get that there exists a run from C' such as:

$$C' \xrightarrow{e_1, t_1} C'_1 \xrightarrow{e_2, t_2} \dots \xrightarrow{e_k, t_k} C'_k$$

With $C'_k \in \text{Future}(v_k)$.

³ Remember that $(s, v) \rightsquigarrow (s_1, v_1)$.

Thanks to Claim 8.3, we get that $v_1(x) = v_k(x)$ for all $x \neq x_{t_f}$. Hence, $C'_k \in \text{Future}(v_1)$. We can then reiterate the reasoning and build an infinite run from C' :

$$C' \xrightarrow{e_1, t_1} C'_1 \xrightarrow{e_2, t_2} \dots \xrightarrow{e_k, t_k} C'_k \xrightarrow{e_{k+1}, t_1} \dots \xrightarrow{e_{2k}, t_k} C'_{2k} \xrightarrow{e_{2k+1}, t_1} \dots$$

By monotonicity⁴ of our networks we get an infinite run from C_0 :

$$C_0 \xrightarrow{e_1, t_1} C_1 \xrightarrow{e_2, t_2} \dots \xrightarrow{e_k, t_k} C_k \xrightarrow{e_{k+1}, t_1} \dots$$

Hence we have proved the lemma. $\square$

Let ρ be the execution of the previous lemma, we now show that in this execution, t_f is indeed fired infinitely often.

We first justify that there must exist $1 \leq i \leq k$ such that $(s_1, v_i) \xrightarrow{t_f} (s_1, v_{i+1})$ if $t_f = (q, \alpha, q')$ with $\alpha \in !\Sigma \cup !!\Sigma$ (i.e. t_f is an action transition), or such that $(s_1, v_i) \rightsquigarrow_{t_f} (s_1, v_{i+1})$ if $t_f = (q, ?a, q')$ for an $a \in \Sigma$ (i.e. t_f is a reception transition). From the construction of the VASS, we have that $v_1(x_{t_f}) = v(x_{t_f}) - 1$, hence there must exist i such that $v_{i+1}(x_{t_f}) \geq v_i(x_{t_f}) + 1$ so that $v' \geq v$, hence $(s_1, v_i) \xrightarrow{t_f} (s_1, v_{i+1})$ if $t_f = (q, \alpha, q')$ with $\alpha \in !\Sigma \cup !!\Sigma$, or $(s_1, v_i) \rightsquigarrow_{t_f} (s_1, v_{i+1})$ if $t_f = (q, ?a, q')$.

If t_f is an action transition, then we just proved that $t_f \in \{t_1, \dots, t_k\}$, hence thanks to Lemma 8.8, we get that t_f is fired infinitely often in ρ .

We now focus on the case where $t_f = (q, ?a, q')$ is a reception transition. Using ??, as there exists $i \in [1, k-1]$ such that $(s_1, v_i) \rightsquigarrow_{t_f} (s_1, v_{i+1})$, there exists a sequence of transitions $\tilde{t}_0 \tilde{t}_1 \dots \tilde{t}_n$ such that

1. $\tilde{t}_j = (q_j, \alpha_j, q_{j+1})$ for all $j \in [0, n]$, with $q_0 = q_{n+1}$, and
2. there exists $i_j \in [1, k-1]$ such that $(s_1, v_{i_j}) \xrightarrow{\tilde{t}_j} (s_1, v_{i_{j+1}})$ or $(s_1, v_{i_j}) \rightsquigarrow_{\tilde{t}_j} (s_1, v_{i_{j+1}})$ for all $j \in [0, n]$, and
3. $t_f \in \{\tilde{t}_j \mid j \in [0, n]\}$.

We assume w.l.o.g. that $t_f = \tilde{t}_n$. We now distinguish between two cases:

- **Case 1: there exists j with $\alpha_j \in !\Sigma \cup !!\Sigma$.** We let j_1 be the index such that:

$$\alpha_{j_1} \in !\Sigma \cup !!\Sigma \text{ and } \alpha_j \in ?\Sigma, \text{ for all } j \in [j_1 + 1, n-1]$$

i.e. to be the last index before going into the reception segment containing $\tilde{t}_f$. Remember that there exists i_{j_1} such that $(s_1, v_{i_{j_1}}) \xrightarrow{\tilde{t}_{j_1}} (s_1, v_{i_{j_1+1}})$, hence $\tilde{t}_{j_1} \in \{t_1, \dots, t_k\}$. Therefore, the transition is fired infinitely often in ρ , and, as the number of processes is fixed throughout the execution, there exists a process e who fires $\tilde{t}_{j_1}$ infinitely often. Let $x \in \mathbb{N}$ such that $C_x \xrightarrow{e, \tilde{t}_{j_1}} C_{x+1}$, there exists $y > x$

⁴ In Chapter 2 and lemma 2.4, we proved that if $C_1 \xrightarrow{e, t} C'_1$ and $C_2 \geq C_1$, then there exists $C'_2 \geq C'_1$ such that $C_2 \xrightarrow{e, t} C'_2$, where $C \leq C'$ if and only if $|C^{-1}(q)| \leq |C'^{-1}(q)|$.

such that $C_y \xrightarrow{e, \tilde{t}_1} C_{y+1}$. Observe that by construction $q_{j_1+1}, q_{j_1+2}, \dots, q_n \in Q_W$, and as the protocol is Single Wait-Only, the only way for process e to go from q_{j_1+1} to q_{j_1} is to go through the reception segment $\tilde{t}_{j_1+1} \dots \tilde{t}_n$, hence it fires $\tilde{t}_n = t_f$ between C_x and C_y , and so it has to fire it infinitely often.

- **Case 2: for all $j \in [0, n]$, $\alpha_j \in ?\Sigma$.** Observe that in that case, if we let $t_f = (q, ?a, q')$ we have that $\text{future}(q) = \{q_0, \dots, q_n\}$. We prove by contradiction that there exists $j \in [0, n]$, such that $\tilde{t}_j$ is infinitely fired.

Assume that there exists N such that for all $M \geq N$, none of the $\tilde{t}_j$ are taken between C_M and C_{M+1} . Let $\tilde{t}_j = (p, ?a, p') \in T$ and i_j such that $(s_1, v_{i_j}) \rightsquigarrow_{\tilde{t}_j} (s_1, v_{i_j+1})$. We let $t = (q, \alpha, q') \in T$ be the action transition such that $(s_1, v_{i_j}) \xrightarrow{t} (s_1, v_{i_j+1})$. By construction, $t \in \{t_1, \dots, t_k\}$. Hence, there exists $K > M$ such that $C_K \xrightarrow{e, t} C_{K+1}$ for a process e . Recall that (from the building of ρ), for all $j \in \mathbb{N}$:

- $C_j = C'_j \oplus C^j$ for two configurations C'_j and C^j s.t. $C'_j \in \text{Future}(C''_j)$ where $C''_j \in \mathbb{C}[v_{i_j}]$.
- $C_{j+1} = C'_{j+1} \oplus C^{j+1}$ for two configurations C'_{j+1} and C^{j+1} s.t. $C'_{j+1} \in \text{Future}(C''_{j+1})$ where $C''_{j+1} \in \text{Future}(v_{i_{j+1}})$ is such that $C''_j \xrightarrow{e, t} C''_{j+1}$. Furthermore, the transition is issued by a process in C'_j and we actually have $C'_j \xrightarrow{e, t} C'_{j+1}$.

Recall that C''_{K+1} is built thanks to Lemma 8.5, hence there exists e' such that $\tilde{t}_j = (C'_K(e'), ?a, C''_{K+1}(e'))$. Configuration C'_{K+1} is then built thanks to Lemma 8.2: either $\tilde{t}_j$ is also taken between C'_K and C'_{K+1} or, $C'^{-1}_K(p) = \emptyset$. As $\tilde{t}_j$ is not taken between C_K and C_{K+1} it is neither between C'_K and C'_{K+1} , and so we obtained that $C'^{-1}_K(p) = \emptyset$.

We now prove that this remains the case for all $J \geq K$, i.e.

$$C'^{-1}_J(p) = \emptyset \text{ for all } J \geq K.$$

Assume it is not, there must exist a transition $t' = (s, \alpha, p)$ which is taken later, and such that $t' \notin \{\tilde{t}_0, \dots, \tilde{t}_n\}$ (we assumed that none of them were taken from now on). Hence, as $C'_j \in \text{Future}(v_{i_j})$ for some $j \in [1, k]$, it must be that t' is part of a cycle $\tilde{t}_0, \dots, \tilde{t}_m$ such that

1. $\tilde{t}_j = (p_j, \beta_j, p_{j+1})$ for all $j \in [0, m]$, with $p_0 = p_{m+1}$, and
2. there exists $r_j \in [1, k-1]$ such that $(s_1, v_{r_j}) \rightsquigarrow_{\tilde{t}_j} (s_1, v_{r_j+1})$ or $(s_1, v_{r_j}) \rightsquigarrow_{\tilde{t}_j} (s_1, v_{r_j+1})$ for all $j \in [0, m]$, and
3. $t' \in \{\tilde{t}_j \mid j \in [0, m]\}$.

Hence, as $t' = (s, \alpha, p)$ and the protocol is SWO, it follows that $\tilde{t}_{i_j} \in \{\tilde{t}_j \mid j \in [0, n]\}$ as it is the only outgoing transition from p . We can generalize: for all $k \in \{i_j + 1, \dots, n, 0, \dots, i_j\}$, the only outgoing transition from q_k is $\tilde{t}_k$, hence, we get that $\{\tilde{t}_j \mid j \in [0, n]\} \subseteq \{\tilde{t}_j \mid j \in [0, m]\}$. Recall that $\tilde{t}_j = (q_j, \alpha_j, q_{j+1})$ for all $j \in [0, n]$ (and $q_{n+1} = q_0$). As the protocol is SWO we get that the set of reachable states from any of the states q_j is $\{q_0, \dots, q_n\}$, and, necessarily, if $t' = (s, \alpha, p)$

it must be that $s \in \{q_0, \dots, q_n\}$ and so as the protocol is SWO, $t' \in \{\tilde{t}_0, \dots, \tilde{t}_n\}$ contradicting our assumption.

Hence, no other incoming transition to p is fired and $C_J'^{-1}(p) = \emptyset$ for all $J \geq K$.

So if none of the transitions $\tilde{t}_0, \dots, \tilde{t}_n$ are fired after C_N' , then there exists K such that, for all $J \geq K$, $C_J'^{-1}(q_j) = \emptyset$ for all $j \in [0, n]$. However, as for all $j \in [0, n]$ there exists i_j such that $(s_1, v_{i_j}) \rightsquigarrow_{\tilde{t}_j} (s_1, v_{i_{j+1}})$, it must be that $C_j' \in \text{Future}(v_{i_{j+1}})$ and so there exists a process e' such that $C_j'(e) \in \{q_0, \dots, q_n\}$. As the set of reachable states from $\{q_0, \dots, q_n\}$ is the set itself, the process stays in this set forever and for all $J \geq j$, there exists $p \in \setminus \{q_0, \dots, q_n\}$ s.t. $C_J'^{-1}(p) \neq \emptyset$, contradicting what we just argued.

Hence, there exists $\tilde{t}_j$ which is taken infinitely often. Hence, there exists e' taking $\tilde{t}_j$ infinitely often. Between two moments where it takes $\tilde{t}_j$, because the protocol is SWO, the only path going back to q_j goes through $\tilde{t}_j \tilde{t}_{j+1} \dots \tilde{t}_n \tilde{t}_0 \dots \tilde{t}_j$ and so, it takes t_f . Hence, there is a process taking infinitely often t_f , concluding the proof. $\square$

We are now ready to prove the main theorem of this section.

Theorem 8.1. REP-COVER^{nb} on Single Wait-Only protocols is in PTIME.

Proof. The set of coverable states of P needed to build V can be computed in polynomial time in the size of P thanks to Theorem 6.5. The size of the VASS V built from protocol P is polynomial in the size of P . Hence, completeness and soundness together with Theorem 8.1 prove the theorem. $\square$

8.4 SYNCHRONIZATION TO AN ACTION STATE IS EASY

In this section, we show that the synchronization problem *when the final state is an action state* belongs to PTIME in rendez-vous networks and broadcast networks with Single Wait-Only protocols. Moreover, for both types of networks, we prove that this upper bound is tight. In the next section, we will see that there is a significant complexity gap when the final state is a *waiting* state. In particular, synchronization on a waiting state in SWO protocols becomes Ackermann-hard in rendez-vous networks.

Let us first observe the following key property: for any configuration C such that $C^{-1}(Q_W) = \emptyset$, i.e., when no process is in a waiting state, the future set of C is exactly $\{C\}$, that is, $\text{Future}(C) = \{C\}$. This property is central to explain the complexity gap. When the final state q_f is an action state, we can compute an approximation of the reachable configurations that guarantees the existence of a configuration in the future set that is indeed reachable. If a configuration where all processes are in q_f belongs to this approximated set, then, since $q_f \in Q_A$, we know that this configuration is actually reachable (because its future set is reduced to itself).

In contrast, if q_f is a waiting state, the future set of a configuration where all processes are in q_f may include configurations where some processes have already moved beyond q_f .

Thus, we cannot conclude that the correct configuration — with all processes exactly in q_f — is reachable from the membership of such a configuration in the future set.

8.4.1 Rendez-vous networks

In this subsection, we explain why the synchronization problem on SWO rendez-vous protocols is in PTIME when $q_f \in Q_A$. To do this, we present a reduction with the synchronization on rendez-vous protocols with a *blocking* semantics, i. e. when non-blocking sendings are not allowed.

Let $P = (Q, \Sigma, q_{init}, T)$ be a rendez-vous protocol. We assume w.l.o.g. that there is no ingoing transition into q_{init} . If it is the case, we can copy the initial state into another state q_{init}' (not initial), whose outgoing and ingoing transitions are the same as the ones of q_{init} , and delete the ingoing transitions of q_{init} . This transformation is equivalent with regard to the synchronization problem. With an analogous reasoning, we assume that q_f (which is an action state by hypothesis) has no outgoing action transitions, i. e. it is a sink state.

We then transform it into a new rendez-vous protocol $P^b = (Q \cup \{q_\tau\}, \Sigma \cup \{\tau\}, q_{init}, T^b)$ such that

$$T^b = T \cup \{(q, !\tau, q') \mid \exists a \in \Sigma, (q, !a, q') \in T\} \\ \cup \{(q_{init}, ?\tau, q_\tau), (q_\tau, ?\tau, q_\tau), (q_f, !f, q_f), (q_\tau, ?f, q_f)\}.$$

The structure of P^b is illustrated in Figure 8.4. Each transition of P is duplicated as shown in Figure 8.5. Observe that P^b is not SWO anymore.

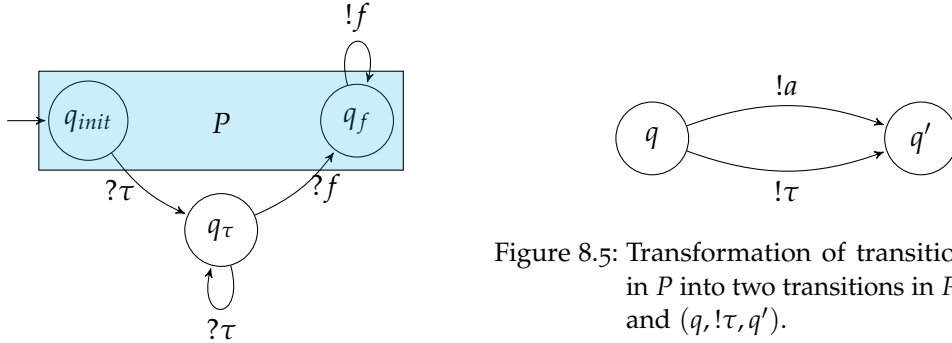


Figure 8.5: Transformation of transition $(q, !a, q')$ in P into two transitions in P^b : $(q, !a, q')$ and $(q, !\tau, q')$.

Figure 8.4: Illustration of P^b built from P .

The idea is as follows. Each rendez-vous between $(q, !a, q')$ and $(p, ?a, p')$ in P is also possible in P^b . However, in the blocking semantics, every message must be received by another process. Thus, whenever a process fires a transition $(q, !a, q')$ in P but the message is not received by any other process (that is, a sending step), in P^b , the process instead fires the transition $(q, !\tau, q')$. This τ message is then received by an extra process initially in q_{init} , whose only role is to receive τ via the transition $(q_{init}, ?\tau, q_\tau)$ first, and $(q_\tau, ?\tau, q_\tau)$ after. This mechanism allows the sending process to move to q' , even if no other process is currently in a state able to receive a .

Moreover, if a process fires the transition $(q, !\tau, q')$ instead of $(q, !a, q')$ when there was actually a process ready to receive a from a state p , this does not create unwanted behavior. Since we require reaching a configuration of P^b where all processes are in state q_f , the process in p will eventually move. Because the protocol P is SWO, there is only one outgoing

transition from p , which is a reception of a . Therefore, postponing the rendez-vous only delays the synchronization but is equivalent with respect to the synchronization problem.

Once all processes except the one on q_τ have reached q_f , one of the processes in q_f sends a special message f . This message is received by the process on q_τ , allowing it to move to q_f as well.

We now prove formally the soundness and completeness of our construction. Recall that $\rightarrow_{P,nb}$ denotes the transition relation associated with protocol P , allowing both rendez-vous and sending steps. On the other hand, $\rightarrow_{P^b,b}$ denotes the transition relation associated with protocol P^b , allowing only rendez-vous steps. We will now compare executions under both relations.

Lemma 8.9. If there exists an execution $C_{init} \rightarrow_{P,nb}^* C$ such that $C(e) = q_f$ for all $e \in [1, |C|]$, then there exists an execution $C'_{init} \rightarrow_{P^b,b}^* C'$ such that $C'(e) = q_f$ for all $e \in [1, |C'|]$.

Proof. We write the execution of P as:

$$C_{init} = C_0 \rightarrow_{P,nb} C_1 \rightarrow_{P,nb} \cdots \rightarrow_{P,nb} C_k = C.$$

If there is no **sending step**⁵ happening between C_0 and C_k , then

$$C_{init} = C_0 \rightarrow_{P^b,b} C_1 \rightarrow_{P^b,b} \cdots \rightarrow_{P^b,b} C_k = C.$$

And so, as any configuration of P is a configuration of P^b , the execution is also an execution of P^b , proving the lemma.

We assume now that there is at least one sending step and we let i_1 be the first index from which it happens. Let $C'_{init} = C_{init} \oplus q_{init}^\bullet$ and $n = |C_{init}|$. We prove by induction that for all $0 \leq i \leq k$, $C_{init} \oplus q_{init}^\bullet \rightarrow^* C_i \oplus q^\bullet$ where $q = q_{init}$ if $i \leq i_1$ and $q = q_\tau$ otherwise. For $i = 0$, it is trivially true as $C'_{init} = C_0 \oplus q_{init}^\bullet$. Assume that we have proved that $C_{init} \oplus q_{init}^\bullet \rightarrow^* C_i \oplus q^\bullet$ where $q = q_{init}$ if $i \leq i_1$ and $q = q_\tau$ otherwise. Let $C_i \xrightarrow{e_1, (p, !a, p')}_{P,nb} C_{i+1}$.

- * If there exists e_2 such that $(C_i(e_2), ?a, C_{i+1}(e_2)) \in T$, then a rendez-vous step happens between C_i and C_{i+1} and so $i_1 \neq i$. Furthermore, $C_i \oplus q^\bullet \xrightarrow{e_1, (p, !a, p')}_{P^b,b} C_{i+1} \oplus q^\bullet$.
- * Otherwise, it is a sending step happening between C_i and C_{i+1} and $i \geq i_1$. From P^b 's construction, we have that $(p, !\tau, p') \in T^b$. Furthermore, by induction hypothesis, either $i = i_1$ and $q = q_{init}$, or $i > i_1$ and $q = q_\tau$. In both cases, $(q, ?\tau, q_\tau) \in T$. Hence, $C_i \oplus q^\bullet \xrightarrow{e_1, (p, !\tau, p')}_{P^b,b} C_{i+1} \oplus q_\tau^\bullet$, proving the property.

Hence, $C'_{init} \rightarrow_{P^b,b} C_k \oplus q_\tau^\bullet$ and hence, $C_k \oplus q_\tau^\bullet \xrightarrow{1, (q_f, !f, q_f)}_{P^b,b} C_k \oplus q_f^\bullet$, proving the lemma. $\square$

Lemma 8.10. If there exists an execution $C_{init} \rightarrow_{P^b,b}^* C$ such that $C(e) = q_f$ for all $e \in [1, |C|]$, then there exists an execution $C_{init} \rightarrow_{P,nb}^* C$.

⁵ A sending step happens when a message is sent and not received by any process.

Proof. We write the execution of P^b as:

$$C_{init} = C_0 \rightarrow_{P^b, b} C_1 \rightarrow_{P^b, b} \cdots \rightarrow_{P^b, b} C_k = C.$$

We let E_τ be the set of processes e such that there exists $i \in [0, k]$ with $C_i(e) = q_\tau$. By construction of P^b , each process in E_τ receives τ (at least once) a certain number of times before receiving f and reaching q_f . It might be that it sends f a certain number of times afterwards. No other action is performed because we have assumed that there were no ingoing transition to q_{init} in P , and q_f is a sink state. Indeed, if a process in E_τ takes a transition $(q_{init}, !a, q)$, then it can not go back into q_{init} and hence can never reach state q_τ , which is absurd given E_τ 's definition. Let $x = |E_\tau|$ and $n = |C_0|$, we assume w.l.o.g. that $E_\tau = \{n - x, n - x + 1, \dots, n\}$ and let C'_i be the configuration C_i restricted to processes *not* in E_τ for all $0 \leq i \leq k$. Formally, C'_i is the configuration of $n - x$ processes such that $C'_i(e) = C_i(e)$ for all $e \in [1, n - x]$. By construction, $C'_i(e) \neq q_\tau$ for all $0 \leq i \leq k$ and $e \in [1, n - x]$, in particular, C'_i is a configuration of P .

We prove the intermediate result.

Claim 8.4. For all $0 \leq i < k - 1$, $C'_i \rightarrow_{P, nb} C'$ with $C' \in \text{Future}(C'_{i+1})$ or $C'_{i+1} = C'_i$.

Proof. Let $i \in [0, n - 2]$, by definition of $\rightarrow_{P^b, b}$, there exists $e_1, e_2 \in [1, n]$ such that

$$t_1 = (C_i(e_1), !a, C_{i+1}(e_1)) \in T^b \text{ and } t_2 = (C_i(e_2), ?a, C_{i+1}(e_2)) \in T^b$$

and for each other process e , $C_i(e) = C_{i+1}(e)$.

1. If $t_1 = (q_f, !f, q_f)$, then $e_2 \in E_\tau$ as the only process able to receive f must be in q_τ . Hence $C'_{i+1} = C'_i$.
2. If $t_1 \neq (q_f, !f, q_f)$, then $e_1 \notin E_\tau$. If $e_2 \notin E_\tau$, then $e_1, e_2 \in [1, n - x]$. Hence, the same rendez-vous can be performed from C'_i , leading precisely to C'_{i+1} . If $e_2 \in E_\tau$, then $t_2 = (q, ?\tau, q_\tau)$ (with $q = q_{init}$ or $q = q_\tau$) and $t_1 = (p, !\tau, p')$ for two states $p, p' \in Q$. By definition of P^b , there exists $a \in \Sigma$ such that $(p, !a, p') \in T$. As $C'_i(e_1) = p$, we define C' as:
 - if for each other process e , $a \notin R(C'_i(e))$, we let C' be the configuration such that $C'(e_1) = p'$ and for each other process e , $C'(e) = C'_i(e)$. We naturally get that $C'_i \xrightarrow{e_1, t_1}_{P, nb} C'$ and $C' = C'_{i+1}$.
 - if there exists a process e_2 and a state $q' \in Q$ such that $(C'_i(e_2), ?a, q') \in T$, then we let C' be the configuration such that $C'(e_1) = p'$, $C'(e_2) = q'$ and for each other process e , $C'(e) = C'_i(e)$. We naturally get that $C'_i \xrightarrow{e_1, t_1}_{P, nb} C'$. By construction, $q' \in \text{Future}(C'_i(e_2))$, and we have that $C' \in \text{Future}(C'_{i+1})$.

□

Hence, we can build an execution of P by applying successively the previous claim and Lemma 8.2 between C'_0 and a configuration $C' \in \text{Future}(C'_k)$. As $C'_k(e) = q_f \in Q_A$ for all $e \in [1, n - x]$, we have that $\text{Future}(C'_k) = \{C'_k\}$. Hence, $C' = C_k$ and so we proved the lemma. □

The two previous lemmas, together with the fact that SYNCHRO^b is in PTIME (Theorem 2.8, [BER21]) lead to the following theorem.

Theorem 8.2. SYNCHRO^{nb} for SWO rendez-vous protocols is in PTIME.

8.4.2 Broadcast networks

In the case of SWO broadcast protocols (and when $q_f \in Q_A$), the synchronization problem is also in PTIME. This time, we reduce it to the synchronization problem on *reconfigurable broadcast networks* (shortly RBN).

Reconfigurable broadcast networks work with broadcast protocols. Broadcasts are received by an arbitrary subset of processes, and this subset can vary throughout the execution.

Formally, given a broadcast protocol $P = (Q, \Sigma, q_{init}, T)$, we define the transition relation $\rightarrow_{\text{RBN}}$ over configurations of P , capturing the behavior of an RBN. Let $C, C' \in \mathcal{C}$ be two configurations, and let $t = (q, !!a, q') \in T$ be a transition. For a process $e \in [1, |C|]$, we say that $C \xrightarrow{e, t}_{\text{RBN}} C'$ if: $(C(e), !!a, C'(e)) = t$, and there exists a subset $R \subseteq [1, |C|] \setminus e$ such that for all $e' \in R$, $(C(e'), ?a, C'(e')) \in T$. For all other processes e'' , $C'(e'') = C(e'')$.

As usual, we write $C \rightarrow_{\text{RBN}} C'$ whenever there exist e and t such that $C \xrightarrow{e, t}_{\text{RBN}} C'$, and let $\rightarrow_{\text{RBN}}^*$ denote its reflexive and transitive closure.

We have the following result from [Fou15]:

Theorem 8.2 (Chapter 2 Section 2 in [Fou15]). SYNCHRO on RBN is in PTIME.

Moreover, by appropriately choosing the subset of receivers, we immediately have :

$$C \rightarrow C' \text{ implies that } C \rightarrow_{\text{RBN}} C'.$$

Conversely, we establish the following lemma.

Lemma 8.11. Let P be an SWO broadcast protocol. If $C \rightarrow_{\text{RBN}} C'$, then there exists $C'' \in \text{Future}(C')$ such that $C \rightarrow C''$.

Proof. Let $C \xrightarrow{e_1, (q, !!a, q')}_{\text{RBN}} C'$, and let $E_?$ be the set of processes e such that:

$$(C(e), ?a, C'(e)) \in T$$

i. e. the processes receiving the message a . Observe that the only processes changing states between C and C' are processes in $E_? \cup \{e_1\}$. We define C'' as follows:

- $C''(e_1) = q' = C'(e_1)$,
- $C''(e) = C'(e)$ for all $e \in E_?$,
- for all process e such that there exists a state $p \in Q$ such that $(C(e), ?a, p) = (C'(e), ?a, p) \in T$, we let $C''(e) = p$. In particular we have that $C''(e) \in \text{future}(C'(e))$,
- for all other process e , we let $C''(e) = C(e) = C'(e)$.

Hence, we naturally get that $C'' \in \text{Future}(C')$ and $C \xrightarrow{e_1, (q, !!a, q')} C''$ concluding the proof. $\square$

Recall that for all configuration C such that $C(e) = q_f \in Q_A$ for all $e \in [1, |C|]$, $\text{Future}(C) = \{C\}$ because q_f is an action state. Thanks to a simple induction together with Lemma 8.2, we get the following lemma.

Lemma 8.12. Let $C_{\text{init}} \in \mathcal{C}_{\text{init}}$ and C such that $C(e) = q_f \in Q_A$ for all $e \in [1, |C|]$. We have that $C_{\text{init}} \rightarrow^* C$ if and only if $C_{\text{init}} \rightarrow_{\text{RBN}}^* C$.

Hence, thanks to Theorem 8.2, we get the following theorem.

Theorem 8.3. *SYNCHRO for SWO broadcast protocols is in PTIME.*

8.4.3 Lower bound

In this subsection, we present a lower bound on the synchronization problem restricted to SWO protocols when the action state to synchronize on is an action state. To do this, we present a reduction from the Circuit Value Problem (CVP), which is known to be PTIME-complete [Lad75], already used to prove PTIME-hardness of state coverability for Wait-Only protocols (cf. Theorem 6.5 in Chapter 6). The reduction we present makes use of a broadcast protocol, but could also be presented with a rendez-vous protocol, hence proving that the two upper bounds of this section are tight.

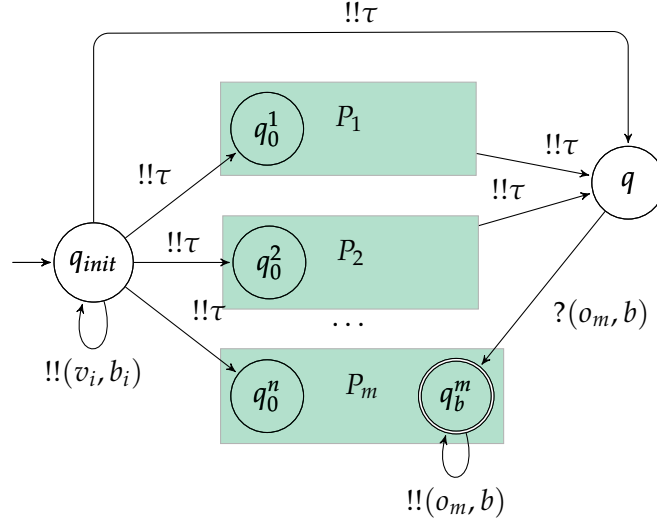
We recall how the CVP problem is stated: given an acyclic Boolean circuit with n input variables, one output variable, m boolean gates of type *and*, *or*, *not*, and a truth assignment for the input variables, is the value of the output equal to a given boolean value? Given an instance of the CVP, we build a protocol in which the processes broadcast variables (input ones or associated with gates) along with their boolean values. These broadcasts will be received by other processes that will use them to compute boolean value of their corresponding gate, and broadcast the obtained value. Hence, different values are propagated through the protocol representing the circuit, until the state representing the output variable value we look for is covered. Once it is covered, the process on it can gather everyone on it thanks to a broadcast.

We start by recalling some notations. We denote an instance of CVP $C = (V, o, G, B, b)$ where:

- $V = \{v_1, \dots, v_n\}$ gives the n input variables,
- o is the output variable,
- $G = \{g_1, \dots, g_m\}$ the m boolean gates,
- $B = \{b_1, \dots, b_n\}$ the boolean assignment such that for all $1 \leq i \leq n$, $b_i \in \{\top, \perp\}$ is the assignment of variable v_i , and
- b the boolean output value to test.

Let $V' = V \cup \{o_1, \dots, o_m\}$ where o_j is the output variable of gate j for $1 \leq j \leq m$. Wlog we can assume that $o = o_m$. For $1 \leq j \leq m$, we denote gate g_j by $g_j(\diamond, x_1, x_2, o_j)$ with $x_1, x_2 \in V'$ and $\diamond \in \{\vee, \wedge\}$ or by $g_j(\neg, x, o_j)$ with $x \in V'$. As C is acyclic, one can assume that $x_1, x_2, x \in V \cup \{o_1, o_2, \dots, o_{j-1}\}$.

The protocol we build from C is depicted in Figure 8.6. The state on which we have to synchronize all processes is q_b^m : as in Section 6.3.2, this state is coverable if and only if

Figure 8.6: Protocol P obtained from C .

$o = o_m$ evaluates to b . Once covered by a process, message (o_m, b) is broadcast, allowing every process to move to q_b^m as well.

Green boxes correspond to subparts of the protocol specific to gates $g_1, \dots, g_m$. The subparts associated with each gate work as follows: from the entering state, a process guesses which input values it will receive, and goes to the corresponding waiting state. If it guesses correctly, it will receive the corresponding message(s) and go to a state from which it can broadcast the output value of the gate. The value of the variables can be broadcast at will from q_{init} . For instance, a subpart P_j associated with an “or” gate is depicted in Figure 8.7: a process on state q_0^j decides which sequence of messages it will subsequently receive, thereby determining the boolean value of the gate. If the process makes the wrong choice, it will be blocked later when waiting for a missing message, and thus will never reach the final state. Otherwise, it will successfully reach the state encoding the correct boolean value and broadcast it before moving forward (if this is not the last gate). If it reaches q_b^m , it stays there.

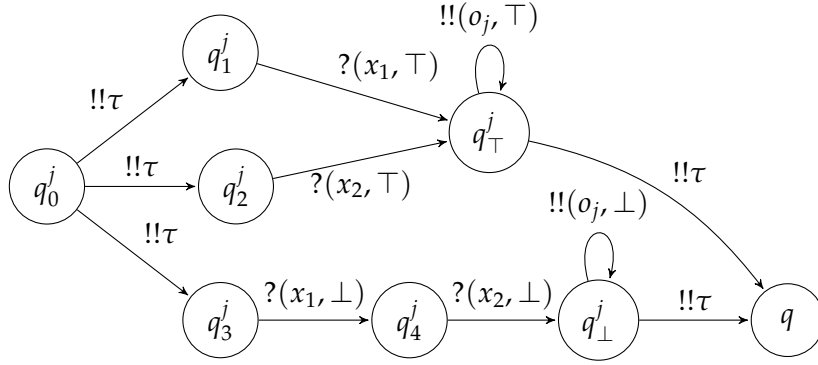
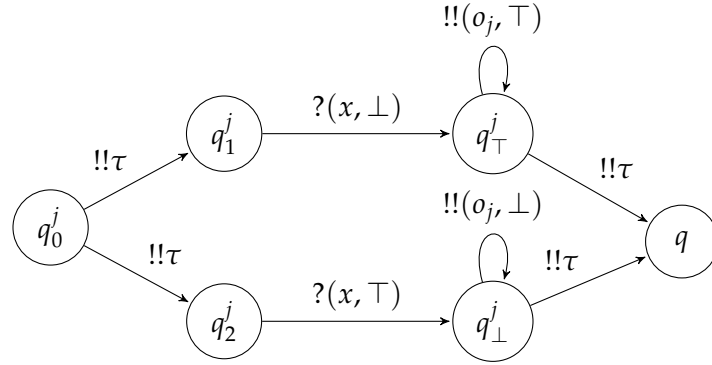
For the “or” gate shown in Figure 8.7, observe that $(o_j, \top)$ is broadcast whenever $q_\top^j$ is reached, that is, when $(x_1, \top)$ or $(x_2, \top)$ is received. In contrast, $(o_j, \perp)$ is broadcast when $q_\perp^j$ is reached, that is, only if both $(x_1, \perp)$ and $(x_2, \perp)$ are received.

Similarly, the subpart associated with a “not” gate is depicted in Figure 8.8, ensuring that the boolean value is negated appropriately.

Observe that P is Single Wait-Only. As the proofs are very similar to the ones presented in Section 6.3.2, we do not present it here.

Theorem 8.4. SYNCHRO^{nb} restricted to Single Wait-Only (broadcast) protocols and with $q_f \in Q_A$ is PTIME-hard.

As in Section 6.3.2, the presented reduction can easily be adapted to work with a rendez-vous protocol: we turn each broadcast transition labeled $!!a$ into a sending transition labeled $!a$, while keeping the corresponding reception transitions unchanged. At the end of the execution, instead of synchronizing all processes to q_b^m in a single broadcast step, the process that first reaches q_b^m repeatedly sends the corresponding message, allowing it to rendez-vous with each remaining process and enabling them to reach q_b^m as well.

Figure 8.7: P_j when $g_j(\vee, x_1, x_2, o_j)$.Figure 8.8: P_j when $g_j(\neg, x, o_j)$.

This yields the following corollary.

Corollary 8.1. SYNCHRO^{nb} restricted to Single Wait-Only rendez-vous (resp. broadcast) protocols and with $q_f \in Q_A$ is PTIME-complete.

8.5 SYNCHRONIZATION TO A WAITING STATE REMAINS HARD

In this section, we explain why the result presented in the last section does not hold when the target state is a waiting state. In fact, we know that it is Ackermann-hard in that case. The decidability status of this problem remains open.

We establish that this result holds when considering **rendez-vous protocols** (i.e. protocols without transitions labeled by $!!$). We show that with SWO rendez-vous protocols, one can simulate a test-free counter machine (shortly test-free CM) such that: a special configuration $(\ell_f, 0, 0)$ is reachable in the test-free CM if and only if there exists an execution of the protocol putting everyone on a special state q_f .

Let $\text{CM} = (\text{Loc}, X, \ell_{in}, \Delta)$ and a location $\ell_f \in \text{Loc}$. We build a SWO rendez-vous protocol P_{CM} depicted in Figure 8.9. Observe that the protocol is the same as the one depicted in Section 4.2, except for the reception $(1_i, ?\text{zero}_i, \ominus)$, which was the only reception preventing P_{CM} to be SWO.

In particular, when considering test-free CM, the protocol presented in Chapter 4 and section 4.2 can be simplified by deleting the transition $(1_i, ?\text{zero}_i, \ominus)$ as no message zero_i is ever sent by the process in the colored zone by definition. This directly leads to the above presented protocol. Hence, by a straightforward adaption of the proofs presented in Section 4.2

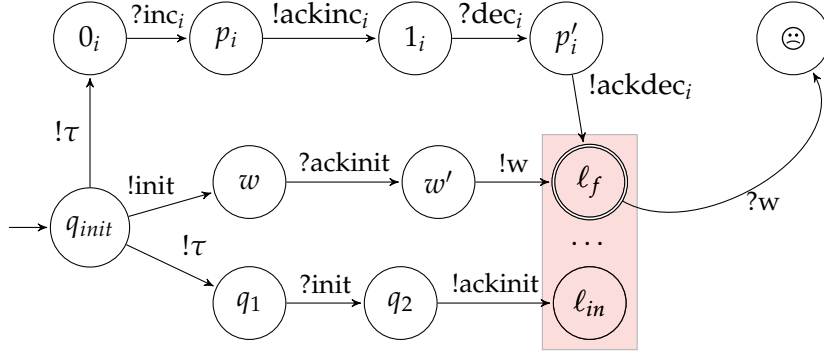


Figure 8.9: The protocol P_{CM} – The colored zone contains transitions pictured in Figure 8.10



Figure 8.10: Translation of $\delta = (\ell, x_i+, \ell')$, and $\delta' = (\ell, x_i-, \ell')$ for $\delta, \delta' \in \Delta$ (respectively left figure and right figure).

and Theorem 3.3 (stating that the reachability problem in VASS is Ackermann-complete [LS19; CO21; Ler21]), we get the following theorem.

Theorem 8.5. SYNCHRO^{nb} restricted to Single Wait-Only (rendez-vous) protocols is Ackermann-hard.

8.6 CONCLUSIONS

In this final chapter, we introduced a new restriction: Single Wait-Only (SWO) protocols. These are Wait-Only protocols in which each waiting state receives exactly one message. We showed that, under this restriction, synchronization becomes easier in some cases, specifically when the target state is an action state and the protocols are restricted to either rendez-vous or broadcast communication. In contrast, when the target is a waiting state, synchronization remains Ackermann-hard, even for rendez-vous protocols. We also proved that the repeated coverability problem is solvable in polynomial time. The results are summarized in Table 8.1. Several open questions remain, which we hope to investigate in future work.

	SYNCHRO ^{nb}		REP-COVER ^{nb}
Rdv Networks (RDV-protocol)	Undecidable (Chapter 4)		Undecidable [EK03; EFM99]
Broadcast Networks (B-protocol)			
Networks			
Wait-Only Rdv Networks (Wait-Only RDV-protocol)	Undecidable (Chapter 6)		?
Wait-Only Networks (Wait-Only protocols)			
Wait-Only Broadcast Networks (Wait-Only B-protocol)	<u>$q_f \in Q_W$:</u> Ackermann-c. (Chapter 7)	<u>$q_f \in Q_A$:</u> EXPSPACE-c. (Chapter 7)	in EXPSPACE and PSPACE-hard (Chapter 7)
SWO Rdv Networks (SWO RDV-protocol)	<u>$q_f \in Q_W$:</u> Ackermann-hard	<u>$q_f \in Q_A$:</u> PTIME-complete	in PTIME
SWO Broadcast Networks (SWO B-protocol)	<u>$q_f \in Q_W$:</u> Decidable	<u>$q_f \in Q_A$:</u> PTIME-complete	
SWO Networks (SWO B-protocol)	<u>$q_f \in Q_W$:</u> Ackermann-hard	<u>$q_f \in Q_A$:</u> PTIME-hard	

Table 8.1: Results presented in this chapter are written in red.

CONCLUSIONS

SUMMARY

In this thesis, we have investigated the verification of distributed systems where the number of processes is a parameter, and all processes execute the same code, modeled as a finite-state machine called a *protocol*. We considered both broadcast and non-blocking rendez-vous communication, either jointly or separately, always under synchronous semantics.

We studied four verification problems (see Chapter 2):

- **The synchronization problem**, which asks whether there exists an execution that puts all processes in a given state. In Chapter 4, we proved that this problem is undecidable when (1) communication is restricted to broadcasts, or (2) communication is restricted to non-blocking rendez-vous. In both cases, the proof relies on simulating a counter machine with zero tests. While result (1) is expected (though, to our knowledge, not documented on the literature), result (2) is more surprising, as the same problem is known to be in PTIME for *blocking* rendez-vous. Interestingly, non-blocking rendez-vous enables a form of leader selection via a simple mechanism.
- **The state and configuration coverability problems**, which ask whether there exists an execution placing at least one process in a given state, or covering a given configuration, respectively. These problems are known to be Ackermann-complete in the general case, including when restricted to broadcast communication. In Chapter 5, we focused on networks where communication is restricted to non-blocking rendez-vous (i.e., no broadcasts), and proved that both state and configuration coverability problems are EXPSpace-complete. To do so, we introduced an extension of counter machines called *non-blocking test-free counter machines with reset* (NB-R-CM), and showed that the coverability problem for these systems is EXPSpace-complete by adapting classical techniques from Lipton and Rackoff for VASS. The main technical challenge lies in the “reset” feature: from any location, the machine may restart its execution from the initial location without resetting the counters. Our solution relies on the non-blocking nature of decrements in NB-R-CM, allowing us to reset counters before simulating the VASS.
- **The repeated coverability problem**, which asks whether a given state can be visited infinitely often. This problem is known to be undecidable under both broadcast and non-blocking rendez-vous communication.

In Chapter 6, we introduced the *Wait-Only* restriction, which enforces that protocols consist only of waiting states (which do not perform actions) and action states (which do

not receive messages). We showed that this restriction significantly simplifies the coverability problems: state coverability becomes PTIME-complete, while configuration coverability becomes PSPACE-complete, in contrast with the high complexities in general broadcast (Ackermann-complete) or rendez-vous networks (EXPSpace-complete, as shown in Chapter 5). These results rely on a property we call the *copypaste property*, which in particular ensures that if one action state and another state are separately coverable, they are also jointly coverable. This naturally leads to a polynomial-time algorithm for state coverability. Proving that configuration coverability is in PSPACE is more involved: to cover a configuration with k processes, we show it suffices to consider configurations with k processes under an extended, carefully chosen transition relation.

We also proved that, under the Wait-Only restriction, configuration coverability becomes solvable in polynomial time if communication is restricted to non-blocking rendez-vous. This again relies on a suitable abstraction of configurations and their evolution. A key consequence is that, in such systems, any coverable state is either coverable by at most one process or by arbitrarily many.

Since the synchronization problem remains undecidable under this restriction with non-blocking rendez-vous, we turned to broadcast networks in Chapter 7. There, we proposed a generic abstraction of Wait-Only protocols into VASS. The abstraction keeps track of the number of processes in action states and in zones of waiting states. The VASS locations encode which waiting states are populated in each zone, and counters track how many processes are present. We argued that this construction faithfully simulates the original network, even without requiring transfer operations. From this construction, we derived upper bounds: synchronization becomes decidable and is in EXPSpace when the target state is an action state; repeated coverability is also in EXPSpace. Regarding lower bounds, we showed that synchronization is Ackermann-hard, repeated coverability is PSPACE-hard, and synchronization with an action target state is EXPSpace-hard.

Finally, in Chapter 8, we introduced *Single Wait-Only (SWO)* protocols, a refinement of Wait-Only protocols in which each waiting state receives exactly one message. We investigated whether this restriction yields lower complexities for problems that remain hard in the Wait-Only setting. We found that synchronization becomes decidable in some specific cases: when the target state is an action state and communication is restricted to either broadcast or non-blocking rendez-vous. In contrast, if the target state is a waiting state, synchronization remains Ackermann-hard, even when restricting to non-blocking rendez-vous communication. We also established that repeated coverability becomes decidable in PTIME for SWO protocols. The key idea is that the behavior of processes in waiting states can be “predicted”, as there is a unique reception path until the next action state, or a reception cycle in which processes become trapped.

These results show that SWO protocols enable more precise analyses in some cases, while leaving others challenging, thus opening several directions for future research.

FUTURE WORK

We present now several future directions, that we believe to be interesting.

Remaining open problems. Several important questions remain open. The synchronization problem in Single Wait-Only protocols is still not fully understood. While repeated coverability becomes tractable, synchronization might still be undecidable as we only es-

established an Ackermann lower bound. We conjecture that if the target is an action state and both broadcast and rendez-vous are allowed, the problem could be tractable. Another open question is repeated coverability in rendez-vous networks with Wait-Only protocols. Solving these, and even study the verification of more expressive logic, would complete our understanding of these networks. We also believe that an interesting path is to explore questions related to distributed synthesis. In [BFS15], authors study state coverability in Reconfigurable Broadcast Networks (Broadcast networks in which each message is received by an arbitrary subset of processes) where all processes follow the same *local strategy*, which is another parameter of the network. The question is then to determine if there exists a local strategy for which a given state is coverable. The authors show that this problem is NP-complete. We believe that (Single) Wait-Only restrictions are promising to obtain decidability results for this problem when the broadcasts are received by all processes.

(Single) Wait-Only in other contexts. It would also be interesting to explore the effect of the (Single) Wait-Only restriction in broader settings. For example, combining it with network topologies (for which coverability is undecidable [DSZ10]), might lead to new decidability or complexity results. Another direction is to consider systems with dynamic message creation (e. g. modeling the dynamic creation of objects on which `notify()` and `notifyAll()` are performed), where processes generate messages instead of relying on a fixed message alphabet. One also might consider adding inputs to the protocols to analyze the expressive power of Wait-Only and SWO protocols from the viewpoint of population protocols, where one studies the predicates or languages computable under a specific set of executions. Finally, in a similar fashion than continuous Petri nets, one might explore networks where a fraction of a process can move while the other stays, and study networks with a continuous semantics.

Towards a non-interference notion between processes. Recently, in his thesis [Wal24], Nicolas Waldburger introduced the notion of copycat systems, where any step taken by a process e_1 from state p to q can be simultaneously taken by any process e_2 in p , without affecting the global behavior. Broadcast and rendez-vous systems generally do not fall into this class: a message sent by e_2 may be received by others and affect them. However, we conjecture that Wait-Only protocols may satisfy a weaker form of this property. For example, if a process in an action state performs a step, it does not interfere with other processes on action states and only affects processes on relevant waiting states. In SWO protocols, this influence is even more controlled, since each waiting state accepts a single reception. Interestingly, the copypaste property implies that processes on action states are preserved under execution duplication, aligning with the intuition that they are interference-free. It would be valuable to define a general framework for non-interference reasoning in distributed systems, parameterized by the type of interference allowed. This could lead to a classification based on the interaction structure between processes.

BIBLIOGRAPHY

- [Abd+96] Parosh Aziz Abdulla, Kārlis Čerāns, Bengt Jonsson, and Yih-Kuen Tsay. “General Decidability Theorems for Infinite-State Systems.” In: *LICS’96*. IEEE Computer Society, 1996, pp. 313–321.
- [Abd+00] Parosh Aziz Abdulla, Kārlis Čerāns, Bengt Jonsson, and Yih-Kuen Tsay. “Algorithmic Analysis of Programs with Well Quasi-ordered Domains.” In: *Information and Computation* 160.1-2 (2000), pp. 109–127.
- [ARZ15] Benjamin Aminof, Sasha Rubin, and Florian Zuleger. “On the Expressive Power of Communication Primitives in Parameterised Systems.” In: *LPAR’15*. Vol. 9450. Lecture Notes in Computer Science. Springer-Verlag, 2015. DOI: 10.1007/978-3-662-48899-7_22.
- [Ang+04] Dana Angluin, James Aspnes, Zoë Diamadi, Michael J. Fischer, and René Peralta. “Computation in networks of passively mobile finite-state sensors.” In: *Proceedings of the Twenty-Third Annual ACM Symposium on Principles of Distributed Computing*. PODC ’04. New York, NY, USA: Association for Computing Machinery, 2004, 290–299. ISBN: 1581138024. DOI: 10.1145/1011767.1011810. URL: <https://doi.org/10.1145/1011767.1011810>.
- [AAE06] Dana Angluin, James Aspnes, and David Eisenstat. “Stably computable predicates are semilinear.” In: *Proceedings of the Twenty-Fifth Annual ACM Symposium on Principles of Distributed Computing*. PODC ’06. New York, NY, USA: Association for Computing Machinery, 2006, 292–299. ISBN: 1595933840. DOI: 10.1145/1146381.1146425. URL: <https://doi.org/10.1145/1146381.1146425>.
- [AK86] Krzysztof R. Apt and Dexter C. Kozen. “Limits for automatic verification of finite-state concurrent systems.” In: *Information Processing Letters* 22.6 (1986), pp. 307–309. ISSN: 0020-0190. DOI: [https://doi.org/10.1016/0020-0190\(86\)90071-2](https://doi.org/10.1016/0020-0190(86)90071-2). URL: <https://www.sciencedirect.com/science/article/pii/0020019086900712>.
- [BBM18] A. R. Balasubramanian, Nathalie Bertrand, and Nicolas Markey. “Parameterized Verification of Synchronization in Constrained Reconfigurable Broadcast Networks.” In: *Tools and Algorithms for the Construction and Analysis of Systems - 24th International Conference, TACAS 2018, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2018, Thessaloniki, Greece, April 14-20, 2018, Proceedings, Part II*. Vol. 10806. Lecture Notes in Computer Science. Springer, 2018, pp. 38–54. DOI: 10.1007/978-3-319-89963-3_3. URL: https://doi.org/10.1007/978-3-319-89963-3_3.
- [BER21] A. R. Balasubramanian, Javier Esparza, and Mikhail Raskin. “Finding Cut-Offs in Leaderless Rendez-Vous Protocols is Easy.” In: *FOSSACS’21*. Vol. 12650. LNCS. Springer, 2021, pp. 42–61.

- [BGW22] A. R. Balasubramanian, Lucie Guillou, and Chana Weil-Kennedy. “Parameterized Analysis of Reconfigurable Broadcast Networks.” In: *Foundations of Software Science and Computation Structures - 25th International Conference, FOSSACS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings*. Vol. 13242. Lecture Notes in Computer Science. Springer, 2022, pp. 61–80. DOI: 10.1007/978-3-030-99253-8_4. URL: https://doi.org/10.1007/978-3-030-99253-8_4.
- [BW21] A. R. Balasubramanian and Chana Weil-Kennedy. “Reconfigurable Broadcast Networks and Asynchronous Shared-Memory Systems are Equivalent.” In: *Proceedings 12th International Symposium on Games, Automata, Logics, and Formal Verification, GandALF 2021, Padua, Italy, 20-22 September 2021*. Vol. 346. EPTCS. 2021, pp. 18–34. DOI: 10.4204/EPTCS.346.2. URL: <https://doi.org/10.4204/EPTCS.346.2>.
- [Bau+23] Pascal Baumann, Khushraj Madnani, Filip Mazowiecki, and Georg Zetsche. “Monus Semantics in Vector Addition Systems with States.” In: *34th International Conference on Concurrency Theory (CONCUR 2023)*. Vol. 279. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 10:1–10:18. ISBN: 978-3-95977-299-0. DOI: 10.4230/LIPIcs.CONCUR.2023.10. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CONCUR.2023.10>.
- [BFS15] Nathalie Bertrand, Paulin Fournier, and Arnaud Sangnier. “Distributed Local Strategies in Broadcast Networks.” In: *26th International Conference on Concurrency Theory, CONCUR 2015, Madrid, Spain, September 1-4, 2015*. Vol. 42. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2015, pp. 44–57. DOI: 10.4230/LIPIcs.CONCUR.2015.44. URL: <https://doi.org/10.4230/LIPIcs.CONCUR.2015.44>.
- [Ber+22] Nathalie Bertrand, Nicolas Markey, Ocan Sankur, and Nicolas Waldburger. “Parameterized Safety Verification of Round-Based Shared-Memory Systems.” In: *49th International Colloquium on Automata, Languages, and Programming, ICALP 2022, July 4-8, 2022, Paris, France*. Vol. 229. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022, 113:1–113:20. DOI: 10.4230/LIPIcs.ICALP.2022.113. URL: <https://doi.org/10.4230/LIPIcs.ICALP.2022.113>.
- [Blo+20] Michael Blondin, Javier Esparza, Blaise Genest, Martin Helfrich, and Stefan Jaax. “Succinct Population Protocols for Presburger Arithmetic.” In: *37th International Symposium on Theoretical Aspects of Computer Science (STACS 2020)*. Vol. 154. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, 40:1–40:15. ISBN: 978-3-95977-140-5. DOI: 10.4230/LIPIcs.STACS.2020.40. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.STACS.2020.40>.
- [BM99] Ahmed Bouajjani and Richard Mayr. “Model checking lossy vector addition systems.” In: *Proceedings of the 16th Annual Conference on Theoretical Aspects of Computer Science. STACS’99*. Berlin, Heidelberg: Springer-Verlag, 1999, 323–333. ISBN: 354065691X.

- [Bou+16] Patricia Bouyer, Nicolas Markey, Mickael Randour, Arnaud Sangnier, and Daniel Stan. “Reachability in Networks of Register Protocols under Stochastic Schedulers.” In: *43rd International Colloquium on Automata, Languages, and Programming (ICALP 2016)*. Vol. 55. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016, 106:1–106:14. ISBN: 978-3-95977-013-2. DOI: 10.4230/LIPIcs.ICALP.2016.106. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2016.106>.
- [CMS19] Peter Chini, Roland Meyer, and Prakash Saivasan. “Liveness in Broadcast Networks.” In: *Networked Systems - 7th International Conference, NETYS 2019, Marrakech, Morocco, June 19-21, 2019, Revised Selected Papers*. Vol. 11704. Lecture Notes in Computer Science. Springer, 2019, pp. 52–66. DOI: 10.1007/978-3-030-31277-0_4. URL: https://doi.org/10.1007/978-3-030-31277-0_4.
- [CO21] Wojciech Czerwinski and Lukasz Orlikowski. “Reachability in Vector Addition Systems is Ackermann-complete.” In: *62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2021, Denver, CO, USA, February 7-10, 2022*. IEEE, 2021, pp. 1229–1240. DOI: 10.1109/FOCS52979.2021.00120. URL: <https://doi.org/10.1109/FOCS52979.2021.00120>.
- [DRVB02] Giorgio Delzanno, Jean-François. Raskin, and Laurent Van Begin. “Towards the Automated Verification of Multithreaded Java Programs.” In: *TACAS 2002, Held as Part of the Joint European Conference on Theory and Practice of Software, ETAPS 2002*. Vol. 2280. Lecture Notes in Computer Science. Springer, 2002.
- [Del+12] Giorgio Delzanno, Arnaud Sangnier, Riccardo Traverso, and Gianluigi Zavattaro. “On the Complexity of Parameterized Reachability in Reconfigurable Broadcast Networks.” In: *IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science, FSTTCS 2012, December 15-17, 2012, Hyderabad, India*. Vol. 18. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2012, pp. 289–300. DOI: 10.4230/LIPIcs.FSTTCS.2012.289. URL: <https://doi.org/10.4230/LIPIcs.FSTTCS.2012.289>.
- [DSZ10] Giorgio Delzanno, Arnaud Sangnier, and Gianluigi Zavattaro. “Parameterized Verification of Ad Hoc Networks.” In: *Proceedings of the 21st International Conference on Concurrency Theory (CONCUR’10)*. Vol. 6269. Lecture Notes in Computer Science. Springer, 2010.
- [DSZ11] Giorgio Delzanno, Arnaud Sangnier, and Gianluigi Zavattaro. “On the Power of Cliques in the Parameterized Verification of Ad Hoc Networks.” In: *Proceedings of the 14th International Conference on Foundations of Software Science and Computation Structures (FoSSaCS’11)*. Vol. 6604. Lecture Notes in Computer Science. Springer, 2011.
- [DSZ12] Giorgio Delzanno, Arnaud Sangnier, and Gianluigi Zavattaro. “Verification of Ad Hoc Networks with Node and Communication Failures.” In: *Proceedings of the 32nd International Conference on Formal Techniques for Distributed Systems (FMOODS/FORTE’12)*. Vol. 7273. Lecture Notes in Computer Science. Springer, 2012.

- [DFS98] Catherine Dufourd, Alain Finkel, and Ph. Schnoebelen. “Reset Nets Between Decidability and Undecidability.” In: *Proceedings of the 25th International Colloquium on Automata, Languages and Programming*. ICALP ’98. Berlin, Heidelberg: Springer-Verlag, 1998, 103–115. ISBN: 3540647813.
- [Dur+17] Antoine Durand-Gasselin, Javier Esparza, Pierre Ganty, and Rupak Majumdar. “Model checking parameterized asynchronous shared-memory systems.” In: *Formal Methods Syst. Des.* 50.2-3 (2017), pp. 140–167. DOI: 10.1007/S10703-016-0258-3. URL: <https://doi.org/10.1007/s10703-016-0258-3>.
- [EK03] E. Allen Emerson and Vineet Kahlon. “Model Checking Guarded Protocols.” In: *LICS 2003*. IEEE Computer Society, 2003. DOI: 10.1109/LICS.2003.1210076.
- [EN98] E. Allen Emerson and Kedar S. Namjoshi. “On model checking for non-deterministic infinite-state systems.” In: *Proceedings. Thirteenth Annual IEEE Symposium on Logic in Computer Science (Cat. No.98CB36226)*. 1998, pp. 70–80. DOI: 10.1109/LICS.1998.705644.
- [Ene+24] Constantin Enea, Dimitra Giannakopoulou, Michalis Kokologiannakis, and Rupak Majumdar. “Model Checking Distributed Protocols in Must.” In: *Proc. ACM Program. Lang.* 8.OOPSLA2 (Oct. 2024). DOI: 10.1145/3689778. URL: <https://doi.org/10.1145/3689778>.
- [Esp96] Javier Esparza. “Decidability and Complexity of Petri Net Problems - An Introduction.” In: *Lectures on Petri Nets I: Basic Models, Advances in Petri Nets, the volumes are based on the Advanced Course on Petri Nets, held in Dagstuhl, September 1996*. Vol. 1491. Lecture Notes in Computer Science. Springer, 1996, pp. 374–428. DOI: 10.1007/3-540-65306-6_20. URL: https://doi.org/10.1007/3-540-65306-6_20.
- [Esp14] Javier Esparza. “Keeping a crowd safe: On the complexity of parameterized verification (invited talk).” In: *31st International Symposium on Theoretical Aspects of Computer Science (STACS 2014)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik. 2014, pp. 1–10.
- [EFM99] Javier Esparza, Alain Finkel, and Richard Mayr. “On the verification of broadcast protocols.” In: *LICS’99*. IEEE Comp. Soc. Press, 1999. DOI: 10.1109/LICS.1999.782630.
- [Esp+17] Javier Esparza, Pierre Ganty, Jérôme Leroux, and Rupak Majumdar. “Verification of population protocols.” In: *Acta Informatica* 54.2 (2017), pp. 191–215. DOI: 10.1007/s00236-016-0272-3. URL: <https://doi.org/10.1007/s00236-016-0272-3>.
- [EGM13] Javier Esparza, Pierre Ganty, and Rupak Majumdar. “Parameterized Verification of Asynchronous Shared-Memory Systems.” In: *Computer Aided Verification - 25th International Conference, CAV 2013, Saint Petersburg, Russia, July 13-19, 2013. Proceedings*. Vol. 8044. Lecture Notes in Computer Science. Springer, 2013, pp. 124–140. DOI: 10.1007/978-3-642-39799-8_8. URL: https://doi.org/10.1007/978-3-642-39799-8_8.

- [Esp+18] Javier Esparza, Pierre Ganty, Rupak Majumdar, and Chana Weil-Kennedy. “Verification of Immediate Observation Population Protocols.” In: *29th International Conference on Concurrency Theory (CONCUR 2018)*. Vol. 118. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018, 31:1–31:16. ISBN: 978-3-95977-087-3. DOI: 10.4230/LIPIcs.CONCUR.2018.31. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CONCUR.2018.31>.
- [Esp+21] Javier Esparza, Stefan Jaax, Mikhail Raskin, and Chana Weil-Kennedy. “The complexity of verifying population protocols.” In: *Distributed Computing* 34.2 (2021), pp. 133–177. DOI: 10.1007/s00446-021-00390-x. URL: <https://doi.org/10.1007/s00446-021-00390-x>.
- [FGH13] Alain Finkel, Stefan Göller, and Christoph Haase. “Reachability in Register Machines with Polynomial Updates.” In: *Mathematical Foundations of Computer Science 2013*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 409–420.
- [FS01] Alain Finkel and Philippe Schnoebelen. “Well-structured transition systems everywhere!” In: *Theoretical Computer Science* 256.1-2 (2001), pp. 63–92.
- [FMW17] Marie Fortin, Anca Muscholl, and Igor Walukiewicz. “Model-Checking Linear-Time Properties of Parametrized Asynchronous Shared-Memory Pushdown Systems.” In: *Computer Aided Verification - 29th International Conference, CAV 2017, Heidelberg, Germany, July 24-28, 2017, Proceedings, Part II*. Vol. 10427. Lecture Notes in Computer Science. Springer, 2017, pp. 155–175. DOI: 10.1007/978-3-319-63390-9_9. URL: https://doi.org/10.1007/978-3-319-63390-9_9.
- [Fou15] Paulin Fournier. “Parameterized verification of networks of many identical processes. (Vérification paramétrée de réseaux composés d’une multitude de processus identiques).” PhD thesis. University of Rennes 1, France, 2015. URL: <https://tel.archives-ouvertes.fr/tel-01355847>.
- [FH15] Estíbaliz Fraca and Serge Haddad. “Complexity Analysis of Continuous Petri Nets.” In: *Fundam. Informaticae* 137.1 (2015), pp. 1–28. DOI: 10.3233/FI-2015-1168. URL: <https://doi.org/10.3233/FI-2015-1168>.
- [GS92] Steven M. German and A. Prasad Sistla. “Reasoning about Systems with Many Processes.” In: *Journal of the ACM* 39.3 (1992), pp. 675–735.
- [Göl+10] Stefan Göller, Christoph Haase, Joël Ouaknine, and James Worrell. “Model Checking Succinct and Parametric One-Counter Automata.” In: *Proceedings of the 37th International Colloquium on Automata, Languages and Programming (ICALP’10) – Part II*. Vol. 6199. Lecture Notes in Computer Science. Springer-Verlag, July 2010, pp. 575–586. DOI: 10.1007/978-3-642-14162-1_48.
- [GSS23] Lucie Guillou, Arnaud Sangnier, and Nathalie Sznajder. “Safety Analysis of Parameterised Networks with Non-Blocking Rendez-Vous.” In: *34th International Conference on Concurrency Theory, CONCUR 2023, September 18-23, 2023, Antwerp, Belgium*. Vol. 279. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 7:1–7:17. DOI: 10.4230/LIPIcs.CONCUR.2023.7. URL: <https://doi.org/10.4230/LIPIcs.CONCUR.2023.7>.

- [GSS24] Lucie Guillou, Arnaud Sangnier, and Nathalie Sznajder. “Safety Verification of Wait-Only Non-Blocking Broadcast Protocols.” In: *Application and Theory of Petri Nets and Concurrency - 45th International Conference, PETRI NETS 2024, Geneva, Switzerland, June 26-28, 2024, Proceedings*. Vol. 14628. Lecture Notes in Computer Science. Springer, 2024, pp. 291–311. DOI: 10.1007/978-3-031-61433-0\14. URL: <https://doi.org/10.1007/978-3-031-61433-0\14>.
- [GSS25] Lucie Guillou, Arnaud Sangnier, and Nathalie Sznajder. “Wait-Only Broadcast Protocols Are Easier to Verify.” In: *50th International Symposium on Mathematical Foundations of Computer Science, MFCS 2025, August 25-29, 2025, Warsaw, Poland*. Ed. by Pawel Gawrychowski, Filip Mazowiecki, and Michal Skrzypczak. Vol. 345. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025, 53:1–53:17. DOI: 10.4230/LIPICS.MFCS.2025.53. URL: <https://doi.org/10.4230/LIPIcs.MFCS.2025.53>.
- [HH14] Christoph Haase and Simon Halfon. “Integer Vector Addition Systems with States.” In: *Reachability Problems*. Cham: Springer International Publishing, 2014, pp. 112–124.
- [Hab97] Peter Habermehl. “On the Complexity of the Linear-Time μ -calculus for Petri-Nets.” In: *Application and Theory of Petri Nets 1997, 18th International Conference, ICATPN ’97, Toulouse, France, June 23-27, 1997, Proceedings*. Vol. 1248. Lecture Notes in Computer Science. Springer, 1997, pp. 102–116. DOI: 10.1007/3-540-63139-9\32. URL: <https://doi.org/10.1007/3-540-63139-9\32>.
- [Hag11] Matthew Hague. “Parameterised Pushdown Systems with Non-Atomic Writes.” In: *IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2011)*. Vol. 13. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2011, pp. 457–468. ISBN: 978-3-939897-34-7. DOI: 10.4230/LIPIcs.FSTTCS.2011.457. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSTTCS.2011.457>.
- [HP79] John Hopcroft and Jean-Jacques Pansiot. “On the reachability problem for 5-dimensional vector addition systems.” In: *Theoretical Computer Science* 8.2 (1979), pp. 135–159. ISSN: 0304-3975. DOI: [https://doi.org/10.1016/0304-3975\(79\)90041-0](https://doi.org/10.1016/0304-3975(79)90041-0). URL: <https://www.sciencedirect.com/science/article/pii/0304397579900410>.
- [HS20] Florian Horn and Arnaud Sangnier. “Deciding the Existence of Cut-Off in Parameterized Rendez-Vous Networks.” In: *CONCUR’20*. Vol. 171. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020, 46:1–46:16.
- [KM69] Richard M. Karp and Raymond E. Miller. “Parallel program schemata.” In: *Journal of Computer and System Sciences* 3.2 (1969), pp. 147–195. ISSN: 0022-0000. DOI: [https://doi.org/10.1016/S0022-0000\(69\)80011-5](https://doi.org/10.1016/S0022-0000(69)80011-5). URL: <https://www.sciencedirect.com/science/article/pii/S0022000069800115>.
- [Kok+22] Michalis Kokologiannakis, Iason Marmanis, Vladimir Gladstein, and Viktor Vafeiadis. “Truly stateless, optimal dynamic partial order reduction.” In: *Proc. ACM Program. Lang.* 6.POPL (Jan. 2022). DOI: 10.1145/3498711. URL: <https://doi.org/10.1145/3498711>.

- [KS88] S. Rao Kosaraju and Gregory F. Sullivan. “Detecting Cycles in Dynamic Graphs in Polynomial Time (Preliminary Version).” In: *Proceedings of the 20th Annual ACM Symposium on Theory of Computing, May 2-4, 1988, Chicago, Illinois, USA*. ACM, 1988, pp. 398–406. DOI: 10.1145/62212.62251. URL: <https://doi.org/10.1145/62212.62251>.
- [Koz77] Dexter Kozen. “Lower bounds for natural proof systems.” In: *18th Annual Symposium on Foundations of Computer Science (sfcs 1977)*. 1977, pp. 254–266. DOI: 10.1109/SFCS.1977.16.
- [Lad75] Richard E. Ladner. “The circuit value problem is log space complete for P.” In: *SIGACT News* 7.1 (Jan. 1975), 18–20. ISSN: 0163-5700. DOI: 10.1145/990518.990519. URL: <https://doi.org/10.1145/990518.990519>.
- [LPY97] Kim G. Larsen, Paul Pettersson, and Wang Yi. “Uppaal in a nutshell.” In: *International Journal on Software Tools for Technology Transfer* 1.1 (1997), pp. 134–152. DOI: 10.1007/s100090050010. URL: <https://doi.org/10.1007/s100090050010>.
- [Ler11] Jérôme Leroux. “Vector Addition System Reversible Reachability Problem.” In: *CONCUR - Concurrency Theory - 22nd International Conference, CONCUR 2011, Aachen, Germany, September 6-9, 2011. Proceedings*. Vol. 6901. Lecture Notes in Computer Science. Springer, 2011, pp. 327–341. DOI: 10.1007/978-3-642-23217-6_22. URL: https://doi.org/10.1007/978-3-642-23217-6_22.
- [Ler21] Jérôme Leroux. “The Reachability Problem for Petri Nets is Not Primitive Recursive.” In: *62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2021, Denver, CO, USA, February 7-10, 2022*. IEEE, 2021, pp. 1241–1252. DOI: 10.1109/FOCS52979.2021.00121. URL: <https://doi.org/10.1109/FOCS52979.2021.00121>.
- [LS19] Jérôme Leroux and Sylvain Schmitz. “Reachability in Vector Addition Systems is Primitive-Recursive in Fixed Dimension.” In: *34th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2019, Vancouver, BC, Canada, June 24-27, 2019*. IEEE, 2019, pp. 1–13. DOI: 10.1109/LICS.2019.8785796. URL: <https://doi.org/10.1109/LICS.2019.8785796>.
- [Lip76] Richard J. Lipton. *The reachability problem requires exponential space*. Research report (Yale University. Department of Computer Science). Department of Computer Science, Yale University, 1976.
- [May81] Ernst W. Mayr. “An algorithm for the general Petri net reachability problem.” In: *Proceedings of the Thirteenth Annual ACM Symposium on Theory of Computing. STOC '81*. New York, NY, USA: Association for Computing Machinery, 1981, 238–246. ISBN: 9781450373920. DOI: 10.1145/800076.802477. URL: <https://doi.org/10.1145/800076.802477>.
- [Min67] Marvin L. Minsky. *Computation: Finite and Infinite Machines*. Upper Saddle River, NJ, USA: Prentice-Hall, Inc., 1967. ISBN: 0-13-165563-9.
- [MSW17] Anca Muscholl, Helmut Seidl, and Igor Walukiewicz. “Reachability for Dynamic Parametric Processes.” In: *Verification, Model Checking, and Abstract Interpretation - 18th International Conference, VMCAI 2017, Paris, France, January 15-17, 2017, Proceedings*. Vol. 10145. Lecture Notes in Computer Science. Springer, 2017, pp. 424–441. DOI: 10.1007/978-3-319-52234-0_23. URL: https://doi.org/10.1007/978-3-319-52234-0_23.

- [MQ08] Madanlal Musuvathi and Shaz Qadeer. “Fair stateless model checking.” In: *SIGPLAN Not.* 43.6 (June 2008), 362–371. ISSN: 0362-1340. DOI: 10.1145/1379022.1375625. URL: <https://doi.org/10.1145/1379022.1375625>.
- [Pet62] Carl Adam Petri. *Kommunikation mit Automaten*. Dissertation, Schriften des IIM 2. Bonn: Rheinisch-Westfälisches Institut für Instrumentelle Mathematik an der Universität Bonn, 1962.
- [Rac78] Charles Rackoff. “The Covering and Boundedness Problems for Vector Addition Systems.” In: *Theoretical Computer Science* 6 (1978), pp. 223–231.
- [RB03] Jean-François Raskin and Laurent Van Begin. “Petri Nets with Non-blocking Arcs are Difficult to Analyze.” In: *INFINITY’03*. Vol. 98. Electronic Notes in Theoretical Computer Science. Elsevier, 2003, pp. 35–55.
- [Rei13] Julien Reichert. “On the Complexity of Counter Reachability Games.” In: *Reachability Problems*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 196–208.
- [SS13] Sylvain Schmitz and Philippe Schnoebelen. “The Power of Well-Structured Systems.” In: *CONCUR 2013*. Vol. 8052. Lecture Notes in Computer Science. Springer, 2013. DOI: 10.1007/978-3-642-40184-8_2.
- [Val78] Rüdiger Valk. “Self-Modifying Nets, a Natural Extension of Petri Nets.” In: *ICALP’78*. Vol. 62. LNCS. Springer, 1978, pp. 464–476.
- [Wal24] Nicolas Waldburger. “Parameterized verification of distributed shared-memory systems.” Thèse de doctorat dirigée par Bertrand, NathalieMarkey, Nicolas et Sankur, Ocan Informatique Université de Rennes (2023-....) 2024. PhD thesis. 2024. URL: <http://www.theses.fr/2024URENS058>.

INDEX

- $?\Sigma$, 16, 149
- $E_{q_a, j}^{\rho, i}$, 159, 159–161, 167, 168, 171, 172, 193
- CoSets, 153
- V_P^{MR} , 180
- V_P , 153, 159, 171, 178, 179, 186
- V_P^{RC} , 186
- $\text{repr}^{\text{RC}}(\cdot, \cdot)$, 191, 192
- T_{Act} , 16, 18, 24
- $Q_A[P]$, 95
- Done, 152
- $\bullet$
- A , 152
- $\mathcal{C}_n[P]$, 17
- $C[v]$, 213
- Future(C), 203, 205
- $\llbracket \cdot \rrbracket^{\text{RC}}$, 188, 189
- $\llbracket \gamma \rrbracket$, 113, 115
- $\text{lab}(\mathcal{S})$, 153
- $\text{lab}(S)$, 152
- 0_X , 15, 15, 34, 34, 60, 61, 75
- $\text{st}(\text{Toks})$, 126
- $\xrightarrow{\text{rdv}, t} p$, 78, 81, 82
- $\xrightarrow{s, t} p$, 78, 81, 82
- $\#\text{proc}(\rho)$, 147
- $\text{na}(\rho, j, e)$, 149, 168, 170
- $\text{na-index}(\rho, j, e)$, 149
- $\text{na-index-set}(\rho, j, q_a)$, 151, 159–161
- $\text{na-index-set}^{\text{conf}}(\rho, j, q_a)$, 186
- $\text{na-state}(\rho, j, e)$, 149
- $\text{print}(S)$, 152
- $\rho(e)$, 147
- $\rho_{\geq i}$, 147
- $\rho_{\leq i}$, 147
- $\rho_{i, j}$, 147
- $\text{set}(C)$, 152
- S_f , 170
- $\min_q$, 97, 97, 101, 103, 105
- $C(A)$, 146
- $C \oplus C'$, 152
- $C^{-1}(Q')$, 17
- $\mathcal{C}_f$, 170, 179
- $Q_W[P]$, 95
- n -configuration, 17, 24
 - (Broadcast step), 18, 20, 24, 100, 147, 148
 - (Rendez-vous step), 18, 20, 24, 26, 27, 55, 78, 147, 148, 205, 206
 - (Sending step), 18, 19, 20, 24, 26, 27, 45, 49, 55, 78, 147, 220
- COVER[$\mathcal{CM}$], 35, 38, 94
- COVER[VASS], 36
- COVER[VAS], 36
- COVER[NB-CM], 39
- COVER[NB-VAS], 40
- COVER[NB-R-CM], 39
- COVER[test-free CM], 38, 38
- REACH[$\mathcal{CM}$], 35, 38
- REACH[VASS], 36
- REACH[VAS], 36
- REACH[test-free CM], 38, 38
- MUTUREACH[VASS], 178
- MUTUREACH[VAS], 178
- NB-CM, 39, 39, 56, 60–68, 71–73, 79, 80
- NB-R-CM, 39, 56, 61, 62, 73, 79, 85, 86, 88
- $R(q)$, 16, 126, 150, 157, 158
- REP-COVER[VASS], 184
- $Q?(a)$, 16, 53, 138, 140, 141
- test-free CM, 37, 37, 38, 62, 65, 73, 74, 173, 182
- 2Exp-bounded, 62, 62, 73
- abstract configuration, 112
- Almost well-formed, 185, 185, 186
- Blocking Networks, 20, 31, 91
- Blocking Rendez-Vous Networks, 20, 28, 31, 41, 91
- Broadcast Networks, 20, 31, 42, 45, 91, 143, 199, 227
- broadcast protocol, 16, 16, 20, 27, 28, 31, 42, 91, 143, 199, 227
- coherent, 153, 153, 161, 162
- configuration, 17, 17, 23

- configuration coverability, **21**, 27, 29, 31, 79, 84, 90, 91, 143
- conflict-free, **126**, 126, 132, 134
- consistent, **127**, 131
- Counter machine, **34**, 34, 61
- coverable, **19**, 96
- execution, **15**, **18**, 34, 96, 147
- finite execution, **18**, 147, 149
- fires t , **184**
- implementation, **156**, 170, 179, 188
- infinite execution, **18**, 147, 149
- mutliset configuration, **23**, 23, 78, 79
- Networks, **20**, 227
- Non-blocking VAS, **40**, 40, 56, 57, 59, 60
- nonconformist, **186**, 191
- P-configuration, **187**
- print, **152**
- protocol, **16**, 16, 17, 20
- reachable, **19**, 23, 26
- Rendez-Vous Networks, **20**, 31, 39, 45, 91, 143, 199, 227
- rendez-vous protocol, **16**, 16, 20, 28, 29, 31, 45, 53, 79, 84–86, 90, 91, 143, 199, 225, 227
- repeated coverability, **22**, 28, 29, 31, 91, 147, 184, 199, 227
- representative, **159**, **160**, 160
- respects, **126**, 127, 133, 134
- run, **15**, 34
- S-configuration, **156**, 162
- Single Wait-Only, **201**, 227
- state coverability, **21**, 27, 29, 31, 79, 84, 85, 90, 91, 143
- summary, **152**
- synchronization, **22**, 31, 53, 91, 143, 147, 199, 227
- token-set of configurations, **126**
- VASS, **35**, 35–38, 56, 61, 62, 178
- Vector Addition Systems, **36**, 36, 37, 56, 178
- Wait-Only protocol, **93**, 103, 143, 145, 199, 201, 227
- well-formed, **151**, 160, 161, 163
- Well-Structured Transitions Systems, **26**, 26, 37

